

库函数使用说明

0.1 Topics	1
1 结构体索引	3
1.1 结构体	3
2 文件索引	5
2.1 文件列表	5
3 Topic Documentation	11
3.1 Clu	11
3.1.1 详细描述	11
3.1.2 函数说明	12
3.1.2.1 CLU_Disable()	12
3.1.2.2 CLU_Enable()	12
3.1.2.3 CLU_Init()	13
3.1.2.4 CLU_ReadOutVoleue()	13
3.1.2.5 CLU_StructInit()	13
3.2 Crc	14
3.2.1 详细描述	14
3.2.2 函数说明	14
3.2.2.1 CRC_Calc()	14
3.2.2.2 CRC_CalcCommon()	15
3.2.2.3 CRC_Disable()	15
3.2.2.4 CRC_Enable()	16
3.2.2.5 CRC_Init()	16
3.2.2.6 CRC_StructInit()	17
3.2.3 变量说明	17
3.2.3.1 CRC_XOROUT	17
3.3 I2c	17
3.3.1 详细描述	18
3.3.2 函数说明	18
3.3.2.1 I2c_ClearBusState()	18
3.3.2.2 I2C_ClearIRQFlag()	19
3.3.2.3 I2C_Disable()	19
3.3.2.4 I2C_Enable()	19
3.3.2.5 I2C_GetFlag()	20
3.3.2.6 I2C_GetIRQFlag()	20
3.3.2.7 I2C_Init()	21
3.3.2.8 I2c_ReadBusState()	21
3.3.2.9 I2C_ReadData()	22
3.3.2.10 I2C_SendData()	22
3.3.2.11 I2C_SetBaudRate()	22
3.3.2.12 I2C_StructInit()	23

4 结构体说明	25
4.1 ADC_InitTypeDef结构体 参考	25
4.1.1 详细描述	26
4.1.2 结构体成员变量说明	26
4.1.2.1 CSMP	26
4.1.2.2 DATA_ALIGN	26
4.1.2.3 GAIN	26
4.1.2.4 GEN	26
4.1.2.5 HTH	26
4.1.2.6 IE	27
4.1.2.7 IS1	27
4.1.2.8 LTH	27
4.1.2.9 NSMP	27
4.1.2.10 OVSR	27
4.1.2.11 RE	27
4.1.2.12 S1	28
4.1.2.13 S2	28
4.1.2.14 TCNT	28
4.1.2.15 TRIG	28
4.1.2.16 TROVS	28
4.2 ADC_TypeDef结构体 参考	28
4.2.1 详细描述	31
4.2.2 结构体成员变量说明	31
4.2.2.1 AMC0	31
4.2.2.2 AMC1	31
4.2.2.3 CFG	31
4.2.2.4 CHN0	31
4.2.2.5 CHN1	32
4.2.2.6 CHN2	32
4.2.2.7 CHN3	32
4.2.2.8 CHNT	32
4.2.2.9 DAT0	32
4.2.2.10 DAT1	32
4.2.2.11 DAT10	33
4.2.2.12 DAT11	33
4.2.2.13 DAT12	33
4.2.2.14 DAT13	33
4.2.2.15 DAT2	33
4.2.2.16 DAT3	33
4.2.2.17 DAT4	34
4.2.2.18 DAT5	34
4.2.2.19 DAT6	34

4.2.2.20 DAT7	34
4.2.2.21 DAT8	34
4.2.2.22 DAT9	34
4.2.2.23 DC0	35
4.2.2.24 DC1	35
4.2.2.25 GAIN	35
4.2.2.26 GEN	35
4.2.2.27 HTH	35
4.2.2.28 ICHN	35
4.2.2.29 IDAT0	36
4.2.2.30 IDAT1	36
4.2.2.31 IE	36
4.2.2.32 IF	36
4.2.2.33 LTH	36
4.2.2.34 null	36
4.2.2.35 null1	37
4.2.2.36 null10	37
4.2.2.37 null11	37
4.2.2.38 null12	37
4.2.2.39 null13	37
4.2.2.40 null14	37
4.2.2.41 null15	38
4.2.2.42 null16	38
4.2.2.43 null2	38
4.2.2.44 null3	38
4.2.2.45 null4	38
4.2.2.46 null5	38
4.2.2.47 null6	39
4.2.2.48 null7	39
4.2.2.49 null8	39
4.2.2.50 null9	39
4.2.2.51 SWT	39
4.2.2.52 TRIG	39
4.3 AFE_TypeDef结构体 参考	40
4.3.1 详细描述	40
4.3.2 结构体成员变量说明	40
4.3.2.1 ADC	40
4.3.2.2 DAC0	41
4.3.2.3 DAC0_AMC	41
4.3.2.4 DAC0_DC	41
4.3.2.5 DAC1	41
4.3.2.6 DAC1_AMC	41

4.3.2.7 DAC1_DC	41
4.3.2.8 DAC_CTRL	41
4.3.2.9 DBG	41
4.3.2.10 INFO	42
4.3.2.11 null1	42
4.3.2.12 null2	42
4.3.2.13 null3	42
4.3.2.14 null4	42
4.3.2.15 null5	42
4.3.2.16 null6	42
4.3.2.17 null7	42
4.3.2.18 REG0	43
4.3.2.19 REG1	43
4.3.2.20 REG2	43
4.3.2.21 REG3	43
4.3.2.22 REG4	43
4.3.2.23 REG5	43
4.3.2.24 REG6	43
4.3.2.25 REG7	43
4.3.2.26 REG8	44
4.3.2.27 REG9	44
4.3.2.28 REGA	44
4.3.2.29 REGB	44
4.3.2.30 REGC	44
4.4 CAN_InitTypeDef结构体 参考	44
4.4.1 详细描述	44
4.4.2 结构体成员变量说明	45
4.4.2.1 ERRINT_IE	45
4.4.2.2 IE	45
4.4.2.3 SBaud	45
4.5 can_parameter结构体 参考	45
4.5.1 详细描述	45
4.5.2 结构体成员变量说明	46
4.5.2.1 id	46
4.5.2.2 ide	46
4.5.2.3 len	46
4.5.2.4 rtr	46
4.5.2.5 RX	46
4.5.2.6 rx_sta	46
4.5.2.7 TX	47
4.6 CAN_TypeDef结构体 参考	47
4.6.1 详细描述	48

4.6.2 结构体成员变量说明	48
4.6.2.1 ACF	48
4.6.2.2 ACFCTRL	48
4.6.2.3 ACFEN	48
4.6.2.4 CFG_STAT	48
4.6.2.5 EALCAP	49
4.6.2.6 ERRINT	49
4.6.2.7 LIMIT	49
4.6.2.8 RBUF	49
4.6.2.9 RCTRL	49
4.6.2.10 RECNT	49
4.6.2.11 resv0	50
4.6.2.12 resv1	50
4.6.2.13 resv2	50
4.6.2.14 resv3	50
4.6.2.15 resv4	50
4.6.2.16 RTIE	50
4.6.2.17 RTIF	51
4.6.2.18 SBAUD	51
4.6.2.19 TBUF	51
4.6.2.20 TCMD	51
4.6.2.21 TCTRL	51
4.6.2.22 TECNT	51
4.7 CANRxMsg结构体 参考	52
4.7.1 详细描述	52
4.7.2 结构体成员变量说明	52
4.7.2.1 buf	52
4.7.2.2 id	52
4.7.2.3 ide	52
4.7.2.4 len	52
4.7.2.5 rtr	53
4.7.2.6 valid	53
4.8 CANTxMsg结构体 参考	53
4.8.1 详细描述	53
4.8.2 结构体成员变量说明	53
4.8.2.1 buf	53
4.8.2.2 id	53
4.8.2.3 ide	54
4.8.2.4 len	54
4.8.2.5 rtr	54
4.8.2.6 ss	54
4.8.2.7 tbsel	54

4.9 CLU_InitTypeDef结构体 参考	54
4.9.1 详细描述	55
4.9.2 结构体成员变量说明	55
4.9.2.1 CLKINV	55
4.9.2.2 CLKSEL	55
4.9.2.3 EN	56
4.9.2.4 FIE	56
4.9.2.5 FN	56
4.9.2.6 MXA	56
4.9.2.7 MXB	56
4.9.2.8 OEN	56
4.9.2.9 OUTSEL	57
4.9.2.10 RIE	57
4.10 CLU_TypeDef结构体 参考	57
4.10.1 详细描述	57
4.10.2 结构体成员变量说明	58
4.10.2.1 CF	58
4.10.2.2 EN	58
4.10.2.3 FN	58
4.10.2.4 IE	58
4.10.2.5 IF	58
4.10.2.6 MX	58
4.10.2.7 OUT	59
4.11 CMP_InitTypeDef结构体 参考	59
4.11.1 详细描述	59
4.11.2 结构体成员变量说明	60
4.11.2.1 CLK_COM_DIV	60
4.11.2.2 CMP0	60
4.11.2.3 CMP1	60
4.11.2.4 CMP2	60
4.11.2.5 FT	60
4.11.2.6 HYS	60
4.12 CMP_SingleTypeDef结构体 参考	61
4.12.1 详细描述	61
4.12.2 结构体成员变量说明	62
4.12.2.1 CHN0P_PWM.EN	62
4.12.2.2 CHN1P_PWM.EN	62
4.12.2.3 CHN2P_PWM.EN	62
4.12.2.4 CHN3P_PWM.EN	62
4.12.2.5 CLK_EN	62
4.12.2.6 EN	62
4.12.2.7 FIL_CLK_DIV16	63

4.12.2.8 FIL_CLK_DIV2	63
4.12.2.9 IE	63
4.12.2.10 IN_EN	63
4.12.2.11 IRQ_TRIG	63
4.12.2.12 POL	64
4.12.2.13 RE	64
4.12.2.14 SELN	64
4.12.2.15 SELP	64
4.12.2.16 W_PWM_POL	65
4.13 CMP_TypeDef结构体 参考	65
4.13.1 详细描述	65
4.13.2 结构体成员变量说明	65
4.13.2.1 BLCWIN	65
4.13.2.2 CFG	66
4.13.2.3 DATA	66
4.13.2.4 IE	66
4.13.2.5 IF	66
4.13.2.6 TCLK	66
4.14 CRC_InitTypeDef结构体 参考	66
4.14.1 详细描述	67
4.14.2 结构体成员变量说明	67
4.14.2.1 INIT	67
4.14.2.2 null	67
4.14.2.3 POLY	67
4.14.2.4 REFIN	68
4.14.2.5 REFOUT	68
4.14.2.6 WIDTH	68
4.14.2.7 XOROUT	68
4.15 CRC_TypeDef结构体 参考	68
4.15.1 详细描述	69
4.15.2 结构体成员变量说明	69
4.15.2.1 CR	69
4.15.2.2 DR	69
4.15.2.3 INIT	69
4.15.2.4 POL	69
4.16 DAC_InitTypeDef结构体 参考	69
4.16.1 详细描述	70
4.16.2 结构体成员变量说明	70
4.16.2.1 DAC_GAIN	70
4.16.2.2 DAC_STEP	70
4.16.2.3 DACOUT_EN	70
4.16.2.4 TIG_CH_EN	71

4.17 DMA.InitTypeDef结构体 参考	71
4.17.1 详细描述	71
4.17.2 结构体成员变量说明	72
4.17.2.1 CIRC	72
4.17.2.2 DADR	72
4.17.2.3 DBTW	72
4.17.2.4 DINC	72
4.17.2.5 EN	72
4.17.2.6 IRQ_EN	72
4.17.2.7 REN	72
4.17.2.8 RMODE	73
4.17.2.9 SADR	73
4.17.2.10 SBTW	73
4.17.2.11 SINC	73
4.17.2.12 TMS	73
4.18 DMA.MODULERegTypeDef结构体 参考	73
4.18.1 详细描述	74
4.18.2 结构体成员变量说明	74
4.18.2.1 CTRL	74
4.18.2.2 IE	74
4.18.2.3 IF	74
4.19 DMA.RegTypeDef结构体 参考	74
4.19.1 详细描述	75
4.19.2 结构体成员变量说明	75
4.19.2.1 CCR	75
4.19.2.2 CTMS	75
4.19.2.3 DADR	75
4.19.2.4 REN	75
4.19.2.5 SADR	75
4.20 DSP.TypeDef结构体 参考	76
4.20.1 详细描述	76
4.20.2 结构体成员变量说明	76
4.20.2.1 ARCTAN	76
4.20.2.2 COS	76
4.20.2.3 DID	76
4.20.2.4 DIS	77
4.20.2.5 MOD	77
4.20.2.6 PC	77
4.20.2.7 QUO	77
4.20.2.8 RAD	77
4.20.2.9 REM	77
4.20.2.10 SC	77

4.20.2.11 SIN	77
4.20.2.12 SQRT	78
4.20.2.13 THETA	78
4.20.2.14 X	78
4.20.2.15 Y	78
4.21 EXTI_TypeDef结构体 参考	78
4.21.1 详细描述	78
4.21.2 结构体成员变量说明	79
4.21.2.1 CR0	79
4.21.2.2 CR1	79
4.21.2.3 IE	79
4.21.2.4 IF	79
4.22 GPIO_InitTypeDef结构体 参考	79
4.22.1 详细描述	80
4.22.2 结构体成员变量说明	80
4.22.2.1 GPIO_Mode	80
4.22.2.2 GPIO_PFLT	80
4.22.2.3 GPIO_Pin	80
4.22.2.4 GPIO_PODEna	80
4.22.2.5 GPIO_PuPd	80
4.23 GPIO_TypeDef结构体 参考	81
4.23.1 详细描述	81
4.23.2 结构体成员变量说明	81
4.23.2.1 BRR	81
4.23.2.2 BSRR	81
4.23.2.3 F3210	82
4.23.2.4 F7654	82
4.23.2.5 FBA98	82
4.23.2.6 FFEDC	82
4.23.2.7 null	82
4.23.2.8 PDI	82
4.23.2.9 PDO	83
4.23.2.10 PFLT	83
4.23.2.11 PIE	83
4.23.2.12 PODE	83
4.23.2.13 POE	83
4.23.2.14 PUE	83
4.24 HALL_InitTypeDef结构体 参考	84
4.24.1 详细描述	84
4.24.2 结构体成员变量说明	84
4.24.2.1 Capture_IRQ_Ena	84
4.24.2.2 ClockDivision	84

4.24.2.3 CountTH	85
4.24.2.4 Filter75_Ena	85
4.24.2.5 FilterLen	85
4.24.2.6 HALL_Ena	85
4.24.2.7 OverFlow_IRQ_Ena	85
4.24.2.8 softIE	85
4.25 HALL_TypeDef结构体 参考	86
4.25.1 详细描述	86
4.25.2 结构体成员变量说明	86
4.25.2.1 CFG	86
4.25.2.2 CNT	86
4.25.2.3 INFO	86
4.25.2.4 TH	87
4.25.2.5 WIDTH	87
4.26 I2C_InitTypeDef结构体 参考	87
4.26.1 详细描述	87
4.26.2 结构体成员变量说明	88
4.26.2.1 ADRCMP	88
4.26.2.2 BaudRate	88
4.26.2.3 BURST_ADDR_CMP	88
4.26.2.4 BURST_NACK	88
4.26.2.5 BUS_ERR_IE	88
4.26.2.6 BUSRT_EN	88
4.26.2.7 IE	89
4.26.2.8 MST_MODE	89
4.26.2.9 SLV_MODE	89
4.26.2.10 STOP_IE	89
4.26.2.11 TC_IE	89
4.27 I2C_TypeDef结构体 参考	89
4.27.1 详细描述	90
4.27.2 结构体成员变量说明	90
4.27.2.1 ADDR	90
4.27.2.2 BCR	90
4.27.2.3 BSIZE	90
4.27.2.4 CFG	91
4.27.2.5 DATA	91
4.27.2.6 MSCR	91
4.27.2.7 SCR	91
4.28 IWDG_InitTypeDef结构体 参考	91
4.28.1 详细描述	92
4.28.2 结构体成员变量说明	92
4.28.2.1 DWK_EN	92

4.28.2.2 RTH	92
4.28.2.3 WDG_EN	92
4.28.2.4 WTH	92
4.29 IWDG_TypeDef结构体 参考	92
4.29.1 详细描述	93
4.29.2 结构体成员变量说明	93
4.29.2.1 CFG	93
4.29.2.2 CLR	93
4.29.2.3 CNT	93
4.29.2.4 PSW	94
4.29.2.5 RTH	94
4.29.2.6 WTH	94
4.30 MCPWM_InitTypeDef结构体 参考	94
4.30.1 详细描述	102
4.30.2 结构体成员变量说明	102
4.30.2.1 AUEN	102
4.30.2.2 BASE_CNT0_EN	102
4.30.2.3 BASE_CNT1_EN	102
4.30.2.4 CH0_NS	102
4.30.2.5 CH0_PS	102
4.30.2.6 CH0N_default_output	103
4.30.2.7 CH0N_FAIL_EN	103
4.30.2.8 CH0N_Polarity_INV	103
4.30.2.9 CH0N_SCTRLN	103
4.30.2.10 CH0P_default_output	103
4.30.2.11 CH0P_FAIL_EN	103
4.30.2.12 CH0P_Polarity_INV	104
4.30.2.13 CH0P_SCTRLP	104
4.30.2.14 CH1_NS	104
4.30.2.15 CH1_PS	104
4.30.2.16 CH1N_default_output	104
4.30.2.17 CH1N_FAIL_EN	104
4.30.2.18 CH1N_Polarity_INV	105
4.30.2.19 CH1N_SCTRLN	105
4.30.2.20 CH1P_default_output	105
4.30.2.21 CH1P_FAIL_EN	105
4.30.2.22 CH1P_Polarity_INV	105
4.30.2.23 CH1P_SCTRLP	105
4.30.2.24 CH2_NS	106
4.30.2.25 CH2_PS	106
4.30.2.26 CH2N_default_output	106
4.30.2.27 CH2N_FAIL_EN	106

4.30.2.28 CH2N_Polarity_INV	106
4.30.2.29 CH2N_SCTRLN	106
4.30.2.30 CH2P_default_output	107
4.30.2.31 CH2P_FAIL_EN	107
4.30.2.32 CH2P_Polarity_INV	107
4.30.2.33 CH2P_SCTRLP	107
4.30.2.34 CH3_NS	107
4.30.2.35 CH3_PS	107
4.30.2.36 CH3N_default_output	108
4.30.2.37 CH3N_FAIL_EN	108
4.30.2.38 CH3N_Polarity_INV	108
4.30.2.39 CH3N_SCTRLN	108
4.30.2.40 CH3P_default_output	108
4.30.2.41 CH3P_FAIL_EN	108
4.30.2.42 CH3P_Polarity_INV	109
4.30.2.43 CH3P_SCTRLP	109
4.30.2.44 CH4_NS	109
4.30.2.45 CH4_PS	109
4.30.2.46 CH4N_default_output	109
4.30.2.47 CH4N_FAIL_EN	109
4.30.2.48 CH4N_Polarity_INV	110
4.30.2.49 CH4N_SCTRLN	110
4.30.2.50 CH4P_default_output	110
4.30.2.51 CH4P_FAIL_EN	110
4.30.2.52 CH4P_Polarity_INV	110
4.30.2.53 CH4P_SCTRLP	110
4.30.2.54 CH5_NS	111
4.30.2.55 CH5_PS	111
4.30.2.56 CH5N_default_output	111
4.30.2.57 CH5N_FAIL_EN	111
4.30.2.58 CH5N_Polarity_INV	111
4.30.2.59 CH5N_SCTRLN	111
4.30.2.60 CH5P_default_output	112
4.30.2.61 CH5P_FAIL_EN	112
4.30.2.62 CH5P_Polarity_INV	112
4.30.2.63 CH5P_SCTRLP	112
4.30.2.64 CLK_DIV	112
4.30.2.65 CMP_CTRL_CNT0	112
4.30.2.66 CMP_CTRL_CNT1	113
4.30.2.67 DeadTimeCH012N	113
4.30.2.68 DeadTimeCH012P	113
4.30.2.69 DeadTimeCH345N	113

4.30.2.70 DeadTimeCH345P	113
4.30.2.71 EVT0	113
4.30.2.72 EVT1	114
4.30.2.73 EVT_CNT0_EN	114
4.30.2.74 EVT_CNT1_EN	114
4.30.2.75 FAIL0_INPUT_EN	114
4.30.2.76 FAIL0_INT_EN	114
4.30.2.77 FAIL0_Polarity	114
4.30.2.78 FAIL0_Signal_Sel	115
4.30.2.79 FAIL1_INPUT_EN	115
4.30.2.80 FAIL1_INT_EN	115
4.30.2.81 FAIL1_Polarity	115
4.30.2.82 FAIL1_Signal_Sel	115
4.30.2.83 FAIL2_INPUT_EN	115
4.30.2.84 FAIL2_INT_EN	116
4.30.2.85 FAIL2_Polarity	116
4.30.2.86 FAIL2_Signal_Sel	116
4.30.2.87 FAIL3_INPUT_EN	116
4.30.2.88 FAIL3_INT_EN	116
4.30.2.89 FAIL3_Polarity	116
4.30.2.90 FAIL3_Signal_Sel	117
4.30.2.91 FAIL_0CAP	117
4.30.2.92 FAIL_1CAP	117
4.30.2.93 HALT_PRT0	117
4.30.2.94 HALT_PRT1	117
4.30.2.95 IO0_PPE	117
4.30.2.96 IO1_PPE	118
4.30.2.97 IO2_PPE	118
4.30.2.98 IO3_PPE	118
4.30.2.99 IO4_PPE	118
4.30.2.100 IO5_PPE	118
4.30.2.101 IO_CMP_FLT_CLKDIV	118
4.30.2.102 MCLK_EN	119
4.30.2.103 MCPWM_WorkModeCH0	119
4.30.2.104 MCPWM_WorkModeCH1	119
4.30.2.105 MCPWM_WorkModeCH2	119
4.30.2.106 MCPWM_WorkModeCH3	119
4.30.2.107 MCPWM_WorkModeCH4	119
4.30.2.108 MCPWM_WorkModeCH5	120
4.30.2.109 STT_HYST	120
4.30.2.110 Switch_CH0N_CH0P	120
4.30.2.111 Switch_CH1N_CH1P	120

4.30.2.112 Switch_CH2N_CH2P	120
4.30.2.113 Switch_CH3N_CH3P	120
4.30.2.114 Switch_CH4N_CH4P	121
4.30.2.115 Switch_CH5N_CH5P	121
4.30.2.116 T0_Update0_INT_EN	121
4.30.2.117 T0_Update1_INT_EN	121
4.30.2.118 T1_Update0_INT_EN	121
4.30.2.119 T1_Update1_INT_EN	121
4.30.2.120 TH0	122
4.30.2.121 TH00	122
4.30.2.122 TH00_Match0_INT_EN	122
4.30.2.123 TH00_Match1_INT_EN	122
4.30.2.124 TH01	122
4.30.2.125 TH01_Match0_INT_EN	122
4.30.2.126 TH01_Match1_INT_EN	123
4.30.2.127 TH1	123
4.30.2.128 TH10	123
4.30.2.129 TH10_Match0_INT_EN	123
4.30.2.130 TH10_Match1_INT_EN	123
4.30.2.131 TH11	123
4.30.2.132 TH11_Match0_INT_EN	124
4.30.2.133 TH11_Match1_INT_EN	124
4.30.2.134 TH20	124
4.30.2.135 TH20_Match0_INT_EN	124
4.30.2.136 TH20_Match1_INT_EN	124
4.30.2.137 TH21	124
4.30.2.138 TH21_Match0_INT_EN	125
4.30.2.139 TH21_Match1_INT_EN	125
4.30.2.140 TH30	125
4.30.2.141 TH31	125
4.30.2.142 TH40	125
4.30.2.143 TH41	125
4.30.2.144 TH50	126
4.30.2.145 TH51	126
4.30.2.146 TMR0	126
4.30.2.147 TMR0_Match0_INT_EN	126
4.30.2.148 TMR0_Match1_INT_EN	126
4.30.2.149 TMR0_RE0	126
4.30.2.150 TMR0_RE1	127
4.30.2.151 TMR1	127
4.30.2.152 TMR1_Match0_INT_EN	127
4.30.2.153 TMR1_Match1_INT_EN	127

4.30.2.154 TMR1_RE0	127
4.30.2.155 TMR1_RE1	127
4.30.2.156 TMR2	128
4.30.2.157 TMR2_Match0_INT_EN	128
4.30.2.158 TMR2_Match1_INT_EN	128
4.30.2.159 TMR2_RE0	128
4.30.2.160 TMR2_RE1	128
4.30.2.161 TMR2_TB	128
4.30.2.162 TMR3	129
4.30.2.163 TMR3_Match0_INT_EN	129
4.30.2.164 TMR3_Match1_INT_EN	129
4.30.2.165 TMR3_RE0	129
4.30.2.166 TMR3_RE1	129
4.30.2.167 TMR3_TB	129
4.30.2.168 TR0_AEC	130
4.30.2.169 TR0_T0_RE0	130
4.30.2.170 TR0_T0_RE1	130
4.30.2.171 TR0_T0_UpdateEN	130
4.30.2.172 TR0_T1_RE0	130
4.30.2.173 TR0_T1_RE1	130
4.30.2.174 TR0_T1_UpdateEN	131
4.30.2.175 TR0_UP_INTV	131
4.30.2.176 TR1_AEC	131
4.30.2.177 TR1_T0_RE0	131
4.30.2.178 TR1_T0_RE1	131
4.30.2.179 TR1_T0_UpdateEN	131
4.30.2.180 TR1_T1_RE0	132
4.30.2.181 TR1_T1_RE1	132
4.30.2.182 TR1_T1_UpdateEN	132
4.30.2.183 TR1_UP_INTV	132
4.30.2.184 Update0_INT_EN	132
4.30.2.185 Update1_INT_EN	132
4.30.2.186 ZCS0_INT_EN	133
4.30.2.187 ZCS1_INT_EN	133
4.30.2.188 ZCS_DELAY	133
4.30.2.189 ZCS_EN	133
4.31 MCPWM.TypeDef结构体 参考	133
4.31.1 详细描述	136
4.31.2 结构体成员变量说明	136
4.31.2.1 AUEN	136
4.31.2.2 CH_DEF	136
4.31.2.3 CH_MSK	136

4.31.2.4 CNT0	136
4.31.2.5 CNT1	136
4.31.2.6 DTH00	137
4.31.2.7 DTH01	137
4.31.2.8 DTH10	137
4.31.2.9 DTH11	137
4.31.2.10 EIE	137
4.31.2.11 EIF	137
4.31.2.12 EVT0	138
4.31.2.13 EVT1	138
4.31.2.14 FAIL012	138
4.31.2.15 FAIL345	138
4.31.2.16 FCNT	138
4.31.2.17 FLT	138
4.31.2.18 IE0	139
4.31.2.19 IE1	139
4.31.2.20 IF0	139
4.31.2.21 IF1	139
4.31.2.22 IO01	139
4.31.2.23 IO23	139
4.31.2.24 IO45	140
4.31.2.25 null	140
4.31.2.26 PP	140
4.31.2.27 PRT	140
4.31.2.28 RE	140
4.31.2.29 SDCFG	140
4.31.2.30 STT_HYST	141
4.31.2.31 TCLK	141
4.31.2.32 TH0	141
4.31.2.33 TH00	141
4.31.2.34 TH01	141
4.31.2.35 TH1	141
4.31.2.36 TH10	142
4.31.2.37 TH11	142
4.31.2.38 TH20	142
4.31.2.39 TH21	142
4.31.2.40 TH30	142
4.31.2.41 TH31	142
4.31.2.42 TH40	143
4.31.2.43 TH41	143
4.31.2.44 TH50	143
4.31.2.45 TH51	143

4.31.2.46 TMR0	143
4.31.2.47 TMR1	143
4.31.2.48 TMR2	144
4.31.2.49 TMR3	144
4.31.2.50 UPDATE	144
4.31.2.51 ZCS_DELAY	144
4.32 OPA_InitTypeDef结构体 参考	144
4.32.1 详细描述	145
4.32.2 结构体成员变量说明	145
4.32.2.1 OPA_CLEna	145
4.32.2.2 OPA_Gain	145
4.32.2.3 OPA_IT	145
4.33 SIF_InitTypeDef结构体 参考	146
4.33.1 详细描述	146
4.33.2 结构体成员变量说明	146
4.33.2.1 SIF_DMA.EN	146
4.33.2.2 SIF_DONE	146
4.33.2.3 SIF_DONE_VLD	146
4.33.2.4 SIF_EN	147
4.33.2.5 SIF_IRQ.EN	147
4.33.2.6 SIF_IRQ_IF	147
4.33.2.7 SIF_MSB	147
4.33.2.8 SIF_RATIO	147
4.33.2.9 SIF_SYNC	147
4.33.2.10 SIF_SYNC_PULSE	147
4.33.2.11 SIF_SYNC_VLD	147
4.33.2.12 SIF_TDTH1	148
4.33.2.13 SIF_TOSC	148
4.33.2.14 SIF_TSTH1	148
4.34 SIF_RegTypeDef结构体 参考	148
4.34.1 详细描述	148
4.34.2 结构体成员变量说明	148
4.34.2.1 SIFx_CFG	148
4.34.2.2 SIFx_IRQ	149
4.34.2.3 SIFx_TDTH1	149
4.34.2.4 SIFx_TOSC	149
4.34.2.5 SIFx_TSTH1	149
4.35 SPI_InitTypeDef结构体 参考	149
4.35.1 详细描述	150
4.35.2 结构体成员变量说明	150
4.35.2.1 BaudRate	150
4.35.2.2 ByteLength	150

4.35.2.3 CPHA	150
4.35.2.4 CPOL	150
4.35.2.5 CS	150
4.35.2.6 Duplex	151
4.35.2.7 EN	151
4.35.2.8 ENDIAN	151
4.35.2.9 IRQEna	151
4.35.2.10 Mode	151
4.35.2.11 TRANS_MODE	151
4.35.2.12 Trig	152
4.36 SPI_TypeDef结构体 参考	152
4.36.1 详细描述	152
4.36.2 结构体成员变量说明	152
4.36.2.1 BAUD	152
4.36.2.2 CFG	153
4.36.2.3 IE	153
4.36.2.4 RX_DATA	153
4.36.2.5 SIZE	153
4.36.2.6 TX_DATA	153
4.37 stru_DMA_REN联合体 参考	153
4.37.1 详细描述	154
4.37.2 结构体成员变量说明	154
4.37.2.1 __pad0__	154
4.37.2.2 __pad1__	155
4.37.2.3 __pad2__	155
4.37.2.4 __pad3__	155
4.37.2.5 __pad4__	155
4.37.2.6 __pad5__	155
4.37.2.7 __pad6__	155
4.37.2.8 __pad7__	155
4.37.2.9 __pad8__	155
4.37.2.10 __pad9__	156
4.37.2.11 ADC0_EN	156
4.37.2.12 ADC1_EN	156
4.37.2.13 [struct]	156
4.37.2.14 CMP_EN	156
4.37.2.15 DSP_EN	156
4.37.2.16 GPIO_EN	156
4.37.2.17 HALL0_EN	156
4.37.2.18 I2C0_RX_EN	157
4.37.2.19 I2C0_TX_EN	157
4.37.2.20 MCPWM_RE0_EN	157

4.37.2.21 MCPWM_RE1_EN	157
4.37.2.22 R	157
4.37.2.23 SIF_EN	157
4.37.2.24 SPI_RX_EN	157
4.37.2.25 SPI_TX_EN	157
4.37.2.26 SW_TRIG_EN	158
4.37.2.27 TIMER0_EN	158
4.37.2.28 TIMER1_EN	158
4.37.2.29 TIMER2_EN	158
4.37.2.30 TIMER3_EN	158
4.37.2.31 UART0_RX_EN	158
4.37.2.32 UART0_TX_EN	158
4.37.2.33 UART1_RX_EN	158
4.37.2.34 UART1_TX_EN	159
4.38 Stru_TempertureCof_Def结构体 参考	159
4.38.1 详细描述	159
4.38.2 结构体成员变量说明	159
4.38.2.1 nCofA	159
4.38.2.2 nOffsetB	159
4.39 SYS_InitTypeDef结构体 参考	160
4.39.1 详细描述	160
4.39.2 结构体成员变量说明	160
4.39.2.1 Clk_DivSPI	160
4.39.2.2 Clk_DivUART	160
4.39.2.3 Clk_FEN	160
4.39.2.4 Clk_Sel	161
4.39.2.5 PLL_DivSel	161
4.39.2.6 PLL_SrcSel	161
4.39.2.7 PORFilter_Ena	161
4.39.2.8 WDT_Ena	161
4.40 SYS_TypeDef结构体 参考	161
4.40.1 详细描述	162
4.40.2 结构体成员变量说明	162
4.40.2.1 CLK_CFG	162
4.40.2.2 CLK_DIV0	162
4.40.2.3 CLK_DIV1	162
4.40.2.4 CLK_DIV2	163
4.40.2.5 CLK_FEN	163
4.40.2.6 DBG_CFG	163
4.40.2.7 FLSE	163
4.40.2.8 FLSP	163
4.40.2.9 FLST	163

4.40.2.10 IO_CFG	163
4.40.2.11 MBIST	163
4.40.2.12 null	164
4.40.2.13 null10	164
4.40.2.14 null7	164
4.40.2.15 null8	164
4.40.2.16 null9	164
4.40.2.17 SBIST	164
4.40.2.18 SFT_RST	164
4.40.2.19 TRIM	164
4.40.2.20 WR.PROTECT	165
4.41 TIM_QEPInitTypeDef结构体 参考	165
4.41.1 详细描述	165
4.41.2 结构体成员变量说明	165
4.41.2.1 QEP_ClockDiv	165
4.41.2.2 QEP_FE_CNT_EN	166
4.41.2.3 QEP_Filter	166
4.41.2.4 QEP_IRQEna	166
4.41.2.5 QEP_Mode	167
4.41.2.6 QEP_TH	167
4.41.2.7 QEP_ZEC	167
4.41.2.8 QEP_ZLC	167
4.41.2.9 QEP_ZNC	167
4.41.2.10 QEP_ZPC	167
4.42 TIM_QEPTypedef结构体 参考	168
4.42.1 详细描述	168
4.42.2 结构体成员变量说明	168
4.42.2.1 CFG	168
4.42.2.2 CNT	168
4.42.2.3 IE	168
4.42.2.4 IF	169
4.42.2.5 TH	169
4.43 TIM_TimerInitTypeDef结构体 参考	169
4.43.1 详细描述	171
4.43.2 结构体成员变量说明	171
4.43.2.1 Timer0_CH0_DEFAULT	171
4.43.2.2 Timer0_CH1_DEFAULT	171
4.43.2.3 Timer0_FAIL_EN	171
4.43.2.4 Timer0_FAIL_POL	171
4.43.2.5 Timer0_FAIL_SEL	171
4.43.2.6 Timer0_HALT_PRT	172
4.43.2.7 Timer0_SHADOW	172

4.43.2.8 Timer0_UPDATE	172
4.43.2.9 Timer_CAP0_CLR_EN	172
4.43.2.10 Timer_CAP1_CLR_EN	172
4.43.2.11 Timer_CENTER	172
4.43.2.12 Timer_CH0_FE_CAP_EN	173
4.43.2.13 Timer_CH0_RE_CAP_EN	173
4.43.2.14 Timer_CH0_WorkMode	173
4.43.2.15 Timer_CH0Output	173
4.43.2.16 Timer_CH1_FE_CAP_EN	173
4.43.2.17 Timer_CH1_RE_CAP_EN	173
4.43.2.18 Timer_CH1_WorkMode	174
4.43.2.19 Timer_CH1Output	174
4.43.2.20 Timer_ClockDiv	174
4.43.2.21 Timer_CMP0	174
4.43.2.22 Timer_CMP1	175
4.43.2.23 Timer_CNT	175
4.43.2.24 Timer_DIR	175
4.43.2.25 Timer_EN	175
4.43.2.26 Timer_ETON	175
4.43.2.27 Timer_EVT	176
4.43.2.28 Timer_FLT	176
4.43.2.29 Timer_GATE_EN	176
4.43.2.30 Timer_IRQEna	176
4.43.2.31 Timer_ONE_TRIG	177
4.43.2.32 Timer_RL_EN	177
4.43.2.33 Timer_SRC0	177
4.43.2.34 Timer_SRC1	178
4.43.2.35 Timer_TH	179
4.43.2.36 Timer_XCLK_EN	179
4.44 TIM_TimerTypeDef结构体 参考	179
4.44.1 详细描述	179
4.44.2 结构体成员变量说明	180
4.44.2.1 CFG	180
4.44.2.2 CMP0	180
4.44.2.3 CMP1	180
4.44.2.4 CNT	180
4.44.2.5 EVT	180
4.44.2.6 FLT	180
4.44.2.7 IE	181
4.44.2.8 IF	181
4.44.2.9 IO	181
4.44.2.10 TH	181

4.45 UART_InitTypeDef结构体 参考	181
4.45.1 详细描述	182
4.45.2 结构体成员变量说明	182
4.45.2.1 BaudRate	182
4.45.2.2 Bit9Value	182
4.45.2.3 DMARE	183
4.45.2.4 DUPLEX	183
4.45.2.5 FirstSend	183
4.45.2.6 IRQEna	183
4.45.2.7 Match485Addr	183
4.45.2.8 MultiDropEna	184
4.45.2.9 ParityMode	184
4.45.2.10 RXD_INV	184
4.45.2.11 StopBits	184
4.45.2.12 TXD_INV	184
4.45.2.13 WordLength	185
4.46 UART_TypeDef结构体 参考	185
4.46.1 详细描述	185
4.46.2 结构体成员变量说明	186
4.46.2.1 ADR	186
4.46.2.2 BUFF	186
4.46.2.3 CTRL	186
4.46.2.4 DIVH	186
4.46.2.5 DIVL	186
4.46.2.6 IE	186
4.46.2.7 IF	187
4.46.2.8 IOC	187
4.46.2.9 RE	187
4.46.2.10 STT	187
4.47 WAKE_InitTypeDef结构体 参考	187
4.47.1 详细描述	187
4.47.2 结构体成员变量说明	188
4.47.2.1 AFE_REG5.RECORD	188
4.47.2.2 CLK_CFG.RECORD	188
5 文件说明	189
5.1 include/lks32mc07x_adc.h 文件参考	189
5.1.1 详细描述	192
5.1.2 宏定义说明	193
5.1.2.1 ADC0	193
5.1.2.2 ADC1	193
5.1.2.3 ADC_ALL_IF	193

5.1.2.4 ADC_AWD0_IE	193
5.1.2.5 ADC_AWD0_IF	193
5.1.2.6 ADC_AWD0_RE	194
5.1.2.7 ADC_CHANNEL_0	194
5.1.2.8 ADC_CHANNEL_1	194
5.1.2.9 ADC_CHANNEL_10	194
5.1.2.10 ADC_CHANNEL_11	194
5.1.2.11 ADC_CHANNEL_12	194
5.1.2.12 ADC_CHANNEL_13	195
5.1.2.13 ADC_CHANNEL_14	195
5.1.2.14 ADC_CHANNEL_15	195
5.1.2.15 ADC_CHANNEL_2	195
5.1.2.16 ADC_CHANNEL_3	195
5.1.2.17 ADC_CHANNEL_4	195
5.1.2.18 ADC_CHANNEL_5	196
5.1.2.19 ADC_CHANNEL_6	196
5.1.2.20 ADC_CHANNEL_7	196
5.1.2.21 ADC_CHANNEL_8	196
5.1.2.22 ADC_CHANNEL_9	196
5.1.2.23 ADC_HERR_IE	196
5.1.2.24 ADC_HERR_IF	197
5.1.2.25 ADC_HERR_RE	197
5.1.2.26 ADC_ISF_IE	197
5.1.2.27 ADC_ISF_IF	197
5.1.2.28 ADC_ISF_RE	197
5.1.2.29 ADC_SERR_IE	197
5.1.2.30 ADC_SERR_IF	198
5.1.2.31 ADC_SERR_RE	198
5.1.2.32 ADC_SF1_IE	198
5.1.2.33 ADC_SF1_IF	198
5.1.2.34 ADC_SF1_RE	198
5.1.2.35 ADC_SF2_IE	198
5.1.2.36 ADC_SF2_IF	199
5.1.2.37 ADC_SF2_RE	199
5.1.2.38 ADC_TRIG_CL_IT0	199
5.1.2.39 ADC_TRIG_CL_IT1	199
5.1.2.40 ADC_TRIG_CL_IT2	199
5.1.2.41 ADC_TRIG_CL_IT3	199
5.1.2.42 ADC_TRIG_CL_NT0	200
5.1.2.43 ADC_TRIG_CL_NT1	200
5.1.2.44 ADC_TRIG_CL_NT2	200
5.1.2.45 ADC_TRIG_CL_NT3	200

5.1.2.46 ADC_TRIG.MCPWM0_T0	200
5.1.2.47 ADC_TRIG.MCPWM0_T1	200
5.1.2.48 ADC_TRIG.MCPWM0_T2	201
5.1.2.49 ADC_TRIG.MCPWM0_T3	201
5.1.2.50 ADC_TRIG.UTIMER0_CMP0	201
5.1.2.51 ADC_TRIG.UTIMER0_CMP1	201
5.1.2.52 ADC_TRIG.UTIMER2_CMP0	201
5.1.2.53 ADC_TRIG.UTIMER2_CMP1	201
5.1.3 枚举类型说明	201
5.1.3.1 ADC_GAINx	201
5.1.3.2 CHNx	202
5.1.3.3 DATx	202
5.1.4 函数说明	203
5.1.4.1 ADC.CHN_GAIN_CFG()	203
5.1.4.2 ADC.ClearIRQFlag()	204
5.1.4.3 ADC.GetConversionValue()	206
5.1.4.4 ADC.GetIRQFlag()	206
5.1.4.5 ADC.Init()	208
5.1.4.6 ADC.SoftTrgEN()	209
5.1.4.7 ADC.StructInit()	210
5.2 lks32mc07x_adc.h	211
5.3 include/lks32mc07x_can.h 文件参考	213
5.3.1 详细描述	217
5.3.2 宏定义说明	218
5.3.2.1 CAN	218
5.3.2.2 CAN.ERRINT_ALIE	218
5.3.2.3 CAN.ERRINT_ALIF	218
5.3.2.4 CAN.ERRINT_BEIE	218
5.3.2.5 CAN.ERRINT_BEIF	218
5.3.2.6 CAN.ERRINT_EPASS	218
5.3.2.7 CAN.ERRINT_EPIE	219
5.3.2.8 CAN.ERRINT_EPIF	219
5.3.2.9 CAN.ERRINT_EWARN	219
5.3.2.10 CAN.MODE_LBME	219
5.3.2.11 CAN.MODE_LBMI	219
5.3.2.12 CAN.MODE_LOM	219
5.3.2.13 CAN.MODE_RESET	220
5.3.2.14 CAN.MODE_TPSS	220
5.3.2.15 CAN.MODE_TSSS	220
5.3.2.16 CAN.RTIE_EIE	220
5.3.2.17 CAN.RTIE_RAFIE	220
5.3.2.18 CAN.RTIE_RFIE	220

5.3.2.19 CAN_RTIE_RIE	221
5.3.2.20 CAN_RTIE_ROIE	221
5.3.2.21 CAN_RTIE_TPIE	221
5.3.2.22 CAN_RTIE_TSFF	221
5.3.2.23 CAN_RTIE_TSIE	221
5.3.2.24 CAN_RTIF_AIF	221
5.3.2.25 CAN_RTIF_EIF	222
5.3.2.26 CAN_RTIF_RAFIF	222
5.3.2.27 CAN_RTIF_RFIF	222
5.3.2.28 CAN_RTIF_RIF	222
5.3.2.29 CAN_RTIF_ROIF	222
5.3.2.30 CAN_RTIF_TPIF	222
5.3.2.31 CAN_RTIF_TSIF	223
5.3.2.32 CANRXMSG_HEADLEN	223
5.3.2.33 ID0_Filter	223
5.3.2.34 ID1_Filter	223
5.3.2.35 ID2_Filter	223
5.3.2.36 ID3_Filter	223
5.3.3 枚举类型说明	223
5.3.3.1 CANErrorKind	223
5.3.3.2 CanRecvRet	224
5.3.3.3 CanSendRet	224
5.3.3.4 IDEFilterOption	224
5.3.3.5 STBTransMode	225
5.3.3.6 STBTransOption	225
5.3.3.7 TBSelOption	225
5.3.4 函数说明	225
5.3.4.1 CAN_ArbitrationLostCap()	225
5.3.4.2 CAN_BusoffGet()	226
5.3.4.3 CAN_Disable()	226
5.3.4.4 CAN_Enable()	227
5.3.4.5 CAN_ErrorPassiveGet()	227
5.3.4.6 CAN_Init()	227
5.3.4.7 CAN_Init1()	228
5.3.4.8 CAN_KindOfErrorGet()	229
5.3.4.9 CAN_LBMECmd()	229
5.3.4.10 CAN_LBMICmd()	230
5.3.4.11 CAN_Receive_Msg()	231
5.3.4.12 CAN_Recv()	232
5.3.4.13 CAN_Reset()	232
5.3.4.14 CAN_ROverflowGet()	233
5.3.4.15 CAN_RxErrorCntGet()	233

5.3.4.16 CAN_Send()	234
5.3.4.17 CAN_Send_Cancel()	234
5.3.4.18 CAN_Send_Trig()	234
5.3.4.19 CAN_SetSTBMode()	235
5.3.4.20 CAN_SS_Enable()	235
5.3.4.21 CAN_STBFullGet()	236
5.3.4.22 CAN_StructInit()	236
5.3.4.23 CAN_TxErrorCntGet()	236
5.3.4.24 IDx_Filter()	236
5.3.4.25 IDx_FilterCmd()	237
5.3.4.26 My_CAN_Send_Msg()	237
5.3.5 变量说明	237
5.3.5.1 g_rxMsg	237
5.4 lks32mc07x.can.h	238
5.5 include/lks32mc07x.clu.h 文件参考	240
5.5.1 详细描述	248
5.5.2 宏定义说明	248
5.5.2.1 CLU_FN_0	248
5.5.2.2 CLU_FN_1	248
5.5.2.3 CLU_FN_A	248
5.5.2.4 CLU_FN_A.B	248
5.5.2.5 CLU_FN_A.B.C	249
5.5.2.6 CLU_FN_A.B.NOTC	249
5.5.2.7 CLU_FN_A.C	249
5.5.2.8 CLU_FN_A.NOTB	249
5.5.2.9 CLU_FN_A.NOTB.C	249
5.5.2.10 CLU_FN_A.NOTB.NOTC	249
5.5.2.11 CLU_FN_A.NOTC	250
5.5.2.12 CLU_FN_B	250
5.5.2.13 CLU_FN_B.C	250
5.5.2.14 CLU_FN_B.NOTC	250
5.5.2.15 CLU_FN_C	250
5.5.2.16 CLU_FN.NOTA	250
5.5.2.17 CLU_FN.NOTA.B	251
5.5.2.18 CLU_FN.NOTA.B.C	251
5.5.2.19 CLU_FN.NOTA.B.NOTC	251
5.5.2.20 CLU_FN.NOTA.C	251
5.5.2.21 CLU_FN.NOTA.NOTB	251
5.5.2.22 CLU_FN.NOTA.NOTB.C	251
5.5.2.23 CLU_FN.NOTA.NOTB.NOTC	252
5.5.2.24 CLU_FN.NOTA.NOTC	252
5.5.2.25 CLU_FN.NOTB	252

5.5.2.26 CLU_FN_NOTB.C	252
5.5.2.27 CLU_FN_NOTB.NOTC	252
5.5.2.28 CLU_FN_NOTC	252
5.5.2.29 CLU_MODEL_CLU0	253
5.5.2.30 CLU_MODEL_CLU1	253
5.5.2.31 CLU_MODEL_CLU2	253
5.5.2.32 CLU_MODEL_CLU3	253
5.5.2.33 CLU_MXA_CLU0.CLU0OUT	253
5.5.2.34 CLU_MXA_CLU0.CLU1OUT	253
5.5.2.35 CLU_MXA_CLU0.CLU2OUT	254
5.5.2.36 CLU_MXA_CLU0.CLU3OUT	254
5.5.2.37 CLU_MXA_CLU0.P0_0	254
5.5.2.38 CLU_MXA_CLU0.P0_12	254
5.5.2.39 CLU_MXA_CLU0.P0_14	254
5.5.2.40 CLU_MXA_CLU0.P0_3	254
5.5.2.41 CLU_MXA_CLU0.P0_5	255
5.5.2.42 CLU_MXA_CLU0.P0_7	255
5.5.2.43 CLU_MXA_CLU0.P1_0	255
5.5.2.44 CLU_MXA_CLU0.P2_12	255
5.5.2.45 CLU_MXA_CLU0.PWM0	255
5.5.2.46 CLU_MXA_CLU0.PWM1	255
5.5.2.47 CLU_MXA_CLU0.Timer0ch0ovfl	256
5.5.2.48 CLU_MXA_CLU0.Timer1ch0ovfl	256
5.5.2.49 CLU_MXA_CLU1.CLU0OUT	256
5.5.2.50 CLU_MXA_CLU1.CLU1OUT	256
5.5.2.51 CLU_MXA_CLU1.CLU2OUT	256
5.5.2.52 CLU_MXA_CLU1.CLU3OUT	256
5.5.2.53 CLU_MXA_CLU1.P0_0	257
5.5.2.54 CLU_MXA_CLU1.P0_11	257
5.5.2.55 CLU_MXA_CLU1.P0_14	257
5.5.2.56 CLU_MXA_CLU1.P0_15	257
5.5.2.57 CLU_MXA_CLU1.P0_2	257
5.5.2.58 CLU_MXA_CLU1.P0_5	257
5.5.2.59 CLU_MXA_CLU1.P0_6	258
5.5.2.60 CLU_MXA_CLU1.P2_12	258
5.5.2.61 CLU_MXA_CLU1.PWM2	258
5.5.2.62 CLU_MXA_CLU1.PWM3	258
5.5.2.63 CLU_MXA_CLU1.Timer0ch1ovfl	258
5.5.2.64 CLU_MXA_CLU1.Timer1ch1ovfl	258
5.5.2.65 CLU_MXA_CLU2.CLU0OUT	259
5.5.2.66 CLU_MXA_CLU2.CLU1OUT	259
5.5.2.67 CLU_MXA_CLU2.CLU2OUT	259

5.5.2.68 CLU_MXA_CLU2_CLU3OUT	259
5.5.2.69 CLU_MXA_CLU2_P1_10	259
5.5.2.70 CLU_MXA_CLU2_P1_13	259
5.5.2.71 CLU_MXA_CLU2_P1_15	260
5.5.2.72 CLU_MXA_CLU2_P2_1	260
5.5.2.73 CLU_MXA_CLU2_P2_14	260
5.5.2.74 CLU_MXA_CLU2_P2_4	260
5.5.2.75 CLU_MXA_CLU2_P2_6	260
5.5.2.76 CLU_MXA_CLU2_P2_8	260
5.5.2.77 CLU_MXA_CLU2_PWM0	261
5.5.2.78 CLU_MXA_CLU2_PWM2	261
5.5.2.79 CLU_MXA_CLU2_Timer2ch0ovfl	261
5.5.2.80 CLU_MXA_CLU2_Timer3ch0ovfl	261
5.5.2.81 CLU_MXA_CLU3_CLU0OUT	261
5.5.2.82 CLU_MXA_CLU3_CLU1OUT	261
5.5.2.83 CLU_MXA_CLU3_CLU2OUT	262
5.5.2.84 CLU_MXA_CLU3_CLU3OUT	262
5.5.2.85 CLU_MXA_CLU3_P1_15	262
5.5.2.86 CLU_MXA_CLU3_P2_0	262
5.5.2.87 CLU_MXA_CLU3_P2_1	262
5.5.2.88 CLU_MXA_CLU3_P2_13	262
5.5.2.89 CLU_MXA_CLU3_P2_3	263
5.5.2.90 CLU_MXA_CLU3_P2_6	263
5.5.2.91 CLU_MXA_CLU3_P2_8	263
5.5.2.92 CLU_MXA_CLU3_P3_9	263
5.5.2.93 CLU_MXA_CLU3_PWM1	263
5.5.2.94 CLU_MXA_CLU3_PWM3	263
5.5.2.95 CLU_MXA_CLU3_Timer2ch1ovfl	264
5.5.2.96 CLU_MXA_CLU3_Timer3ch1ovfl	264
5.5.2.97 CLU_MXB_CLU0_ADBUSY0	264
5.5.2.98 CLU_MXB_CLU0_ADBUSY1	264
5.5.2.99 CLU_MXB_CLU0_CLU0OUT	264
5.5.2.100 CLU_MXB_CLU0_CLU1OUT	264
5.5.2.101 CLU_MXB_CLU0_CLU2OUT	265
5.5.2.102 CLU_MXB_CLU0_CLU3OUT	265
5.5.2.103 CLU_MXB_CLU0_P0_11	265
5.5.2.104 CLU_MXB_CLU0_P0_13	265
5.5.2.105 CLU_MXB_CLU0_P0_15	265
5.5.2.106 CLU_MXB_CLU0_P0_2	265
5.5.2.107 CLU_MXB_CLU0_P0_4	266
5.5.2.108 CLU_MXB_CLU0_P0_6	266
5.5.2.109 CLU_MXB_CLU0_P2_11	266

5.5.2.110 CLU_MXB_CLU0_P2.7	266
5.5.2.111 CLU_MXB_CLU0_PWM3	266
5.5.2.112 CLU_MXB_CLU0_SPIMOSI	266
5.5.2.113 CLU_MXB_CLU1_ADBUSY0	267
5.5.2.114 CLU_MXB_CLU1_ADBUSY1	267
5.5.2.115 CLU_MXB_CLU1_CLU0OUT	267
5.5.2.116 CLU_MXB_CLU1_CLU1OUT	267
5.5.2.117 CLU_MXB_CLU1_CLU2OUT	267
5.5.2.118 CLU_MXB_CLU1_CLU3OUT	267
5.5.2.119 CLU_MXB_CLU1_P0.12	268
5.5.2.120 CLU_MXB_CLU1_P0.13	268
5.5.2.121 CLU_MXB_CLU1_P0.3	268
5.5.2.122 CLU_MXB_CLU1_P0.4	268
5.5.2.123 CLU_MXB_CLU1_P0.7	268
5.5.2.124 CLU_MXB_CLU1_P1.0	268
5.5.2.125 CLU_MXB_CLU1_P2.11	269
5.5.2.126 CLU_MXB_CLU1_P2.7	269
5.5.2.127 CLU_MXB_CLU1_PWM1	269
5.5.2.128 CLU_MXB_CLU1_SPIMISO	269
5.5.2.129 CLU_MXB_CLU2_CLU0OUT	269
5.5.2.130 CLU_MXB_CLU2_CLU1OUT	269
5.5.2.131 CLU_MXB_CLU2_CLU2OUT	270
5.5.2.132 CLU_MXB_CLU2_CLU3OUT	270
5.5.2.133 CLU_MXB_CLU2_I2CSDA	270
5.5.2.134 CLU_MXB_CLU2_P1.11	270
5.5.2.135 CLU_MXB_CLU2_P1.14	270
5.5.2.136 CLU_MXB_CLU2_P2.0	270
5.5.2.137 CLU_MXB_CLU2_P2.13	271
5.5.2.138 CLU_MXB_CLU2_P2.15	271
5.5.2.139 CLU_MXB_CLU2_P2.3	271
5.5.2.140 CLU_MXB_CLU2_P2.5	271
5.5.2.141 CLU_MXB_CLU2_P3.9	271
5.5.2.142 CLU_MXB_CLU2_PWM1	271
5.5.2.143 CLU_MXB_CLU2_PWM3	272
5.5.2.144 CLU_MXB_CLU2_SPICLK	272
5.5.2.145 CLU_MXB_CLU3_CLU0OUT	272
5.5.2.146 CLU_MXB_CLU3_CLU1OUT	272
5.5.2.147 CLU_MXB_CLU3_CLU2OUT	272
5.5.2.148 CLU_MXB_CLU3_CLU3OUT	272
5.5.2.149 CLU_MXB_CLU3_I2CSCL	273
5.5.2.150 CLU_MXB_CLU3_P1.10	273
5.5.2.151 CLU_MXB_CLU3_P1.11	273

5.5.2.152 CLU_MXB_CLU3_P1_13	273
5.5.2.153 CLU_MXB_CLU3_P1_14	273
5.5.2.154 CLU_MXB_CLU3_P2_14	273
5.5.2.155 CLU_MXB_CLU3_P2_15	274
5.5.2.156 CLU_MXB_CLU3_P2_4	274
5.5.2.157 CLU_MXB_CLU3_P2_5	274
5.5.2.158 CLU_MXB_CLU3_PWM0	274
5.5.2.159 CLU_MXB_CLU3_PWM2	274
5.5.2.160 CLU_MXB_CLU3_SPICSN	274
5.6 lks32mc07x_clu.h	275
5.7 include/lks32mc07x_cmp.h 文件参考	277
5.7.1 详细描述	280
5.7.2 宏定义说明	280
5.7.2.1 CMP	280
5.7.2.2 CMP0_SELN_HALL0_MID	280
5.7.2.3 CMP0_SELP_IP0	281
5.7.2.4 CMP0_SELP_IP1	281
5.7.2.5 CMP0_SELP_IP2	281
5.7.2.6 CMP0_SELP_IP3	281
5.7.2.7 CMP0_SELP_IP4	281
5.7.2.8 CMP0_SELP_OPA0_IP	281
5.7.2.9 CMP0_SELP_OPA0_OUT	282
5.7.2.10 CMP0_SELP_OPA1_OUT	282
5.7.2.11 CMP1_SELN_HALL1_MID	282
5.7.2.12 CMP1_SELP_DAC0	282
5.7.2.13 CMP1_SELP_IP0	282
5.7.2.14 CMP1_SELP_IP1	282
5.7.2.15 CMP1_SELP_IP2	283
5.7.2.16 CMP1_SELP_IP3	283
5.7.2.17 CMP1_SELP_OPA2_OUT	283
5.7.2.18 CMP1_SELP_OPA3_IP	283
5.7.2.19 CMP1_SELP_OPA3_OUT	283
5.7.2.20 CMP2_SELN_DAC1	283
5.7.2.21 CMP2_SELP_IP0	284
5.7.2.22 CMP2_SELP_IP1	284
5.7.2.23 CMP2_SELP_OPA2_OUT	284
5.7.2.24 CMP2_SELP_OPA3_OUT	284
5.7.2.25 CMP_CHN_0	284
5.7.2.26 CMP_CHN_1	284
5.7.2.27 CMP_CHN_2	285
5.7.2.28 CMP_HIGH_LEVEL	285
5.7.2.29 CMP_HYS_0mV	285

5.7.2.30 CMP_HYS_20mV	285
5.7.2.31 CMP_LOW_LEVEL	285
5.7.2.32 CMP_SELN_DAC0	285
5.7.2.33 CMP_SELN_IN	286
5.7.2.34 CMP_SELN_REF	286
5.7.2.35 IRQ_EDGE_TRIG_MODE	286
5.7.2.36 IRQ_LEVEL_TRIG_MODE	286
5.7.2.37 W_PWM_HIGH_LEVEL	286
5.7.2.38 W_PWM_LOW_LEVEL	286
5.7.3 函数说明	286
5.7.3.1 CMP_ClearIRQFlag()	286
5.7.3.2 CMP_Cmd()	287
5.7.3.3 CMP_GetFilterValue()	287
5.7.3.4 CMP_GetIRQFlag()	288
5.7.3.5 CMP_GetValue()	289
5.7.3.6 CMP_Init()	290
5.7.3.7 CMP_StructInit()	291
5.8 lks32mc07x_cmp.h	292
5.9 include/lks32mc07x_crc.h 文件参考	294
5.9.1 详细描述	295
5.9.2 宏定义说明	296
5.9.2.1 CRC_MODEL_CRC_16_CCITT	296
5.9.2.2 CRC_MODEL_CRC_16_CCITT_FALSE	296
5.9.2.3 CRC_MODEL_CRC_16_DNP	296
5.9.2.4 CRC_MODEL_CRC_16_IBMx1	296
5.9.2.5 CRC_MODEL_CRC_16_MAXIM	296
5.9.2.6 CRC_MODEL_CRC_16_MODBU	296
5.9.2.7 CRC_MODEL_CRC_16_USBx1	297
5.9.2.8 CRC_MODEL_CRC_16_X25	297
5.9.2.9 CRC_MODEL_CRC_16_XMODEM	297
5.9.2.10 CRC_MODEL_CRC_32	297
5.9.2.11 CRC_MODEL_CRC_32_MPEG2	297
5.9.2.12 CRC_MODEL_CRC_8	297
5.9.2.13 CRC_MODEL_CRC_8_ITU	298
5.9.2.14 CRC_MODEL_CRC_8_MAXIMx	298
5.9.2.15 CRC_MODEL_CRC_8_ROHC	298
5.9.2.16 CRC_WIDTH_16	298
5.9.2.17 CRC_WIDTH_32	298
5.9.2.18 CRC_WIDTH_7	298
5.9.2.19 CRC_WIDTH_8	299
5.10 lks32mc07x_crc.h	299
5.11 include/lks32mc07x_dac.h 文件参考	300

5.11.1 详细描述	301
5.11.2 宏定义说明	301
5.11.2.1 DAC0_UTIMER0_TRG_EN	301
5.11.2.2 DAC0_UTIMER1_TRG_EN	302
5.11.2.3 DAC0_UTIMER2_TRG_EN	302
5.11.2.4 DAC0_UTIMER3_TRG_EN	302
5.11.2.5 DAC1_UTIMER0_TRG_EN	302
5.11.2.6 DAC1_UTIMER1_TRG_EN	302
5.11.2.7 DAC1_UTIMER2_TRG_EN	302
5.11.2.8 DAC1_UTIMER3_TRG_EN	303
5.11.2.9 DAC_Channel0	303
5.11.2.10 DAC_Channel1	303
5.11.2.11 DAC_RANGE_1V2	303
5.11.2.12 DAC_RANGE_4V85	303
5.11.3 函数说明	303
5.11.3.1 DAC_Cmd()	303
5.11.3.2 DAC_Init()	304
5.11.3.3 DAC_Output()	307
5.11.3.4 DAC_OutputValue()	307
5.11.3.5 DAC_OutputVoltage()	308
5.11.3.6 DAC_StructInit()	309
5.12 lks32mc07x_dac.h	310
5.13 include/lks32mc07x_dma.h 文件参考	310
5.13.1 详细描述	312
5.13.2 宏定义说明	313
5.13.2.1 DMA_BYTE_TRANS	313
5.13.2.2 DMA_CHN0	313
5.13.2.3 DMA_CHN1	313
5.13.2.4 DMA_CHN2	313
5.13.2.5 DMA_CHN3	313
5.13.2.6 DMA_HALFWORD_TRANS	313
5.13.2.7 DMA_IE_CH0	314
5.13.2.8 DMA_IE_CH1	314
5.13.2.9 DMA_IE_CH2	314
5.13.2.10 DMA_IE_CH3	314
5.13.2.11 DMA_IF_CH0	314
5.13.2.12 DMA_IF_CH1	314
5.13.2.13 DMA_IF_CH2	314
5.13.2.14 DMA_IF_CH3	315
5.13.2.15 DMA_REQ_ADC0	315
5.13.2.16 DMA_REQ_ADC1	315
5.13.2.17 DMA_REQ_CMP	315

5.13.2.18 DMA_REQ_GPIO	315
5.13.2.19 DMA_REQ_HALL	315
5.13.2.20 DMA_REQ_I2C0_RX	315
5.13.2.21 DMA_REQ_I2C0_TX	315
5.13.2.22 DMA_REQ_MCPWM0	316
5.13.2.23 DMA_REQ_MCPWM1	316
5.13.2.24 DMA_REQ_SIF	316
5.13.2.25 DMA_REQ_SPI_RX	316
5.13.2.26 DMA_REQ_SPI_TX	316
5.13.2.27 DMA_REQ_SW	316
5.13.2.28 DMA_REQ_TIMER0	316
5.13.2.29 DMA_REQ_TIMER1	317
5.13.2.30 DMA_REQ_TIMER2	317
5.13.2.31 DMA_REQ_TIMER3	317
5.13.2.32 DMA_REQ_TIMER4	317
5.13.2.33 DMA_REQ_UART0_RX	317
5.13.2.34 DMA_REQ_UART0_TX	317
5.13.2.35 DMA_REQ_UART1_RX	317
5.13.2.36 DMA_REQ_UART1_TX	317
5.13.2.37 DMA_TCIE	318
5.13.2.38 DMA_TEIE	318
5.13.2.39 DMA_TOP	318
5.13.2.40 DMA_WORD_TRANS	318
5.13.2.41 MEMORY2PERI	318
5.13.2.42 PERI2MEMORY	318
5.13.3 函数说明	318
5.13.3.1 DMA_CHx_EN()	318
5.13.3.2 DMA_ClearIRQFlag()	319
5.13.3.3 DMA_Disable()	320
5.13.3.4 DMA_Enable()	321
5.13.3.5 DMA_GetFlag()	322
5.13.3.6 DMA_GetIRQFlag()	323
5.13.3.7 DMA_Init()	324
5.13.3.8 DMA_SetAddr()	326
5.13.3.9 DMA_SetLength()	327
5.13.3.10 DMA_SetRequest()	328
5.13.3.11 DMA_StructInit()	329
5.13.3.12 DMA_Trigger()	330
5.14 lks32mc07x_dma.h	331
5.15 include/lks32mc07x_dsp.h 文件参考	333
5.15.1 详细描述	334
5.15.2 宏定义说明	334

5.15.2.1 DSP0_arctan	334
5.15.2.2 DSP0_cos	335
5.15.2.3 DSP0_div	335
5.15.2.4 DSP0_mod	335
5.15.2.5 DSP0_rms	335
5.15.2.6 DSP0_sin	335
5.15.2.7 DSP0_sqrt	335
5.15.2.8 DSP_ARCTAN_MODE	335
5.15.2.9 DSP_CALC_ARCTAN	336
5.15.2.10 DSP_CALC_COS	336
5.15.2.11 DSP_CALC_DIV	336
5.15.2.12 DSP_CALC_MOD	337
5.15.2.13 DSP_CALC_RMS	337
5.15.2.14 DSP_CALC_SIN	337
5.15.2.15 DSP_CALC_SQRT	338
5.15.2.16 DSP_REG	338
5.15.2.17 DSP_SIN_COS_MODE	338
5.15.3 函数说明	338
5.15.3.1 DSP_arctan()	338
5.15.3.2 DSP_ArctanMode()	339
5.15.3.3 DSP_cos()	339
5.15.3.4 DSP_div()	340
5.15.3.5 DSP_Init()	341
5.15.3.6 DSP_mod()	342
5.15.3.7 DSP_rms()	343
5.15.3.8 DSP_sin()	344
5.15.3.9 DSP_SinCosMode()	345
5.15.3.10 DSP_sqrt()	345
5.16 lks32mc07x.dsp.h	346
5.17 include/lks32mc07x_exti.h 文件参考	348
5.17.1 详细描述	349
5.17.2 宏定义说明	350
5.17.2.1 EXTI	350
5.17.3 枚举类型说明	350
5.17.3.1 EXTI_Mode_TypeDef	350
5.17.4 函数说明	350
5.17.4.1 EXTI_ClearIRQFlag()	350
5.17.4.2 EXTI_GetIRQFlag()	351
5.17.4.3 EXTI_ReadIFFlag()	352
5.17.4.4 EXTI_ResetIF()	352
5.17.4.5 EXTI_Trigger_Config()	353
5.18 lks32mc07x_exti.h	354

5.19 include/lks32mc07x_flash.h 文件参考	354
5.19.1 详细描述	355
5.19.2 宏定义说明	356
5.19.2.1 Flash_MAIN	356
5.19.2.2 Flash_NVR	356
5.19.3 函数说明	356
5.19.3.1 EraseSector()	356
5.19.3.2 ProgramPage()	357
5.19.3.3 Read_More_Flash()	358
5.19.4 变量说明	360
5.19.4.1 erase_flag	360
5.19.4.2 progm_flag	360
5.20 lks32mc07x_flash.h	360
5.21 include/lks32mc07x_gpio.h 文件参考	361
5.21.1 详细描述	363
5.21.2 宏定义说明	364
5.21.2.1 AF0_GPIO	364
5.21.2.2 AF10_CAN	364
5.21.2.3 AF11_SIF	364
5.21.2.4 AF12_CL	364
5.21.2.5 AF1_SYSAF	364
5.21.2.6 AF2_HALL	364
5.21.2.7 AF3_MCPWM	365
5.21.2.8 AF4_UART	365
5.21.2.9 AF5_SPI	365
5.21.2.10 AF6_I2C	365
5.21.2.11 AF7_TIMER01	365
5.21.2.12 AF8_TIMER23	365
5.21.2.13 AF9_DEBUG	366
5.21.2.14 GPIO0	366
5.21.2.15 GPIO1	366
5.21.2.16 GPIO2	366
5.21.2.17 GPIO3	366
5.21.2.18 GPIO_Pin_0	366
5.21.2.19 GPIO_Pin_1	367
5.21.2.20 GPIO_Pin_10	367
5.21.2.21 GPIO_Pin_11	367
5.21.2.22 GPIO_Pin_12	367
5.21.2.23 GPIO_Pin_13	367
5.21.2.24 GPIO_Pin_14	367
5.21.2.25 GPIO_Pin_15	368
5.21.2.26 GPIO_Pin_2	368

5.21.2.27 GPIO_Pin_3	368
5.21.2.28 GPIO_Pin_4	368
5.21.2.29 GPIO_Pin_5	368
5.21.2.30 GPIO_Pin_6	368
5.21.2.31 GPIO_Pin_7	369
5.21.2.32 GPIO_Pin_8	369
5.21.2.33 GPIO_Pin_9	369
5.21.2.34 GPIO_Pin_ALL	369
5.21.2.35 GPIO_Pin_NONE	369
5.21.2.36 GPIO_PinSource_0	369
5.21.2.37 GPIO_PinSource_1	370
5.21.2.38 GPIO_PinSource_10	370
5.21.2.39 GPIO_PinSource_11	370
5.21.2.40 GPIO_PinSource_12	370
5.21.2.41 GPIO_PinSource_13	370
5.21.2.42 GPIO_PinSource_14	370
5.21.2.43 GPIO_PinSource_15	371
5.21.2.44 GPIO_PinSource_2	371
5.21.2.45 GPIO_PinSource_3	371
5.21.2.46 GPIO_PinSource_4	371
5.21.2.47 GPIO_PinSource_5	371
5.21.2.48 GPIO_PinSource_6	371
5.21.2.49 GPIO_PinSource_7	372
5.21.2.50 GPIO_PinSource_8	372
5.21.2.51 GPIO_PinSource_9	372
5.21.3 枚举类型说明	372
5.21.3.1 BitAction	372
5.21.3.2 GPIOMode_TypeDef	372
5.21.3.3 GPIOPuPd_TypeDef	373
5.21.4 函数说明	373
5.21.4.1 GPIO_Init()	373
5.21.4.2 GPIO_PinAFConfig()	375
5.21.4.3 GPIO_ReadInputData()	376
5.21.4.4 GPIO_ReadInputDataBit()	377
5.21.4.5 GPIO_ReadOutputData()	378
5.21.4.6 GPIO_ReadOutputDataBit()	379
5.21.4.7 GPIO_ResetBits()	380
5.21.4.8 GPIO_SetBits()	381
5.21.4.9 GPIO_StructInit()	382
5.21.4.10 GPIO_Write()	383
5.21.4.11 GPIO_WriteBit()	384
5.22 lks32mc07x_gpio.h	385

5.23 include/lks32mc07x_hall.h 文件参考	386
5.23.1 详细描述	388
5.23.2 宏定义说明	389
5.23.2.1 HALL	389
5.23.2.2 HALL_CAPTURE_EVENT	389
5.23.2.3 HALL_CLK_DIV1	389
5.23.2.4 HALL_CLK_DIV2	389
5.23.2.5 HALL_CLK_DIV4	389
5.23.2.6 HALL_CLK_DIV8	389
5.23.2.7 HALL_OVERFLOW_EVENT	390
5.23.3 函数说明	390
5.23.3.1 HALL_Clear_IRQ()	390
5.23.3.2 HALL_Cmd()	391
5.23.3.3 HALL_GetCaptureValue()	392
5.23.3.4 HALL_GetCount()	393
5.23.3.5 HALL_GetFilterValue()	394
5.23.3.6 HALL_GetFlag()	395
5.23.3.7 HALL_GetIRQFlag()	395
5.23.3.8 HALL_Init()	396
5.23.3.9 HALL_IsCaptureEvent()	397
5.23.3.10 HALL_IsOverFlowEvent()	398
5.23.3.11 HALL_SoftTrigger()	399
5.23.3.12 HALL_StructInit()	400
5.24 lks32mc07x_hall.h	401
5.25 include/lks32mc07x_i2c.h 文件参考	402
5.25.1 详细描述	404
5.25.2 宏定义说明	404
5.25.2.1 I2C_BUS_STATE_ADDR_DATA	404
5.25.2.2 I2C_BUS_STATE_Done	404
5.25.2.3 I2C_BUS_STATE_LOST_ARB	404
5.25.2.4 I2C_BUS_STATE_RX_ACK	404
5.25.2.5 I2C_BUS_STATE_STOP_EVT	405
5.25.2.6 I2C_BUS_STATE_STT_ERR	405
5.25.2.7 I2C_IF_ADDR_CMP	405
5.25.2.8 I2C_IF_BUS_ERR	405
5.25.2.9 I2C_IF_NACK	405
5.25.2.10 I2C_IF_STOP	405
5.25.2.11 I2C_IF_TC	406
5.26 lks32mc07x_i2c.h	406
5.27 include/lks32mc07x_lib.h 文件参考	407
5.27.1 详细描述	408
5.27.2 宏定义说明	408

5.27.2.1 REG_IF_WRITE_BIT	408
5.27.2.2 REG_READ	408
5.27.2.3 REG_READ_BIT	408
5.27.2.4 REG_READ_BIT_POS_MASK	409
5.27.2.5 REG_RESET	409
5.27.2.6 REG_SET	409
5.27.2.7 REG_WRITE	409
5.27.2.8 REG_WRITE_BIT	409
5.27.2.9 REG_WRITE_BIT_POS_MASK	409
5.28 lks32mc07x.lib.h	410
5.29 include/lks32mc07x.mcpwm.h 文件参考	410
5.29.1 详细描述	415
5.29.2 宏定义说明	416
5.29.2.1 MCPWM0	416
5.29.2.2 MCPWM0_ALL_AUPDAT	416
5.29.2.3 MCPWM0_CENTRAL_PWM_MODE	416
5.29.2.4 MCPWM0_CNT0_AUPDAT	416
5.29.2.5 MCPWM0_CNT1_AUPDAT	416
5.29.2.6 MCPWM0_EDGE_PWM_MODE	416
5.29.2.7 MCPWM0_FAIL0_IRQ_IF	417
5.29.2.8 MCPWM0_FAIL1_IRQ_IF	417
5.29.2.9 MCPWM0_FAIL2_IRQ_IF	417
5.29.2.10 MCPWM0_FAIL3_IRQ_IF	417
5.29.2.11 MCPWM0_FAIL_SEL_CMP	417
5.29.2.12 MCPWM0_FAIL_SEL_IO	417
5.29.2.13 MCPWM0_HIGH_LEVEL	418
5.29.2.14 MCPWM0_HIGH_LEVEL_ACTIVE	418
5.29.2.15 MCPWM0_LOW_LEVEL	418
5.29.2.16 MCPWM0_LOW_LEVEL_ACTIVE	418
5.29.2.17 MCPWM0_TH00_AUPDATE	418
5.29.2.18 MCPWM0_TH01_AUPDATE	418
5.29.2.19 MCPWM0_TH0_AUPDATE	419
5.29.2.20 MCPWM0_TH10_AUPDATE	419
5.29.2.21 MCPWM0_TH11_AUPDATE	419
5.29.2.22 MCPWM0_TH1_AUPDATE	419
5.29.2.23 MCPWM0_TH20_AUPDATE	419
5.29.2.24 MCPWM0_TH21_AUPDATE	419
5.29.2.25 MCPWM0_TH30_AUPDATE	420
5.29.2.26 MCPWM0_TH31_AUPDATE	420
5.29.2.27 MCPWM0_TH40_AUPDATE	420
5.29.2.28 MCPWM0_TH41_AUPDATE	420
5.29.2.29 MCPWM0_TH50_AUPDATE	420

5.29.2.30 MCPWM0_TH51_AUPDATE	420
5.29.2.31 MCPWM0_TMR0_AUPDATE	421
5.29.2.32 MCPWM0_TMR1_AUPDATE	421
5.29.2.33 MCPWM0_TMR2_AUPDATE	421
5.29.2.34 MCPWM0_TMR3_AUPDATE	421
5.29.2.35 MCPWM0_ZCS0_IRQ_IF	421
5.29.2.36 MCPWM0_ZCS1_IRQ_IF	421
5.29.2.37 MCPWM0CNT0_T0_IRQ_IF	422
5.29.2.38 MCPWM0CNT0_T1_IRQ_IF	422
5.29.2.39 MCPWM0CNT0_TH00_IRQ_IF	422
5.29.2.40 MCPWM0CNT0_TH01_IRQ_IF	422
5.29.2.41 MCPWM0CNT0_TH10_IRQ_IF	422
5.29.2.42 MCPWM0CNT0_TH11_IRQ_IF	422
5.29.2.43 MCPWM0CNT0_TH20_IRQ_IF	423
5.29.2.44 MCPWM0CNT0_TH21_IRQ_IF	423
5.29.2.45 MCPWM0CNT0_TMR0_IRQ_IF	423
5.29.2.46 MCPWM0CNT0_TMR1_IRQ_IF	423
5.29.2.47 MCPWM0CNT0_TMR2_IRQ_IF	423
5.29.2.48 MCPWM0CNT0_TMR3_IRQ_IF	423
5.29.2.49 MCPWM0CNT0_UP_IRQ_IF	424
5.29.2.50 MCPWM0CNT1_T0_IRQ_IF	424
5.29.2.51 MCPWM0CNT1_T1_IRQ_IF	424
5.29.2.52 MCPWM0CNT1_TH30_IRQ_IF	424
5.29.2.53 MCPWM0CNT1_TH31_IRQ_IF	424
5.29.2.54 MCPWM0CNT1_TH40_IRQ_IF	424
5.29.2.55 MCPWM0CNT1_TH41_IRQ_IF	425
5.29.2.56 MCPWM0CNT1_TH50_IRQ_IF	425
5.29.2.57 MCPWM0CNT1_TH51_IRQ_IF	425
5.29.2.58 MCPWM0CNT1_TMR2_IRQ_IF	425
5.29.2.59 MCPWM0CNT1_TMR3_IRQ_IF	425
5.29.2.60 MCPWM0CNT1_UP_IRQ_IF	425
5.29.2.61 MCPWM{EIF_FAIL0	426
5.29.2.62 MCPWM{EIF_FAIL1	426
5.29.2.63 MCPWM{EIF_FAIL2	426
5.29.2.64 MCPWM{EIF_FAIL3	426
5.29.2.65 MCPWM{EIF_ZCS0	426
5.29.2.66 MCPWM{EIF_ZCS1	426
5.29.2.67 MCPWM_IF0_T0_IF	427
5.29.2.68 MCPWM_IF0_T1_IF	427
5.29.2.69 MCPWM_IF0_TH00_IF	427
5.29.2.70 MCPWM_IF0_TH01_IF	427
5.29.2.71 MCPWM_IF0_TH10_IF	427

5.29.2.72 MCPWM_IF0_TH11_IF	427
5.29.2.73 MCPWM_IF0_TH20_IF	428
5.29.2.74 MCPWM_IF0_TH21_IF	428
5.29.2.75 MCPWM_IF0_TMR0_IF	428
5.29.2.76 MCPWM_IF0_TMR1_IF	428
5.29.2.77 MCPWM_IF0_TMR2_IF	428
5.29.2.78 MCPWM_IF0_TMR3_IF	428
5.29.2.79 MCPWM_IF0_UP_IF	429
5.29.2.80 MCPWM_IF1_T0_IF	429
5.29.2.81 MCPWM_IF1_T1_IF	429
5.29.2.82 MCPWM_IF1_TH30_IF	429
5.29.2.83 MCPWM_IF1_TH31_IF	429
5.29.2.84 MCPWM_IF1_TH40_IF	429
5.29.2.85 MCPWM_IF1_TH41_IF	430
5.29.2.86 MCPWM_IF1_TH50_IF	430
5.29.2.87 MCPWM_IF1_TH51_IF	430
5.29.2.88 MCPWM_IF1_TMR0_IF	430
5.29.2.89 MCPWM_IF1_TMR1_IF	430
5.29.2.90 MCPWM_IF1_TMR2_IF	430
5.29.2.91 MCPWM_IF1_TMR3_IF	431
5.29.2.92 MCPWM_IF1_UP_IF	431
5.29.3 枚举类型说明	431
5.29.3.1 MCPWM0CNTx	431
5.29.4 函数说明	431
5.29.4.1 MCPWM_CH012_IESet()	431
5.29.4.2 MCPWM_CH012_Init()	431
5.29.4.3 MCPWM_CH345_IESet()	432
5.29.4.4 MCPWM_CH345_Init()	432
5.29.4.5 MCPWM_ClearIRQFlag()	432
5.29.4.6 MCPWM_Clk_Enable()	435
5.29.4.7 MCPWM_CNTx_Enable()	435
5.29.4.8 MCPWM_DIV_Set()	436
5.29.4.9 MCPWM_GetIRQFlag()	436
5.29.4.10 MCPWM_Init()	439
5.29.4.11 MCPWM_ModuleClockCmd()	441
5.29.4.12 MCPWM_StructInit()	441
5.29.4.13 PWMOutputs()	442
5.30 lks32mc07x_mcpwm.h	443
5.31 include/lks32mc07x_nvr.h 文件参考	448
5.31.1 详细描述	448
5.31.2 函数说明	449
5.31.2.1 Read_NVR()	449

5.31.2.2 Read_Trim()	449
5.32 lks32mc07x_nvr.h	450
5.33 include/lks32mc07x_opa.h 文件参考	450
5.33.1 详细描述	451
5.33.2 宏定义说明	452
5.33.2.1 PGA_GAIN_16	452
5.33.2.2 PGA_GAIN_32	452
5.33.2.3 PGA_GAIN_4	452
5.33.2.4 PGA_GAIN_8	452
5.33.2.5 PGA_IT_1	452
5.33.2.6 PGA_IT_1P2	453
5.33.2.7 PGA_IT_1P5	453
5.33.2.8 PGA_IT_2	453
5.33.2.9 PGA_IT_DISABLE	453
5.33.3 枚举类型说明	453
5.33.3.1 enumOPA	453
5.33.4 函数说明	454
5.33.4.1 OPA_Init()	454
5.33.4.2 OPA_OUT()	455
5.33.4.3 OPA_StructInit()	455
5.34 lks32mc07x_opa.h	456
5.35 include/lks32mc07x_qep.h 文件参考	457
5.35.1 详细描述	459
5.35.2 宏定义说明	459
5.35.2.1 QEP0	459
5.35.2.2 QEP1	460
5.35.2.3 QEP_CLK_DIV1	460
5.35.2.4 QEP_CLK_DIV128	460
5.35.2.5 QEP_CLK_DIV16	460
5.35.2.6 QEP_CLK_DIV2	460
5.35.2.7 QEP_CLK_DIV32	460
5.35.2.8 QEP_CLK_DIV4	461
5.35.2.9 QEP_CLK_DIV64	461
5.35.2.10 QEP_CLK_DIV8	461
5.35.2.11 QEP_IF_OVERFLOW	461
5.35.2.12 QEP_IF_UNDERFLOW	461
5.35.2.13 QEP_IRQEna_None	461
5.35.2.14 QEP_IRQEna_OverFlow	462
5.35.2.15 QEP_IRQEna_UnderFlow	462
5.35.2.16 QEP_Mode_CCWCW_UpDown	462
5.35.2.17 QEP_Mode_CCWSIGN_UpDown	462
5.35.2.18 QEP_Mode_T1	462

5.35.2.19 QEP_Mode_T1_T2	462
5.35.3 函数说明	462
5.35.3.1 QEP_ClearIRQFlag()	462
5.35.3.2 QEP_GetFlag()	463
5.35.3.3 QEP_GetIRQFlag()	464
5.35.3.4 TIM_QEP_GetCount()	465
5.35.3.5 TIM_QEPCmd()	466
5.35.3.6 TIM_QEPInit()	467
5.35.3.7 TIM_QEPStructInit()	469
5.36 lks32mc07x_qep.h	469
5.37 include/lks32mc07x_sif.h 文件参考	470
5.37.1 详细描述	472
5.37.2 宏定义说明	472
5.37.2.1 SIF0	472
5.37.2.2 SIF_DMA_Enable	472
5.37.2.3 SIF_IE_Enable	472
5.37.2.4 SPI_IF_CLEAR	473
5.37.3 函数说明	473
5.37.3.1 SIF_Clear_Irq()	473
5.37.3.2 SIF_DMA_IsEnable()	474
5.37.3.3 SIF_Init()	475
5.37.3.4 SIF_Irq_IsEnable()	476
5.37.3.5 SIF_Senddata()	477
5.37.3.6 SIF_StructInit()	478
5.37.3.7 SIF_TH_set()	478
5.37.3.8 SIF_Timebase_set()	479
5.38 lks32mc07x_sif.h	480
5.39 include/lks32mc07x_spi.h 文件参考	481
5.39.1 宏定义说明	482
5.39.1.1 SPI0	482
5.39.1.2 SPI_DMA_DISABLE	482
5.39.1.3 SPI_DMA_ENABLE	483
5.39.1.4 SPI_FIRSTSEND_LSB	483
5.39.1.5 SPI_FIRSTSEND_MSB	483
5.39.1.6 SPI_IF_DataOver	483
5.39.1.7 SPI_IF_SSErr	483
5.39.1.8 SPI_IF_TranDone	483
5.39.1.9 SPI_IRQEna_DataOver	484
5.39.1.10 SPI_IRQEna_Enable	484
5.39.1.11 SPI_IRQEna_SSErr	484
5.39.1.12 SPI_IRQEna_TranDone	484
5.39.2 枚举类型说明	484

5.39.2.1 SPI_Duplex	484
5.39.2.2 SPI_Mode	485
5.39.3 函数说明	485
5.39.3.1 SPI_ClearIRQFlag()	485
5.39.3.2 SPI_GetIRQFlag()	486
5.39.3.3 SPI_Init()	487
5.39.3.4 SPI_ReadData()	488
5.39.3.5 SPI_SendData()	489
5.39.3.6 SPI_StructInit()	490
5.40 lks32mc07x_spi.h	491
5.41 include/lks32mc07x_sys.h 文件参考	492
5.41.1 详细描述	495
5.41.2 宏定义说明	496
5.41.2.1 AFE	496
5.41.2.2 CLK_SEL_HS	496
5.41.2.3 CLK_SEL_PLL	496
5.41.2.4 SECOND2IWDGCNT	496
5.41.2.5 SYS	496
5.41.2.6 SYS_AnalogModule_ADC	497
5.41.2.7 SYS_AnalogModule_BGP	497
5.41.2.8 SYS_AnalogModule_CMP0	497
5.41.2.9 SYS_AnalogModule_CMP1	497
5.41.2.10 SYS_AnalogModule_CMP2	497
5.41.2.11 SYS_AnalogModule_DAC0	497
5.41.2.12 SYS_AnalogModule_DAC1	497
5.41.2.13 SYS_AnalogModule_OPA0	497
5.41.2.14 SYS_AnalogModule_OPA1	498
5.41.2.15 SYS_AnalogModule_OPA2	498
5.41.2.16 SYS_AnalogModule_OPA3	498
5.41.2.17 SYS_AnalogModule_PLL	498
5.41.2.18 SYS_AnalogModule_RCH	498
5.41.2.19 SYS_AnalogModule_TMP	498
5.41.2.20 SYS_AnalogModule_XTAL	498
5.41.2.21 SYS_Clk_SPIDiv1	498
5.41.2.22 SYS_Clk_SPIDiv2	499
5.41.2.23 SYS_Clk_SPIDiv4	499
5.41.2.24 SYS_Clk_SPIDiv8	499
5.41.2.25 SYS_Clk_UARTDiv1	499
5.41.2.26 SYS_Clk_UARTDiv2	499
5.41.2.27 SYS_Clk_UARTDiv4	499
5.41.2.28 SYS_Clk_UARTDiv8	499
5.41.2.29 SYS_MCLK_12M_RC	499

5.41.2.30 SYS_MCLK_12M_XTAL	500
5.41.2.31 SYS_MCLK_24M_RC	500
5.41.2.32 SYS_MCLK_24M_XTAL	500
5.41.2.33 SYS_MCLK_32K_RC	500
5.41.2.34 SYS_MCLK_48M_RC	500
5.41.2.35 SYS_MCLK_48M_XTAL	500
5.41.2.36 SYS_MCLK_96M_RC	500
5.41.2.37 SYS_MCLK_96M_XTAL	500
5.41.2.38 SYS_MCU_POWER_3v3	501
5.41.2.39 SYS_MCU_POWER_5v0	501
5.41.2.40 SYS_Module_CAN0	501
5.41.2.41 SYS_Module_CL0	501
5.41.2.42 SYS_Module_CMP	501
5.41.2.43 SYS_Module_CRC0	501
5.41.2.44 SYS_Module_DSP0	501
5.41.2.45 SYS_Module_GPIO	501
5.41.2.46 SYS_Module_HALL0	502
5.41.2.47 SYS_Module_I2C0	502
5.41.2.48 SYS_Module_MCPWM0	502
5.41.2.49 SYS_Module_QEP0	502
5.41.2.50 SYS_Module_QEP1	502
5.41.2.51 SYS_Module_SIF0	502
5.41.2.52 SYS_Module_SPI0	502
5.41.2.53 SYS_Module_TIMER0	502
5.41.2.54 SYS_Module_TIMER1	503
5.41.2.55 SYS_Module_TIMER2	503
5.41.2.56 SYS_Module_TIMER3	503
5.41.2.57 SYS_Module_UART0	503
5.41.2.58 SYS_Module_UART1	503
5.41.2.59 SYS_PLLSRSEL_CRYSTAL	503
5.41.2.60 SYS_PLLSRSEL_RCH	503
5.41.2.61 SYS_POWER_2V5	503
5.41.2.62 SYS_POWER_3V25	504
5.41.2.63 SYS_POWER_3V5	504
5.41.2.64 SYS_POWER_3V75	504
5.41.2.65 SYS_POWER_4V0	504
5.41.3 函数说明	504
5.41.3.1 IWDG_DISABLE()	504
5.41.3.2 IWDG_ENABLE()	505
5.41.3.3 IWDG_Feed()	506
5.41.3.4 IWDG_Init()	507
5.41.3.5 IWDG_StrutInit()	508

5.41.3.6 SYS_AnalogModuleClockCmd()	509
5.41.3.7 SYS_ClearRst()	510
5.41.3.8 SYS_I2CClkDiv()	512
5.41.3.9 SYS_Init()	513
5.41.3.10 SYS_MclkChoice()	514
5.41.3.11 SYS_ModuleClockCmd()	516
5.41.3.12 SYS_ReadMcuClk()	518
5.41.3.13 SYS_SoftReset_all()	519
5.41.3.14 SYS_SoftResetModule()	520
5.41.3.15 SYS_SPIClkDiv()	522
5.41.3.16 SYS_StructInit()	523
5.41.3.17 SYS_UartClkDiv()	524
5.41.3.18 SYS_VolSelModule()	525
5.41.3.19 SYS_VolSelModuleEnableIRQ()	525
5.41.3.20 SYS_VolSelModuleIRQ()	525
5.42 lks32mc07x_sys.h	526
5.43 include/lks32mc07x_tim.h 文件参考	529
5.43.1 详细描述	532
5.43.2 宏定义说明	533
5.43.2.1 CAP_SRC_CHXOR	533
5.43.2.2 CAP_SRC_CLU0	533
5.43.2.3 CAP_SRC_CLU1	533
5.43.2.4 CAP_SRC_CLU2	533
5.43.2.5 CAP_SRC_CLU3	533
5.43.2.6 CAP_SRC_CMP0	533
5.43.2.7 CAP_SRC_CMP1	534
5.43.2.8 CAP_SRC_CMP2	534
5.43.2.9 CAP_SRC_TCH0	534
5.43.2.10 CAP_SRC_TCH1	534
5.43.2.11 TIMER0_EVT_CH0	534
5.43.2.12 TIMER0_EVT_CH1	534
5.43.2.13 TIMER1_EVT_CH0	535
5.43.2.14 TIMER1_EVT_CH1	535
5.43.2.15 TIMER2_EVT_CH0	535
5.43.2.16 TIMER2_EVT_CH1	535
5.43.2.17 TIMER3_EVT_CH0	535
5.43.2.18 TIMER3_EVT_CH1	535
5.43.2.19 TIMER_CH0_IF	536
5.43.2.20 TIMER_CH1_IF	536
5.43.2.21 TIMER_CLK_DIV1	536
5.43.2.22 TIMER_CLK_DIV128	536
5.43.2.23 TIMER_CLK_DIV16	536

5.43.2.24	TIMER_CLK_DIV2	536
5.43.2.25	TIMER_CLK_DIV32	537
5.43.2.26	TIMER_CLK_DIV4	537
5.43.2.27	TIMER_CLK_DIV64	537
5.43.2.28	TIMER_CLK_DIV8	537
5.43.2.29	TIMER_EVT_CLU0	537
5.43.2.30	TIMER_EVT_CLU1	537
5.43.2.31	TIMER_EVT_CLU2	538
5.43.2.32	TIMER_EVT_CLU3	538
5.43.2.33	TIMER_EVT_TADC0	538
5.43.2.34	TIMER_EVT_TADC1	538
5.43.2.35	TIMER_EVT_TADC2	538
5.43.2.36	TIMER_EVT_TADC3	538
5.43.2.37	TIMER_FAIL_IF	539
5.43.2.38	TIMER_IF_CH0	539
5.43.2.39	TIMER_IF_CH1	539
5.43.2.40	TIMER_IF_FAIL	539
5.43.2.41	TIMER_IF_ZERO	539
5.43.2.42	TIMER_OPMODE_CAP	539
5.43.2.43	TIMER_OPMODE_CMP	540
5.43.2.44	TIMER_POL_HIGH	540
5.43.2.45	TIMER_POL_LOW	540
5.43.2.46	TIMER_ZC_IF	540
5.43.2.47	UTIMER0	540
5.43.2.48	UTIMER1	540
5.43.2.49	UTIMER2	541
5.43.2.50	UTIMER3	541
5.43.3	枚举类型说明	541
5.43.3.1	Timer_IRQEnaDef	541
5.43.4	函数说明	541
5.43.4.1	TIM_ClearIRQFlag()	541
5.43.4.2	TIM_CMP0()	542
5.43.4.3	TIM_CMP1()	543
5.43.4.4	TIM_GetFlag()	544
5.43.4.5	TIM_GetIRQFlag()	545
5.43.4.6	TIM_Timer_GetCMPT0()	546
5.43.4.7	TIM_Timer_GetCMPT1()	547
5.43.4.8	TIM_Timer_GetCount()	548
5.43.4.9	TIM_TimerCmd()	549
5.43.4.10	TIM_TimerInit()	550
5.43.4.11	TIM_TimerStrutInit()	552
5.43.4.12	UTIMER_Disable()	553

5.43.4.13 UTIMER_Enable()	553
5.44 lks32mc07x_tim.h	554
5.45 include/lks32mc07x_tmp.h 文件参考	555
5.45.1 详细描述	556
5.45.2 函数说明	557
5.45.2.1 GetCurrentTemperature()	557
5.45.2.2 TempSensor_Init()	558
5.46 lks32mc07x_tmp.h	559
5.47 include/lks32mc07x_uart.h 文件参考	559
5.47.1 详细描述	562
5.47.2 宏定义说明	562
5.47.2.1 RX_DMA_RE	562
5.47.2.2 TX_BUF_DMA_RE	563
5.47.2.3 TX_DMA_RE	563
5.47.2.4 UART0	563
5.47.2.5 UART1	563
5.47.2.6 UART_FIRSTSEND_LSB	563
5.47.2.7 UART_FIRSTSEND_MSB	563
5.47.2.8 UART_IF_CheckError	564
5.47.2.9 UART_IF_IDLE	564
5.47.2.10 UART_IF_LBD	564
5.47.2.11 UART_IF_RcvOver	564
5.47.2.12 UART_IF_RX_OV	564
5.47.2.13 UART_IF_SendBufEmpty	564
5.47.2.14 UART_IF_SendOver	565
5.47.2.15 UART_IF_StopError	565
5.47.2.16 UART_IF_TX_OV	565
5.47.2.17 UART_IRQEna_CheckError	565
5.47.2.18 UART_IRQEna_IDLE	565
5.47.2.19 UART_IRQEna_LBD	565
5.47.2.20 UART_IRQEna_RcvOver	566
5.47.2.21 UART_IRQEna_RX_OV	566
5.47.2.22 UART_IRQEna_SendBuffEmpty	566
5.47.2.23 UART_IRQEna_SendOver	566
5.47.2.24 UART_IRQEna_StopError	566
5.47.2.25 UART_IRQEna_TX_OV	566
5.47.2.26 UART_STOPBITS_1b	567
5.47.2.27 UART_STOPBITS_2b	567
5.47.2.28 UART_WORDLENGTH_8b	567
5.47.2.29 UART_WORDLENGTH_9b	567
5.47.3 枚举类型说明	567
5.47.3.1 UART_ParityMode	567

5.47.4 函数说明	568
5.47.4.1 UART_ClearIRQFlag()	568
5.47.4.2 UART_GetIRQFlag()	569
5.47.4.3 UART_Init()	570
5.47.4.4 UART_ReadData()	571
5.47.4.5 UART_SendAddr_485()	572
5.47.4.6 UART_SendData()	573
5.47.4.7 UART_SendData_485()	574
5.47.4.8 UART_StructInit()	575
5.48 lks32mc07x_uart.h	576
5.49 include/lks32mc07x_wake.h 文件参考	577
5.49.1 详细描述	580
5.49.2 宏定义说明	580
5.49.2.1 IOWK_FLT_DI	580
5.49.2.2 IOWK_FLT_EN	580
5.49.2.3 SYS_WakeInterval_025s	580
5.49.2.4 SYS_WakeInterval_05s	581
5.49.2.5 SYS_WakeInterval_16s	581
5.49.2.6 SYS_WakeInterval_1s	581
5.49.2.7 SYS_WakeInterval_2s	581
5.49.2.8 SYS_WakeInterval_32s	581
5.49.2.9 SYS_WakeInterval_4s	581
5.49.2.10 SYS_WakeInterval_8s	582
5.49.2.11 WAKEIO_CLUOUT0	582
5.49.2.12 WAKEIO_CLUOUT1	582
5.49.2.13 WAKEIO_CLUOUT2	582
5.49.2.14 WAKEIO_CLUOUT3	582
5.49.2.15 WAKEIO_P0_0	582
5.49.2.16 WAKEIO_P0_11	583
5.49.2.17 WAKEIO_P0_14	583
5.49.2.18 WAKEIO_P0_2	583
5.49.2.19 WAKEIO_P0_6	583
5.49.2.20 WAKEIO_P2_15	583
5.49.2.21 WAKEIO_P2_4	583
5.49.2.22 WAKEIO_P2_7	583
5.49.3 枚举类型说明	583
5.49.3.1 enumWakeTrigLevel	583
5.49.4 函数说明	584
5.49.4.1 SetWakeIO()	584
5.49.4.2 Switch2HRC()	585
5.49.4.3 Switch2PLL()	586
5.49.4.4 SYS_EnterSleep()	587

5.49.4.5 SYS_EVTCLRSleep()	588
5.49.4.6 SYS_FallSleep()	588
5.49.4.7 SYS_SetWakeInterval()	589
5.50 lks32mc07x_wake.h	591
5.51 source/lks32mc07x_adc.c 文件参考	591
5.51.1 详细描述	592
5.51.2 函数说明	593
5.51.2.1 ADC_CHN_GAIN_CFG()	593
5.51.2.2 ADC_ClearIRQFlag()	594
5.51.2.3 ADC_GetConversionValue()	595
5.51.2.4 ADC_GetIRQFlag()	596
5.51.2.5 ADC_Init()	597
5.51.2.6 ADC_SoftTrgEN()	599
5.51.2.7 ADC_StructInit()	599
5.52 lks32mc07x_adc.c	600
5.53 source/lks32mc07x_can.c 文件参考	602
5.53.1 详细描述	604
5.53.2 函数说明	605
5.53.2.1 CAN_ArbitrationLostCap()	605
5.53.2.2 CAN_BusoffGet()	605
5.53.2.3 CAN_Disable()	605
5.53.2.4 CAN_Enable()	606
5.53.2.5 CAN_ErrorPassiveGet()	606
5.53.2.6 CAN_Init()	606
5.53.2.7 CAN_Init1()	607
5.53.2.8 CAN_KindOfErrorGet()	608
5.53.2.9 CAN_LBMECmd()	608
5.53.2.10 CAN_LBMICmd()	609
5.53.2.11 CAN_Receive_Msg()	610
5.53.2.12 CAN_Recv()	611
5.53.2.13 CAN_Reset()	611
5.53.2.14 CAN_ROverflowGet()	612
5.53.2.15 CAN_RxErrorCntGet()	612
5.53.2.16 CAN_Send()	613
5.53.2.17 CAN_Send_Cancel()	613
5.53.2.18 CAN_Send_Trig()	613
5.53.2.19 CAN_SetSTBMode()	614
5.53.2.20 CAN_SS_Enable()	614
5.53.2.21 CAN_STBFullGet()	615
5.53.2.22 CAN_StructInit()	615
5.53.2.23 CAN_TxErrorCntGet()	615
5.53.2.24 CAN_Write_TBUF()	615

5.53.2.25 Delay()	616
5.53.2.26 GetBaudParam()	616
5.53.2.27 IDx_Filter()	617
5.53.2.28 IDx_FilterCmd()	617
5.53.2.29 My_CAN_Send_Msg()	618
5.53.2.30 Reset_CFG_STAT()	618
5.53.2.31 Set_CFG_STAT()	619
5.53.3 变量说明	620
5.53.3.1 g_rxMsg	620
5.54 lks32mc07x_can.c	620
5.55 source/lks32mc07x_clu.c 文件参考	628
5.56 lks32mc07x_clu.c	629
5.57 source/lks32mc07x_cmp.c 文件参考	631
5.57.1 详细描述	631
5.57.2 函数说明	632
5.57.2.1 CMP_ClearIRQFlag()	632
5.57.2.2 CMP_GetFilterValue()	633
5.57.2.3 CMP_GetIRQFlag()	634
5.57.2.4 CMP_GetValue()	635
5.57.2.5 CMP_Init()	636
5.57.2.6 CMP_StructInit()	637
5.58 lks32mc07x_cmp.c	638
5.59 source/lks32mc07x_crc.c 文件参考	639
5.60 lks32mc07x_crc.c	640
5.61 source/lks32mc07x_dac.c 文件参考	643
5.61.1 详细描述	643
5.61.2 函数说明	644
5.61.2.1 DAC_Cmd()	644
5.61.2.2 DAC_Init()	645
5.61.2.3 DAC_Output()	646
5.61.2.4 DAC_OutputValue()	646
5.61.2.5 DAC_OutputVoltage()	647
5.61.2.6 DAC_StructInit()	648
5.62 lks32mc07x_dac.c	649
5.63 source/lks32mc07x_dma.c 文件参考	650
5.63.1 详细描述	651
5.63.2 函数说明	652
5.63.2.1 DMA_CHx_EN()	652
5.63.2.2 DMA_ClearIRQFlag()	653
5.63.2.3 DMA_Disable()	654
5.63.2.4 DMA_Enable()	655
5.63.2.5 DMA_GetFlag()	656

5.63.2.6 DMA_GetIRQFlag()	656
5.63.2.7 DMA_Init()	658
5.63.2.8 DMA_SetAddr()	659
5.63.2.9 DMA_SetLength()	660
5.63.2.10 DMA_SetRequest()	661
5.63.2.11 DMA_StructInit()	663
5.63.2.12 DMA_Trigger()	664
5.64 lks32mc07x_dma.c	665
5.65 source/lks32mc07x_dsp.c 文件参考	666
5.65.1 详细描述	667
5.65.2 函数说明	667
5.65.2.1 DSP_arctan()	667
5.65.2.2 DSP_ArctanMode()	668
5.65.2.3 DSP_cos()	669
5.65.2.4 DSP_div()	669
5.65.2.5 DSP_Init()	670
5.65.2.6 DSP_mod()	671
5.65.2.7 DSP_rms()	672
5.65.2.8 DSP_sin()	673
5.65.2.9 DSP_SinCosMode()	674
5.65.2.10 DSP_sqrt()	674
5.66 lks32mc07x_dsp.c	675
5.67 source/lks32mc07x_exti.c 文件参考	677
5.67.1 详细描述	677
5.67.2 函数说明	678
5.67.2.1 EXTI_ClearIRQFlag()	678
5.67.2.2 EXTI_GetIRQFlag()	679
5.67.2.3 EXTI_ReadIFFlag()	680
5.67.2.4 EXTI_ResetIF()	680
5.67.2.5 EXTI_Trigger_Config()	680
5.68 lks32mc07x_exti.c	681
5.69 source/lks32mc07x_flash.c 文件参考	686
5.69.1 详细描述	687
5.69.2 函数说明	687
5.69.2.1 EraseSector()	687
5.69.2.2 ProgramPage()	688
5.69.2.3 Read_Flash()	690
5.69.2.4 Read_More_Flash()	692
5.69.3 变量说明	693
5.69.3.1 erase_flag	693
5.69.3.2 progm_flag	693
5.70 lks32mc07x_flash.c	694

5.71 source/lks32mc07x_gpio.c 文件参考	695
5.71.1 详细描述	696
5.71.2 函数说明	697
5.71.2.1 GPIO_Init()	697
5.71.2.2 GPIO_PinAFConfig()	698
5.71.2.3 GPIO_ReadInputData()	700
5.71.2.4 GPIO_ReadInputDataBit()	700
5.71.2.5 GPIO_ReadOutputData()	701
5.71.2.6 GPIO_ReadOutputDataBit()	702
5.71.2.7 GPIO_ResetBits()	703
5.71.2.8 GPIO_SetBits()	704
5.71.2.9 GPIO_StructInit()	705
5.71.2.10 GPIO_Write()	706
5.71.2.11 GPIO_WriteBit()	707
5.72 lks32mc07x_gpio.c	708
5.73 source/lks32mc07x_hall.c 文件参考	711
5.73.1 详细描述	712
5.73.2 函数说明	712
5.73.2.1 HALL_Clear_IRQ()	712
5.73.2.2 HALL_Cmd()	713
5.73.2.3 HALL_GetCaptureValue()	714
5.73.2.4 HALL_GetCount()	715
5.73.2.5 HALL_GetFilterValue()	717
5.73.2.6 HALL_GetFlag()	718
5.73.2.7 HALL_GetIRQFlag()	719
5.73.2.8 HALL_Init()	720
5.73.2.9 HALL_IsCaptureEvent()	721
5.73.2.10 HALL_IsOverFlowEvent()	722
5.73.2.11 HALL_SoftTrigger()	723
5.73.2.12 HALL_StructInit()	724
5.74 lks32mc07x_hall.c	725
5.75 source/lks32mc07x_i2c.c 文件参考	726
5.76 lks32mc07x_i2c.c	726
5.77 source/lks32mc07x_mcpwm.c 文件参考	730
5.77.1 详细描述	731
5.77.2 函数说明	731
5.77.2.1 MCPWM_CH012.IESet()	731
5.77.2.2 MCPWM_CH012.Init()	731
5.77.2.3 MCPWM_CH345.IESet()	731
5.77.2.4 MCPWM_CH345.Init()	732
5.77.2.5 MCPWM_ClearIRQFlag()	732
5.77.2.6 MCPWM_Clk_Enable()	735

5.77.2.7 MCPWM_CNTx_Enable()	735
5.77.2.8 MCPWM_DIV_Set()	736
5.77.2.9 MCPWM_GetIRQFlag()	736
5.77.2.10 MCPWM_Init()	739
5.77.2.11 MCPWM_ModuleClockCmd()	741
5.77.2.12 MCPWM_StructInit()	741
5.77.2.13 PWMOutputs()	742
5.77.3 变量说明	743
5.77.3.1 ClockDivflg	743
5.78 lks32mc07x_mcpwm.c	743
5.79 source/lks32mc07x_opa.c 文件参考	749
5.79.1 详细描述	750
5.79.2 函数说明	750
5.79.2.1 OPA_Init()	750
5.79.2.2 OPA_OUT()	751
5.79.2.3 OPA_StructInit()	752
5.79.3 变量说明	753
5.79.3.1 maxmagflg	753
5.80 lks32mc07x_opa.c	753
5.81 source/lks32mc07x_qep.c 文件参考	755
5.81.1 详细描述	755
5.81.2 函数说明	756
5.81.2.1 QEP_ClearIRQFlag()	756
5.81.2.2 QEP_GetFlag()	757
5.81.2.3 QEP_GetIRQFlag()	758
5.81.2.4 TIM_QEP_GetCount()	759
5.81.2.5 TIM_QEPCmd()	760
5.81.2.6 TIM_QEPInit()	761
5.81.2.7 TIM_QEPStructInit()	762
5.82 lks32mc07x_qep.c	763
5.83 source/lks32mc07x_sif.c 文件参考	765
5.83.1 详细描述	765
5.83.2 函数说明	766
5.83.2.1 SIF_Clear_Irq()	766
5.83.2.2 SIF_DMA_IsEnable()	767
5.83.2.3 SIF_Init()	768
5.83.2.4 SIF_Irq_IsEnable()	769
5.83.2.5 SIF_Senddata()	770
5.83.2.6 SIF_StructInit()	771
5.83.2.7 SIF_TH_set()	771
5.83.2.8 SIF_Timebase_set()	772
5.84 lks32mc07x_sif.c	773

5.85 source/lks32mc07x_spi.c 文件参考	774
5.85.1 函数说明	774
5.85.1.1 SPI_ClearIRQFlag()	774
5.85.1.2 SPI_GetIRQFlag()	775
5.85.1.3 SPI_Init()	777
5.85.1.4 SPI_ReadData()	779
5.85.1.5 SPI_SendData()	779
5.85.1.6 SPI_StructInit()	780
5.86 lks32mc07x_spi.c	781
5.87 source/lks32mc07x_sys.c 文件参考	783
5.87.1 详细描述	784
5.87.2 函数说明	784
5.87.2.1 IWDG_DISABLE()	784
5.87.2.2 IWDG_ENABLE()	785
5.87.2.3 IWDG_Feed()	786
5.87.2.4 IWDG_Init()	787
5.87.2.5 IWDG_StructInit()	788
5.87.2.6 SoftDelay()	789
5.87.2.7 SYS_AnalogModuleClockCmd()	790
5.87.2.8 SYS_ClearRst()	792
5.87.2.9 SYS_I2CClkDiv()	793
5.87.2.10 SYS_Init()	794
5.87.2.11 SYS_MclkChoice()	795
5.87.2.12 SYS_ModuleClockCmd()	796
5.87.2.13 SYS_ReadMcuClk()	798
5.87.2.14 SYS_SoftReset_all()	799
5.87.2.15 SYS_SoftResetModule()	800
5.87.2.16 SYS_SPIClkDiv()	802
5.87.2.17 SYS_StructInit()	803
5.87.2.18 SYS_UartClkDiv()	804
5.87.2.19 SYS_VolSelModule()	805
5.87.2.20 SYS_VolSelModuleEnableIRQ()	805
5.87.2.21 SYS_VolSelModuleIRQ()	805
5.87.3 变量说明	806
5.87.3.1 sys_clk	806
5.88 lks32mc07x_sys.c	806
5.89 source/lks32mc07x_tim.c 文件参考	811
5.89.1 详细描述	812
5.89.2 函数说明	812
5.89.2.1 TIM_ClearIRQFlag()	812
5.89.2.2 TIM_CMP0()	813
5.89.2.3 TIM_CMP1()	814

5.89.2.4 TIM_Disable()	815
5.89.2.5 TIM_Enable()	816
5.89.2.6 TIM_GetFlag()	817
5.89.2.7 TIM_GetIRQFlag()	818
5.89.2.8 TIM_Timer_GetCMPT0()	819
5.89.2.9 TIM_Timer_GetCMPT1()	820
5.89.2.10 TIM_Timer_GetCount()	821
5.89.2.11 TIM_TimerCmd()	822
5.89.2.12 TIM_TimerInit()	823
5.89.2.13 TIM_TimerStrutInit()	825
5.89.2.14 UTIMER_Disable()	826
5.89.2.15 UTIMER_Enable()	826
5.90 lks32mc07x_tim.c	827
5.91 source/lks32mc07x_tmp.c 文件参考	829
5.91.1 详细描述	830
5.91.2 函数说明	830
5.91.2.1 GetCurrentTemperature()	830
5.91.2.2 TempSensor_Init()	831
5.91.3 变量说明	832
5.91.3.1 m_TempertureCof	832
5.92 lks32mc07x_tmp.c	833
5.93 source/lks32mc07x_uart.c 文件参考	833
5.93.1 详细描述	834
5.93.2 宏定义说明	834
5.93.2.1 MCU_MCLK	834
5.93.3 函数说明	835
5.93.3.1 UART_ClearIRQFlag()	835
5.93.3.2 UART_GetIRQFlag()	836
5.93.3.3 UART_Init()	837
5.93.3.4 UART_ReadBaud()	838
5.93.3.5 UART_ReadData()	839
5.93.3.6 UART_SendAddr_485()	840
5.93.3.7 UART_SendData()	841
5.93.3.8 UART_SendData_485()	842
5.93.3.9 UART_SetBaud()	843
5.93.3.10 UART_StructInit()	843
5.94 lks32mc07x_uart.c	844
5.95 source/lks32mc07x_wake.c 文件参考	846
5.95.1 详细描述	847
5.95.2 函数说明	848
5.95.2.1 SetWakeIO()	848
5.95.2.2 Switch2HRC()	850

5.95.2.3 Switch2PLL()	851
5.95.2.4 SYS_EnterSleep()	852
5.95.2.5 SYS_EVTCLRSleep()	852
5.95.2.6 SYS_FallSleep()	853
5.95.2.7 SYS_SetWakeInterval()	854
5.95.2.8 wake_SoftDelay()	856
5.95.3 变量说明	857
5.95.3.1 WAKE_InitStruct	857
5.96 lks32mc07x.wake.c	857

0.1 Topics

Here is a list of all topics with brief descriptions:

Clu	11
Crc	14
I2c	17

Chapter 1

结构体索引

1.1 结构体

这里列出了所有结构体，并附带简要说明：

ADC_InitTypeDef	25
ADC_TypeDef	28
AFE_TypeDef	
模拟模块系统寄存器对象定义	40
CAN_InitTypeDef	44
can_parameter	
< Includes -----	45
CAN_TypeDef	47
CANRxMsg	52
CANTxMsg	53
CLU_InitTypeDef	54
CLU_TypeDef	
< Includes -----	57
CMP_InitTypeDef	59
CMP_SingleTypeDef	
CMP初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参	61
CMP_TypeDef	
CMP寄存器结构体句柄	65
CRC_InitTypeDef	66
CRC_TypeDef	
< Includes -----	68
DAC_InitTypeDef	69
DMA_InitTypeDef	
DMA初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参	71
DMA_MODULERegTypeDef	
DMA控制状态寄存器对象定义	73
DMA_RegTypeDef	
DMA通道相关寄存器对象定义	74
DSP_TypeDef	
DSP寄存器结构体句柄	76
EXTI_TypeDef	
EXTI寄存器结构体句柄	78
GPIO_InitTypeDef	
GPIO功能配置结构体句柄	79
GPIO_TypeDef	
< Includes -----*/	81

HALL_InitTypeDef	
HALL初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参	84
HALL_TypeDef	
HALL寄存器结构体句柄	86
I2C_InitTypeDef	87
I2C_TypeDef	89
IWDG_InitTypeDef	
IWDG结构体基地址定义	91
IWDG_TypeDef	
IWDG寄存器结构体句柄	92
MCPWM_InitTypeDef	94
MCPWM_TypeDef	133
OPA_InitTypeDef	
OPA初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参	144
SIF_InitTypeDef	146
SIF_RegTypeDef	148
SPI_InitTypeDef	149
SPI_TypeDef	
SPI寄存器结构体句柄	152
stru_DMA_REN	
DAM搬运使能源定义	153
Stru_TemperatureCof_Def	
芯片内置温度传感器初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参	159
SYS_InitTypeDef	
系统初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参	160
SYS_TypeDef	
数字模块系统配置寄存器对象定义	161
TIM_QEPInitTypeDef	165
TIM_QEPTypeDef	
编码器寄存器结构体句柄	168
TIM_TimerInitTypeDef	169
TIM_TimerTypeDef	179
UART_InitTypeDef	181
UART_TypeDef	
UART寄存器结构体句柄	185
WAKE_InitTypeDef	
低功耗时钟配置中间变量	187

Chapter 2

文件索引

2.1 文件列表

这里列出了所有文件，并附带简要说明：

[include/lks32mc07x_adc.h](#)

文件名称： LKS32MC07x_adc.h

文件标识： 无

内容摘要： ADC外设驱动程序

其它说明： 无

[189](#)

[include/lks32mc07x_can.h](#)

文件名称： LKS32MC07x_can.h

文件标识： 无

内容摘要： CAN外设驱动程序

其它说明： 无

[213](#)

[include/lks32mc07x_clu.h](#)

CLU库函数头文件 [240](#)

[include/lks32mc07x_cmp.h](#)

文件名称： [lks32mc07x_cmp.h](#)

文件标识： 无

内容摘要： CMP外设驱动程序头文件

其它说明： 无

[277](#)

[include/lks32mc07x_crc.h](#)

CRC库函数头文件 [294](#)

[include/lks32mc07x_dac.h](#)

文件名称： [lks32mc07x_dac.h](#)

文件标识： 无

内容摘要： DAC外设驱动程序头文件

其它说明： 无

[300](#)

[include/lks32mc07x_dma.h](#)

文件名称： [lks32mc07x_dma.h](#)

文件标识： 无

内容摘要： DMA外设驱动程序头文件

其它说明： 无

[310](#)

- include/lks32mc07x_dsp.h
 文件名称: lks32mc07x_dsp.h
 文件标识: 无
 内容摘要: DSP驱动头文件
 其它说明: 无
 333
- include/lks32mc07x_exti.h
 文件名称: lks32mc07x_exti.h
 文件标识: 无
 内容摘要: EXTI外部中断驱动头文件
 其它说明: 无
 348
- include/lks32mc07x_flash.h
 文件名称: LKS32MC07x_flash.h
 文件标识: 无
 内容摘要: FLASH外设驱动程序
 其它说明: 无
 354
- include/lks32mc07x_gpio.h
 文件名称: lks32mc07x_gpio.h
 文件标识: 无
 内容摘要: GPIO外设驱动程序头文件
 其它说明: 无
 361
- include/lks32mc07x_hall.h
 文件名称: lks32mc07x_hall.h
 文件标识: 无
 内容摘要: HALL驱动头文件
 其它说明: 无
 386
- include/lks32mc07x_i2c.h
 I2C库函数头文件 402
- include/lks32mc07x_lib.h
 文件名称: lks32mc07x_lib.h
 文件标识: 无
 内容摘要: 库函数头文件
 其它说明: 无
 407
- include/lks32mc07x_mcpwm.h
 文件名称: LKS32MC07x_mcpwm.h
 文件标识: 无
 内容摘要: MCPWM外设驱动程序
 其它说明: 无
 410
- include/lks32mc07x_nvr.h
 文件名称: lks32mc07x_nvr.h
 文件标识: 无
 内容摘要: NVR驱动头文件
 其它说明: 无
 448
- include/lks32mc07x_opa.h
 文件名称: lks32mc07x_opa.h
 文件标识: 无
 内容摘要: OPA驱动头文件
 其它说明: 无
 450

include/lks32mc07x_qep.h	
文件名称: lks32mc07x_qep.h	
文件标识: 无	
内容摘要: QEP驱动头文件	
其它说明: 无	
457	
include/lks32mc07x_sif.h	
文件名称: lks32mc07x_sif.h	
文件标识: 无	
内容摘要: SIF驱动头文件	
其它说明: 无	
470	
include/lks32mc07x_spi.h	481
include/lks32mc07x_sys.h	
文件名称: lks32mc07x_sys.h	
文件标识: 无	
内容摘要: sys驱动头文件	
其它说明: 无	
492	
include/lks32mc07x_tim.h	
文件名称: lks32mc07x_tim.h	
文件标识: 无	
内容摘要: TIM驱动头文件	
其它说明: 无	
529	
include/lks32mc07x_tmp.h	
文件名称: lks32mc08x_tmp.h	
文件标识: 无	
内容摘要: 芯片内置温度传感器驱动头文件	
其它说明: 无	
555	
include/lks32mc07x_uart.h	
文件名称: lks32mc07x_uart.h	
文件标识: 无	
内容摘要: UART驱动头文件	
其它说明: 无	
559	
include/lks32mc07x_wake.h	
文件名称: lks32mc07x_wake.h	
文件标识: 无	
内容摘要: wake驱动头文件	
其它说明: 无	
577	
source/lks32mc07x_adc.c	
文件名称: lks32mc07x_adc.c	
文件标识: 无	
内容摘要: ADC外设驱动程序	
其它说明: 无	
591	
source/lks32mc07x_can.c	
文件名称: lks32mc07x_can.c	
文件标识: 无	
内容摘要: CAN外设驱动程序	
其它说明: 无	
602	
source/lks32mc07x_clu.c	628

source/lks32mc07x_cmp.c	
文件名称: lks32mc07x_cmp.c	
文件标识: 无	
内容摘要: cmp外设驱动程序	
其它说明: 无	
631	
source/lks32mc07x_crc.c	639
source/lks32mc07x_dac.c	
文件名称: LKS32MC07x_dac.c	
文件标识: 无	
内容摘要: DAC外设驱动程序	
其它说明: 无	
643	
source/lks32mc07x_dma.c	
文件名称: LKS32MC07x_dma.c	
文件标识: 无	
内容摘要: DMA外设驱动程序	
其它说明: 无	
650	
source/lks32mc07x_dsp.c	
文件名称: lks32mc07x_dsp.c	
文件标识: 无	
内容摘要: DSP驱动程序	
其它说明: 无	
666	
source/lks32mc07x_exti.c	
文件名称: lks32mc07x_exti.c	
文件标识: 无	
内容摘要: EXTI外设驱动程序	
其它说明: 无	
677	
source/lks32mc07x_flash.c	
文件名称: lks32mc07x_flash.c	
文件标识: 无	
内容摘要: FLASH外设驱动程序	
其它说明: 无	
686	
source/lks32mc07x_gpio.c	
文件名称: lks32mc07x_gpio.c	
文件标识: 无	
内容摘要: GPIO外设驱动程序	
其它说明: 无	
695	
source/lks32mc07x_hall.c	
文件名称: lks32mc07x_hall.c	
文件标识: 无	
内容摘要: HALL外设驱动程序	
其它说明: 无	
711	
source/lks32mc07x_i2c.c	726
source/lks32mc07x_mcpwm.c	
文件名称: lks32mc07x_mcpwm.c	
文件标识: 无	
内容摘要: MCPWM外设驱动程序	
其它说明: 无	
730	

[source/lks32mc07x_opa.c](#)

文件名称: [lks32mc07x_opa.c](#)
文件标识: 无
内容摘要: OPA运放外设驱动程序
其它说明: 无

[749](#)[source/lks32mc07x_qep.c](#)

文件名称: [lks32mc07x_qep.c](#)
文件标识: 无
内容摘要: 编码器外设驱动程序
其它说明: 无

[755](#)[source/lks32mc07x_sif.c](#)

文件名称: [LKS32MC07x_sif.c](#)
文件标识: 无
内容摘要: SIF外设驱动程序
其它说明: 无

[765](#)[source/lks32mc07x_spi.c](#) [774](#)[source/lks32mc07x_sys.c](#)

文件名称: [lks32mc07x_sys.c](#)
文件标识: 无
内容摘要: SYS外设驱动程序
其它说明: 无

[783](#)[source/lks32mc07x_tim.c](#)

文件名称: [lks32mc07x_tim.c](#)
文件标识: 无
内容摘要: 定时器外设驱动程序
其它说明: 无

[811](#)[source/lks32mc07x_tmp.c](#)

文件名称: [lks32mc08x_tmp.c](#)
文件标识: 无
内容摘要: 芯片内置温度传感器外设驱动程序
其它说明: 无

[829](#)[source/lks32mc07x_uart.c](#)

文件名称: [lks32mc07x_uart.c](#)
文件标识: 无
内容摘要: UART外设驱动程序
其它说明: 无

[833](#)[source/lks32mc07x_wake.c](#)

文件名称: [lks32mc07x_wake.c](#)
文件标识: 无
内容摘要: wake外设驱动程序
其它说明: 无

[846](#)

Chapter 3

Topic Documentation

3.1 Clu

CLU库函数

函数

- void **CLU_Init** (**CLU_TypeDef** *CLUx, u8 CLU_MODEL_CLUx, **CLU_InitTypeDef** *this)
CLU计算初始化
- void **CLU_StructInit** (**CLU_InitTypeDef** *this)
CLU初始化结构体初始化
- void **CLU_Enable** (**CLU_TypeDef** *CLUx)
CLU模块时钟使能
- void **CLU_Disable** (**CLU_TypeDef** *CLUx)
CLU模块时钟关闭
- u8 **CLU_ReadOutValue** (**CLU_TypeDef** *CLUx, u8 CLU_MODEL_CLUx)
获取CLU的输出值

3.1.1 详细描述

CLU库函数

作者

YangZJ

日期

2023-02-27

3.1.2 函数说明

3.1.2.1 CLU_Disable()

```
void CLU_Disable (
    CLU_TypeDef * CLUX )
```

CLU模块时钟关闭

在文件 [lks32mc07x.clu.c](#) 第 100 行定义.

函数调用图:



3.1.2.2 CLU_Enable()

```
void CLU_Enable (
    CLU_TypeDef * CLUX )
```

CLU模块时钟使能

在文件 [lks32mc07x.clu.c](#) 第 93 行定义.

函数调用图:



这是这个函数的调用关系图:



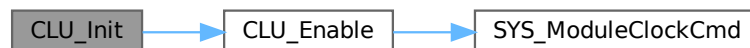
3.1.2.3 CLU_Init()

```
void CLU_Init (
    CLU_TypeDef * CLUx,
    u8 CLU_MODEL_CLUx,
    CLU_InitTypeDef * this )
```

CLU计算初始化

在文件 [lks32mc07x_clu.c](#) 第 14 行定义.

函数调用图:



3.1.2.4 CLU_ReadOutVouue()

```
u8 CLU_ReadOutVouue (
    CLU_TypeDef * CLUx,
    u8 CLU_MODEL_CLUx )
```

获取CLU的输出值

在文件 [lks32mc07x_clu.c](#) 第 107 行定义.

3.1.2.5 CLU_StructInit()

```
void CLU_StructInit (
    CLU_InitTypeDef * this )
```

CLU初始化结构体初始化

< 下降沿中断使能

< 上升沿中断使能

< 模块使能 0关闭模块 1打开模块

< A输入复用选择 格式 CLU.MXA.CLUa.b a取值范围为[0 1 2 3] 对应不同的CLU b表示选择不同的输入信号

< B输入复用选择 格式 CLU.MXB.CLUa.b a取值范围为[0 1 2 3] 对应不同的CLU b表示选择不同的输入信号

< 真值表 格式 CLU.FN_x x取值范围为[A NOTA A_NOTB ...]对应为最简与或式下的最小相 例如 $y=a+bc+lc$ 表示为 CLU.FN_A|CLU.FN.B_C|CLU.FN.NOTC

< 输出选择 0: D 触发器输出 1: LUT 输出

< 输出使能 0: 关闭 1: 使能

< D 触发器时钟电平 0: 同相 1: 反向

< D 触发器时钟选择 0x0: 进位输入 0x1: MXA 输入 0x2: ALTCLK0 0x3: ALTCLK1

在文件 [lks32mc07x_clu.c](#) 第 77 行定义.

3.2 Crc

CRC库函数

函数

- void **CRC_Init** (**CRC_TypeDef** *CRCx, **CRC_InitTypeDef** *this)
CRC计算初始化
- void **CRC_StructInit** (**CRC_InitTypeDef** *this)
CRC初始化结构体初始化
- u32 **CRC_Calc** (**CRC_TypeDef** *CRCx, u8 *data, u32 len)
CRC计算
- u32 **CRC_CalcCommon** (**CRC_TypeDef** *CRCx, u8 *data, u32 len, u16 CRC_MODEL_x)
CRC计算接口
- void **CRC_Enable** (**CRC_TypeDef** *CRCx)
CRC模块时钟使能
- void **CRC_Disable** (**CRC_TypeDef** *CRCx)
CRC模块时钟关闭

变量

- static u32 **CRC_XOROUT** = 0

3.2.1 详细描述

CRC库函数

作者

YangZJ

日期

2023-01-02

3.2.2 函数说明

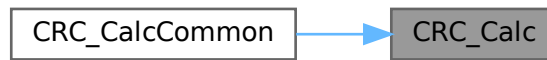
3.2.2.1 CRC_Calc()

```
u32 CRC_Calc (
    CRC_TypeDef * CRCx,
    u8 * data,
    u32 len )
```

CRC计算

在文件 [lks32mc07x_crc.c](#) 第 68 行定义.

这是这个函数的调用关系图:



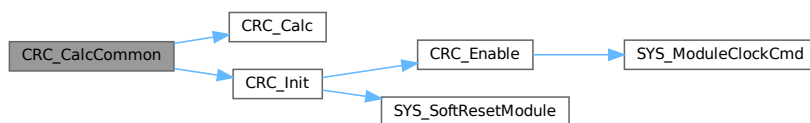
3.2.2.2 CRC_CalcCommon()

```
u32 CRC_CalcCommon (
    CRC_TypeDef * CRCx,
    u8 * data,
    u32 len,
    u16 CRC_MODEL_x )
```

CRC计算接口

在文件 [lks32mc07x_crc.c](#) 第 89 行定义.

函数调用图:



3.2.2.3 CRC_Disable()

```
void CRC_Disable (
    CRC_TypeDef * CRCx )
```

CRC模块时钟关闭

在文件 [lks32mc07x_crc.c](#) 第 273 行定义.

函数调用图:



3.2.2.4 CRC_Enable()

```
void CRC_Enable (
    CRC_TypeDef * CRCx )
```

CRC模块时钟使能

在文件 [lks32mc07x_crc.c](#) 第 263 行定义.

函数调用图:



这是这个函数的调用关系图:



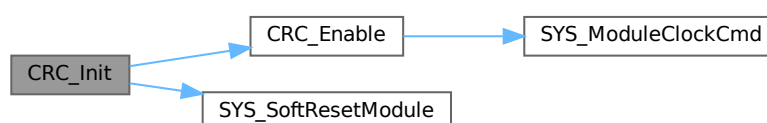
3.2.2.5 CRC_Init()

```
void CRC_Init (
    CRC_TypeDef * CRCx,
    CRC_InitTypeDef * this )
```

CRC计算初始化

在文件 [lks32mc07x_crc.c](#) 第 15 行定义.

函数调用图:



这是这个函数的调用关系图:



3.2.2.6 CRC_StructInit()

```
void CRC_StructInit (
    CRC_InitTypeDef * this )
```

CRC初始化结构体初始化

在文件 [lks32mc07x_crc.c](#) 第 56 行定义.

3.2.3 变量说明

3.2.3.1 CRC_XOROUT

```
u32 CRC_XOROUT = 0 [static]
```

在文件 [lks32mc07x_crc.c](#) 第 10 行定义.

3.3 I2c

i2c库函数

函数

- void [I2C_Init](#) ([I2C_TypeDef](#) *I2Cx, [I2C_InitTypeDef](#) *this)
I2C初始化
- void [I2C_StructInit](#) ([I2C_InitTypeDef](#) *this)
I2C结构体初始化
- u8 [I2c_ReadBusState](#) ([I2C_TypeDef](#) *I2Cx, u16 n)
读I2C总线状态
- void [I2c_ClearBusState](#) ([I2C_TypeDef](#) *I2Cx, u16 n)
I2C总线状态复位
- void [I2C_SendData](#) ([I2C_TypeDef](#) *I2Cx, u8 n)
I2C发送数据
- u8 [I2C_ReadData](#) ([I2C_TypeDef](#) *I2Cx)
I2C读缓冲区数据

- void [I2C_ClearIRQFlag](#) ([I2C_TypeDef](#) *I2Cx, u16 I2C_IF_x)
I2C清除中断 操作的表: 输出参数:
- u8 [I2C_GetIRQFlag](#) ([I2C_TypeDef](#) *I2Cx, u16 I2C_IF_x)
I2C获取当前中断状态 操作的表: 输出参数:
- u8 [I2C_GetFlag](#) ([I2C_TypeDef](#) *I2Cx, u16 I2C_IF_x)
I2C清除中断
- void [I2C_Enable](#) ([I2C_TypeDef](#) *I2Cx)
I2C模块使能
- void [I2C_Disable](#) ([I2C_TypeDef](#) *I2Cx)
I2C模块关闭
- void [I2C_SetBaudRate](#) (u32 BaudRate)
设置I2C模块的波特率

3.3.1 详细描述

i2c库函数

作者

YangZJ

日期

2023-01-02

3.3.2 函数说明

3.3.2.1 I2c_ClearBusState()

```
void I2c_ClearBusState (  
    I2C\_TypeDef * I2Cx,  
    u16 n )
```

I2C总线状态复位

参数

<i>this</i>	I2C配置结构体
-------------	----------

更新记录 V1.0 2021/11/10 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 80 行定义.

3.3.2.2 I2C_ClearIRQFlag()

```
void I2C_ClearIRQFlag (
    I2C_TypeDef * I2Cx,
    u16 I2C_IF_x )
```

I2C清除中断 操作的表： 输出参数：

更新记录 V1.0 2022/08/30 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 110 行定义.

3.3.2.3 I2C_Disable()

```
void I2C_Disable (
    I2C_TypeDef * I2Cx )
```

I2C模块关闭

参数

<i>I2C_TypeDef</i>	*I2Cx
--------------------	-------

更新记录 V1.0 2022/08/30 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 322 行定义.

函数调用图:



3.3.2.4 I2C_Enable()

```
void I2C_Enable (
    I2C_TypeDef * I2Cx )
```

I2C模块使能

参数

<i>I2C_TypeDef</i>	*I2Cx
--------------------	-------

更新记录 V1.0 2022/08/30 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 310 行定义.

函数调用图:



3.3.2.5 I2C_GetFlag()

```
u8 I2C_GetFlag (
    I2C_TypeDef * I2Cx,
    u16 I2C_IF_x )
```

I2C清除中断

参数

<i>I2C_TypeDef</i>	*I2Cx
--------------------	-------

更新记录 V1.0 2022/08/30 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 244 行定义.

3.3.2.6 I2C_GetIRQFlag()

```
u8 I2C_GetIRQFlag (
    I2C_TypeDef * I2Cx,
    u16 I2C_IF_x )
```

I2C获取当前中断状态 操作的表: 输出参数:

更新记录 V1.0 2022/08/30 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 139 行定义.

3.3.2.7 I2C_Init()

```
void I2C_Init (
    I2C_TypeDef * I2Cx,
    I2C_InitTypeDef * this )
```

I2C初始化

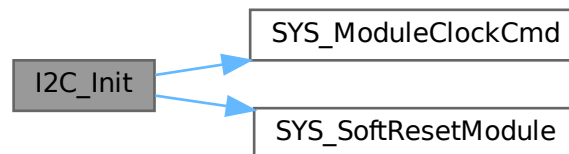
参数

<i>this</i>	I2C配置结构体
-------------	----------

更新记录 V1.0 2021/11/10 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 18 行定义.

函数调用图:



3.3.2.8 I2c_ReadBusState()

```
u8 I2c_ReadBusState (
    I2C_TypeDef * I2Cx,
    u16 n )
```

读I2C总线状态

参数

<i>this</i>	I2C配置结构体
-------------	----------

更新记录 V1.0 2021/11/10 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 64 行定义.

3.3.2.9 I2C_ReadData()

```
u8 I2C_ReadData (
    I2C_TypeDef * I2Cx )
```

I2C读缓冲区数据

参数

<i>I2C_TypeDef</i>	*I2Cx
--------------------	-------

返回

缓冲区数据

更新记录 V1.0 2016/05/21 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 99 行定义.

3.3.2.10 I2C_SendData()

```
void I2C_SendData (
    I2C_TypeDef * I2Cx,
    u8 n )
```

I2C发送数据

更新记录 V1.0 2016/05/21 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 88 行定义.

3.3.2.11 I2C_SetBaudRate()

```
void I2C_SetBaudRate (
    u32 BaudRate )
```

设置I2C模块的波特率

参数

<i>BaudRate</i>	波特率
-----------------	-----

更新记录 V1.0 2022/08/30 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 335 行定义.

3.3.2.12 I2C_StructInit()

```
void I2C_StructInit (  
    I2C_InitTypeDef * this )
```

I2C结构体初始化

I2C配置结构体初始化

参数

<i>this</i>	I2C配置结构体
-------------	----------

更新记录 V1.0 2021/11/10 YangZJ 创建

在文件 [lks32mc07x_i2c.c](#) 第 45 行定义.

Chapter 4

结构体说明

4.1 ADC_InitTypeDef结构体 参考

```
#include <lks32mc07x_adc.h>
```

成员变量

- u16 **IE**
中断使能
- u16 **RE**
DMA请求使能
- u16 **NSMP**
两段采样使能
- u16 **DATA_ALIGN**
DAT右对齐使能
- u16 **CSMP**
连续采样使能
- u16 **TCNT**
触发一次采样所需的事件数
- u16 **TROVS**
手动触发过采样使能，开启后一次采样需要多次触发
- u16 **OVSr**
过采样率
- u16 **TRIG**
触发信号
- u16 **S1**
第一段常规采样次数
- u16 **S2**
第二段常规采样次数
- u16 **IS1**
空闲采样次数
- u16 **GAIN**
低增益（7.2V量程）使能
- u16 **LTH**
ADC 模拟看门狗 0 下阈值
- u16 **HTH**
ADC 模拟看门狗 0 上阈值
- u16 **GEN**
ADC 模拟看门狗 0 对应使能位

4.1.1 详细描述

在文件 [lks32mc07x_adc.h](#) 第 92 行定义.

4.1.2 结构体成员变量说明

4.1.2.1 CSMP

u16 CSMP

连续采样使能

在文件 [lks32mc07x_adc.h](#) 第 98 行定义.

4.1.2.2 DATA_ALIGN

u16 DATA_ALIGN

DAT右对齐使能

在文件 [lks32mc07x_adc.h](#) 第 97 行定义.

4.1.2.3 GAIN

u16 GAIN

低增益（7.2V量程）使能

在文件 [lks32mc07x_adc.h](#) 第 106 行定义.

4.1.2.4 GEN

u16 GEN

ADC 模拟看门狗 0 对应使能位

在文件 [lks32mc07x_adc.h](#) 第 109 行定义.

4.1.2.5 HTH

u16 HTH

ADC 模拟看门狗 0 上阈值

在文件 [lks32mc07x_adc.h](#) 第 108 行定义.

4.1.2.6 IE

u16 IE

中断使能

在文件 [lks32mc07x_adc.h](#) 第 94 行定义.

4.1.2.7 IS1

u16 IS1

空闲采样次数

在文件 [lks32mc07x_adc.h](#) 第 105 行定义.

4.1.2.8 LTH

u16 LTH

ADC 模拟看门狗 0 下阈值

在文件 [lks32mc07x_adc.h](#) 第 107 行定义.

4.1.2.9 NSMP

u16 NSMP

两段采样使能

在文件 [lks32mc07x_adc.h](#) 第 96 行定义.

4.1.2.10 OVSR

u16 OVSR

过采样率

在文件 [lks32mc07x_adc.h](#) 第 101 行定义.

4.1.2.11 RE

u16 RE

DMA请求使能

在文件 [lks32mc07x_adc.h](#) 第 95 行定义.

4.1.2.12 S1

u16 S1

第一段常规采样次数

在文件 [lks32mc07x_adc.h](#) 第 103 行定义.

4.1.2.13 S2

u16 S2

第二段常规采样次数

在文件 [lks32mc07x_adc.h](#) 第 104 行定义.

4.1.2.14 TCNT

u16 TCNT

触发一次采样所需的事件数

在文件 [lks32mc07x_adc.h](#) 第 99 行定义.

4.1.2.15 TRIG

u16 TRIG

触发信号

在文件 [lks32mc07x_adc.h](#) 第 102 行定义.

4.1.2.16 TROVS

u16 TROVS

手动触发过采样使能，开启后一次采样需要多次触发

在文件 [lks32mc07x_adc.h](#) 第 100 行定义.

4.2 ADC_TypeDef结构体 参考

```
#include <lks32mc07x_adc.h>
```

成员变量

- `__IO uint32_t DAT0`
偏移地址: `0x00` ADCx 第 0 次常规采样数据
- `__IO uint32_t DAT1`
偏移地址: `0x04` ADCx 第 1 次常规采样数据
- `__IO uint32_t DAT2`
偏移地址: `0x08` ADCx 第 2 次常规采样数据
- `__IO uint32_t DAT3`
偏移地址: `0x0C` ADCx 第 3 次常规采样数据
- `__IO uint32_t DAT4`
偏移地址: `0x10` ADCx 第 4 次常规采样数据
- `__IO uint32_t DAT5`
偏移地址: `0x14` ADCx 第 5 次常规采样数据
- `__IO uint32_t DAT6`
偏移地址: `0x18` ADCx 第 6 次常规采样数据
- `__IO uint32_t DAT7`
偏移地址: `0x1C` ADCx 第 7 次常规采样数据
- `__IO uint32_t DAT8`
偏移地址: `0x20` ADCx 第 8 次常规采样数据
- `__IO uint32_t DAT9`
偏移地址: `0x24` ADCx 第 9 次常规采样数据
- `__IO uint32_t DAT10`
偏移地址: `0x28` ADCx 第 10 次常规采样数据
- `__IO uint32_t DAT11`
偏移地址: `0x2C` ADCx 第 11 次常规采样数据
- `__IO uint32_t DAT12`
偏移地址: `0x30` ADCx 第 12 次常规采样数据
- `__IO uint32_t DAT13`
偏移地址: `0x34` ADCx 第 13 次常规采样数据
- `__IO uint32_t IDAT0`
偏移地址: `0x38` ADCx 第 0 次空闲采样数据
- `__IO uint32_t IDAT1`
偏移地址: `0x3C` ADCx 第 1 次空闲采样数据
- `__IO uint32_t ICHN`
偏移地址: `0x40` ADCx 空闲采样通道配置
- `__IO uint32_t null`
偏移地址: `0x44` 地址对齐, 无意义
- `__IO uint32_t null1`
偏移地址: `0x48` 地址对齐, 无意义
- `__IO uint32_t null2`
偏移地址: `0x4C` 地址对齐, 无意义
- `__IO uint32_t CHN0`
偏移地址: `0x50` ADCx 第 0~3 次常规采样输入信号选择 *Signal 0 Signal 0 Signal 0 Signal 0 Signal 0 Signal 0*
- `__IO uint32_t CHN1`
偏移地址: `0x54` ADCx 第 4~7 次常规采样输入信号选择
- `__IO uint32_t CHN2`
偏移地址: `0x58` ADCx 第 8~11 次常规采样输入信号选择
- `__IO uint32_t CHN3`
偏移地址: `0x5C` ADCx 第 12~15 次常规采样输入信号选择
- `__IO uint32_t CHNT`

- 偏移地址: 0x60 ADCx 各种触发模式下采样次数
- `__IO uint32_t GAIN`
偏移地址: 0x64 ADCx 增益选择
- `__IO uint32_t null3`
偏移地址: 0x68 地址对齐, 无意义
- `__IO uint32_t null4`
偏移地址: 0x6C 地址对齐, 无意义
- `__IO uint32_t null5`
偏移地址: 0x70 地址对齐, 无意义
- `__IO uint32_t CFG`
偏移地址: 0x74 ADCx 模式配置
- `__IO uint32_t TRIG`
偏移地址: 0x78 ADCx 采样触发配置
- `__IO uint32_t SWT`
偏移地址: 0x7C ADCx 软件触发
- `__IO uint32_t DC0`
偏移地址: 0x80 ADCx 增益为 0 时 DC offset
- `__IO uint32_t AMC0`
偏移地址: 0x84 ADCx 增益为 0 时增益校正系数
- `__IO uint32_t DC1`
偏移地址: 0x88 ADCx 增益为 1 时 DC offset
- `__IO uint32_t AMC1`
偏移地址: 0x8C ADCx 增益为 1 时增益校正系数
- `__IO uint32_t IE`
偏移地址: 0x90 ADCx 中断使能
- `__IO uint32_t IF`
偏移地址: 0x94 ADCx 中断标志
- `__IO uint32_t null6`
偏移地址: 0x98 地址对齐, 无意义
- `__IO uint32_t null7`
偏移地址: 0x9C 地址对齐, 无意义
- `__IO uint32_t null8`
偏移地址: 0xA0 地址对齐, 无意义
- `__IO uint32_t null9`
偏移地址: 0xA4 地址对齐, 无意义
- `__IO uint32_t null10`
偏移地址: 0xA8 地址对齐, 无意义
- `__IO uint32_t null11`
偏移地址: 0xAC 地址对齐, 无意义
- `__IO uint32_t null12`
偏移地址: 0xB0 地址对齐, 无意义
- `__IO uint32_t null13`
偏移地址: 0xB4 地址对齐, 无意义
- `__IO uint32_t null14`
偏移地址: 0xB8 地址对齐, 无意义
- `__IO uint32_t null15`
偏移地址: 0xBC 地址对齐, 无意义
- `__IO uint32_t null16`
偏移地址: 0xCC 地址对齐, 无意义
- `__IO uint32_t LTH`
偏移地址: 0xC4 ADCx 数据低阈值

- `__IO uint32_t HTH`
偏移地址: `0xC8 ADCx` 数据高阈值
- `__IO uint32_t GEN`
偏移地址: `0xCC ADCx` 阈值监测使能

4.2.1 详细描述

在文件 `lks32mc07x_adc.h` 第 21 行定义.

4.2.2 结构体成员变量说明

4.2.2.1 AMC0

```
__IO uint32_t AMC0
```

偏移地址: `0x84 ADCx` 增益为 0 时增益校正系数

在文件 `lks32mc07x_adc.h` 第 64 行定义.

4.2.2.2 AMC1

```
__IO uint32_t AMC1
```

偏移地址: `0x8C ADCx` 增益为 1 时增益校正系数

在文件 `lks32mc07x_adc.h` 第 66 行定义.

4.2.2.3 CFG

```
__IO uint32_t CFG
```

偏移地址: `0x74 ADCx` 模式配置

在文件 `lks32mc07x_adc.h` 第 59 行定义.

4.2.2.4 CHN0

```
__IO uint32_t CHN0
```

偏移地址: `0x50 ADCx` 第 0~3 次常规采样输入信号选择 Signal 0 Signal 0 Signal 0 Signal 0 Signal 0 Signal 0

在文件 `lks32mc07x_adc.h` 第 48 行定义.

4.2.2.5 CHN1

```
--IO uint32_t CHN1
```

偏移地址: 0x54 ADCx 第 4~7 次常规采样输入信号选择

在文件 [lks32mc07x_adc.h](#) 第 49 行定义.

4.2.2.6 CHN2

```
--IO uint32_t CHN2
```

偏移地址: 0x58 ADCx 第 8~11 次常规采样输入信号选择

在文件 [lks32mc07x_adc.h](#) 第 50 行定义.

4.2.2.7 CHN3

```
--IO uint32_t CHN3
```

偏移地址: 0x5C ADCx 第 12~15 次常规采样输入信号选择

在文件 [lks32mc07x_adc.h](#) 第 51 行定义.

4.2.2.8 CHNT

```
--IO uint32_t CHNT
```

偏移地址: 0x60 ADCx 各种触发模式下采样次数

在文件 [lks32mc07x_adc.h](#) 第 53 行定义.

4.2.2.9 DAT0

```
--IO uint32_t DAT0
```

偏移地址: 0x00 ADCx 第 0 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 23 行定义.

4.2.2.10 DAT1

```
--IO uint32_t DAT1
```

偏移地址: 0x04 ADCx 第 1 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 24 行定义.

4.2.2.11 DAT10

```
--IO uint32_t DAT10
```

偏移地址: 0x28 ADCx 第 10 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 35 行定义.

4.2.2.12 DAT11

```
--IO uint32_t DAT11
```

偏移地址: 0x2C ADCx 第 11 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 36 行定义.

4.2.2.13 DAT12

```
--IO uint32_t DAT12
```

偏移地址: 0x30 ADCx 第 12 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 38 行定义.

4.2.2.14 DAT13

```
--IO uint32_t DAT13
```

偏移地址: 0x34 ADCx 第 13 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 39 行定义.

4.2.2.15 DAT2

```
--IO uint32_t DAT2
```

偏移地址: 0x08 ADCx 第 2 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 25 行定义.

4.2.2.16 DAT3

```
--IO uint32_t DAT3
```

偏移地址: 0x0C ADCx 第 3 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 26 行定义.

4.2.2.17 DAT4

```
--IO uint32_t DAT4
```

偏移地址: 0x10 ADCx 第 4 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 28 行定义.

4.2.2.18 DAT5

```
--IO uint32_t DAT5
```

偏移地址: 0x14 ADCx 第 5 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 29 行定义.

4.2.2.19 DAT6

```
--IO uint32_t DAT6
```

偏移地址: 0x18 ADCx 第 6 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 30 行定义.

4.2.2.20 DAT7

```
--IO uint32_t DAT7
```

偏移地址: 0x1C ADCx 第 7 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 31 行定义.

4.2.2.21 DAT8

```
--IO uint32_t DAT8
```

偏移地址: 0x20 ADCx 第 8 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 33 行定义.

4.2.2.22 DAT9

```
--IO uint32_t DAT9
```

偏移地址: 0x24 ADCx 第 9 次常规采样数据

在文件 [lks32mc07x_adc.h](#) 第 34 行定义.

4.2.2.23 DC0

```
--IO uint32_t DC0
```

偏移地址: 0x80 ADCx 增益为 0 时 DC offset

在文件 [lks32mc07x_adc.h](#) 第 63 行定义.

4.2.2.24 DC1

```
--IO uint32_t DC1
```

偏移地址: 0x88 ADCx 增益为 1 时 DC offset

在文件 [lks32mc07x_adc.h](#) 第 65 行定义.

4.2.2.25 GAIN

```
--IO uint32_t GAIN
```

偏移地址: 0x64 ADCx 增益选择

在文件 [lks32mc07x_adc.h](#) 第 54 行定义.

4.2.2.26 GEN

```
--IO uint32_t GEN
```

偏移地址: 0xCC ADCx 阈值监测使能

在文件 [lks32mc07x_adc.h](#) 第 86 行定义.

4.2.2.27 HTH

```
--IO uint32_t HTH
```

偏移地址: 0xC8 ADCx 数据高阈值

在文件 [lks32mc07x_adc.h](#) 第 85 行定义.

4.2.2.28 ICHN

```
--IO uint32_t ICHN
```

偏移地址: 0x40 ADCx 空闲采样通道配置

在文件 [lks32mc07x_adc.h](#) 第 43 行定义.

4.2.2.29 IDAT0

```
--IO uint32_t IDAT0
```

偏移地址: 0x38 ADCx 第 0 次空闲采样数据

在文件 [lks32mc07x_adc.h](#) 第 40 行定义.

4.2.2.30 IDAT1

```
--IO uint32_t IDAT1
```

偏移地址: 0x3C ADCx 第 1 次空闲采样数据

在文件 [lks32mc07x_adc.h](#) 第 41 行定义.

4.2.2.31 IE

```
--IO uint32_t IE
```

偏移地址: 0x90 ADCx 中断使能

在文件 [lks32mc07x_adc.h](#) 第 68 行定义.

4.2.2.32 IF

```
--IO uint32_t IF
```

偏移地址: 0x94 ADCx 中断标志

在文件 [lks32mc07x_adc.h](#) 第 69 行定义.

4.2.2.33 LTH

```
--IO uint32_t LTH
```

偏移地址: 0xC4 ADCx 数据低阈值

在文件 [lks32mc07x_adc.h](#) 第 84 行定义.

4.2.2.34 null

```
--IO uint32_t null
```

偏移地址: 0x44 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 44 行定义.

4.2.2.35 null1

```
--IO uint32_t null1
```

偏移地址: 0x48 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 45 行定义.

4.2.2.36 null10

```
--IO uint32_t null10
```

偏移地址: 0xA8 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 75 行定义.

4.2.2.37 null11

```
--IO uint32_t null11
```

偏移地址: 0xAC 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 76 行定义.

4.2.2.38 null12

```
--IO uint32_t null12
```

偏移地址: 0xB0 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 78 行定义.

4.2.2.39 null13

```
--IO uint32_t null13
```

偏移地址: 0xB4 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 79 行定义.

4.2.2.40 null14

```
--IO uint32_t null14
```

偏移地址: 0xB8 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 80 行定义.

4.2.2.41 null15

```
--IO uint32_t null15
```

偏移地址: 0xBC 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 81 行定义.

4.2.2.42 null16

```
--IO uint32_t null16
```

偏移地址: 0xCC 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 83 行定义.

4.2.2.43 null2

```
--IO uint32_t null2
```

偏移地址: 0x4C 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 46 行定义.

4.2.2.44 null3

```
--IO uint32_t null3
```

偏移地址: 0x68 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 55 行定义.

4.2.2.45 null4

```
--IO uint32_t null4
```

偏移地址: 0x6C 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 56 行定义.

4.2.2.46 null5

```
--IO uint32_t null5
```

偏移地址: 0x70 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 58 行定义.

4.2.2.47 null6

```
--IO uint32_t null6
```

偏移地址: 0x98 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 70 行定义.

4.2.2.48 null7

```
--IO uint32_t null7
```

偏移地址: 0x9C 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 71 行定义.

4.2.2.49 null8

```
--IO uint32_t null8
```

偏移地址: 0xA0 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 73 行定义.

4.2.2.50 null9

```
--IO uint32_t null9
```

偏移地址: 0xA4 地址对齐, 无意义

在文件 [lks32mc07x_adc.h](#) 第 74 行定义.

4.2.2.51 SWT

```
--IO uint32_t SWT
```

偏移地址: 0x7C ADCx 软件触发

在文件 [lks32mc07x_adc.h](#) 第 61 行定义.

4.2.2.52 TRIG

```
--IO uint32_t TRIG
```

偏移地址: 0x78 ADCx 采样触发配置

在文件 [lks32mc07x_adc.h](#) 第 60 行定义.

4.3 AFE_TypeDef结构体 参考

模拟模块系统寄存器对象定义

```
#include <lks32mc07x_sys.h>
```

成员变量

- __IO u32 ADC
- __IO u32 INFO
- __IO u32 DBG
- __IO u32 null7
- __IO u32 REG0
- __IO u32 REG1
- __IO u32 REG2
- __IO u32 REG3
- __IO u32 REG4
- __IO u32 REG5
- __IO u32 REG6
- __IO u32 REG7
- __IO u32 REG8
- __IO u32 REG9
- __IO u32 REGA
- __IO u32 REGB
- __IO u32 REGC
- __IO u32 null1
- __IO u32 null2
- __IO u32 null3
- __IO u32 null4
- __IO u32 null5
- __IO u32 null6
- __IO u32 DAC_CTRL
- __IO u32 DAC0
- __IO u32 DAC1
- __IO u32 DAC0_AMC
- __IO u32 DAC0_DC
- __IO u32 DAC1_AMC
- __IO u32 DAC1_DC

4.3.1 详细描述

模拟模块系统寄存器对象定义

在文件 [lks32mc07x_sys.h](#) 第 23 行定义.

4.3.2 结构体成员变量说明

4.3.2.1 ADC

```
__IO u32 ADC
```

在文件 [lks32mc07x_sys.h](#) 第 25 行定义.

4.3.2.2 DAC0

```
--IO u32 DAC0
```

在文件 [lks32mc07x.sys.h](#) 第 50 行定义.

4.3.2.3 DAC0_AMC

```
--IO u32 DAC0_AMC
```

在文件 [lks32mc07x.sys.h](#) 第 52 行定义.

4.3.2.4 DAC0_DC

```
--IO u32 DAC0_DC
```

在文件 [lks32mc07x.sys.h](#) 第 53 行定义.

4.3.2.5 DAC1

```
--IO u32 DAC1
```

在文件 [lks32mc07x.sys.h](#) 第 51 行定义.

4.3.2.6 DAC1_AMC

```
--IO u32 DAC1_AMC
```

在文件 [lks32mc07x.sys.h](#) 第 54 行定义.

4.3.2.7 DAC1_DC

```
--IO u32 DAC1_DC
```

在文件 [lks32mc07x.sys.h](#) 第 55 行定义.

4.3.2.8 DAC_CTRL

```
--IO u32 DAC_CTRL
```

在文件 [lks32mc07x.sys.h](#) 第 49 行定义.

4.3.2.9 DBG

```
--IO u32 DBG
```

在文件 [lks32mc07x.sys.h](#) 第 27 行定义.

4.3.2.10 INFO

```
--IO u32 INFO
```

在文件 [lks32mc07x.sys.h](#) 第 26 行定义.

4.3.2.11 null1

```
--IO u32 null1
```

在文件 [lks32mc07x.sys.h](#) 第 43 行定义.

4.3.2.12 null2

```
--IO u32 null2
```

在文件 [lks32mc07x.sys.h](#) 第 44 行定义.

4.3.2.13 null3

```
--IO u32 null3
```

在文件 [lks32mc07x.sys.h](#) 第 45 行定义.

4.3.2.14 null4

```
--IO u32 null4
```

在文件 [lks32mc07x.sys.h](#) 第 46 行定义.

4.3.2.15 null5

```
--IO u32 null5
```

在文件 [lks32mc07x.sys.h](#) 第 47 行定义.

4.3.2.16 null6

```
--IO u32 null6
```

在文件 [lks32mc07x.sys.h](#) 第 48 行定义.

4.3.2.17 null7

```
--IO u32 null7
```

在文件 [lks32mc07x.sys.h](#) 第 29 行定义.

4.3.2.18 REG0

```
--IO u32 REG0
```

在文件 [lks32mc07x.sys.h](#) 第 30 行定义.

4.3.2.19 REG1

```
--IO u32 REG1
```

在文件 [lks32mc07x.sys.h](#) 第 31 行定义.

4.3.2.20 REG2

```
--IO u32 REG2
```

在文件 [lks32mc07x.sys.h](#) 第 32 行定义.

4.3.2.21 REG3

```
--IO u32 REG3
```

在文件 [lks32mc07x.sys.h](#) 第 33 行定义.

4.3.2.22 REG4

```
--IO u32 REG4
```

在文件 [lks32mc07x.sys.h](#) 第 34 行定义.

4.3.2.23 REG5

```
--IO u32 REG5
```

在文件 [lks32mc07x.sys.h](#) 第 35 行定义.

4.3.2.24 REG6

```
--IO u32 REG6
```

在文件 [lks32mc07x.sys.h](#) 第 36 行定义.

4.3.2.25 REG7

```
--IO u32 REG7
```

在文件 [lks32mc07x.sys.h](#) 第 37 行定义.

4.3.2.26 REG8

```
--IO u32 REG8
```

在文件 [lks32mc07x_sys.h](#) 第 38 行定义.

4.3.2.27 REG9

```
--IO u32 REG9
```

在文件 [lks32mc07x_sys.h](#) 第 39 行定义.

4.3.2.28 REGA

```
--IO u32 REGA
```

在文件 [lks32mc07x_sys.h](#) 第 40 行定义.

4.3.2.29 REGB

```
--IO u32 REGB
```

在文件 [lks32mc07x_sys.h](#) 第 41 行定义.

4.3.2.30 REGC

```
--IO u32 REGC
```

在文件 [lks32mc07x_sys.h](#) 第 42 行定义.

4.4 CAN_InitTypeDef结构体 参考

```
#include <lks32mc07x_can.h>
```

成员变量

- u32 [SBaud](#)
- u32 [IE](#)
- u32 [ERRINT_IE](#)

4.4.1 详细描述

在文件 [lks32mc07x_can.h](#) 第 123 行定义.

4.4.2 结构体成员变量说明

4.4.2.1 ERRINT_IE

u32 ERRINT_IE

在文件 [lks32mc07x.can.h](#) 第 128 行定义.

4.4.2.2 IE

u32 IE

在文件 [lks32mc07x.can.h](#) 第 127 行定义.

4.4.2.3 SBaud

u32 SBaud

在文件 [lks32mc07x.can.h](#) 第 125 行定义.

4.5 can_parameter结构体 参考

< Includes -----

```
#include <lks32mc07x.can.h>
```

成员变量

- u32 [id](#)
接收ID号
- u8 [len](#)
接收数据长度
- u8 [ide](#)
接收数据类型0标准/1扩展帧
- u8 [rtr](#)
接收数据类型0数据帧/1远程帧
- u8 [TX](#) [8]
数据发生缓冲区
- u8 [RX](#) [8]
接收数据缓冲区
- u32 [rx_sta](#)
接收错误标志位

4.5.1 详细描述

< Includes -----

在文件 [lks32mc07x.can.h](#) 第 20 行定义.

4.5.2 结构体成员变量说明

4.5.2.1 id

u32 id

接收ID号

在文件 [lks32mc07x_can.h](#) 第 22 行定义.

4.5.2.2 ide

u8 ide

接收数据类型0标准/1扩展帧

在文件 [lks32mc07x_can.h](#) 第 24 行定义.

4.5.2.3 len

u8 len

接收数据长度

在文件 [lks32mc07x_can.h](#) 第 23 行定义.

4.5.2.4 rtr

u8 rtr

接收数据类型0数据帧/1远程帧

在文件 [lks32mc07x_can.h](#) 第 25 行定义.

4.5.2.5 RX

u8 RX[8]

接收数据缓冲区

在文件 [lks32mc07x_can.h](#) 第 27 行定义.

4.5.2.6 rx_sta

u32 rx_sta

接收错误标志位

在文件 [lks32mc07x_can.h](#) 第 28 行定义.

4.5.2.7 TX

u8 TX[8]

数据发生缓冲区

在文件 [lks32mc07x_can.h](#) 第 26 行定义.

4.6 CAN_TypeDef结构体 参考

```
#include <lks32mc07x_can.h>
```

成员变量

- __IO u32 [RBUF](#) [4]
0x00~0x0C CAN Rx Buffer寄存器
- __IO u32 [resv0](#) [16]
0x10~0x4F
- __IO u32 [TBUF](#) [4]
0x50~0x5C CAN Tx Buffer寄存器
- __IO u32 [resv1](#) [16]
0x60~0x9F
- __IO u8 [CFG_STAT](#)
0xA0 CAN配置和状态寄存器
- __IO u8 [TCMD](#)
0xA1 CAN发送命令寄存器
- __IO u8 [TCTRL](#)
0xA2 CAN发送控制寄存器
- __IO u8 [RCTRL](#)
0xA3 CAN接收控制寄存器
- __IO u8 [RTIE](#)
0xA4 CAN发送接收中断控制寄存器
- __IO u8 [RTIF](#)
0xA5 CAN发送接收中断标志寄存器
- __IO u8 [ERRINT](#)
0xA6 CAN错误中断使能和标志寄存器
- __IO u8 [LIMIT](#)
0xA7 CAN警告寄存器
- __IO u32 [SBAUD](#)
0xA8 CAN波特率配置寄存器
- __IO u32 [resv2](#)
0xAC
- __IO u8 [EALCAP](#)
0xB0 CAN错误信息和丢失仲裁信息记录寄存器
- __IO u8 [resv3](#)
0xB1
- __IO u8 [RECNT](#)
0xB2 CAN接收错误计数器寄存器
- __IO u8 [TECNT](#)

- 0xB3 CAN发送错误计数器寄存器*
 - __IO u8 [ACFCTRL](#)
0xB4 CAN ID过滤器控制寄存器
 - __IO u8 [resv4](#)
0xB5
 - __IO u16 [ACFEN](#)
0xB6 CAN ID过滤器使能寄存器
 - __IO u32 [ACF](#)
0xB8 CAN ID过滤器选择寄存器

4.6.1 详细描述

在文件 [lks32mc07x.can.h](#) 第 33 行定义.

4.6.2 结构体成员变量说明

4.6.2.1 ACF

__IO u32 ACF

0xB8 CAN ID过滤器选择寄存器

在文件 [lks32mc07x.can.h](#) 第 65 行定义.

4.6.2.2 ACFCTRL

__IO u8 ACFCTRL

0xB4 CAN ID过滤器控制寄存器

在文件 [lks32mc07x.can.h](#) 第 60 行定义.

4.6.2.3 ACFEN

__IO u16 ACFEN

0xB6 CAN ID过滤器使能寄存器

在文件 [lks32mc07x.can.h](#) 第 64 行定义.

4.6.2.4 CFG_STAT

__IO u8 CFG_STAT

0xA0 CAN配置和状态寄存器

在文件 [lks32mc07x.can.h](#) 第 41 行定义.

4.6.2.5 EALCAP

```
--IO u8 EALCAP
```

0xB0 CAN错误信息和丢失仲裁信息记录寄存器

在文件 [lks32mc07x_can.h](#) 第 54 行定义.

4.6.2.6 ERRINT

```
--IO u8 ERRINT
```

0xA6 CAN错误中断使能和标志寄存器

在文件 [lks32mc07x_can.h](#) 第 48 行定义.

4.6.2.7 LIMIT

```
--IO u8 LIMIT
```

0xA7 CAN警告寄存器

在文件 [lks32mc07x_can.h](#) 第 49 行定义.

4.6.2.8 RBUF

```
--IO u32 RBUF[4]
```

0x00~0x0C CAN Rx Buffer寄存器

在文件 [lks32mc07x_can.h](#) 第 35 行定义.

4.6.2.9 RCTRL

```
--IO u8 RCTRL
```

0xA3 CAN接收控制寄存器

在文件 [lks32mc07x_can.h](#) 第 44 行定义.

4.6.2.10 RECNT

```
--IO u8 RECNT
```

0xB2 CAN接收错误计数器寄存器

在文件 [lks32mc07x_can.h](#) 第 58 行定义.

4.6.2.11 resv0

```
--IO u32 resv0[16]
```

0x10~0x4F

在文件 [lks32mc07x_can.h](#) 第 36 行定义.

4.6.2.12 resv1

```
--IO u32 resv1[16]
```

0x60~0x9F

在文件 [lks32mc07x_can.h](#) 第 39 行定义.

4.6.2.13 resv2

```
--IO u32 resv2
```

0xAC

在文件 [lks32mc07x_can.h](#) 第 52 行定义.

4.6.2.14 resv3

```
--IO u8 resv3
```

0xB1

在文件 [lks32mc07x_can.h](#) 第 56 行定义.

4.6.2.15 resv4

```
--IO u8 resv4
```

0xB5

在文件 [lks32mc07x_can.h](#) 第 62 行定义.

4.6.2.16 RTIE

```
--IO u8 RTIE
```

0xA4 CAN发送接收中断控制寄存器

在文件 [lks32mc07x_can.h](#) 第 46 行定义.

4.6.2.17 RTIF

```
--IO u8 RTIF
```

0xA5 CAN发送接收中断标志寄存器

在文件 [lks32mc07x_can.h](#) 第 47 行定义.

4.6.2.18 SBAUD

```
--IO u32 SBAUD
```

0xA8 CAN波特率配置寄存器

在文件 [lks32mc07x_can.h](#) 第 50 行定义.

4.6.2.19 TBUF

```
--IO u32 TBUF[4]
```

0x50~0x5C CAN Tx Buffer寄存器

在文件 [lks32mc07x_can.h](#) 第 38 行定义.

4.6.2.20 TCMD

```
--IO u8 TCMD
```

0xA1 CAN发送命令寄存器

在文件 [lks32mc07x_can.h](#) 第 42 行定义.

4.6.2.21 TCTRL

```
--IO u8 TCTRL
```

0xA2 CAN发送控制寄存器

在文件 [lks32mc07x_can.h](#) 第 43 行定义.

4.6.2.22 TECNT

```
--IO u8 TECNT
```

0xB3 CAN发送错误计数器寄存器

在文件 [lks32mc07x_can.h](#) 第 59 行定义.

4.7 CANRxMsg结构体 参考

```
#include <lks32mc07x_can.h>
```

成员变量

- u32 [id](#)
- u8 [ide](#)
- u8 [rtr](#)
- u8 [valid](#)
- u8 [len](#)
- u8 [buf](#) [64]

4.7.1 详细描述

在文件 [lks32mc07x_can.h](#) 第 190 行定义.

4.7.2 结构体成员变量说明

4.7.2.1 buf

```
u8 buf[64]
```

在文件 [lks32mc07x_can.h](#) 第 198 行定义.

4.7.2.2 id

```
u32 id
```

在文件 [lks32mc07x_can.h](#) 第 192 行定义.

4.7.2.3 ide

```
u8 ide
```

在文件 [lks32mc07x_can.h](#) 第 193 行定义.

4.7.2.4 len

```
u8 len
```

在文件 [lks32mc07x_can.h](#) 第 197 行定义.

4.7.2.5 rtr

u8 rtr

在文件 [lks32mc07x_can.h](#) 第 194 行定义.

4.7.2.6 valid

u8 valid

在文件 [lks32mc07x_can.h](#) 第 195 行定义.

4.8 CANTxMsg结构体 参考

```
#include <lks32mc07x_can.h>
```

成员变量

- u32 id
- u8 ide
- u8 rtr
- u8 * buf
- u8 len
- TBSelOption tbsel
- u8 ss

4.8.1 详细描述

在文件 [lks32mc07x_can.h](#) 第 176 行定义.

4.8.2 结构体成员变量说明

4.8.2.1 buf

u8* buf

在文件 [lks32mc07x_can.h](#) 第 182 行定义.

4.8.2.2 id

u32 id

在文件 [lks32mc07x_can.h](#) 第 178 行定义.

4.8.2.3 ide

u8 ide

在文件 [lks32mc07x_can.h](#) 第 179 行定义.

4.8.2.4 len

u8 len

在文件 [lks32mc07x_can.h](#) 第 183 行定义.

4.8.2.5 rtr

u8 rtr

在文件 [lks32mc07x_can.h](#) 第 180 行定义.

4.8.2.6 ss

u8 ss

在文件 [lks32mc07x_can.h](#) 第 186 行定义.

4.8.2.7 tbsel

TBSelOption tbsel

在文件 [lks32mc07x_can.h](#) 第 185 行定义.

4.9 CLU_InitTypeDef结构体 参考

```
#include <lks32mc07x_clu.h>
```


成员变量

- u32 **EN**
模块使能 0:关闭模块 1:打开模块
- u32 **FIE**
下降沿中断使能
- u32 **RIE**
上升沿中断使能
- u32 **MXA**
A输入复用选择 格式 *CLU_MXA_CLUa_b* a取值范围为[0 1 2 3] 对应不同的CLU b表示选择不同的输入信号
- u32 **MXB**
B输入复用选择 格式 *CLU_MXB_CLUa_b* a取值范围为[0 1 2 3] 对应不同的CLU b表示选择不同的输入信号
- u32 **FN**
真值表 格式 *CLU_FN_x* x取值范围为[A NOTA A_NOTB ...]对应为最简与或式下的最小相 例如 $y=a+bc+!c$ 表示为 *CLU_FN_A|CLU_FN_B.C|CLU_FN_NOTC*
- u32 **OUTSEL**
输出选择 0: D 触发器输出 1: LUT 输出
- u32 **OEN**
输出使能 0: 关闭 1: 使能
- u32 **CLKINV**
D 触发器时钟电平 0: 同相 1: 反向
- u32 **CLKSEL**
D 触发器时钟选择 0x0: 进位输入 0x1: MXA 输入 0x2: ALTCLK0 0x3: ALTCLK1

4.9.1 详细描述

在文件 [lks32mc07x.clu.h](#) 第 28 行定义.

4.9.2 结构体成员变量说明

4.9.2.1 CLKINV

u32 CLKINV

D 触发器时钟电平 0: 同相 1: 反向

在文件 [lks32mc07x.clu.h](#) 第 38 行定义.

4.9.2.2 CLKSEL

u32 CLKSEL

D 触发器时钟选择 0x0: 进位输入 0x1: MXA 输入 0x2: ALTCLK0 0x3: ALTCLK1

在文件 [lks32mc07x.clu.h](#) 第 39 行定义.

4.9.2.3 EN

u32 EN

模块使能 0关闭模块 1打开模块

在文件 [lks32mc07x_clu.h](#) 第 30 行定义.

4.9.2.4 FIE

u32 FIE

下降沿中断使能

在文件 [lks32mc07x_clu.h](#) 第 31 行定义.

4.9.2.5 FN

u32 FN

真值表 格式 CLU_FN_x x取值范围为[A NOTA A.NOTB ...]对应为最简与或式下的最小相 例如 $y=a+bc+!c$ 表示为 CLU_FN_A|CLU_FN_B.C|CLU_FN_NOTC

在文件 [lks32mc07x_clu.h](#) 第 35 行定义.

4.9.2.6 MXA

u32 MXA

A输入复用选择 格式 CLU_MXA.CLUa_b a取值范围为[0 1 2 3] 对应不同的CLU b表示选择不同的输入信号

在文件 [lks32mc07x_clu.h](#) 第 33 行定义.

4.9.2.7 MXB

u32 MXB

B输入复用选择 格式 CLU_MXB.CLUa_b a取值范围为[0 1 2 3] 对应不同的CLU b表示选择不同的输入信号

在文件 [lks32mc07x_clu.h](#) 第 34 行定义.

4.9.2.8 OEN

u32 OEN

输出使能 0: 关闭 1: 使能

在文件 [lks32mc07x_clu.h](#) 第 37 行定义.

4.9.2.9 OUTSEL

u32 OUTSEL

输出选择 0: D 触发器输出 1: LUT 输出

在文件 [lks32mc07x_clu.h](#) 第 36 行定义.

4.9.2.10 RIE

u32 RIE

上升沿中断使能

在文件 [lks32mc07x_clu.h](#) 第 32 行定义.

4.10 CLU_TypeDef结构体 参考

< Includes -----

```
#include <lks32mc07x_clu.h>
```

成员变量

- `__IO u32 EN`
0x00 CL0 使能寄存器
- `__IO u32 IE`
0x04 CL0 中断使能寄存器
- `__IO u32 IF`
0x08 CL0 中断标志寄存器
- `__IO u32 OUT`
0x0C CL0 输出寄存器
- `__IO u32 MX`
0x10 CL0 输入多路复选寄存器
- `__IO u32 FN`
0x14 CL0 功能选择寄存器
- `__IO u32 CF`
0x18 CL0 配置寄存器

4.10.1 详细描述

< Includes -----

在文件 [lks32mc07x_clu.h](#) 第 13 行定义.

4.10.2 结构体成员变量说明

4.10.2.1 CF

`--IO u32 CF`

0x18 CL0 配置寄存器

在文件 [lks32mc07x_clu.h](#) 第 21 行定义.

4.10.2.2 EN

`--IO u32 EN`

0x00 CL0 使能寄存器

在文件 [lks32mc07x_clu.h](#) 第 15 行定义.

4.10.2.3 FN

`--IO u32 FN`

0x14 CL0 功能选择寄存器

在文件 [lks32mc07x_clu.h](#) 第 20 行定义.

4.10.2.4 IE

`--IO u32 IE`

0x04 CL0 中断使能寄存器

在文件 [lks32mc07x_clu.h](#) 第 16 行定义.

4.10.2.5 IF

`--IO u32 IF`

0x08 CL0 中断标志寄存器

在文件 [lks32mc07x_clu.h](#) 第 17 行定义.

4.10.2.6 MX

`--IO u32 MX`

0x10 CL0 输入多路复选寄存器

在文件 [lks32mc07x_clu.h](#) 第 19 行定义.

4.10.2.7 OUT

--IO u32 OUT

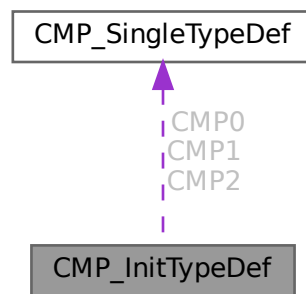
0x0C CL0 输出寄存器

在文件 [lks32mc07x.clu.h](#) 第 18 行定义.

4.11 CMP_InitTypeDef结构体 参考

```
#include <lks32mc07x_cmp.h>
```

CMP_InitTypeDef 的协作图:



成员变量

- u8 [HYS](#)
比较器回差选择
- FuncState [FT](#)
比较器快速比较使能 *DISABLE/ENABLE*
- u32 [CLK_COM_DIV](#)
比较器共用分频系数，取值0-4
- [CMP_SingleTypeDef](#) [CMP0](#)
*CMP0*配置
- [CMP_SingleTypeDef](#) [CMP1](#)
*CMP1*配置
- [CMP_SingleTypeDef](#) [CMP2](#)
*CMP2*配置

4.11.1 详细描述

在文件 [lks32mc07x_cmp.h](#) 第 68 行定义.

4.11.2 结构体成员变量说明

4.11.2.1 CLK_COM_DIV

u32 CLK_COM_DIV

比较器共用分频系数，取值0-4

在文件 [lks32mc07x_cmp.h](#) 第 77 行定义.

4.11.2.2 CMP0

[CMP_SingleTypeDef](#) CMP0

CMP0配置

在文件 [lks32mc07x_cmp.h](#) 第 78 行定义.

4.11.2.3 CMP1

[CMP_SingleTypeDef](#) CMP1

CMP1配置

在文件 [lks32mc07x_cmp.h](#) 第 79 行定义.

4.11.2.4 CMP2

[CMP_SingleTypeDef](#) CMP2

CMP2配置

在文件 [lks32mc07x_cmp.h](#) 第 80 行定义.

4.11.2.5 FT

FuncState FT

比较器快速比较使能 DISABLE/ENABLE

在文件 [lks32mc07x_cmp.h](#) 第 76 行定义.

4.11.2.6 HYS

u8 HYS

比较器回差选择

参见

[CMP_HYS_20mV](#)

[CMP_HYS_0mV](#) 比较器回差选择

在文件 [lks32mc07x_cmp.h](#) 第 75 行定义.

4.12 CMP_SingleTypeDef结构体 参考

CMP初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

```
#include <lks32mc07x_cmp.h>
```

成员变量

- u16 SELP
比较器信号正端选择
- u16 SELN
比较器 信号负端选择
- u32 FIL_CLK_DIV16
 CMP 滤波时钟频率 $Freq(CMP_Filter) = (Freq(MCLK)/2^{FIL_CLK_DIV2})/(FIL_CLK_DIV16+1)$ ，其中 $MCLK$ 为系统的主时钟，通常为 96MHz 全速时钟。需要注意的是，产生 CMP 滤波时钟需要使能 CLK_EN 。
- u32 FIL_CLK_DIV2
比较器 012 滤波系数， $0x0:1$ 分频， $0x1:2$ 分频， $0x2:4$ 分频， $0x3:8$ 分频， $0x4:16$ 分频， $0x5:32$ 分频， $0x6:64$ 分频， $0x7:128$ 分频
- u16 POL
比较器输出极性选择
- u16 W_PWM_POL
比较器开窗极性选择
- u16 IRQ_TRIG
比较器触发中断方式选择
- FuncState CLK_EN
比较器时钟使能， $DISABLE$:失能， $ENABLE$: 使能
- FuncState EN
比较器使能
 $DISABLE$:失能， $ENABLE$: 使能
- FuncState IE
比较器中断使能， $DISABLE$:失能， $ENABLE$: 使能
- FuncState RE
比较器 DMA 请求使能， $DISABLE$:失能， $ENABLE$: 使能
- FuncState IN_EN
比较器信号输入使能， $DISABLE$:失能， $ENABLE$: 使能
- FuncState CHN3P_PWM_EN
 $MCPWM$ 模块 $CHN3_P$ 通道使能比较器 $0/1$ 开窗， $DISABLE$:失能， $ENABLE$: 使能
- FuncState CHN2P_PWM_EN
 $MCPWM$ 模块 $CHN2_P$ 通道使能比较器 $0/1$ 开窗， $DISABLE$:失能， $ENABLE$: 使能
- FuncState CHN1P_PWM_EN
 $MCPWM$ 模块 $CHN1_P$ 通道使能比较器 $0/1$ 开窗， $DISABLE$:失能， $ENABLE$: 使能
- FuncState CHN0P_PWM_EN
 $MCPWM$ 模块 $CHN0_P$ 通道使能比较器 $0/1$ 开窗， $DISABLE$:失能， $ENABLE$: 使能

4.12.1 详细描述

CMP初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

在文件 [lks32mc07x_cmp.h](#) 第 21 行定义。

4.12.2 结构体成员变量说明

4.12.2.1 CHN0P_PWM_EN

FuncState CHN0P_PWM_EN

MCPWM 模块 CHN0_P 通道使能比较器 0/1 开窗, DISABLE:失能, ENABLE: 使能
在文件 [lks32mc07x_cmp.h](#) 第 66 行定义.

4.12.2.2 CHN1P_PWM_EN

FuncState CHN1P_PWM_EN

MCPWM 模块 CHN1_P 通道使能比较器 0/1 开窗, DISABLE:失能, ENABLE: 使能
在文件 [lks32mc07x_cmp.h](#) 第 65 行定义.

4.12.2.3 CHN2P_PWM_EN

FuncState CHN2P_PWM_EN

MCPWM 模块 CHN2_P 通道使能比较器 0/1 开窗, DISABLE:失能, ENABLE: 使能
在文件 [lks32mc07x_cmp.h](#) 第 64 行定义.

4.12.2.4 CHN3P_PWM_EN

FuncState CHN3P_PWM_EN

MCPWM 模块 CHN3_P 通道使能比较器 0/1 开窗, DISABLE:失能, ENABLE: 使能
在文件 [lks32mc07x_cmp.h](#) 第 63 行定义.

4.12.2.5 CLK_EN

FuncState CLK_EN

比较器时钟使能, DISABLE:失能, ENABLE: 使能
在文件 [lks32mc07x_cmp.h](#) 第 58 行定义.

4.12.2.6 EN

FuncState EN

比较器使能
DISABLE:失能, ENABLE: 使能

在文件 [lks32mc07x_cmp.h](#) 第 59 行定义.

4.12.2.7 FIL_CLK_DIV16

u32 FIL_CLK_DIV16

CMP 滤波时钟频率 $\text{Freq}(\text{CMP_Filter}) = (\text{Freq}(\text{MCLK})/2^{\text{FIL_CLK_DIV2}})/(\text{FIL_CLK_DIV16}+1)$ ，其中MCLK 为系统的主时钟，通常为 96MHz 全速时钟。需要注意的是，产生 CMP 滤波时钟需要使能CLK.EN。

比较器012滤波系数，赋值范围0-0xF

在文件 [lks32mc07x_cmp.h](#) 第 38 行定义.

4.12.2.8 FIL_CLK_DIV2

u32 FIL_CLK_DIV2

比较器012滤波系数，0x0:1分频，0x1:2分频,0x2:4分频,0x3:8分频，0x4:16分频，0x5:32分频,0x6:64分频,0x7:128分频

在文件 [lks32mc07x_cmp.h](#) 第 39 行定义.

4.12.2.9 IE

FuncState IE

比较器中断使能，DISABLE:失能，ENABLE: 使能

在文件 [lks32mc07x_cmp.h](#) 第 60 行定义.

4.12.2.10 IN_EN

FuncState IN_EN

比较器信号输入使能，DISABLE:失能，ENABLE: 使能

在文件 [lks32mc07x_cmp.h](#) 第 62 行定义.

4.12.2.11 IRQ_TRIG

u16 IRQ_TRIG

比较器触发中断方式选择

参见

[IRQ_LEVEL_TRIG_MODE](#)

[IRQ_EDGE_TRIG_MODE](#) 比较器边沿/电平触发中断选择

在文件 [lks32mc07x_cmp.h](#) 第 57 行定义.

4.12.2.12 POL

u16 POL

比较器输出极性选择

参见

[CMP_HIGH_LEVEL](#)

[CMP_LOW_LEVEL](#) 比较器有效输出极性选择

在文件 [lks32mc07x_cmp.h](#) 第 45 行定义.

4.12.2.13 RE

FuncState RE

比较器DMA 请求使能, DISABLE:失能, ENABLE: 使能

在文件 [lks32mc07x_cmp.h](#) 第 61 行定义.

4.12.2.14 SELN

u16 SELN

比较器 信号负端选择

参见

[CMP_SELN_IN](#)

[CMP_SELN_REF](#)

[CMP_SELN_DAC0](#)

[CMP0_SELN_HALL0_MID](#)

[CMP1_SELN_HALL1_MID](#)

[CMP2_SELN_DAC1](#) 比较器信号负端选择

在文件 [lks32mc07x_cmp.h](#) 第 33 行定义.

4.12.2.15 SELP

u16 SELP

比较器信号正端选择

在文件 [lks32mc07x_cmp.h](#) 第 23 行定义.

4.12.2.16 W_PWM_POL

u16 W_PWM_POL

比较器开窗极性选择

参见

[W_PWM_HIGH_LEVEL](#)

[W_PWM_LOW_LEVEL](#) 比较器开窗 PWM 信号极性选择,比较器 0/1

在文件 [lks32mc07x_cmp.h](#) 第 51 行定义.

4.13 CMP_TypeDef结构体 参考

CMP寄存器结构体句柄

```
#include <lks32mc07x_cmp.h>
```

成员变量

- [__IO u32 IE](#)
比较器中断使能寄存器
- [__IO u32 IF](#)
比较器中断标志寄存器
- [__IO u32 TCLK](#)
比较器分频时钟控制寄存器
- [__IO u32 CFG](#)
比较器控制寄存器
- [__IO u32 BLCWIN](#)
比较器开窗控制寄存器
- [__IO u32 DATA](#)
比较器输出数值寄存器

4.13.1 详细描述

CMP寄存器结构体句柄

在文件 [lks32mc07x_cmp.h](#) 第 86 行定义.

4.13.2 结构体成员变量说明

4.13.2.1 BLCWIN

[__IO u32 BLCWIN](#)

比较器开窗控制寄存器

在文件 [lks32mc07x_cmp.h](#) 第 92 行定义.

4.13.2.2 CFG

```
--IO u32 CFG
```

比较器控制寄存器

在文件 [lks32mc07x_cmp.h](#) 第 91 行定义.

4.13.2.3 DATA

```
--IO u32 DATA
```

比较器输出数值寄存器

在文件 [lks32mc07x_cmp.h](#) 第 93 行定义.

4.13.2.4 IE

```
--IO u32 IE
```

比较器中断使能寄存器

在文件 [lks32mc07x_cmp.h](#) 第 88 行定义.

4.13.2.5 IF

```
--IO u32 IF
```

比较器中断标志寄存器

在文件 [lks32mc07x_cmp.h](#) 第 89 行定义.

4.13.2.6 TCLK

```
--IO u32 TCLK
```

比较器分频时钟控制寄存器

在文件 [lks32mc07x_cmp.h](#) 第 90 行定义.

4.14 CRC_InitTypeDef结构体 参考

```
#include <lks32mc07x_crc.h>
```

成员变量

- u32 **POLY**
多项式
- u32 **INIT**
初始值
- u32 **XOROUT**
结果异或值
- u8 **REFIN**
输入数据反转
- u8 **REFOUT**
输出数据反转
- u8 **WIDTH**
宽度
- u8 **null**
地址对齐

4.14.1 详细描述

在文件 [lks32mc07x_crc.h](#) 第 25 行定义.

4.14.2 结构体成员变量说明

4.14.2.1 INIT

u32 INIT

初始值

在文件 [lks32mc07x_crc.h](#) 第 28 行定义.

4.14.2.2 null

u8 null

地址对齐

在文件 [lks32mc07x_crc.h](#) 第 33 行定义.

4.14.2.3 POLY

u32 POLY

多项式

在文件 [lks32mc07x_crc.h](#) 第 27 行定义.

4.14.2.4 REFIN

u8 REFIN

输入数据反转

在文件 [lks32mc07x_crc.h](#) 第 30 行定义.

4.14.2.5 REFOUT

u8 REFOUT

输出数据反转

在文件 [lks32mc07x_crc.h](#) 第 31 行定义.

4.14.2.6 WIDTH

u8 WIDTH

宽度

在文件 [lks32mc07x_crc.h](#) 第 32 行定义.

4.14.2.7 XOROUT

u32 XOROUT

结果异或值

在文件 [lks32mc07x_crc.h](#) 第 29 行定义.

4.15 CRC_TypeDef结构体 参考

< Includes -----

```
#include <lks32mc07x_crc.h>
```

成员变量

- `__IO u32 DR`
0x00 CRC 数据（输入信息码/输出编码）寄存器
- `__IO u32 CR`
0x04 CRC 控制寄存器
- `__IO u32 INIT`
0x08 CRC 初始码寄存器
- `__IO u32 POL`
0x0C CRC 生成多项式对应的二进制码寄存器

4.15.1 详细描述

< Includes -----

在文件 [lks32mc07x_crc.h](#) 第 13 行定义.

4.15.2 结构体成员变量说明

4.15.2.1 CR

--IO u32 CR

0x04 CRC 控制寄存器

在文件 [lks32mc07x_crc.h](#) 第 16 行定义.

4.15.2.2 DR

--IO u32 DR

0x00 CRC 数据（输入信息码/输出编码）寄存器

在文件 [lks32mc07x_crc.h](#) 第 15 行定义.

4.15.2.3 INIT

--IO u32 INIT

0x08 CRC 初始码寄存器

在文件 [lks32mc07x_crc.h](#) 第 17 行定义.

4.15.2.4 POL

--IO u32 POL

0x0C CRC 生成多项式对应的二进制码寄存器

在文件 [lks32mc07x_crc.h](#) 第 18 行定义.

4.16 DAC_InitTypeDef结构体 参考

```
#include <lks32mc07x_dac.h>
```

成员变量

- `uint8_t DAC_GAIN`
DAC输出档位: 1, 1.2V; 0, 4.85V
- `uint8_t DACOUT_EN`
DAC输出使能: *DISABLE*, 不使能; *ENABLE*, 使能
- `int8_t DAC_STEP`
步进值, 3bit 有符号数, 范围-4~3
- `uint8_t TIG_CH_EN`
UTIMER0/1/2/3触发步进使能,*DISABLE*, 不使能; *ENABLE*, 使能

4.16.1 详细描述

在文件 `lks32mc07x-dac.h` 第 20 行定义.

4.16.2 结构体成员变量说明

4.16.2.1 DAC_GAIN

`uint8_t DAC_GAIN`

DAC输出档位: 1, 1.2V; 0, 4.85V

参见

`DAC_RANGE_1V2`

`DAC_RANGE_4V85` DAC输出档位: 1:1.2V; 0:4.85V

在文件 `lks32mc07x-dac.h` 第 27 行定义.

4.16.2.2 DAC_STEP

`int8_t DAC_STEP`

步进值, 3bit 有符号数, 范围-4~3

在文件 `lks32mc07x-dac.h` 第 29 行定义.

4.16.2.3 DACOUT_EN

`uint8_t DACOUT_EN`

DAC输出使能: *DISABLE*, 不使能; *ENABLE*, 使能

在文件 `lks32mc07x-dac.h` 第 28 行定义.

4.16.2.4 TIG_CH_EN

```
uint8_t TIG_CH_EN
```

UTIMER0/1/2/3触发步进使能,DISABLE, 不使能; ENABLE, 使能

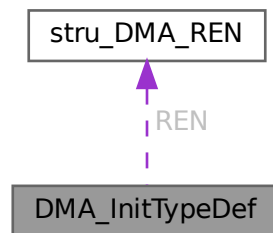
在文件 [lks32mc07x_dac.h](#) 第 30 行定义.

4.17 DMA_InitTypeDef结构体 参考

DMA初始化结构体句柄, 初始化时请定义该句柄, 并用其它地址来传参

```
#include <lks32mc07x_dma.h>
```

DMA_InitTypeDef 的协作图:



成员变量

- u8 [IRQ_EN](#)
- u8 [SBTW](#)
- u8 [DBTW](#)
- u8 [SINC](#)
- u8 [DINC](#)
- u8 [CIRC](#)
- u8 [RMODE](#)
- u8 [EN](#)
- u8 [TMS](#)
- u32 [SADR](#)
- u32 [DADR](#)
- [stru_DMA_REN](#) [REN](#)

4.17.1 详细描述

DMA初始化结构体句柄, 初始化时请定义该句柄, 并用其它地址来传参

在文件 [lks32mc07x_dma.h](#) 第 97 行定义.

4.17.2 结构体成员变量说明

4.17.2.1 CIRC

u8 CIRC

在文件 [lks32mc07x_dma.h](#) 第 104 行定义.

4.17.2.2 DADR

u32 DADR

在文件 [lks32mc07x_dma.h](#) 第 109 行定义.

4.17.2.3 DBTW

u8 DBTW

在文件 [lks32mc07x_dma.h](#) 第 101 行定义.

4.17.2.4 DINC

u8 DINC

在文件 [lks32mc07x_dma.h](#) 第 103 行定义.

4.17.2.5 EN

u8 EN

在文件 [lks32mc07x_dma.h](#) 第 106 行定义.

4.17.2.6 IRQ_EN

u8 IRQ_EN

在文件 [lks32mc07x_dma.h](#) 第 99 行定义.

4.17.2.7 REN

[stru_DMA_REN](#) REN

在文件 [lks32mc07x_dma.h](#) 第 110 行定义.

4.17.2.8 RMODE

u8 RMODE

在文件 [lks32mc07x_dma.h](#) 第 105 行定义.

4.17.2.9 SADR

u32 SADR

在文件 [lks32mc07x_dma.h](#) 第 108 行定义.

4.17.2.10 SBTW

u8 SBTW

在文件 [lks32mc07x_dma.h](#) 第 100 行定义.

4.17.2.11 SINC

u8 SINC

在文件 [lks32mc07x_dma.h](#) 第 102 行定义.

4.17.2.12 TMS

u8 TMS

在文件 [lks32mc07x_dma.h](#) 第 107 行定义.

4.18 DMA_MODULERegTypeDef结构体 参考

DMA控制状态寄存器对象定义

```
#include <lks32mc07x_dma.h>
```

成员变量

- `__IO u32 CTRL`
DMA 控制寄存器
- `__IO u32 IE`
DMA 中断使能寄存器
- `__IO u32 IF`
DMA 中断标志寄存器

4.18.1 详细描述

DMA控制状态寄存器对象定义

在文件 [lks32mc07x_dma.h](#) 第 35 行定义.

4.18.2 结构体成员变量说明

4.18.2.1 CTRL

```
--IO u32 CTRL
```

DMA 控制寄存器

在文件 [lks32mc07x_dma.h](#) 第 37 行定义.

4.18.2.2 IE

```
--IO u32 IE
```

DMA 中断使能寄存器

在文件 [lks32mc07x_dma.h](#) 第 38 行定义.

4.18.2.3 IF

```
--IO u32 IF
```

DMA 中断标志寄存器

在文件 [lks32mc07x_dma.h](#) 第 39 行定义.

4.19 DMA_RegTypeDef结构体 参考

DMA通道相关寄存器对象定义

```
#include <lks32mc07x_dma.h>
```

成员变量

- `--IO uint32_t CCR`
DMA 通道配置寄存器
- `--IO uint32_t REN`
DMA 请求使能寄存器
- `--IO uint32_t CTMS`
DMA传输次数设置寄存器
- `--IO uint32_t SADR`
DMA 源地址设置寄存器
- `--IO uint32_t DADR`
DMA 目的地址设置

4.19.1 详细描述

DMA通道相关寄存器对象定义

在文件 [lks32mc07x_dma.h](#) 第 23 行定义.

4.19.2 结构体成员变量说明

4.19.2.1 CCR

```
--IO uint32_t CCR
```

DMA 通道配置寄存器

在文件 [lks32mc07x_dma.h](#) 第 25 行定义.

4.19.2.2 CTMS

```
--IO uint32_t CTMS
```

DMA传输次数设置寄存器

在文件 [lks32mc07x_dma.h](#) 第 27 行定义.

4.19.2.3 DADR

```
--IO uint32_t DADR
```

DMA 目的地址设置

在文件 [lks32mc07x_dma.h](#) 第 29 行定义.

4.19.2.4 REN

```
--IO uint32_t REN
```

DMA 请求使能寄存器

在文件 [lks32mc07x_dma.h](#) 第 26 行定义.

4.19.2.5 SADR

```
--IO uint32_t SADR
```

DMA 源地址设置寄存器

在文件 [lks32mc07x_dma.h](#) 第 28 行定义.

4.20 DSP_Typedef结构体 参考

DSP寄存器结构体句柄

```
#include <lks32mc07x_dsp.h>
```

成员变量

- __IO u32 SC
- __IO u32 THETA
- __IO u32 X
- __IO u32 Y
- __IO u32 SIN
- __IO u32 COS
- __IO u32 MOD
- __IO u32 ARCTAN
- __IO u32 DID
- __IO u32 DIS
- __IO u32 QUO
- __IO u32 REM
- __IO u32 RAD
- __IO u32 SQRT
- __IO u32 PC

4.20.1 详细描述

DSP寄存器结构体句柄

在文件 [lks32mc07x_dsp.h](#) 第 25 行定义.

4.20.2 结构体成员变量说明

4.20.2.1 ARCTAN

```
__IO u32 ARCTAN
```

在文件 [lks32mc07x_dsp.h](#) 第 34 行定义.

4.20.2.2 COS

```
__IO u32 COS
```

在文件 [lks32mc07x_dsp.h](#) 第 32 行定义.

4.20.2.3 DID

```
__IO u32 DID
```

在文件 [lks32mc07x_dsp.h](#) 第 35 行定义.

4.20.2.4 DIS

```
--IO u32 DIS
```

在文件 [lks32mc07x_dsp.h](#) 第 36 行定义.

4.20.2.5 MOD

```
--IO u32 MOD
```

在文件 [lks32mc07x_dsp.h](#) 第 33 行定义.

4.20.2.6 PC

```
--IO u32 PC
```

在文件 [lks32mc07x_dsp.h](#) 第 41 行定义.

4.20.2.7 QUO

```
--IO u32 QUO
```

在文件 [lks32mc07x_dsp.h](#) 第 37 行定义.

4.20.2.8 RAD

```
--IO u32 RAD
```

在文件 [lks32mc07x_dsp.h](#) 第 39 行定义.

4.20.2.9 REM

```
--IO u32 REM
```

在文件 [lks32mc07x_dsp.h](#) 第 38 行定义.

4.20.2.10 SC

```
--IO u32 SC
```

在文件 [lks32mc07x_dsp.h](#) 第 27 行定义.

4.20.2.11 SIN

```
--IO u32 SIN
```

在文件 [lks32mc07x_dsp.h](#) 第 31 行定义.

4.20.2.12 SQRT

```
--IO u32 SQRT
```

在文件 [lks32mc07x_dsp.h](#) 第 40 行定义.

4.20.2.13 THETA

```
--IO u32 THETA
```

在文件 [lks32mc07x_dsp.h](#) 第 28 行定义.

4.20.2.14 X

```
--IO u32 X
```

在文件 [lks32mc07x_dsp.h](#) 第 29 行定义.

4.20.2.15 Y

```
--IO u32 Y
```

在文件 [lks32mc07x_dsp.h](#) 第 30 行定义.

4.21 EXTI_TypeDef结构体 参考

EXTI寄存器结构体句柄

```
#include <lks32mc07x_exti.h>
```

成员变量

- `_IO uint32_t CR0`
- `_IO uint32_t CR1`
- `_IO uint32_t IE`
- `_IO uint32_t IF`

4.21.1 详细描述

EXTI寄存器结构体句柄

在文件 [lks32mc07x_exti.h](#) 第 25 行定义.

4.21.2 结构体成员变量说明

4.21.2.1 CR0

```
__IO uint32_t CR0
```

在文件 [lks32mc07x_exti.h](#) 第 27 行定义.

4.21.2.2 CR1

```
__IO uint32_t CR1
```

在文件 [lks32mc07x_exti.h](#) 第 28 行定义.

4.21.2.3 IE

```
__IO uint32_t IE
```

在文件 [lks32mc07x_exti.h](#) 第 29 行定义.

4.21.2.4 IF

```
__IO uint32_t IF
```

在文件 [lks32mc07x_exti.h](#) 第 30 行定义.

4.22 GPIO_InitTypeDef结构体 参考

GPIO功能配置结构体句柄

```
#include <lks32mc07x_gpio.h>
```

成员变量

- u32 [GPIO.Pin](#)
要配置的引脚
- u32 [GPIO.Mode](#)
GPIO模式
- u32 [GPIO.PuPd](#)
上拉使能
- u32 [GPIO.PODEna](#)
开漏使能
- u32 [GPIO.PFLT](#)

4.22.1 详细描述

GPIO功能配置结构体句柄

在文件 [lks32mc07x_gpio.h](#) 第 92 行定义.

4.22.2 结构体成员变量说明

4.22.2.1 GPIO_Mode

u32 GPIO_Mode

GPIO模式

在文件 [lks32mc07x_gpio.h](#) 第 95 行定义.

4.22.2.2 GPIO_PFLT

u32 GPIO_PFLT

在文件 [lks32mc07x_gpio.h](#) 第 98 行定义.

4.22.2.3 GPIO_Pin

u32 GPIO_Pin

要配置的引脚

在文件 [lks32mc07x_gpio.h](#) 第 94 行定义.

4.22.2.4 GPIO_PODEna

u32 GPIO_PODEna

开漏使能

在文件 [lks32mc07x_gpio.h](#) 第 97 行定义.

4.22.2.5 GPIO_PuPd

u32 GPIO_PuPd

上拉使能

在文件 [lks32mc07x_gpio.h](#) 第 96 行定义.

4.23 GPIO_TypeDef结构体 参考

< Includes -----*/

```
#include <lks32mc07x_gpio.h>
```

成员变量

- __IO uint32_t [PIE](#)
- __IO uint32_t [POE](#)
- __IO uint32_t [PDI](#)
- __IO uint32_t [PDO](#)
- __IO uint32_t [PUE](#)
- __IO uint32_t [null](#)
- __IO uint32_t [PODE](#)
- __IO uint32_t [PFLT](#)
- __IO uint32_t [F3210](#)
- __IO uint32_t [F7654](#)
- __IO uint32_t [FBA98](#)
- __IO uint32_t [FFEDC](#)
- __IO uint32_t [BSRR](#)
- __IO uint32_t [BRR](#)

4.23.1 详细描述

< Includes -----*/

GPIO寄存器结构体句柄

在文件 [lks32mc07x_gpio.h](#) 第 22 行定义.

4.23.2 结构体成员变量说明

4.23.2.1 BRR

```
__IO uint32_t BRR
```

0x34 GPIO x 位清零寄存器

在文件 [lks32mc07x_gpio.h](#) 第 37 行定义.

4.23.2.2 BSRR

```
__IO uint32_t BSRR
```

0x30 GPIO x 位操作寄存器

在文件 [lks32mc07x_gpio.h](#) 第 36 行定义.

4.23.2.3 F3210

```
--IO uint32_t F3210
```

0x20 GPIO x [3:0]功能选择

在文件 [lks32mc07x_gpio.h](#) 第 32 行定义.

4.23.2.4 F7654

```
--IO uint32_t F7654
```

0x24 GPIO x [7:4]功能选择

在文件 [lks32mc07x_gpio.h](#) 第 33 行定义.

4.23.2.5 FBA98

```
--IO uint32_t FBA98
```

0x28 GPIO x [11:8]功能选择

在文件 [lks32mc07x_gpio.h](#) 第 34 行定义.

4.23.2.6 FFEDC

```
--IO uint32_t FFEDC
```

0x2C GPIO x [15:12]功能选择

在文件 [lks32mc07x_gpio.h](#) 第 35 行定义.

4.23.2.7 null

```
--IO uint32_t null
```

0x14 结构体地址对齐, 无意义

在文件 [lks32mc07x_gpio.h](#) 第 29 行定义.

4.23.2.8 PDI

```
--IO uint32_t PDI
```

0x08 GPIO x 输入数据

在文件 [lks32mc07x_gpio.h](#) 第 26 行定义.

4.23.2.9 PDO

--IO uint32_t PDO

0x0C GPIO x 输出数据

在文件 [lks32mc07x_gpio.h](#) 第 27 行定义.

4.23.2.10 PFLT

--IO uint32_t PFLT

0x1c GPIO x 滤波使能

在文件 [lks32mc07x_gpio.h](#) 第 31 行定义.

4.23.2.11 PIE

--IO uint32_t PIE

0x00 GPIO x 输入使能

在文件 [lks32mc07x_gpio.h](#) 第 24 行定义.

4.23.2.12 PODE

--IO uint32_t PODE

0x18 GPIO x 开漏使能

在文件 [lks32mc07x_gpio.h](#) 第 30 行定义.

4.23.2.13 POE

--IO uint32_t POE

0x04 GPIO x 输出使能

在文件 [lks32mc07x_gpio.h](#) 第 25 行定义.

4.23.2.14 PUE

--IO uint32_t PUE

0x10 GPIO x 上拉使能

在文件 [lks32mc07x_gpio.h](#) 第 28 行定义.

4.24 HALL_InitTypeDef结构体 参考

HALL初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

```
#include <lks32mc07x_hall.h>
```

成员变量

- `uint32_t CountTH`
HALL计数器门限值，有效位为24位
- `uint16_t FilterLen`
滤波长度,0对应长度1,1023对应长度1024滤波长度
- `uint8_t ClockDivision`
分频 0~3:/1 /2 /4 /8
- `uint8_t Filter75.Ena`
使能第一级7/5滤波,高电平有效,ENABLE: 使能 DISABLE: 失能
- `uint8_t HALL.Ena`
使能HALL,高电平有效,ENABLE: 使能 DISABLE: 失能
- `uint8_t Capture_IRQ.Ena`
HALL信号变化中断使能,高电平有效,ENABLE: 使能 DISABLE: 失能
- `uint8_t OverFlow_IRQ.Ena`
HALL计数器溢出中断使能,高电平有效,ENABLE: 使能 DISABLE: 失能
- `uint8_t softIE`
软件中断使能,ENABLE: 使能 DISABLE: 失能

4.24.1 详细描述

HALL初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

在文件 `lks32mc07x_hall.h` 第 41 行定义.

4.24.2 结构体成员变量说明

4.24.2.1 Capture_IRQ.Ena

```
uint8_t Capture_IRQ.Ena
```

HALL信号变化中断使能,高电平有效,ENABLE: 使能 DISABLE: 失能

在文件 `lks32mc07x_hall.h` 第 48 行定义.

4.24.2.2 ClockDivision

```
uint8_t ClockDivision
```

分频 0~3:/1 /2 /4 /8

在文件 `lks32mc07x_hall.h` 第 45 行定义.

4.24.2.3 CountTH

uint32_t CountTH

HALL计数器门限值,有效位为24位

在文件 [lks32mc07x_hall.h](#) 第 43 行定义.

4.24.2.4 Filter75_Ena

uint8_t Filter75_Ena

使能第一级7/5滤波,高电平有效,ENABLE: 使能 DISABLE: 失能

在文件 [lks32mc07x_hall.h](#) 第 46 行定义.

4.24.2.5 FilterLen

uint16_t FilterLen

滤波长度,0对应长度1,1023对应长度1024滤波长度

在文件 [lks32mc07x_hall.h](#) 第 44 行定义.

4.24.2.6 HALL_Ena

uint8_t HALL_Ena

使能HALL,高电平有效,ENABLE: 使能 DISABLE: 失能

在文件 [lks32mc07x_hall.h](#) 第 47 行定义.

4.24.2.7 OverFlow_IRQ_Ena

uint8_t OverFlow_IRQ_Ena

HALL计数器溢出中断使能,高电平有效,ENABLE: 使能 DISABLE: 失能

在文件 [lks32mc07x_hall.h](#) 第 49 行定义.

4.24.2.8 softIE

uint8_t softIE

软件中断使能,ENABLE: 使能 DISABLE: 失能

在文件 [lks32mc07x_hall.h](#) 第 50 行定义.

4.25 HALL_TypeDef结构体 参考

HALL寄存器结构体句柄

```
#include <lks32mc07x_hall.h>
```

成员变量

- `__IO uint32_t CFG`
HALL 模块配置寄存器
- `__IO uint32_t INFO`
HALL 模块信息寄存器
- `__IO uint32_t WIDTH`
HALL 宽度计数值寄存器
- `__IO uint32_t TH`
HALL 模块计数器门限值寄存器
- `__IO uint32_t CNT`
HALL 计数寄存器

4.25.1 详细描述

HALL寄存器结构体句柄

在文件 [lks32mc07x_hall.h](#) 第 24 行定义.

4.25.2 结构体成员变量说明

4.25.2.1 CFG

```
__IO uint32_t CFG
```

HALL 模块配置寄存器

在文件 [lks32mc07x_hall.h](#) 第 26 行定义.

4.25.2.2 CNT

```
__IO uint32_t CNT
```

HALL 计数寄存器

在文件 [lks32mc07x_hall.h](#) 第 30 行定义.

4.25.2.3 INFO

```
__IO uint32_t INFO
```

HALL 模块信息寄存器

在文件 [lks32mc07x_hall.h](#) 第 27 行定义.

4.25.2.4 TH

```
--IO uint32_t TH
```

HALL 模块计数器门限值寄存器

在文件 [lks32mc07x_hall.h](#) 第 29 行定义.

4.25.2.5 WIDTH

```
--IO uint32_t WIDTH
```

HALL 宽度计数值寄存器

在文件 [lks32mc07x_hall.h](#) 第 28 行定义.

4.26 I2C_InitTypeDef结构体 参考

```
#include <lks32mc07x_i2c.h>
```

成员变量

- u32 [ADRCMP](#)
I2C 硬件地址比较使能开关
- u32 [MST.MODE](#)
I2C 主模式使能信号
- u32 [SLV.MODE](#)
I2C 从模式使能信号
- u32 [BUSRT_EN](#)
I2C DMA传输使能
- u32 [BaudRate](#)
I2C 波特率
- u32 [IE](#)
I2C 中断使能信号
- u32 [TC_IE](#)
I2C 数据传输完成中断使能
- u32 [BUS_ERR_IE](#)
I2C 总线错误事件中断使能信号
- u32 [STOP_IE](#)
I2C STOP 事件中断使能信号
- u32 [BURST_NACK](#)
NACK 事件中断使能
- u32 [BURST_ADDR_CMP](#)
硬件地址匹配中断使能

4.26.1 详细描述

在文件 [lks32mc07x_i2c.h](#) 第 45 行定义.

4.26.2 结构体成员变量说明

4.26.2.1 ADRCMP

u32 ADRCMP

I2C 硬件地址比较使能开关

在文件 [lks32mc07x_i2c.h](#) 第 47 行定义.

4.26.2.2 BaudRate

u32 BaudRate

I2C 波特率

在文件 [lks32mc07x_i2c.h](#) 第 51 行定义.

4.26.2.3 BURST_ADDR_CMP

u32 BURST_ADDR_CMP

硬件地址匹配中断使能

在文件 [lks32mc07x_i2c.h](#) 第 57 行定义.

4.26.2.4 BURST_NACK

u32 BURST_NACK

NACK 事件中断使能

在文件 [lks32mc07x_i2c.h](#) 第 56 行定义.

4.26.2.5 BUS_ERR_IE

u32 BUS_ERR_IE

I2C 总线错误事件中断使能信号

在文件 [lks32mc07x_i2c.h](#) 第 54 行定义.

4.26.2.6 BUSRT_EN

u32 BUSRT_EN

I2C DMA传输使能

在文件 [lks32mc07x_i2c.h](#) 第 50 行定义.

4.26.2.7 IE

u32 IE

I2C 中断使能信号

在文件 [lks32mc07x_i2c.h](#) 第 52 行定义.

4.26.2.8 MST_MODE

u32 MST_MODE

I2C 主模式使能信号

在文件 [lks32mc07x_i2c.h](#) 第 48 行定义.

4.26.2.9 SLV_MODE

u32 SLV_MODE

I2C 从模式使能信号

在文件 [lks32mc07x_i2c.h](#) 第 49 行定义.

4.26.2.10 STOP_IE

u32 STOP_IE

I2C STOP 事件中断使能信号

在文件 [lks32mc07x_i2c.h](#) 第 55 行定义.

4.26.2.11 TC_IE

u32 TC_IE

I2C 数据传输完成中断使能

在文件 [lks32mc07x_i2c.h](#) 第 53 行定义.

4.27 I2C_TypeDef结构体 参考

```
#include <lks32mc07x_i2c.h>
```

成员变量

- `__IO u32 ADDR`
0x00 I2C 地址寄存器
- `__IO u32 CFG`
0x04 I2C 配置寄存器
- `__IO u32 SCR`
0x08 I2C 状态寄存器
- `__IO u32 DATA`
0x0C I2C 数据寄存器
- `__IO u32 MSCR`
0x10 I2C 主模式寄存器
- `__IO u32 BCR`
0x14 I2C 传输控制寄存器
- `__IO u32 BSIZE`
0x18 I2C 传输长度寄存器

4.27.1 详细描述

在文件 `lks32mc07x_i2c.h` 第 30 行定义.

4.27.2 结构体成员变量说明

4.27.2.1 ADDR

```
__IO u32 ADDR
```

0x00 I2C 地址寄存器

在文件 `lks32mc07x_i2c.h` 第 32 行定义.

4.27.2.2 BCR

```
__IO u32 BCR
```

0x14 I2C 传输控制寄存器

在文件 `lks32mc07x_i2c.h` 第 37 行定义.

4.27.2.3 BSIZE

```
__IO u32 BSIZE
```

0x18 I2C 传输长度寄存器

在文件 `lks32mc07x_i2c.h` 第 38 行定义.

4.27.2.4 CFG

```
--IO u32 CFG
```

0x04 I2C 配置寄存器

在文件 [lks32mc07x_i2c.h](#) 第 33 行定义.

4.27.2.5 DATA

```
--IO u32 DATA
```

0x0C I2C 数据寄存器

在文件 [lks32mc07x_i2c.h](#) 第 35 行定义.

4.27.2.6 MSCR

```
--IO u32 MSCR
```

0x10 I2C 主模式寄存器

在文件 [lks32mc07x_i2c.h](#) 第 36 行定义.

4.27.2.7 SCR

```
--IO u32 SCR
```

0x08 I2C 状态寄存器

在文件 [lks32mc07x_i2c.h](#) 第 34 行定义.

4.28 IWDG_InitTypeDef结构体 参考

IWDG结构体基地址定义

```
#include <lks32mc07x_sys.h>
```

成员变量

- u32 [DWK_EN](#)
深度休眠定时唤醒使能
- u32 [WDG_EN](#)
独立看门狗使能
- u32 [WTH](#)
看门狗定时唤醒时间 (21位计数器, 但低12恒位0)
- u32 [RTH](#)
看门狗超时复位时间

4.28.1 详细描述

IWDG结构体基地址定义

IWDG初始化结构体句柄

在文件 [lks32mc07x-sys.h](#) 第 237 行定义.

4.28.2 结构体成员变量说明

4.28.2.1 DWK_EN

u32 DWK_EN

深度休眠定时唤醒使能

在文件 [lks32mc07x-sys.h](#) 第 239 行定义.

4.28.2.2 RTH

u32 RTH

看门狗超时复位时间

在文件 [lks32mc07x-sys.h](#) 第 242 行定义.

4.28.2.3 WDG_EN

u32 WDG_EN

独立看门狗使能

在文件 [lks32mc07x-sys.h](#) 第 240 行定义.

4.28.2.4 WTH

u32 WTH

看门狗定时唤醒时间 (21位计数器, 但低12恒位0)

在文件 [lks32mc07x-sys.h](#) 第 241 行定义.

4.29 IWDG_TypeDef结构体 参考

IWDG寄存器结构体句柄

```
#include <lks32mc07x-sys.h>
```

成员变量

- `__IO uint32_t PSW`
IWDG密码寄存器
- `__IO uint32_t CFG`
IWDG配置寄存器
- `__IO uint32_t CLR`
IWDG清零寄存器
- `__IO uint32_t WTH`
IWDG计数器定时唤醒门限值寄存器
- `__IO uint32_t RTH`
IWDG计数器超时复位门限值寄存器
- `__IO uint32_t CNT`
IWDG计数器当前计数值寄存器

4.29.1 详细描述

IWDG寄存器结构体句柄

在文件 `lks32mc07x_sys.h` 第 219 行定义.

4.29.2 结构体成员变量说明

4.29.2.1 CFG

```
__IO uint32_t CFG
```

IWDG配置寄存器

在文件 `lks32mc07x_sys.h` 第 222 行定义.

4.29.2.2 CLR

```
__IO uint32_t CLR
```

IWDG清零寄存器

在文件 `lks32mc07x_sys.h` 第 223 行定义.

4.29.2.3 CNT

```
__IO uint32_t CNT
```

IWDG计数器当前计数值寄存器

在文件 `lks32mc07x_sys.h` 第 226 行定义.

4.29.2.4 PSW

```
--IO uint32_t PSW
```

IWDG密码寄存器

在文件 [lks32mc07x_sys.h](#) 第 221 行定义.

4.29.2.5 RTH

```
--IO uint32_t RTH
```

IWDG计数器超时复位门限值寄存器

在文件 [lks32mc07x_sys.h](#) 第 225 行定义.

4.29.2.6 WTH

```
--IO uint32_t WTH
```

IWDG计数器定时唤醒门限值寄存器

在文件 [lks32mc07x_sys.h](#) 第 224 行定义.

4.30 MCPWM_InitTypeDef结构体 参考

```
#include <lks32mc07x_mcpwm.h>
```

成员变量

- uint16_t [TH0](#)
MCPWM时基0计数器门限值 (PWM周期)
- uint16_t [TH1](#)
MCPWM时基1计数器门限值 (PWM周期)
- uint16_t [TH00](#)
MCPWM时基0 CH0P比较门限值 (PWM占空比)
- uint16_t [TH01](#)
MCPWM时基0 CH0N比较门限值 (PWM占空比)
- uint16_t [TH10](#)
MCPWM时基0 CH1P比较门限值 (PWM占空比)
- uint16_t [TH11](#)
MCPWM时基0 CH1N比较门限值 (PWM占空比)
- uint16_t [TH20](#)
MCPWM时基0 CH2P比较门限值 (PWM占空比)
- uint16_t [TH21](#)
MCPWM时基0 CH2N比较门限值 (PWM占空比)
- uint16_t [TH30](#)
MCPWM时基1 CH3P比较门限值 (PWM占空比)

- uint16_t **TH31**
MCPWM时基1 CH3N比较门限值 (PWM占空比)
- uint16_t **TH40**
MCPWM时基1 CH4P比较门限值 (PWM占空比)
- uint16_t **TH41**
MCPWM时基1 CH4N比较门限值 (PWM占空比)
- uint16_t **TH50**
MCPWM时基1 CH5P比较门限值 (PWM占空比)
- uint16_t **TH51**
MCPWM时基1 CH5N比较门限值 (PWM占空比)
- uint8_t **CMP_CTRL_CNT1**
比较器控制计数器1使能
- uint8_t **CMP_CTRL_CNT0**
比较器控制计数器0使能
- uint8_t **BASE_CNT1_EN**
MCPWM 计数器1使能开关
- uint8_t **BASE_CNT0_EN**
MCPWM 计数器0使能开关
- uint8_t **EVT_CNT1_EN**
*< MCPWM 计数器1外部触发使能
- uint8_t **EVT_CNT0_EN**
MCPWM 计数器0外部触发使能
- uint8_t **TMR3_TB**
MCPWM TMR3时基 (计数器) 选择 0: 时基0; 1: 时基1
- uint8_t **TMR2_TB**
MCPWM TMR2时基 (计数器) 选择 0: 时基0; 1: 时基1
- uint8_t **ZCS_EN**
使能ZCS事件检测 高有效
- uint8_t **MCLK_EN**
MCPWM时钟使能开关
- uint8_t **CLK_DIV**
MCPWM 分频系数
- uint16_t **MCPWM_WorkModeCH0**
MCPWM CH0工作模式: 边沿对齐/中心对齐
- uint16_t **MCPWM_WorkModeCH1**
MCPWM CH1工作模式: 边沿对齐/中心对齐
- uint16_t **MCPWM_WorkModeCH2**
MCPWM CH2工作模式: 边沿对齐/中心对齐
- uint16_t **MCPWM_WorkModeCH3**
MCPWM CH3工作模式: 边沿对齐/中心对齐
- uint16_t **MCPWM_WorkModeCH4**
MCPWM CH4工作模式: 边沿对齐/中心对齐
- uint16_t **MCPWM_WorkModeCH5**
MCPWM CH5工作模式: 边沿对齐/中心对齐
- uint16_t **TMR0**
PWM触发ADC事件0, 时间点设置
- uint16_t **TMR1**
PWM触发ADC事件1, 时间点设置
- uint16_t **TMR2**
PWM触发ADC事件2, 时间点设置
- uint16_t **TMR3**

- PWM*触发*ADC*事件3, 时间点设置
- uint16_t [DeadTimeCH012N](#)
 - CH0N*、*CH1N*、*Ch2N*死区时间设置
- uint16_t [DeadTimeCH012P](#)
 - CH0P*、*CH1P*、*Ch2P*死区时间设置
- uint16_t [DeadTimeCH345N](#)
 - CH3N*、*CH4N*、*Ch5N*死区时间设置
- uint16_t [DeadTimeCH345P](#)
 - CH3P*、*CH4P*、*Ch5P*死区时间设置
- uint16_t [EVT0](#)
 - 外部事件触发*MCPWM_CNT0* (时基0)从0开始使能
- uint16_t [EVT1](#)
 - 外部事件触发*MCPWM_CNT1* (时基1)从0开始使能
- uint8_t [CH0N.Polarity_INV](#)
 - CH0N*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH0P.Polarity_INV](#)
 - CH0P*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH1N.Polarity_INV](#)
 - CH1N*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH1P.Polarity_INV](#)
 - CH1P*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH2N.Polarity_INV](#)
 - CH2N*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH2P.Polarity_INV](#)
 - CH2P*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH3N.Polarity_INV](#)
 - CH3N*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH3P.Polarity_INV](#)
 - CH3P*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH4N.Polarity_INV](#)
 - CH4N*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH4P.Polarity_INV](#)
 - CH4P*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH5N.Polarity_INV](#)
 - CH5N*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH5P.Polarity_INV](#)
 - CH5P*输出极性取反, 0:正常输出; 1:取反输出
- uint8_t [CH0P.SCTRLP](#)
 - 当*CH0_PS* = 1时, 输出到*CH0_P*通道的值
- uint8_t [CH0N.SCTRLN](#)
 - 当*CH0_NS* = 1时, 输出到*CH0_N*通道的值
- uint8_t [CH1P.SCTRLP](#)
 - 当*CH1_PS* = 1时, 输出到*CH1_P*通道的值
- uint8_t [CH1N.SCTRLN](#)
 - 当*CH1_NS* = 1时, 输出到*CH1_N*通道的值
- uint8_t [CH2P.SCTRLP](#)
 - 当*CH2_PS* = 1时, 输出到*CH2_P*通道的值
- uint8_t [CH2N.SCTRLN](#)
 - 当*CH2_NS* = 1时, 输出到*CH2_N*通道的值
- uint8_t [CH3P.SCTRLP](#)
 - 当*CH3_PS* = 1时, 输出到*CH3_P*通道的值

- uint8_t [CH3N.SCTRLN](#)
当 $CH3.NS = 1$ 时, 输出到 $CH3.N$ 通道的值
- uint8_t [CH4P.SCTRLP](#)
当 $CH4.PS = 1$ 时, 输出到 $CH4.P$ 通道的值
- uint8_t [CH4N.SCTRLN](#)
当 $CH4.NS = 1$ 时, 输出到 $CH4.N$ 通道的值
- uint8_t [CH5P.SCTRLP](#)
当 $CH5.PS = 1$ 时, 输出到 $CH5.P$ 通道的值
- uint8_t [CH5N.SCTRLN](#)
当 $CH5.NS = 1$ 时, 输出到 $CH5.N$ 通道的值
- uint8_t [CH0.PS](#)
 $CH0.P$ 信号来源, 1: 来自 $CH0.SCTRLP$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH0.NS](#)
 $CH0.N$ 信号来源, 1: 来自 $CH0.SCTRLN$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH1.PS](#)
 $CH1.P$ 信号来源, 1: 来自 $CH1.SCTRLP$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH1.NS](#)
 $CH1.N$ 信号来源, 1: 来自 $CH1.SCTRLN$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH2.PS](#)
 $CH2.P$ 信号来源, 1: 来自 $CH2.SCTRLP$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH2.NS](#)
 $CH2.N$ 信号来源, 1: 来自 $CH2.SCTRLN$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH3.PS](#)
 $CH3.P$ 信号来源, 1: 来自 $CH3.SCTRLP$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH3.NS](#)
 $CH3.N$ 信号来源, 1: 来自 $CH3.SCTRLN$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH4.PS](#)
 $CH4.P$ 信号来源, 1: 来自 $CH4.SCTRLP$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH4.NS](#)
 $CH4.N$ 信号来源, 1: 来自 $CH4.SCTRLN$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH5.PS](#)
 $CH5.P$ 信号来源, 1: 来自 $CH5.SCTRLP$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [CH5.NS](#)
 $CH5.N$ 信号来源, 1: 来自 $CH5.SCTRLN$; 0: 来自 $MCPWM$ 实际运行计数器
- uint8_t [Switch_CH0N.CH0P](#)
交换 $CH0N$, $CH0P$ 信号输出使能开关
- uint8_t [Switch_CH1N.CH1P](#)
交换 $CH1N$, $CH1P$ 信号输出使能开关
- uint8_t [Switch_CH2N.CH2P](#)
交换 $CH2N$, $CH2P$ 信号输出使能开关
- uint8_t [Switch_CH3N.CH3P](#)
交换 $CH3N$, $CH3P$ 信号输出使能开关
- uint8_t [Switch_CH4N.CH4P](#)
交换 $CH2N$, $CH2P$ 信号输出使能开关
- uint8_t [Switch_CH5N.CH5P](#)
交换 $CH3N$, $CH3P$ 信号输出使能开关
- uint8_t [TR0.UP_INTV](#)
 $MCPWM.CNT0$ 更新间隔 $T0/T1$ 事件次数
- uint8_t [TR0.T0.UpdateEN](#)
 $MCPWM.CNT0$ $T0$ 事件更新使能
- uint8_t [TR0.T1.UpdateEN](#)

- MCPWM_CNT0 T1事件更新使能*
- uint8_t [TR0_AEC](#)
 - MCPWM 更新事件是否自动清除Fail0和Fail1事件（恢复CH0CH1CH2输出） 1: 使能*
- uint8_t [TR1_UP_INTV](#)
 - MCPWM_CNT1 更新间隔 T0/T1事件次数*
- uint8_t [TR1_T0_UpdateEN](#)
 - MCPWM_CNT1 T0事件更新使能*
- uint8_t [TR1_T1_UpdateEN](#)
 - MCPWM_CNT1 T1事件更新使能*
- uint8_t [TR1_AEC](#)
 - MCPWM 更新事件是否自动清除Fail2和Fail3事件（恢复CH3CH4CH5输出） 1: 使能*
- uint8_t [FAIL0_INPUT_EN](#)
 - FAIL0 输入功能使能*
- uint8_t [FAIL1_INPUT_EN](#)
 - FAIL1 输入功能使能*
- uint8_t [FAIL0CAP](#)
 - FAIL01事件发生时 将MCPWM0_CNT0值存入MCPWM0_FCNT 使能*
- uint8_t [FAIL0_Signal_Sel](#)
 - FAIL0 信号选择, 比较器0或GPIO*
- uint8_t [FAIL1_Signal_Sel](#)
 - FAIL1 信号选择, 比较器0或GPIO*
- uint8_t [FAIL0_Polarity](#)
 - FAIL0 信号极性设置, 高有效或低有效*
- uint8_t [FAIL1_Polarity](#)
 - FAIL1 信号极性设置, 高有效或低有效*
- uint8_t [HALT_PRT0](#)
 - MCU进入HALT状态, MCPWM(CH0CH1CH2)信号是否正常输出 1正常输出; 0强制输出保护值*
- uint8_t [FAIL2_INPUT_EN](#)
 - FAIL2 输入功能使能*
- uint8_t [FAIL3_INPUT_EN](#)
 - FAIL3 输入功能使能*
- uint8_t [FAIL1CAP](#)
 - FAIL23事件发生时 将MCPWM0_CNT1值存入MCPWM0_FCNT*
- uint8_t [FAIL2_Signal_Sel](#)
 - FAIL2 信号选择, 比较器0或GPIO或CLUOUT0或CLUOUT1*
- uint8_t [FAIL3_Signal_Sel](#)
 - FAIL3 信号选择, 比较器1或GPIO或CLUOUT2或CLUOUT3*
- uint8_t [FAIL2_Polarity](#)
 - FAIL2 信号极性设置, 高有效或低有效*
- uint8_t [FAIL3_Polarity](#)
 - FAIL3 信号极性设置, 高有效或低有效*
- uint8_t [HALT_PRT1](#)
 - MCU进入HALT状态, MCPWM(CH3CH4CH5)信号是否正常输出 1正常输出; 0强制输出保护值*
- uint8_t [CH0P.default.output](#)
 - CH0P MOE为0时或发生FAIL事件时, 默认电平输出*
- uint8_t [CH0N.default.output](#)
 - CH0N MOE为0时或发生FAIL事件时, 默认电平输出*
- uint8_t [CH1P.default.output](#)
 - CH1P MOE为0时或发生FAIL事件时, 默认电平输出*
- uint8_t [CH1N.default.output](#)
 - CH1N MOE为0时或发生FAIL事件时, 默认电平输出*

- uint8_t [CH2P.default_output](#)
CH2P MOE为0时或发生FAIL事件时，默认电平输出
- uint8_t [CH2N.default_output](#)
CH2N MOE为0时或发生FAIL事件时，默认电平输出
- uint8_t [CH3P.default_output](#)
CH3P MOE为0时或发生FAIL事件时，默认电平输出
- uint8_t [CH3N.default_output](#)
CH3N MOE为0时或发生FAIL事件时，默认电平输出
- uint8_t [CH4P.default_output](#)
CH4P MOE为0时或发生FAIL事件时，默认电平输出
- uint8_t [CH4N.default_output](#)
CH4N MOE为0时或发生FAIL事件时，默认电平输出
- uint8_t [CH5P.default_output](#)
CH5P MOE为0时或发生FAIL事件时，默认电平输出
- uint8_t [CH5N.default_output](#)
CH5N MOE为0时或发生FAIL事件时，默认电平输出
- uint8_t [CH0N.FAIL_EN](#)
CH0N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH0P.FAIL_EN](#)
CH0P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH1N.FAIL_EN](#)
CH1N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH1P.FAIL_EN](#)
CH1P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH2N.FAIL_EN](#)
CH2N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH2P.FAIL_EN](#)
CH2P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH3N.FAIL_EN](#)
CH3N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH3P.FAIL_EN](#)
CH3P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH4N.FAIL_EN](#)
CH4N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH4P.FAIL_EN](#)
CH4P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH5N.FAIL_EN](#)
CH5N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [CH5P.FAIL_EN](#)
CH5P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响
- uint8_t [T0_Update0.INT_EN](#)
MCPWM0.CNT0 计数到-MCPWMTH (T0更新事件)中断使能
- uint8_t [T1_Update0.INT_EN](#)
MCPWM0.CNT0 计数到0 (T1更新事件)中断使能
- uint8_t [TMR0.Match0.INT_EN](#)
MCPWM0.CNT0 计数到 MCPWM0.TMR0 产生的计数事件匹配事件中断使能
- uint8_t [TMR1.Match0.INT_EN](#)
MCPWM0.CNT0 计数到 MCPWM0.TMR1 产生的计数事件匹配事件中断使能
- uint8_t [TMR2.Match0.INT_EN](#)
MCPWM0.CNT0 计数到 MCPWM0.TMR2 产生的计数事件匹配事件中断使能
- uint8_t [TMR3.Match0.INT_EN](#)

- MCPWM0.CNT0* 计数到 *MCPWM0.TMR3* 产生的计数事件匹配事件中斷使能
- uint8_t [TH00_Match0.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH00* 产生的计数事件匹配事件中斷使能
- uint8_t [TH01_Match0.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH01* 产生的计数事件匹配事件中斷使能
- uint8_t [TH10_Match0.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH10* 产生的计数事件匹配事件中斷使能
- uint8_t [TH11_Match0.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH11* 产生的计数事件匹配事件中斷使能
- uint8_t [TH20_Match0.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH20* 产生的计数事件匹配事件中斷使能
- uint8_t [TH21_Match0.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH21* 产生的计数事件匹配事件中斷使能
- uint8_t [Update0.INT_EN](#)
 - MCPWM0.CNT0* 计数器0更新中斷使能
- uint8_t [T0_Update1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH0* (T0更新事件)中斷使能
- uint8_t [T1_Update1.INT_EN](#)
 - MCPWM0.CNT0* 计数到0 (T1更新事件)中斷使能
- uint8_t [TMR0_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TMR0* 产生的计数事件匹配事件中斷使能
- uint8_t [TMR1_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TMR1* 产生的计数事件匹配事件中斷使能
- uint8_t [TMR2_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TMR2* 产生的计数事件匹配事件中斷使能
- uint8_t [TMR3_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TMR3* 产生的计数事件匹配事件中斷使能
- uint8_t [TH00_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH00* 产生的计数事件匹配事件中斷使能
- uint8_t [TH01_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH01* 产生的计数事件匹配事件中斷使能
- uint8_t [TH10_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH10* 产生的计数事件匹配事件中斷使能
- uint8_t [TH11_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH11* 产生的计数事件匹配事件中斷使能
- uint8_t [TH20_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH20* 产生的计数事件匹配事件中斷使能
- uint8_t [TH21_Match1.INT_EN](#)
 - MCPWM0.CNT0* 计数到 *MCPWM0.TH21* 产生的计数事件匹配事件中斷使能
- uint8_t [Update1.INT_EN](#)
 - MCPWM0.CNT0* 计数器0更新中斷使能
- uint8_t [FAIL0.INT_EN](#)
- uint8_t [FAIL1.INT_EN](#)
- uint8_t [FAIL2.INT_EN](#)
- uint8_t [FAIL3.INT_EN](#)
- uint8_t [IO_CMP_FLT_CLKDIV](#)
 - GPIO*或*CMP*信号输入滤波时钟设置1-16
- uint8_t [ZCS0.INT_EN](#)
 - ZCS0*事件中斷使能 数字电源应用
- uint8_t [ZCS1.INT_EN](#)
 - ZCS1*事件中斷使能 数字电源应用

- uint8_t **IO0_PPE**
IO0推挽模式使能: 1使能, 0关闭
- uint8_t **IO1_PPE**
IO1推挽模式使能: 1使能, 0关闭
- uint8_t **IO2_PPE**
IO2推挽模式使能: 1使能, 0关闭
- uint8_t **IO3_PPE**
IO3推挽模式使能: 1使能, 0关闭
- uint8_t **IO4_PPE**
IO4推挽模式使能: 1使能, 0关闭
- uint8_t **IO5_PPE**
IO5推挽模式使能: 1使能, 0关闭
- uint8_t **TR1_T1.RE1**
MCPWM_CNT1 计数到0 (T1) 事件 DMA请求1 使能: 1使能, 0关闭
- uint8_t **TR1_T0.RE1**
MCPWM_CNT1 计数到-MCPWM.TH(T0)事件 DMA请求1 使能: 1使能, 0关闭
- uint8_t **TR0_T1.RE1**
MCPWM_CNT0 计数到0 (T1) 事件 DMA请求1 使能: 1使能, 0关闭
- uint8_t **TR0_T0.RE1**
MCPWM_CNT0 计数到-MCPWM.TH(T0)事件 DMA请求1 使能: 1使能, 0关闭
- uint8_t **TMR3.RE1**
MCPWM_CNT01 计数到MCPWM.TMR3 事件 DMA请求1 使能: 1使能, 0关闭
- uint8_t **TMR2.RE1**
MCPWM_CNT01 计数到MCPWM.TMR2 事件 DMA请求1 使能: 1使能, 0关闭
- uint8_t **TMR1.RE1**
MCPWM_CNT01 计数到MCPWM.TMR1 事件 DMA请求1 使能: 1使能, 0关闭
- uint8_t **TMR0.RE1**
MCPWM_CNT01 计数到MCPWM.TMR0 事件 DMA请求1 使能: 1使能, 0关闭
- uint8_t **TR1_T1.RE0**
MCPWM_CNT1 计数到0 (T1) 事件 DMA请求0 使能: 1使能, 0关闭
- uint8_t **TR1_T0.RE0**
MCPWM_CNT1 计数到-MCPWM.TH(T0)事件 DMA请求0 使能: 1使能, 0关闭
- uint8_t **TR0_T1.RE0**
MCPWM_CNT0 计数到0 (T1) 事件 DMA请求0 使能: 1使能, 0关闭
- uint8_t **TR0_T0.RE0**
MCPWM_CNT0 计数到-MCPWM.TH(T0)事件 DMA请求0 使能: 1使能, 0关闭
- uint8_t **TMR3.RE0**
MCPWM_CNT01 计数到MCPWM.TMR3 事件 DMA请求0 使能: 1使能, 0关闭
- uint8_t **TMR2.RE0**
MCPWM_CNT01 计数到MCPWM.TMR2 事件 DMA请求0 使能: 1使能, 0关闭
- uint8_t **TMR1.RE0**
MCPWM_CNT01 计数到MCPWM.TMR1 事件 DMA请求0 使能: 1使能, 0关闭
- uint8_t **TMR0.RE0**
MCPWM_CNT01 计数到MCPWM.TMR0 事件 DMA请求0 使能: 1使能, 0关闭
- uint16_t **STT_HYST**
状态切换延时, 12位无符号数
- uint16_t **ZCS_DELAY**
ZCS状态停留时间, 15位无符号数 数字电源应用
- uint32_t **AUEN**
自动更新使能

4.30.1 详细描述

在文件 [lks32mc07x_mcpwm.h](#) 第 90 行定义.

4.30.2 结构体成员变量说明

4.30.2.1 AUEN

`uint32_t AUEN`

自动更新使能

在文件 [lks32mc07x_mcpwm.h](#) 第 318 行定义.

4.30.2.2 BASE_CNT0_EN

`uint8_t BASE_CNT0_EN`

MCPWM 计数器0使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 113 行定义.

4.30.2.3 BASE_CNT1_EN

`uint8_t BASE_CNT1_EN`

MCPWM 计数器1使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 112 行定义.

4.30.2.4 CH0_NS

`uint8_t CH0_NS`

CH0.N信号来源, 1: 来自CH0.SCTRLN; 0: 来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 173 行定义.

4.30.2.5 CH0_PS

`uint8_t CH0_PS`

CH0.P信号来源, 1: 来自CH0.SCTRLP; 0: 来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 172 行定义.

4.30.2.6 CH0N_default_output

uint8_t CH0N_default_output

CH0N MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 221 行定义.

4.30.2.7 CH0N_FAIL_EN

uint8_t CH0N_FAIL_EN

CH0N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 233 行定义.

4.30.2.8 CH0N.Polarity_INV

uint8_t CH0N.Polarity_INV

CH0N输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 146 行定义.

4.30.2.9 CH0N_SCTRLN

uint8_t CH0N_SCTRLN

当CH0_NS = 1时，输出到CH0.N通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 160 行定义.

4.30.2.10 CH0P_default_output

uint8_t CH0P_default_output

CH0P MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 220 行定义.

4.30.2.11 CH0P_FAIL_EN

uint8_t CH0P_FAIL_EN

CH0P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 234 行定义.

4.30.2.12 CH0P.Polarity_INV

uint8_t CH0P_Polarity_INV

CH0P输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 147 行定义.

4.30.2.13 CH0P.SCTRLP

uint8_t CH0P_SCTRLP

当CH0_PS = 1时，输出到CH0.P通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 159 行定义.

4.30.2.14 CH1.NS

uint8_t CH1_NS

CH1.N信号来源，1：来自CH1.SCTRLN；0：来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 175 行定义.

4.30.2.15 CH1.PS

uint8_t CH1_PS

CH1.P信号来源，1：来自CH1.SCTRLP；0：来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 174 行定义.

4.30.2.16 CH1N.default_output

uint8_t CH1N_default_output

CH1N MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 223 行定义.

4.30.2.17 CH1N.FAIL_EN

uint8_t CH1N_FAIL_EN

CH1N通道FAIL事件使能 1：MOE为0时或发生FAIL事件时，输出默认电平；0：输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 235 行定义.

4.30.2.18 CH1N.Polarity_INV

uint8_t CH1N.Polarity_INV

CH1N输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 148 行定义.

4.30.2.19 CH1N.SCTRLN

uint8_t CH1N.SCTRLN

当CH1_NS = 1时，输出到CH1_N通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 162 行定义.

4.30.2.20 CH1P.default_output

uint8_t CH1P.default_output

CH1P MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 222 行定义.

4.30.2.21 CH1P.FAIL_EN

uint8_t CH1P.FAIL_EN

CH1P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 236 行定义.

4.30.2.22 CH1P.Polarity_INV

uint8_t CH1P.Polarity_INV

CH1P输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 149 行定义.

4.30.2.23 CH1P.SCTRLP

uint8_t CH1P.SCTRLP

当CH1_PS = 1时，输出到CH1_P通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 161 行定义.

4.30.2.24 CH2_NS

uint8_t CH2_NS

CH2_N信号来源, 1: 来自CH2.SCTRLN; 0: 来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 177 行定义.

4.30.2.25 CH2_PS

uint8_t CH2_PS

CH2_P信号来源, 1: 来自CH2.SCTRLP; 0: 来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 176 行定义.

4.30.2.26 CH2N_default_output

uint8_t CH2N_default_output

CH2N MOE为0时或发生FAIL事件时, 默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 225 行定义.

4.30.2.27 CH2N_FAIL_EN

uint8_t CH2N_FAIL_EN

CH2N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时, 输出默认电平; 0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 237 行定义.

4.30.2.28 CH2N_Polarity_INV

uint8_t CH2N_Polarity_INV

CH2N输出极性取反, 0:正常输出; 1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 150 行定义.

4.30.2.29 CH2N_SCTRLN

uint8_t CH2N_SCTRLN

当CH2_NS = 1时, 输出到CH2_N通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 164 行定义.

4.30.2.30 CH2P_default_output

uint8_t CH2P_default_output

CH2P MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 224 行定义.

4.30.2.31 CH2P_FAIL_EN

uint8_t CH2P_FAIL_EN

CH2P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 238 行定义.

4.30.2.32 CH2P_Polarity_INV

uint8_t CH2P_Polarity_INV

CH2P输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 151 行定义.

4.30.2.33 CH2P_SCTRLP

uint8_t CH2P_SCTRLP

当CH2_PS = 1时，输出到CH2.P通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 163 行定义.

4.30.2.34 CH3_NS

uint8_t CH3_NS

CH3_N信号来源，1: 来自CH3.SCTRLN；0: 来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 179 行定义.

4.30.2.35 CH3_PS

uint8_t CH3_PS

CH3.P信号来源，1: 来自CH3.SCTRLP；0: 来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 178 行定义.

4.30.2.36 CH3N.default_output

uint8_t CH3N_default_output

CH3N MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 227 行定义.

4.30.2.37 CH3N.FAIL_EN

uint8_t CH3N_FAIL_EN

CH3N通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 239 行定义.

4.30.2.38 CH3N.Polarity_INV

uint8_t CH3N_Polarity_INV

CH3N输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 152 行定义.

4.30.2.39 CH3N.SCTRLN

uint8_t CH3N_SCTRLN

当CH3_NS = 1时，输出到CH3_N通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 166 行定义.

4.30.2.40 CH3P.default_output

uint8_t CH3P_default_output

CH3P MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 226 行定义.

4.30.2.41 CH3P.FAIL_EN

uint8_t CH3P_FAIL_EN

CH3P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 240 行定义.

4.30.2.42 CH3P.Polarity_INV

uint8_t CH3P.Polarity_INV

CH3P输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 153 行定义.

4.30.2.43 CH3P.SCTRLP

uint8_t CH3P.SCTRLP

当CH3_PS = 1时，输出到CH3.P通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 165 行定义.

4.30.2.44 CH4.NS

uint8_t CH4.NS

CH4.N信号来源，1：来自CH4.SCTRLN；0：来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 181 行定义.

4.30.2.45 CH4.PS

uint8_t CH4.PS

CH4.P信号来源，1：来自CH4.SCTRLP；0：来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 180 行定义.

4.30.2.46 CH4N.default_output

uint8_t CH4N.default_output

CH4N MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 229 行定义.

4.30.2.47 CH4N.FAIL_EN

uint8_t CH4N.FAIL_EN

CH4N通道FAIL事件使能 1：MOE为0时或发生FAIL事件时，输出默认电平；0：输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 241 行定义.

4.30.2.48 CH4N.Polarity_INV

uint8_t CH4N.Polarity_INV

CH4N输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 154 行定义.

4.30.2.49 CH4N.SCTRLN

uint8_t CH4N.SCTRLN

当CH4_NS = 1时，输出到CH4.N通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 168 行定义.

4.30.2.50 CH4P.default_output

uint8_t CH4P.default_output

CH4P MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 228 行定义.

4.30.2.51 CH4P.FAIL_EN

uint8_t CH4P.FAIL_EN

CH4P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 242 行定义.

4.30.2.52 CH4P.Polarity_INV

uint8_t CH4P.Polarity_INV

CH4P输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 155 行定义.

4.30.2.53 CH4P.SCTRLP

uint8_t CH4P.SCTRLP

当CH4_PS = 1时，输出到CH4.P通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 167 行定义.

4.30.2.54 CH5_NS

uint8_t CH5_NS

CH5_N信号来源，1：来自CH5.SCTRLN；0：来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 183 行定义.

4.30.2.55 CH5_PS

uint8_t CH5_PS

CH5_P信号来源，1：来自CH5.SCTRLP；0：来自MCPWM实际运行计数器

在文件 [lks32mc07x_mcpwm.h](#) 第 182 行定义.

4.30.2.56 CH5N_default_output

uint8_t CH5N_default_output

CH5N MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 231 行定义.

4.30.2.57 CH5N_FAIL_EN

uint8_t CH5N_FAIL_EN

CH5N通道FAIL事件使能 1：MOE为0时或发生FAIL事件时，输出默认电平；0：输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 243 行定义.

4.30.2.58 CH5N_Polarity_INV

uint8_t CH5N_Polarity_INV

CH5N输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 156 行定义.

4.30.2.59 CH5N_SCTRLN

uint8_t CH5N_SCTRLN

当CH5_NS = 1时，输出到CH5_N通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 170 行定义.

4.30.2.60 CH5P_default_output

uint8_t CH5P_default_output

CH5P MOE为0时或发生FAIL事件时，默认电平输出

在文件 [lks32mc07x_mcpwm.h](#) 第 230 行定义.

4.30.2.61 CH5P_FAIL_EN

uint8_t CH5P_FAIL_EN

CH5P通道FAIL事件使能 1: MOE为0时或发生FAIL事件时，输出默认电平；0: 输出不受影响

在文件 [lks32mc07x_mcpwm.h](#) 第 244 行定义.

4.30.2.62 CH5P_Polarity_INV

uint8_t CH5P_Polarity_INV

CH5P输出极性取反，0:正常输出；1:取反输出

在文件 [lks32mc07x_mcpwm.h](#) 第 157 行定义.

4.30.2.63 CH5P_SCTRLP

uint8_t CH5P_SCTRLP

当CH5_PS = 1时，输出到CH5_P通道的值

在文件 [lks32mc07x_mcpwm.h](#) 第 169 行定义.

4.30.2.64 CLK_DIV

uint8_t CLK_DIV

MCPWM 分频系数

在文件 [lks32mc07x_mcpwm.h](#) 第 124 行定义.

4.30.2.65 CMP_CTRL_CNT0

uint8_t CMP_CTRL_CNT0

比较器控制计数器0使能

在文件 [lks32mc07x_mcpwm.h](#) 第 110 行定义.

4.30.2.66 CMP_CTRL_CNT1

```
uint8_t CMP_CTRL_CNT1
```

比较器控制计数器1使能

在文件 [lks32mc07x_mcpwm.h](#) 第 109 行定义.

4.30.2.67 DeadTimeCH012N

```
uint16_t DeadTimeCH012N
```

CH0N、CH1N、Ch2N死区时间设置

在文件 [lks32mc07x_mcpwm.h](#) 第 138 行定义.

4.30.2.68 DeadTimeCH012P

```
uint16_t DeadTimeCH012P
```

CH0P、CH1P、Ch2P死区时间设置

在文件 [lks32mc07x_mcpwm.h](#) 第 139 行定义.

4.30.2.69 DeadTimeCH345N

```
uint16_t DeadTimeCH345N
```

CH3N、CH4N、Ch5N死区时间设置

在文件 [lks32mc07x_mcpwm.h](#) 第 140 行定义.

4.30.2.70 DeadTimeCH345P

```
uint16_t DeadTimeCH345P
```

CH3P、CH4P、Ch5P死区时间设置

在文件 [lks32mc07x_mcpwm.h](#) 第 141 行定义.

4.30.2.71 EVT0

```
uint16_t EVT0
```

外部事件触发MCPWM_CNT0 (时基0)从0开始使能

在文件 [lks32mc07x_mcpwm.h](#) 第 143 行定义.

4.30.2.72 EVT1

uint16_t EVT1

外部事件触发MCPWM_CNT1 (时基1)从0开始使能

在文件 [lks32mc07x_mcpwm.h](#) 第 144 行定义.

4.30.2.73 EVT_CNT0_EN

uint8_t EVT_CNT0_EN

MCPWM 计数器0外部触发使能

在文件 [lks32mc07x_mcpwm.h](#) 第 116 行定义.

4.30.2.74 EVT_CNT1_EN

uint8_t EVT_CNT1_EN

*< MCPWM 计数器1外部触发使能

在文件 [lks32mc07x_mcpwm.h](#) 第 115 行定义.

4.30.2.75 FAIL0.INPUT_EN

uint8_t FAIL0.INPUT_EN

FAIL0 输入功能使能

在文件 [lks32mc07x_mcpwm.h](#) 第 202 行定义.

4.30.2.76 FAIL0.INT_EN

uint8_t FAIL0.INT_EN

在文件 [lks32mc07x_mcpwm.h](#) 第 280 行定义.

4.30.2.77 FAIL0.Polarity

uint8_t FAIL0.Polarity

FAIL0 信号极性设置, 高有效或低有效

在文件 [lks32mc07x_mcpwm.h](#) 第 207 行定义.

4.30.2.78 FAIL0.Signal.Sel

```
uint8_t FAIL0.Signal.Sel
```

FAIL0 信号选择，比较器0或GPIO

在文件 [lks32mc07x_mcpwm.h](#) 第 205 行定义.

4.30.2.79 FAIL1.INPUT.EN

```
uint8_t FAIL1.INPUT.EN
```

FAIL1 输入功能使能

在文件 [lks32mc07x_mcpwm.h](#) 第 203 行定义.

4.30.2.80 FAIL1.INT.EN

```
uint8_t FAIL1.INT.EN
```

在文件 [lks32mc07x_mcpwm.h](#) 第 281 行定义.

4.30.2.81 FAIL1.Polarity

```
uint8_t FAIL1.Polarity
```

FAIL1 信号极性设置，高有效或低有效

在文件 [lks32mc07x_mcpwm.h](#) 第 208 行定义.

4.30.2.82 FAIL1.Signal.Sel

```
uint8_t FAIL1.Signal.Sel
```

FAIL1 信号选择，比较器0或GPIO

在文件 [lks32mc07x_mcpwm.h](#) 第 206 行定义.

4.30.2.83 FAIL2.INPUT.EN

```
uint8_t FAIL2.INPUT.EN
```

FAIL2 输入功能使能

在文件 [lks32mc07x_mcpwm.h](#) 第 211 行定义.

4.30.2.84 FAIL2.INT.EN

uint8_t FAIL2.INT.EN

在文件 [lks32mc07x_mcpwm.h](#) 第 282 行定义.

4.30.2.85 FAIL2.Polarity

uint8_t FAIL2.Polarity

FAIL2 信号极性设置, 高有效或低有效

在文件 [lks32mc07x_mcpwm.h](#) 第 216 行定义.

4.30.2.86 FAIL2.Signal.Sel

uint8_t FAIL2.Signal.Sel

FAIL2 信号选择, 比较器0或GPIO或CLUOUT0或CLUOUT1

在文件 [lks32mc07x_mcpwm.h](#) 第 214 行定义.

4.30.2.87 FAIL3.INPUT.EN

uint8_t FAIL3.INPUT.EN

FAIL3 输入功能使能

在文件 [lks32mc07x_mcpwm.h](#) 第 212 行定义.

4.30.2.88 FAIL3.INT.EN

uint8_t FAIL3.INT.EN

在文件 [lks32mc07x_mcpwm.h](#) 第 283 行定义.

4.30.2.89 FAIL3.Polarity

uint8_t FAIL3.Polarity

FAIL3 信号极性设置, 高有效或低有效

在文件 [lks32mc07x_mcpwm.h](#) 第 217 行定义.

4.30.2.90 FAIL3_Signal_Sel

```
uint8_t FAIL3_Signal_Sel
```

FAIL3 信号选择, 比较器1或GPIO或CLUOUT2或CLUOUT3

在文件 [lks32mc07x_mcpwm.h](#) 第 215 行定义.

4.30.2.91 FAIL_0CAP

```
uint8_t FAIL_0CAP
```

FAIL01事件发生时 将MCPWM0_CNT0值存入MCPWM0_FCNT 使能

在文件 [lks32mc07x_mcpwm.h](#) 第 204 行定义.

4.30.2.92 FAIL_1CAP

```
uint8_t FAIL_1CAP
```

FAIL23事件发生时 将MCPWM0_CNT1值存入MCPWM0_FCNT

在文件 [lks32mc07x_mcpwm.h](#) 第 213 行定义.

4.30.2.93 HALT_PRT0

```
uint8_t HALT_PRT0
```

MCU进入HALT状态, MCPWM(CH0CH1CH2)信号是否正常输出 1正常输出; 0强制输出保护值

在文件 [lks32mc07x_mcpwm.h](#) 第 209 行定义.

4.30.2.94 HALT_PRT1

```
uint8_t HALT_PRT1
```

MCU进入HALT状态, MCPWM(CH3CH4CH5)信号是否正常输出 1正常输出; 0强制输出保护值

在文件 [lks32mc07x_mcpwm.h](#) 第 218 行定义.

4.30.2.95 IO0_PPE

```
uint8_t IO0_PPE
```

IO0推挽模式使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 290 行定义.

4.30.2.96 IO1_PPE

uint8_t IO1_PPE

IO1推挽模式使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 291 行定义.

4.30.2.97 IO2_PPE

uint8_t IO2_PPE

IO2推挽模式使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 292 行定义.

4.30.2.98 IO3_PPE

uint8_t IO3_PPE

IO3推挽模式使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 293 行定义.

4.30.2.99 IO4_PPE

uint8_t IO4_PPE

IO4推挽模式使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 294 行定义.

4.30.2.100 IO5_PPE

uint8_t IO5_PPE

IO5推挽模式使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 295 行定义.

4.30.2.101 IO_CMP_FLT_CLKDIV

uint8_t IO_CMP_FLT_CLKDIV

GPIO或CMP信号输入滤波时钟设置1-16

在文件 [lks32mc07x_mcpwm.h](#) 第 285 行定义.

4.30.2.102 MCLK_EN

uint8_t MCLK_EN

MCPWM 时钟使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 123 行定义.

4.30.2.103 MCPWM_WorkModeCH0

uint16_t MCPWM_WorkModeCH0

MCPWM CH0工作模式: 边沿对齐/中心对齐

在文件 [lks32mc07x_mcpwm.h](#) 第 126 行定义.

4.30.2.104 MCPWM_WorkModeCH1

uint16_t MCPWM_WorkModeCH1

MCPWM CH1工作模式: 边沿对齐/中心对齐

在文件 [lks32mc07x_mcpwm.h](#) 第 127 行定义.

4.30.2.105 MCPWM_WorkModeCH2

uint16_t MCPWM_WorkModeCH2

MCPWM CH2工作模式: 边沿对齐/中心对齐

在文件 [lks32mc07x_mcpwm.h](#) 第 128 行定义.

4.30.2.106 MCPWM_WorkModeCH3

uint16_t MCPWM_WorkModeCH3

MCPWM CH3工作模式: 边沿对齐/中心对齐

在文件 [lks32mc07x_mcpwm.h](#) 第 129 行定义.

4.30.2.107 MCPWM_WorkModeCH4

uint16_t MCPWM_WorkModeCH4

MCPWM CH4工作模式: 边沿对齐/中心对齐

在文件 [lks32mc07x_mcpwm.h](#) 第 130 行定义.

4.30.2.108 MCPWM_WorkModeCH5

uint16_t MCPWM_WorkModeCH5

MCPWM CH5工作模式：边沿对齐/中心对齐

在文件 [lks32mc07x_mcpwm.h](#) 第 131 行定义.

4.30.2.109 STT_HYST

uint16_t STT_HYST

状态切换延时，12位无符号数

在文件 [lks32mc07x_mcpwm.h](#) 第 315 行定义.

4.30.2.110 Switch_CH0N_CH0P

uint8_t Switch_CH0N_CH0P

交换CH0N, CH0P信号输出使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 185 行定义.

4.30.2.111 Switch_CH1N_CH1P

uint8_t Switch_CH1N_CH1P

交换CH1N, CH1P信号输出使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 186 行定义.

4.30.2.112 Switch_CH2N_CH2P

uint8_t Switch_CH2N_CH2P

交换CH2N, CH2P信号输出使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 187 行定义.

4.30.2.113 Switch_CH3N_CH3P

uint8_t Switch_CH3N_CH3P

交换CH3N, CH3P信号输出使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 188 行定义.

4.30.2.114 Switch_CH4N_CH4P

```
uint8_t Switch_CH4N_CH4P
```

交换CH2N, CH2P信号输出使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 189 行定义.

4.30.2.115 Switch_CH5N_CH5P

```
uint8_t Switch_CH5N_CH5P
```

交换CH3N, CH3P信号输出使能开关

在文件 [lks32mc07x_mcpwm.h](#) 第 190 行定义.

4.30.2.116 T0_Update0.INT_EN

```
uint8_t T0_Update0.INT_EN
```

MCPWM0.CNT0 计数到-MCPWMTH (T0更新事件)中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 246 行定义.

4.30.2.117 T0_Update1.INT_EN

```
uint8_t T0_Update1.INT_EN
```

MCPWM0.CNT0 计数到-MCPWMTH (T0更新事件)中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 263 行定义.

4.30.2.118 T1_Update0.INT_EN

```
uint8_t T1_Update0.INT_EN
```

MCPWM0.CNT0 计数到0 (T1更新事件)中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 247 行定义.

4.30.2.119 T1_Update1.INT_EN

```
uint8_t T1_Update1.INT_EN
```

MCPWM0.CNT0 计数到0 (T1更新事件)中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 264 行定义.

4.30.2.120 TH0

```
uint16_t TH0
```

MCPWM时基0计数器门限值（PWM周期）

在文件 [lks32mc07x_mcpwm.h](#) 第 92 行定义.

4.30.2.121 TH00

```
uint16_t TH00
```

MCPWM时基0 CH0P比较门限值（PWM占空比）

在文件 [lks32mc07x_mcpwm.h](#) 第 95 行定义.

4.30.2.122 TH00_Match0_INT_EN

```
uint8_t TH00_Match0_INT_EN
```

MCPWM0.CNT0 计数到 MCPWM0.TH00 产生的计数事件匹配事件中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 254 行定义.

4.30.2.123 TH00_Match1_INT_EN

```
uint8_t TH00_Match1_INT_EN
```

MCPWM0.CNT0 计数到 MCPWM0.TH00 产生的计数事件匹配事件中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 271 行定义.

4.30.2.124 TH01

```
uint16_t TH01
```

MCPWM时基0 CH0N比较门限值（PWM占空比）

在文件 [lks32mc07x_mcpwm.h](#) 第 96 行定义.

4.30.2.125 TH01_Match0_INT_EN

```
uint8_t TH01_Match0_INT_EN
```

MCPWM0.CNT0 计数到 MCPWM0.TH01 产生的计数事件匹配事件中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 255 行定义.

4.30.2.126 TH01_Match1_INT_EN

```
uint8_t TH01_Match1_INT_EN
```

MCPWM0_CNT0 计数到 MCPWM0_TH01 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 272 行定义.

4.30.2.127 TH1

```
uint16_t TH1
```

MCPWM时基1计数器门限值 (PWM周期)
在文件 [lks32mc07x_mcpwm.h](#) 第 93 行定义.

4.30.2.128 TH10

```
uint16_t TH10
```

MCPWM时基0 CH1P比较门限值 (PWM占空比)
在文件 [lks32mc07x_mcpwm.h](#) 第 97 行定义.

4.30.2.129 TH10_Match0_INT_EN

```
uint8_t TH10_Match0_INT_EN
```

MCPWM0_CNT0 计数到 MCPWM0_TH10 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 256 行定义.

4.30.2.130 TH10_Match1_INT_EN

```
uint8_t TH10_Match1_INT_EN
```

MCPWM0_CNT0 计数到 MCPWM0_TH10 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 273 行定义.

4.30.2.131 TH11

```
uint16_t TH11
```

MCPWM时基0 CH1N比较门限值 (PWM占空比)
在文件 [lks32mc07x_mcpwm.h](#) 第 98 行定义.

4.30.2.132 TH11_Match0_INT_EN

```
uint8_t TH11_Match0_INT_EN
```

MCPWM0_CNT0 计数到 MCPWM0_TH11 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 257 行定义.

4.30.2.133 TH11_Match1_INT_EN

```
uint8_t TH11_Match1_INT_EN
```

MCPWM0_CNT0 计数到 MCPWM0_TH11 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 274 行定义.

4.30.2.134 TH20

```
uint16_t TH20
```

MCPWM时基0 CH2P比较门限值 (PWM占空比)
在文件 [lks32mc07x_mcpwm.h](#) 第 99 行定义.

4.30.2.135 TH20_Match0_INT_EN

```
uint8_t TH20_Match0_INT_EN
```

MCPWM0_CNT0 计数到 MCPWM0_TH20 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 258 行定义.

4.30.2.136 TH20_Match1_INT_EN

```
uint8_t TH20_Match1_INT_EN
```

MCPWM0_CNT0 计数到 MCPWM0_TH20 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 275 行定义.

4.30.2.137 TH21

```
uint16_t TH21
```

MCPWM时基0 CH2N比较门限值 (PWM占空比)
在文件 [lks32mc07x_mcpwm.h](#) 第 100 行定义.

4.30.2.138 TH21_Match0_INT_EN

```
uint8_t TH21_Match0_INT_EN
```

MCPWM0.CNT0 计数到 MCPWM0.TH21 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 259 行定义.

4.30.2.139 TH21_Match1_INT_EN

```
uint8_t TH21_Match1_INT_EN
```

MCPWM0.CNT0 计数到 MCPWM0.TH21 产生的计数事件匹配事件中断使能
在文件 [lks32mc07x_mcpwm.h](#) 第 276 行定义.

4.30.2.140 TH30

```
uint16_t TH30
```

MCPWM时基1 CH3P比较门限值 (PWM占空比)
在文件 [lks32mc07x_mcpwm.h](#) 第 102 行定义.

4.30.2.141 TH31

```
uint16_t TH31
```

MCPWM时基1 CH3N比较门限值 (PWM占空比)
在文件 [lks32mc07x_mcpwm.h](#) 第 103 行定义.

4.30.2.142 TH40

```
uint16_t TH40
```

MCPWM时基1 CH4P比较门限值 (PWM占空比)
在文件 [lks32mc07x_mcpwm.h](#) 第 104 行定义.

4.30.2.143 TH41

```
uint16_t TH41
```

MCPWM时基1 CH4N比较门限值 (PWM占空比)
在文件 [lks32mc07x_mcpwm.h](#) 第 105 行定义.

4.30.2.144 TH50

uint16_t TH50

MCPWM时基1 CH5P比较门限值（PWM占空比）

在文件 [lks32mc07x_mcpwm.h](#) 第 106 行定义.

4.30.2.145 TH51

uint16_t TH51

MCPWM时基1 CH5N比较门限值（PWM占空比）

在文件 [lks32mc07x_mcpwm.h](#) 第 107 行定义.

4.30.2.146 TMR0

uint16_t TMR0

PWM触发ADC事件0，时间点设置

在文件 [lks32mc07x_mcpwm.h](#) 第 133 行定义.

4.30.2.147 TMR0_Match0_INT_EN

uint8_t TMR0_Match0_INT_EN

MCPWM0.CNT0 计数到 MCPWM0.TMR0 产生的计数事件匹配事件中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 249 行定义.

4.30.2.148 TMR0_Match1_INT_EN

uint8_t TMR0_Match1_INT_EN

MCPWM0.CNT0 计数到 MCPWM0.TMR0 产生的计数事件匹配事件中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 266 行定义.

4.30.2.149 TMR0_RE0

uint8_t TMR0_RE0

MCPWM.CNT01 计数到MCPWM.TMR0 事件 DMA请求0 使能： 1使能， 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 313 行定义.

4.30.2.150 TMR0_RE1

uint8_t TMR0_RE1

MCPWM_CNT01 计数到MCPWM_TMR0 事件 DMA请求1 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 304 行定义.

4.30.2.151 TMR1

uint16_t TMR1

PWM触发ADC事件1, 时间点设置

在文件 [lks32mc07x_mcpwm.h](#) 第 134 行定义.

4.30.2.152 TMR1_Match0_INT_EN

uint8_t TMR1_Match0_INT_EN

MCPWM0_CNT0 计数到 MCPWM0_TMR1 产生的计数事件匹配事件中中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 250 行定义.

4.30.2.153 TMR1_Match1_INT_EN

uint8_t TMR1_Match1_INT_EN

MCPWM0_CNT0 计数到 MCPWM0_TMR1 产生的计数事件匹配事件中中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 267 行定义.

4.30.2.154 TMR1_RE0

uint8_t TMR1_RE0

MCPWM_CNT01 计数到MCPWM_TMR1 事件 DMA请求0 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 312 行定义.

4.30.2.155 TMR1_RE1

uint8_t TMR1_RE1

MCPWM_CNT01 计数到MCPWM_TMR1 事件 DMA请求1 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 303 行定义.

4.30.2.156 TMR2

uint16_t TMR2

PWM触发ADC事件2，时间点设置

在文件 [lks32mc07x_mcpwm.h](#) 第 135 行定义.

4.30.2.157 TMR2.Match0_INT_EN

uint8_t TMR2.Match0_INT_EN

MCPWM0.CNT0 计数到 MCPWM0.TMR2 产生的计数事件匹配事件中中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 251 行定义.

4.30.2.158 TMR2.Match1_INT_EN

uint8_t TMR2.Match1_INT_EN

MCPWM0.CNT0 计数到 MCPWM0.TMR2 产生的计数事件匹配事件中中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 268 行定义.

4.30.2.159 TMR2.RE0

uint8_t TMR2.RE0

MCPWM.CNT01 计数到MCPWM.TMR2 事件 DMA请求0 使能：1使能，0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 311 行定义.

4.30.2.160 TMR2.RE1

uint8_t TMR2.RE1

MCPWM.CNT01 计数到MCPWM.TMR2 事件 DMA请求1 使能：1使能，0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 302 行定义.

4.30.2.161 TMR2.TB

uint8_t TMR2.TB

MCPWM TMR2时基（计数器）选择 0：时基0； 1：时基1

在文件 [lks32mc07x_mcpwm.h](#) 第 119 行定义.

4.30.2.162 TMR3

uint16_t TMR3

PWM触发ADC事件3，时间点设置

在文件 [lks32mc07x_mcpwm.h](#) 第 136 行定义.

4.30.2.163 TMR3.Match0_INT_EN

uint8_t TMR3.Match0_INT_EN

MCPWM0.CNT0 计数到 MCPWM0.TMR3 产生的计数事件匹配事件中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 252 行定义.

4.30.2.164 TMR3.Match1_INT_EN

uint8_t TMR3.Match1_INT_EN

MCPWM0.CNT0 计数到 MCPWM0.TMR3 产生的计数事件匹配事件中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 269 行定义.

4.30.2.165 TMR3.RE0

uint8_t TMR3.RE0

MCPWM.CNT01 计数到MCPWM.TMR3 事件 DMA请求0 使能：1使能，0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 310 行定义.

4.30.2.166 TMR3.RE1

uint8_t TMR3.RE1

MCPWM.CNT01 计数到MCPWM.TMR3 事件 DMA请求1 使能：1使能，0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 301 行定义.

4.30.2.167 TMR3.TB

uint8_t TMR3.TB

MCPWM TMR3时基（计数器）选择 0：时基0；1：时基1

在文件 [lks32mc07x_mcpwm.h](#) 第 118 行定义.

4.30.2.168 TR0_AEC

```
uint8_t TR0_AEC
```

MCPWM 更新事件是否自动清除Fail0和Fail1事件（恢复CH0CH1CH2输出） 1：使能

在文件 [lks32mc07x_mcpwm.h](#) 第 195 行定义.

4.30.2.169 TR0_T0_RE0

```
uint8_t TR0_T0_RE0
```

MCPWM_CNT0 计数到-MCPWM_TH(T0)事件 DMA请求0 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 309 行定义.

4.30.2.170 TR0_T0_RE1

```
uint8_t TR0_T0_RE1
```

MCPWM_CNT0 计数到-MCPWM_TH(T0)事件 DMA请求1 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 300 行定义.

4.30.2.171 TR0_T0_UpdateEN

```
uint8_t TR0_T0_UpdateEN
```

MCPWM_CNT0 T0事件更新使能

在文件 [lks32mc07x_mcpwm.h](#) 第 193 行定义.

4.30.2.172 TR0_T1_RE0

```
uint8_t TR0_T1_RE0
```

MCPWM_CNT0 计数到0 (T1) 事件 DMA请求0 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 308 行定义.

4.30.2.173 TR0_T1_RE1

```
uint8_t TR0_T1_RE1
```

MCPWM_CNT0 计数到0 (T1) 事件 DMA请求1 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 299 行定义.

4.30.2.174 TR0_T1_UpdateEN

```
uint8_t TR0_T1_UpdateEN
```

MCPWM_CNT0 T1事件更新使能

在文件 [lks32mc07x_mcpwm.h](#) 第 194 行定义.

4.30.2.175 TR0_UP_INTV

```
uint8_t TR0_UP_INTV
```

MCPWM_CNT0 更新间隔 T0/T1事件次数

在文件 [lks32mc07x_mcpwm.h](#) 第 192 行定义.

4.30.2.176 TR1_AEC

```
uint8_t TR1_AEC
```

MCPWM 更新事件是否自动清除Fail2和Fail3事件（恢复CH3CH4CH5输出） 1: 使能

在文件 [lks32mc07x_mcpwm.h](#) 第 200 行定义.

4.30.2.177 TR1_T0_RE0

```
uint8_t TR1_T0_RE0
```

MCPWM_CNT1 计数到-MCPWM_TH(T0)事件 DMA请求0 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 307 行定义.

4.30.2.178 TR1_T0_RE1

```
uint8_t TR1_T0_RE1
```

MCPWM_CNT1 计数到-MCPWM_TH(T0)事件 DMA请求1 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 298 行定义.

4.30.2.179 TR1_T0_UpdateEN

```
uint8_t TR1_T0_UpdateEN
```

MCPWM_CNT1 T0事件更新使能

在文件 [lks32mc07x_mcpwm.h](#) 第 198 行定义.

4.30.2.180 TR1_T1_RE0

```
uint8_t TR1_T1_RE0
```

MCPWM_CNT1 计数到0 (T1) 事件 DMA请求0 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 306 行定义.

4.30.2.181 TR1_T1_RE1

```
uint8_t TR1_T1_RE1
```

MCPWM_CNT1 计数到0 (T1) 事件 DMA请求1 使能: 1使能, 0关闭

在文件 [lks32mc07x_mcpwm.h](#) 第 297 行定义.

4.30.2.182 TR1_T1_UpdateEN

```
uint8_t TR1_T1_UpdateEN
```

MCPWM_CNT1 T1事件更新使能

在文件 [lks32mc07x_mcpwm.h](#) 第 199 行定义.

4.30.2.183 TR1_UP_INTV

```
uint8_t TR1_UP_INTV
```

MCPWM_CNT1 更新间隔 T0/T1事件次数

在文件 [lks32mc07x_mcpwm.h](#) 第 197 行定义.

4.30.2.184 Update0_INT_EN

```
uint8_t Update0_INT_EN
```

MCPWM0_CNT0 计数器0更新中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 261 行定义.

4.30.2.185 Update1_INT_EN

```
uint8_t Update1_INT_EN
```

MCPWM0_CNT0 计数器0更新中断使能

在文件 [lks32mc07x_mcpwm.h](#) 第 278 行定义.

4.30.2.186 ZCS0_INT_EN

```
uint8_t ZCS0_INT_EN
```

ZCS0事件中断使能 数字电源应用

在文件 [lks32mc07x_mcpwm.h](#) 第 287 行定义.

4.30.2.187 ZCS1_INT_EN

```
uint8_t ZCS1_INT_EN
```

ZCS1事件中断使能 数字电源应用

在文件 [lks32mc07x_mcpwm.h](#) 第 288 行定义.

4.30.2.188 ZCS_DELAY

```
uint16_t ZCS_DELAY
```

ZCS状态停留时间, 15位无符号数 数字电源应用

在文件 [lks32mc07x_mcpwm.h](#) 第 316 行定义.

4.30.2.189 ZCS_EN

```
uint8_t ZCS_EN
```

使能ZCS事件检测 高有效

在文件 [lks32mc07x_mcpwm.h](#) 第 121 行定义.

4.31 MCPWM_TypeDef结构体 参考

```
#include <lks32mc07x_mcpwm.h>
```

成员变量

- `__IO u32 TH00`
偏移地址: `0x00 MCPWM CH0.P` 比较门限值寄存器
- `__IO u32 TH01`
偏移地址: `0x04 MCPWM CH0.N` 比较门限值寄存器
- `__IO u32 TH10`
偏移地址: `0x08 MCPWM CH1.P` 比较门限值寄存器
- `__IO u32 TH11`
偏移地址: `0x0C MCPWM CH1.N` 比较门限值寄存器
- `__IO u32 TH20`
偏移地址: `0x10 MCPWM CH2.P` 比较门限值寄存器
- `__IO u32 TH21`
偏移地址: `0x14 MCPWM CH2.N` 比较门限值寄存器
- `__IO u32 TH30`
偏移地址: `0x18 MCPWM CH3.P` 比较门限值寄存器
- `__IO u32 TH31`
偏移地址: `0x1C MCPWM CH3.N` 比较门限值寄存器
- `__IO u32 TH40`
偏移地址: `0x20 MCPWM CH4.P` 比较门限值寄存器
- `__IO u32 TH41`
偏移地址: `0x24 MCPWM CH4.N` 比较门限值寄存器
- `__IO u32 TH50`
偏移地址: `0x28 MCPWM CH5.P` 比较门限值寄存器
- `__IO u32 TH51`
偏移地址: `0x2C MCPWM CH5.N` 比较门限值寄存器
- `__IO u32 TMR0`
偏移地址: `0x30 ADC` 采样定时器比较门限 0 寄存器
- `__IO u32 TMR1`
偏移地址: `0x34 ADC` 采样定时器比较门限 1 寄存器
- `__IO u32 TMR2`
偏移地址: `0x38 ADC` 采样定时器比较门限 2 寄存器
- `__IO u32 TMR3`
偏移地址: `0x3C ADC` 采样定时器比较门限 3 寄存器
- `__IO u32 TH0`
偏移地址: `0x40 MCPWM` 时基 0 门限值寄存器
- `__IO u32 TH1`
偏移地址: `0x44 MCPWM` 时基 1 门限值寄存器
- `__IO u32 CNT0`
偏移地址: `0x48 MCPWM` 时基 0 计数器寄存器
- `__IO u32 CNT1`
偏移地址: `0x4C MCPWM` 时基 1 计数器寄存器
- `__IO u32 UPDATE`
偏移地址: `0x50 MCPWM` 加载控制寄存器
- `__IO u32 FCNT`
偏移地址: `0x54 MCPWM FAIL` 时刻 CNT 值
- `__IO u32 EVT0`
偏移地址: `0x58 MCPWM` 时基 0 外部触发
- `__IO u32 EVT1`
偏移地址: `0x5C MCPWM` 时基 1 外部触发
- `__IO u32 DTH00`

- 偏移地址: 0x60 MCPWM CH0/1/2 N 通道死区宽度控制寄存器
- `__IO u32 DTH01`
- 偏移地址: 0x64 MCPWM CH0/1/2 P 通道死区宽度控制寄存器
- `__IO u32 DTH10`
- 偏移地址: 0x68 MCPWM CH3/4/5 N 通道死区宽度控制寄存器
- `__IO u32 DTH11`
- 偏移地址: 0x6C MCPWM CH3/4/5 P 通道死区宽度控制寄存器
- `__IO u32 FLT`
- 偏移地址: 0x70 MCPWM 滤波时钟分频寄存器
- `__IO u32 SDCFG`
- 偏移地址: 0x74 MCPWM 加载配置寄存器
- `__IO u32 AUEN`
- 偏移地址: 0x78 MCPWM 自动更新使能寄存器
- `__IO u32 TCLK`
- 偏移地址: 0x7C MCPWM 时钟分频控制寄存器
- `__IO u32 IE0`
- 偏移地址: 0x80 MCPWM 时基 0 中断控制寄存器
- `__IO u32 IF0`
- 偏移地址: 0x84 MCPWM 时基 0 中断标志位寄存器
- `__IO u32 IE1`
- 偏移地址: 0x88 MCPWM 中断控制寄存器
- `__IO u32 IF1`
- 偏移地址: 0x8C MCPWM 中断标志位寄存器
- `__IO u32 EIE`
- 偏移地址: 0x90 MCPWM 异常中断控制寄存器
- `__IO u32 EIF`
- 偏移地址: 0x94 MCPWM 异常中断标志位寄存器
- `__IO u32 RE`
- 偏移地址: 0x98 MCPWM DMA 请求控制寄存器
- `__IO u32 PP`
- 偏移地址: 0x9C MCPWM 推挽模式使能寄存器
- `__IO u32 IO01`
- 偏移地址: 0xA0 MCPWM CH0 CH1 IO 控制寄存器
- `__IO u32 IO23`
- 偏移地址: 0xA4 MCPWM CH2 CH3 IO 控制寄存器
- `__IO u32 IO45`
- 偏移地址: 0xA8 MCPWM CH4 CH5 IO 控制寄存器
- `__IO u32 null`
- 偏移地址: 0xAC 地址对齐
- `__IO u32 FAIL012`
- 偏移地址: 0xB0 MCPWM CH0 CH1 CH2 短路控制寄存器
- `__IO u32 FAIL345`
- 偏移地址: 0xB4 MCPWM CH3 CH4 CH5 短路控制寄存器
- `__IO u32 CH_DEF`
- 偏移地址: 0xB8 MCPWM 短路保护通道输出默认值
- `__IO u32 CH_MSK`
- 偏移地址: 0xBC MCPWM 通道屏蔽寄存器
- `__IO u32 PRT`
- 偏移地址: 0xC0 MCPWM 保护寄存器
- `__IO u32 STT_HYST`
- 偏移地址: 0xC8 MCPWM 状态停留延时寄存器
- `__IO u32 ZCS_DELAY`
- 偏移地址: 0xCC MCPWM ZCS 状态延时寄存器

4.31.1 详细描述

在文件 [lks32mc07x_mcpwm.h](#) 第 21 行定义.

4.31.2 结构体成员变量说明

4.31.2.1 AUEN

```
--IO u32 AUEN
```

偏移地址: 0x78 MCPWM 自动更新使能寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 60 行定义.

4.31.2.2 CH_DEF

```
--IO u32 CH_DEF
```

偏移地址: 0xB8 MCPWM 短路保护通道输出默认值

在文件 [lks32mc07x_mcpwm.h](#) 第 80 行定义.

4.31.2.3 CH_MSK

```
--IO u32 CH_MSK
```

偏移地址: 0xBC MCPWM 通道屏蔽寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 81 行定义.

4.31.2.4 CNT0

```
--IO u32 CNT0
```

偏移地址: 0x48 MCPWM 时基 0 计数器寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 45 行定义.

4.31.2.5 CNT1

```
--IO u32 CNT1
```

偏移地址: 0x4C MCPWM 时基 1 计数器寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 46 行定义.

4.31.2.6 DTH00

```
--IO u32 DTH00
```

偏移地址: 0x60 MCPWM CH0/1/2 N 通道死区宽度控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 53 行定义.

4.31.2.7 DTH01

```
--IO u32 DTH01
```

偏移地址: 0x64 MCPWM CH0/1/2 P 通道死区宽度控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 54 行定义.

4.31.2.8 DTH10

```
--IO u32 DTH10
```

偏移地址: 0x68 MCPWM CH3/4/5 N 通道死区宽度控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 55 行定义.

4.31.2.9 DTH11

```
--IO u32 DTH11
```

偏移地址: 0x6C MCPWM CH3/4/5 P 通道死区宽度控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 56 行定义.

4.31.2.10 EIE

```
--IO u32 EIE
```

偏移地址: 0x90 MCPWM 异常中断控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 68 行定义.

4.31.2.11 EIF

```
--IO u32 EIF
```

偏移地址: 0x94 MCPWM 异常中断标志位寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 69 行定义.

4.31.2.12 EVT0

--IO u32 EVT0

偏移地址: 0x58 MCPWM 时基 0 外部触发

在文件 [lks32mc07x_mcpwm.h](#) 第 50 行定义.

4.31.2.13 EVT1

--IO u32 EVT1

偏移地址: 0x5C MCPWM 时基 1 外部触发

在文件 [lks32mc07x_mcpwm.h](#) 第 51 行定义.

4.31.2.14 FAIL012

--IO u32 FAIL012

偏移地址: 0xB0 MCPWM CH0 CH1 CH2 短路控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 78 行定义.

4.31.2.15 FAIL345

--IO u32 FAIL345

偏移地址: 0xB4 MCPWM CH3 CH4 CH5 短路控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 79 行定义.

4.31.2.16 FCNT

--IO u32 FCNT

偏移地址: 0x54 MCPWM FAIL 时刻 CNT 值

在文件 [lks32mc07x_mcpwm.h](#) 第 49 行定义.

4.31.2.17 FLT

--IO u32 FLT

偏移地址: 0x70 MCPWM 滤波时钟分频寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 58 行定义.

4.31.2.18 IE0

```
--IO u32 IE0
```

偏移地址: 0x80 MCPWM 时基 0 中断控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 63 行定义.

4.31.2.19 IE1

```
--IO u32 IE1
```

偏移地址: 0x88 MCPWM 中断控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 65 行定义.

4.31.2.20 IF0

```
--IO u32 IF0
```

偏移地址: 0x84 MCPWM 时基 0 中断标志位寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 64 行定义.

4.31.2.21 IF1

```
--IO u32 IF1
```

偏移地址: 0x8C MCPWM 中断标志位寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 66 行定义.

4.31.2.22 IO01

```
--IO u32 IO01
```

偏移地址: 0xA0 MCPWM CH0 CH1 IO 控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 73 行定义.

4.31.2.23 IO23

```
--IO u32 IO23
```

偏移地址: 0xA4 MCPWM CH2 CH3 IO 控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 74 行定义.

4.31.2.24 IO45

--IO u32 IO45

偏移地址: 0xA8 MCPWM CH4 CH5 IO 控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 75 行定义.

4.31.2.25 null

--IO u32 null

偏移地址: 0xAC 地址对齐

在文件 [lks32mc07x_mcpwm.h](#) 第 76 行定义.

4.31.2.26 PP

--IO u32 PP

偏移地址: 0x9C MCPWM 推挽模式使能寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 71 行定义.

4.31.2.27 PRT

--IO u32 PRT

偏移地址: 0xC0 MCPWM 保护寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 83 行定义.

4.31.2.28 RE

--IO u32 RE

偏移地址: 0x98 MCPWM DMA 请求控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 70 行定义.

4.31.2.29 SDCFG

--IO u32 SDCFG

偏移地址: 0x74 MCPWM 加载配置寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 59 行定义.

4.31.2.30 STT_HYST

```
--IO u32 STT_HYST
```

偏移地址: 0xC8 MCPWM 状态停留延时寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 84 行定义.

4.31.2.31 TCLK

```
--IO u32 TCLK
```

偏移地址: 0x7C MCPWM 时钟分频控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 61 行定义.

4.31.2.32 TH0

```
--IO u32 TH0
```

偏移地址: 0x40 MCPWM 时基 0 门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 43 行定义.

4.31.2.33 TH00

```
--IO u32 TH00
```

偏移地址: 0x00 MCPWM CH0_P 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 23 行定义.

4.31.2.34 TH01

```
--IO u32 TH01
```

偏移地址: 0x04 MCPWM CH0_N 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 24 行定义.

4.31.2.35 TH1

```
--IO u32 TH1
```

偏移地址: 0x44 MCPWM 时基 1 门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 44 行定义.

4.31.2.36 TH10

```
--IO u32 TH10
```

偏移地址: 0x08 MCPWM CH1_P 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 25 行定义.

4.31.2.37 TH11

```
--IO u32 TH11
```

偏移地址: 0x0C MCPWM CH1_N 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 26 行定义.

4.31.2.38 TH20

```
--IO u32 TH20
```

偏移地址: 0x10 MCPWM CH2_P 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 28 行定义.

4.31.2.39 TH21

```
--IO u32 TH21
```

偏移地址: 0x14 MCPWM CH2_N 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 29 行定义.

4.31.2.40 TH30

```
--IO u32 TH30
```

偏移地址: 0x18 MCPWM CH3_P 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 30 行定义.

4.31.2.41 TH31

```
--IO u32 TH31
```

偏移地址: 0x1C MCPWM CH3_N 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 31 行定义.

4.31.2.42 TH40

```
--IO u32 TH40
```

偏移地址: 0x20 MCPWM CH4_P 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 33 行定义.

4.31.2.43 TH41

```
--IO u32 TH41
```

偏移地址: 0x24 MCPWM CH4_N 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 34 行定义.

4.31.2.44 TH50

```
--IO u32 TH50
```

偏移地址: 0x28 MCPWM CH5_P 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 35 行定义.

4.31.2.45 TH51

```
--IO u32 TH51
```

偏移地址: 0x2C MCPWM CH5_N 比较门限值寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 36 行定义.

4.31.2.46 TMR0

```
--IO u32 TMR0
```

偏移地址: 0x30 ADC 采样定时器比较门限 0 寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 38 行定义.

4.31.2.47 TMR1

```
--IO u32 TMR1
```

偏移地址: 0x34 ADC 采样定时器比较门限 1 寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 39 行定义.

4.31.2.48 TMR2

```
--IO u32 TMR2
```

偏移地址: 0x38 ADC 采样定时器比较门限 2 寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 40 行定义.

4.31.2.49 TMR3

```
--IO u32 TMR3
```

偏移地址: 0x3C ADC 采样定时器比较门限 3 寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 41 行定义.

4.31.2.50 UPDATE

```
--IO u32 UPDATE
```

偏移地址: 0x50 MCPWM 加载控制寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 48 行定义.

4.31.2.51 ZCS_DELAY

```
--IO u32 ZCS_DELAY
```

偏移地址: 0xCC MCPWM ZCS 状态延时寄存器

在文件 [lks32mc07x_mcpwm.h](#) 第 85 行定义.

4.32 OPA_InitTypeDef结构体 参考

OPA初始化结构体句柄, 初始化时请定义该句柄, 并用其它地址来传参

```
#include <lks32mc07x_opa.h>
```

成员变量

- [uint32_t OPA_IT](#)
运放偏置电流调节
- [uint32_t OPA_Gain](#)
运放闭环增益
- [uint32_t OPA_CLEna](#)
运放闭环使能:*Enable,Disable*

4.32.1 详细描述

OPA初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参
在文件 [lks32mc07x_opa.h](#) 第 31 行定义.

4.32.2 结构体成员变量说明

4.32.2.1 OPA_CLEna

uint32_t OPA_CLEna

运放闭环使能:Enable,Disable

在文件 [lks32mc07x_opa.h](#) 第 49 行定义.

4.32.2.2 OPA_Gain

uint32_t OPA_Gain

运放闭环增益

参见

[PGA_GAIN_32](#)

[PGA_GAIN_16](#)

[PGA_GAIN_8](#)

[PGA_GAIN_4](#) 运放闭环增益

在文件 [lks32mc07x_opa.h](#) 第 48 行定义.

4.32.2.3 OPA_IT

uint32_t OPA_IT

运放偏置电流调节

参见

[PGA_IT_1](#)

[PGA_IT_1P2](#)

[PGA_IT_1P5](#)

[PGA_IT_DISABLE](#) 运放偏置电流调节

在文件 [lks32mc07x_opa.h](#) 第 40 行定义.

4.33 SIF_InitTypeDef结构体 参考

```
#include <lks32mc07x-sif.h>
```

成员变量

- uint8_t SIF_DONE
- uint8_t SIF_SYNC
- uint8_t SIF_SYNC_PULSE
- uint8_t SIF_DONE_VLD
- uint8_t SIF_SYNC_VLD
- uint8_t SIF_RATIO
- uint8_t SIF_MSB
- uint8_t SIF_EN
- uint16_t SIF_TOSC
- uint16_t SIF_TSTH1
- uint8_t SIF_TDTH1
- uint8_t SIF_IRQ_IF
- uint8_t SIF_DMA_EN
- uint8_t SIF_IRQ_EN

4.33.1 详细描述

在文件 [lks32mc07x-sif.h](#) 第 19 行定义.

4.33.2 结构体成员变量说明

4.33.2.1 SIF_DMA_EN

```
uint8_t SIF_DMA_EN
```

在文件 [lks32mc07x-sif.h](#) 第 33 行定义.

4.33.2.2 SIF_DONE

```
uint8_t SIF_DONE
```

在文件 [lks32mc07x-sif.h](#) 第 21 行定义.

4.33.2.3 SIF_DONE_VLD

```
uint8_t SIF_DONE_VLD
```

在文件 [lks32mc07x-sif.h](#) 第 24 行定义.

4.33.2.4 SIF_EN

uint8_t SIF_EN

在文件 [lks32mc07x_sif.h](#) 第 28 行定义.

4.33.2.5 SIF_IRQ_EN

uint8_t SIF_IRQ_EN

在文件 [lks32mc07x_sif.h](#) 第 34 行定义.

4.33.2.6 SIF_IRQ_IF

uint8_t SIF_IRQ_IF

在文件 [lks32mc07x_sif.h](#) 第 32 行定义.

4.33.2.7 SIF_MSB

uint8_t SIF_MSB

在文件 [lks32mc07x_sif.h](#) 第 27 行定义.

4.33.2.8 SIF_RATIO

uint8_t SIF_RATIO

在文件 [lks32mc07x_sif.h](#) 第 26 行定义.

4.33.2.9 SIF_SYNC

uint8_t SIF_SYNC

在文件 [lks32mc07x_sif.h](#) 第 22 行定义.

4.33.2.10 SIF_SYNC_PULSE

uint8_t SIF_SYNC_PULSE

在文件 [lks32mc07x_sif.h](#) 第 23 行定义.

4.33.2.11 SIF_SYNC_VLD

uint8_t SIF_SYNC_VLD

在文件 [lks32mc07x_sif.h](#) 第 25 行定义.

4.33.2.12 SIF_TDTH1

```
uint8_t SIF_TDTH1
```

在文件 [lks32mc07x_sif.h](#) 第 31 行定义.

4.33.2.13 SIF_TOSC

```
uint16_t SIF_TOSC
```

在文件 [lks32mc07x_sif.h](#) 第 29 行定义.

4.33.2.14 SIF_TSTH1

```
uint16_t SIF_TSTH1
```

在文件 [lks32mc07x_sif.h](#) 第 30 行定义.

4.34 SIF_RegTypeDef结构体 参考

```
#include <lks32mc07x_sif.h>
```

成员变量

- __IO uint32_t [SIFx_CFG](#)
- __IO uint32_t [SIFx_TOSC](#)
- __IO uint32_t [SIFx_TSTH1](#)
- __IO uint32_t [SIFx_TDTH1](#)
- __IO uint32_t [SIFx_IRQ](#)

4.34.1 详细描述

在文件 [lks32mc07x_sif.h](#) 第 38 行定义.

4.34.2 结构体成员变量说明

4.34.2.1 SIFx_CFG

```
__IO uint32_t SIFx_CFG
```

在文件 [lks32mc07x_sif.h](#) 第 40 行定义.

4.34.2.2 SIFx_IRQ

```
--IO uint32_t SIFx_IRQ
```

在文件 [lks32mc07x_sif.h](#) 第 44 行定义.

4.34.2.3 SIFx_TDTH1

```
--IO uint32_t SIFx_TDTH1
```

在文件 [lks32mc07x_sif.h](#) 第 43 行定义.

4.34.2.4 SIFx_TOSC

```
--IO uint32_t SIFx_TOSC
```

在文件 [lks32mc07x_sif.h](#) 第 41 行定义.

4.34.2.5 SIFx_TSTH1

```
--IO uint32_t SIFx_TSTH1
```

在文件 [lks32mc07x_sif.h](#) 第 42 行定义.

4.35 SPI_InitTypeDef结构体 参考

```
#include <lks32mc07x_spi.h>
```

成员变量

- [SPI_Duplex Duplex](#)
数据传输模式选择, 全双工`0x00`, 半双工仅发送`0x10`, 半双工仅接收`0x11` 模式选择
- `uint8_t CS`
从设备下片选信号来源: `0`, 恒为`0`; `1`, 来源于主设备
- [SPI_Mode Mode](#)
主从模式选择: `0`, 从模式; `1`, 主模式
- `uint8_t CPHA`
相位选择: `0`对应时钟从默认电平到第一个跳变边沿为采样数据时刻; `1`对应发送数据时刻
- `uint8_t CPOL`
极性选择: `0`对应默认时钟电平为低电平; `1`对应默认时钟为高电平
- `uint8_t ENDIAN`
传输顺序选择: `0`对应`MSB`高位先传输; `1`对应`LSB`低位先传输
- `uint8_t EN`
模块使能: `1` 开启模块
- `uint8_t IRQEna`
中断使能
- `uint8_t Trig`
传输触发方式选择 `1`: 外部触发; `0`内部自动执行 (仅主模式有效)
- `uint8_t TRANS.MODE`
数据搬移方式选择 `0`: `DMA`搬运; `1``MCU`搬运
- `uint16_t BaudRate`
波特率设置 取值范围`3-4095`
- `uint8_t ByteLength`
数据长度

4.35.1 详细描述

在文件 [lks32mc07x_spi.h](#) 第 61 行定义.

4.35.2 结构体成员变量说明

4.35.2.1 BaudRate

```
uint16_t BaudRate
```

波特率设置 取值范围3-4095

在文件 [lks32mc07x_spi.h](#) 第 75 行定义.

4.35.2.2 ByteLength

```
uint8_t ByteLength
```

数据长度

在文件 [lks32mc07x_spi.h](#) 第 77 行定义.

4.35.2.3 CPHA

```
uint8_t CPHA
```

相位选择: 0对应时钟从默认电平到第一个跳变边沿为采样数据时刻; 1对应发送数据时刻

在文件 [lks32mc07x_spi.h](#) 第 66 行定义.

4.35.2.4 CPOL

```
uint8_t CPOL
```

极性选择: 0对应默认时钟电平为低电平; 1对应默认时钟为高电平

在文件 [lks32mc07x_spi.h](#) 第 67 行定义.

4.35.2.5 CS

```
uint8_t CS
```

从设备下片选信号来源: 0, 恒为0; 1, 来源于主设备

在文件 [lks32mc07x_spi.h](#) 第 64 行定义.

4.35.2.6 Duplex

`SPI_Duplex` Duplex

数据传输模式选择，全双工0x00, 半双工仅发送0x10, 半双工仅接收0x11 模式选择

在文件 `lks32mc07x_spi.h` 第 63 行定义.

4.35.2.7 EN

`uint8_t` EN

模块使能: 1 开启模块

在文件 `lks32mc07x_spi.h` 第 69 行定义.

4.35.2.8 ENDIAN

`uint8_t` ENDIAN

传输顺序选择: 0对应MSB高位先传输; 1对应LSB低位先传输

在文件 `lks32mc07x_spi.h` 第 68 行定义.

4.35.2.9 IRQEna

`uint8_t` IRQEna

中断使能

在文件 `lks32mc07x_spi.h` 第 71 行定义.

4.35.2.10 Mode

`SPI_Mode` Mode

主从模式选择: 0, 从模式; 1, 主模式

在文件 `lks32mc07x_spi.h` 第 65 行定义.

4.35.2.11 TRANS_MODE

`uint8_t` TRANS_MODE

数据搬运方式选择 0: DMA搬运; 1MCU搬运

在文件 `lks32mc07x_spi.h` 第 74 行定义.

4.35.2.12 Trig

uint8_t Trig

传输触发方式选择 1: 外部触发; 0内部自动执行 (仅主模式有效)

在文件 [lks32mc07x_spi.h](#) 第 72 行定义.

4.36 SPI_TypeDef结构体 参考

SPI寄存器结构体句柄

```
#include <lks32mc07x_spi.h>
```

成员变量

- __IO uint32_t [CFG](#)
SPI控制寄存器
- __IO uint32_t [IE](#)
SPI中断寄存器
- __IO uint32_t [BAUD](#)
SPI波特率控制寄存器
- __IO uint32_t [TX_DATA](#)
SPI发送数据寄存器
- __IO uint32_t [RX_DATA](#)
SPI接收数据寄存器
- __IO uint32_t [SIZE](#)
SPI传输数据长度寄存器

4.36.1 详细描述

SPI寄存器结构体句柄

在文件 [lks32mc07x_spi.h](#) 第 49 行定义.

4.36.2 结构体成员变量说明

4.36.2.1 BAUD

__IO uint32_t BAUD

SPI波特率控制寄存器

在文件 [lks32mc07x_spi.h](#) 第 53 行定义.

4.36.2.2 CFG

```
--IO uint32_t CFG
```

SPI控制寄存器

在文件 [lks32mc07x_spi.h](#) 第 51 行定义.

4.36.2.3 IE

```
--IO uint32_t IE
```

SPI中断寄存器

在文件 [lks32mc07x_spi.h](#) 第 52 行定义.

4.36.2.4 RX_DATA

```
--IO uint32_t RX_DATA
```

SPI接收数据寄存器

在文件 [lks32mc07x_spi.h](#) 第 55 行定义.

4.36.2.5 SIZE

```
--IO uint32_t SIZE
```

SPI传输数据长度寄存器

在文件 [lks32mc07x_spi.h](#) 第 56 行定义.

4.36.2.6 TX_DATA

```
--IO uint32_t TX_DATA
```

SPI发送数据寄存器

在文件 [lks32mc07x_spi.h](#) 第 54 行定义.

4.37 stru_DMA_REN联合体 参考

DAM搬运使能源定义

```
#include <lks32mc07x_dma.h>
```

成员变量

- u32 **R**
- struct {
 - u16 **ADC0_EN**: 1
 - u16 **ADC1_EN**: 1
 - u16 : 1
 - u16 **DSP_EN**: 1
 - u16 **TIMER0_EN**: 1
 - u16 **TIMER1_EN**: 1
 - u16 **TIMER2_EN**: 1
 - u16 **TIMER3_EN**: 1
 - u16 : 1
 - u16 : 1
 - u16 **SPI_RX_EN**: 1
 - u16 **SPI_TX_EN**: 1
 - u16 **MCPWM_RE0_EN**: 1
 - u16 **MCPWM_RE1_EN**: 1
 - u16 **I2C0_RX_EN**: 1
 - u16 **I2C0_TX_EN**: 1
 - u16 **UART0_RX_EN**: 1
 - u16 **UART0_TX_EN**: 1
 - u16 **UART1_RX_EN**: 1
 - u16 **UART1_TX_EN**: 1
 - u16 : 1
 - u16 : 1
 - u16 : 1
 - u16 : 1
 - u16 **HALL0_EN**: 1
 - u16 **SIF_EN**: 1
 - u16 **CMP_EN**: 1
 - u16 : 1
 - u16 : 1
 - u16 **GPIO_EN**: 1
 - u16 : 1
 - u16 **SW_TRIG_EN**: 1
- } **B**

4.37.1 详细描述

DAM搬运使能源定义

在文件 [lks32mc07x_dma.h](#) 第 54 行定义.

4.37.2 结构体成员变量说明

4.37.2.1 __pad0__

u16 __pad0__

在文件 [lks32mc07x_dma.h](#) 第 61 行定义.

4.37.2.2 __pad1__

u16 __pad1__

在文件 [lks32mc07x_dma.h](#) 第 67 行定义.

4.37.2.3 __pad2__

u16 __pad2__

在文件 [lks32mc07x_dma.h](#) 第 68 行定义.

4.37.2.4 __pad3__

u16 __pad3__

在文件 [lks32mc07x_dma.h](#) 第 79 行定义.

4.37.2.5 __pad4__

u16 __pad4__

在文件 [lks32mc07x_dma.h](#) 第 80 行定义.

4.37.2.6 __pad5__

u16 __pad5__

在文件 [lks32mc07x_dma.h](#) 第 81 行定义.

4.37.2.7 __pad6__

u16 __pad6__

在文件 [lks32mc07x_dma.h](#) 第 82 行定义.

4.37.2.8 __pad7__

u16 __pad7__

在文件 [lks32mc07x_dma.h](#) 第 86 行定义.

4.37.2.9 __pad8__

u16 __pad8__

在文件 [lks32mc07x_dma.h](#) 第 87 行定义.

4.37.2.10 __pad9__

u16 __pad9__

在文件 [lks32mc07x_dma.h](#) 第 89 行定义.

4.37.2.11 ADC0_EN

u16 ADC0_EN

在文件 [lks32mc07x_dma.h](#) 第 59 行定义.

4.37.2.12 ADC1_EN

u16 ADC1_EN

在文件 [lks32mc07x_dma.h](#) 第 60 行定义.

4.37.2.13 [struct]

struct { ... } B

4.37.2.14 CMP_EN

u16 CMP_EN

在文件 [lks32mc07x_dma.h](#) 第 85 行定义.

4.37.2.15 DSP_EN

u16 DSP_EN

在文件 [lks32mc07x_dma.h](#) 第 62 行定义.

4.37.2.16 GPIO_EN

u16 GPIO_EN

在文件 [lks32mc07x_dma.h](#) 第 88 行定义.

4.37.2.17 HALL0_EN

u16 HALL0_EN

在文件 [lks32mc07x_dma.h](#) 第 83 行定义.

4.37.2.18 I2C0_RX_EN

u16 I2C0_RX_EN

在文件 [lks32mc07x.dma.h](#) 第 73 行定义.

4.37.2.19 I2C0_TX_EN

u16 I2C0_TX_EN

在文件 [lks32mc07x.dma.h](#) 第 74 行定义.

4.37.2.20 MCPWM_RE0_EN

u16 MCPWM_RE0_EN

在文件 [lks32mc07x.dma.h](#) 第 71 行定义.

4.37.2.21 MCPWM_RE1_EN

u16 MCPWM_RE1_EN

在文件 [lks32mc07x.dma.h](#) 第 72 行定义.

4.37.2.22 R

u32 R

在文件 [lks32mc07x.dma.h](#) 第 56 行定义.

4.37.2.23 SIF_EN

u16 SIF_EN

在文件 [lks32mc07x.dma.h](#) 第 84 行定义.

4.37.2.24 SPI_RX_EN

u16 SPI_RX_EN

在文件 [lks32mc07x.dma.h](#) 第 69 行定义.

4.37.2.25 SPI_TX_EN

u16 SPI_TX_EN

在文件 [lks32mc07x.dma.h](#) 第 70 行定义.

4.37.2.26 SW_TRIG_EN

u16 SW_TRIG_EN

在文件 [lks32mc07x.dma.h](#) 第 90 行定义.

4.37.2.27 TIMER0_EN

u16 TIMER0_EN

在文件 [lks32mc07x.dma.h](#) 第 63 行定义.

4.37.2.28 TIMER1_EN

u16 TIMER1_EN

在文件 [lks32mc07x.dma.h](#) 第 64 行定义.

4.37.2.29 TIMER2_EN

u16 TIMER2_EN

在文件 [lks32mc07x.dma.h](#) 第 65 行定义.

4.37.2.30 TIMER3_EN

u16 TIMER3_EN

在文件 [lks32mc07x.dma.h](#) 第 66 行定义.

4.37.2.31 UART0_RX_EN

u16 UART0_RX_EN

在文件 [lks32mc07x.dma.h](#) 第 75 行定义.

4.37.2.32 UART0_TX_EN

u16 UART0_TX_EN

在文件 [lks32mc07x.dma.h](#) 第 76 行定义.

4.37.2.33 UART1_RX_EN

u16 UART1_RX_EN

在文件 [lks32mc07x.dma.h](#) 第 77 行定义.

4.37.2.34 UART1_TX_EN

u16 UART1_TX_EN

在文件 [lks32mc07x_dma.h](#) 第 78 行定义.

4.38 Stru_TempertureCof_Def结构体 参考

芯片内置温度传感器初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

```
#include <lks32mc07x_tmp.h>
```

成员变量

- u16 [nCofA](#)
温度系数A，来源Read.Trim(0x00000398)输出值
- u16 [nOffsetB](#)
温度系数偏置，来源Read.Trim(0x0000039C)输出值

4.38.1 详细描述

芯片内置温度传感器初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

在文件 [lks32mc07x_tmp.h](#) 第 23 行定义.

4.38.2 结构体成员变量说明

4.38.2.1 nCofA

u16 nCofA

温度系数A，来源Read.Trim(0x00000398)输出值

在文件 [lks32mc07x_tmp.h](#) 第 25 行定义.

4.38.2.2 nOffsetB

u16 nOffsetB

温度系数偏置，来源Read.Trim(0x0000039C)输出值

在文件 [lks32mc07x_tmp.h](#) 第 26 行定义.

4.39 SYS_InitTypeDef结构体 参考

系统初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

```
#include <lks32mc07x-sys.h>
```

成员变量

- uint32_t PLL_SrcSel
- uint32_t PLL_DivSel
- uint32_t Clk_Sel
- uint32_t Clk_DivSPI
选择4MHz HS时钟或者PLL作为主时钟
- uint32_t Clk_DivUART
- uint32_t Clk_FEN
- uint32_t WDT_Ena
- uint32_t PORFilter_Ena

4.39.1 详细描述

系统初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

在文件 [lks32mc07x-sys.h](#) 第 90 行定义.

4.39.2 结构体成员变量说明

4.39.2.1 Clk_DivSPI

```
uint32_t Clk_DivSPI
```

选择4MHz HS时钟或者PLL作为主时钟

在文件 [lks32mc07x-sys.h](#) 第 96 行定义.

4.39.2.2 Clk_DivUART

```
uint32_t Clk_DivUART
```

在文件 [lks32mc07x-sys.h](#) 第 97 行定义.

4.39.2.3 Clk_FEN

```
uint32_t Clk_FEN
```

在文件 [lks32mc07x-sys.h](#) 第 98 行定义.

4.39.2.4 Clk_Sel

```
uint32_t Clk_Sel
```

在文件 [lks32mc07x_sys.h](#) 第 94 行定义.

4.39.2.5 PLL_DivSel

```
uint32_t PLL_DivSel
```

在文件 [lks32mc07x_sys.h](#) 第 93 行定义.

4.39.2.6 PLL_SrcSel

```
uint32_t PLL_SrcSel
```

在文件 [lks32mc07x_sys.h](#) 第 92 行定义.

4.39.2.7 PORFilter_Ena

```
uint32_t PORFilter_Ena
```

在文件 [lks32mc07x_sys.h](#) 第 101 行定义.

4.39.2.8 WDT_Ena

```
uint32_t WDT_Ena
```

在文件 [lks32mc07x_sys.h](#) 第 100 行定义.

4.40 SYS_TypeDef结构体 参考

数字模块系统配置寄存器对象定义

```
#include <lks32mc07x_sys.h>
```

成员变量

- __IO u32 CLK_CFG
- __IO u32 IO_CFG
- __IO u32 DBG_CFG
- __IO u32 null
- __IO u32 CLK_DIV0
- __IO u32 CLK_DIV1
- __IO u32 CLK_DIV2
- __IO u32 CLK_FEN
- __IO u32 TRIM
- __IO u32 SFT_RST
- __IO u32 WR_PROTECT
- __IO u32 MBIST
- __IO u32 null7
- __IO u32 null8
- __IO u32 null9
- __IO u32 FLSE
- __IO u32 FLSP
- __IO u32 FLST
- __IO u32 null10
- __IO u32 SBIST

4.40.1 详细描述

数字模块系统配置寄存器对象定义

在文件 [lks32mc07x.sys.h](#) 第 62 行定义.

4.40.2 结构体成员变量说明

4.40.2.1 CLK_CFG

```
__IO u32 CLK_CFG
```

在文件 [lks32mc07x.sys.h](#) 第 64 行定义.

4.40.2.2 CLK_DIV0

```
__IO u32 CLK_DIV0
```

在文件 [lks32mc07x.sys.h](#) 第 68 行定义.

4.40.2.3 CLK_DIV1

```
__IO u32 CLK_DIV1
```

在文件 [lks32mc07x.sys.h](#) 第 69 行定义.

4.40.2.4 CLK_DIV2

```
--IO u32 CLK_DIV2
```

在文件 [lks32mc07x.sys.h](#) 第 70 行定义.

4.40.2.5 CLK_FEN

```
--IO u32 CLK_FEN
```

在文件 [lks32mc07x.sys.h](#) 第 71 行定义.

4.40.2.6 DBG_CFG

```
--IO u32 DBG_CFG
```

在文件 [lks32mc07x.sys.h](#) 第 66 行定义.

4.40.2.7 FLSE

```
--IO u32 FLSE
```

在文件 [lks32mc07x.sys.h](#) 第 79 行定义.

4.40.2.8 FLSP

```
--IO u32 FLSP
```

在文件 [lks32mc07x.sys.h](#) 第 80 行定义.

4.40.2.9 FLST

```
--IO u32 FLST
```

在文件 [lks32mc07x.sys.h](#) 第 81 行定义.

4.40.2.10 IO_CFG

```
--IO u32 IO_CFG
```

在文件 [lks32mc07x.sys.h](#) 第 65 行定义.

4.40.2.11 MBIST

```
--IO u32 MBIST
```

在文件 [lks32mc07x.sys.h](#) 第 75 行定义.

4.40.2.12 null

```
--IO u32 null
```

在文件 [lks32mc07x.sys.h](#) 第 67 行定义.

4.40.2.13 null10

```
--IO u32 null10
```

在文件 [lks32mc07x.sys.h](#) 第 82 行定义.

4.40.2.14 null7

```
--IO u32 null7
```

在文件 [lks32mc07x.sys.h](#) 第 76 行定义.

4.40.2.15 null8

```
--IO u32 null8
```

在文件 [lks32mc07x.sys.h](#) 第 77 行定义.

4.40.2.16 null9

```
--IO u32 null9
```

在文件 [lks32mc07x.sys.h](#) 第 78 行定义.

4.40.2.17 SBIST

```
--IO u32 SBIST
```

在文件 [lks32mc07x.sys.h](#) 第 83 行定义.

4.40.2.18 SFT_RST

```
--IO u32 SFT_RST
```

在文件 [lks32mc07x.sys.h](#) 第 73 行定义.

4.40.2.19 TRIM

```
--IO u32 TRIM
```

在文件 [lks32mc07x.sys.h](#) 第 72 行定义.

4.40.2.20 WR_PROTECT

```
--IO u32 WR_PROTECT
```

在文件 [lks32mc07x_sys.h](#) 第 74 行定义.

4.41 TIM_QEPInitTypeDef结构体 参考

```
#include <lks32mc07x_qep.h>
```

成员变量

- uint32_t [QEP_Mode](#)
编码器模式选择
- uint32_t [QEP_FE_CNT_EN](#)
- uint32_t [QEP_ZNC](#)
- uint32_t [QEP_ZPC](#)
- uint32_t [QEP_ZLC](#)
- uint32_t [QEP_ZEC](#)
- uint16_t [QEP_TH](#)
- uint32_t [QEP_ClockDiv](#)
编码器对应时钟分频系数
- uint32_t [QEP_Filter](#)
- uint32_t [QEP_IRQEna](#)
编码器中断使能

4.41.1 详细描述

在文件 [lks32mc07x_qep.h](#) 第 43 行定义.

4.41.2 结构体成员变量说明

4.41.2.1 QEP_ClockDiv

```
uint32_t QEP_ClockDiv
```

编码器对应时钟分频系数

参见

[ECD_Clk_Div1](#)

[ECD_Clk_Div2](#)

[ECD_Clk_Div4](#)

[ECD_Clk_Div8](#)

[ECD_Clk_Div16](#)

[ECD_Clk_Div32](#)

[ECD_Clk_Div64](#)

[ECD_Clk_Div128](#)

在文件 [lks32mc07x_qep.h](#) 第 70 行定义.

4.41.2.2 QEP_FE_CNT_EN

`uint32_t QEP_FE_CNT_EN`

在文件 [lks32mc07x_qep.h](#) 第 53 行定义.

4.41.2.3 QEP_Filter

`uint32_t QEP_Filter`

在文件 [lks32mc07x_qep.h](#) 第 71 行定义.

4.41.2.4 QEP_IRQEna

`uint32_t QEP_IRQEna`

编码器中断使能

参见

[QEP_IRQEna.None](#)

[QEP_IRQEna.UnderFlow](#)

[QEP_IRQEna.Overflow](#)

在文件 [lks32mc07x_qep.h](#) 第 78 行定义.

4.41.2.5 QEP_Mode

uint32_t QEP_Mode

编码器模式选择

参见

[QEP_Mode_T1](#)

[QEP_Mode_T1_T2](#)

[QEP_Mode_CCWSIGN_UpDown](#)

[QEP_Mode_CCWCW_UpDown](#)

在文件 [lks32mc07x_qep.h](#) 第 52 行定义.

4.41.2.6 QEP_TH

uint16_t QEP_TH

在文件 [lks32mc07x_qep.h](#) 第 58 行定义.

4.41.2.7 QEP_ZEC

uint32_t QEP_ZEC

在文件 [lks32mc07x_qep.h](#) 第 57 行定义.

4.41.2.8 QEP_ZLC

uint32_t QEP_ZLC

在文件 [lks32mc07x_qep.h](#) 第 56 行定义.

4.41.2.9 QEP_ZNC

uint32_t QEP_ZNC

在文件 [lks32mc07x_qep.h](#) 第 54 行定义.

4.41.2.10 QEP_ZPC

uint32_t QEP_ZPC

在文件 [lks32mc07x_qep.h](#) 第 55 行定义.

4.42 TIM_QETypeDef结构体 参考

编码器寄存器结构体句柄

```
#include <lks32mc07x-qep.h>
```

成员变量

- `__IO uint32_t CFG`
编码器配置寄存器
- `__IO uint32_t TH`
编码器计数门限寄存器
- `__IO uint32_t CNT`
编码器计数值寄存器
- `__IO uint32_t IE`
编码器中断使能
- `__IO uint32_t IF`
编码器中断标志位

4.42.1 详细描述

编码器寄存器结构体句柄

在文件 [lks32mc07x-qep.h](#) 第 24 行定义.

4.42.2 结构体成员变量说明

4.42.2.1 CFG

```
__IO uint32_t CFG
```

编码器配置寄存器

在文件 [lks32mc07x-qep.h](#) 第 26 行定义.

4.42.2.2 CNT

```
__IO uint32_t CNT
```

编码器计数值寄存器

在文件 [lks32mc07x-qep.h](#) 第 28 行定义.

4.42.2.3 IE

```
__IO uint32_t IE
```

编码器中断使能

在文件 [lks32mc07x-qep.h](#) 第 29 行定义.

4.42.2.4 IF

```
--IO uint32_t IF
```

编码器中断标志位

在文件 [lks32mc07x_qep.h](#) 第 30 行定义.

4.42.2.5 TH

```
--IO uint32_t TH
```

编码器计数门限寄存器

在文件 [lks32mc07x_qep.h](#) 第 27 行定义.

4.43 TIM_TimerInitTypeDef结构体 参考

```
#include <lks32mc07x_tim.h>
```

成员变量

- FuncState [Timer_EN](#)
定时器使能, *ENABLE*使能, *DISABLE*失能
- uint32_t [Timer_CAP1_CLR_EN](#)
当发生*CAP1*捕获事件时, 清零 *Timer* 计数器, 高有效
- uint32_t [Timer_CAP0_CLR_EN](#)
当发生*CAP0*捕获事件时, 清零 *Timer* 计数器, 高有效
- uint32_t [Timer_ONE_TRIG](#)
单次发送模式, 此位需要在*Timer*比较模式下使用, 且对应定时器使能需设置为0
- uint32_t [Timer_CENTER](#)
中心计数模式使能, 0: *Timer*向上从0计数至*TH*, 然后回0, 或*Timer*向下 从*TH*计数至0, 然后回到*TH*, 1: *Timer*向上从0计数至*TH*, 然后向下计数至0
- uint32_t [Timer_DIR](#)
0: 0->*TH*递增计数, 1: *TH*->递减计数
- uint32_t [Timer_ClockDiv](#)
定时器分频
- uint32_t [Timer_ETON](#)
*Timer*计数器计数外部启动使能, 0: 自动运行, 1: 外部事件触发计数
- uint32_t [Timer_GATE_EN](#)
*Timer*暂停使能, 0: 不暂停 1: 当外部信号为低时, *Timer*暂停计数, 外部信号根据*TIMER1_EVT.EVT_SRC* 进行选择
- uint32_t [Timer_RL_EN](#)
*Timer*重装使能, 0: 禁用外部事件重装, 1: 使能外部事件重装
- uint32_t [Timer_XCLK_EN](#)
*Timer*时钟源, 0: 芯片内部时钟, 1: 外部时钟
- uint32_t [Timer_SRC1](#)

定时器分频

• uint32_t [Timer_CH1Output](#)

Timer 通道 1 在比较模式下的输出极性控制, 当计数器 $CNT < CMP1$ 时的输出值, 0: 低电平, 1: 高电平

• uint32_t [Timer_CH1_WorkMode](#)

Timer 通道 1 的工作模式选择, 0: 比较模式, 1: 捕获模式

• uint32_t [Timer_CH1_FE_CAP_EN](#)

Timer 通道 1 下降沿捕获事件使能, 1: 使能, 0: 关闭

• uint32_t [Timer_CH1_RE_CAP_EN](#)

Timer 通道 1 上升沿捕获事件使能, 1: 使能, 0: 关闭

• uint32_t [Timer_SRC0](#)

定时器分频

• uint32_t [Timer_CH0Output](#)

Timer 通道 0 在比较模式下的输出极性控制, 当计数器 $CNT < CMP1$ 时的输出值, 0: 低电平, 1: 高电平

• uint32_t [Timer_CH0_WorkMode](#)

Timer 通道 0 的工作模式选择, 0: 比较模式, 1: 捕获模式

• uint32_t [Timer_CH0_FE_CAP_EN](#)

Timer 通道 0 下降沿捕获事件使能, 1: 使能, 0: 关闭

• uint32_t [Timer_CH0_RE_CAP_EN](#)

Timer 通道 0 上升沿捕获事件使能, 1: 使能, 0: 关闭

• uint32_t [Timer_TH](#)

Timer 计数器计数门限

• uint32_t [Timer_CNT](#)

Timer 计数器计数门限

• uint32_t [Timer_CMP0](#)

通道 0 比较捕获寄存器

• uint32_t [Timer_CMP1](#)

通道 1 比较捕获寄存器

• uint32_t [Timer_EVT](#)

定时器分频

• uint32_t [Timer_FLT](#)

通道 0/1 信号滤波宽度选择。取值范围 0~255, 滤波公式 = $Timer_FLT * 8 * \text{定时器时钟}$

• uint32_t [Timer_IRQEna](#)

定时器中断使能

• uint32_t [Timer0_UPDATE](#)

软件更新, 1: 将 $TH/CMP0/CMP1$ 的预装载值加载到影子寄存器 $STH/SCMP0/SCMP1$ 之中, 0: 无作用

• uint32_t [Timer0_SHADOW](#)

0: 软件写入 $TH/CMP0/CMP1$ 时直接更新对应的影子寄存器, 1: 启用影子寄存器, 软件写入 $TH/CMP0/$ $CMP1$ 时进仅仅更新预装载值, 当 *Timer* 发生过零事件时才将预装载值写入影子寄存器

• uint32_t [Timer0_CH1_DEFAULT](#)

定时器 0 发生 *Fail* 事件 *MOE* 清零后, *CH1* 通道输出值

• uint32_t [Timer0_CH0_DEFAULT](#)

定时器 0 发生 *Fail* 事件 *MOE* 清零后, *CH0* 通道输出值

• uint32_t [Timer0_HALT_PRT](#)

MCU 进入 *HALT* 状态, *Timer* 输出值选择。1: 正常输出; 0: 输出 *CH1_DEFAULT* 和 *CH0_DEFAULT*。

• uint32_t [Timer0_FAIL_SEL](#)

FAIL 信号选择, 0: *TIMER0_FAIL*, 来自 *GPIO*, 1: *CLU0* 输出

• uint32_t [Timer0_FAIL_POL](#)

FAIL 信号极性, 0: 高电平 *FAIL*, 1: 低电平 *FAIL*

• uint32_t [Timer0_FAIL_EN](#)

FAIL 信号使能, 0: 禁用 *FAIL*, 1: 使能 *FAIL*

4.43.1 详细描述

在文件 [lks32mc07x_tim.h](#) 第 124 行定义.

4.43.2 结构体成员变量说明

4.43.2.1 Timer0.CH0_DEFAULT

```
uint32_t Timer0_CH0_DEFAULT
```

定时器0发生Fail事件MOE清零后, CH0通道输出值

在文件 [lks32mc07x_tim.h](#) 第 222 行定义.

4.43.2.2 Timer0.CH1_DEFAULT

```
uint32_t Timer0_CH1_DEFAULT
```

定时器0发生Fail事件MOE清零后, CH1通道输出值

在文件 [lks32mc07x_tim.h](#) 第 221 行定义.

4.43.2.3 Timer0.FAIL_EN

```
uint32_t Timer0_FAIL_EN
```

FAIL 信号使能,0: 禁用 FAIL,1: 使能 FAIL

在文件 [lks32mc07x_tim.h](#) 第 226 行定义.

4.43.2.4 Timer0.FAIL_POL

```
uint32_t Timer0_FAIL_POL
```

FAIL信号极性,0: 高电平FAIL,1: 低电平FAIL

在文件 [lks32mc07x_tim.h](#) 第 225 行定义.

4.43.2.5 Timer0.FAIL_SEL

```
uint32_t Timer0_FAIL_SEL
```

FAIL信号选择, 0: TIMER0_FAIL, 来自GPIO, 1: CLU0输出

在文件 [lks32mc07x_tim.h](#) 第 224 行定义.

4.43.2.6 Timer0_HALT_PRT

```
uint32_t Timer0_HALT_PRT
```

MCU 进入HALT状态, Timer输出值选择。1:正常输出; 0:输出 CH1_DEFAULT 和 CH0_DEFAULT。

在文件 [lks32mc07x_tim.h](#) 第 223 行定义.

4.43.2.7 Timer0_SHADOW

```
uint32_t Timer0_SHADOW
```

0: 软件写入TH/CMP0/CMP1 时直接更新对应的影子寄存器, 1: 启用影子寄存器, 软件写入 TH/CMP0/←CMP1 时进仅仅更新预装载值, 当Timer发生过零事件时才将预 装载值写入影子寄存器

在文件 [lks32mc07x_tim.h](#) 第 218 行定义.

4.43.2.8 Timer0_UPDATE

```
uint32_t Timer0_UPDATE
```

软件更新, 1: 将 TH/CMP0/CMP1 的预装载值加载到影子寄存器 STH/SCMP0/SCMP1 之中, 0: 无作用

在文件 [lks32mc07x_tim.h](#) 第 216 行定义.

4.43.2.9 Timer_CAP0_CLR_EN

```
uint32_t Timer_CAP0_CLR_EN
```

当发生CAP0捕获事件时, 清零 Timer 计数器, 高有效

在文件 [lks32mc07x_tim.h](#) 第 128 行定义.

4.43.2.10 Timer_CAP1_CLR_EN

```
uint32_t Timer_CAP1_CLR_EN
```

当发生CAP1捕获事件时, 清零 Timer 计数器, 高有效

在文件 [lks32mc07x_tim.h](#) 第 127 行定义.

4.43.2.11 Timer_CENTER

```
uint32_t Timer_CENTER
```

中心计数模式使能,0:Timer向上从0计数至TH, 然后回0, 或Timer向下 从TH计数至0, 然后回到TH, 1←:Timer向上从0计数至TH, 然后向下计数至0

在文件 [lks32mc07x_tim.h](#) 第 132 行定义.

4.43.2.12 Timer.CH0_FE_CAP_EN

```
uint32_t Timer.CH0_FE_CAP_EN
```

Timer通道0下升沿捕获事件使能,1:使能,0:关闭

在文件 [lks32mc07x_tim.h](#) 第 186 行定义.

4.43.2.13 Timer.CH0_RE_CAP_EN

```
uint32_t Timer.CH0_RE_CAP_EN
```

Timer通道0上升沿捕获事件使能,1:使能,0:关闭

在文件 [lks32mc07x_tim.h](#) 第 187 行定义.

4.43.2.14 Timer.CH0_WorkMode

```
uint32_t Timer.CH0_WorkMode
```

Timer 通道0的工作模式选择,0: 比较模式 ,1: 捕获模式

在文件 [lks32mc07x_tim.h](#) 第 185 行定义.

4.43.2.15 Timer.CH0Output

```
uint32_t Timer.CH0Output
```

Timer 通道0在比较模式下的输出极性控制, 当计数器 $CNT < CMP1$ 时的输出值, 0: 低电平, 1: 高电平

在文件 [lks32mc07x_tim.h](#) 第 184 行定义.

4.43.2.16 Timer.CH1_FE_CAP_EN

```
uint32_t Timer.CH1_FE_CAP_EN
```

Timer通道1下升沿捕获事件使能,1:使能,0:关闭

在文件 [lks32mc07x_tim.h](#) 第 168 行定义.

4.43.2.17 Timer.CH1_RE_CAP_EN

```
uint32_t Timer.CH1_RE_CAP_EN
```

Timer通道1上升沿捕获事件使能,1:使能,0:关闭

在文件 [lks32mc07x_tim.h](#) 第 169 行定义.

4.43.2.18 Timer.CH1.WorkMode

```
uint32_t Timer.CH1.WorkMode
```

Timer 通道1的工作模式选择,0: 比较模式 ,1: 捕获模式

在文件 [lks32mc07x_tim.h](#) 第 167 行定义.

4.43.2.19 Timer.CH1Output

```
uint32_t Timer.CH1Output
```

Timer 通道1在比较模式下的输出极性控制, 当计数器 $CNT < CMP1$ 时的输出值, 0: 低电平, 1: 高电平

在文件 [lks32mc07x_tim.h](#) 第 166 行定义.

4.43.2.20 Timer.ClockDiv

```
uint32_t Timer.ClockDiv
```

定时器分频

参见

TIM_Clk_Div1

TIM_Clk_Div2

TIM_Clk_Div4

TIM_Clk_Div8

TIM_Clk_Div16

TIM_Clk_Div32

TIM_Clk_Div64

TIM_Clk_Div128

在文件 [lks32mc07x_tim.h](#) 第 146 行定义.

4.43.2.21 Timer.CMP0

```
uint32_t Timer.CMP0
```

通道0比较捕获寄存器

在文件 [lks32mc07x_tim.h](#) 第 191 行定义.

4.43.2.22 Timer_CMP1

```
uint32_t Timer_CMP1
```

通道1比较捕获寄存器

在文件 [lks32mc07x_tim.h](#) 第 192 行定义.

4.43.2.23 Timer_CNT

```
uint32_t Timer_CNT
```

Timer计数器计数门限

在文件 [lks32mc07x_tim.h](#) 第 190 行定义.

4.43.2.24 Timer_DIR

```
uint32_t Timer_DIR
```

0:0->TH 递增计数, 1: TH->递减计数

在文件 [lks32mc07x_tim.h](#) 第 134 行定义.

4.43.2.25 Timer_EN

```
FuncState Timer_EN
```

定时器使能, ENABLE使能, DISABLE失能

在文件 [lks32mc07x_tim.h](#) 第 126 行定义.

4.43.2.26 Timer_ETON

```
uint32_t Timer_ETON
```

Timer计数器计数外部启动使能,0: 自动运行,1:外部事件触发计数

在文件 [lks32mc07x_tim.h](#) 第 147 行定义.

4.43.2.27 Timer_EVT

uint32_t Timer_EVT

定时器分频

参见

[TIMER0_EVT_CH0](#)

[TIMER0_EVT_CH1](#)

.....

[TIMER_EVT_TADC2](#)

[TIMER_EVT_TADC3](#) Timer外部事件选择

在文件 [lks32mc07x_tim.h](#) 第 201 行定义.

4.43.2.28 Timer_FLT

uint32_t Timer_FLT

通道0/1信号滤波宽度选择。取值范围0~255,滤波公式=Timer_FLT*8*定时器时钟

在文件 [lks32mc07x_tim.h](#) 第 202 行定义.

4.43.2.29 Timer_GATE_EN

uint32_t Timer_GATE_EN

Timer暂停使能, 0: 不暂停 1:当外部信号为低时, Timer暂停计数, 外部信号根据TIMER1_EVT.EVT_SRC进行选择

在文件 [lks32mc07x_tim.h](#) 第 148 行定义.

4.43.2.30 Timer_IRQEna

uint32_t Timer_IRQEna

定时器中断使能

参见

[Timer_IRQEna_CH0](#)

[Timer_IRQEna_CH1](#)

[Timer_IRQEna_ZC](#)

[Timer0_IRQEna_FAIL](#)

[Timer_IRQEna_CH0_RE](#)

[Timer_IRQEna_CH1_RE](#)

[Timer_IRQEna_ZC_RE](#)

[Timer0_IRQEna_FAIL_RE](#) 中断使能

在文件 [lks32mc07x.tim.h](#) 第 214 行定义.

4.43.2.31 Timer.ONE_TRIG

uint32_t Timer.ONE_TRIG

单次发送模式，此位需要在Timer比较模式下使用，且对应定时器使能需设置为0

在文件 [lks32mc07x.tim.h](#) 第 130 行定义.

4.43.2.32 Timer.RL_EN

uint32_t Timer.RL_EN

Timer重装使能,0: 禁用外部事件重装，1: 使能外部事件重装

在文件 [lks32mc07x.tim.h](#) 第 150 行定义.

4.43.2.33 Timer.SRC0

uint32_t Timer.SRC0

定时器分频

参见

[CAP_SRC_TCH0](#)

[CAP_SRC_TCH1](#)

[CAP_SRC_CLU0](#)

[CAP_SRC_CLU1](#)

[CAP_SRC_CLU2](#)

[CAP_SRC_CLU3](#)

[CAP_SRC_CMP0](#)

[CAP_SRC_CMP1](#)

[CAP_SRC_CMP2](#)

[CAP_SRC_CHXOR](#) Timer 捕获模式通道0信号来源

在文件 [lks32mc07x.tim.h](#) 第 183 行定义.

4.43.2.34 Timer_SRC1

uint32_t Timer_SRC1

定时器分频

参见

[CAP_SRC_TCH0](#)

[CAP_SRC_TCH1](#)

[CAP_SRC_CLU0](#)

[CAP_SRC_CLU1](#)

[CAP_SRC_CLU2](#)

[CAP_SRC_CLU3](#)

[CAP_SRC_CMP0](#)

[CAP_SRC_CMP1](#)

[CAP_SRC_CMP2](#)

[CAP_SRC_CHXOR](#)

在文件 [lks32mc07x.tim.h](#) 第 165 行定义.

4.43.2.35 Timer.TH

uint32_t Timer.TH

Timer计数器计数门限

在文件 [lks32mc07x.tim.h](#) 第 189 行定义.

4.43.2.36 Timer.XCLK_EN

uint32_t Timer.XCLK_EN

Timer时钟源,0: 芯片内部时钟,1:外部时钟

在文件 [lks32mc07x.tim.h](#) 第 151 行定义.

4.44 TIM_TimerTypeDef结构体 参考

```
#include <lks32mc07x.tim.h>
```

成员变量

- __IO uint32_t [CFG](#)
定时器配置寄存器
- __IO uint32_t [TH](#)
定时器计数门限寄存器
- __IO uint32_t [CNT](#)
定时器计数值寄存器
- __IO uint32_t [CMP0](#)
定时器比较/捕获寄存器 0
- __IO uint32_t [CMP1](#)
定时器比较/捕获寄存器 1
- __IO uint32_t [EVT](#)
定时器外部事件选择寄存器
- __IO uint32_t [FLT](#)
定时器滤波控制寄存器
- __IO uint32_t [IE](#)
定时器中断使能寄存器
- __IO uint32_t [IF](#)
定时器中断标志寄存器
- __IO uint32_t [IO](#)
定时器IO控制寄存器

4.44.1 详细描述

在文件 [lks32mc07x.tim.h](#) 第 20 行定义.

4.44.2 结构体成员变量说明

4.44.2.1 CFG

```
--IO uint32_t CFG
```

定时器配置寄存器

在文件 [lks32mc07x_tim.h](#) 第 22 行定义.

4.44.2.2 CMP0

```
--IO uint32_t CMP0
```

定时器比较/捕获寄存器 0

在文件 [lks32mc07x_tim.h](#) 第 25 行定义.

4.44.2.3 CMP1

```
--IO uint32_t CMP1
```

定时器比较/捕获寄存器 1

在文件 [lks32mc07x_tim.h](#) 第 26 行定义.

4.44.2.4 CNT

```
--IO uint32_t CNT
```

定时器计数值寄存器

在文件 [lks32mc07x_tim.h](#) 第 24 行定义.

4.44.2.5 EVT

```
--IO uint32_t EVT
```

定时器外部事件选择寄存器

在文件 [lks32mc07x_tim.h](#) 第 27 行定义.

4.44.2.6 FLT

```
--IO uint32_t FLT
```

定时器滤波控制寄存器

在文件 [lks32mc07x_tim.h](#) 第 28 行定义.

4.44.2.7 IE

```
--IO uint32_t IE
```

定时器中断使能寄存器

在文件 [lks32mc07x_tim.h](#) 第 29 行定义.

4.44.2.8 IF

```
--IO uint32_t IF
```

定时器中断标志寄存器

在文件 [lks32mc07x_tim.h](#) 第 30 行定义.

4.44.2.9 IO

```
--IO uint32_t IO
```

定时器IO控制寄存器

在文件 [lks32mc07x_tim.h](#) 第 31 行定义.

4.44.2.10 TH

```
--IO uint32_t TH
```

定时器计数门限寄存器

在文件 [lks32mc07x_tim.h](#) 第 23 行定义.

4.45 UART_InitTypeDef结构体 参考

```
#include <lks32mc07x_uart.h>
```

成员变量

- `uint32_t BaudRate`
波特率
- `uint8_t WordLength`
数据长度
- `uint8_t StopBits`
停止位长度
- `uint8_t FirstSend`
发送数据大小端设置
- `UART_ParityMode ParityMode`
奇偶校验
- `uint8_t MultiDropEna`
使能 *Multi-drop, 0:Disable 1:Enable*
- `uint8_t Bit9Value`
*Multi-drop Master*模式时,第9个数据位值
- `uint8_t DUPLEX`
双工选择,0:全双工 1:半双工
- `uint16_t Match485Addr`
用作485通信时的匹配地址
- `uint16_t IRQEna`
中断使能寄存器
- `uint8_t DMARE`
DMA 请求使能
- `uint8_t RXD_INV`
接收电平取反
- `uint8_t TXD_INV`
发送电平取反

4.45.1 详细描述

在文件 `lks32mc07x_uart.h` 第 56 行定义.

4.45.2 结构体成员变量说明

4.45.2.1 BaudRate

```
uint32_t BaudRate
```

波特率

在文件 `lks32mc07x_uart.h` 第 58 行定义.

4.45.2.2 Bit9Value

```
uint8_t Bit9Value
```

*Multi-drop Master*模式时,第9个数据位值

在文件 `lks32mc07x_uart.h` 第 80 行定义.

4.45.2.3 DMARE

uint8_t DMARE

DMA 请求使能

在文件 [lks32mc07x_uart.h](#) 第 85 行定义.

4.45.2.4 DUPLEX

uint8_t DUPLEX

双工选择,0:全双工 1:半双工

在文件 [lks32mc07x_uart.h](#) 第 81 行定义.

4.45.2.5 FirstSend

uint8_t FirstSend

发送数据大小端设置

参见

[UART_FIRSTSEND_LSB](#)

[UART_FIRSTSEND_MSB](#)

在文件 [lks32mc07x_uart.h](#) 第 73 行定义.

4.45.2.6 IRQEna

uint16_t IRQEna

中断使能寄存器

在文件 [lks32mc07x_uart.h](#) 第 84 行定义.

4.45.2.7 Match485Addr

uint16_t Match485Addr

用作485通信时的匹配地址

在文件 [lks32mc07x_uart.h](#) 第 83 行定义.

4.45.2.8 MultiDropEna

uint8_t MultiDropEna

使能Multi-drop,0:Disable 1:Enable

在文件 [lks32mc07x_uart.h](#) 第 79 行定义.

4.45.2.9 ParityMode

UART_ParityMode ParityMode

奇偶校验

参见

[UART_ParityMode](#)

在文件 [lks32mc07x_uart.h](#) 第 77 行定义.

4.45.2.10 RXD_INV

uint8_t RXD_INV

接收电平取反

在文件 [lks32mc07x_uart.h](#) 第 86 行定义.

4.45.2.11 StopBits

uint8_t StopBits

停止位长度

参见

[UART_STOPBITS_1b](#)

[UART_STOPBITS_2b](#)

在文件 [lks32mc07x_uart.h](#) 第 68 行定义.

4.45.2.12 TXD_INV

uint8_t TXD_INV

发送电平取反

在文件 [lks32mc07x_uart.h](#) 第 87 行定义.

4.45.2.13 WordLength

uint8_t WordLength

数据长度

参见

[UART_WORDLENGTH_8b](#)

[UART_WORDLENGTH_9b](#)

在文件 [lks32mc07x_uart.h](#) 第 63 行定义.

4.46 UART_TypeDef结构体 参考

UART寄存器结构体句柄

```
#include <lks32mc07x_uart.h>
```

成员变量

- [__IO uint32_t CTRL](#)
UART控制寄存器
- [__IO uint32_t DIVH](#)
UART波特率设置高字节寄存器
- [__IO uint32_t DIVL](#)
UART波特率设置低字节寄存器
- [__IO uint32_t BUFF](#)
UART收发缓冲寄存器
- [__IO uint32_t ADR](#)
485通信地址匹配寄存器
- [__IO uint32_t STT](#)
UART状态寄存器
- [__IO uint32_t RE](#)
UART DMA 请求使能寄存器
- [__IO uint32_t IE](#)
UART中断使能寄存器
- [__IO uint32_t IF](#)
UART中断标志寄存器
- [__IO uint32_t IOC](#)
UART IO翻转使能

4.46.1 详细描述

UART寄存器结构体句柄

在文件 [lks32mc07x_uart.h](#) 第 32 行定义.

4.46.2 结构体成员变量说明

4.46.2.1 ADR

```
--IO uint32_t ADR
```

485通信地址匹配寄存器

在文件 [lks32mc07x_uart.h](#) 第 38 行定义.

4.46.2.2 BUFF

```
--IO uint32_t BUFF
```

UART收发缓冲寄存器

在文件 [lks32mc07x_uart.h](#) 第 37 行定义.

4.46.2.3 CTRL

```
--IO uint32_t CTRL
```

UART控制寄存器

在文件 [lks32mc07x_uart.h](#) 第 34 行定义.

4.46.2.4 DIVH

```
--IO uint32_t DIVH
```

UART波特率设置高字节寄存器

在文件 [lks32mc07x_uart.h](#) 第 35 行定义.

4.46.2.5 DIVL

```
--IO uint32_t DIVL
```

UART波特率设置低字节寄存器

在文件 [lks32mc07x_uart.h](#) 第 36 行定义.

4.46.2.6 IE

```
--IO uint32_t IE
```

UART中断使能寄存器

在文件 [lks32mc07x_uart.h](#) 第 41 行定义.

4.46.2.7 IF

```
--IO uint32_t IF
```

UART中断标志寄存器

在文件 [lks32mc07x_uart.h](#) 第 42 行定义.

4.46.2.8 IOC

```
--IO uint32_t IOC
```

UART IO翻转使能

在文件 [lks32mc07x_uart.h](#) 第 43 行定义.

4.46.2.9 RE

```
--IO uint32_t RE
```

UART DMA 请求使能寄存器

在文件 [lks32mc07x_uart.h](#) 第 40 行定义.

4.46.2.10 STT

```
--IO uint32_t STT
```

UART状态寄存器

在文件 [lks32mc07x_uart.h](#) 第 39 行定义.

4.47 WAKE_InitTypeDef结构体 参考

低功耗时钟配置中间变量

```
#include <lks32mc07x_wake.h>
```

成员变量

- uint32_t [AFE_REG5_RECORD](#)
- uint32_t [CLK_CFG_RECORD](#)

4.47.1 详细描述

低功耗时钟配置中间变量

在文件 [lks32mc07x_wake.h](#) 第 22 行定义.

4.47.2 结构体成员变量说明

4.47.2.1 AFE_REG5_RECORD

```
uint32_t AFE_REG5_RECORD
```

在文件 [lks32mc07x_wake.h](#) 第 24 行定义.

4.47.2.2 CLK_CFG_RECORD

```
uint32_t CLK_CFG_RECORD
```

在文件 [lks32mc07x_wake.h](#) 第 25 行定义.

Chapter 5

文件说明

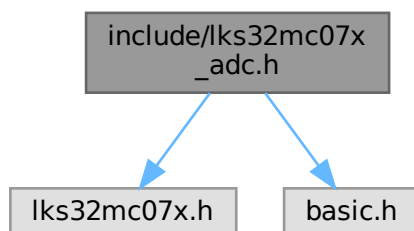
5.1 include/lks32mc07x_adc.h 文件参考

文件名称: LKS32MC07x_adc.h
文件标识: 无
内容摘要: ADC外设驱动程序
其它说明: 无

```
#include "lks32mc07x.h"
```

```
#include "basic.h"
```

lks32mc07x_adc.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [ADC_TypeDef](#)
- struct [ADC_InitTypeDef](#)

宏定义

- **#define ADC0** ((ADC_TypeDef *)ADC0_BASE)
- **#define ADC1** ((ADC_TypeDef *)ADC1_BASE)
- **#define ADC_ISF_RE** BIT15
空闲采样完成 DMA 请求使能
- **#define ADC_HERR_RE** BIT14
硬件触发发生在非空闲状态 DMA 请求使能
- **#define ADC_SERR_RE** BIT13
软件触发发生在非空闲状态 DMA 请求使能
- **#define ADC_AWD0_RE** BIT10
阈值监测 0 超限 DMA 请求使能
- **#define ADC_SF2_RE** BIT9
第二段采样完成 DMA 请求使能
- **#define ADC_SF1_RE** BIT8
第一段采样完成 DMA 请求使能
- **#define ADC_ISF_IE** BIT7
空闲采样完成中断使能
- **#define ADC_HERR_IE** BIT6
硬件触发发生在非空闲状态中断使能
- **#define ADC_SERR_IE** BIT5
软件触发发生在非空闲状态中断使能
- **#define ADC_AWD0_IE** BIT2
阈值监测 0 超限中断使能
- **#define ADC_SF2_IE** BIT1
第二段常规采样完成中断使能
- **#define ADC_SF1_IE** BIT0
第一段常规采样完成中断使能
- **#define ADC_ISF_IF** BIT7
空闲采样完成中断标志
- **#define ADC_HERR_IF** BIT6
硬件触发发生在非空闲状态中断标志
- **#define ADC_SERR_IF** BIT5
软件触发发生在非空闲状态中断标志
- **#define ADC_AWD0_IF** BIT2
阈值监测 0 超限中断标志
- **#define ADC_SF2_IF** BIT1
第二段常规采样完成中断标志
- **#define ADC_SF1_IF** BIT0
第一段常规采样完成中断标志
- **#define ADC_ALL_IF** (0xf)
ADC全部中断标志位——可用于初始化
- **#define ADC_TRIG_CL_IT3** BIT15
CL_OUTPUT[3]触发 ADC 空闲采样使能, 高有效
- **#define ADC_TRIG_CL_IT2** BIT14
CL_OUTPUT[2]触发 ADC 空闲采样使能, 高有效
- **#define ADC_TRIG_CL_IT1** BIT13
CL_OUTPUT[1]触发 ADC 空闲采样使能, 高有效
- **#define ADC_TRIG_CL_IT0** BIT12
CL_OUTPUT[0]触发 ADC 空闲采样使能, 高有效
- **#define ADC_TRIG_CL_NT3** BIT11

- CL_OUTPUT[3]*触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.CL_NT2** BIT10
*CL_OUTPUT[2]*触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.CL_NT1** BIT9
*CL_OUTPUT[1]*触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.CL_NT0** BIT8
*CL_OUTPUT[0]*触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.UTIMER2_CMP1** BIT7
UTIMER2 比较事件 1 触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.UTIMER2_CMP0** BIT6
UTIMER2 比较事件 0 触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.UTIMER0_CMP1** BIT5
UTIMER0 比较事件 1 触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.UTIMER0_CMP0** BIT4
UTIMER0 比较事件 0 触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.MCPWM0_T3** BIT3
MCPWM0 T3 事件触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.MCPWM0_T2** BIT2
MCPWM0 T2 事件触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.MCPWM0_T1** BIT1
MCPWM0 T1 事件触发 ADC 常规采样使能, 高有效
- #define **ADC_TRIG.MCPWM0_T0** BIT0
MCPWM0 T0 事件触发 ADC 常规采样使能, 高有效
- #define **ADC_CHANNEL_0** 0x0
OPA0 输出 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_1** 0x1
OPA1 输出 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_2** 0x2
OPA2 输出 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_3** 0x3
OPA3 输出 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_4** 0x4
ADC01.CH4 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_5** 0x5
ADC01.CH5 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_6** 0x6
ADC01.CH6 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_7** 0x7
ADC01.CH7 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_8** 0x8
ADC01.CH8 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_9** 0x9
ADC01.CH9 *ADC0*和*ADC1*通道0-9共用
- #define **ADC_CHANNEL_10** 0xA
ADC0.CH10 *ADC1.CH10* *ADC0*和*ADC1*通道对应IO口不同
- #define **ADC_CHANNEL_11** 0xB
ADC0.CH11 *ADC1.CH11* *ADC0*和*ADC1*通道对应IO口不同
- #define **ADC_CHANNEL_12** 0xC
ADC0.CH12 *ADC1.CH12* *ADC0*和*ADC1*通道对应IO口不同
- #define **ADC_CHANNEL_13** 0xD
ADC0.CH13 *ADC1.CH13* *ADC0*和*ADC1*通道对应IO口不同

- `#define ADC_CHANNEL_14 0xE`
ADC0通道14对应DAC0输出 ADC1通道14对应温度传感器
- `#define ADC_CHANNEL_15 0xF`
ADC0通道15对应DAC1输出 ADC1通道15对应内部地 AVSS

枚举

- `enum CHNx {`
 `CHN0 = 0, CHN1, CHN2, CHN3,`
 `CHN4, CHN5, CHN6, CHN7,`
 `CHN8, CHN9, CHN10, CHN11,`
 `CHN12, CHN13, CHN14, CHN15 }`
ADC采样通道序号
- `enum DATx {`
 `DAT0 = 0, DAT1, DAT2, DAT3,`
 `DAT4, DAT5, DAT6, DAT7,`
 `DAT8, DAT9, DAT10, DAT11,`
 `DAT12, DAT13, DAT14, DAT15 }`
ADC数据寄存器序号
- `enum ADC_GAINx { ADC_GAIN3V6 = 0, ADC_GAIN7V2 = 1 }`

函数

- `void ADC_Init (ADC_TypeDef *ADCx, ADC_InitTypeDef *ADC_InitStruct)`
函数名称: `void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct)`
- `void ADC_StructInit (ADC_InitTypeDef *ADC_InitStruct)`
函数名称: `void ADC_StructInit(ADC_InitTypeDef* ADC_InitStruct)`
- `uint16_t ADC_GetIRQFlag (ADC_TypeDef *ADCx, uint16_t INT_flag)`
函数名称: `uint16_t ADC_GetIRQFlag(ADC_TypeDef* ADCx, uint16_t INT_flag)`
- `void ADC_ClearIRQFlag (ADC_TypeDef *ADCx, uint16_t INT_flag)`
函数名称: `void ADC_ClearIRQFlag(ADC_TypeDef* ADCx, uint16_t INT_flag)`
- `void ADC_CHN_GAIN_CFG (ADC_TypeDef *ADCx, CHNx CHNum, u8 nChannel, ADC_GAINx GAIN3V6_or_7V2)`
函数名称: `void ADC_CHN_GAIN_CFG(ADC_TypeDef* ADCx, CHNx CHNum, u8 passageway, ADC_GAINx GAIN3V6_or_7V2)`
- `s16_t ADC_GetConversionValue (ADC_TypeDef *ADCx, DATx DATNum)`
函数名称: `s16_t ADC_GetConversionValue(ADC_TypeDef* ADCx, DATx DATNum)`
- `void ADC_SoftTrgEN (ADC_TypeDef *ADCx, FuncState state)`
函数名称: `void ADC_SoftTrgEN(ADC_TypeDef* ADCx, FuncState state)`

5.1.1 详细描述

文件名称: LKS32MC07x_adc.h
文件标识: 无
内容摘要: ADC外设驱动程序
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月22日	1.0	zhujie	创建

修改日志:

在文件 [lks32mc07x_adc.h](#) 中定义.

5.1.2 宏定义说明

5.1.2.1 ADC0

```
#define ADC0 ((ADC_TypeDef *)ADC0_BASE)
```

在文件 [lks32mc07x_adc.h](#) 第 89 行定义.

5.1.2.2 ADC1

```
#define ADC1 ((ADC_TypeDef *)ADC1_BASE)
```

在文件 [lks32mc07x_adc.h](#) 第 90 行定义.

5.1.2.3 ADC_ALL_IF

```
#define ADC_ALL_IF (0x7f)
```

ADC全部中断标志位——可用于初始化

在文件 [lks32mc07x_adc.h](#) 第 132 行定义.

5.1.2.4 ADC_AWD0_IE

```
#define ADC_AWD0_IE BIT2
```

阈值监测 0 超限中断使能

在文件 [lks32mc07x_adc.h](#) 第 122 行定义.

5.1.2.5 ADC_AWD0_IF

```
#define ADC_AWD0_IF BIT2
```

阈值监测 0 超限中断标志

在文件 [lks32mc07x_adc.h](#) 第 129 行定义.

5.1.2.6 ADC_AWD0_RE

```
#define ADC_AWD0_RE BIT10
```

阈值监测 0 超限 DMA 请求使能

在文件 [lks32mc07x_adc.h](#) 第 115 行定义.

5.1.2.7 ADC_CHANNEL_0

```
#define ADC_CHANNEL_0 0x0
```

OPA0 输出 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 151 行定义.

5.1.2.8 ADC_CHANNEL_1

```
#define ADC_CHANNEL_1 0x1
```

OPA1 输出 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 152 行定义.

5.1.2.9 ADC_CHANNEL_10

```
#define ADC_CHANNEL_10 0xA
```

ADC0.CH10 ADC1.CH10 ADC0和ADC1通道对应IO口不同

在文件 [lks32mc07x_adc.h](#) 第 161 行定义.

5.1.2.10 ADC_CHANNEL_11

```
#define ADC_CHANNEL_11 0xB
```

ADC0.CH11 ADC1.CH11 ADC0和ADC1通道对应IO口不同

在文件 [lks32mc07x_adc.h](#) 第 162 行定义.

5.1.2.11 ADC_CHANNEL_12

```
#define ADC_CHANNEL_12 0xC
```

ADC0.CH12 ADC1.CH12 ADC0和ADC1通道对应IO口不同

在文件 [lks32mc07x_adc.h](#) 第 163 行定义.

5.1.2.12 ADC_CHANNEL_13

```
#define ADC_CHANNEL_13 0xD
```

ADC0.CH13 ADC1.CH13 ADC0和ADC1通道对应IO口不同

在文件 [lks32mc07x_adc.h](#) 第 164 行定义.

5.1.2.13 ADC_CHANNEL_14

```
#define ADC_CHANNEL_14 0xE
```

ADC0通道14对应DAC0输出 ADC1通道14对应温度传感器

在文件 [lks32mc07x_adc.h](#) 第 165 行定义.

5.1.2.14 ADC_CHANNEL_15

```
#define ADC_CHANNEL_15 0xF
```

ADC0通道15对应DAC1输出 ADC1通道15对应内部地 AVSS

在文件 [lks32mc07x_adc.h](#) 第 166 行定义.

5.1.2.15 ADC_CHANNEL_2

```
#define ADC_CHANNEL_2 0x2
```

OPA2 输出 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 153 行定义.

5.1.2.16 ADC_CHANNEL_3

```
#define ADC_CHANNEL_3 0x3
```

OPA3 输出 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 154 行定义.

5.1.2.17 ADC_CHANNEL_4

```
#define ADC_CHANNEL_4 0x4
```

ADC01.CH4 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 155 行定义.

5.1.2.18 ADC_CHANNEL_5

```
#define ADC_CHANNEL_5 0x5
```

ADC01.CH5 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 156 行定义.

5.1.2.19 ADC_CHANNEL_6

```
#define ADC_CHANNEL_6 0x6
```

ADC01.CH6 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 157 行定义.

5.1.2.20 ADC_CHANNEL_7

```
#define ADC_CHANNEL_7 0x7
```

ADC01.CH7 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 158 行定义.

5.1.2.21 ADC_CHANNEL_8

```
#define ADC_CHANNEL_8 0x8
```

ADC01.CH8 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 159 行定义.

5.1.2.22 ADC_CHANNEL_9

```
#define ADC_CHANNEL_9 0x9
```

ADC01.CH9 ADC0和ADC1通道0-9共用

在文件 [lks32mc07x_adc.h](#) 第 160 行定义.

5.1.2.23 ADC_HERR_IE

```
#define ADC_HERR_IE BIT6
```

硬件触发发生在非空闲状态中断使能

在文件 [lks32mc07x_adc.h](#) 第 120 行定义.

5.1.2.24 ADC_HERR_IF

```
#define ADC_HERR_IF BIT6
```

硬件触发发生在非空闲状态中断标志

在文件 [lks32mc07x_adc.h](#) 第 127 行定义.

5.1.2.25 ADC_HERR_RE

```
#define ADC_HERR_RE BIT14
```

硬件触发发生在非空闲状态 DMA 请求使能

在文件 [lks32mc07x_adc.h](#) 第 113 行定义.

5.1.2.26 ADC_ISF_IE

```
#define ADC_ISF_IE BIT7
```

空闲采样完成中断使能

在文件 [lks32mc07x_adc.h](#) 第 119 行定义.

5.1.2.27 ADC_ISF_IF

```
#define ADC_ISF_IF BIT7
```

空闲采样完成中断标志

在文件 [lks32mc07x_adc.h](#) 第 126 行定义.

5.1.2.28 ADC_ISF_RE

```
#define ADC_ISF_RE BIT15
```

空闲采样完成 DMA 请求使能

在文件 [lks32mc07x_adc.h](#) 第 112 行定义.

5.1.2.29 ADC_SERR_IE

```
#define ADC_SERR_IE BIT5
```

软件触发发生在非空闲状态中断使能

在文件 [lks32mc07x_adc.h](#) 第 121 行定义.

5.1.2.30 ADC_SERR_IF

```
#define ADC_SERR_IF BIT5
```

软件触发发生在非空闲状态中断标志

在文件 [lks32mc07x_adc.h](#) 第 128 行定义.

5.1.2.31 ADC_SERR_RE

```
#define ADC_SERR_RE BIT13
```

软件触发发生在非空闲状态 DMA 请求使能

在文件 [lks32mc07x_adc.h](#) 第 114 行定义.

5.1.2.32 ADC_SF1_IE

```
#define ADC_SF1_IE BIT0
```

第一段常规采样完成中断使能

在文件 [lks32mc07x_adc.h](#) 第 124 行定义.

5.1.2.33 ADC_SF1_IF

```
#define ADC_SF1_IF BIT0
```

第一段常规采样完成中断标志

在文件 [lks32mc07x_adc.h](#) 第 131 行定义.

5.1.2.34 ADC_SF1_RE

```
#define ADC_SF1_RE BIT8
```

第一段采样完成 DMA 请求使能

在文件 [lks32mc07x_adc.h](#) 第 117 行定义.

5.1.2.35 ADC_SF2_IE

```
#define ADC_SF2_IE BIT1
```

第二段常规采样完成中断使能

在文件 [lks32mc07x_adc.h](#) 第 123 行定义.

5.1.2.36 ADC_SF2_IF

```
#define ADC_SF2_IF BIT1
```

第二段常规采样完成中断标志

在文件 [lks32mc07x_adc.h](#) 第 130 行定义.

5.1.2.37 ADC_SF2_RE

```
#define ADC_SF2_RE BIT9
```

第二段采样完成 DMA 请求使能

在文件 [lks32mc07x_adc.h](#) 第 116 行定义.

5.1.2.38 ADC_TRIG_CL_IT0

```
#define ADC_TRIG_CL_IT0 BIT12
```

CL_OUTPUT[0]触发 ADC 空闲采样使能, 高有效

在文件 [lks32mc07x_adc.h](#) 第 137 行定义.

5.1.2.39 ADC_TRIG_CL_IT1

```
#define ADC_TRIG_CL_IT1 BIT13
```

CL_OUTPUT[1]触发 ADC 空闲采样使能, 高有效

在文件 [lks32mc07x_adc.h](#) 第 136 行定义.

5.1.2.40 ADC_TRIG_CL_IT2

```
#define ADC_TRIG_CL_IT2 BIT14
```

CL_OUTPUT[2]触发 ADC 空闲采样使能, 高有效

在文件 [lks32mc07x_adc.h](#) 第 135 行定义.

5.1.2.41 ADC_TRIG_CL_IT3

```
#define ADC_TRIG_CL_IT3 BIT15
```

CL_OUTPUT[3]触发 ADC 空闲采样使能, 高有效

在文件 [lks32mc07x_adc.h](#) 第 134 行定义.

5.1.2.42 ADC_TRIG_CL_NT0

```
#define ADC_TRIG_CL_NT0 BIT8
```

CL.OUTPUT[0]触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 141 行定义.

5.1.2.43 ADC_TRIG_CL_NT1

```
#define ADC_TRIG_CL_NT1 BIT9
```

CL.OUTPUT[1]触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 140 行定义.

5.1.2.44 ADC_TRIG_CL_NT2

```
#define ADC_TRIG_CL_NT2 BIT10
```

CL.OUTPUT[2]触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 139 行定义.

5.1.2.45 ADC_TRIG_CL_NT3

```
#define ADC_TRIG_CL_NT3 BIT11
```

CL.OUTPUT[3]触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 138 行定义.

5.1.2.46 ADC_TRIG_MCPWM0_T0

```
#define ADC_TRIG_MCPWM0_T0 BIT0
```

MCPWM0 T0 事件触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 149 行定义.

5.1.2.47 ADC_TRIG_MCPWM0_T1

```
#define ADC_TRIG_MCPWM0_T1 BIT1
```

MCPWM0 T1 事件触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 148 行定义.

5.1.2.48 ADC_TRIG_MCPWM0_T2

```
#define ADC_TRIG_MCPWM0_T2 BIT2
```

MCPWM0 T2 事件触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 147 行定义.

5.1.2.49 ADC_TRIG_MCPWM0_T3

```
#define ADC_TRIG_MCPWM0_T3 BIT3
```

MCPWM0 T3 事件触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 146 行定义.

5.1.2.50 ADC_TRIG_UTIMER0_CMP0

```
#define ADC_TRIG_UTIMER0_CMP0 BIT4
```

UTIMER0 比较事件 0 触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 145 行定义.

5.1.2.51 ADC_TRIG_UTIMER0_CMP1

```
#define ADC_TRIG_UTIMER0_CMP1 BIT5
```

UTIMER0 比较事件 1 触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 144 行定义.

5.1.2.52 ADC_TRIG_UTIMER2_CMP0

```
#define ADC_TRIG_UTIMER2_CMP0 BIT6
```

UTIMER2 比较事件 0 触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 143 行定义.

5.1.2.53 ADC_TRIG_UTIMER2_CMP1

```
#define ADC_TRIG_UTIMER2_CMP1 BIT7
```

UTIMER2 比较事件 1 触发 ADC 常规采样使能，高有效

在文件 [lks32mc07x_adc.h](#) 第 142 行定义.

5.1.3 枚举类型说明

5.1.3.1 ADC_GAINx

```
enum ADC\_GAINx
```

枚举值

ADC_GAIN3V6	3.6V 量程
ADC_GAIN7V2	7.2V 量程

在文件 [lks32mc07x_adc.h](#) 第 210 行定义.

5.1.3.2 CHNx

enum [CHNx](#)

ADC采样通道序号

枚举值

CHN0	第0次采样通道序号
CHN1	第1次采样通道序号
CHN2	第2次采样通道序号
CHN3	第3次采样通道序号
CHN4	第4次采样通道序号
CHN5	第5次采样通道序号
CHN6	第6次采样通道序号
CHN7	第7次采样通道序号
CHN8	第8次采样通道序号
CHN9	第9次采样通道序号
CHN10	第10次采样通道序号
CHN11	第11次采样通道序号
CHN12	第12次采样通道序号
CHN13	第13次采样通道序号
CHN14	第14次采样通道序号
CHN15	第15次采样通道序号

在文件 [lks32mc07x_adc.h](#) 第 169 行定义.

5.1.3.3 DATx

enum [DATx](#)

ADC数据寄存器序号

枚举值

DAT0	第0次采样信号对应结果寄存器序号
DAT1	第1次采样信号对应结果寄存器序号
DAT2	第2次采样信号对应结果寄存器序号
DAT3	第3次采样信号对应结果寄存器序号

枚举值

DAT4	第4次采样信号对应结果寄存器序号
DAT5	第5次采样信号对应结果寄存器序号
DAT6	第6次采样信号对应结果寄存器序号
DAT7	第7次采样信号对应结果寄存器序号
DAT8	第8次采样信号对应结果寄存器序号
DAT9	第9次采样信号对应结果寄存器序号
DAT10	第10次采样信号对应结果寄存器序号
DAT11	第11次采样信号对应结果寄存器序号
DAT12	第12次采样信号对应结果寄存器序号
DAT13	第13次采样信号对应结果寄存器序号
DAT14	第14次采样信号对应结果寄存器序号
DAT15	第15次采样信号对应结果寄存器序号

在文件 [lks32mc07x_adc.h](#) 第 189 行定义.

5.1.4 函数说明

5.1.4.1 ADC_CHN_GAIN_CFG()

```
void ADC_CHN_GAIN_CFG (
    ADC_TypeDef * ADCx,
    CHNx CHNum,
    u8 nChannel,
    ADC_GAINx GAIN3V6_or_7V2 )
```

函数名称: void [ADC_CHN_GAIN_CFG\(ADC_TypeDef *ADCx,CHNx CHNum,u8 passageway, ADC_GAINx GAIN3V6_or_7V2\)](#)

功能描述: ADC采样通道设置

参见

被引用内容: [CHNx](#)

参数

输入参数:	ADCx, 07系列有ADC0和ADC1 CHNum为枚举类型, 对应0-15; Channel_0为采样通道序号对应0 nChannel: ADC_CHANNEL_0 ~ ADC_CHANNEL_15 通道选择 ADC_GAINx: ADC_GAIN3V6,ADC_GAIN7V2 量程选择
输出参数:	无

返回

返回值: 无

注解

其它说明： 无

警告

无

示例代码:

```
ADC_CHN_GAIN_CFG (ADC0, CHN0, ADC_CHANNEL_9, ADC_GAIN3V6); //配置ADC0第0次采样ADC通道9的信号, 选择量程为3.6V
ADC_CHN_GAIN_CFG (ADC1, CHN1, ADC_CHANNEL_9, ADC_GAIN7V2); //配置ADC1第1次采样ADC通道9的信号, 选择量程为7.2V
```

Date	Version	Author	Description
2023年03月9日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 222 行定义.

5.1.4.2 ADC_ClearIRQFlag()

```
void ADC_ClearIRQFlag (
    ADC_TypeDef * ADCx,
    uint16_t INT_flag )
```

函数名称: void [ADC_ClearIRQFlag\(ADC_TypeDef* ADCx, uint16_t INT_flag\)](#)

功能描述: ADC中断标志位清零

参见

被引用内容： 无

参数

输入参数:	ADCx:	ADC0
	INT_flag:	ADC中断标志
	ADC_ISF_IF	空闲采样完成中断标志
	ADC_HERR_IF	硬件触发发生在非空闲状态中断标志
	ADC_SERR_IF	软件触发发生在非空闲状态中断标志
	ADC_AWD0_IF	阈值监测 0 超限中断标志
	ADC_SF2_IF	第二段扫描结束中断标志
	ADC_SF1_IF	第一段扫描结束中断标志
	ADC_ALL_IF	所有ADC中断标志
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (ADC_GetIRQFlag (ADC0, ADC_EOS0_IRQ_IF) ) // 获取第一段扫描结束中断标志
{
    ADC_ClearIRQFlag (ADC0, ADC_EOS0_IRQ_IF) // 第一段扫描结束中断标志
}
```

Date	Version	Author	Description
2023年02月27日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 148 行定义.

5.1.4.3 ADC_GetConversionValue()

```
s16 ADC_GetConversionValue (
    ADC_TypeDef * ADCx,
    DATx DATNum )
```

函数名称: s16 ADC_GetConversionValue(ADC_TypeDef *ADCx,DATx DATNum)

功能描述: 读ADC采样值

参见

被引用内容: ADCx: ADC0或ADC1

参数

输入参数:	DATx: 读取第几次采样结果值，取值为0-15
输出参数:	无

返回

返回 值: 对应x次采样结果值

注解

其它说明: 无

警告

无

示例代码:

```
s16 ADC_samp0 = 0;
ADC_samp0 = ADC_GetConversionValue (ADC0, DAT3); //读取ADC0第3次采样结果
```

Date	Version	Author	Description
2023年02月27日	1.0	Zhu Jie	创建

修改日志:

在文件 lks32mc07x_adc.c 第 260 行定义.

5.1.4.4 ADC_GetIRQFlag()

```
u16 ADC_GetIRQFlag (
```



```
ADC_TypeDef * ADCx,
uint16_t INT_flag )
```

函数名称: uint16_t ADC_GetIRQFlag(ADC_TypeDef* ADCx, uint16_t INT_flag)

功能描述: 获取ADC中断标志

参见
被引用内容: 无

参数

输入参数:	ADCx:	ADC0
	INT_flag:	ADC中断标志
	ADC_ISF_IF	空闲采样完成中断标志
	ADC_HERR_IF	硬件触发发生在非空闲状态中断标志
	ADC_SERR_IF	软件触发发生在非空闲状态中断标志
	ADC_AWD0_IF	阈值监测 0 超限中断标志
	ADC_SF2_IF	第二段扫描结束中断标志
	ADC_SF1_IF	第一段扫描结束中断标志
输出参数:	无	

返回
返回 值: 读对应标志位是否为1, 1位置位, 0为未置位

注解
其它说明: 无

警告
只有对应中断使能后, 改为才能读取, 如果对应中断未使能, 读取结果一直为0

示例代码:

```
if(ADC_GetIRQFlag(ADC0,ADC_SF1_IF))//获取第一段扫描结束中断标志
{
    ADC_ClearIRQFlag(ADC0,ADC_SF1_IF) //第一段扫描结束中断标志
}
```

Date	Version	Author	Description
2023年02月27日	1.0	Zhu Jie	创建

修改日志:

在文件 `lks32mc07x_adc.c` 第 184 行定义.

5.1.4.5 ADC_Init()

```
void ADC_Init (
    ADC_TypeDef * ADCx,
    ADC_InitTypeDef * this )
```

函数名称: `void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct)`

功能描述: ADC初始化函数

参见

被引用内容: `SYS_AnalogModuleClockCmd()`

参数

输入参数:	ADCx: ADC0 , <code>ADC_InitTypeDef *ADC_InitTypeDef</code>
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
ADC_InitTypeDef ADC_InitStructure;
ADC_StructInit (&ADC_InitStructure);
ADC_InitStructure.ADC_CLK_DIV = ADC_Clock_48M;
ADC_InitStructure.Align = ADC_LEFT_ALIGN;
ADC_InitStructure.Trigger_Mode = ADC_2SEG_TRG;
ADC_InitStructure.FirSeg_Ch = ADC_2_TIMES_SAMPLE;
ADC_InitStructure.SecSeg_Ch = ADC_2_TIMES_SAMPLE;
ADC_InitStructure.ThrSeg_Ch = 0;
ADC_InitStructure.FouSeg_Ch = 0;
ADC_InitStructure.RefVol = RefVol_2V4;
// 初始化结构体
// ADC时钟选择48MHz
// ADC数据输出左对齐
// 设置ADC转换模式为双段式采样
// 第一段共采样2个通道
// 第二段共采样2个通道
// 第三段共采样0个通道
// 第四段共采样0个通道
// ADC基准电压2.4V
```

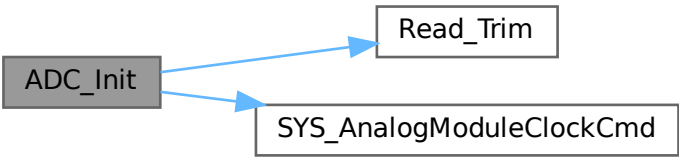
```
ADC.InitStructure.Gain0 = 0x0; // ADC_DAT0-ADC_DAT9 (BIT0~BIT9) 采样保持电
路增益设置2/3
ADC.InitStructure.Gain1 = 0x0; // ADC_DAT10-ADC_DAT19 (BIT10~BIT19) 采样保
持电路增益设置2/3
ADC.InitStructure.Con.Sample = DISABLE; // 关闭连续采样
// 打开 UTIMER T0 T1 硬件触发采样
ADC.InitStructure.MCPWM.Trigger.En = ADC_MCPWM_T0_TRG|ADC_MCPWM_T1_TRG; // 开启UTIMER硬件第一段和第二段触发采样
ADC.InitStructure.UTIMER.Trigger.En = DISABLE; // 关闭UTIMER硬件触发采样
ADC.InitStructure.Trigger.Cnt = 0; // 发生 (Trigger.Cnt+1) 次触发事件执行一段转换
动作
ADC.InitStructure.IE = ADC_EOS0_IRQ_EN | ADC_EOS1_IRQ_EN; // ADC第一、二段采样结束中断使能
ADC_Init (ADC0, &ADC.InitStructure);
```

Date	Version	Author	Description
2015年11月26日	1.0	Howlet Li	创建

修改日志:

在文件 `lks32mc07x_adc.c` 第 54 行定义.

函数调用图:



5.1.4.6 ADC_SoftTrgEN()

```
void ADC_SoftTrgEN (
    ADC_TypeDef * ADCx,
    FuncState state )
```

函数名称: void `ADC_SoftTrgEN(ADC_TypeDef* ADCx, FuncState state)`

功能描述: ADC采样软件触发使能

参见

被引用内容: FuncState

参数

输入参数:	ADCx, 08系列固定选择ADC0 state: ENABLE触发, DISABLE不触发
输出参数:	无

返回

返回 值： 无

注解

其它说明： ADC0_SWT写0x5AA5仅触发一次，完成后自动清零，等待下次软件触发

警告

无

示例代码：

```
ADC_SoftTrgEN(ADC0, ENABLE); //软件触发一次ADC采样
```

Date	Version	Author	Description
2023年02月27日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 353 行定义.

5.1.4.7 ADC_StructInit()

```
void ADC_StructInit (
    ADC_InitTypeDef * this )
```

函数名称: void [ADC_StructInit\(ADC_InitTypeDef* ADC_InitStruct\)](#)

功能描述: ADC结构体初始化

参见

被引用内容： 无

参数

输入参数:	ADC_InitTypeDef *ADC_InitStruct
输出参数:	无

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

```
ADC_InitTypeDef ADC_InitStructure;
ADC_StructInit (&ADC_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2015年11月26日	1.0	Howlet	修改

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 99 行定义.

5.2 lks32mc07x_adc.h

[浏览该文件的文档.](#)

```
00001
00014 #ifndef _LKS32MC07x_ADC_H_
00015 #define _LKS32MC07x_ADC_H_
00016
00017 /* Includes -----*/
00018 #include "lks32mc07x.h"
00019 #include "basic.h"
00020
00021 typedef struct
00022 {
00023     __IO uint32_t DAT0;
00024     __IO uint32_t DAT1;
00025     __IO uint32_t DAT2;
00026     __IO uint32_t DAT3;
00028     __IO uint32_t DAT4;
00029     __IO uint32_t DAT5;
00030     __IO uint32_t DAT6;
00031     __IO uint32_t DAT7;
00033     __IO uint32_t DAT8;
00034     __IO uint32_t DAT9;
00035     __IO uint32_t DAT10;
00036     __IO uint32_t DAT11;
00038     __IO uint32_t DAT12;
00039     __IO uint32_t DAT13;
00040     __IO uint32_t IDAT0;
00041     __IO uint32_t IDAT1;
00043     __IO uint32_t ICHN;
00044     __IO uint32_t null;
00045     __IO uint32_t null1;
00046     __IO uint32_t null12;
00048     __IO uint32_t CHN0;
00049     __IO uint32_t CHN1;
00050     __IO uint32_t CHN2;
00051     __IO uint32_t CHN3;
00053     __IO uint32_t CHNT;
00054     __IO uint32_t GAIN;
00055     __IO uint32_t null13;
00056     __IO uint32_t null14;
00058     __IO uint32_t null15;
```

```

00059     __IO uint32_t CFG;
00060     __IO uint32_t TRIG;
00061     __IO uint32_t SWT;
00063     __IO uint32_t DC0;
00064     __IO uint32_t AMC0;
00065     __IO uint32_t DC1;
00066     __IO uint32_t AMC1;
00068     __IO uint32_t IE;
00069     __IO uint32_t IF;
00070     __IO uint32_t null16;
00071     __IO uint32_t null17;
00073     __IO uint32_t null18;
00074     __IO uint32_t null19;
00075     __IO uint32_t null110;
00076     __IO uint32_t null111;
00078     __IO uint32_t null112;
00079     __IO uint32_t null113;
00080     __IO uint32_t null114;
00081     __IO uint32_t null115;
00083     __IO uint32_t null116;
00084     __IO uint32_t LTH;
00085     __IO uint32_t HTH;
00086     __IO uint32_t GEN;
00087 } ADC_TypeDef;
00088
00089 #define ADC0 ((ADC_TypeDef *)ADC0_BASE)
00090 #define ADC1 ((ADC_TypeDef *)ADC1_BASE)
00091
00092 typedef struct
00093 {
00094     u16 IE;
00095     u16 RE;
00096     u16 NSMP;
00097     u16 DATA_ALIGN;
00098     u16 CSMP;
00099     u16 TCNT;
00100     u16 TROVS;
00101     u16 OVSR;
00102     u16 TRIG;
00103     u16 S1;
00104     u16 S2;
00105     u16 IS1;
00106     u16 GAIN;
00107     u16 LTH;
00108     u16 HTH;
00109     u16 GEN;
00110 } ADC_InitTypeDef;
00111
00112 #define ADC_ISF_RE          BIT15
00113 #define ADC_HERR_RE         BIT14
00114 #define ADC_SERR_RE         BIT13
00115 #define ADC_AWD0_RE         BIT10
00116 #define ADC_SF2_RE          BIT9
00117 #define ADC_SF1_RE          BIT8
00119 #define ADC_ISF_IE          BIT7
00120 #define ADC_HERR_IE         BIT6
00121 #define ADC_SERR_IE         BIT5
00122 #define ADC_AWD0_IE         BIT2
00123 #define ADC_SF2_IE          BIT1
00124 #define ADC_SF1_IE          BIT0
00126 #define ADC_ISF_IF          BIT7
00127 #define ADC_HERR_IF         BIT6
00128 #define ADC_SERR_IF         BIT5
00129 #define ADC_AWD0_IF         BIT2
00130 #define ADC_SF2_IF          BIT1
00131 #define ADC_SF1_IF          BIT0
00132 #define ADC_ALL_IF          (0x7f)
00134 #define ADC_TRIG_CL_IT3     BIT15
00135 #define ADC_TRIG_CL_IT2     BIT14
00136 #define ADC_TRIG_CL_IT1     BIT13
00137 #define ADC_TRIG_CL_IT0     BIT12
00138 #define ADC_TRIG_CL_NT3     BIT11
00139 #define ADC_TRIG_CL_NT2     BIT10
00140 #define ADC_TRIG_CL_NT1     BIT9
00141 #define ADC_TRIG_CL_NT0     BIT8
00142 #define ADC_TRIG_UTIMER2_CMP1 BIT7
00143 #define ADC_TRIG_UTIMER2_CMP0 BIT6
00144 #define ADC_TRIG_UTIMER0_CMP1 BIT5
00145 #define ADC_TRIG_UTIMER0_CMP0 BIT4
00146 #define ADC_TRIG_MCPWM0_T3  BIT3
00147 #define ADC_TRIG_MCPWM0_T2  BIT2
00148 #define ADC_TRIG_MCPWM0_T1  BIT1
00149 #define ADC_TRIG_MCPWM0_T0  BIT0
00151 #define ADC_CHANNEL_0       0x0
00152 #define ADC_CHANNEL_1       0x1
00153 #define ADC_CHANNEL_2       0x2
00154 #define ADC_CHANNEL_3       0x3

```

```

00155 #define ADC_CHANNEL_4          0x4
00156 #define ADC_CHANNEL_5          0x5
00157 #define ADC_CHANNEL_6          0x6
00158 #define ADC_CHANNEL_7          0x7
00159 #define ADC_CHANNEL_8          0x8
00160 #define ADC_CHANNEL_9          0x9
00161 #define ADC_CHANNEL_10         0xA
00162 #define ADC_CHANNEL_11         0xB
00163 #define ADC_CHANNEL_12         0xC
00164 #define ADC_CHANNEL_13         0xD
00165 #define ADC_CHANNEL_14         0xE
00166 #define ADC_CHANNEL_15         0xF
00169 typedef enum
00170 {
00171     CHN0 = 0,
00172     CHN1,
00173     CHN2,
00174     CHN3,
00175     CHN4,
00176     CHN5,
00177     CHN6,
00178     CHN7,
00179     CHN8,
00180     CHN9,
00181     CHN10,
00182     CHN11,
00183     CHN12,
00184     CHN13,
00185     CHN14,
00186     CHN15,
00187 } CHNx;
00189 typedef enum
00190 {
00191     DAT0 = 0,
00192     DAT1,
00193     DAT2,
00194     DAT3,
00195     DAT4,
00196     DAT5,
00197     DAT6,
00198     DAT7,
00199     DAT8,
00200     DAT9,
00201     DAT10,
00202     DAT11,
00203     DAT12,
00204     DAT13,
00205     DAT14,
00206     DAT15,
00207 } DATx;
00208
00209 /*定义ADC增益序号*/
00210 typedef enum
00211 {
00212     ADC_GAIN3V6 = 0,
00213     ADC_GAIN7V2 = 1,
00214 } ADC_GAINx;
00215
00216 void ADC_Init(ADC_TypeDef *ADCx, ADC_InitTypeDef *ADC_InitStruct);
00217 void ADC_StructInit(ADC_InitTypeDef *ADC_InitStruct);
00218 uint16_t ADC_GetIRQFlag(ADC_TypeDef *ADCx, uint16_t INT_flag);
00219 void ADC_ClearIRQFlag(ADC_TypeDef *ADCx, uint16_t INT_flag);
00220 void ADC_CHN_GAIN_CFG(ADC_TypeDef *ADCx, CHNx CHNum, uint8_t nChannel, ADC_GAINx GAIN3V6_or_7V2);
00221 uint16_t ADC_GetConversionValue(ADC_TypeDef *ADCx, DATx DATNum);
00222 void ADC_SoftTrgEN(ADC_TypeDef *ADCx, FuncState state);
00223
00224 #endif /*_LKS32MC07x_ADC_H_*/
00225
00226 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.3 include/lks32mc07x_can.h 文件参考

文件名称: LKS32MC07x_can.h
 文件标识: 无
 内容摘要: CAN外设驱动程序
 其它说明: 无

```
#include "lks32mc07x_lib.h"
```

lks32mc07x.can.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [can_parameter](#)

< Includes -----

- struct [CAN_TypeDef](#)
- struct [CAN_InitTypeDef](#)
- struct [CAN_TxMsg](#)
- struct [CAN_RxMsg](#)

宏定义

- #define [CAN](#) (([CAN_TypeDef](#) *)CAN0.BASE)
- #define [CAN_MODE_LOM](#) BIT8
[8] 监听模式使能位
- #define [CAN_MODE_RESET](#) BIT7
[7] 模块复位模式
- #define [CAN_MODE_LBME](#) BIT6
[6] 外部回环模式使能位
- #define [CAN_MODE_LBMI](#) BIT5
[5] 内部回环模式使能位
- #define [CAN_MODE_TPSS](#) BIT4
[4] PTB 单次传输模式使能位
- #define [CAN_MODE_TSSS](#) BIT3
[3] STB 单次传输模式使能位
- #define [CAN_RTIE_RIE](#) BIT7
[7] 接收中断使能 (Receive Interrupt Enable)
- #define [CAN_RTIE_ROIE](#) BIT6
[6] 接收溢出中断使能 (Receive Overrun Interrupt Enable)
- #define [CAN_RTIE_RFIE](#) BIT5
[5] Rx Buffer 满中断使能 (Rx Buffer Full Interrupt Enable)
- #define [CAN_RTIE RAFIE](#) BIT4
[4] Rx Buffer 将满中断使能 (Rx Buffer Almost Full Interrupt Enable)
- #define [CAN_RTIE_TPIE](#) BIT3
[3] PTB 发送中断使能 (Transmission Primary Interrupt Enable)
- #define [CAN_RTIE_TSIE](#) BIT2

- [2] STB 发送中断使能 (*Transmission Secondary Interrupt Enable*)
- #define CAN_RTIE_EIE BIT1
 - [1] 错误中断使能 (*Error Interrupt Enable*)
- #define CAN_RTIE_TSFF BIT0
 - [0] Tx Buffer 标志位 1: STB Slot 被全部填满 0: STB Slot 没有被全部填满
- #define CAN_ERRINT_EPIE BIT5
 - [13] 被动错误中断使能 (*Error Passive Interrupt Enable*) 1: 使能 0: 禁止
- #define CAN_ERRINT_ALIE BIT3
 - [11] 仲裁失败中断使能 (*Arbitration Lost Interrupt Enable*) 1: 使能 0: 禁止
- #define CAN_ERRINT_BEIE BIT1
 - [8] 总线错误中断使能 (*Bus Error Interrupt Enable*) 1: 使能 0: 禁止
- #define CAN_RTIF_RIF BIT7
 - [7] 接收中断标志位 (*Receive Interrupt Flag*) 1: 接收到有效帧 (数据帧或者远程帧), 写 1 清零 0: 没有接收到有效帧
- #define CAN_RTIF_ROIF BIT6
 - [6] 接收溢出中断标志位 (*Receive Overrun Interrupt Flag*) 1: Rx Buffer 至少有一帧未读走的数据被覆盖 0: Rx Buffer 无覆盖 溢出时 ROIF 和 RFIF 同时置 1, 写 1 清零
- #define CAN_RTIF_RFIF BIT5
 - [5] Rx Buffer 满中断标志位 (*Rx Buffer Full Interrupt Flag*) 1: Rx Buffer 满, 写 1 清零 0: Rx Buffer 未满
- #define CAN_RTIF_RAFIF BIT4
 - [4] Rx Buffer 将满中断标志位 (*Rx Buffer Almost Full Interrupt Flag*) 1: 被填充的 Rx Buffer Slot 数目大于等于 AFWL 设定值 0: 被填充的 Rx Buffer Slot 数目小于 AFWL 设定值
- #define CAN_RTIF_TPIF BIT3
 - [3] PTB 发送中断标志 (*Transmission Primary Interrupt Flag*) 1: 触发 PTB 发送, 发送成功完成。写 1 清零 0: 无 PTB 发送请求, 无完成标志
- #define CAN_RTIF_TSIF BIT2
 - [2] STB 发送中断标志 (*Transmission Secondary Interrupt Flag*) 1: 触发 STB 发送, 发送成功完成。写 1 清零 0: 无 STB 发送请求, 无完成标志
- #define CAN_RTIF_EIF BIT1
 - [1] 错误中断标志 (*Error Interrupt Flag*) 1: 错误计数器的值发生变化, 大于或者小于错误警告寄存器的设定值。写 1 清零 0: 无错误计数器的值从小于错误警告寄存器的设定值变为大于设定值, 或者从大于设定值变为小于设定值, 均会触发中断标志。另外, 进入 Bus Off 或从 Bus Off 退出, 也会触发。
- #define CAN_RTIF_AIF BIT0
 - [0] 取消发送中断标志 (*Abort Interrupt Flag*) 1: 通过 TPA 和 TSA 请求的发送消息被成功取消。写 1 清零 0: 未取消发送数据 AIF 没有对应的使能寄存器
- #define CAN_ERRINT_EWARN BIT7
 - [15] 错误计数值超过门限中断标志位 1: RECNT 或者 TECNT 大于等于 EWL 设定值, 写 1 清零 0: RECNT 或者 TECNT 小于 EWL 设定值
- #define CAN_ERRINT_EPASS BIT6
 - [14] CAN 设备处于被动错误状态 1: CAN 设备处于被动错误状态 0: CAN 设备处于主动错误状态
- #define CAN_ERRINT_EPIF BIT4
 - [12] 被动错误中断标志 (*Error Passive Interrupt Flag*) 1: 发生主动错误到被动错误 (或者相反) 的改变, 写 1 清零 0: 未发生
- #define CAN_ERRINT_ALIF BIT2
 - [10] 仲裁失败中断标志位 (*Arbitration Lost Interrupt Flag*) 1: 仲裁失败, 写 1 清零 0: 仲裁成功
- #define CAN_ERRINT_BEIF BIT0
 - [8] 总线错误中断标志 (*Bus Error Interrupt Flag*) 1: 总线错误, 写 1 清零 0: 无总线错误
- #define CANRXMSG_HEADLEN 8 /* CANRxMsg 中 id~len 的字节数 */
- #define ID0_Filter(a, b, c) IDx_Filter(0, a, b, c)
- #define ID1_Filter(a, b, c) IDx_Filter(1, a, b, c)
- #define ID2_Filter(a, b, c) IDx_Filter(2, a, b, c)
- #define ID3_Filter(a, b, c) IDx_Filter(3, a, b, c)

枚举

- enum **CANErrorKind** {
CanError_None = 0 , **CanErrorr_Bit** = 1 , **CanErrorr_Form** = 2 , **CanErrorr_Stuff** = 3 ,
CanErrorr_Ack = 4 , **CanErrorr_Crc** = 5 , **CanErrorr_Unknown** = 6 }
- enum **CanSendRet** {
CanSend_Succ , **CanSend_DataTooLong** , **CanSend_NotIdle** , **CanSend_Timeout** ,
CanSend_STBFull }
- enum **CanRecvRet** { **CanRecv_Succ** , **CanRecv_NoData** }
- enum **STBTransOption** { **STB_TransNo** , **STB_TransOne** , **STB_TransAll** }
- enum **STBTransMode** { **STB_FIFOmode** , **STB_PriorityMode** }
- enum **TBSelOption** { **TBSel_PTB** = 0 , **TBSel_STB** = 1 }
- enum **IDFilterOption** { **IDFilter_St.Ex** , **IDFilter_St** , **IDFilter.Ex** }

函数

- void **CAN_Init** (u8 BTR3, u8 BTR2, u8 BTR1, u8 BTR0)
 函数名称: *void CAN_Init(u8 BTR3,u8 BTR2,u8 BTR1,u8 BTR0)*
- void **CAN_StructInit** (**CAN_InitTypeDef** *this)
 CAN配置结构体初始化, 并设置默认参数
- void **CAN_Init1** (**CAN_InitTypeDef** *this)
 CAN初始化函数,采用2.0B/FD协议 输入参数: **CAN_InitTypeDef***: 初始化参数
- **CanSendRet CAN_Send** (**CANTxMsg** *tx_msg)
- **CanSendRet CAN_Send_Trig** (**STBTransOption** stb.trans)
 CANFD数据发送函数, 采用CAN-FD协议, STB发送。默认STB为FIFO模式。输入参数: *id*:发送id *ide*: 标准/扩展帧 *rtr*:远程帧/数据帧 *fdf*:CAN2.0/CANFD *brs*:切换波特率 **msg*:发送的数据 *len*:发送字节长度 输出参数: 无 返回值: 0: 发送成功 !0: 发送失败 其它说明: *id*:标准ID(11位)/扩展ID(11位+18位) *ide*:0,标准帧;1,扩展帧 *rtr*:0,数据帧;1,遥控帧 *fdf*:0,CAN2.0;1,CANFD *brs*:0,不切换波特率;1,切换波特率 *option*:0,本次不发送;1,发送一帧;2,发送所有帧
- **CanSendRet My_CAN_Send_Msg** (u32 id, u8 ide, u8 rtr, u8 *msg, u8 len)
 CANFD数据发送函数, 采用CAN-FD协议, PTB发送 输入参数: *id*: 发送id *ide*: 标准/扩展帧 *rtr*: 远程帧/数据帧 *fdf*:CAN2.0/CANFD *brs*:切换波特率 **msg*: 发送的数据 *len*:发送字节长度 输出参数: 无 返回值: 0: 发送成功 !0: 发送失败 其它说明: *id*:标准ID(11位)/扩展ID(11位+18位) *ide*:0,标准帧;1,扩展帧 *rtr*:0,数据帧;1,遥控帧 *fdf*:0,CAN2.0;1,CANFD *brs*:0,不切换波特率;1,切换波特率 *len*:要发送的数据长度, CAN2.0最大8字节, CANFD最大64字节
- **CanRecvRet CAN_Recv** (**CANRxMsg** *rx_msg)
 CAN接收数据 输入参数: **rx_msg*: 接收数据存储对象 返回值: **CanRecvRet**
- **CanRecvRet CAN_Receive_Msg** (u32 *id, u8 *len, u8 *ide, u8 *rtr, u8 *buf)
 函数名称: *u8 CAN_Receive_Msg(u32* id,u8* len ,u8* ide,u8* rtr,u8 *buf)*
- void **CAN_Send_Cancel** (**TBSelOption** tbsel)
- void **IDx_Filter** (u8 x, u32 id, u32 mask, **IDFilterOption** ide_filter)
 CAN IDx接收滤波函数 输入参数: *x*: 滤波通道号。 *id*: ID值。 *mask*: 屏蔽位。 *bit*=0表示需要位匹配, *bit*=1表示不需要匹配。 *ide_filter*: **IDFilter_St.Ex** / **IDFilter_St** / **IDFilter.Ex**
- void **IDx_FilterCmd** (u8 x, FuncState state)
 CAN接收过滤器使能与禁止
- void **CAN_LBMICmd** (FuncState state)
 内部回环模式使能/禁止
- void **CAN_LBMECmd** (FuncState state)
 外部回环模式使能/禁止
- void **CAN_SS_Enable** (**TBSelOption** tbsel, FuncState state)
 单次传输模式使能/禁止
- void **CAN_Enable** (void)
 打开CAN模块时钟

- void [CAN_Disable](#) (void)
关闭CAN模块时钟
- void [CAN_Reset](#) (void)
CAN模块进入复位模式
- void [CAN_SetSTBMode](#) (STBTransMode tsMode)
- u8 [CAN_STBFullGet](#) (void)
- u8 [CAN_RxErrorCntGet](#) (void)
获取can接收错误计数
- u8 [CAN_TxErrorCntGet](#) (void)
获取can发送错误计数
- u8 [CAN_ErrorPassiveGet](#) (void)
获取can的error passive state
- u8 [CAN_BusoffGet](#) (void)
获取can的bus off state
- [CANErrorKind](#) [CAN_KindOfErrorGet](#) (void)
获取can的kind of error，错误类别读取
- u8 [CAN_ArbitrationLostCap](#) (void)
获取can仲裁失败位置
- u8 [CAN_ROverflowGet](#) (void)
获取can的Rx Buffer是否溢出

变量

- [CANRxMsg](#) g_rxMsg

5.3.1 详细描述

文件名称: LKS32MC07x_can.h
文件标识: 无
内容摘要: CAN外设驱动程序
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_can.h](#) 中定义.

5.3.2 宏定义说明

5.3.2.1 CAN

```
#define CAN ((CAN_TypeDef *) CAN0_BASE)
```

在文件 [lks32mc07x.can.h](#) 第 69 行定义.

5.3.2.2 CAN_ERRINT_ALIE

```
#define CAN_ERRINT_ALIE BIT3
```

[11] 仲裁失败中断使能 (Arbitration Lost Interrupt Enable) 1: 使能0: 禁止

在文件 [lks32mc07x.can.h](#) 第 91 行定义.

5.3.2.3 CAN_ERRINT_ALIF

```
#define CAN_ERRINT_ALIF BIT2
```

[10] 仲裁失败中断标志位 (Arbitration Lost Interrupt Flag) 1: 仲裁失败, 写 1 清零 0: 仲裁成功

在文件 [lks32mc07x.can.h](#) 第 108 行定义.

5.3.2.4 CAN_ERRINT_BEIE

```
#define CAN_ERRINT_BEIE BIT1
```

[8] 总线错误中断使能 (Bus Error Interrupt Enable) 1: 使能0: 禁止

在文件 [lks32mc07x.can.h](#) 第 92 行定义.

5.3.2.5 CAN_ERRINT_BEIF

```
#define CAN_ERRINT_BEIF BIT0
```

[8] 总线错误中断标志 (Bus Error Interrupt Flag) 1: 总线错误, 写 1 清零0: 无总线错误

在文件 [lks32mc07x.can.h](#) 第 109 行定义.

5.3.2.6 CAN_ERRINT_EPASS

```
#define CAN_ERRINT_EPASS BIT6
```

[14] CAN 设备处于被动错误状态1: CAN 设备处于被动错误状态0: CAN 设备处于主动错误状态

在文件 [lks32mc07x.can.h](#) 第 106 行定义.

5.3.2.7 CAN_ERRINT_EPIE

```
#define CAN_ERRINT_EPIE BIT5
```

[13] 被动错误中断使能（Error Passive Interrupt Enable） 1：使能0：禁止

在文件 [lks32mc07x.can.h](#) 第 90 行定义.

5.3.2.8 CAN_ERRINT_EPIF

```
#define CAN_ERRINT_EPIF BIT4
```

[12] 被动错误中断标志（Error Passive Interrupt Flag） 1：发生主动错误到被动错误（或者相反）的改变，写 1 清零0：未发生

在文件 [lks32mc07x.can.h](#) 第 107 行定义.

5.3.2.9 CAN_ERRINT_EWARN

```
#define CAN_ERRINT_EWARN BIT7
```

[15] 错误计数值超过门限中断标志位1： RECNT 或者 TECNT 大于等于 EWL 设定值，写 1 清零0： RECNT 或者 TECNT 小于 EWL 设定值

在文件 [lks32mc07x.can.h](#) 第 105 行定义.

5.3.2.10 CAN_MODE_LBME

```
#define CAN_MODE_LBME BIT6
```

[6] 外部回环模式使能位

在文件 [lks32mc07x.can.h](#) 第 74 行定义.

5.3.2.11 CAN_MODE_LBMI

```
#define CAN_MODE_LBMI BIT5
```

[5] 内部回环模式使能位

在文件 [lks32mc07x.can.h](#) 第 75 行定义.

5.3.2.12 CAN_MODE_LOM

```
#define CAN_MODE_LOM BIT8
```

[8] 监听模式使能位

在文件 [lks32mc07x.can.h](#) 第 72 行定义.

5.3.2.13 CAN_MODE_RESET

```
#define CAN_MODE_RESET BIT7
```

[7] 模块复位模式

在文件 [lks32mc07x_can.h](#) 第 73 行定义.

5.3.2.14 CAN_MODE_TPSS

```
#define CAN_MODE_TPSS BIT4
```

[4] PTB 单次传输模式使能位

在文件 [lks32mc07x_can.h](#) 第 76 行定义.

5.3.2.15 CAN_MODE_TSSS

```
#define CAN_MODE_TSSS BIT3
```

[3] STB 单次传输模式使能位

在文件 [lks32mc07x_can.h](#) 第 77 行定义.

5.3.2.16 CAN_RTIE_EIE

```
#define CAN_RTIE_EIE BIT1
```

[1] 错误中断使能 (Error Interrupt Enable)

在文件 [lks32mc07x_can.h](#) 第 86 行定义.

5.3.2.17 CAN_RTIE_RAFIE

```
#define CAN_RTIE_RAFIE BIT4
```

[4] Rx Buffer 将满中断使能 (Rx Buffer Almost Full Interrupt Enable)

在文件 [lks32mc07x_can.h](#) 第 83 行定义.

5.3.2.18 CAN_RTIE_RFIE

```
#define CAN_RTIE_RFIE BIT5
```

[5] Rx Buffer 满中断使能 (Rx Buffer Full Interrupt Enable)

在文件 [lks32mc07x_can.h](#) 第 82 行定义.

5.3.2.19 CAN_RTIE_RIE

```
#define CAN_RTIE_RIE BIT7
```

[7] 接收中断使能 (Receive Interrupt Enable)

在文件 [lks32mc07x_can.h](#) 第 80 行定义.

5.3.2.20 CAN_RTIE_ROIE

```
#define CAN_RTIE_ROIE BIT6
```

[6] 接收溢出中断使能 (Receive Overrun Interrupt Enable)

在文件 [lks32mc07x_can.h](#) 第 81 行定义.

5.3.2.21 CAN_RTIE_TPIE

```
#define CAN_RTIE_TPIE BIT3
```

[3] PTB 发送中断使能 (Transmission Primary Interrupt Enable)

在文件 [lks32mc07x_can.h](#) 第 84 行定义.

5.3.2.22 CAN_RTIE_TSFF

```
#define CAN_RTIE_TSFF BIT0
```

[0] Tx Buffer 标志位 1: STB Slot 被全部填满 0: STB Slot 没有被全部填满

在文件 [lks32mc07x_can.h](#) 第 87 行定义.

5.3.2.23 CAN_RTIE_TSIE

```
#define CAN_RTIE_TSIE BIT2
```

[2] STB 发送中断使能 (Transmission Secondary Interrupt Enable)

在文件 [lks32mc07x_can.h](#) 第 85 行定义.

5.3.2.24 CAN_RTIF_AIF

```
#define CAN_RTIF_AIF BIT0
```

[0] 取消发送中断标志 (Abort Interrupt Flag) 1: 通过 TPA 和 TSA 请求的发送消息被成功取消。写 1 清零 0: 未取消发送数据 AIF 没有对应的使能寄存器

在文件 [lks32mc07x_can.h](#) 第 102 行定义.

5.3.2.25 CAN_RTIF_EIF

```
#define CAN_RTIF_EIF BIT1
```

[1] 错误中断标志 (Error Interrupt Flag) 1: 错误计数器的值发生变化, 大于或者小于错误警告寄存器的设定值。写 1 清零 0: 无错误计数器的值从小于错误警告寄存器的设定值变为大于设定值, 或者从大于设定值变为小于设定值, 均会触发中断标志。另外, 进入 Bus Off 或从 Bus Off 退出, 也会触发。

在文件 [lks32mc07x.can.h](#) 第 101 行定义。

5.3.2.26 CAN_RTIF_RAFIF

```
#define CAN_RTIF_RAFIF BIT4
```

[4] Rx Buffer 将满中断标志位 (Rx Buffer Almost Full Interrupt Flag) 1: 被填充的 Rx Buffer Slot 数目大于等于 AFWL 设定值 0: 被填充的 Rx Buffer Slot 数目小于 AFWL 设定值

在文件 [lks32mc07x.can.h](#) 第 98 行定义。

5.3.2.27 CAN_RTIF_RFIF

```
#define CAN_RTIF_RFIF BIT5
```

[5] Rx Buffer 满中断标志位 (Rx Buffer Full Interrupt Flag) 1: Rx Buffer 满, 写 1 清零 0: Rx Buffer 未满

在文件 [lks32mc07x.can.h](#) 第 97 行定义。

5.3.2.28 CAN_RTIF_RIF

```
#define CAN_RTIF_RIF BIT7
```

[7] 接收中断标志位 (Receive Interrupt Flag) 1: 接收到有效帧 (数据帧或者远程帧), 写 1 清零 0: 没有接收到有效帧

在文件 [lks32mc07x.can.h](#) 第 95 行定义。

5.3.2.29 CAN_RTIF_ROIF

```
#define CAN_RTIF_ROIF BIT6
```

[6] 接收溢出中断标志位 (Receive Overrun Interrupt Flag) 1: Rx Buffer 至少有一帧未读走的数据被覆盖 0: Rx Buffer 无覆盖 溢出时 ROIF 和 RFIF 同时置 1, 写 1 清零

在文件 [lks32mc07x.can.h](#) 第 96 行定义。

5.3.2.30 CAN_RTIF_TPIF

```
#define CAN_RTIF_TPIF BIT3
```

[3] PTB 发送中断标志 (Transmission Primary Interrupt Flag) 1: 触发 PTB 发送, 发送成功完成。写 1 清零 0: 无 PTB 发送请求, 无完成标志

在文件 [lks32mc07x.can.h](#) 第 99 行定义。

5.3.2.31 CAN_RTIF_TSIF

```
#define CAN_RTIF_TSIF BIT2
```

[2] STB 发送中断标志 (Transmission Secondary Interrupt Flag) 1: 触发 STB 发送, 发送成功完成。写 1 清零 0: 无 STB 发送请求, 无完成标志

在文件 [lks32mc07x_can.h](#) 第 100 行定义.

5.3.2.32 CANRXMSG_HEADLEN

```
#define CANRXMSG_HEADLEN 8 /* CANRxMsg中id~len的字节数 */
```

在文件 [lks32mc07x_can.h](#) 第 189 行定义.

5.3.2.33 ID0_Filter

```
#define ID0_Filter(  
    a,  
    b,  
    c ) IDx_Filter(0, a, b, c)
```

在文件 [lks32mc07x_can.h](#) 第 244 行定义.

5.3.2.34 ID1_Filter

```
#define ID1_Filter(  
    a,  
    b,  
    c ) IDx_Filter(1, a, b, c)
```

在文件 [lks32mc07x_can.h](#) 第 245 行定义.

5.3.2.35 ID2_Filter

```
#define ID2_Filter(  
    a,  
    b,  
    c ) IDx_Filter(2, a, b, c)
```

在文件 [lks32mc07x_can.h](#) 第 246 行定义.

5.3.2.36 ID3_Filter

```
#define ID3_Filter(  
    a,  
    b,  
    c ) IDx_Filter(3, a, b, c)
```

在文件 [lks32mc07x_can.h](#) 第 247 行定义.

5.3.3 枚举类型说明

5.3.3.1 CANErrorKind

```
enum CANErrorKind
```

枚举值

CanError_None	无错误
CanErrr_Bit	位错误
CanErrr_Form	形式错误
CanErrr_Stuff	填充错误
CanErrr_Ack	应答错误
CanErrr_Crc	CRC 错误
CanErrr_Unknown	其他错误

在文件 [lks32mc07x.can.h](#) 第 111 行定义.

5.3.3.2 CanRecvRet

enum [CanRecvRet](#)

枚举值

CanRecv_Succ	
CanRecv_NoData	

在文件 [lks32mc07x.can.h](#) 第 142 行定义.

5.3.3.3 CanSendRet

enum [CanSendRet](#)

枚举值

CanSend_Succ	
CanSend_DataTooLong	
CanSend_NotIdle	
CanSend_Timeout	
CanSend_STBFull	

在文件 [lks32mc07x.can.h](#) 第 132 行定义.

5.3.3.4 IDEFilterOption

enum [IDEFilterOption](#)

枚举值

IDEFilter_St_Ext	
IDEFilter_St	
IDEFilter_Ext	

在文件 [lks32mc07x_can.h](#) 第 169 行定义.

5.3.3.5 STBTransMode

enum [STBTransMode](#)

枚举值

STB_FIFOMode	
STB_PriorityMode	

在文件 [lks32mc07x_can.h](#) 第 156 行定义.

5.3.3.6 STBTransOption

enum [STBTransOption](#)

枚举值

STB_TransNo	
STB_TransOne	
STB_TransAll	

在文件 [lks32mc07x_can.h](#) 第 149 行定义.

5.3.3.7 TBSeIOption

enum [TBSeIOption](#)

枚举值

TBSeI_PTB	
TBSeI_STB	

在文件 [lks32mc07x_can.h](#) 第 162 行定义.

5.3.4 函数说明

5.3.4.1 CAN_ArbitrationLostCap()

```
u8 CAN_ArbitrationLostCap (
    void )
```

获取can仲裁失败位置

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 870 行定义.

5.3.4.2 CAN_BusoffGet()

```
u8 CAN_BusoffGet (
    void )
```

获取can的bus off state

更新记录

在文件 [lks32mc07x.can.c](#) 第 851 行定义.

5.3.4.3 CAN_Disable()

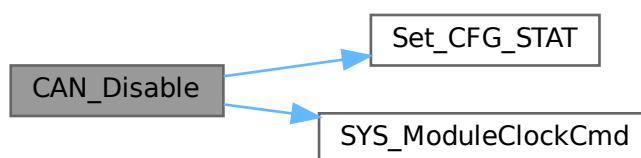
```
void CAN_Disable (
    void )
```

关闭CAN模块时钟

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 777 行定义.

函数调用图:



5.3.4.4 CAN_Enable()

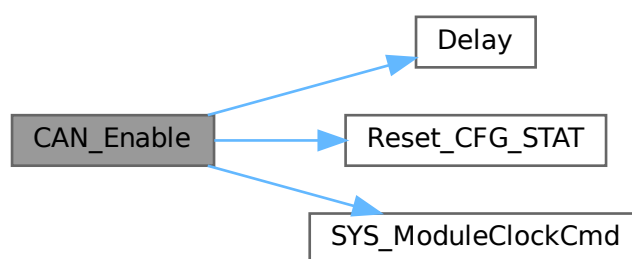
```
void CAN_Enable (
    void )
```

打开CAN模块时钟

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x_can.c](#) 第 767 行定义.

函数调用图:



5.3.4.5 CAN_ErrorPassiveGet()

```
u8 CAN_ErrorPassiveGet (
    void )
```

获取can的error passive state

更新记录

在文件 [lks32mc07x_can.c](#) 第 842 行定义.

5.3.4.6 CAN_Init()

```
void CAN_Init (
    u8 BTR3,
    u8 BTR2,
    u8 BTR1,
    u8 BTR0 )
```

函数名称: void [CAN_Init\(u8 BTR3,u8 BTR2,u8 BTR1,u8 BTR0\)](#)

功能描述: CAN初始化函数,采用2.0B协议

参见

被调用函数: 无

参数

输入参数:	波特率 AN_SBAUD.PRESC[7:0] CAN_SBAUD.SJW[6:0] CAN_SBAUD.SEG2[6:0] CAN_SBAUD.SEG1[7:0] Sample Poin 1Mbps 0x05 0x02 0x05 0x08 63% 800Kbps 0x0B 0x01 0x03 0x04 60% 666Kbps 0x0B 0x01 0x03 0x06 67% 500Kbps 0x0B 0x02 0x05 0x08 63% 400Kbps 0x0B 0x02 0x06 0x0B 65% 250Kbps 0x17 0x02 0x05 0x08 63% 200Kbps 0x17 0x02 0x06 0x0B 65% 125Kbps 0x2F 0x02 0x05 0x08 63% 100Kbps 0x2F 0x02 0x06 0x0B 65% 80Kbps 0x35 0x02 0x06 0x0B 65% 50Kbps 0x5F 0x02 0x06 0x0B 65% 40Kbps 0x77 0x02 0x06 0x0B 65% 25Kbps 0xBF 0x02 0x06 0x0B 65% 20Kbps 0xEF 0x02 0x06 0x0B 65% 10Kbps 0xEF 0x06 0x0D 0x18 65% 05Kbps 0xEF 0x0D 0x1B 0x32 65%
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
CAN_Init (0x0B, 0x02, 0x05, 0x08); //CAN波特率500Khz
```

Date	Version	Author	Description
2023年2月23日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_can.c](#) 第 130 行定义.

5.3.4.7 CAN_Init1()

```
void CAN_Init1 (
```

```
CAN_InitTypeDef * this )
```

CAN初始化函数,采用2.0B/FD协议 输入参数: CAN_InitTypeDef*: 初始化参数

更新记录 V1.0 2024/07/03 YangZJ 创建

在文件 [lks32mc07x-can.c](#) 第 41 行定义.

函数调用图:



5.3.4.8 CAN_KindOfErrorGet()

```
CANErrorKind CAN_KindOfErrorGet (
    void )
```

获取can的kind of error, 错误类别读取

更新记录

在文件 [lks32mc07x-can.c](#) 第 860 行定义.

5.3.4.9 CAN_LBMECmd()

```
void CAN_LBMECmd (
    FuncState state )
```

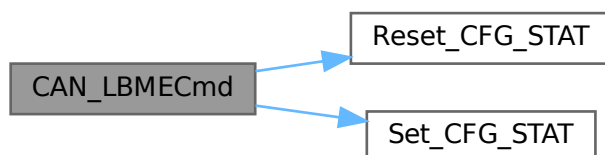
外部回环模式使能/禁止

更新记录

自发自收，自应答，需将芯片RX和TX接入CAN收发器，收发器可以不挂入总线。应在CAN初始化和ID滤波设置之后再使能。

在文件 [lks32mc07x_can.c](#) 第 717 行定义.

函数调用图:



5.3.4.10 CAN_LBMICmd()

```
void CAN_LBMICmd (
    FuncState state )
```

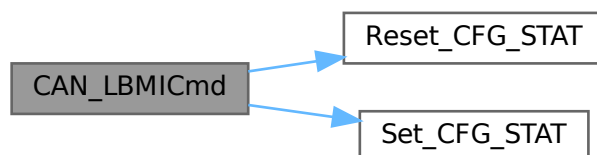
内部回环模式使能/禁止

更新记录

自发自收，无需自应答，芯片RX和TX引脚可悬空。应在CAN初始化和ID滤波设置之后再使能。

在文件 [lks32mc07x_can.c](#) 第 699 行定义.

函数调用图:



5.3.4.11 CAN_Receive_Msg()

```
CanRecvRet CAN_Receive_Msg (
    u32 * id,
    u8 * len,
    u8 * ide,
    u8 * rtr,
    u8 * buf )
```

函数名称: u8 CAN_Receive_Msg(u32* id,u8* len ,u8* ide,u8* rtr,u8 *buf)

功能描述: CAN轮询接收数据程序

参见
被调用函数: 无

参数

输入参数:	无
输出参数:	id: 接收帧id len: 接收数据长度 ide: 接收帧的类型, 标准/扩展帧 rtr: 接收帧的类型, 远程帧/数据帧 *buf: 接收帧的数据存储缓存区

返回
返 回 值: 0: 接收完成 !0: 接收失败

注解
其它说明: 1、id:标准ID(11位)/扩展ID(11位+18位)
2、ide:0,标准帧;1,扩展帧
3、rtr:0,数据帧;1,远程帧。

警告
无

示例代码:

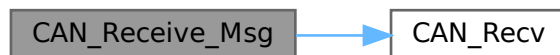
```
//can_par.id存储接收帧ID, can_par.len接收数据个数
can_par.ide存储接收帧类型标准/扩展帧,
can_par.rtr存储接收帧类型远程帧/数据帧, can_par.RX存储接收帧数据
CAN_Receive_Msg (&( can_par.id), &(can_par.len), &(can_par.ide), &(can_par.rtr), can_par.RX);
```

Date	Version	Author	Description
2023年2月23日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_can.c](#) 第 405 行定义.

函数调用图:



5.3.4.12 CAN_Recv()

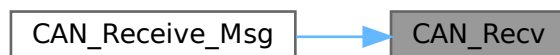
```
CanRecvRet CAN_Recv (  
    CANRxMsg * rx_msg )
```

CAN接收数据 输入参数: *rx_msg: 接收数据存储对象 返回值: CanRecvRet

更新记录

在文件 [lks32mc07x_can.c](#) 第 314 行定义.

这是这个函数的调用关系图:



5.3.4.13 CAN_Reset()

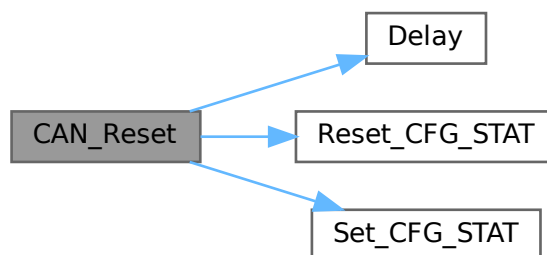
```
void CAN_Reset (  
    void )
```

CAN模块进入复位模式

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 786 行定义.

函数调用图:



5.3.4.14 CAN_ROverflowGet()

```
u8 CAN_ROverflowGet (  
    void )
```

获取can的Rx Buffer是否溢出

更新记录

在文件 [lks32mc07x.can.c](#) 第 879 行定义.

5.3.4.15 CAN_RxErrorCntGet()

```
u8 CAN_RxErrorCntGet (  
    void )
```

获取can接收错误计数

更新记录 V1.0 2022/09/19 YangZJ 创建

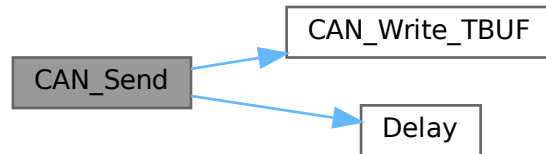
在文件 [lks32mc07x.can.c](#) 第 824 行定义.

5.3.4.16 CAN_Send()

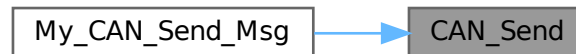
```
CanSendRet CAN_Send (
    CANTxMsg * tx_msg )
```

在文件 [lks32mc07x.can.c](#) 第 171 行定义.

函数调用图:



这是这个函数的调用关系图:



5.3.4.17 CAN_Send_Cancel()

```
void CAN_Send_Cancel (
    TBSeIOption tbsel )
```

在文件 [lks32mc07x.can.c](#) 第 296 行定义.

5.3.4.18 CAN_Send_Trig()

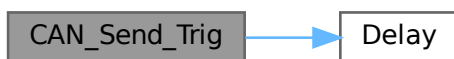
```
CanSendRet CAN_Send_Trig (
    STBTransOption stb_trans )
```

CANFD数据发送函数，采用CAN-FD协议，STB发送。默认STB为FIFO模式。输入参数: id:发送id ide:0:标准/扩展帧 rtr:远程帧/数据帧 fdf:CAN2.0/CANFD brs:切换波特率 *msg:发送的数据 len:发送字节长度 输出参数: 无 返回值: 0: 发送成功 !0: 发送失败 其它说明: id:标准ID(11位)/扩展ID(11位+18位) ide:0,标准帧;1,扩展帧 rtr:0,数据帧;1,遥控帧 fdf:0,CAN2.0;1,CANFD brs:0,不切换波特率;1,切换波特率 option:0,本次不发送;1,发送一帧;2,发送所有帧

更新记录 V1.0 2023/02/13 HuangMG 创建

在文件 [lks32mc07x_can.c](#) 第 257 行定义.

函数调用图:



5.3.4.19 CAN_SetSTBMode()

```
void CAN_SetSTBMode (
    STBTransMode tsMode )
```

在文件 [lks32mc07x_can.c](#) 第 797 行定义.

5.3.4.20 CAN_SS_Enable()

```
void CAN_SS_Enable (
    TBSelOption tbsel,
    FuncState state )
```

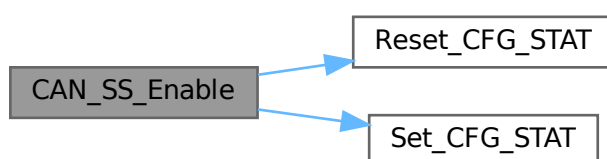
单次传输模式使能/禁止

更新记录

PTB/STB单次传输模式使能，使能后错误不重发。应在CAN初始化和ID滤波设置之后再使能。

在文件 [lks32mc07x_can.c](#) 第 737 行定义.

函数调用图:



5.3.4.21 CAN_STBFullGet()

```
u8 CAN_STBFullGet (
    void )
```

在文件 [lks32mc07x.can.c](#) 第 815 行定义.

5.3.4.22 CAN_StructInit()

```
void CAN_StructInit (
    CAN_InitTypeDef * this )
```

CAN配置结构体初始化，并设置默认参数

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 72 行定义.

5.3.4.23 CAN_TxErrorCntGet()

```
u8 CAN_TxErrorCntGet (
    void )
```

获取can发送错误计数

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 833 行定义.

5.3.4.24 IDx_Filter()

```
void IDx_Filter (
    u8 x,
    u32 id,
    u32 mask,
    IDEFilterOption ide_filter )
```

CAN IDx接收滤波函数 输入参数: x: 滤波通道号。 id: ID值。 mask: 屏蔽位。 bit=0表示需要位匹配, bit=1表示不需要匹配。 ide_filter: IDEFilter_St_Ex / IDEFilter_St / IDEFilter_Ex

更新记录

在文件 [lks32mc07x.can.c](#) 第 442 行定义.

函数调用图:



5.3.4.25 IDx_FilterCmd()

```
void IDx_FilterCmd (
    u8 x,
    FuncState state )
```

CAN接收过滤器使能与禁止

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x_can.c](#) 第 483 行定义.

5.3.4.26 My_CAN_Send_Msg()

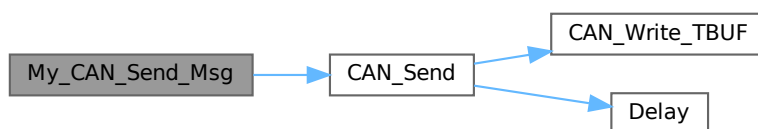
```
CanSendRet My_CAN_Send_Msg (
    u32 id,
    u8 ide,
    u8 rtr,
    u8 * msg,
    u8 len )
```

CANFD数据发送函数，采用CAN-FD协议，PTB发送 输入参数: id: 发送id ide: 标准/扩展帧 rtr: 远程帧/数据帧 fdf:CAN2.0/CANFD brs:切换波特率 *msg: 发送的数据 len:发送字节长度 输出参数: 无 返回值: 0: 发送成功 !0: 发送失败 其它说明: id:标准ID(11位)/扩展ID(11位+18位) ide:0,标准帧;1,扩展帧 rtr:0,数据帧;1,遥控帧 fdf:0,CAN2.0;1,CANFD brs:0,不切换波特率;1,切换波特率 len:要发送的数据长度，CAN2.0最大8字节，↔CANFD最大64字节

更新记录

在文件 [lks32mc07x_can.c](#) 第 230 行定义.

函数调用图:



5.3.5 变量说明

5.3.5.1 g_rxMsg

```
CANRxMsg g_rxMsg [extern]
```

在文件 [lks32mc07x_can.c](#) 第 18 行定义.

5.4 lks32mc07x_can.h

[浏览该文件的文档.](#)

```

00001
00014 #ifndef _LKS32MC07X_CAN_H_
00015 #define _LKS32MC07X_CAN_H_
00016
00018 #include "lks32mc07x.lib.h"
00019
00020 typedef struct
00021 {
00022     u32 id;
00023     u8 len;
00024     u8 ide;
00025     u8 rtr;
00026     u8 TX[8];
00027     u8 RX[8];
00028     u32 rx_sta;
00029 } can_parameter;
00030
00031 // extern can_parameter can_par;
00032
00033 typedef struct
00034 {
00035     __IO u32 RBUF[4];
00036     __IO u32 resv0[16];
00037
00038     __IO u32 TBUF[4];
00039     __IO u32 resv1[16];
00040
00041     __IO u8 CFG_STAT;
00042     __IO u8 TCMD;
00043     __IO u8 TCTRL;
00044     __IO u8 RCTRL;
00045
00046     __IO u8 RTIE;
00047     __IO u8 RTIF;
00048     __IO u8 ERRINT;
00049     __IO u8 LIMIT;
00050     __IO u32 SBAUD;
00051
00052     __IO u32 resv2;
00053
00054     __IO u8 EALCAP;
00055
00056     __IO u8 resv3;
00057
00058     __IO u8 RECNT;
00059     __IO u8 TECNT;
00060     __IO u8 ACFCTRL;
00061
00062     __IO u8 resv4;
00063
00064     __IO u16 ACFEN;
00065     __IO u32 ACF;
00066 } CAN_TypeDef;
00067
00068 #ifndef CAN
00069 #define CAN ((CAN_TypeDef *)CAN0_BASE)
00070 #endif
00071
00072 #define CAN_MODE_LOM BIT8
00073 #define CAN_MODE_RESET BIT7
00074 #define CAN_MODE_LBME BIT6
00075 #define CAN_MODE_LBMI BIT5
00076 #define CAN_MODE_TPSS BIT4
00077 #define CAN_MODE_TSSS BIT3
00078
00079 // RTIE中断使能定义
00080 #define CAN_RTIE_RIE BIT7
00081 #define CAN_RTIE_ROIE BIT6
00082 #define CAN_RTIE_RFIE BIT5
00083 #define CAN_RTIE_RAFIE BIT4
00084 #define CAN_RTIE_TPIE BIT3
00085 #define CAN_RTIE_TSIE BIT2
00086 #define CAN_RTIE_EIE BIT1
00087 #define CAN_RTIE_TSFF BIT0
00088
00089 // ERRINT中断使能定义
00090 #define CAN_ERRINT_EPIE BIT5
00091 #define CAN_ERRINT_ALIE BIT3
00092 #define CAN_ERRINT_BEIE BIT1
00093
00094 // RTIE中断标志位
00095 #define CAN_RTIF_RIF BIT7

```



```

00096 #define CAN_RTIF_ROIF BIT6
00097 #define CAN_RTIF_RFIF BIT5
00098 #define CAN_RTIF_RAFIF BIT4
00099 #define CAN_RTIF_TPIF BIT3
00100 #define CAN_RTIF_TSIF BIT2
00101 #define CAN_RTIF_EIF BIT1
00102 #define CAN_RTIF_AIF BIT0
00103
00104 // ERRINT中断标志位
00105 #define CAN_ERRINT_EWARN BIT7
00106 #define CAN_ERRINT_EPASS BIT6
00107 #define CAN_ERRINT_EPIF BIT4
00108 #define CAN_ERRINT_ALIF BIT2
00109 #define CAN_ERRINT_BEIF BIT0
00110
00111 typedef enum
00112 {
00113     CanError_None = 0,
00114     CanErrorr_Bit = 1,
00115     CanErrorr_Form = 2,
00116     CanErrorr_Stuff = 3,
00117     CanErrorr_Ack = 4,
00118     CanErrorr_Crc = 5,
00119     CanErrorr_Unknown = 6,
00120 } CANErrorKind;
00121
00122 // CAN模块初始化
00123 typedef struct
00124 {
00125     u32 SBaud; /* 2.0B波特率, 必须大于0。 */
00126
00127     u32 IE; /* 初始化时要使能的中断 */
00128     u32 ERRINT_IE; /* 错误中断使能 */
00129 } CAN_InitTypeDef;
00130
00131 /* CAN_Send执行结果 */
00132 typedef enum
00133 {
00134     CanSend_Succ, /* CAN发送成功 */
00135     CanSend_DataTooLong, /* CAN发送失败, 数据过长 */
00136     CanSend_NotIdle, /* CAN发送成功, 当前非空闲 */
00137     CanSend_Timeout, /* CAN发送超时 */
00138     CanSend_STBFull, /* STB缓冲区已满 */
00139 } CanSendRet;
00140
00141 /* CAN_Recv执行结果 */
00142 typedef enum
00143 {
00144     CanRecv_Succ, /* 执行成功, 取得数据 */
00145     CanRecv_NoData, /* 执行失败, 无数据 */
00146 } CanRecvRet;
00147
00148 /* STB发送时, 需要填入的触发参数 */
00149 typedef enum
00150 {
00151     STB_TransNo, /* 本次调用仅添加进STB, 暂不发送 */
00152     STB_TransOne, /* 本次调用添加到STB后, 发送一帧 */
00153     STB_TransAll, /* 本次调用添加到STB后, 发送所有帧 */
00154 } STBTransOption;
00155
00156 typedef enum
00157 {
00158     STB_FIFOMode, /* 发送缓冲区FIFO模式 */
00159     STB_PriorityMode, /* 发送缓冲区优先级模式 */
00160 } STBTransMode;
00161
00162 typedef enum
00163 {
00164     TBSel_PTB = 0, /* Tx Buffer 选择 PTB */
00165     TBSel_STB = 1, /* Tx Buffer 选择 STB */
00166 } TBSelOption;
00167
00168 /* 设置ID过滤时, 对标准帧和扩展帧的设置选项 */
00169 typedef enum
00170 {
00171     IDEFilter_St_Ex, /* 接收标准帧+扩展帧 */
00172     IDEFilter_St, /* 只接收标准帧 */
00173     IDEFilter_Ex, /* 只接收扩展帧 */
00174 } IDEFilterOption;
00175
00176 typedef struct
00177 {
00178     u32 id;
00179     u8 ide;
00180     u8 rtr;
00181
00182     u8 *buf;

```

```

00183     u8 len;
00184
00185     TBSELoption tbsel; /* TBSEL_PTB或TBSEL_STB。如果指定STB, CAN_Send()之后还需调用CAN_Send_Trig()触发发送 */
00186     u8 ss; /* 0-错误时自动重发, 1-单次发送不重发 */
00187 } CAN_TxMsg;
00188
00189 #define CAN_RXMSG_HEADLEN 8 /* CAN_RxMsg中id-len的字节数 */
00190 typedef struct
00191 {
00192     u32 id;
00193     u8 ide;
00194     u8 rtr;
00195     u8 valid; /* 是否有效, 接收成功valid=1, 处理完设置valid=0 */
00196
00197     u8 len;
00198     u8 buf[64];
00199 } CAN_RxMsg;
00200
00201 extern CAN_RxMsg g_rxMsg;
00202
00203 void CAN_Init(u8 BTR3, u8 BTR2, u8 BTR1, u8 BTR0);
00204
00205 void CAN_StructInit(CAN_InitTypeDef *this);
00206 void CAN_Init1(CAN_InitTypeDef *this);
00207
00208 CanSendRet CAN_Send(CAN_TxMsg *tx_msg);
00209 CanSendRet CAN_Send_Trig(STBTransOption stb_trans); /* 触发STB缓冲区数据发出 */
00210
00211 CanSendRet My_CAN_Send_Msg(u32 id, u8 ide, u8 rtr, u8 *msg, u8 len);
00212
00213 CanRecvRet CAN_Recv(CAN_RxMsg *rx_msg);
00214 CanRecvRet CAN_Receive_Msg(u32 *id, u8 *len, u8 *ide, u8 *rtr, u8 *buf);
00215
00216 void CAN_Send_Cancel(TBSELoption tbsel);
00217
00218 void IDx_Filter(u8 x, u32 id, u32 mask, IDEFilterOption ide_filter);
00219 void IDx_FilterCmd(u8 x, FuncState state);
00220
00221 void CAN_LBMCmd(FuncState state); /* 内部回环模式使能/禁止 */
00222 void CAN_LBMECmd(FuncState state); /* 外部回环模式使能/禁止 */
00223
00224 void CAN_SS_Enable(TBSELoption tbsel, FuncState state); /* 单次传输模式使能 */
00225
00226 void CAN_Enable(void);
00227 void CAN_Disable(void);
00228
00229 void CAN_Reset(void);
00230
00231 void CAN_Set_STB_Mode(STBTransMode tsMode);
00232
00233 u8 CAN_STB_FullGet(void);
00234
00235 u8 CAN_Rx_ErrorCntGet(void);
00236 u8 CAN_Tx_ErrorCntGet(void);
00237 u8 CAN_ErrorPassiveGet(void);
00238 u8 CAN_BusOffGet(void);
00239
00240 CANErrorKind CAN_KindOfErrorGet(void);
00241 u8 CAN_ArbitrationLostCap(void);
00242 u8 CAN_R0verflowGet(void);
00243
00244 #define ID0_Filter(a, b, c) IDx_Filter(0, a, b, c)
00245 #define ID1_Filter(a, b, c) IDx_Filter(1, a, b, c)
00246 #define ID2_Filter(a, b, c) IDx_Filter(2, a, b, c)
00247 #define ID3_Filter(a, b, c) IDx_Filter(3, a, b, c)
00248
00249 #endif

```

5.5 include/lks32mc07x_clu.h 文件参考

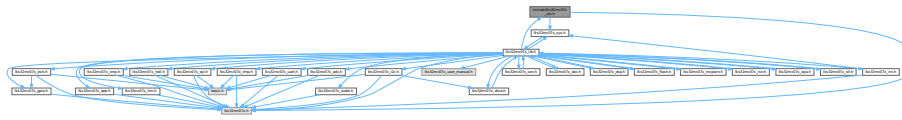
CLU库函数头文件

```

#include "lks32mc07x.h"
#include "lks32mc07x_sys.h"

```

lks32mc07x.clu.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct **CLU_TypeDef**
 < Includes -----
- struct **CLU_InitTypeDef**

宏定义

- #define **CLU_MODEL_CLU0** BIT0
 CLU0 CL0里面有4个CLU 这里用BIT0 BIT1 BIT2 BIT3做区分 不可修改
- #define **CLU_MODEL_CLU1** BIT1
 CLU1
- #define **CLU_MODEL_CLU2** BIT2
 CLU2
- #define **CLU_MODEL_CLU3** BIT3
 CLU3
- #define **CLU_MXA_CLU0_CLU0OUT** 0
 使用 CLU0OUT 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_CLU1OUT** 1
 使用 CLU1OUT 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_CLU2OUT** 2
 使用 CLU2OUT 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_CLU3OUT** 3
 使用 CLU3OUT 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_Timer0ch0ovfl** 4
 使用 Timer0ch0ovfl 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_Timer1ch0ovfl** 5
 使用 Timer1ch0ovfl 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_PWM0** 6
 使用 PWM0 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_PWM1** 7
 使用 PWM1 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_P0_0** 8
 使用 P0_0 作为CLU0.A的输入
- #define **CLU_MXA_CLU0_P0_3** 9

- 使用 *P0_3* 作为 *CLU0.A* 的输入
- #define *CLU_MXA_CLU0.P0_5* 10
使用 *P0_5* 作为 *CLU0.A* 的输入
- #define *CLU_MXA_CLU0.P0_7* 11
使用 *P0_7* 作为 *CLU0.A* 的输入
- #define *CLU_MXA_CLU0.P2_12* 12
使用 *P2_12* 作为 *CLU0.A* 的输入
- #define *CLU_MXA_CLU0.P0_12* 13
使用 *P0_12* 作为 *CLU0.A* 的输入
- #define *CLU_MXA_CLU0.P0_14* 14
使用 *P0_14* 作为 *CLU0.A* 的输入
- #define *CLU_MXA_CLU0.P1_0* 15
使用 *P1_0* 作为 *CLU0.A* 的输入
- #define *CLU_MXA_CLU1.CLU0OUT* 0
使用 *CLU0OUT* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.CLU1OUT* 1
使用 *CLU1OUT* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.CLU2OUT* 2
使用 *CLU2OUT* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.CLU3OUT* 3
使用 *CLU3OUT* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.Timer0ch1ovfl* 4
使用 *Timer0ch1ovfl* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.Timer1ch1ovfl* 5
使用 *Timer1ch1ovfl* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.PWM2* 6
使用 *PWM2* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.PWM3* 7
使用 *PWM3* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.P0_0* 8
使用 *P0_0* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.P0_2* 9
使用 *P0_2* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.P0_5* 10
使用 *P0_5* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.P0_6* 11
使用 *P0_6* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.P2_12* 12
使用 *P2_12* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.P0_11* 13
使用 *P0_11* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.P0_14* 14
使用 *P0_14* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU1.P0_15* 15
使用 *P0_15* 作为 *CLU1.A* 的输入
- #define *CLU_MXA_CLU2.CLU0OUT* 0
使用 *CLU0OUT* 作为 *CLU2.A* 的输入
- #define *CLU_MXA_CLU2.CLU1OUT* 1
使用 *CLU1OUT* 作为 *CLU2.A* 的输入
- #define *CLU_MXA_CLU2.CLU2OUT* 2
使用 *CLU2OUT* 作为 *CLU2.A* 的输入

- `#define CLU_MXA_CLU2_CLU3OUT 3`
使用 `CLU3OUT` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_Timer2ch0ovfl 4`
使用 `Timer2ch0ovfl` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_Timer3ch0ovfl 5`
使用 `Timer3ch0ovfl` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_PWM0 6`
使用 `PWM0` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_PWM2 7`
使用 `PWM2` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_P2_8 8`
使用 `P2.8` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_P1_13 9`
使用 `P1_13` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_P1_15 10`
使用 `P1_15` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_P1_10 11`
使用 `P1_10` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_P2_1 12`
使用 `P2_1` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_P2_4 13`
使用 `P2.4` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_P2_6 14`
使用 `P2.6` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU2_P2_14 15`
使用 `P2.14` 作为 `CLU2.A` 的输入
- `#define CLU_MXA_CLU3_CLU0OUT 0`
使用 `CLU0OUT` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_CLU1OUT 1`
使用 `CLU1OUT` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_CLU2OUT 2`
使用 `CLU2OUT` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_CLU3OUT 3`
使用 `CLU3OUT` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_Timer2ch1ovfl 4`
使用 `Timer2ch1ovfl` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_Timer3ch1ovfl 5`
使用 `Timer3ch1ovfl` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_PWM1 6`
使用 `PWM1` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_PWM3 7`
使用 `PWM3` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_P2_8 8`
使用 `P2.8` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_P3_9 9`
使用 `P3.9` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_P1_15 10`
使用 `P1_15` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_P2_0 11`
使用 `P2.0` 作为 `CLU3.A` 的输入
- `#define CLU_MXA_CLU3_P2_1 12`

- 使用 *P2_1* 作为 *CLU3.A* 的输入
- #define *CLU_MXA_CLU3.P2_3* 13
使用 *P2_3* 作为 *CLU3.A* 的输入
- #define *CLU_MXA_CLU3.P2_6* 14
使用 *P2_6* 作为 *CLU3.A* 的输入
- #define *CLU_MXA_CLU3.P2_13* 15
使用 *P2_13* 作为 *CLU3.A* 的输入
- #define *CLU_MXB_CLU0.CLU0OUT* 0
使用 *CLU0OUT* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.CLU1OUT* 1
使用 *CLU1OUT* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.CLU2OUT* 2
使用 *CLU2OUT* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.CLU3OUT* 3
使用 *CLU3OUT* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.ADBUSY0* 4
使用 *ADBUSY0* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.ADBUSY1* 5
使用 *ADBUSY1* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.PWM3* 6
使用 *PWM3* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.SPIMOSI* 7
使用 *SPIMOSI* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.P0_2* 8
使用 *P0_2* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.P0_4* 9
使用 *P0_4* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.P0_6* 10
使用 *P0_6* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.P2_11* 11
使用 *P2_11* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.P0_11* 12
使用 *P0_11* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.P0_13* 13
使用 *P0_13* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.P0_15* 14
使用 *P0_15* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU0.P2_7* 15
使用 *P2_7* 作为 *CLU0.B* 的输入
- #define *CLU_MXB_CLU1.CLU0OUT* 0
使用 *CLU0OUT* 作为 *CLU1.B* 的输入
- #define *CLU_MXB_CLU1.CLU1OUT* 1
使用 *CLU1OUT* 作为 *CLU1.B* 的输入
- #define *CLU_MXB_CLU1.CLU2OUT* 2
使用 *CLU2OUT* 作为 *CLU1.B* 的输入
- #define *CLU_MXB_CLU1.CLU3OUT* 3
使用 *CLU3OUT* 作为 *CLU1.B* 的输入
- #define *CLU_MXB_CLU1.ADBUSY0* 4
使用 *ADBUSY0* 作为 *CLU1.B* 的输入
- #define *CLU_MXB_CLU1.ADBUSY1* 5
使用 *ADBUSY1* 作为 *CLU1.B* 的输入

- `#define CLU_MXB_CLU1_PWM1 6`
使用 *PWM1* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_SPIMISO 7`
使用 *SPIMISO* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_P0_3 8`
使用 *P0_3* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_P0_4 9`
使用 *P0_4* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_P0_7 10`
使用 *P0_7* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_P2_11 11`
使用 *P2_11* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_P0_12 12`
使用 *P0_12* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_P0_13 13`
使用 *P0_13* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_P1_0 14`
使用 *P1_0* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU1_P2_7 15`
使用 *P2_7* 作为 *CLU1.B* 的输入
- `#define CLU_MXB_CLU2_CLU0OUT 0`
使用 *CLU0OUT* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_CLU1OUT 1`
使用 *CLU1OUT* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_CLU2OUT 2`
使用 *CLU2OUT* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_CLU3OUT 3`
使用 *CLU3OUT* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_PWM1 4`
使用 *PWM1* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_PWM3 5`
使用 *PWM3* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_I2CSDA 6`
使用 *I2CSDA* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_SPICLK 7`
使用 *SPICLK* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_P3_9 8`
使用 *P3_9* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_P1_14 9`
使用 *P1_14* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_P2_0 10`
使用 *P2_0* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_P1_11 11`
使用 *P1_11* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_P2_3 12`
使用 *P2_3* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_P2_5 13`
使用 *P2_5* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_P2_13 14`
使用 *P2_13* 作为 *CLU2.B* 的输入
- `#define CLU_MXB_CLU2_P2_15 15`

- 使用 *P2_15* 作为 *CLU2.B* 的输入
- #define *CLU_MXB_CLU3_CLU0OUT* 0
使用 *CLU0OUT* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_CLU1OUT* 1
使用 *CLU1OUT* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_CLU2OUT* 2
使用 *CLU2OUT* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_CLU3OUT* 3
使用 *CLU3OUT* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_PWM0* 4
使用 *PWM0* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_PWM2* 5
使用 *PWM2* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_I2CSCL* 6
使用 *I2CSCL* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_SPICSN* 7
使用 *SPICSN* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_P1_13* 8
使用 *P1_13* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_P1_14* 9
使用 *P1_14* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_P1_10* 10
使用 *P1_10* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_P1_11* 11
使用 *P1_11* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_P2_4* 12
使用 *P2_4* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_P2_5* 13
使用 *P2_5* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_P2_14* 14
使用 *P2_14* 作为 *CLU3.B* 的输入
- #define *CLU_MXB_CLU3_P2_15* 15
使用 *P2_15* 作为 *CLU3.B* 的输入
- #define *CLU_FN_0* 0x0
*CLU*真值表的最小相 0 注意最小相之间只能用 | (C语言按位或)
- #define *CLU_FN_1* 0xff
*CLU*真值表的最小相 1 注意最小相之间只能用 | (C语言按位或)
- #define *CLU_FN_A* 0xf0
*CLU*真值表的最小相 a 注意最小相之间只能用 | (C语言按位或)
- #define *CLU_FN_NOTA* 0x0f
*CLU*真值表的最小相 !a 注意最小相之间只能用 | (C语言按位或)
- #define *CLU_FN_B* 0xcc
*CLU*真值表的最小相 b 注意最小相之间只能用 | (C语言按位或)
- #define *CLU_FN_NOTB* 0x33
*CLU*真值表的最小相 !b 注意最小相之间只能用 | (C语言按位或)
- #define *CLU_FN_C* 0xaa
*CLU*真值表的最小相 c 注意最小相之间只能用 | (C语言按位或)
- #define *CLU_FN_NOTC* 0x55
*CLU*真值表的最小相 !c 注意最小相之间只能用 | (C语言按位或)
- #define *CLU_FN_A_B* 0xc0
*CLU*真值表的最小相 a&b 注意最小相之间只能用 | (C语言按位或)

- `#define CLU_FN_NOTA_NOTB 0x03`
 CLU 真值表的最小相 $!a \& !b$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTA_B 0x0c`
 CLU 真值表的最小相 $!a \& b$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_A_NOTB 0x30`
 CLU 真值表的最小相 $a \& !b$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_A_C 0xa0`
 CLU 真值表的最小相 $a \& c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTA_NOTC 0x05`
 CLU 真值表的最小相 $!a \& !c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTA_C 0x0a`
 CLU 真值表的最小相 $!a \& c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_A_NOTC 0x50`
 CLU 真值表的最小相 $a \& !c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_B_C 0x88`
 CLU 真值表的最小相 $b \& c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTB_NOTC 0x11`
 CLU 真值表的最小相 $!b \& !c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTB_C 0x22`
 CLU 真值表的最小相 $!b \& c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_B_NOTC 0x44`
 CLU 真值表的最小相 $b \& !c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTA_NOTB_NOTC 0x01`
 CLU 真值表的最小相 $!a \& !b \& !c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTA_NOTB_C 0x02`
 CLU 真值表的最小相 $!a \& !b \& c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTA_B_NOTC 0x04`
 CLU 真值表的最小相 $!a \& b \& !c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_NOTA_B_C 0x08`
 CLU 真值表的最小相 $!a \& b \& c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_A_NOTB_NOTC 0x10`
 CLU 真值表的最小相 $a \& !b \& !c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_A_NOTB_C 0x20`
 CLU 真值表的最小相 $a \& !b \& c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_A_B_NOTC 0x40`
 CLU 真值表的最小相 $a \& b \& !c$ 注意最小相之间只能用 `|` (C语言按位或)
- `#define CLU_FN_A_B_C 0x80`
 CLU 真值表的最小相 $a \& b \& c$ 注意最小相之间只能用 `|` (C语言按位或)

函数

- `void CLU_Init (CLU_TypeDef *CLUx, u8 CLU_MODEL_CLUx, CLU_InitTypeDef *CLU_InitStruct)`
 CLU 计算初始化
- `void CLU_StructInit (CLU_InitTypeDef *CLU_InitStruct)`
 CLU 初始化结构体初始化
- `void CLU_Enable (CLU_TypeDef *CLUx)`
 CLU 模块时钟使能
- `void CLU_Disable (CLU_TypeDef *CLUx)`
 CLU 模块时钟关闭
- `u8 CLU_ReadOutValue (CLU_TypeDef *CLUx, u8 CLU_MODEL_CLUx)`
 获取 CLU 的输出值

5.5.1 详细描述

CLU库函数头文件

作者

YangZJ

日期

2023-02-27

在文件 [lks32mc07x_clu.h](#) 中定义.

5.5.2 宏定义说明

5.5.2.1 CLU_FN_0

```
#define CLU_FN_0 0x0
```

CLU真值表的最小相 0 注意最小相之间只能用 | (C语言按位或)

在文件 [lks32mc07x_clu.h](#) 第 183 行定义.

5.5.2.2 CLU_FN_1

```
#define CLU_FN_1 0xff
```

CLU真值表的最小相 1 注意最小相之间只能用 | (C语言按位或)

在文件 [lks32mc07x_clu.h](#) 第 184 行定义.

5.5.2.3 CLU_FN_A

```
#define CLU_FN_A 0xf0
```

CLU真值表的最小相 a 注意最小相之间只能用 | (C语言按位或)

在文件 [lks32mc07x_clu.h](#) 第 185 行定义.

5.5.2.4 CLU_FN_A_B

```
#define CLU_FN_A_B 0xc0
```

CLU真值表的最小相 a&b 注意最小相之间只能用 | (C语言按位或)

在文件 [lks32mc07x_clu.h](#) 第 191 行定义.

5.5.2.5 CLU_FN_A_B_C

```
#define CLU_FN_A_B_C 0x80
```

CLU真值表的最小相 $a \& b \& c$ 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 210 行定义.

5.5.2.6 CLU_FN_A_B_NOTC

```
#define CLU_FN_A_B_NOTC 0x40
```

CLU真值表的最小相 $a \& b \& !c$ 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 209 行定义.

5.5.2.7 CLU_FN_A_C

```
#define CLU_FN_A_C 0xa0
```

CLU真值表的最小相 $a \& c$ 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 195 行定义.

5.5.2.8 CLU_FN_A_NOTB

```
#define CLU_FN_A_NOTB 0x30
```

CLU真值表的最小相 $a \& !b$ 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 194 行定义.

5.5.2.9 CLU_FN_A_NOTB_C

```
#define CLU_FN_A_NOTB_C 0x20
```

CLU真值表的最小相 $a \& !b \& c$ 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 208 行定义.

5.5.2.10 CLU_FN_A_NOTB_NOTC

```
#define CLU_FN_A_NOTB_NOTC 0x10
```

CLU真值表的最小相 $a \& !b \& !c$ 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 207 行定义.

5.5.2.11 CLU_FN_A_NOTC

```
#define CLU_FN_A_NOTC 0x50
```

CLU真值表的最小相 a&lc 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 198 行定义.

5.5.2.12 CLU_FN_B

```
#define CLU_FN_B 0xcc
```

CLU真值表的最小相 b 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 187 行定义.

5.5.2.13 CLU_FN_B_C

```
#define CLU_FN_B_C 0x88
```

CLU真值表的最小相 b&c 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 199 行定义.

5.5.2.14 CLU_FN_B_NOTC

```
#define CLU_FN_B_NOTC 0x44
```

CLU真值表的最小相 b&lc 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 202 行定义.

5.5.2.15 CLU_FN_C

```
#define CLU_FN_C 0xaa
```

CLU真值表的最小相 c 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 189 行定义.

5.5.2.16 CLU_FN_NOTA

```
#define CLU_FN_NOTA 0x0f
```

CLU真值表的最小相 la 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 186 行定义.

5.5.2.17 CLU_FN_NOTA_B

```
#define CLU_FN_NOTA_B 0x0c
```

CLU真值表的最小相 $!a \& b$ 注意最小相之间只能用 $|$ (C语言按位或)

在文件 [lks32mc07x.clu.h](#) 第 193 行定义.

5.5.2.18 CLU_FN_NOTA_B_C

```
#define CLU_FN_NOTA_B_C 0x08
```

CLU真值表的最小相 $!a \& b \& c$ 注意最小相之间只能用 $|$ (C语言按位或)

在文件 [lks32mc07x.clu.h](#) 第 206 行定义.

5.5.2.19 CLU_FN_NOTA_B_NOTC

```
#define CLU_FN_NOTA_B_NOTC 0x04
```

CLU真值表的最小相 $!a \& b \& !c$ 注意最小相之间只能用 $|$ (C语言按位或)

在文件 [lks32mc07x.clu.h](#) 第 205 行定义.

5.5.2.20 CLU_FN_NOTA_C

```
#define CLU_FN_NOTA_C 0x0a
```

CLU真值表的最小相 $!a \& c$ 注意最小相之间只能用 $|$ (C语言按位或)

在文件 [lks32mc07x.clu.h](#) 第 197 行定义.

5.5.2.21 CLU_FN_NOTA_NOTB

```
#define CLU_FN_NOTA_NOTB 0x03
```

CLU真值表的最小相 $!a \& !b$ 注意最小相之间只能用 $|$ (C语言按位或)

在文件 [lks32mc07x.clu.h](#) 第 192 行定义.

5.5.2.22 CLU_FN_NOTA_NOTB_C

```
#define CLU_FN_NOTA_NOTB_C 0x02
```

CLU真值表的最小相 $!a \& !b \& c$ 注意最小相之间只能用 $|$ (C语言按位或)

在文件 [lks32mc07x.clu.h](#) 第 204 行定义.

5.5.2.23 CLU_FN_NOTA_NOTB_NOTC

```
#define CLU_FN_NOTA_NOTB_NOTC 0x01
```

CLU真值表的最小相 **!a&!b&!c** 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 203 行定义.

5.5.2.24 CLU_FN_NOTA_NOTC

```
#define CLU_FN_NOTA_NOTC 0x05
```

CLU真值表的最小相 **!a&!c** 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 196 行定义.

5.5.2.25 CLU_FN_NOTB

```
#define CLU_FN_NOTB 0x33
```

CLU真值表的最小相 **!b** 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 188 行定义.

5.5.2.26 CLU_FN_NOTB_C

```
#define CLU_FN_NOTB_C 0x22
```

CLU真值表的最小相 **!b&c** 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 201 行定义.

5.5.2.27 CLU_FN_NOTB_NOTC

```
#define CLU_FN_NOTB_NOTC 0x11
```

CLU真值表的最小相 **!b&!c** 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 200 行定义.

5.5.2.28 CLU_FN_NOTC

```
#define CLU_FN_NOTC 0x55
```

CLU真值表的最小相 **!c** 注意最小相之间只能用 | （C语言按位或）

在文件 [lks32mc07x.clu.h](#) 第 190 行定义.

5.5.2.29 CLU_MODEL_CLU0

```
#define CLU_MODEL_CLU0 BIT0
```

CLU0 CL0里面有4个CLU 这里用BIT0 BIT1 BIT2 BIT3做区分 不可修改

在文件 [lks32mc07x.clu.h](#) 第 42 行定义.

5.5.2.30 CLU_MODEL_CLU1

```
#define CLU_MODEL_CLU1 BIT1
```

CLU1

在文件 [lks32mc07x.clu.h](#) 第 43 行定义.

5.5.2.31 CLU_MODEL_CLU2

```
#define CLU_MODEL_CLU2 BIT2
```

CLU2

在文件 [lks32mc07x.clu.h](#) 第 44 行定义.

5.5.2.32 CLU_MODEL_CLU3

```
#define CLU_MODEL_CLU3 BIT3
```

CLU3

在文件 [lks32mc07x.clu.h](#) 第 45 行定义.

5.5.2.33 CLU_MXA_CLU0_CLU0OUT

```
#define CLU_MXA_CLU0_CLU0OUT 0
```

使用 CLU0OUT 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 47 行定义.

5.5.2.34 CLU_MXA_CLU0_CLU1OUT

```
#define CLU_MXA_CLU0_CLU1OUT 1
```

使用 CLU1OUT 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 48 行定义.

5.5.2.35 CLU_MXA_CLU0_CLU2OUT

```
#define CLU_MXA_CLU0_CLU2OUT 2
```

使用 CLU2OUT 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 49 行定义.

5.5.2.36 CLU_MXA_CLU0_CLU3OUT

```
#define CLU_MXA_CLU0_CLU3OUT 3
```

使用 CLU3OUT 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 50 行定义.

5.5.2.37 CLU_MXA_CLU0_P0_0

```
#define CLU_MXA_CLU0_P0_0 8
```

使用 P0_0 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 55 行定义.

5.5.2.38 CLU_MXA_CLU0_P0_12

```
#define CLU_MXA_CLU0_P0_12 13
```

使用 P0_12 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 60 行定义.

5.5.2.39 CLU_MXA_CLU0_P0_14

```
#define CLU_MXA_CLU0_P0_14 14
```

使用 P0_14 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 61 行定义.

5.5.2.40 CLU_MXA_CLU0_P0_3

```
#define CLU_MXA_CLU0_P0_3 9
```

使用 P0_3 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 56 行定义.

5.5.2.41 CLU_MXA_CLU0_P0_5

```
#define CLU_MXA_CLU0_P0_5 10
```

使用 P0_5 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 57 行定义.

5.5.2.42 CLU_MXA_CLU0_P0_7

```
#define CLU_MXA_CLU0_P0_7 11
```

使用 P0_7 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 58 行定义.

5.5.2.43 CLU_MXA_CLU0_P1_0

```
#define CLU_MXA_CLU0_P1_0 15
```

使用 P1_0 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 62 行定义.

5.5.2.44 CLU_MXA_CLU0_P2_12

```
#define CLU_MXA_CLU0_P2_12 12
```

使用 P2_12 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 59 行定义.

5.5.2.45 CLU_MXA_CLU0_PWM0

```
#define CLU_MXA_CLU0_PWM0 6
```

使用 PWM0 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 53 行定义.

5.5.2.46 CLU_MXA_CLU0_PWM1

```
#define CLU_MXA_CLU0_PWM1 7
```

使用 PWM1 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 54 行定义.

5.5.2.47 CLU_MXA_CLU0_Timer0ch0ovfl

```
#define CLU_MXA_CLU0_Timer0ch0ovfl 4
```

使用 Timer0ch0ovfl 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 51 行定义.

5.5.2.48 CLU_MXA_CLU0_Timer1ch0ovfl

```
#define CLU_MXA_CLU0_Timer1ch0ovfl 5
```

使用 Timer1ch0ovfl 作为CLU0_A的输入

在文件 [lks32mc07x.clu.h](#) 第 52 行定义.

5.5.2.49 CLU_MXA_CLU1_CLU0OUT

```
#define CLU_MXA_CLU1_CLU0OUT 0
```

使用 CLU0OUT 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 64 行定义.

5.5.2.50 CLU_MXA_CLU1_CLU1OUT

```
#define CLU_MXA_CLU1_CLU1OUT 1
```

使用 CLU1OUT 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 65 行定义.

5.5.2.51 CLU_MXA_CLU1_CLU2OUT

```
#define CLU_MXA_CLU1_CLU2OUT 2
```

使用 CLU2OUT 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 66 行定义.

5.5.2.52 CLU_MXA_CLU1_CLU3OUT

```
#define CLU_MXA_CLU1_CLU3OUT 3
```

使用 CLU3OUT 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 67 行定义.

5.5.2.53 CLU_MXA_CLU1_P0_0

```
#define CLU_MXA_CLU1_P0_0 8
```

使用 P0_0 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 72 行定义.

5.5.2.54 CLU_MXA_CLU1_P0_11

```
#define CLU_MXA_CLU1_P0_11 13
```

使用 P0_11 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 77 行定义.

5.5.2.55 CLU_MXA_CLU1_P0_14

```
#define CLU_MXA_CLU1_P0_14 14
```

使用 P0_14 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 78 行定义.

5.5.2.56 CLU_MXA_CLU1_P0_15

```
#define CLU_MXA_CLU1_P0_15 15
```

使用 P0_15 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 79 行定义.

5.5.2.57 CLU_MXA_CLU1_P0_2

```
#define CLU_MXA_CLU1_P0_2 9
```

使用 P0_2 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 73 行定义.

5.5.2.58 CLU_MXA_CLU1_P0_5

```
#define CLU_MXA_CLU1_P0_5 10
```

使用 P0_5 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 74 行定义.

5.5.2.59 CLU_MXA_CLU1_P0_6

```
#define CLU_MXA_CLU1_P0_6 11
```

使用 P0_6 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 75 行定义.

5.5.2.60 CLU_MXA_CLU1_P2_12

```
#define CLU_MXA_CLU1_P2_12 12
```

使用 P2_12 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 76 行定义.

5.5.2.61 CLU_MXA_CLU1_PWM2

```
#define CLU_MXA_CLU1_PWM2 6
```

使用 PWM2 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 70 行定义.

5.5.2.62 CLU_MXA_CLU1_PWM3

```
#define CLU_MXA_CLU1_PWM3 7
```

使用 PWM3 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 71 行定义.

5.5.2.63 CLU_MXA_CLU1_Timer0ch1ovfl

```
#define CLU_MXA_CLU1_Timer0ch1ovfl 4
```

使用 Timer0ch1ovfl 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 68 行定义.

5.5.2.64 CLU_MXA_CLU1_Timer1ch1ovfl

```
#define CLU_MXA_CLU1_Timer1ch1ovfl 5
```

使用 Timer1ch1ovfl 作为CLU1_A的输入

在文件 [lks32mc07x.clu.h](#) 第 69 行定义.

5.5.2.65 CLU_MXA_CLU2_CLU0OUT

```
#define CLU_MXA_CLU2_CLU0OUT 0
```

使用 CLU0OUT 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 81 行定义.

5.5.2.66 CLU_MXA_CLU2_CLU1OUT

```
#define CLU_MXA_CLU2_CLU1OUT 1
```

使用 CLU1OUT 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 82 行定义.

5.5.2.67 CLU_MXA_CLU2_CLU2OUT

```
#define CLU_MXA_CLU2_CLU2OUT 2
```

使用 CLU2OUT 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 83 行定义.

5.5.2.68 CLU_MXA_CLU2_CLU3OUT

```
#define CLU_MXA_CLU2_CLU3OUT 3
```

使用 CLU3OUT 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 84 行定义.

5.5.2.69 CLU_MXA_CLU2_P1_10

```
#define CLU_MXA_CLU2_P1_10 11
```

使用 P1_10 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 92 行定义.

5.5.2.70 CLU_MXA_CLU2_P1_13

```
#define CLU_MXA_CLU2_P1_13 9
```

使用 P1_13 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 90 行定义.

5.5.2.71 CLU_MXA_CLU2_P1_15

```
#define CLU_MXA_CLU2_P1_15 10
```

使用 P1_15 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 91 行定义.

5.5.2.72 CLU_MXA_CLU2_P2_1

```
#define CLU_MXA_CLU2_P2_1 12
```

使用 P2_1 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 93 行定义.

5.5.2.73 CLU_MXA_CLU2_P2_14

```
#define CLU_MXA_CLU2_P2_14 15
```

使用 P2_14 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 96 行定义.

5.5.2.74 CLU_MXA_CLU2_P2_4

```
#define CLU_MXA_CLU2_P2_4 13
```

使用 P2_4 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 94 行定义.

5.5.2.75 CLU_MXA_CLU2_P2_6

```
#define CLU_MXA_CLU2_P2_6 14
```

使用 P2_6 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 95 行定义.

5.5.2.76 CLU_MXA_CLU2_P2_8

```
#define CLU_MXA_CLU2_P2_8 8
```

使用 P2_8 作为CLU2_A的输入

在文件 [lks32mc07x.clu.h](#) 第 89 行定义.

5.5.2.77 CLU_MXA_CLU2_PWM0

```
#define CLU_MXA_CLU2_PWM0 6
```

使用 PWM0 作为CLU2.A的输入

在文件 [lks32mc07x.clu.h](#) 第 87 行定义.

5.5.2.78 CLU_MXA_CLU2_PWM2

```
#define CLU_MXA_CLU2_PWM2 7
```

使用 PWM2 作为CLU2.A的输入

在文件 [lks32mc07x.clu.h](#) 第 88 行定义.

5.5.2.79 CLU_MXA_CLU2_Timer2ch0ovfl

```
#define CLU_MXA_CLU2_Timer2ch0ovfl 4
```

使用 Timer2ch0ovfl 作为CLU2.A的输入

在文件 [lks32mc07x.clu.h](#) 第 85 行定义.

5.5.2.80 CLU_MXA_CLU2_Timer3ch0ovfl

```
#define CLU_MXA_CLU2_Timer3ch0ovfl 5
```

使用 Timer3ch0ovfl 作为CLU2.A的输入

在文件 [lks32mc07x.clu.h](#) 第 86 行定义.

5.5.2.81 CLU_MXA_CLU3_CLU0OUT

```
#define CLU_MXA_CLU3_CLU0OUT 0
```

使用 CLU0OUT 作为CLU3.A的输入

在文件 [lks32mc07x.clu.h](#) 第 98 行定义.

5.5.2.82 CLU_MXA_CLU3_CLU1OUT

```
#define CLU_MXA_CLU3_CLU1OUT 1
```

使用 CLU1OUT 作为CLU3.A的输入

在文件 [lks32mc07x.clu.h](#) 第 99 行定义.

5.5.2.83 CLU_MXA_CLU3_CLU2OUT

```
#define CLU_MXA_CLU3_CLU2OUT 2
```

使用 CLU2OUT 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 100 行定义.

5.5.2.84 CLU_MXA_CLU3_CLU3OUT

```
#define CLU_MXA_CLU3_CLU3OUT 3
```

使用 CLU3OUT 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 101 行定义.

5.5.2.85 CLU_MXA_CLU3_P1_15

```
#define CLU_MXA_CLU3_P1_15 10
```

使用 P1_15 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 108 行定义.

5.5.2.86 CLU_MXA_CLU3_P2_0

```
#define CLU_MXA_CLU3_P2_0 11
```

使用 P2_0 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 109 行定义.

5.5.2.87 CLU_MXA_CLU3_P2_1

```
#define CLU_MXA_CLU3_P2_1 12
```

使用 P2_1 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 110 行定义.

5.5.2.88 CLU_MXA_CLU3_P2_13

```
#define CLU_MXA_CLU3_P2_13 15
```

使用 P2_13 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 113 行定义.

5.5.2.89 CLU_MXA_CLU3_P2.3

```
#define CLU_MXA_CLU3_P2_3 13
```

使用 P2.3 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 111 行定义.

5.5.2.90 CLU_MXA_CLU3_P2.6

```
#define CLU_MXA_CLU3_P2_6 14
```

使用 P2.6 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 112 行定义.

5.5.2.91 CLU_MXA_CLU3_P2.8

```
#define CLU_MXA_CLU3_P2_8 8
```

使用 P2.8 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 106 行定义.

5.5.2.92 CLU_MXA_CLU3_P3.9

```
#define CLU_MXA_CLU3_P3_9 9
```

使用 P3.9 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 107 行定义.

5.5.2.93 CLU_MXA_CLU3_PWM1

```
#define CLU_MXA_CLU3_PWM1 6
```

使用 PWM1 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 104 行定义.

5.5.2.94 CLU_MXA_CLU3_PWM3

```
#define CLU_MXA_CLU3_PWM3 7
```

使用 PWM3 作为CLU3_A的输入

在文件 [lks32mc07x.clu.h](#) 第 105 行定义.

5.5.2.95 CLU_MXA.CLU3_Timer2ch1ovfl

```
#define CLU_MXA.CLU3_Timer2ch1ovfl 4
```

使用 Timer2ch1ovfl 作为CLU3.A的输入

在文件 [lks32mc07x.clu.h](#) 第 102 行定义.

5.5.2.96 CLU_MXA.CLU3_Timer3ch1ovfl

```
#define CLU_MXA.CLU3_Timer3ch1ovfl 5
```

使用 Timer3ch1ovfl 作为CLU3.A的输入

在文件 [lks32mc07x.clu.h](#) 第 103 行定义.

5.5.2.97 CLU_MXB.CLU0_ADBUSY0

```
#define CLU_MXB.CLU0_ADBUSY0 4
```

使用 ADBUSY0 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 119 行定义.

5.5.2.98 CLU_MXB.CLU0_ADBUSY1

```
#define CLU_MXB.CLU0_ADBUSY1 5
```

使用 ADBUSY1 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 120 行定义.

5.5.2.99 CLU_MXB.CLU0_CLU0OUT

```
#define CLU_MXB.CLU0_CLU0OUT 0
```

使用 CLU0OUT 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 115 行定义.

5.5.2.100 CLU_MXB.CLU0_CLU1OUT

```
#define CLU_MXB.CLU0_CLU1OUT 1
```

使用 CLU1OUT 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 116 行定义.

5.5.2.101 CLU_MXB_CLU0_CLU2OUT

```
#define CLU_MXB_CLU0_CLU2OUT 2
```

使用 CLU2OUT 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 117 行定义.

5.5.2.102 CLU_MXB_CLU0_CLU3OUT

```
#define CLU_MXB_CLU0_CLU3OUT 3
```

使用 CLU3OUT 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 118 行定义.

5.5.2.103 CLU_MXB_CLU0_P0_11

```
#define CLU_MXB_CLU0_P0_11 12
```

使用 P0_11 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 127 行定义.

5.5.2.104 CLU_MXB_CLU0_P0_13

```
#define CLU_MXB_CLU0_P0_13 13
```

使用 P0_13 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 128 行定义.

5.5.2.105 CLU_MXB_CLU0_P0_15

```
#define CLU_MXB_CLU0_P0_15 14
```

使用 P0_15 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 129 行定义.

5.5.2.106 CLU_MXB_CLU0_P0_2

```
#define CLU_MXB_CLU0_P0_2 8
```

使用 P0_2 作为CLU0.B的输入

在文件 [lks32mc07x.clu.h](#) 第 123 行定义.

5.5.2.107 CLU_MXB_CLU0_P0_4

```
#define CLU_MXB_CLU0_P0_4 9
```

使用 P0_4 作为CLU0.B的输入

在文件 [lks32mc07x_clu.h](#) 第 124 行定义.

5.5.2.108 CLU_MXB_CLU0_P0_6

```
#define CLU_MXB_CLU0_P0_6 10
```

使用 P0_6 作为CLU0.B的输入

在文件 [lks32mc07x_clu.h](#) 第 125 行定义.

5.5.2.109 CLU_MXB_CLU0_P2_11

```
#define CLU_MXB_CLU0_P2_11 11
```

使用 P2_11 作为CLU0.B的输入

在文件 [lks32mc07x_clu.h](#) 第 126 行定义.

5.5.2.110 CLU_MXB_CLU0_P2_7

```
#define CLU_MXB_CLU0_P2_7 15
```

使用 P2_7 作为CLU0.B的输入

在文件 [lks32mc07x_clu.h](#) 第 130 行定义.

5.5.2.111 CLU_MXB_CLU0_PWM3

```
#define CLU_MXB_CLU0_PWM3 6
```

使用 PWM3 作为CLU0.B的输入

在文件 [lks32mc07x_clu.h](#) 第 121 行定义.

5.5.2.112 CLU_MXB_CLU0_SPI MOSI

```
#define CLU_MXB_CLU0_SPI MOSI 7
```

使用 SPI MOSI 作为CLU0.B的输入

在文件 [lks32mc07x_clu.h](#) 第 122 行定义.

5.5.2.113 CLU_MXB_CLU1_ADBUSY0

```
#define CLU_MXB_CLU1_ADBUSY0 4
```

使用 ADBUSY0 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 136 行定义.

5.5.2.114 CLU_MXB_CLU1_ADBUSY1

```
#define CLU_MXB_CLU1_ADBUSY1 5
```

使用 ADBUSY1 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 137 行定义.

5.5.2.115 CLU_MXB_CLU1_CLU0OUT

```
#define CLU_MXB_CLU1_CLU0OUT 0
```

使用 CLU0OUT 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 132 行定义.

5.5.2.116 CLU_MXB_CLU1_CLU1OUT

```
#define CLU_MXB_CLU1_CLU1OUT 1
```

使用 CLU1OUT 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 133 行定义.

5.5.2.117 CLU_MXB_CLU1_CLU2OUT

```
#define CLU_MXB_CLU1_CLU2OUT 2
```

使用 CLU2OUT 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 134 行定义.

5.5.2.118 CLU_MXB_CLU1_CLU3OUT

```
#define CLU_MXB_CLU1_CLU3OUT 3
```

使用 CLU3OUT 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 135 行定义.

5.5.2.119 CLU_MXB_CLU1_P0_12

```
#define CLU_MXB_CLU1_P0_12 12
```

使用 P0_12 作为CLU1_B的输入

在文件 [lks32mc07x.clu.h](#) 第 144 行定义.

5.5.2.120 CLU_MXB_CLU1_P0_13

```
#define CLU_MXB_CLU1_P0_13 13
```

使用 P0_13 作为CLU1_B的输入

在文件 [lks32mc07x.clu.h](#) 第 145 行定义.

5.5.2.121 CLU_MXB_CLU1_P0_3

```
#define CLU_MXB_CLU1_P0_3 8
```

使用 P0_3 作为CLU1_B的输入

在文件 [lks32mc07x.clu.h](#) 第 140 行定义.

5.5.2.122 CLU_MXB_CLU1_P0_4

```
#define CLU_MXB_CLU1_P0_4 9
```

使用 P0_4 作为CLU1_B的输入

在文件 [lks32mc07x.clu.h](#) 第 141 行定义.

5.5.2.123 CLU_MXB_CLU1_P0_7

```
#define CLU_MXB_CLU1_P0_7 10
```

使用 P0_7 作为CLU1_B的输入

在文件 [lks32mc07x.clu.h](#) 第 142 行定义.

5.5.2.124 CLU_MXB_CLU1_P1_0

```
#define CLU_MXB_CLU1_P1_0 14
```

使用 P1_0 作为CLU1_B的输入

在文件 [lks32mc07x.clu.h](#) 第 146 行定义.

5.5.2.125 CLU_MXB_CLU1_P2_11

```
#define CLU_MXB_CLU1_P2_11 11
```

使用 P2_11 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 143 行定义.

5.5.2.126 CLU_MXB_CLU1_P2_7

```
#define CLU_MXB_CLU1_P2_7 15
```

使用 P2_7 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 147 行定义.

5.5.2.127 CLU_MXB_CLU1_PWM1

```
#define CLU_MXB_CLU1_PWM1 6
```

使用 PWM1 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 138 行定义.

5.5.2.128 CLU_MXB_CLU1_SPIMISO

```
#define CLU_MXB_CLU1_SPIMISO 7
```

使用 SPIMISO 作为CLU1_B的输入

在文件 [lks32mc07x_clu.h](#) 第 139 行定义.

5.5.2.129 CLU_MXB_CLU2_CLU0OUT

```
#define CLU_MXB_CLU2_CLU0OUT 0
```

使用 CLU0OUT 作为CLU2_B的输入

在文件 [lks32mc07x_clu.h](#) 第 149 行定义.

5.5.2.130 CLU_MXB_CLU2_CLU1OUT

```
#define CLU_MXB_CLU2_CLU1OUT 1
```

使用 CLU1OUT 作为CLU2_B的输入

在文件 [lks32mc07x_clu.h](#) 第 150 行定义.

5.5.2.131 CLU_MXB_CLU2_CLU2OUT

```
#define CLU_MXB_CLU2_CLU2OUT 2
```

使用 CLU2OUT 作为CLU2_B的输入

在文件 [lks32mc07x.clu.h](#) 第 151 行定义.

5.5.2.132 CLU_MXB_CLU2_CLU3OUT

```
#define CLU_MXB_CLU2_CLU3OUT 3
```

使用 CLU3OUT 作为CLU2_B的输入

在文件 [lks32mc07x.clu.h](#) 第 152 行定义.

5.5.2.133 CLU_MXB_CLU2_I2CSDA

```
#define CLU_MXB_CLU2_I2CSDA 6
```

使用 I2CSDA 作为CLU2_B的输入

在文件 [lks32mc07x.clu.h](#) 第 155 行定义.

5.5.2.134 CLU_MXB_CLU2_P1_11

```
#define CLU_MXB_CLU2_P1_11 11
```

使用 P1_11 作为CLU2_B的输入

在文件 [lks32mc07x.clu.h](#) 第 160 行定义.

5.5.2.135 CLU_MXB_CLU2_P1_14

```
#define CLU_MXB_CLU2_P1_14 9
```

使用 P1_14 作为CLU2_B的输入

在文件 [lks32mc07x.clu.h](#) 第 158 行定义.

5.5.2.136 CLU_MXB_CLU2_P2_0

```
#define CLU_MXB_CLU2_P2_0 10
```

使用 P2_0 作为CLU2_B的输入

在文件 [lks32mc07x.clu.h](#) 第 159 行定义.

5.5.2.137 CLU_MXB_CLU2_P2_13

```
#define CLU_MXB_CLU2_P2_13 14
```

使用 P2_13 作为CLU2_B的输入

在文件 [lks32mc07x_clu.h](#) 第 163 行定义.

5.5.2.138 CLU_MXB_CLU2_P2_15

```
#define CLU_MXB_CLU2_P2_15 15
```

使用 P2_15 作为CLU2_B的输入

在文件 [lks32mc07x_clu.h](#) 第 164 行定义.

5.5.2.139 CLU_MXB_CLU2_P2_3

```
#define CLU_MXB_CLU2_P2_3 12
```

使用 P2_3 作为CLU2_B的输入

在文件 [lks32mc07x_clu.h](#) 第 161 行定义.

5.5.2.140 CLU_MXB_CLU2_P2_5

```
#define CLU_MXB_CLU2_P2_5 13
```

使用 P2_5 作为CLU2_B的输入

在文件 [lks32mc07x_clu.h](#) 第 162 行定义.

5.5.2.141 CLU_MXB_CLU2_P3_9

```
#define CLU_MXB_CLU2_P3_9 8
```

使用 P3_9 作为CLU2_B的输入

在文件 [lks32mc07x_clu.h](#) 第 157 行定义.

5.5.2.142 CLU_MXB_CLU2_PWM1

```
#define CLU_MXB_CLU2_PWM1 4
```

使用 PWM1 作为CLU2_B的输入

在文件 [lks32mc07x_clu.h](#) 第 153 行定义.

5.5.2.143 CLU_MXB_CLU2_PWM3

```
#define CLU_MXB_CLU2_PWM3 5
```

使用 PWM3 作为CLU2.B的输入

在文件 [lks32mc07x_clu.h](#) 第 154 行定义.

5.5.2.144 CLU_MXB_CLU2_SPICLK

```
#define CLU_MXB_CLU2_SPICLK 7
```

使用 SPICLK 作为CLU2.B的输入

在文件 [lks32mc07x_clu.h](#) 第 156 行定义.

5.5.2.145 CLU_MXB_CLU3_CLU0OUT

```
#define CLU_MXB_CLU3_CLU0OUT 0
```

使用 CLU0OUT 作为CLU3.B的输入

在文件 [lks32mc07x_clu.h](#) 第 166 行定义.

5.5.2.146 CLU_MXB_CLU3_CLU1OUT

```
#define CLU_MXB_CLU3_CLU1OUT 1
```

使用 CLU1OUT 作为CLU3.B的输入

在文件 [lks32mc07x_clu.h](#) 第 167 行定义.

5.5.2.147 CLU_MXB_CLU3_CLU2OUT

```
#define CLU_MXB_CLU3_CLU2OUT 2
```

使用 CLU2OUT 作为CLU3.B的输入

在文件 [lks32mc07x_clu.h](#) 第 168 行定义.

5.5.2.148 CLU_MXB_CLU3_CLU3OUT

```
#define CLU_MXB_CLU3_CLU3OUT 3
```

使用 CLU3OUT 作为CLU3.B的输入

在文件 [lks32mc07x_clu.h](#) 第 169 行定义.

5.5.2.149 CLU_MXB_CLU3_I2CSCL

```
#define CLU_MXB_CLU3_I2CSCL 6
```

使用 I2CSCL 作为CLU3_B的输入

在文件 [lks32mc07x.clu.h](#) 第 172 行定义.

5.5.2.150 CLU_MXB_CLU3_P1_10

```
#define CLU_MXB_CLU3_P1_10 10
```

使用 P1_10 作为CLU3_B的输入

在文件 [lks32mc07x.clu.h](#) 第 176 行定义.

5.5.2.151 CLU_MXB_CLU3_P1_11

```
#define CLU_MXB_CLU3_P1_11 11
```

使用 P1_11 作为CLU3_B的输入

在文件 [lks32mc07x.clu.h](#) 第 177 行定义.

5.5.2.152 CLU_MXB_CLU3_P1_13

```
#define CLU_MXB_CLU3_P1_13 8
```

使用 P1_13 作为CLU3_B的输入

在文件 [lks32mc07x.clu.h](#) 第 174 行定义.

5.5.2.153 CLU_MXB_CLU3_P1_14

```
#define CLU_MXB_CLU3_P1_14 9
```

使用 P1_14 作为CLU3_B的输入

在文件 [lks32mc07x.clu.h](#) 第 175 行定义.

5.5.2.154 CLU_MXB_CLU3_P2_14

```
#define CLU_MXB_CLU3_P2_14 14
```

使用 P2_14 作为CLU3_B的输入

在文件 [lks32mc07x.clu.h](#) 第 180 行定义.

5.5.2.155 CLU_MXB_CLU3_P2_15

```
#define CLU_MXB_CLU3_P2_15 15
```

使用 P2_15 作为CLU3_B的输入

在文件 [lks32mc07x_clu.h](#) 第 181 行定义.

5.5.2.156 CLU_MXB_CLU3_P2_4

```
#define CLU_MXB_CLU3_P2_4 12
```

使用 P2_4 作为CLU3_B的输入

在文件 [lks32mc07x_clu.h](#) 第 178 行定义.

5.5.2.157 CLU_MXB_CLU3_P2_5

```
#define CLU_MXB_CLU3_P2_5 13
```

使用 P2_5 作为CLU3_B的输入

在文件 [lks32mc07x_clu.h](#) 第 179 行定义.

5.5.2.158 CLU_MXB_CLU3_PWM0

```
#define CLU_MXB_CLU3_PWM0 4
```

使用 PWM0 作为CLU3_B的输入

在文件 [lks32mc07x_clu.h](#) 第 170 行定义.

5.5.2.159 CLU_MXB_CLU3_PWM2

```
#define CLU_MXB_CLU3_PWM2 5
```

使用 PWM2 作为CLU3_B的输入

在文件 [lks32mc07x_clu.h](#) 第 171 行定义.

5.5.2.160 CLU_MXB_CLU3_SPICSN

```
#define CLU_MXB_CLU3_SPICSN 7
```

使用 SPICSN 作为CLU3_B的输入

在文件 [lks32mc07x_clu.h](#) 第 173 行定义.

5.6 lks32mc07x_clu.h

[浏览该文件的文档.](#)

```

00001
00007 #ifndef __LKS32MC07x_CLU_H
00008 #define __LKS32MC07x_CLU_H
00010 #include "lks32mc07x.h"
00011 #include "lks32mc07x-sys.h"
00012
00013 typedef struct
00014 {
00015     __IO u32 EN;
00016     __IO u32 IE;
00017     __IO u32 IF;
00018     __IO u32 OUT;
00019     __IO u32 MX;
00020     __IO u32 FN;
00021     __IO u32 CF;
00022 } CLU_TypeDef;
00023
00024 #ifndef CLU
00025 #define CLU ((CLU_TypeDef *)CL0_BASE)
00026 #endif
00027
00028 typedef struct
00029 {
00030     u32 EN;
00031     u32 FIE;
00032     u32 RIE;
00033     u32 MXA;
00034     u32 MXB;
00035     u32 FN;
00036     u32 OUTSEL;
00037     u32 OEN;
00038     u32 CLKINV;
00039     u32 CLKSEL;
00040 } CLU_InitTypeDef;
00041
00042 #define CLU_MODEL_CLU0          BIT0
00043 #define CLU_MODEL_CLU1          BIT1
00044 #define CLU_MODEL_CLU2          BIT2
00045 #define CLU_MODEL_CLU3          BIT3
00046
00047 #define CLU_MXA_CLU0_CLU0OUT      0
00048 #define CLU_MXA_CLU0_CLU1OUT      1
00049 #define CLU_MXA_CLU0_CLU2OUT      2
00050 #define CLU_MXA_CLU0_CLU3OUT      3
00051 #define CLU_MXA_CLU0_Timer0ch0ovfl 4
00052 #define CLU_MXA_CLU0_Timer1ch0ovfl 5
00053 #define CLU_MXA_CLU0_PWM0         6
00054 #define CLU_MXA_CLU0_PWM1         7
00055 #define CLU_MXA_CLU0_P0_0         8
00056 #define CLU_MXA_CLU0_P0_3         9
00057 #define CLU_MXA_CLU0_P0_5        10
00058 #define CLU_MXA_CLU0_P0_7        11
00059 #define CLU_MXA_CLU0_P2_12        12
00060 #define CLU_MXA_CLU0_P0_12        13
00061 #define CLU_MXA_CLU0_P0_14        14
00062 #define CLU_MXA_CLU0_P1_0        15
00063
00064 #define CLU_MXA_CLU1_CLU0OUT      0
00065 #define CLU_MXA_CLU1_CLU1OUT      1
00066 #define CLU_MXA_CLU1_CLU2OUT      2
00067 #define CLU_MXA_CLU1_CLU3OUT      3
00068 #define CLU_MXA_CLU1_Timer0ch1ovfl 4
00069 #define CLU_MXA_CLU1_Timer1ch1ovfl 5
00070 #define CLU_MXA_CLU1_PWM2         6
00071 #define CLU_MXA_CLU1_PWM3         7
00072 #define CLU_MXA_CLU1_P0_0         8
00073 #define CLU_MXA_CLU1_P0_2         9
00074 #define CLU_MXA_CLU1_P0_5        10
00075 #define CLU_MXA_CLU1_P0_6        11
00076 #define CLU_MXA_CLU1_P2_12        12
00077 #define CLU_MXA_CLU1_P0_11        13
00078 #define CLU_MXA_CLU1_P0_14        14
00079 #define CLU_MXA_CLU1_P0_15        15
00080
00081 #define CLU_MXA_CLU2_CLU0OUT      0
00082 #define CLU_MXA_CLU2_CLU1OUT      1
00083 #define CLU_MXA_CLU2_CLU2OUT      2
00084 #define CLU_MXA_CLU2_CLU3OUT      3
00085 #define CLU_MXA_CLU2_Timer2ch0ovfl 4
00086 #define CLU_MXA_CLU2_Timer3ch0ovfl 5
00087 #define CLU_MXA_CLU2_PWM0         6
00088 #define CLU_MXA_CLU2_PWM2         7

```

```

00089 #define CLU.MXA.CLU2.P2.8      8
00090 #define CLU.MXA.CLU2.P1.13      9
00091 #define CLU.MXA.CLU2.P1.15     10
00092 #define CLU.MXA.CLU2.P1.10     11
00093 #define CLU.MXA.CLU2.P2.1      12
00094 #define CLU.MXA.CLU2.P2.4      13
00095 #define CLU.MXA.CLU2.P2.6      14
00096 #define CLU.MXA.CLU2.P2.14     15
00097
00098 #define CLU.MXA.CLU3.CLU0OUT      0
00099 #define CLU.MXA.CLU3.CLU1OUT      1
00100 #define CLU.MXA.CLU3.CLU2OUT      2
00101 #define CLU.MXA.CLU3.CLU3OUT      3
00102 #define CLU.MXA.CLU3.Timer2chlovf1 4
00103 #define CLU.MXA.CLU3.Timer3chlovf1 5
00104 #define CLU.MXA.CLU3.PWM1         6
00105 #define CLU.MXA.CLU3.PWM3         7
00106 #define CLU.MXA.CLU3.P2.8         8
00107 #define CLU.MXA.CLU3.P3.9         9
00108 #define CLU.MXA.CLU3.P1.15        10
00109 #define CLU.MXA.CLU3.P2.0         11
00110 #define CLU.MXA.CLU3.P2.1         12
00111 #define CLU.MXA.CLU3.P2.3         13
00112 #define CLU.MXA.CLU3.P2.6         14
00113 #define CLU.MXA.CLU3.P2.13        15
00114
00115 #define CLU.MXB.CLU0.CLU0OUT      0
00116 #define CLU.MXB.CLU0.CLU1OUT      1
00117 #define CLU.MXB.CLU0.CLU2OUT      2
00118 #define CLU.MXB.CLU0.CLU3OUT      3
00119 #define CLU.MXB.CLU0.ADBUSY0      4
00120 #define CLU.MXB.CLU0.ADBUSY1      5
00121 #define CLU.MXB.CLU0.PWM3         6
00122 #define CLU.MXB.CLU0.SPIMOSI      7
00123 #define CLU.MXB.CLU0.P0.2         8
00124 #define CLU.MXB.CLU0.P0.4         9
00125 #define CLU.MXB.CLU0.P0.6        10
00126 #define CLU.MXB.CLU0.P2.11        11
00127 #define CLU.MXB.CLU0.P0.11        12
00128 #define CLU.MXB.CLU0.P0.13        13
00129 #define CLU.MXB.CLU0.P0.15        14
00130 #define CLU.MXB.CLU0.P2.7        15
00131
00132 #define CLU.MXB.CLU1.CLU0OUT      0
00133 #define CLU.MXB.CLU1.CLU1OUT      1
00134 #define CLU.MXB.CLU1.CLU2OUT      2
00135 #define CLU.MXB.CLU1.CLU3OUT      3
00136 #define CLU.MXB.CLU1.ADBUSY0      4
00137 #define CLU.MXB.CLU1.ADBUSY1      5
00138 #define CLU.MXB.CLU1.PWM1         6
00139 #define CLU.MXB.CLU1.SPIMISO      7
00140 #define CLU.MXB.CLU1.P0.3         8
00141 #define CLU.MXB.CLU1.P0.4         9
00142 #define CLU.MXB.CLU1.P0.7        10
00143 #define CLU.MXB.CLU1.P2.11        11
00144 #define CLU.MXB.CLU1.P0.12        12
00145 #define CLU.MXB.CLU1.P0.13        13
00146 #define CLU.MXB.CLU1.P1.0        14
00147 #define CLU.MXB.CLU1.P2.7        15
00148
00149 #define CLU.MXB.CLU2.CLU0OUT      0
00150 #define CLU.MXB.CLU2.CLU1OUT      1
00151 #define CLU.MXB.CLU2.CLU2OUT      2
00152 #define CLU.MXB.CLU2.CLU3OUT      3
00153 #define CLU.MXB.CLU2.PWM1         4
00154 #define CLU.MXB.CLU2.PWM3         5
00155 #define CLU.MXB.CLU2.I2CSDA       6
00156 #define CLU.MXB.CLU2.SPICK        7
00157 #define CLU.MXB.CLU2.P3.9         8
00158 #define CLU.MXB.CLU2.P1.14        9
00159 #define CLU.MXB.CLU2.P2.0        10
00160 #define CLU.MXB.CLU2.P1.11        11
00161 #define CLU.MXB.CLU2.P2.3         12
00162 #define CLU.MXB.CLU2.P2.5         13
00163 #define CLU.MXB.CLU2.P2.13        14
00164 #define CLU.MXB.CLU2.P2.15        15
00165
00166 #define CLU.MXB.CLU3.CLU0OUT      0
00167 #define CLU.MXB.CLU3.CLU1OUT      1
00168 #define CLU.MXB.CLU3.CLU2OUT      2
00169 #define CLU.MXB.CLU3.CLU3OUT      3
00170 #define CLU.MXB.CLU3.PWM0         4
00171 #define CLU.MXB.CLU3.PWM2         5
00172 #define CLU.MXB.CLU3.I2CSCL       6
00173 #define CLU.MXB.CLU3.SPICSN       7
00174 #define CLU.MXB.CLU3.P1.13        8
00175 #define CLU.MXB.CLU3.P1.14        9

```

```

00176 #define CLU_MXB_CLU3_P1_10      10
00177 #define CLU_MXB_CLU3_P1_11      11
00178 #define CLU_MXB_CLU3_P2_4       12
00179 #define CLU_MXB_CLU3_P2_5       13
00180 #define CLU_MXB_CLU3_P2_14      14
00181 #define CLU_MXB_CLU3_P2_15      15
00182
00183 #define CLU_FN_0                  0x0
00184 #define CLU_FN_1                  0xff
00185 #define CLU_FN_A                  0xf0
00186 #define CLU_FN_NOTA               0x0f
00187 #define CLU_FN_B                  0xcc
00188 #define CLU_FN_NOTB               0x33
00189 #define CLU_FN_C                  0xaa
00190 #define CLU_FN_NOTC               0x55
00191 #define CLU_FN_A_B                0xc0
00192 #define CLU_FN_NOTA_NOTB          0x03
00193 #define CLU_FN_NOTA_B             0x0c
00194 #define CLU_FN_A_NOTB             0x30
00195 #define CLU_FN_A_C                0xa0
00196 #define CLU_FN_NOTA_NOTC          0x05
00197 #define CLU_FN_NOTA_C             0x0a
00198 #define CLU_FN_A_NOTC             0x50
00199 #define CLU_FN_B_C                0x88
00200 #define CLU_FN_NOTB_NOTC          0x11
00201 #define CLU_FN_NOTB_C             0x22
00202 #define CLU_FN_B_NOTC             0x44
00203 #define CLU_FN_NOTA_NOTB_NOTC     0x01
00204 #define CLU_FN_NOTA_NOTB_C        0x02
00205 #define CLU_FN_NOTA_B_NOTC        0x04
00206 #define CLU_FN_NOTA_B_C           0x08
00207 #define CLU_FN_A_NOTB_NOTC        0x10
00208 #define CLU_FN_A_NOTB_C           0x20
00209 #define CLU_FN_A_B_NOTC           0x40
00210 #define CLU_FN_A_B_C              0x80
00211
00212 void CLU_Init(CLU_TypeDef *CLUx, u8 CLU_MODEL_CLUx, CLU_InitTypeDef *CLU_InitStruct);
00213 void CLU_StructInit(CLU_InitTypeDef *CLU_InitStruct);
00214
00215 void CLU_Enable(CLU_TypeDef *CLUx);
00216 void CLU_Disable(CLU_TypeDef *CLUx);
00217
00218 u8 CLU_ReadOutVole(CLU_TypeDef *CLUx, u8 CLU_MODEL_CLUx);
00219 #endif /*_LKS32MC08x_CLU_H */

```

5.7 include/lks32mc07x_cmp.h 文件参考

文件名称: **lks32mc07x_cmp.h**

文件标识: 无

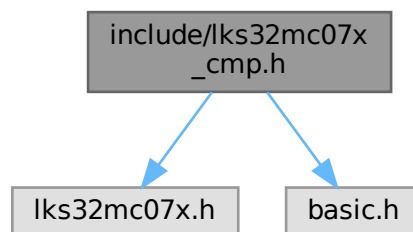
内容摘要: **CMP**外设驱动程序头文件

其它说明: 无

```
#include "lks32mc07x.h"
```

```
#include "basic.h"
```

lks32mc07x_cmp.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了：



结构体

- struct **CMP_SingleTypeDef**
CMP初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参
- struct **CMP_InitTypeDef**
- struct **CMP_TypeDef**
CMP寄存器结构体句柄

宏定义

- #define **CMP** ((CMP_TypeDef *)CMP.BASE)
CMP结构体基地址定义
- #define **CMP_SELN_IN** 0
N端输入
- #define **CMP_SELN_REF** 1
N端内部参考REF输入
- #define **CMP_SELN_DAC0** 2
N端内部DAC0输入
- #define **CMP0_SELN_HALL0_MID** 3
CMP0-N端反电势过零中点输入
- #define **CMP1_SELN_HALL1_MID** 3
CMP1-N端反电势过零中点输入
- #define **CMP2_SELN_DAC1** 3
CMP2-N端内部DAC1输入
- #define **CMP0_SELP_IP0** 0
P端IP0输入
- #define **CMP0_SELP_OPA0_IP** 1
P端OPA0 IP输入
- #define **CMP0_SELP_OPA0_OUT** 2
P端OPA0 OUT
- #define **CMP0_SELP_OPA1_OUT** 3
P端OPA1 OUT
- #define **CMP0_SELP_IP1** 4
P端IP1输入
- #define **CMP0_SELP_IP2** 5
P端IP2输入
- #define **CMP0_SELP_IP3** 6
P端IP3输入
- #define **CMP0_SELP_IP4** 7
P端IP4输入
- #define **CMP1_SELP_IP0** 0
P端IP0输入
- #define **CMP1_SELP_OPA3_IP** 1

- P端OPA3 IP输入*
- #define **CMP1_SEL_P_OPA2_OUT** 2
P端OPA2 OUT
- #define **CMP1_SEL_P_OPA3_OUT** 3
P端OPA3 OUT
- #define **CMP1_SEL_P_IP1** 4
P端IP1输入
- #define **CMP1_SEL_P_IP2** 5
P端IP2输入
- #define **CMP1_SEL_P_IP3** 6
P端IP3输入
- #define **CMP1_SEL_P_DAC0** 7
P端DAC0输入
- #define **CMP2_SEL_P_IP0** 0
P端IP0输入
- #define **CMP2_SEL_P_IP1** 1
P端IP1输入
- #define **CMP2_SEL_P_OPA2_OUT** 2
P端OPA2 OUT
- #define **CMP2_SEL_P_OPA3_OUT** 3
P端OPA3 OUT
- #define **CMP_HIGH_LEVEL** 0 /**高电平输出有效 */
- #define **CMP_LOW_LEVEL** 1 /**低电平输出有效 */
- #define **IRQ_LEVEL_TRIG_MODE** 0
电平触发中断模式
- #define **IRQ_EDGE_TRIG_MODE** 1
边沿触发中断模式
- #define **W_PWM_HIGH_LEVEL** 1
比较器开窗高电平有效
- #define **W_PWM_LOW_LEVEL** 0
比较器开窗低电平有效
- #define **CMP_CHN_0** 0
通道0
- #define **CMP_CHN_1** 1
通道1
- #define **CMP_CHN_2** 2
通道2
- #define **CMP_HYS_20mV** 0
20mV
- #define **CMP_HYS_0mV** 1
0mV

函数

- void **CMP_StructInit** (CMP_InitTypeDef *CMP_InitStruct)
函数名称: *void CMP_StructInit(CMP_InitTypeDef* CMP_InitStruct)*
- void **CMP_Init** (CMP_InitTypeDef *CMP_InitStruct)
函数名称: *CMP_Init(CMP_InitTypeDef* this)*
- void **CMP_Cmd** (u8 CMP_CHN_x, FuncState state)
- void **CMP_ClearIRQFlag** (u8 CMP_CHN_x)

- 函数名称: void CMP_ClearIRQFlag(u8 CMP_CHN_x)
- u8 CMP_GetIRQFlag (u8 CMP_CHN_x)

函数名称: void CMP_Cmd(u8 CMP_CHN_x,FuncState state)
 - uint8_t CMP_GetFilterValue (void)

函数名称: uint8_t CMP_GetFilterValue(void)
 - uint8_t CMP_GetValue (void)

函数名称: uint8_t CMP_GetValue(void)

5.7.1 详细描述

文件名称: lks32mc07x_cmp.h
文件标识: 无
内容摘要: CMP外设驱动程序头文件
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 lks32mc07x_cmp.h 中定义.

5.7.2 宏定义说明

5.7.2.1 CMP

#define CMP ((CMP_TypeDef *)CMP_BASE)

CMP结构体基地址定义

在文件 lks32mc07x_cmp.h 第 98 行定义.

5.7.2.2 CMP0_SELN_HALL0_MID

#define CMP0_SELN_HALL0_MID 3

CMP0-N端反电势过零中点输入

在文件 lks32mc07x_cmp.h 第 104 行定义.

5.7.2.3 CMP0_SELP_IP0

```
#define CMP0_SELP_IP0 0
```

P端IP0输入

在文件 [lks32mc07x_cmp.h](#) 第 109 行定义.

5.7.2.4 CMP0_SELP_IP1

```
#define CMP0_SELP_IP1 4
```

P端IP1输入

在文件 [lks32mc07x_cmp.h](#) 第 113 行定义.

5.7.2.5 CMP0_SELP_IP2

```
#define CMP0_SELP_IP2 5
```

P端IP2输入

在文件 [lks32mc07x_cmp.h](#) 第 114 行定义.

5.7.2.6 CMP0_SELP_IP3

```
#define CMP0_SELP_IP3 6
```

P端IP3输入

在文件 [lks32mc07x_cmp.h](#) 第 115 行定义.

5.7.2.7 CMP0_SELP_IP4

```
#define CMP0_SELP_IP4 7
```

P端IP4输入

在文件 [lks32mc07x_cmp.h](#) 第 116 行定义.

5.7.2.8 CMP0_SELP_OPA0_IP

```
#define CMP0_SELP_OPA0_IP 1
```

P端OPA0 IP输入

在文件 [lks32mc07x_cmp.h](#) 第 110 行定义.

5.7.2.9 CMP0_SELP_OPA0_OUT

```
#define CMP0_SELP_OPA0_OUT 2
```

P端OPA0 OUT

在文件 [lks32mc07x_cmp.h](#) 第 111 行定义.

5.7.2.10 CMP0_SELP_OPA1_OUT

```
#define CMP0_SELP_OPA1_OUT 3
```

P端OPA1 OUT

在文件 [lks32mc07x_cmp.h](#) 第 112 行定义.

5.7.2.11 CMP1_SELN_HALL1_MID

```
#define CMP1_SELN_HALL1_MID 3
```

CMP1-N端反电势过零中点输入

在文件 [lks32mc07x_cmp.h](#) 第 105 行定义.

5.7.2.12 CMP1_SELP_DAC0

```
#define CMP1_SELP_DAC0 7
```

P端DAC0输入

在文件 [lks32mc07x_cmp.h](#) 第 126 行定义.

5.7.2.13 CMP1_SELP_IP0

```
#define CMP1_SELP_IP0 0
```

P端IP0输入

在文件 [lks32mc07x_cmp.h](#) 第 119 行定义.

5.7.2.14 CMP1_SELP_IP1

```
#define CMP1_SELP_IP1 4
```

P端IP1输入

在文件 [lks32mc07x_cmp.h](#) 第 123 行定义.

5.7.2.15 CMP1_SELP_IP2

```
#define CMP1_SELP_IP2 5
```

P端IP2输入

在文件 [lks32mc07x_cmp.h](#) 第 124 行定义.

5.7.2.16 CMP1_SELP_IP3

```
#define CMP1_SELP_IP3 6
```

P端IP3输入

在文件 [lks32mc07x_cmp.h](#) 第 125 行定义.

5.7.2.17 CMP1_SELP_OPA2_OUT

```
#define CMP1_SELP_OPA2_OUT 2
```

P端OPA2 OUT

在文件 [lks32mc07x_cmp.h](#) 第 121 行定义.

5.7.2.18 CMP1_SELP_OPA3_IP

```
#define CMP1_SELP_OPA3_IP 1
```

P端OPA3 IP输入

在文件 [lks32mc07x_cmp.h](#) 第 120 行定义.

5.7.2.19 CMP1_SELP_OPA3_OUT

```
#define CMP1_SELP_OPA3_OUT 3
```

P端OPA3 OUT

在文件 [lks32mc07x_cmp.h](#) 第 122 行定义.

5.7.2.20 CMP2_SELN_DAC1

```
#define CMP2_SELN_DAC1 3
```

CMP2-N端内部DAC1输入

在文件 [lks32mc07x_cmp.h](#) 第 106 行定义.

5.7.2.21 CMP2_SELP_IP0

```
#define CMP2_SELP_IP0 0
```

P端IP0输入

在文件 [lks32mc07x_cmp.h](#) 第 129 行定义.

5.7.2.22 CMP2_SELP_IP1

```
#define CMP2_SELP_IP1 1
```

P端IP1输入

在文件 [lks32mc07x_cmp.h](#) 第 130 行定义.

5.7.2.23 CMP2_SELP_OPA2_OUT

```
#define CMP2_SELP_OPA2_OUT 2
```

P端OPA2 OUT

在文件 [lks32mc07x_cmp.h](#) 第 131 行定义.

5.7.2.24 CMP2_SELP_OPA3_OUT

```
#define CMP2_SELP_OPA3_OUT 3
```

P端OPA3 OUT

在文件 [lks32mc07x_cmp.h](#) 第 132 行定义.

5.7.2.25 CMP_CHN_0

```
#define CMP_CHN_0 0
```

通道0

在文件 [lks32mc07x_cmp.h](#) 第 147 行定义.

5.7.2.26 CMP_CHN_1

```
#define CMP_CHN_1 1
```

通道1

在文件 [lks32mc07x_cmp.h](#) 第 148 行定义.

5.7.2.27 CMP_CHN_2

```
#define CMP_CHN_2 2
```

通道2

比较器回差选择

在文件 [lks32mc07x_cmp.h](#) 第 151 行定义.

5.7.2.28 CMP_HIGH_LEVEL

```
#define CMP_HIGH_LEVEL 0 /**高电平输出有效 */
```

在文件 [lks32mc07x_cmp.h](#) 第 135 行定义.

5.7.2.29 CMP_HYS_0mV

```
#define CMP_HYS_0mV 1
```

0mV

在文件 [lks32mc07x_cmp.h](#) 第 153 行定义.

5.7.2.30 CMP_HYS_20mV

```
#define CMP_HYS_20mV 0
```

20mV

在文件 [lks32mc07x_cmp.h](#) 第 152 行定义.

5.7.2.31 CMP_LOW_LEVEL

```
#define CMP_LOW_LEVEL 1 /**低电平输出有效 */
```

在文件 [lks32mc07x_cmp.h](#) 第 136 行定义.

5.7.2.32 CMP_SELN_DAC0

```
#define CMP_SELN_DAC0 2
```

N端内部DAC0输入

在文件 [lks32mc07x_cmp.h](#) 第 103 行定义.

5.7.2.33 CMP_SELN_IN

```
#define CMP_SELN_IN 0
```

N端输入

在文件 [lks32mc07x_cmp.h](#) 第 101 行定义.

5.7.2.34 CMP_SELN_REF

```
#define CMP_SELN_REF 1
```

N端内部参考REF输入

在文件 [lks32mc07x_cmp.h](#) 第 102 行定义.

5.7.2.35 IRQ_EDGE_TRIG_MODE

```
#define IRQ_EDGE_TRIG_MODE 1
```

边沿触发中断模式

在文件 [lks32mc07x_cmp.h](#) 第 140 行定义.

5.7.2.36 IRQ_LEVEL_TRIG_MODE

```
#define IRQ_LEVEL_TRIG_MODE 0
```

电平触发中断模式

在文件 [lks32mc07x_cmp.h](#) 第 139 行定义.

5.7.2.37 W_PWM_HIGH_LEVEL

```
#define W_PWM_HIGH_LEVEL 1
```

比较器开窗高电平有效

在文件 [lks32mc07x_cmp.h](#) 第 143 行定义.

5.7.2.38 W_PWM_LOW_LEVEL

```
#define W_PWM_LOW_LEVEL 0
```

比较器开窗低电平有效

在文件 [lks32mc07x_cmp.h](#) 第 144 行定义.

5.7.3 函数说明

5.7.3.1 CMP_ClearIRQFlag()

```
void CMP_ClearIRQFlag (  
    u8 CMP_CHN_x )
```

函数名称: void [CMP_ClearIRQFlag\(u8 CMP_CHN_x\)](#)

功能描述: 清除TIM中断标志

参见

被调用函数: 无

参数

输入参数:	CMP_CHN_0, CMP_CHN_1即获取CMP0和CMP1中断标志位
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (CMP_GetIRQFlag (CMP_CHN_1)) //获取CMP1中断标志位
{
    CMP_ClearIRQFlag (CMP_CHN_1); //清除CMP1中断标志位
}
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 214 行定义.

5.7.3.2 CMP_Cmd()

```
void CMP_Cmd (
    u8 CMP_CHN_x,
    FuncState state )
```

5.7.3.3 CMP_GetFilterValue()

```
uint8_t CMP_GetFilterValue (
    void )
```

函数名称: [uint8_t CMP_GetFilterValue\(void\)](#)

功能描述: 读取滤波后的CMP值

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
u8 cmpVal;  
cmpVal = CMP_GetFilterValue();
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 239 行定义.

5.7.3.4 CMP_GetIRQFlag()

```
u8 CMP_GetIRQFlag (  
    u8 CMP_CHN_x )
```

函数名称: void [CMP_Cmd\(u8 CMP_CHN_x,FuncState state\)](#)

功能描述: CMP时钟使能

参见

被调用函数: 无

参数

输入参数:	CMP_CHN_x: CMP_CHN_0/CMP_CHN_1/CMP_CHN_2,state: ENABLE/DISABLE
输出参数:	无

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

```

        CMP_Cmd(CMP_CHN_0,ENABLE); //CMP0使能
@par 修改日志:


| Date        | Version | Author     | Description |
|-------------|---------|------------|-------------|
| 2023年02月22日 | 1.0     | Olive Wang | 创建          |


/
void CMP_Cmd(u8 CMP_CHN_x, FuncState state)
{
    if (CMP_CHN_0 == CMP_CHN_x)
    {
        SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP0, state); // CMP0使能
    }
    else if (CMP_CHN_1 == CMP_CHN_x)
    {
        SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP1, state); // CMP1使能
    }
    else if (CMP_CHN_2 == CMP_CHN_x)
    {
        SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP2, state); // CMP2使能
    }
    SYS_ModuleClockCmd(SYS_Module_CMP, state); // 数字电路时钟使能
}
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 180 行定义.

5.7.3.5 CMP_GetValue()

```
uint8_t CMP_GetValue (
    void )
```

函数名称: uint8_t [CMP_GetValue\(void\)](#)

功能描述: 读取未滤波的CMP值

参见

被调用函数： 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
u8 cmpVal;  
cmpVal = CMP_GetValue();
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 263 行定义.

5.7.3.6 CMP_Init()

```
void CMP_Init (  
    CMP_InitTypeDef * this )
```

函数名称: [CMP_Init\(CMP_InitTypeDef* this\)](#)

功能描述: CMP初始化

参见

被调用函数: [SYS_AnalogModuleClockCmd\(\)](#)

参数

输入参数:	CMP_InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
CMP_InitTypeDef CMP_InitStructure;
CMP_StructInit (&CMP_InitStructure);

CMP_InitStructure.CLK_COM_DIV = 0; // 比较器共用滤波时钟分频
CMP_InitStructure.FT = DISABLE; // 比较器快速比较
CMP_InitStructure.HYS = CMP_HYS_20mV; // 比较器滞回电压

CMP_InitStructure.CMP0_SELP = CMP0_SELP_IP1; // 比较器0正端信号选择
CMP_InitStructure.CMP0_SELN = CMP_SELN_REF; // 比较器0负端信号选择
CMP_InitStructure.CMP0_RE = DISABLE; // 比较器0DMA失能
CMP_InitStructure.CMP0_POL = CMP_HIGH_LEVEL; // 比较器0高电平有效
CMP_InitStructure.CMP0_IRQ_TRIG = IRQ_LEVEL_TRIG_MODE; // 比较器0电平触发中断模式
CMP_InitStructure.CMP0_IN_EN = ENABLE; // 比较器0信号输入使能
CMP_InitStructure.CMP0_IE = ENABLE; // 比较器0信号中断使能
CMP_InitStructure.CMP0_FIL_CLK_DIV16 = 2; // 即滤波宽度=tclk 周期*16*CMP_FltCnt (CMP_FltCnt分频系数, 0~15)
CMP_InitStructure.CMP0_FIL_CLK_DIV2 = 2; // 比较器 2/1/0 滤波时钟使能
CMP_InitStructure.CMP0_CLK_EN = ENABLE; // 比较器时钟使能
CMP_InitStructure.CMP0_EN = ENABLE; // 比较器0开关 操作SYS_AFE_REG5

CMP_Init (&CMP_InitStructure); // 比较器初始化
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 `lks32mc07x_cmp.c` 第 57 行定义.

5.7.3.7 CMP_StructInit()

```
void CMP_StructInit (
    CMP_InitTypeDef * CMP_InitStructure )
```

函数名称: void `CMP_StructInit(CMP_InitTypeDef* CMP_InitStructure)`

功能描述: ADC结构体初始化

参见

被调用函数: 无

参数

输入参数:	CMP.InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
CMP_StructInit CMP_InitStructure;  
CMP_StructInit (&CMP_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 117 行定义.

5.8 lks32mc07x_cmp.h

[浏览该文件的文档.](#)

```
00001  
00014 #ifndef __lks32mc07x_CMP_H  
00015 #define __lks32mc07x_CMP_H  
00016 #include "lks32mc07x.h"  
00017 #include "basic.h"  
00021 typedef struct  
00022 {  
00023     u16 SELP;  
00033     u16 SELN;  
00038     u32 FIL_CLK_DIV16;  
00039     u32 FIL_CLK_DIV2;  
00045     u16 POL;  
00051     u16 W_PWM_POL;  
00057     u16 IRQ_TRIG;  
00058     FuncState CLK_EN;  
00059     FuncState EN;  
00060     FuncState IE;  
00061     FuncState RE;  
00062     FuncState IN_EN;  
00063     FuncState CHN3P_PWM_EN;
```

```

00064     FuncState CHN2P_PWM_EN;
00065     FuncState CHN1P_PWM_EN;
00066     FuncState CHN0P_PWM_EN;
00067 } CMP_SingleTypeDef;
00068 typedef struct
00069 {
00070     u8 HYS;
00071     FuncState FT;
00072     u32 CLK_COM_DIV;
00073     CMP_SingleTypeDef CMP0;
00074     CMP_SingleTypeDef CMP1;
00075     CMP_SingleTypeDef CMP2;
00076 } CMP_InitTypeDef;
00077 typedef struct
00078 {
00079     __IO u32 IE;
00080     __IO u32 IF;
00081     __IO u32 TCLK;
00082     __IO u32 CFG;
00083     __IO u32 BLCWIN;
00084     __IO u32 DATA;
00085 } CMP_TypeDef;
00086 #define CMP ((CMP_TypeDef *)CMP_BASE)
00087
00088 // 比较器信号N端选择
00089 #define CMP_SELN_IN 0
00090 #define CMP_SELN_REF 1
00091 #define CMP_SELN_DAC0 2
00092 #define CMP0_SELN_HALL0_MID 3
00093 #define CMP1_SELN_HALL1_MID 3
00094 #define CMP2_SELN_DAC1 3
00095 // 比较器0 P端选择
00096 #define CMP0_SELP_IP0 0
00097 #define CMP0_SELP_OPA0_IP 1
00098 #define CMP0_SELP_OPA0_OUT 2
00099 #define CMP0_SELP_OPA1_OUT 3
00100 #define CMP0_SELP_IP1 4
00101 #define CMP0_SELP_IP2 5
00102 #define CMP0_SELP_IP3 6
00103 #define CMP0_SELP_IP4 7
00104 // 比较器1 P端选择
00105 #define CMP1_SELP_IP0 0
00106 #define CMP1_SELP_OPA3_IP 1
00107 #define CMP1_SELP_OPA2_OUT 2
00108 #define CMP1_SELP_OPA3_OUT 3
00109 #define CMP1_SELP_IP1 4
00110 #define CMP1_SELP_IP2 5
00111 #define CMP1_SELP_IP3 6
00112 #define CMP1_SELP_DAC0 7
00113 // 比较器2 P端选择
00114 #define CMP2_SELP_IP0 0
00115 #define CMP2_SELP_IP1 1
00116 #define CMP2_SELP_OPA2_OUT 2
00117 #define CMP2_SELP_OPA3_OUT 3
00118 // 有效极性选择
00119 #define CMP_HIGH_LEVEL 0
00120 #define CMP_LOW_LEVEL 1
00121 // 开窗 PWM 信号极性选择
00122 #define IRQ_LEVEL_TRIG_MODE 0
00123 #define IRQ_EDGE_TRIG_MODE 1
00124 // 开窗 PWM 信号极性选择
00125 #define W_PWM_HIGH_LEVEL 1
00126 #define W_PWM_LOW_LEVEL 0
00127 // 比较通道选择
00128 #define CMP_CHN_0 0
00129 #define CMP_CHN_1 1
00130 #define CMP_CHN_2 2
00131 #define CMP_HYS_20mV 0
00132 #define CMP_HYS_0mV 1
00133 void CMP_StructInit(CMP_InitTypeDef *CMP_InitStruct); /* CMP结构体初始化*/
00134 void CMP_Init(CMP_InitTypeDef *CMP_InitStruct); /* CMP初始化*/
00135 void CMP_Cmd(u8 CMP_CHN_x, FuncState state); /*CMP使能 */
00136 void CMP_ClearIRQFlag(u8 CMP_CHN_x); /* 清CMP中断 */
00137 u8 CMP_GetIRQFlag(u8 CMP_CHN_x); /* 读CMP中断 */
00138 uint8_t CMP_GetFilterValue(void); /*读取滤波后的比较器值*/
00139 uint8_t CMP_GetValue(void); /*读取未滤波的比较器值*/
00140
00141 #endif /*_lks32mc07x_OPA_H */
00142 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/
00143 /* -----END OF FILE----- */

```

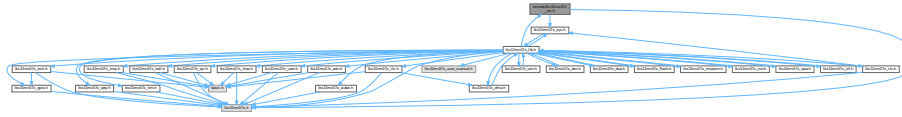
5.9 include/lks32mc07x_crc.h 文件参考

CRC库函数头文件

```
#include "lks32mc07x.h"
```

```
#include "lks32mc07x_sys.h"
```

lks32mc07x_crc.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [CRC_TypeDef](#)
 < Includes -----
- struct [CRC_InitTypeDef](#)

宏定义

- #define [CRC_MODEL_CRC_8_0](#)
 8 07 00 00 false false
- #define [CRC_MODEL_CRC_8_ITU_1](#)
 8 07 00 55 false false
- #define [CRC_MODEL_CRC_8_ROHC_2](#)
 8 07 FF 00 true true
- #define [CRC_MODEL_CRC_8_MAXIMx_3](#)
 8 31 00 00 true true
- #define [CRC_MODEL_CRC_16_IBMx1_4](#)
 16 8005 0000 0000 true true
- #define [CRC_MODEL_CRC_16_MAXIM_5](#)
 16 8005 0000 FFFF true true
- #define [CRC_MODEL_CRC_16_USBx1_6](#)
 16 8005 FFFF FFFF true true
- #define [CRC_MODEL_CRC_16_MODBU_7](#)
 16 8005 FFFF 0000 true true
- #define [CRC_MODEL_CRC_16_CCITT_8](#)
 16 1021 0000 0000 true true
- #define [CRC_MODEL_CRC_16_CCITT_FALSE_9](#)
 16 1021 FFFF 0000 false false
- #define [CRC_MODEL_CRC_16_X25_10](#)

- 16 1021 FFFF FFFF true true*
- #define [CRC_MODEL_CRC_16_XMODEM](#) 11
- 16 1021 0000 0000 false false*
- #define [CRC_MODEL_CRC_16_DNP](#) 12
- 16 3D65 0000 FFFF true true*
- #define [CRC_MODEL_CRC_32](#) 13
- 32 04C11DB7 FFFFFFFF FFFFFFFF true true*
- #define [CRC_MODEL_CRC_32_MPEG_2](#) 14
- 32 04C11DB7 FFFFFFFF 00000000 false false*
- #define [CRC_WIDTH_32](#) 0
- 32bits*
- #define [CRC_WIDTH_16](#) 1
- 16bits*
- #define [CRC_WIDTH_8](#) 2
- 8bits*
- #define [CRC_WIDTH_7](#) 3
- 7bits*

函数

- void [CRC_Init](#) ([CRC_TypeDef](#) *CRCx, [CRC_InitTypeDef](#) *CRC_InitStruct)
CRC计算初始化
- void [CRC_StructInit](#) ([CRC_InitTypeDef](#) *CRC_InitStruct)
CRC初始化结构体初始化
- u32 [CRC_Calc](#) ([CRC_TypeDef](#) *CRCx, u8 *data, u32 len)
CRC计算
- u32 [CRC_CalcCommon](#) ([CRC_TypeDef](#) *CRCx, u8 *data, u32 len, u16 CRC_MODEL_x)
CRC计算接口
- void [CRC_Enable](#) ([CRC_TypeDef](#) *CRCx)
CRC模块时钟使能
- void [CRC_Disable](#) ([CRC_TypeDef](#) *CRCx)
CRC模块时钟关闭

5.9.1 详细描述

CRC库函数头文件

作者

YangZJ

日期

2023-01-02

在文件 [lks32mc07x_crc.h](#) 中定义.

5.9.2 宏定义说明

5.9.2.1 CRC_MODEL_CRC_16_CCITT

```
#define CRC_MODEL_CRC_16_CCITT 8
```

```
16 1021 0000 0000 true true
```

在文件 [lks32mc07x_crc.h](#) 第 44 行定义.

5.9.2.2 CRC_MODEL_CRC_16_CCITT_FALSE

```
#define CRC_MODEL_CRC_16_CCITT_FALSE 9
```

```
16 1021 FFFF 0000 false false
```

在文件 [lks32mc07x_crc.h](#) 第 45 行定义.

5.9.2.3 CRC_MODEL_CRC_16_DNP

```
#define CRC_MODEL_CRC_16_DNP 12
```

```
16 3D65 0000 FFFF true true
```

在文件 [lks32mc07x_crc.h](#) 第 48 行定义.

5.9.2.4 CRC_MODEL_CRC_16_IBMx1

```
#define CRC_MODEL_CRC_16_IBMx1 4
```

```
16 8005 0000 0000 true true
```

在文件 [lks32mc07x_crc.h](#) 第 40 行定义.

5.9.2.5 CRC_MODEL_CRC_16_MAXIM

```
#define CRC_MODEL_CRC_16_MAXIM 5
```

```
16 8005 0000 FFFF true true
```

在文件 [lks32mc07x_crc.h](#) 第 41 行定义.

5.9.2.6 CRC_MODEL_CRC_16_MODBU

```
#define CRC_MODEL_CRC_16_MODBU 7
```

```
16 8005 FFFF 0000 true true
```

在文件 [lks32mc07x_crc.h](#) 第 43 行定义.

5.9.2.7 CRC_MODEL_CRC_16_USBx1

```
#define CRC_MODEL_CRC_16_USBx1 6
```

```
16 8005 FFFF FFFF true true
```

在文件 [lks32mc07x_crc.h](#) 第 42 行定义.

5.9.2.8 CRC_MODEL_CRC_16_X25

```
#define CRC_MODEL_CRC_16_X25 10
```

```
16 1021 FFFF FFFF true true
```

在文件 [lks32mc07x_crc.h](#) 第 46 行定义.

5.9.2.9 CRC_MODEL_CRC_16_XMODEM

```
#define CRC_MODEL_CRC_16_XMODEM 11
```

```
16 1021 0000 0000 false false
```

在文件 [lks32mc07x_crc.h](#) 第 47 行定义.

5.9.2.10 CRC_MODEL_CRC_32

```
#define CRC_MODEL_CRC_32 13
```

```
32 04C11DB7 FFFFFFFF FFFFFFFF true true
```

在文件 [lks32mc07x_crc.h](#) 第 49 行定义.

5.9.2.11 CRC_MODEL_CRC_32_MPEG_2

```
#define CRC_MODEL_CRC_32_MPEG_2 14
```

```
32 04C11DB7 FFFFFFFF 00000000 false false
```

在文件 [lks32mc07x_crc.h](#) 第 50 行定义.

5.9.2.12 CRC_MODEL_CRC_8

```
#define CRC_MODEL_CRC_8 0
```

```
8 07 00 00 false false
```

在文件 [lks32mc07x_crc.h](#) 第 36 行定义.

5.9.2.13 CRC_MODEL_CRC_8_ITU

```
#define CRC_MODEL_CRC_8_ITU 1
```

8 07 00 55 false false

在文件 [lks32mc07x_crc.h](#) 第 37 行定义.

5.9.2.14 CRC_MODEL_CRC_8_MAXIMx

```
#define CRC_MODEL_CRC_8_MAXIMx 3
```

8 31 00 00 true true

在文件 [lks32mc07x_crc.h](#) 第 39 行定义.

5.9.2.15 CRC_MODEL_CRC_8_ROHC

```
#define CRC_MODEL_CRC_8_ROHC 2
```

8 07 FF 00 true true

在文件 [lks32mc07x_crc.h](#) 第 38 行定义.

5.9.2.16 CRC_WIDTH_16

```
#define CRC_WIDTH_16 1
```

16bits

在文件 [lks32mc07x_crc.h](#) 第 53 行定义.

5.9.2.17 CRC_WIDTH_32

```
#define CRC_WIDTH_32 0
```

32bits

在文件 [lks32mc07x_crc.h](#) 第 52 行定义.

5.9.2.18 CRC_WIDTH_7

```
#define CRC_WIDTH_7 3
```

7bits

在文件 [lks32mc07x_crc.h](#) 第 55 行定义.

5.9.2.19 CRC_WIDTH_8

```
#define CRC_WIDTH_8 2
```

8bits

在文件 `lks32mc07x_crc.h` 第 54 行定义。

5.10 lks32mc07x_crc.h

[浏览该文件的文档。](#)

```
00001
00007 #ifndef __LKS32MC07X_CRC_H
00008 #define __LKS32MC07X_CRC_H
00010 #include "lks32mc07x.h"
00011 #include "lks32mc07x_sys.h"
00012
00013 typedef struct
00014 {
00015     __IO u32 DR;
00016     __IO u32 CR;
00017     __IO u32 INIT;
00018     __IO u32 POL;
00019 } CRC_TypeDef;
00020
00021 #ifndef CRC0
00022 #define CRC0 ((CRC_TypeDef *)CRC0_BASE)
00023 #endif
00024
00025 typedef struct
00026 {
00027     u32 POLY;
00028     u32 INIT;
00029     u32 XOROUT;
00030     u8 REFIN;
00031     u8 REFOUT;
00032     u8 WIDTH;
00033     u8 null;
00034 } CRC_InitTypeDef;
00035
00036 #define CRC_MODEL_CRC_8 0
00037 #define CRC_MODEL_CRC_8_ITU 1
00038 #define CRC_MODEL_CRC_8_ROHC 2
00039 #define CRC_MODEL_CRC_8_MAXIMx 3
00040 #define CRC_MODEL_CRC_16_IBMx1 4
00041 #define CRC_MODEL_CRC_16_MAXIM 5
00042 #define CRC_MODEL_CRC_16_USBx1 6
00043 #define CRC_MODEL_CRC_16_MODBU 7
00044 #define CRC_MODEL_CRC_16_CCITT 8
00045 #define CRC_MODEL_CRC_16_CCITT_FALSE 9
00046 #define CRC_MODEL_CRC_16_X25 10
00047 #define CRC_MODEL_CRC_16_XMODEM 11
00048 #define CRC_MODEL_CRC_16_DNP 12
00049 #define CRC_MODEL_CRC_32 13
00050 #define CRC_MODEL_CRC_32_MPEG_2 14
00051
00052 #define CRC_WIDTH_32 0
00053 #define CRC_WIDTH_16 1
00054 #define CRC_WIDTH_8 2
00055 #define CRC_WIDTH_7 3
00056
00057 void CRC_Init(CRC_TypeDef *CRCx, CRC_InitTypeDef *CRC_InitStruct);
00058 void CRC_StructInit(CRC_InitTypeDef *CRC_InitStruct);
00059
00060 u32 CRC_Calc(CRC_TypeDef *CRCx, u8 *data, u32 len);
00061 u32 CRC_CalcCommon(CRC_TypeDef *CRCx, u8 *data, u32 len, u16 CRC_MODEL_x);
00062 void CRC_Enable(CRC_TypeDef *CRCx);
00063 void CRC_Disable(CRC_TypeDef *CRCx);
00064
00065 #endif /*__LKS32MC08X_CRC_H */
```

5.11 include/lks32mc07x_dac.h 文件参考

文件名称: [lks32mc07x_dac.h](#)

文件标识: 无

内容摘要: DAC外设驱动程序头文件

其它说明: 无

```
#include "lks32mc07x_lib.h"
```

[lks32mc07x_dac.h](#) 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [DAC_InitTypeDef](#)

宏定义

- #define [DAC_RANGE_4V85](#) 0
DAC 4.85V量程
- #define [DAC_RANGE_1V2](#) 1
DAC 1.2V量程
- #define [DAC_Channel_0](#) 0
DAC 通道0
- #define [DAC_Channel_1](#) 1
DAC 通道1
- #define [DAC0_UTIMER0_TRG_EN](#) BIT0
TIMER0 过零事件触发 DAC0 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能
- #define [DAC0_UTIMER1_TRG_EN](#) BIT1
TIMER1 过零事件触发 DAC0 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能
- #define [DAC0_UTIMER2_TRG_EN](#) BIT2
TIMER2 过零事件触发 DAC0 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能
- #define [DAC0_UTIMER3_TRG_EN](#) BIT3
TIMER3 过零事件触发 DAC0 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能
- #define [DAC1_UTIMER0_TRG_EN](#) BIT8
TIMER0 过零事件触发 DAC1 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能
- #define [DAC1_UTIMER1_TRG_EN](#) BIT9
TIMER1 过零事件触发 DAC1 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能
- #define [DAC1_UTIMER2_TRG_EN](#) BIT10
TIMER2 过零事件触发 DAC1 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能
- #define [DAC1_UTIMER3_TRG_EN](#) BIT11
TIMER3 过零事件触发 DAC1 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能

函数

- void **DAC_StructInit** (**DAC_InitTypeDef** *DAC_InitStruct)
函数名称: void **DAC_StructInit**(**DAC_InitTypeDef*** **DAC_InitStruct**)
- void **DAC_Init** (uint8_t DAC_Channel, **DAC_InitTypeDef** *DAC_InitStruct)
函数名称: void **DAC_Init**(uint8_t DAC_Channel, **DAC_InitTypeDef*** **DAC_InitStruct**)
- void **DAC_OutputValue** (uint8_t DAC_Channel, uint32_t DACValue)
函数名称: void **DAC_OutputValue**(uint8_t DAC_Channel, uint32_t DACValue)
- void **DAC_OutputVoltage** (uint8_t DAC_Channel, uint16_t DACVoltage)
函数名称: void **DAC_OutputVoltage**(uint8_t DAC_Channel, uint32_t DACVoltage)
- void **DAC_Cmd** (uint8_t DAC_Channel, FuncState state)
函数名称: void **DAC_Cmd**(uint8_t DAC_Channel, **FuncState** state)
- void **DAC_Output** (uint8_t DAC_Channel, **DAC_InitTypeDef** *DAC_InitStruct)

5.11.1 详细描述

文件名称: **lks32mc07x_dac.h**
文件标识: 无
内容摘要: DAC外设驱动程序头文件
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年3月22日	1.0	Olive Wang	创建

修改日志:

在文件 **lks32mc07x_dac.h** 中定义.

5.11.2 宏定义说明

5.11.2.1 DAC0.UTIMER0.TRG.EN

```
#define DAC0_UTIMER0_TRG_EN BIT0
```

TIMER0 过零事件触发 DAC0 步进使能, 高有效,DISABLE, 不使能; ENABLE, 使能

在文件 **lks32mc07x_dac.h** 第 41 行定义.

5.11.2.2 DAC0_UTIMER1_TRG_EN

```
#define DAC0_UTIMER1_TRG_EN BIT1
```

TIMER1 过零事件触发 DAC0 步进使能，高有效,DISABLE，不使能；ENABLE，使能
在文件 [lks32mc07x_dac.h](#) 第 42 行定义.

5.11.2.3 DAC0_UTIMER2_TRG_EN

```
#define DAC0_UTIMER2_TRG_EN BIT2
```

TIMER2 过零事件触发 DAC0 步进使能，高有效,DISABLE，不使能；ENABLE，使能
在文件 [lks32mc07x_dac.h](#) 第 43 行定义.

5.11.2.4 DAC0_UTIMER3_TRG_EN

```
#define DAC0_UTIMER3_TRG_EN BIT3
```

TIMER3 过零事件触发 DAC0 步进使能，高有效,DISABLE，不使能；ENABLE，使能
在文件 [lks32mc07x_dac.h](#) 第 44 行定义.

5.11.2.5 DAC1_UTIMER0_TRG_EN

```
#define DAC1_UTIMER0_TRG_EN BIT8
```

TIMER0 过零事件触发 DAC1 步进使能，高有效,DISABLE，不使能；ENABLE，使能
在文件 [lks32mc07x_dac.h](#) 第 46 行定义.

5.11.2.6 DAC1_UTIMER1_TRG_EN

```
#define DAC1_UTIMER1_TRG_EN BIT9
```

TIMER1 过零事件触发 DAC1 步进使能，高有效,DISABLE，不使能；ENABLE，使能
在文件 [lks32mc07x_dac.h](#) 第 47 行定义.

5.11.2.7 DAC1_UTIMER2_TRG_EN

```
#define DAC1_UTIMER2_TRG_EN BIT10
```

TIMER2 过零事件触发 DAC1 步进使能，高有效,DISABLE，不使能；ENABLE，使能
在文件 [lks32mc07x_dac.h](#) 第 48 行定义.

5.11.2.8 DAC1_UTIMER3_TRG_EN

```
#define DAC1_UTIMER3_TRG_EN BIT11
```

TIMER3 过零事件触发 DAC1 步进使能，高有效,DISABLE，不使能；ENABLE，使能
在文件 [lks32mc07x-dac.h](#) 第 49 行定义.

5.11.2.9 DAC_Channel_0

```
#define DAC_Channel_0 0
```

DAC 通道0

在文件 [lks32mc07x-dac.h](#) 第 38 行定义.

5.11.2.10 DAC_Channel_1

```
#define DAC_Channel_1 1
```

DAC 通道1

在文件 [lks32mc07x-dac.h](#) 第 39 行定义.

5.11.2.11 DAC_RANGE_1V2

```
#define DAC_RANGE_1V2 1
```

DAC 1.2V量程

在文件 [lks32mc07x-dac.h](#) 第 36 行定义.

5.11.2.12 DAC_RANGE_4V85

```
#define DAC_RANGE_4V85 0
```

DAC 4.85V量程

在文件 [lks32mc07x-dac.h](#) 第 35 行定义.

5.11.3 函数说明

5.11.3.1 DAC_Cmd()

```
void DAC_Cmd (
    uint8_t DAC_Channel,
    FuncState state )
```

函数名称: void [DAC_Cmd\(uint8_t DAC_Channel,FuncState state\)](#)

功能描述: DAC外设使能

参见

被调用函数: 无

参数

输入参数:	DAC_Channel: 通道DAC_Channel_0/DAC_Channel_1, state: ENABLE/DISABLE
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
DAC_Cmd(DAC_Channel_0, ENABLE);
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x-dac.c](#) 第 201 行定义.

函数调用图:



5.11.3.2 DAC_Init()

```
void DAC_Init (
    uint8_t DAC_Channel,
    DAC_InitTypeDef * DAC_InitStruct )
```

函数名称: void DAC_init(uint8_t DAC_Channel,DAC_InitTypeDef* DAC_InitStruct)

功能描述: DAC初始化函数

参数

输入参数:	DAC_Channel:DAC_Channel_0/DAC_Channel_1,DAC_InitTypeDef
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

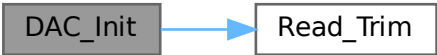
```
DAC_InitTypeDef DAC_InitStructure;
DAC_StructInit (&DAC_InitStructure);
DAC_InitStructure.DAC_GAIN = DAC_RANGE_1V2; //DAC量程为1.2V
DAC_InitStructure.DACOUT_EN = DISABLE ; //DAC通过IO P0[0]输出
DAC_InitStructure.TIG_CH_EN = DISABLE; //失能UTIMER触发
DAC_InitStructure.DAC_STEP = 0; //步进值为0
DAC_Init (DAC_Channel_0, &DAC_InitStructure); // DAC初始化
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x-dac.c](#) 第 67 行定义.

函数调用图:



5.11.3.3 DAC_Output()

```
void DAC_Output (
    uint8_t DAC_Channel,
    DAC_InitTypeDef * DAC_InitStruct )
```

在文件 lks32mc07x.dac.c 第 213 行定义.

5.11.3.4 DAC_OutputValue()

```
void DAC_OutputValue (
    uint8_t DAC_Channel,
    uint32_t DACValue )
```

函数名称: void DAC_OutputValue(uint8_t DAC_Channel,uint32_t DACValue)

功能描述: DAC输出数字量数值设置

参见
被调用函数: 无

参数

输入参数:	DAC_Channel: DAC_Channel_0/DAC_Channel_1, DACValue:DAC输出电压对应数字量
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
DAC_OutputValue(DAC_Channel_0,2048); //DAC输出2048*1.2/4096 = 0.6V
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_dac.c](#) 第 122 行定义.

5.11.3.5 DAC_OutputVoltage()

```
void DAC_OutputVoltage (
    uint8_t DAC_Channel,
    uint16_t DACVoltage )
```

函数名称: void DAC_OutputVoltage(uint8_t DAC_Channel,uint16_t DACVoltage)

功能描述: DAC输出模拟量数值设置

参见

被调用函数: 无

参数

输入参数:	DAC_Channel: DAC_Channel.0/DAC_Channel.1, DACVoltage:DAC输出电压对应数字量
输出参数:	无

返回

返回值: 无

注解

其它说明: 输入变量DACVoltage为Q12格式, 即输出1V对应DACVoltage = $1 < 12 = 4096$

警告

无

示例代码:

```
DAC_OutputVoltage(DAC_Channel1_0,BIT12 * 0.6); //DAC输出0.6V
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x-dac.c](#) 第 155 行定义.

5.11.3.6 DAC_StructInit()

```
void DAC_StructInit (  
    DAC_InitTypeDef * DAC_InitStruct )
```

函数名称: void [DAC_StructInit\(DAC_InitTypeDef* DAC_InitStruct\)](#)

功能描述: DAC结构体初始化

参见

被调用函数: 无

参数

输入参数:	ADC_InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
DAC\_StructInit DAC_InitStructure;  
DAC\_StructInit (&DAC_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x-dac.c](#) 第 37 行定义.

5.12 lks32mc07x_dac.h

浏览该文件的文档.

```

00001
00014 #ifndef _LKS32MC07X_DAC_H_
00015 #define _LKS32MC07X_DAC_H_
00016 /* Includes ----- */
00017
00018 #include "lks32mc07x.lib.h"
00019
00020 typedef struct
00021 {
00022     uint8_t DAC_GAIN;
00023     uint8_t DACOUT_EN;
00024     int8_t DAC_STEP;
00025     uint8_t TIG_CH_EN;
00026 } DAC_InitTypeDef;
00027
00028 /* -----DAC操作相关定义 ----- */
00029
00030 #define DAC_RANGE_4V85 0
00031 #define DAC_RANGE_1V2 1
00032 #define DAC_Channel_0 0
00033 #define DAC_Channel_1 1
00034 #define DAC0_UTIMER0_TRG_EN BIT0
00035 #define DAC0_UTIMER1_TRG_EN BIT1
00036 #define DAC0_UTIMER2_TRG_EN BIT2
00037 #define DAC0_UTIMER3_TRG_EN BIT3
00038 #define DAC1_UTIMER0_TRG_EN BIT8
00039 #define DAC1_UTIMER1_TRG_EN BIT9
00040 #define DAC1_UTIMER2_TRG_EN BIT10
00041 #define DAC1_UTIMER3_TRG_EN BIT11
00042 void DAC_StructInit(DAC_InitTypeDef *DAC_InitStruct); /* DAC结构体初始化 */
00043 void DAC_Init(uint8_t DAC_Channel, DAC_InitTypeDef *DAC_InitStruct); /* DAC初始化 */
00044 void DAC_OutputValue(uint8_t DAC_Channel, uint32_t DACValue); /* DAC输出数值设置--数字量 */
00045 void DAC_OutputVoltage(uint8_t DAC_Channel, uint16_t DACVoltage); /* DAC输出模拟量数值设置--模拟量 */
00046 void DAC_Cmd(uint8_t DAC_Channel, FuncState state); /* DAC通道时钟使能 */
00047 void DAC_Output(uint8_t DAC_Channel, DAC_InitTypeDef *DAC_InitStruct); /* DAC输出至IO口使能配置 */
00048
00049 #endif /* _lks32mc07x_OPA_H */
00050
00051 /****** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE***** */

```

5.13 include/lks32mc07x_dma.h 文件参考

文件名称: [lks32mc07x_dma.h](#)

文件标识: 无

内容摘要: DMA外设驱动程序头文件

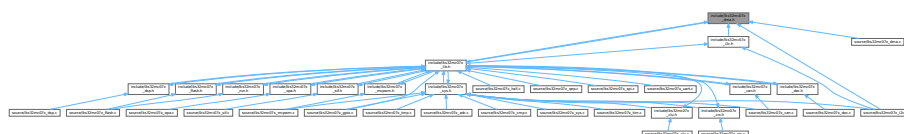
其它说明: 无

#include "lks32mc07x.lib.h"

lks32mc07x_dma.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [DMA_RegTypeDef](#)
DMA通道相关寄存器对象定义
- struct [DMA_MODULERegTypeDef](#)
DMA控制状态寄存器对象定义
- union [stru_DMA_REN](#)
DMA搬运使能源定义
- struct [DMA_InitTypeDef](#)
DMA初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

宏定义

- #define [DMA_CHN0](#) (([DMA_RegTypeDef](#) *)0x40011200)
DMA通道0 1 2 3结构体基地址定义
- #define [DMA_CHN1](#) (([DMA_RegTypeDef](#) *)([DMA0_BASE](#) + 0x20))
- #define [DMA_CHN2](#) (([DMA_RegTypeDef](#) *)([DMA0_BASE](#) + 0x40))
- #define [DMA_CHN3](#) (([DMA_RegTypeDef](#) *)([DMA0_BASE](#) + 0x60))
- #define [DMA_TOP](#) (([DMA_MODULERegTypeDef](#) *)([DMA0_BASE](#) + 0x80))
- #define [DMA_TCIE](#) BIT0 /* 传输完成中断使能，高有效 */
- #define [DMA_TEIE](#) BIT1 /* 错误中断使能，高有效 */
- #define [PERI2MEMORY](#) 0 /* 外设至内存 */
- #define [MEMORY2PERI](#) 1 /* 内存至外设 */
- #define [DMA_BYTE_TRANS](#) 0 /* 访问位宽， 0:byte */
- #define [DMA_HALFWORD_TRANS](#) 1 /* 访问位宽， 1:half-word */
- #define [DMA_WORD_TRANS](#) 2 /* 访问位宽， 2:word */
- #define [DMA_REQ_SW](#) BIT31 /* SW 触发*/
DMA搬运触发源位宏定义
- #define [DMA_REQ_GPIO](#) BIT29 /* GPIO 触发*/
- #define [DMA_REQ_CMP](#) BIT26 /* [CMP](#) 触发*/
- #define [DMA_REQ_SIF](#) BIT25 /* [HALL1](#) 触发*/
- #define [DMA_REQ_HALL](#) BIT24 /* [HALL0](#) 触发*/
- #define [DMA_REQ_UART1_TX](#) BIT19 /* [UART1_TX](#) 触发*/
- #define [DMA_REQ_UART1_RX](#) BIT18 /* [UART1_RX](#) 触发*/
- #define [DMA_REQ_UART0_TX](#) BIT17 /* [UART0_TX](#) 触发*/
- #define [DMA_REQ_UART0_RX](#) BIT16 /* [UART0_RX](#) 触发*/
- #define [DMA_REQ_I2C0_TX](#) BIT15 /* [I2C0_TX](#) 触发*/
- #define [DMA_REQ_I2C0_RX](#) BIT14 /* [I2C0_RX](#) 触发*/
- #define [DMA_REQ_MCPWM1](#) BIT13 /* [MCPWM1](#) 触发*/
- #define [DMA_REQ_MCPWM0](#) BIT12 /* [MCPWM0](#) 触发*/
- #define [DMA_REQ_SPI_TX](#) BIT11 /* [SPI1_TX](#) 触发*/
- #define [DMA_REQ_SPI_RX](#) BIT10 /* [SPI1_RX](#) 触发*/
- #define [DMA_REQ_TIMER4](#) BIT7 /* [TIMER4](#) 触发*/
- #define [DMA_REQ_TIMER3](#) BIT6 /* [TIMER3](#) 触发*/
- #define [DMA_REQ_TIMER2](#) BIT5 /* [TIMER2](#) 触发*/
- #define [DMA_REQ_TIMER1](#) BIT4 /* [TIMER1](#) 触发*/
- #define [DMA_REQ_TIMER0](#) BIT3 /* [TIMER0](#) 触发*/
- #define [DMA_REQ_ADC1](#) BIT1 /* [ADC1](#) 触发*/
- #define [DMA_REQ_ADC0](#) BIT0 /* [ADC0](#) 触发*/
- #define [DMA_IF_CH0](#) BIT0
DMA中断标志位MASK
- #define [DMA_IF_CH1](#) BIT1
- #define [DMA_IF_CH2](#) BIT2

- #define DMA_IF_CH3 BIT3
- #define DMA_IE_CH0 BIT0
DMA中断使能位MASK
- #define DMA_IE_CH1 BIT1
- #define DMA_IE_CH2 BIT2
- #define DMA_IE_CH3 BIT3

函数

- void DMA_Init (DMA_RegTypeDef *DMAx, DMA_InitTypeDef *DMAInitStruct)
函数名称: void DMA_Init(DMA_RegTypeDef *DMAx, DMA_InitTypeDef *DMAInitStruct)
- void DMA_StructInit (DMA_InitTypeDef *DMAInitStruct)
函数名称: void DMA_StructInit(DMA_InitTypeDef *DMAInitStruct)
- u32 DMA_GetIRQFlag (u32 DMA_IF_x)
函数名称: uint32_t DMA_GetIRQFlag(DMA_RegTypeDef *DMAx, u32 timer_if)
- void DMA_ClearIRQFlag (u32 DMA_IF_x)
函数名称: void DMA_ClearIRQFlag(DMA_RegTypeDef *DMAx, uint32_t tempFlag)
- u32 DMA_GetFlag (u32 DMA_IF_x)
- void DMA_Enable (void)
函数名称: void DMA_Enable(void)
- void DMA_Disable (void)
函数名称: void DMA_Disable(void)
- void DMA_SetAddr (DMA_RegTypeDef *DMA_CHx, u32 addr1, u32 addr2)
函数名称: void DMA_SetAddr(DMA_RegTypeDef *DMA_CHx, u32 addr1, u32 addr2)
- void DMA_SetLength (DMA_RegTypeDef *DMA_CHx, u32 len)
函数名称: void DMA_SetLength(DMA_RegTypeDef *DMA_CHx, u32 len)
- void DMA_Trigger (DMA_RegTypeDef *DMA_CHx)
函数名称: void DMA_Trigger(DMA_RegTypeDef *DMA_CHx)
- void DMA_SetRequest (DMA_RegTypeDef *DMA_CHx, u32 DMA_REQ_x)
函数名称: void DMA_SetRequest(DMA_RegTypeDef *DMA_CHx, u32 DMA_REQ_x)
- void DMA_CHx_EN (DMA_RegTypeDef *DMAx, u32 Channel_EN)
函数名称: void DMA_CHx_EN(DMA_RegTypeDef *DMAx, u32 Channel_EN)

5.13.1 详细描述

文件名称: lks32mc07x.dma.h
 文件标识: 无
 内容摘要: DMA外设驱动程序头文件
 其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.h](#) 中定义.

5.13.2 宏定义说明

5.13.2.1 DMA_BYTE_TRANS

```
#define DMA_BYTE_TRANS 0 /* 访问位宽, 0:byte */
```

在文件 [lks32mc07x_dma.h](#) 第 119 行定义.

5.13.2.2 DMA_CHN0

```
#define DMA_CHN0 ((DMA_RegTypeDef *)0x40011200)
```

DAM通道0 1 2 3结构体基地址定义

在文件 [lks32mc07x_dma.h](#) 第 45 行定义.

5.13.2.3 DMA_CHN1

```
#define DMA_CHN1 ((DMA_RegTypeDef *) (DMA0_BASE + 0x20))
```

在文件 [lks32mc07x_dma.h](#) 第 46 行定义.

5.13.2.4 DMA_CHN2

```
#define DMA_CHN2 ((DMA_RegTypeDef *) (DMA0_BASE + 0x40))
```

在文件 [lks32mc07x_dma.h](#) 第 47 行定义.

5.13.2.5 DMA_CHN3

```
#define DMA_CHN3 ((DMA_RegTypeDef *) (DMA0_BASE + 0x60))
```

在文件 [lks32mc07x_dma.h](#) 第 48 行定义.

5.13.2.6 DMA_HALFWORD_TRANS

```
#define DMA_HALFWORD_TRANS 1 /* 访问位宽, 1:half-word */
```

在文件 [lks32mc07x_dma.h](#) 第 120 行定义.

5.13.2.7 DMA_IE_CH0

```
#define DMA_IE_CH0 BIT0
```

DMA中断使能位MASK

在文件 [lks32mc07x_dma.h](#) 第 160 行定义.

5.13.2.8 DMA_IE_CH1

```
#define DMA_IE_CH1 BIT1
```

在文件 [lks32mc07x_dma.h](#) 第 161 行定义.

5.13.2.9 DMA_IE_CH2

```
#define DMA_IE_CH2 BIT2
```

在文件 [lks32mc07x_dma.h](#) 第 162 行定义.

5.13.2.10 DMA_IE_CH3

```
#define DMA_IE_CH3 BIT3
```

在文件 [lks32mc07x_dma.h](#) 第 163 行定义.

5.13.2.11 DMA_IF_CH0

```
#define DMA_IF_CH0 BIT0
```

DMA中断标志位MASK

在文件 [lks32mc07x_dma.h](#) 第 152 行定义.

5.13.2.12 DMA_IF_CH1

```
#define DMA_IF_CH1 BIT1
```

在文件 [lks32mc07x_dma.h](#) 第 153 行定义.

5.13.2.13 DMA_IF_CH2

```
#define DMA_IF_CH2 BIT2
```

在文件 [lks32mc07x_dma.h](#) 第 154 行定义.

5.13.2.14 DMA_IF_CH3

```
#define DMA_IF_CH3 BIT3
```

在文件 [lks32mc07x_dma.h](#) 第 155 行定义.

5.13.2.15 DMA_REQ_ADC0

```
#define DMA_REQ_ADC0 BIT0 /* ADC0 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 147 行定义.

5.13.2.16 DMA_REQ_ADC1

```
#define DMA_REQ_ADC1 BIT1 /* ADC1 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 146 行定义.

5.13.2.17 DMA_REQ_CMP

```
#define DMA_REQ_CMP BIT26 /* CMP 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 128 行定义.

5.13.2.18 DMA_REQ_GPIO

```
#define DMA_REQ_GPIO BIT29 /* GPIO 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 127 行定义.

5.13.2.19 DMA_REQ_HALL

```
#define DMA_REQ_HALL BIT24 /* HALL0 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 130 行定义.

5.13.2.20 DMA_REQ_I2C0_RX

```
#define DMA_REQ_I2C0_RX BIT14 /* I2C0_RX 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 136 行定义.

5.13.2.21 DMA_REQ_I2C0_TX

```
#define DMA_REQ_I2C0_TX BIT15 /* I2C0_TX 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 135 行定义.

5.13.2.22 DMA_REQ_MCPWM0

```
#define DMA_REQ_MCPWM0 BIT12 /* MCPWM0 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 138 行定义.

5.13.2.23 DMA_REQ_MCPWM1

```
#define DMA_REQ_MCPWM1 BIT13 /* MCPWM1 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 137 行定义.

5.13.2.24 DMA_REQ_SIF

```
#define DMA_REQ_SIF BIT25 /* HAL11 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 129 行定义.

5.13.2.25 DMA_REQ_SPI_RX

```
#define DMA_REQ_SPI_RX BIT10 /* SPI1_RX 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 140 行定义.

5.13.2.26 DMA_REQ_SPI_TX

```
#define DMA_REQ_SPI_TX BIT11 /* SPI1_TX 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 139 行定义.

5.13.2.27 DMA_REQ_SW

```
#define DMA_REQ_SW BIT31 /* SW 触发*/
```

DMA搬运触发源位宏定义

在文件 [lks32mc07x_dma.h](#) 第 126 行定义.

5.13.2.28 DMA_REQ_TIMER0

```
#define DMA_REQ_TIMER0 BIT3 /* TIMER0 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 145 行定义.

5.13.2.29 DMA_REQ_TIMER1

```
#define DMA_REQ_TIMER1 BIT4 /* TIMER1 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 144 行定义.

5.13.2.30 DMA_REQ_TIMER2

```
#define DMA_REQ_TIMER2 BIT5 /* TIMER2 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 143 行定义.

5.13.2.31 DMA_REQ_TIMER3

```
#define DMA_REQ_TIMER3 BIT6 /* TIMER3 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 142 行定义.

5.13.2.32 DMA_REQ_TIMER4

```
#define DMA_REQ_TIMER4 BIT7 /* TIMER4 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 141 行定义.

5.13.2.33 DMA_REQ_UART0_RX

```
#define DMA_REQ_UART0_RX BIT16 /* UART0_RX 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 134 行定义.

5.13.2.34 DMA_REQ_UART0_TX

```
#define DMA_REQ_UART0_TX BIT17 /* UART0_TX 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 133 行定义.

5.13.2.35 DMA_REQ_UART1_RX

```
#define DMA_REQ_UART1_RX BIT18 /* UART1_RX 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 132 行定义.

5.13.2.36 DMA_REQ_UART1_TX

```
#define DMA_REQ_UART1_TX BIT19 /* UART1_TX 触发*/
```

在文件 [lks32mc07x_dma.h](#) 第 131 行定义.

5.13.2.37 DMA_TCIE

```
#define DMA_TCIE BIT0 /* 传输完成中断使能, 高有效 */
```

在文件 [lks32mc07x_dma.h](#) 第 113 行定义.

5.13.2.38 DMA_TEIE

```
#define DMA_TEIE BIT1 /* 错误中断使能, 高有效 */
```

在文件 [lks32mc07x_dma.h](#) 第 114 行定义.

5.13.2.39 DMA_TOP

```
#define DMA_TOP ((DMA_MODULERegTypeDef *) (DMA0_BASE + 0x80))
```

在文件 [lks32mc07x_dma.h](#) 第 49 行定义.

5.13.2.40 DMA_WORD_TRANS

```
#define DMA_WORD_TRANS 2 /* 访问位宽, 2:word */
```

在文件 [lks32mc07x_dma.h](#) 第 121 行定义.

5.13.2.41 MEMORY2PERI

```
#define MEMORY2PERI 1 /* 内存至外设 */
```

在文件 [lks32mc07x_dma.h](#) 第 117 行定义.

5.13.2.42 PERI2MEMORY

```
#define PERI2MEMORY 0 /* 外设至内存 */
```

在文件 [lks32mc07x_dma.h](#) 第 116 行定义.

5.13.3 函数说明

5.13.3.1 DMA_CHx.EN()

```
void DMA_CHx_EN (  
    DMA_RegTypeDef * DMAx,  
    u32 Channel_EN )
```

函数名称: void [DMA_CHx_EN\(DMA_RegTypeDef *DMAx,u32 Channel_EN\)](#)

功能描述: DMA通道使能

参见

被引用内容:

参数

输入参数:	DMA.CHX :DMA 通道选择 DMA.CH0 , DMA.CH1 , DMA.CH2 , DMA.CH3 Channel.EN : ENABLE/DISABLE
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 407 行定义.

5.13.3.2 DMA_ClearIRQFlag()

```
void DMA_ClearIRQFlag (  
    u32 DMA_IF_x )
```

函数名称: void DMA_ClearIRQFlag(DMA_RegTypeDef *DMAx , uint32_t tempFlag)

功能描述: 清除DMA中断标志

参见

被引用内容: DMAx可选: DMA.CH0 , DMA.CH1 , DMA.CH2 , DMA.CH3

参数

输入参数:	DMAx: DMA通道选择	
	timer_if参数可选:	
	宏定义	说明
	DMA_CH0_FIF	通道 0 完成中断标志
	DMA_CH1_FIF	通道 1 完成中断标志
	DMA_CH2_FIF	通道 2 完成中断标志
	DMA_CH3_FIF	通道 3 完成中断标志
	DMA_CH0{EIF	通道 0 错误中断标志
	DMA_CH1{EIF	通道 1 错误中断标志
	DMA_CH2{EIF	通道 2 错误中断标志
	DMA_CH3{EIF	通道 3 错误中断标志
输出参数:	无	

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (DMA_GetIRQFlag (DMA_CH0,DMA_CH0_FIF) ) //获取DMA通道0完成中断标志
{
    DMA_ClearIRQFlag (DMA_CH0,DMA_CH0_FIF) //清除DMA通道0完成中断标志
}
```

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x-dma.c](#) 第 209 行定义.

5.13.3.3 DMA_Disable()

```
void DMA_Disable (
    void )
```

函数名称: void DMA_Enable(void)

功能描述: DMA模块 关闭函数

参见

被引用内容:

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 254 行定义.

5.13.3.4 DMA_Enable()

```
void DMA_Enable (  
    void )
```

函数名称: void DMA_Enable(void)

功能描述: DMA模块 使能函数

参见

被引用内容: DMAx可选: DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3

参数

输入参数:	无
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 232 行定义.

这是这个函数的调用关系图:



5.13.3.5 DMA_GetFlag()

```
u32 DMA_GetFlag (
    u32 DMA_IF_x )
```

在文件 [lks32mc07x_dma.c](#) 第 418 行定义.

函数调用图:



5.13.3.6 DMA_GetIRQFlag()

```
u32 DMA_GetIRQFlag (
    u32 DMA_IF_x )
```

函数名称: uint32_t DMA_GetIRQFlag(DMA_RegTypeDef *DMAx , u32 timer_if)

功能描述: 获取DMA中断标志

参见

被引用内容: DMAx可选: DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3

参数

输入参数:	DMAx: DMA通道选择	
	timer_if参数可选:	
	宏定义	说明
	DMA_CH0_FIF	通道 0 完成中断标志
	DMA_CH1_FIF	通道 1 完成中断标志
	DMA_CH2_FIF	通道 2 完成中断标志
	DMA_CH3_FIF	通道 3 完成中断标志
	DMA_CH0{EIF	通道 0 错误中断标志
	DMA_CH1{EIF	通道 1 错误中断标志
	DMA_CH2{EIF	通道 2 错误中断标志
	DMA_CH3{EIF	通道 3 错误中断标志

参见

参数

输出参数:	无
-------	---

返回

返回 值： 无

注解

其它说明： 无

警告

只有对应中断使能后，改为才能读取，如果对应中断未使能，读取结果一直为0

示例代码:

```
if (DMA_GetIRQFlag (DMA_CH0,DMA_CH0_FIF) ) //获取DMA通道0完成中断标志
{
}
```

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 166 行定义.

这是这个函数的调用关系图:



5.13.3.7 DMA_Init()

```
void DMA_Init (
    DMA_RegTypeDef * DMAx,
    DMA_InitTypeDef * this )
```

函数名称: void [DMA_Init\(DMA_RegTypeDef *DMAx, DMA_InitTypeDef *DMAInitStruct\)](#)

功能描述: DMA初始化函数

参见

被引用内容： 无

参数

输入参数:	ADCx: ADC0 , ADC_InitTypeDef
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
DMA_InitTypeDef DMA_InitStructure;
DMA_StructInit (&DMA_InitStructure);
DMA_InitStructure.EN = ENABLE;
DMA_InitStructure.TREQ_EN = ENABLE;
DMA_InitStructure.CIRC = DISABLE;
DMA_InitStructure.SINC = ENABLE;
// DMA通道使能
// DMA 传输完成中断使能
// DMA传输模式: 循环模式, 高有效
// 源地址递增, 高有效, 地址按照 SBTW
对应大小递增 1/2/4
DMA_InitStructure.DINC = DISABLE;
// 目的地址递增, 高有效, 地址按照 DBTW
对应大小递增 1/2/4
DMA_InitStructure.SBTW = DMA_BYTE_TRANS;
// 源访问位宽, 0:byte,
1:half-word, 2:word
DMA_InitStructure.DBTW = DMA_BYTE_TRANS;
// 目的访问位宽, 0:byte,
1:half-word, 2:word
DMA_InitStructure.REN.B.SIF_EN = ENABLE;
// SIF DMA请求使能, 高有效
DMA_InitStructure.TMS = 12;
// DMA 通道 x 数据搬运次数 1~255
DMA_InitStructure.RMODE = 1;
// 0:单轮传输, 一轮连续传输多次 /
1:多轮, 每轮进行一次数据传输
DMA_InitStructure.SADR = (u32)stru_yxtsenddata.byxt_send_data;
// DMA 通道 x 源地址 &CAN0_TXBUF0
DMA_InitStructure.DADR = (u32)&SIF0.WDATA;
// DMA 通道 x 目的地址 CAN0_TXRX0
CAN0_RFIFO00
DMA_Init (DMA_CHNO, &DMA_InitStructure);
```

Date	Version	Author	Description
2015年11月26日	1.0	Howlet Li	创建

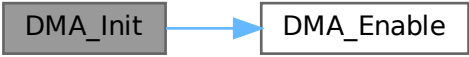
修改日志:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:去除初始化使能DMA通道, 改为DMA.CHx.EN () 函数使能DMA通道

在文件 [lks32mc07x_dma.c](#) 第 97 行定义.

函数调用图:



5.13.3.8 DMA_SetAddr()

```
void DMA_SetAddr (
    DMA_RegTypeDef * DMA_CHx,
    u32 addr1,
    u32 addr2 )
```

函数名称: `void DMA_SetAddr(DMA_RegTypeDef *DMA_CHx, u32 addr1, u32 addr2)`

功能描述: DMA传输地址设置

参见
被引用内容:

参数

输入参数:	DMA_CHX :DMA 通道选择 DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3 addr1 : 源地 址设置 addr2 : 目的地址设置
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 280 行定义.

5.13.3.9 DMA_SetLength()

```
void DMA_SetLength (
    DMA_RegTypeDef * DMA_CHx,
    u32 len )
```

函数名称: void [DMA_SetLength\(DMA_RegTypeDef *DMA_CHx, u32 len\)](#)

功能描述: DMA传输长度设置

参见

被引用内容:

参数

输入参数:	DMA_CHX :DMA 通道选择 DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3 len : 传输长度设置
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 308 行定义.

5.13.3.10 DMA_SetRequest()

```
void DMA_SetRequest (
    DMA_RegTypeDef * DMA_CHx,
    u32 DMA_REQ_x )
```

函数名称: void [DMA_SetRequest\(DMA_RegTypeDef *DMA_CHx, u32 DMA_REQ_x\)](#)

功能描述: DMA 外设请求使能

参见

被引用内容:

参数

输入参数:	DMA.CHX :DMA 通道选择 DMA.CH0 , DMA.CH1 , DMA.CH2 , DMA.CH3 DMA.REQ.x : 参数可选:		
	宏定义	原始值	说明
	DMA_REQ_UART1_TX	BIT19	
	DMA_REQ_UART1_RX	BIT18	
	DMA_REQ_UART0_TX	BIT17	
	DMA_REQ_UART0_RX	BIT16	
	DMA_REQ_I2C0_TX	BIT15	
	DMA_REQ_I2C0_RX	BIT14	
	DMA_REQ_MCPWM1	BIT13	
	DMA_REQ_MCPWM0	BIT12	
	DMA_REQ_SPI_TX	BIT11	
	DMA_REQ_SPI_RX	BIT10	
	DMA_REQ_TIMER4	BIT7	
	DMA_REQ_TIMER3	BIT6	
	DMA_REQ_TIMER2	BIT5	
	DMA_REQ_TIMER1	BIT4	
	DMA_REQ_TIMER0	BIT3	
	DMA_REQ_DSP	BIT2	
	DMA_REQ_ADC1	BIT1	
	DMA_REQ_ADC0	BIT0	
输出参数:	无		

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 382 行定义.

5.13.3.11 DMA_StructInit()

```
void DMA_StructInit (  
    DMA_InitTypeDef * DMAInitStruct )
```

函数名称: void [DMA_StructInit\(DMA_InitTypeDef *DMAInitStruct\)](#)

功能描述: DMA结构体初始化

参见

被引用内容: 无

参数

输入参数:	DMA_InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明： 无

警告

无

示例代码:

```
DMA_InitTypeDef DMA_InitStructure;  
DMA_StructInit (&DMA_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 54 行定义.

5.13.3.12 DMA_Trigger()

```
void DMA_Trigger (  
    DMA_RegTypeDef * DMA_CHx )
```

函数名称: void [DMA_Trigger\(DMA_RegTypeDef *DMA_CHx\)](#)

功能描述: DMA 软件触发设置

参见

被引用内容:

参数

输入参数:	DMA_CHX :DMA 通道选择 DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3
输出参数:	无

返回

返回 值: 无

注解

其它说明： 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 336 行定义.

5.14 lks32mc07x_dma.h

[浏览该文件的文档.](#)

```
00001
00015 #ifndef __lks32mc07x_dma_H
00016 #define __lks32mc07x_dma_H
00017
00018 #include "lks32mc07x.lib.h"
00019
00023 typedef struct
00024 {
00025     __IO uint32_t CCR;
00026     __IO uint32_t REN;
00027     __IO uint32_t CTMS;
00028     __IO uint32_t SADR;
00029     __IO uint32_t DADR;
00030 } DMA_RegTypeDef;
00031
00035 typedef struct
00036 {
00037     __IO u32 CTRL;
00038     __IO u32 IE;
00039     __IO u32 IF;
00040 } DMA_MODULETypeDef;
00041
00045 #define DMA_CHN0 ((DMA_RegTypeDef *)0x40011200)
00046 #define DMA_CHN1 ((DMA_RegTypeDef *) (DMA0_BASE + 0x20))
00047 #define DMA_CHN2 ((DMA_RegTypeDef *) (DMA0_BASE + 0x40))
00048 #define DMA_CHN3 ((DMA_RegTypeDef *) (DMA0_BASE + 0x60))
00049 #define DMA_TOP ((DMA_MODULETypeDef *) (DMA0_BASE + 0x80))
00050
00054 typedef union
00055 {
00056     u32 R;
00057     struct
00058     {
00059         u16 ADC0_EN : 1;        // BIT0
00060         u16 ADC1_EN : 1;        // BIT1
00061         u16 : 1;                // BIT2
00062         u16 DSP_EN : 1;         // BIT3
00063         u16 TIMER0_EN : 1;      // BIT4
00064         u16 TIMER1_EN : 1;      // BIT5
00065         u16 TIMER2_EN : 1;      // BIT6
00066         u16 TIMER3_EN : 1;      // BIT7
00067         u16 : 1;                // BIT8
00068         u16 : 1;                // BIT9
00069         u16 SPI_RX_EN : 1;       // BIT10
00070         u16 SPI_TX_EN : 1;       // BIT11
00071         u16 MCPWM_RE0_EN : 1;    // BIT12
00072         u16 MCPWM_RE1_EN : 1;    // BIT13
00073         u16 I2C0_RX_EN : 1;      // BIT14
```

```

00074         u16 I2C0_TX.EN : 1;    // BIT15
00075         u16 UART0_RX.EN : 1;    // BIT16
00076         u16 UART0_TX.EN : 1;    // BIT17
00077         u16 UART1_RX.EN : 1;    // BIT18
00078         u16 UART1_TX.EN : 1;    // BIT19
00079         u16 : 1;                // BIT20
00080         u16 : 1;                // BIT21
00081         u16 : 1;                // BIT22
00082         u16 : 1;                // BIT23
00083         u16 HALLO.EN : 1;        // BIT24
00084         u16 SIF.EN : 1;          // BIT25
00085         u16 CMP.EN : 1;          // BIT26
00086         u16 : 1;                // BIT27
00087         u16 : 1;                // BIT28
00088         u16 GPIO.EN : 1;         // BIT29
00089         u16 : 1;                // BIT30
00090         u16 SW.TRIG.EN : 1;      // BIT31
00091     } B;
00092 } stru_DMA.REN;
00093
00097 typedef struct
00098 {
00099     u8 IRQ.EN;
00100     u8 SBTW;                    // 源地址访问位宽 0: Byte 1: Halfword 2: Word
00101     u8 DBTW;                    // 目的地址访问位宽 0: Byte 1: Halfword 2: Word
00102     u8 SINC;                    // 源地址递增使能
00103     u8 DINC;                    // 目的地址地政使能
00104     u8 CIRC;                    // 循环模式使能
00105     u8 RMODE;                   // 多轮传输使能, 多个DMA 请求完成传输
00106     u8 EN;                      // 通道使能
00107     u8 TMS;                     // DMA 通道 x 数据搬运次数。
00108     u32 SADR;                   // DMA 通道 x 源地址
00109     u32 DADR;                   // DMA 通道 x 目的地址
00110     stru_DMA.REN REN;          // DMA 请求使能
00111 } DMA.InitTypeDef;
00112
00113 #define DMA.TCIE                BIT0 /* 传输完成中断使能, 高有效 */
00114 #define DMA.TEIE                BIT1 /* 错误中断使能, 高有效 */
00115
00116 #define PERI2MEMORY              0 /* 外设至内存 */
00117 #define MEMORY2PERI              1 /* 内存至外设 */
00118
00119 #define DMA.BYTE.TRANS            0 /* 访问位宽, 0:byte */
00120 #define DMA.HALFWORD.TRANS       1 /* 访问位宽, 1:half-word */
00121 #define DMA.WORD.TRANS           2 /* 访问位宽, 2:word */
00122
00126 #define DMA.REQ_SW               BIT31 /* SW 触发 */
00127 #define DMA.REQ_GPIO             BIT29 /* GPIO 触发 */
00128 #define DMA.REQ_CMP              BIT26 /* CMP 触发 */
00129 #define DMA.REQ_SIF              BIT25 /* HALL1 触发 */
00130 #define DMA.REQ_HALL             BIT24 /* HALL0 触发 */
00131 #define DMA.REQ_UART1_TX         BIT19 /* UART1_TX 触发 */
00132 #define DMA.REQ_UART1_RX         BIT18 /* UART1_RX 触发 */
00133 #define DMA.REQ_UART0_TX         BIT17 /* UART0_TX 触发 */
00134 #define DMA.REQ_UART0_RX         BIT16 /* UART0_RX 触发 */
00135 #define DMA.REQ_I2C0_TX          BIT15 /* I2C0_TX 触发 */
00136 #define DMA.REQ_I2C0_RX          BIT14 /* I2C0_RX 触发 */
00137 #define DMA.REQ_MCPWM1           BIT13 /* MCPWM1 触发 */
00138 #define DMA.REQ_MCPWM0           BIT12 /* MCPWM0 触发 */
00139 #define DMA.REQ_SPI1_TX          BIT11 /* SPI1_TX 触发 */
00140 #define DMA.REQ_SPI1_RX          BIT10 /* SPI1_RX 触发 */
00141 #define DMA.REQ_TIMER4           BIT7  /* TIMER4 触发 */
00142 #define DMA.REQ_TIMER3           BIT6  /* TIMER3 触发 */
00143 #define DMA.REQ_TIMER2           BIT5  /* TIMER2 触发 */
00144 #define DMA.REQ_TIMER1           BIT4  /* TIMER1 触发 */
00145 #define DMA.REQ_TIMER0           BIT3  /* TIMER0 触发 */
00146 #define DMA.REQ_ADC1             BIT1  /* ADC1 触发 */
00147 #define DMA.REQ_ADC0             BIT0  /* ADC0 触发 */
00148
00152 #define DMA.IF.CH0 BIT0
00153 #define DMA.IF.CH1 BIT1
00154 #define DMA.IF.CH2 BIT2
00155 #define DMA.IF.CH3 BIT3
00156
00160 #define DMA.IE.CH0 BIT0
00161 #define DMA.IE.CH1 BIT1
00162 #define DMA.IE.CH2 BIT2
00163 #define DMA.IE.CH3 BIT3
00164
00165 void DMA.Init(DMA.RegTypeDef *DMAx, DMA.InitTypeDef *DMAInitStruct);
00166 void DMA.StructInit(DMA.InitTypeDef *DMAInitStruct);
00167 u32 DMA.GetIRQFlag(u32 DMA_IF_x);
00168 void DMA.ClearIRQFlag(u32 DMA_IF_x);
00169 u32 DMA.GetFlag(u32 DMA_IF_x);
00170
00171 void DMA.Enable(void);
00172 void DMA.Disable(void);

```

```

00173 void DMA_SetAddr(DMA_RegTypeDef *DMA_CHx, u32 addr1, u32 addr2);
00174 void DMA_SetLength(DMA_RegTypeDef *DMA_CHx, u32 len);
00175 void DMA_Trigger(DMA_RegTypeDef *DMA_CHx);
00176 void DMA_SetRequest(DMA_RegTypeDef *DMA_CHx, u32 DMA_REQ_x);
00177 void DMA_CHx_EN(DMA_RegTypeDef *DMAx, u32 Channel_EN);
00178
00179 #endif /*_lks32mc08x_dma_H */
00180
00181 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/

```

5.15 include/lks32mc07x_dsp.h 文件参考

文件名称: [lks32mc07x_dsp.h](#)

文件标识: 无

内容摘要: DSP驱动头文件

其它说明: 无

#include "lks32mc07x_lib.h"

lks32mc07x_dsp.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [DSP_TypeDef](#)
DSP寄存器结构体句柄

宏定义

- #define [DSP_REG](#) (([DSP_TypeDef](#) *) (DSP0.REG.BASE + 0x00))
DSP基地址
- #define [DSP0_arctan](#) [DSP_arctan](#)
- #define [DSP0_div](#) [DSP_div](#)
- #define [DSP0_sin](#) [DSP_sin](#)
- #define [DSP0_cos](#) [DSP_cos](#)
- #define [DSP0_rms](#) [DSP_rms](#)
- #define [DSP0_mod](#) [DSP_mod](#)
- #define [DSP0_sqrt](#) [DSP_sqrt](#)
- #define [DSP_SIN_COS_MODE](#)()
- #define [DSP_ARCTAN_MODE](#)()
- #define [DSP_CALC_SIN](#)(sin, angle)
- #define [DSP_CALC_COS](#)(cos, angle)
- #define [DSP_CALC_RMS](#)(rms, a, b)
- #define [DSP_CALC_ARCTAN](#)(arctan, x, y)
- #define [DSP_CALC_DIV](#)(c, a, b)
- #define [DSP_CALC_MOD](#)(mod, x, y)
- #define [DSP_CALC_SQRT](#)(sqrt, val)

函数

- void **DSP_Init** (void)
函数名称: *void DSP_Init(void)*
- s16 **DSP_arctan** (s16 x, s16 y)
函数名称: *DSP_arctan(s16 x,s16 y)*
- s32 **DSP_div** (s32 a, s32 b)
函数名称: *s32 DSP_div(s32 a,s32 b)*
- s16 **DSP_sin** (s16 angle)
函数名称: *s16 DSP.sin(u16 angle)*
- s16 **DSP_cos** (s16 angle)
函数名称: *s16 DSP.cos(u16 angle)*
- s32 **DSP_rms** (s32 a, s32 b)
函数名称: *s16 u16 DSP_rms(s16 a,s16 b)*
- u16 **DSP_mod** (s16 x, s16 y)
函数名称: *s16 DSP.mod(s32 x,s16 y)*
- u16 **DSP_sqrt** (u32 val)
函数名称: *u16 DSP.sqrt(u32 val)*
- void **DSP_SinCosMode** (void)
*sincos*模式
- void **DSP_ArctanMode** (void)
*Arctan*模式

5.15.1 详细描述

文件名称: **lks32mc07x.dsp.h**
文件标识: 无
内容摘要: DSP驱动头文件
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2018年07月25日	1.0	Howlet	创建

修改日志:

在文件 **lks32mc07x.dsp.h** 中定义.

5.15.2 宏定义说明

5.15.2.1 DSP0.arctan

#define DSP0_arctan **DSP_arctan**

在文件 [lks32mc07x_dsp.h](#) 第 60 行定义.

5.15.2.2 DSP0.cos

```
#define DSP0_cos DSP_cos
```

在文件 [lks32mc07x_dsp.h](#) 第 63 行定义.

5.15.2.3 DSP0.div

```
#define DSP0_div DSP_div
```

在文件 [lks32mc07x_dsp.h](#) 第 61 行定义.

5.15.2.4 DSP0.mod

```
#define DSP0_mod DSP_mod
```

在文件 [lks32mc07x_dsp.h](#) 第 65 行定义.

5.15.2.5 DSP0.rms

```
#define DSP0_rms DSP_rms
```

在文件 [lks32mc07x_dsp.h](#) 第 64 行定义.

5.15.2.6 DSP0.sin

```
#define DSP0_sin DSP_sin
```

在文件 [lks32mc07x_dsp.h](#) 第 62 行定义.

5.15.2.7 DSP0.sqrt

```
#define DSP0_sqrt DSP_sqrt
```

在文件 [lks32mc07x_dsp.h](#) 第 66 行定义.

5.15.2.8 DSP_ARCTAN_MODE

```
#define DSP_ARCTAN_MODE( )
```

值:

```
{  
    DSP_REG->SC = BIT1; \  
}
```

在文件 [lks32mc07x_dsp.h](#) 第 72 行定义.

5.15.2.9 DSP_CALC_ARCTAN

```
#define DSP_CALC_ARCTAN(  
    arctan,  
    x,  
    y )
```

值:

```
{  
    int32_t flg = __get_PRIMASK(); \\  
    __disable_irq(); \\  
    DSP_ARCTAN_MODE(); \\  
    DSP_REG->X = x; \\  
    DSP_REG->Y = y; \\  
    arctan = DSP_REG->ARCTAN; \\  
    __set_PRIMASK(flg); \\  
}
```

在文件 [lks32mc07x_dsp.h](#) 第 105 行定义.

5.15.2.10 DSP_CALC_COS

```
#define DSP_CALC_COS(  
    cos,  
    angle )
```

值:

```
{  
    int32_t flg = __get_PRIMASK(); \\  
    __disable_irq(); \\  
    DSP_SIN_COS_MODE(); \\  
    DSP_REG->THETA = (u16)angle; \\  
    cos = DSP_REG->COS; \\  
    __set_PRIMASK(flg); \\  
}
```

在文件 [lks32mc07x_dsp.h](#) 第 86 行定义.

5.15.2.11 DSP_CALC_DIV

```
#define DSP_CALC_DIV(  
    c,  
    a,  
    b )
```

值:

```
{  
    int32_t flg = __get_PRIMASK(); \\  
    __disable_irq(); \\  
    DSP_REG->DID = (a); \\  
    DSP_REG->DIS = (b); \\  
    c = DSP_REG->QUO; \\  
    __set_PRIMASK(flg); \\  
}
```

在文件 [lks32mc07x_dsp.h](#) 第 115 行定义.

5.15.2.12 DSP_CALC_MOD

```
#define DSP_CALC_MOD(
```

```
    mod,
```

```
    x,
```

```
    y )
```

值:

```
{
    int32_t flg = __get_PRIMASK(); \
    __disable_irq(); \
    DSP_ARCTAN_MODE(); \
    DSP_REG->X = x; \
    DSP_REG->Y = y; \
    mod = DSP_REG->MOD; \
    __set_PRIMASK(flgs); \
}
```

在文件 [lks32mc07x_dsp.h](#) 第 124 行定义.

5.15.2.13 DSP_CALC_RMS

```
#define DSP_CALC_RMS(
```

```
    rms,
```

```
    a,
```

```
    b )
```

值:

```
{
    int32_t flg = __get_PRIMASK(); \
    __disable_irq(); \
    DSP_REG->DID = a; \
    DSP_REG->DIS = b; \
    rms = DSP_REG->REM; \
    __set_PRIMASK(flgs); \
}
```

在文件 [lks32mc07x_dsp.h](#) 第 95 行定义.

5.15.2.14 DSP_CALC_SIN

```
#define DSP_CALC_SIN(
```

```
    sin,
```

```
    angle )
```

值:

```
{
    int32_t flg = __get_PRIMASK(); \
    __disable_irq(); \
    DSP_SIN_COS_MODE(); \
    DSP_REG->THETA = (u16)angle; \
    sin = DSP_REG->SIN; \
    __set_PRIMASK(flgs); \
}
```

在文件 [lks32mc07x_dsp.h](#) 第 77 行定义.

5.15.2.15 DSP_CALC_SQRT

```
#define DSP_CALC_SQRT(
    sqrt,
    val )
```

值:

```
{
    int32_t flg = __get_PRIMASK(); \
    __disable_irq(); \
    DSP_REG->RAD = val; \
    sqrt = DSP_REG->SQRT; \
    __set_PRIMASK(flg); \
}
```

在文件 [lks32mc07x_dsp.h](#) 第 134 行定义.

5.15.2.16 DSP_REG

```
#define DSP_REG ((DSP_TypeDef *) (DSP0_REG_BASE + 0x00))
```

DSP基地址

在文件 [lks32mc07x_dsp.h](#) 第 46 行定义.

5.15.2.17 DSP_SIN_COS_MODE

```
#define DSP_SIN_COS_MODE( )
```

值:

```
{
    DSP_REG->SC = BIT2 | BIT1; \
}
```

在文件 [lks32mc07x_dsp.h](#) 第 68 行定义.

5.15.3 函数说明

5.15.3.1 DSP_arctan()

```
s16 DSP_arctan (
    s16 x,
    s16 y )
```

函数名称: [DSP_arctan\(s16 x,s16 y\)](#)

功能描述: DSP 反正切计算函数

参见

被引用内容:

参数

输入参数:	x: 分母 y:分子
输出参数:	无

返回

返回 值: 反正切值

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 155 行定义.

5.15.3.2 DSP_ArctanMode()

```
void DSP_ArctanMode (
    void )
```

Arctan模式

在文件 [lks32mc07x_dsp.c](#) 第 268 行定义.

5.15.3.3 DSP_cos()

```
s16 DSP_cos (
    s16 angle )
```

函数名称: s16 DSP_cos(u16 angle)

功能描述: DSP cos计算函数

参见

被引用内容:

参数

输入参数:	angle: 输入角度
输出参数:	无

返回

返回 值: $\cos$ 角度值

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 93 行定义.

5.15.3.4 DSP_div()

```
s32 DSP_div (  
    s32 a,  
    s32 b )
```

函数名称: [s32 DSP_div\(s32 a,s32 b\)](#)

功能描述: DSP 除法计算

参见

被引用内容:

参数

输入参数:	x: 被除数 y:除数
输出参数:	无

返回

返回 值: 两数的商

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 187 行定义.

5.15.3.5 DSP_Init()

```
void DSP_Init (
    void )
```

函数名称: [void DSP_Init\(void\)](#)

功能描述: DSP使能和关闭

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

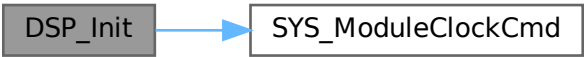
示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 40 行定义.

函数调用图:



5.15.3.6 DSP_mod()

```
u16 DSP_mod (
    s16 x,
    s16 y )
```

函数名称: s16 DSP_mod(s32 x,s16 y)

功能描述: DSP 计算向量的模

参见

被引用内容:

参数

输入参数:	x: x轴 y: y轴
输出参数:	无

返回

返回 值: 向量的模

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 218 行定义.

5.15.3.7 DSP_rms()

```
s32 DSP_rms (
    s32 a,
    s32 b )
```

函数名称: s16 u16 DSP_rms(s16 a,s16 b)

功能描述: DSP 除法求余数计算函数

参见

被引用内容:

参数

输入参数:	a: 被除数 b:除数
输出参数:	无

返回

返回 值: 两数的余数

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 124 行定义.

5.15.3.8 DSP_sin()

```
s16 DSP_sin (
    s16 angle )
```

函数名称: s16 DSP_sin(u16 angle)

功能描述: DSP sin计算函数

参见

被引用内容:

参数

输入参数:	angle: 输入角度
输出参数:	无

返回

返回 值: `sin`角度值

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 `lks32mc07x_dsp.c` 第 63 行定义.

5.15.3.9 DSP_SinCosMode()

```
void DSP_SinCosMode (
    void )
```

`sincos`模式

在文件 `lks32mc07x_dsp.c` 第 261 行定义.

5.15.3.10 DSP_sqrt()

```
u16 DSP_sqrt (
    u32 val )
```

函数名称: `u16 DSP_sqrt(u32 val)`

功能描述: 开平方函数

参见

被引用内容:

参数

输入参数:	val：被开方数
输出参数:	无

返回

返回 值： 向量的莫

注解

其它说明： 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 lks32mc07x_dsp.c 第 248 行定义.

5.16 lks32mc07x_dsp.h

浏览该文件的文档.

```
00001
00015 #ifndef _CONNIE_DSP_H_
00016 #define _CONNIE_DSP_H_
00017
00018 /* Includes ----- */
00019 #include "lks32mc07x.lib.h"
00020
00025 typedef struct
00026 {
00027     __IO u32 SC;           // 0x00)
00028     __IO u32 THETA;       // 0x04)
00029     __IO u32 X;           // 0x08)
00030     __IO u32 Y;           // 0x0C)
00031     __IO u32 SIN;         // 0x10)
00032     __IO u32 COS;         // 0x14)
00033     __IO u32 MOD;         // 0x18)
00034     __IO u32 ARCTAN;      // 0x1C)
00035     __IO u32 DID;         // 0x20)
00036     __IO u32 DIS;         // 0x24)
00037     __IO u32 QUO;         // 0x28)
```

```

00038     __IO u32 REM;      // 0x2C)
00039     __IO u32 RAD;      // 0x30)
00040     __IO u32 SQRT;     // 0x34)
00041     __IO u32 PC;       // 0x38)
00042 } DSP_TypeDef;
00046 #define DSP_REG ((DSP_TypeDef *) (DSP0_REG_BASE + 0x00))
00047
00048 void DSP_Init(void);
00049 s16 DSP_arctan(s16 x, s16 y);
00050 s32 DSP_div(s32 a, s32 b);
00051 s16 DSP_sin(s16 angle);
00052 s16 DSP_cos(s16 angle);
00053 s32 DSP_rms(s32 a, s32 b);
00054 u16 DSP_mod(s16 x, s16 y);
00055 u16 DSP_sqrt(u32 val);
00056 void DSP_SinCosMode(void);
00057 void DSP_ArctanMode(void);
00058
00059 // 命名由DSP0_div更新为DSP_div, 使用下面的宏以兼容之前的代码
00060 #define DSP0_arctan DSP_arctan
00061 #define DSP0_div DSP_div
00062 #define DSP0_sin DSP_sin
00063 #define DSP0_cos DSP_cos
00064 #define DSP0_rms DSP_rms
00065 #define DSP0_mod DSP_mod
00066 #define DSP0_sqrt DSP_sqrt
00067
00068 #define DSP_SIN_COS_MODE() \
00069 { \
00070     DSP_REG->SC = BIT2 | BIT1; \
00071 }
00072 #define DSP_ARCTAN_MODE() \
00073 { \
00074     DSP_REG->SC = BIT1; \
00075 }
00076 // 增加DSP_CALC系列的接口, 以降低参数传递带来的时间损耗
00077 #define DSP_CALC_SIN(sin, angle) \
00078 { \
00079     int32_t flg = __get_PRIMASK(); \
00080     __disable_irq(); \
00081     DSP_SIN_COS_MODE(); \
00082     DSP_REG->THETA = (u16)angle; \
00083     sin = DSP_REG->SIN; \
00084     __set_PRIMASK(flgs); \
00085 }
00086 #define DSP_CALC_COS(cos, angle) \
00087 { \
00088     int32_t flg = __get_PRIMASK(); \
00089     __disable_irq(); \
00090     DSP_SIN_COS_MODE(); \
00091     DSP_REG->THETA = (u16)angle; \
00092     cos = DSP_REG->COS; \
00093     __set_PRIMASK(flgs); \
00094 }
00095 #define DSP_CALC_RMS(rms, a, b) \
00096 { \
00097     int32_t flg = __get_PRIMASK(); \
00098     __disable_irq(); \
00099     DSP_REG->DID = a; \
00100     DSP_REG->DIS = b; \
00101     rms = DSP_REG->REM; \
00102     __set_PRIMASK(flgs); \
00103 }
00104
00105 #define DSP_CALC_ARCTAN(arctan, x, y) \
00106 { \
00107     int32_t flg = __get_PRIMASK(); \
00108     __disable_irq(); \
00109     DSP_ARCTAN_MODE(); \
00110     DSP_REG->X = x; \
00111     DSP_REG->Y = y; \
00112     arctan = DSP_REG->ARCTAN; \
00113     __set_PRIMASK(flgs); \
00114 }
00115 #define DSP_CALC_DIV(c, a, b) \
00116 { \
00117     int32_t flg = __get_PRIMASK(); \
00118     __disable_irq(); \
00119     DSP_REG->DID = (a); \
00120     DSP_REG->DIS = (b); \
00121     c = DSP_REG->QUO; \
00122     __set_PRIMASK(flgs); \
00123 }
00124 #define DSP_CALC_MOD(mod, x, y) \
00125 { \
00126     int32_t flg = __get_PRIMASK(); \
00127     __disable_irq(); \

```

```

00128     DSP_ARCTAN_MODE();
00129     DSP_REG->X = x;
00130     DSP_REG->Y = y;
00131     mod      = DSP_REG->MOD;
00132     __set_PRIMASK(flq);
00133 }
00134 #define DSP_CALC_SQRT(sqrt, val)
00135 {
00136     int32_t flq = __get_PRIMASK();
00137     __disable_irq();
00138     DSP_REG->RAD = val;
00139     sqrt        = DSP_REG->SQRT;
00140     __set_PRIMASK(flq);
00141 }
00142
00143 #endif
00144 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.17 include/lks32mc07x_exti.h 文件参考

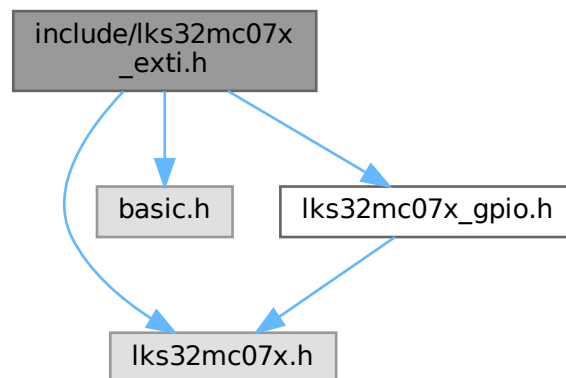
文件名称: [lks32mc07x_exti.h](#)
 文件标识: 无
 内容摘要: EXTI外部中断驱动头文件
 其它说明: 无

```

#include "lks32mc07x.h"
#include "basic.h"
#include "lks32mc07x_gpio.h"

```

lks32mc07x_exti.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [EXTI_TypeDef](#)
EXTI寄存器结构体句柄

宏定义

- #define [EXTI](#) (([EXTI_TypeDef](#) *)EXTI_BASE)
EXTI结构体基地址定义

枚举

- enum [EXTIMode_TypeDef](#) { [EXTI_Trigger_None](#) = 0x00 , [EXTI_Trigger_Negedge](#) = 0x01 , [EXTI_Trigger_Posedge](#) = 0x02 , [EXTI_Trigger_Edge](#) = 0x03 }
EXTI触发信号定义

函数

- uint32_t [EXTI_ReadIFFlag](#) ([EXTI_TypeDef](#) *EXTIx)
- void [EXTI_ResetIF](#) ([EXTI_TypeDef](#) *EXTIx, uint32_t temp)
- void [EXTI_Trigger_Config](#) ([GPIO_TypeDef](#) *GPIOx, uint32_t GPIO_PinSource, uint16_t EXTI_Trigger)
*函数名称: void EXTI_Trigger_Config(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint16_t EXTI_Trigger)*
- void [EXTI_ClearIRQFlag](#) ([GPIO_TypeDef](#) *GPIOx, uint32_t GPIO_PinSource)
*函数名称: void EXTI_ClearIRQFlag(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource)*
- u8 [EXTI_GetIRQFlag](#) ([GPIO_TypeDef](#) *GPIOx, uint32_t GPIO_PinSource)
*函数名称: u8 EXTI_GetIRQFlag(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource)*

5.17.1 详细描述

文件名称: [lks32mc07x_exti.h](#)
文件标识: 无
内容摘要: EXTI外部中断驱动头文件
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_exti.h](#) 中定义.

5.17.2 宏定义说明

5.17.2.1 EXTI

```
#define EXTI ((EXTI_TypeDef *)EXTI_BASE)
```

EXTI结构体基地址定义

在文件 [lks32mc07x-exti.h](#) 第 37 行定义.

5.17.3 枚举类型说明

5.17.3.1 EXTI_Mode_TypeDef

```
enum EXTI_Mode_TypeDef
```

EXTI触发信号定义

枚举值

EXTI_Trigger_None	不触发
EXTI_Trigger_Negedge	下降沿触发
EXTI_Trigger_Posedge	上升沿触发
EXTI_Trigger_Edge	上升沿、下降沿均触发

在文件 [lks32mc07x-exti.h](#) 第 43 行定义.

5.17.4 函数说明

5.17.4.1 EXTI_ClearIRQFlag()

```
void EXTI_ClearIRQFlag (  
    GPIO_TypeDef * GPIOx,  
    uint32_t GPIO_PinSource )
```

函数名称: void [EXTI_ClearIRQFlag\(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource\)](#)

功能描述: 清除外部中断标志位

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象可选(GPIO0~GPIO3) GPIO_PinSource:指定的PinSource可选(GPIO_PinSource_0~GPIO_PinSource_15)
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if(EXTI_GetIRQFlag(GPIO0,GPIO_PinSource_0))//获取P0.0外部中断标志位
{
    EXTI_ClearIRQFlag(GPIO0,GPIO_PinSource_0); //清除P0.0外部中断标志位
}
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_exti.c](#) 第 390 行定义.

5.17.4.2 EXTI_GetIRQFlag()

```
u8 EXTI_GetIRQFlag (
    GPIO_TypeDef * GPIOx,
    uint32_t GPIO_PinSource )
```

函数名称: u8 EXTI_GetIRQFlag(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource)

功能描述: 读取外部中断标志位

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象可选(GPIO0~GPIO3) GPIO_PinSource:指定的PinSource可选(GPIO_PinSource_0~GPIO_PinSource_15)
输出参数:	无

返回

返回 值： 中断标志，高有效

注解

其它说明： 无

警告

无

示例代码:

```
if(EXTI_GetIRQFlag(GPIO0,GPIO.PinSource_0))//获取P0.0外部中断标志位
{
}
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_exti.c](#) 第 159 行定义.

5.17.4.3 EXTI_ReadIFFlag()

```
uint32_t EXTI_ReadIFFlag (
    EXTI_TypeDef * EXTIx )
```

在文件 [lks32mc07x_exti.c](#) 第 468 行定义.

5.17.4.4 EXTI_ResetIF()

```
void EXTI_ResetIF (
    EXTI_TypeDef * EXTIx,
    uint32_t temp )
```

在文件 [lks32mc07x_exti.c](#) 第 472 行定义.

5.17.4.5 EXTI_Trigger_Config()

```
void EXTI_Trigger_Config (
    GPIO_TypeDef * GPIOx,
    uint32_t GPIO_PinSource,
    uint16_t EXTI_Trigger )
```

函数名称: void EXTI_Trigger_Config(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint16_t EXTI_Trigger)

功能描述: GPIO外部触发极性选择

参见

被引用内容: GPIO_TypeDef

参数

输入参数:	GPIOx:GPIO对象可选(GPIO0~GPIO3) GPIO_PinSource:指定的PinSource可选(GPIO_PinSource_0~GPIO_PinSource_15) EXTI_Trigger:指定的触发极性
-------	--

参见

EXTI_Trigger参数来源: EXTI_Mode_TypeDef

参数

输出参数:	无
-------	---

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
EXTI_Trigger_Config (GPIO0, GPIO_PinSource_0, EXTI_Trigger_Negedge); //P0.0信号下降沿触发外部中断事件
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_exti.c](#) 第 39 行定义.

5.18 lks32mc07x_exti.h

[浏览该文件的文档.](#)

```
00001
00014 #ifndef __lks32mc07x_EXTI_H
00015 #define __lks32mc07x_EXTI_H
00016
00017 /* Includes -----*/
00018 #include "lks32mc07x.h"
00019 #include "basic.h"
00020 #include "lks32mc07x_gpio.h"
00021
00025 typedef struct
00026 {
00027     __IO uint32_t CR0;
00028     __IO uint32_t CR1;
00029     __IO uint32_t IE;
00030     __IO uint32_t IF;
00031 } EXTI_TypeDef;
00032
00036 #ifndef EXTI
00037 #define EXTI ((EXTI_TypeDef *)EXTI_BASE)
00038 #endif
00039
00043 typedef enum
00044 {
00045     EXTI_Trigger_None    = 0x00,
00046     EXTI_Trigger_Negedge = 0x01,
00047     EXTI_Trigger_Posedge = 0x02,
00048     EXTI_Trigger_Edge    = 0x03
00049 } EXTI_Mode_TypeDef;
00050
00051 uint32_t EXTI_ReadIFFlag(EXTI_TypeDef *EXTIx);
00052 void EXTI_ResetIF(EXTI_TypeDef *EXTIx, uint32_t temp);
00053
00054 void EXTI_Trigger_Config(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint16_t EXTI_Trigger);
00055 void EXTI_ClearIRQFlag(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource);
00056 u8 EXTI_GetIRQFlag(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource);
00057
00058 #endif /*__lks32mc07x_EXTI_H */
00059
00060 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/
```

5.19 include/lks32mc07x_flash.h 文件参考

文件名称: LKS32MC07x_flash.h
文件标识: 无
内容摘要: FLASH外设驱动程序
其它说明: 无

#include "lks32mc07x_lib.h"
lks32mc07x_flash.h 的引用(include)关系图:



此图展示该文件直接或间接的被哪些文件引用了：



宏定义

- #define Flash_MAIN 0x000
Flash的mian区域定义
- #define Flash_NVR 0x800
Flash的mian区域定义

函数

- int ProgramPage (u32 adr, u32 sz, u8 *buf, u16 nvr)
函数名称: int ProgramPage(u32 adr, u32 sz, u8 *buf, u16 nvr)
- void EraseSector (u32 adr, u16 nvr)
函数名称: void EraseSector(u32 adr, u16 nvr)
- void Read.More.Flash (u32 adr, u32 sz, u32 *buf, u16 nvr)
函数名称: void Read.More.Flash(u32 adr, u32 sz, u32 *buf, u16 nvr)

变量

- volatile u32 erase_flag
flash擦除操作启动变量
- volatile u32 prog_m_flag
flash编程操作启动变量

5.19.1 详细描述

文件名称: LKS32MC07x_flash.h
文件标识: 无
内容摘要: FLASH外设驱动程序
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_flash.h](#) 中定义.

5.19.2 宏定义说明

5.19.2.1 Flash.MAIN

```
#define Flash_MAIN 0x000
```

Flash的mian区域定义

在文件 [lks32mc07x_flash.h](#) 第 18 行定义.

5.19.2.2 Flash.NVR

```
#define Flash_NVR 0x800
```

Flash的mian区域定义

在文件 [lks32mc07x_flash.h](#) 第 19 行定义.

5.19.3 函数说明

5.19.3.1 EraseSector()

```
void EraseSector (
    u32 adr,
    u16 nvr )
```

函数名称: void [EraseSector\(u32 adr, u16 nvr\)](#)

功能描述: Flash按扇区擦除操作

参见

被引用内容: 无

参数

输入参数:	adr: 扇区地址 (一个扇区512字节) nvr: 为0x800的时候擦除NVR, 为0时擦除主Flash erase_flag: 调用函数前必须赋值0x9A0D361F, 否则不执行擦除, 用来防程序跑飞
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

事例1:

```
erase_flag = 0x9A0D361F; //擦除解锁
EraseSector(0x00000000 , Flash.MAIN); //擦除flash的main区域即存储程序区域
erase_flag = 0x00; //擦除上锁
```

事例2:

```
erase_flag = 0x9A0D361F; //擦除解锁
EraseSector(0x00000000 , Flash.NVR); //擦除flash的NVR区域即独立的1KB用户存储区
erase_flag = 0x00; //擦除上锁
```

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_flash.c](#) 第 55 行定义.

5.19.3.2 ProgramPage()

```
int ProgramPage (
    u32 adr,
    u32 sz,
    u8 * buf,
    u16 nvr )
```

函数名称: int [ProgramPage\(u32 adr, u32 sz, u8 *buf, u16 nvr\)](#)

功能描述: Flash编程操作

参见

被引用内容: 无

参数

输入参数:	adr: 扇区地址 sz : 编程字节数量(0~512字节) buf: 要编程的数据数组指针 nvr: 为0x800的时候编程NVR, 为0时编程主Flash progm_flag: 调用函数前必须赋值0x9AFDA40C, 否则不执行编程, 用来防程序跑飞
输出参数:	无

返回

返回 值: 1: 编程成功; 0: 编程失败

注解

其它说明: 无

警告

无

示例代码:

```
u8 Flash_Buff[2] = {0x12,0x23};
    事例1:
    erase_flag = 0x9A0D361F; //擦除解锁
    EraseSector(0x00000000 , Flash_MAIN); //擦除flash的main区域即存储程序区域
    erase_flag = 0x00; //擦除上锁

    prog_flag = 0x9AFDA40C; //编程解锁
    ProgramPage(0x00000000 , 2,Flash_Buff, Flash_MAIN); //编程flash的main区域即存储程序区域
    erase_flag = 0x00; //编程上锁
    事例2:
    erase_flag = 0x9A0D361F; //擦除解锁
    EraseSector(0x00000000 , Flash_NVR); //擦除flash的NVR区域即独立的1KB用户存储区
    erase_flag = 0x00; //擦除上锁

    erase_flag = 0x9A0D361F; //编程解锁
    EraseSector(0x00000000, 2,Flash_Buff,Flash_NVR); //编程flash的NVR区域即独立的1KB用户存储区
    erase_flag = 0x00; //编程上锁
```

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_flash.c](#) 第 137 行定义.

5.19.3.3 Read_More_Flash()

```
void Read_More_Flash (
    u32 adr,
```



```
u32 sz,
u32 * buf,
u16 nvr )
```

函数名称: void **Read_More_Flash**(u32 adr, u32 sz, u32 *buf, u16 nvr)

功能描述: 读取FLASH多地址数据函数

参见
被引用内容: 无

参数

输入参数:	adr: 扇区地址 (一个扇区512字节) nvr: 为0x800, 读取MAIN数据, 为0x0, 读取NVR数据 buf:读取数据存储的地址 sz :读取sz个字
输出参数:	无

返回
返 回 值: 读取的数据值

注解
其它说明: 无

警告
无

示例代码:

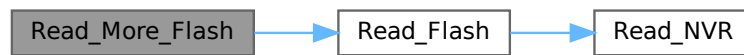
```
u32 Flash_Read_Buff[4] = {0};
事例1:
Read_More_Flash(0x00000000 ,4,Flash_Read_Buff,Flash.MAIN); //获取flash的main区域0x00000000地址4个word数据
事例2:
Read_More_Flash(0x00000000 ,4,Flash_Read_Buff,Flash.NVR); //获取flash的NVR区域0x00000000地址4个word数据
```

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 **lks32mc07x_flash.c** 第 **276** 行定义.

函数调用图:



5.19.4 变量说明

5.19.4.1 erase_flag

```
volatile u32 erase_flag [extern]
```

flash擦除操作启动变量

在文件 [lks32mc07x_flash.c](#) 第 20 行定义.

5.19.4.2 prog_flag

```
volatile u32 prog_flag [extern]
```

flash编程操作启动变量

在文件 [lks32mc07x_flash.c](#) 第 24 行定义.

5.20 lks32mc07x_flash.h

[浏览该文件的文档.](#)

```
00001
00014 #ifndef __lks32mc07x_FLASH__
00015 #define __lks32mc07x_FLASH__
00016 #include "lks32mc07x_lib.h"
00017
00018 #define Flash_MAIN 0x000
00019 #define Flash_NVR 0x800
00021 extern volatile u32 erase_flag;
00022 extern volatile u32 prog_flag;
00023
00024 int ProgramPage(u32 adr, u32 sz, u8 *buf, u16 nvr);
00025 void EraseSector(u32 adr, u16 nvr);
00026 void ReadMoreFlash(u32 adr, u32 sz, u32 *buf, u16 nvr);
00027
00028 #endif
```

5.21 include/lks32mc07x_gpio.h 文件参考

文件名称: [lks32mc07x_gpio.h](#)

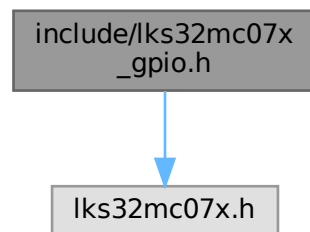
文件标识: 无

内容摘要: GPIO外设驱动程序头文件

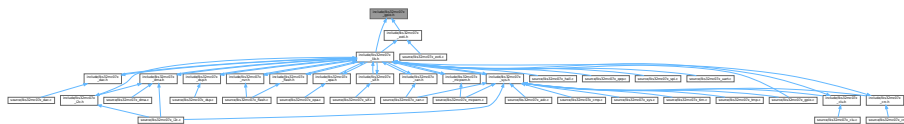
其它说明: 无

```
#include "lks32mc07x.h"
```

[lks32mc07x_gpio.h](#) 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [GPIO_TypeDef](#)
`< Includes -----*/`
- struct [GPIO_InitTypeDef](#)
 GPIO功能配置结构体句柄

宏定义

- #define [GPIO0](#) (([GPIO_TypeDef](#) *)GPIO0_BASE)
 GPIO0结构体基地址定义
- #define [GPIO1](#) (([GPIO_TypeDef](#) *)GPIO1_BASE)
 GPIO1结构体基地址定义
- #define [GPIO2](#) (([GPIO_TypeDef](#) *)GPIO2_BASE)
 GPIO2结构体基地址定义
- #define [GPIO3](#) (([GPIO_TypeDef](#) *)GPIO3_BASE)
 GPIO3结构体基地址定义

- #define AF0_GPIO 0
- #define AF1_SYSAF 1
- #define AF2_HALL 2
- #define AF3_MCPWM 3
- #define AF4_UART 4
- #define AF5_SPI 5
- #define AF6_I2C 6
- #define AF7_TIMER01 7
- #define AF8_TIMER23 8
- #define AF9_DEBUG 9
- #define AF10_CAN 10
- #define AF11_SIF 11
- #define AF12_CL 12
- #define GPIO_Pin_NONE 0
- #define GPIO_Pin_0 BIT0
- #define GPIO_Pin_1 BIT1
- #define GPIO_Pin_2 BIT2
- #define GPIO_Pin_3 BIT3
- #define GPIO_Pin_4 BIT4
- #define GPIO_Pin_5 BIT5
- #define GPIO_Pin_6 BIT6
- #define GPIO_Pin_7 BIT7
- #define GPIO_Pin_8 BIT8
- #define GPIO_Pin_9 BIT9
- #define GPIO_Pin_10 BIT10
- #define GPIO_Pin_11 BIT11
- #define GPIO_Pin_12 BIT12
- #define GPIO_Pin_13 BIT13
- #define GPIO_Pin_14 BIT14
- #define GPIO_Pin_15 BIT15
- #define GPIO_Pin_ALL (uint32_t)0xFFFF
- #define GPIO_PinSource_0 0
- #define GPIO_PinSource_1 1
- #define GPIO_PinSource_2 2
- #define GPIO_PinSource_3 3
- #define GPIO_PinSource_4 4
- #define GPIO_PinSource_5 5
- #define GPIO_PinSource_6 6
- #define GPIO_PinSource_7 7
- #define GPIO_PinSource_8 8
- #define GPIO_PinSource_9 9
- #define GPIO_PinSource_10 10
- #define GPIO_PinSource_11 11
- #define GPIO_PinSource_12 12
- #define GPIO_PinSource_13 13
- #define GPIO_PinSource_14 14
- #define GPIO_PinSource_15 15

枚举

- enum `GPIOMode_TypeDef` { `GPIO_Mode_IN` = 0x00 , `GPIO_Mode_OUT` = 0x01 , `GPIO_Mode_ANA` = 0x02 , `GPIO_Mode_IO` = 0x03 }
- GPIO模式选择枚举*
- enum `GPIOPuPd_TypeDef` { `GPIO_PuPd_NOPULL` = 0x00 , `GPIO_PuPd_UP` = 0x01 }
- GPIO输出模式选择枚举*
- enum `BitAction` { `Bit_RESET` = 0 , `Bit_SET` }
- GPIO寄存器写入量枚举*

函数

- void `GPIO_Init` (`GPIO_TypeDef` *GPIOx, `GPIO_InitTypeDef` *GPIO_InitStruct)
函数名称: `void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)`
- void `GPIO_StructInit` (`GPIO_InitTypeDef` *GPIO_InitStruct)
函数名称: `void GPIO_StructInit(GPIO_InitTypeDef* GPIO_InitStruct)`
- `uint8_t` `GPIO_ReadInputDataBit` (`GPIO_TypeDef` *GPIOx, `uint16_t` GPIO_Pin)
函数名称: `uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)`
- `uint32_t` `GPIO_ReadInputData` (`GPIO_TypeDef` *GPIOx)
函数名称: `uint32_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx)`
- `uint8_t` `GPIO_ReadOutputDataBit` (`GPIO_TypeDef` *GPIOx, `uint16_t` GPIO_Pin)
函数名称: `uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)`
- `uint32_t` `GPIO_ReadOutputData` (`GPIO_TypeDef` *GPIOx)
函数名称: `uint32_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx)`
- void `GPIO_SetBits` (`GPIO_TypeDef` *GPIOx, `uint16_t` GPIO_Pin)
函数名称: `void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)`
- void `GPIO_ResetBits` (`GPIO_TypeDef` *GPIOx, `uint16_t` GPIO_Pin)
函数名称: `void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)`
- void `GPIO_Write` (`GPIO_TypeDef` *GPIOx, `uint32_t` Val)
函数名称: `void GPIO_Write(GPIO_TypeDef* GPIOx, uint32_t Val)`
- void `GPIO_WriteBit` (`GPIO_TypeDef` *GPIOx, `uint16_t` GPIO_Pin, `BitAction` BitVal)
函数名称: `void GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal)`
- void `GPIO_PinAFConfig` (`GPIO_TypeDef` *GPIOx, `uint32_t` GPIO_PinSource, `uint32_t` GPIO_AF)
函数名称: `void GPIO_PinAFConfig(GPIO_TypeDef* GPIOx, uint32_t GPIO_PinSource, uint32_t GPIO_AF)`

5.21.1 详细描述

文件名称: `lks32mc07x_gpio.h`
文件标识: 无
内容摘要: GPIO外设驱动程序头文件
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 `lks32mc07x_gpio.h` 中定义.

5.21.2 宏定义说明

5.21.2.1 AF0.GPIO

```
#define AF0_GPIO 0
```

GPIO复用功能0，模拟功能

在文件 [lks32mc07x_gpio.h](#) 第 101 行定义.

5.21.2.2 AF10.CAN

```
#define AF10_CAN 10
```

GPIO复用功能10，CAN功能

在文件 [lks32mc07x_gpio.h](#) 第 111 行定义.

5.21.2.3 AF11.SIF

```
#define AF11_SIF 11
```

GPIO复用功能11，SIF功能

在文件 [lks32mc07x_gpio.h](#) 第 112 行定义.

5.21.2.4 AF12.CL

```
#define AF12_CL 12
```

GPIO复用功能12，CL功能

在文件 [lks32mc07x_gpio.h](#) 第 113 行定义.

5.21.2.5 AF1.SYSAF

```
#define AF1_SYSAF 1
```

GPIO复用功能1，SYS.AF、CMP、CLOCK等数字信号输出功能

在文件 [lks32mc07x_gpio.h](#) 第 102 行定义.

5.21.2.6 AF2.HALL

```
#define AF2_HALL 2
```

GPIO复用功能2，HALL功能

在文件 [lks32mc07x_gpio.h](#) 第 103 行定义.

5.21.2.7 AF3.MCPWM

```
#define AF3_MCPWM 3
```

GPIO复用功能3，MCPWM功能

在文件 [lks32mc07x_gpio.h](#) 第 104 行定义.

5.21.2.8 AF4.UART

```
#define AF4_UART 4
```

GPIO复用功能4，UART功能

在文件 [lks32mc07x_gpio.h](#) 第 105 行定义.

5.21.2.9 AF5.SPI

```
#define AF5_SPI 5
```

GPIO复用功能5，SPI功能

在文件 [lks32mc07x_gpio.h](#) 第 106 行定义.

5.21.2.10 AF6.I2C

```
#define AF6_I2C 6
```

GPIO复用功能6，I2C功能

在文件 [lks32mc07x_gpio.h](#) 第 107 行定义.

5.21.2.11 AF7.TIMER01

```
#define AF7_TIMER01 7
```

GPIO复用功能7，Timer0、Timer1功能

在文件 [lks32mc07x_gpio.h](#) 第 108 行定义.

5.21.2.12 AF8.TIMER23

```
#define AF8_TIMER23 8
```

GPIO复用功能8，Timer2、Timer3、QEP0、QEP1功能

在文件 [lks32mc07x_gpio.h](#) 第 109 行定义.

5.21.2.13 AF9_DEBUG

```
#define AF9_DEBUG 9
```

GPIO复用功能9，ADC trigger debug功能

在文件 [lks32mc07x_gpio.h](#) 第 110 行定义.

5.21.2.14 GPIO0

```
#define GPIO0 ((GPIO_TypeDef *)GPIO0_BASE)
```

GPIO0结构体基地址定义

在文件 [lks32mc07x_gpio.h](#) 第 43 行定义.

5.21.2.15 GPIO1

```
#define GPIO1 ((GPIO_TypeDef *)GPIO1_BASE)
```

GPIO1结构体基地址定义

在文件 [lks32mc07x_gpio.h](#) 第 48 行定义.

5.21.2.16 GPIO2

```
#define GPIO2 ((GPIO_TypeDef *)GPIO2_BASE)
```

GPIO2结构体基地址定义

在文件 [lks32mc07x_gpio.h](#) 第 53 行定义.

5.21.2.17 GPIO3

```
#define GPIO3 ((GPIO_TypeDef *)GPIO3_BASE)
```

GPIO3结构体基地址定义

在文件 [lks32mc07x_gpio.h](#) 第 58 行定义.

5.21.2.18 GPIO_Pin_0

```
#define GPIO_Pin_0 BIT0
```

GPIO pin脚定义，pin_0脚

在文件 [lks32mc07x_gpio.h](#) 第 117 行定义.

5.21.2.19 GPIO_Pin_1

```
#define GPIO_Pin_1 BIT1
```

GPIO pin脚定义, pin_1脚

在文件 [lks32mc07x-gpio.h](#) 第 118 行定义.

5.21.2.20 GPIO_Pin_10

```
#define GPIO_Pin_10 BIT10
```

GPIO pin脚定义, pin_10脚

在文件 [lks32mc07x-gpio.h](#) 第 127 行定义.

5.21.2.21 GPIO_Pin_11

```
#define GPIO_Pin_11 BIT11
```

GPIO pin脚定义, pin_11脚

在文件 [lks32mc07x-gpio.h](#) 第 128 行定义.

5.21.2.22 GPIO_Pin_12

```
#define GPIO_Pin_12 BIT12
```

GPIO pin脚定义, pin_12脚

在文件 [lks32mc07x-gpio.h](#) 第 129 行定义.

5.21.2.23 GPIO_Pin_13

```
#define GPIO_Pin_13 BIT13
```

GPIO pin脚定义, pin_13脚

在文件 [lks32mc07x-gpio.h](#) 第 130 行定义.

5.21.2.24 GPIO_Pin_14

```
#define GPIO_Pin_14 BIT14
```

GPIO pin脚定义, pin_14脚

在文件 [lks32mc07x-gpio.h](#) 第 131 行定义.

5.21.2.25 GPIO_Pin_15

```
#define GPIO_Pin_15 BIT15
```

GPIO pin脚定义，pin_15脚

在文件 [lks32mc07x_gpio.h](#) 第 132 行定义.

5.21.2.26 GPIO_Pin_2

```
#define GPIO_Pin_2 BIT2
```

GPIO pin脚定义，pin_2脚

在文件 [lks32mc07x_gpio.h](#) 第 119 行定义.

5.21.2.27 GPIO_Pin_3

```
#define GPIO_Pin_3 BIT3
```

GPIO pin脚定义，pin_3脚

在文件 [lks32mc07x_gpio.h](#) 第 120 行定义.

5.21.2.28 GPIO_Pin_4

```
#define GPIO_Pin_4 BIT4
```

GPIO pin脚定义，pin_4脚

在文件 [lks32mc07x_gpio.h](#) 第 121 行定义.

5.21.2.29 GPIO_Pin_5

```
#define GPIO_Pin_5 BIT5
```

GPIO pin脚定义，pin_5脚

在文件 [lks32mc07x_gpio.h](#) 第 122 行定义.

5.21.2.30 GPIO_Pin_6

```
#define GPIO_Pin_6 BIT6
```

GPIO pin脚定义，pin_6脚

在文件 [lks32mc07x_gpio.h](#) 第 123 行定义.

5.21.2.31 GPIO_Pin_7

```
#define GPIO_Pin_7 BIT7
```

GPIO pin脚定义，pin.7脚

在文件 [lks32mc07x_gpio.h](#) 第 124 行定义.

5.21.2.32 GPIO_Pin_8

```
#define GPIO_Pin_8 BIT8
```

GPIO pin脚定义，pin.8脚

在文件 [lks32mc07x_gpio.h](#) 第 125 行定义.

5.21.2.33 GPIO_Pin_9

```
#define GPIO_Pin_9 BIT9
```

GPIO pin脚定义，pin.9脚

在文件 [lks32mc07x_gpio.h](#) 第 126 行定义.

5.21.2.34 GPIO_Pin_ALL

```
#define GPIO_Pin_ALL (uint32_t)0xFFFF
```

GPIO pin脚定义，所有pin脚

在文件 [lks32mc07x_gpio.h](#) 第 133 行定义.

5.21.2.35 GPIO_Pin_NONE

```
#define GPIO_Pin_NONE 0
```

GPIO pin脚定义，无pin脚

在文件 [lks32mc07x_gpio.h](#) 第 116 行定义.

5.21.2.36 GPIO_PinSource_0

```
#define GPIO_PinSource_0 0
```

GPIO pin脚定义，pin.0脚

在文件 [lks32mc07x_gpio.h](#) 第 136 行定义.

5.21.2.37 GPIO_PinSource_1

```
#define GPIO_PinSource_1 1
```

GPIO pin脚定义，pin_1脚

在文件 [lks32mc07x_gpio.h](#) 第 137 行定义.

5.21.2.38 GPIO_PinSource_10

```
#define GPIO_PinSource_10 10
```

GPIO pin脚定义，pin_10脚

在文件 [lks32mc07x_gpio.h](#) 第 146 行定义.

5.21.2.39 GPIO_PinSource_11

```
#define GPIO_PinSource_11 11
```

GPIO pin脚定义，pin_11脚

在文件 [lks32mc07x_gpio.h](#) 第 147 行定义.

5.21.2.40 GPIO_PinSource_12

```
#define GPIO_PinSource_12 12
```

GPIO pin脚定义，pin_12脚

在文件 [lks32mc07x_gpio.h](#) 第 148 行定义.

5.21.2.41 GPIO_PinSource_13

```
#define GPIO_PinSource_13 13
```

GPIO pin脚定义，pin_13脚

在文件 [lks32mc07x_gpio.h](#) 第 149 行定义.

5.21.2.42 GPIO_PinSource_14

```
#define GPIO_PinSource_14 14
```

GPIO pin脚定义，pin_14脚

在文件 [lks32mc07x_gpio.h](#) 第 150 行定义.

5.21.2.43 GPIO_PinSource_15

```
#define GPIO_PinSource_15 15
```

GPIO pin脚定义，pin.15脚

在文件 [lks32mc07x_gpio.h](#) 第 151 行定义.

5.21.2.44 GPIO_PinSource_2

```
#define GPIO_PinSource_2 2
```

GPIO pin脚定义，pin.2脚

在文件 [lks32mc07x_gpio.h](#) 第 138 行定义.

5.21.2.45 GPIO_PinSource_3

```
#define GPIO_PinSource_3 3
```

GPIO pin脚定义，pin.3脚

在文件 [lks32mc07x_gpio.h](#) 第 139 行定义.

5.21.2.46 GPIO_PinSource_4

```
#define GPIO_PinSource_4 4
```

GPIO pin脚定义，pin.4脚

在文件 [lks32mc07x_gpio.h](#) 第 140 行定义.

5.21.2.47 GPIO_PinSource_5

```
#define GPIO_PinSource_5 5
```

GPIO pin脚定义，pin.5脚

在文件 [lks32mc07x_gpio.h](#) 第 141 行定义.

5.21.2.48 GPIO_PinSource_6

```
#define GPIO_PinSource_6 6
```

GPIO pin脚定义，pin.6脚

在文件 [lks32mc07x_gpio.h](#) 第 142 行定义.

5.21.2.49 GPIO_PinSource_7

```
#define GPIO_PinSource_7 7
```

GPIO pin脚定义，pin.7脚

在文件 [lks32mc07x_gpio.h](#) 第 143 行定义.

5.21.2.50 GPIO_PinSource_8

```
#define GPIO_PinSource_8 8
```

GPIO pin脚定义，pin.8脚

在文件 [lks32mc07x_gpio.h](#) 第 144 行定义.

5.21.2.51 GPIO_PinSource_9

```
#define GPIO_PinSource_9 9
```

GPIO pin脚定义，pin.9脚

在文件 [lks32mc07x_gpio.h](#) 第 145 行定义.

5.21.3 枚举类型说明

5.21.3.1 BitAction

```
enum BitAction
```

GPIO寄存器写入量枚举

枚举值

Bit_RESET	
Bit_SET	

在文件 [lks32mc07x_gpio.h](#) 第 83 行定义.

5.21.3.2 GPIOMode_TypeDef

```
enum GPIOMode_TypeDef
```

GPIO模式选择枚举

枚举值

GPIO_Mode_IN	GPIO 输入&高阻，模拟和数字同时生效
GPIO_Mode_OUT	GPIO 输出
GPIO_Mode_ANA	GPIO 高阻
GPIO_Mode_IO	

在文件 [lks32mc07x_gpio.h](#) 第 63 行定义.

5.21.3.3 GPIOPuPd_TypeDef

enum GPIOPuPd_TypeDef

GPIO输出模式选择枚举

枚举值

GPIO_PuPd_NOPULL	高阻态
GPIO_PuPd_UP	上拉

在文件 [lks32mc07x_gpio.h](#) 第 74 行定义.

5.21.4 函数说明

5.21.4.1 GPIO_Init()

```
void GPIO_Init (
    GPIO_TypeDef * GPIOx,
    GPIO_InitTypeDef * this )
```

函数名称: void [GPIO_Init\(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct\)](#)

功能描述: GPIO初始化函数

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_InitStruct: GPIO初始化结构体
输出参数:	无

返回

返回值：无

注解

其它说明：无

警告

无

示例代码:

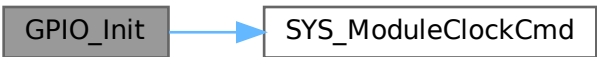
```
GPIO_InitTypeDef GPIO_InitStructure;  
//P2.11输入模式  
GPIO_StructInit(&GPIO_InitStructure); //初始化结构体  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN; //GPIO输入模式  
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11;  
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;  
GPIO_Init(GPIO2, &GPIO_InitStructure);  
//P0.2输出模式  
GPIO_StructInit(&GPIO_InitStructure); //初始化结构体  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT; //GPIO输出模式  
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;  
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;  
GPIO_Init(GPIO0, &GPIO_InitStructure);  
//P3.2开漏输出  
GPIO_StructInit(&GPIO_InitStructure); //初始化结构体  
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT; //GPIO输出模式  
GPIO_InitStructure.GPIO_PODENa = ENABLE; //使能开漏模式  
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;  
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL; //关闭上拉，配置为高阻态  
GPIO_Init(GPIO3, &GPIO_InitStructure);
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 56 行定义.

函数调用图:



5.21.4.2 GPIO_PinAFConfig()

```
void GPIO_PinAFConfig (
    GPIO_TypeDef * GPIOx,
    uint32_t GPIO_PinSource,
    uint32_t GPIO_AF )
```

函数名称: void GPIO_PinAFConfig(GPIO_TypeDef* GPIOx, uint32_t GPIO_PinSource, uint32_t GPIO_AF)

功能描述: GPIO复用功能选择

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3)	
	GPIO_PinSource:指定的PadSource (GPIO_PinSource_0~GPIO_PinSource_15)	
	GPIO_AF:指定复用功能	
	复用功能名称	说明
	AF0.GPIO	模拟功能
	AF1.SYSAF	SYS_AF、CMP、CLOCK等数字信号输出功能
	AF2.HALL	HALL功能
	AF3.MCPWM	MCPWM功能
	AF4.UART	UART功能
	AF5.SPI	SPI功能
	AF6.I2C	I2C功能
	AF7.TIMER01	Timer0、Timer1功能
	AF8.TIMER23	Timer2、Timer3、QEP0、QEP1功能
	AF9.DEBUG	ADC trigger debug功能
	AF10.CAN	CAN功能
	AF11.SIF	SIF功能
	AF12.CL	CL功能
输出参数:	无	

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
GPIO_PinAFConfig(GPIO1,GPIO_PinSource_0,AF4_UART); //P1.0复用为串口功能
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 399 行定义.

5.21.4.3 GPIO_ReadInputData()

```
uint32_t GPIO_ReadInputData (
    GPIO_TypeDef * GPIOx )
```

函数名称: `uint32_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx)`

功能描述: 读取GPIO的输入数据

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3)
输出参数:	无

返回

返回值: 输入数据

注解

其它说明: 无

警告

无

示例代码:

```
u8 GPIO_IN_VALUE = 0;
GPIO_IN_VALUE = GPIO_ReadInputData(GPIO0); //读取P0.0~P0.15输入电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 181 行定义.

5.21.4.4 GPIO_ReadInputDataBit()

```
uint8_t GPIO_ReadInputDataBit (  
    GPIO_TypeDef * GPIOx,  
    uint16_t GPIO_Pin )
```

函数名称: `uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)`

功能描述: 读取GPIO的指定Pin的输入值

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15)
输出参数:	无

返回
返回值: 指定Pin输入数据

注解
其它说明: 无

警告
无

示例代码:

```
u8 GPIO_IN_VALUE = 0;  
GPIO_IN_VALUE = GPIO_ReadInputDataBit (GPIO0,GPIO_Pin_0); //读取P0.0输入电平 1为高电平, 0为低电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 146 行定义.

5.21.4.5 GPIO_ReadOutputData()

```
uint32_t GPIO_ReadOutputData (
    GPIO_TypeDef * GPIOx )
```

函数名称: uint32_t [GPIO_ReadOutputData\(GPIO_TypeDef* GPIOx\)](#)

功能描述: 读取GPIO的输出数据

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3)
输出参数:	无

返回

返回值: 输出数据

注解

其它说明: 无

警告

无

示例代码:

```
u8 GPIO_OUT_VALUE = 0;
GPIO_OUT_VALUE = GPIO\_ReadOutputData\(GPIO0\); //读取P0.0~P0.15输出电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 243 行定义.

5.21.4.6 GPIO_ReadOutputDataBit()

```
uint8_t GPIO_ReadOutputDataBit (
    GPIO_TypeDef * GPIOx,
    uint16_t GPIO_Pin )
```

函数名称: `uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)`

功能描述: 读取GPIO的指定Pin的输出值

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15)
输出参数:	无

返回
返回值: 指定IO输出数据

注解
其它说明: 无

警告
无

示例代码:

```
u8 GPIO_OUT_VALUE = 0;
GPIO_OUT_VALUE = GPIO_ReadOutputDataBit (GPIO0,GPIO_Pin_0); //读取P0.0输出电平 1为高电平，0为低电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 208 行定义.

5.21.4.7 GPIO_ResetBits()

```
void GPIO_ResetBits (
    GPIO_TypeDef * GPIOx,
    uint16_t GPIO_Pin )
```

函数名称: void [GPIO_ResetBits\(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin\)](#)

功能描述: GPIO指定Pin置0

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15)
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
GPIO\_ResetBits (GPIO0, GPIO_Pin_0); //P0.0输出低电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 295 行定义.

5.21.4.8 GPIO_SetBits()

```
void GPIO_SetBits (
    GPIO_TypeDef * GPIOx,
    uint16_t GPIO_Pin )
```

函数名称: void [GPIO_SetBits\(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin\)](#)

功能描述: GPIO指定Pin置1

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15)
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
GPIO\_SetBits\(GPIO0,GPIO\_Pin\_0\) ; //P0.0输出高电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 269 行定义.

5.21.4.9 GPIO_StructInit()

```
void GPIO_StructInit (
    GPIO_InitTypeDef * GPIO_InitStruct )
```

函数名称: void [GPIO_StructInit\(GPIO_InitTypeDef* GPIO_InitStruct\)](#)

功能描述: GPIO结构体初始化

参见
被引用内容: 无

参数

输入参数:	GPIO_InitStruct: GPIO初始化结构体
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
GPIO_InitTypeDef GPIO_InitStruct;
GPIO_StructInit (&GPIO_InitStruct);           //初始化结构体
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 115 行定义.

5.21.4.10 GPIO_Write()

```
void GPIO_Write (
    GPIO_TypeDef * GPIOx,
    uint32_t Val )
```

函数名称: void [GPIO_Write\(GPIO_TypeDef* GPIOx, uint32_t Val\)](#)

功能描述: 向GPIO写入数据

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) BitVal:写入的Bit值 (0x00~0xffff)
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
GPIO\_WriteBit\(GPIO0, 0x01\); //P0.0置1,输出高电平, P0.1~P0.15值不变保持上一时刻状态
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 355 行定义.

5.21.4.11 GPIO_WriteBit()

```
void GPIO_WriteBit (
    GPIO_TypeDef * GPIOx,
    uint16_t GPIO_Pin,
    BitAction BitVal )
```

函数名称: void **GPIO_WriteBit**(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal)

功能描述: 向GPIO指定的Pin写入数据

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15) BitVal:写入的Bit值 (Bit_RESET 或 Bit_SET)
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
GPIO_WriteBit (GPIO0,GPIO_Pin_0,Bit_RESET); //P0.0输出低电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 322 行定义.

5.22 lks32mc07x_gpio.h

[浏览该文件的文档.](#)

```

00001
00014 #ifndef __lks32mc07x_GPIO_H
00015 #define __lks32mc07x_GPIO_H
00016
00018 #include "lks32mc07x.h"
00022 typedef struct
00023 {
00024     __IO uint32_t PIE;
00025     __IO uint32_t POE;
00026     __IO uint32_t PDI;
00027     __IO uint32_t PDO;
00028     __IO uint32_t PUE;
00029     __IO uint32_t null;
00030     __IO uint32_t PODE;
00031     __IO uint32_t PFLT;
00032     __IO uint32_t F3210;
00033     __IO uint32_t F7654;
00034     __IO uint32_t FBA98;
00035     __IO uint32_t FFEDC;
00036     __IO uint32_t BSRR;
00037     __IO uint32_t BRR;
00038 } GPIO_TypeDef;
00039
00043 #define GPIO0 ((GPIO_TypeDef *)GPIO0_BASE)
00044
00048 #define GPIO1 ((GPIO_TypeDef *)GPIO1_BASE)
00049
00053 #define GPIO2 ((GPIO_TypeDef *)GPIO2_BASE)
00054
00058 #define GPIO3 ((GPIO_TypeDef *)GPIO3_BASE)
00059
00063 typedef enum
00064 {
00065     GPIO_Mode_IN = 0x00,
00066     GPIO_Mode_OUT = 0x01,
00067     GPIO_Mode_ANA = 0x02,
00068     GPIO_Mode_IO = 0x03
00069 } GPIO_Mode_TypeDef;
00070
00074 typedef enum
00075 {
00076     GPIO_PuPd_NOPULL = 0x00,
00077     GPIO_PuPd_UP = 0x01,
00078 } GPIO_PuPd_TypeDef;
00079
00083 typedef enum
00084 {
00085     Bit_RESET = 0,
00086     Bit_SET
00087 } BitAction;
00088
00092 typedef struct
00093 {
00094     u32 GPIO_Pin;
00095     u32 GPIO_Mode;
00096     u32 GPIO_PuPd;
00097     u32 GPIO_PODena;
00098     u32 GPIO_PFLT;
00099 } GPIO_InitTypeDef;
00100
00101 #define AF0_GPIO 0
00102 #define AF1_SYSAF 1
00103 #define AF2_HALL 2
00104 #define AF3_MCPWM 3
00105 #define AF4_UART 4
00106 #define AF5_SPI 5
00107 #define AF6_I2C 6
00108 #define AF7_TIMER01 7
00109 #define AF8_TIMER23 8
00110 #define AF9_DEBUG 9
00111 #define AF10_CAN 10
00112 #define AF11_SIF 11
00113 #define AF12_CL 12
00115 /*GPIO_Pad define*/
00116 #define GPIO_Pin_NONE 0
00117 #define GPIO_Pin_0 BIT0
00118 #define GPIO_Pin_1 BIT1
00119 #define GPIO_Pin_2 BIT2
00120 #define GPIO_Pin_3 BIT3
00121 #define GPIO_Pin_4 BIT4
00122 #define GPIO_Pin_5 BIT5
00123 #define GPIO_Pin_6 BIT6

```

```

00124 #define GPIO_Pin_7      BIT7
00125 #define GPIO_Pin_8      BIT8
00126 #define GPIO_Pin_9      BIT9
00127 #define GPIO_Pin_10     BIT10
00128 #define GPIO_Pin_11     BIT11
00129 #define GPIO_Pin_12     BIT12
00130 #define GPIO_Pin_13     BIT13
00131 #define GPIO_Pin_14     BIT14
00132 #define GPIO_Pin_15     BIT15
00133 #define GPIO_Pin_ALL    (uint32_t)0xFFFF
00135 /*GPIO_Pad_sources define*/
00136 #define GPIO_PinSource_0  0
00137 #define GPIO_PinSource_1  1
00138 #define GPIO_PinSource_2  2
00139 #define GPIO_PinSource_3  3
00140 #define GPIO_PinSource_4  4
00141 #define GPIO_PinSource_5  5
00142 #define GPIO_PinSource_6  6
00143 #define GPIO_PinSource_7  7
00144 #define GPIO_PinSource_8  8
00145 #define GPIO_PinSource_9  9
00146 #define GPIO_PinSource_10 10
00147 #define GPIO_PinSource_11 11
00148 #define GPIO_PinSource_12 12
00149 #define GPIO_PinSource_13 13
00150 #define GPIO_PinSource_14 14
00151 #define GPIO_PinSource_15 15
00153 /* 初始化、配置函数 *****/
00154 void GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *GPIO_InitStruct);
00155 void GPIO_StructInit(GPIO_InitTypeDef *GPIO_InitStruct);
00156
00157 /* GPIO读写函数 *****/
00158 uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
00159 uint32_t GPIO_ReadInputData(GPIO_TypeDef *GPIOx);
00160 uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
00161 uint32_t GPIO_ReadOutputData(GPIO_TypeDef *GPIOx);
00162
00163 void GPIO_SetBits(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
00164 void GPIO_ResetBits(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
00165 void GPIO_Write(GPIO_TypeDef *GPIOx, uint32_t Val);
00166 void GPIO_WriteBit(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, BitAction BitVal);
00167
00168 /* GPIO功能选择函数 *****/
00169 void GPIO_PinAFConfig(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint32_t GPIO_AF);
00170
00171 #endif /*_lks32mc07x_GPIO_H */
00172
00173 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.23 include/lks32mc07x_hall.h 文件参考

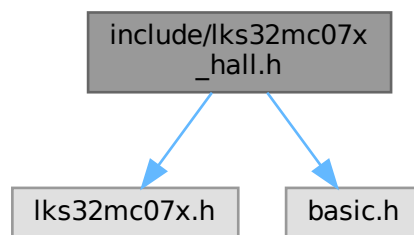
文件名称: [lks32mc07x_hall.h](#)
 文件标识: 无
 内容摘要: HALL驱动头文件
 其它说明: 无

```

#include "lks32mc07x.h"
#include "basic.h"

```

lks32mc07x_hall.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [HALL_TypeDef](#)
*HALL*寄存器结构体句柄
- struct [HALL_InitTypeDef](#)
*HALL*初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

宏定义

- #define [HALL](#) (([HALL_TypeDef](#) *)HALL0.BASE)
*HALL*结构体基地址定义
- #define [HALL_CLK_DIV1](#) ((uint32_t)0x00)
*HALL*时钟为系统主时钟1分频
- #define [HALL_CLK_DIV2](#) ((uint32_t)0x01)
*HALL*时钟为系统主时钟2分频
- #define [HALL_CLK_DIV4](#) ((uint32_t)0x02)
*HALL*时钟为系统主时钟4分频
- #define [HALL_CLK_DIV8](#) ((uint32_t)0x03)
*HALL*时钟为系统主时钟8分频
- #define [HALL_CAPTURE_EVENT](#) ((uint32_t)0x00010000)
*HALL*中断使能和标志定义，*HALL*信号变化中断
- #define [HALL_OVERFLOW_EVENT](#) ((uint32_t)0x00020000)
*HALL*中断使能和标志定义，*HALL*计数溢出中断

函数

- void [HALL_StructInit](#) (HALL_InitTypeDef *HALL_InitStruct)
函数名称: void [HALL_StructInit\(HALL_InitTypeDef* HALL_InitStruct\)](#)
- void [HALL_Init](#) (HALL_InitTypeDef *HALL_InitStruct)
函数名称: void [HALL_Init\(HALL_InitTypeDef* HALL_InitStruct\)](#)
- void [HALL_Cmd](#) (FuncState state)
函数名称: void [HALL_Cmd\(FuncState state\)](#)
- uint32_t [HALL_GetFilterValue](#) (void)
函数名称: uint32_t [HALL_GetFilterValue\(void\)](#)
- uint32_t [HALL_GetCaptureValue](#) (void)
函数名称: uint32_t [HALL_GetCaptureValue\(void\)](#)
- uint32_t [HALL_GetCount](#) (void)
函数名称: uint32_t [HALL_GetCount\(void\)](#)
- uint32_t [HALL_IsCaptureEvent](#) (void)
函数名称: uint32_t [HALL_IsCaptureEvent\(void\)](#)
- uint32_t [HALL_IsOverFlowEvent](#) (void)
函数名称: uint32_t [HALL_IsCaptureEvent\(void\)](#)
- uint32_t [HALL_GetFlag](#) (uint32_t tempFlag)
• void [HALL_Clear_IRQ](#) (uint32_t tempFlag)
函数名称: uint32_t [HALL_ClearIRQFlag\(uint32_t tempFlag\)](#)
- uint32_t [HALL_GetIRQFlag](#) (uint32_t tempFlag)
函数名称: uint32_t [HALL_GetIRQFlag\(uint32_t tempFlag\)](#)
- void [HALL_SoftTrigger](#) (void)
函数名称: uint32_t [HALL_SoftTrigger\(void\)](#)

5.23.1 详细描述

文件名称: [lks32mc07x_hall.h](#)
文件标识: 无
内容摘要: HALL驱动头文件
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.h](#) 中定义.

5.23.2 宏定义说明

5.23.2.1 HALL

```
#define HALL ((HALL_TypeDef *)HALL0_BASE)
```

HALL结构体基地址定义

在文件 [lks32mc07x_hall.h](#) 第 36 行定义.

5.23.2.2 HALL_CAPTURE_EVENT

```
#define HALL_CAPTURE_EVENT ((uint32_t)0x00010000)
```

HALL中断使能和标志定义，HALL信号变化中断

在文件 [lks32mc07x_hall.h](#) 第 58 行定义.

5.23.2.3 HALL_CLK_DIV1

```
#define HALL_CLK_DIV1 ((uint32_t)0x00)
```

HALL时钟为系统主时钟1分频

在文件 [lks32mc07x_hall.h](#) 第 53 行定义.

5.23.2.4 HALL_CLK_DIV2

```
#define HALL_CLK_DIV2 ((uint32_t)0x01)
```

HALL时钟为系统主时钟2分频

在文件 [lks32mc07x_hall.h](#) 第 54 行定义.

5.23.2.5 HALL_CLK_DIV4

```
#define HALL_CLK_DIV4 ((uint32_t)0x02)
```

HALL时钟为系统主时钟4分频

在文件 [lks32mc07x_hall.h](#) 第 55 行定义.

5.23.2.6 HALL_CLK_DIV8

```
#define HALL_CLK_DIV8 ((uint32_t)0x03)
```

HALL时钟为系统主时钟8分频

在文件 [lks32mc07x_hall.h](#) 第 56 行定义.

5.23.2.7 HALL_OVERFLOW_EVENT

```
#define HALL_OVERFLOW_EVENT ((uint32_t)0x00020000)
```

HALL中断使能和标志定义，HALL计数溢出中断

在文件 [lks32mc07x_hall.h](#) 第 59 行定义.

5.23.3 函数说明

5.23.3.1 HALL_Clear_IRQ()

```
void HALL_Clear_IRQ (
    uint32_t tempFlag )
```

函数名称: uint32_t HALL_ClearIRQFlag(uint32_t tempFlag)

功能描述: 清除HALL中断标志位

参见
被引用内容: 无

参数

输入参数:	tempFlag:	
	HALL_CAPTURE_EVENT	HALL信号变化中断
	HALL_OVERFLOW_EVENT	HALL计数溢出中断
输出参数:	无	

返回
返回 值: 无

注解
其它说明: 无

警告
无

示例代码:

```
if (HALL_GetIRQFlag (HALL_CAPTURE_EVENT)) //读取HALL信号变化中断标志位
{
    HALL_ClearIRQFlag (HALL_CAPTURE_EVENT); //清除HALL信号变化中断标志位
}
```


Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 240 行定义.

5.23.3.2 HALL_Cmd()

```
void HALL_Cmd (
    FuncState state )
```

函数名称: void [HALL_Cmd\(FuncState state\)](#)

功能描述: HALL时钟使能

参见

被引用内容: 无

参数

输入参数:	state:	
	ENABLE	HALL使能
	DISABLE	HALL失能
输出参数:	无	

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
HALL\_Cmd (ENABLE); //HALL使能
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 323 行定义.

函数调用图:



5.23.3.3 HALL_GetCaptureValue()

```
uint32_t HALL_GetCaptureValue (
    void )
```

函数名称: `uint32_t HALL_GetCaptureValue(void)`

功能描述: 取得HALL值，未滤波结果，即采样的原始HALL状态值

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返 回 值: 未滤波的HALL信号结果值，前三位有效，其它位恒为0

注解

其它说明: 无

警告

无

示例代码:

```
uint32_t HALL_Value = 0;
HALL_Value = HALL_GetCaptureValue(); //读取未滤波的HALL结果值
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 132 行定义.

5.23.3.4 HALL_GetCount()

```
uint32_t HALL_GetCount (
    void )
```

函数名称: uint32_t [HALL_GetCount\(void\)](#)

功能描述: 读取实时HALL计数器值

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回 值: 实时HALL计数器值

注解

其它说明: 无

警告

无

示例代码:

```
uint32_t HALLValue = 0;
HALLValue = HALL_GetCount(); //读取实时HALL计数器值
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 157 行定义.

5.23.3.5 HALL_GetFilterValue()

```
uint32_t HALL_GetFilterValue (
    void )
```

函数名称: uint32_t [HALL_GetFilterValue\(void\)](#)

功能描述: 取得HALL值，滤波结果

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回 值: 经过滤波的HALL信号结果值，前三位有效，其它位恒为0

注解

其它说明: 无

警告

无

示例代码:

```
uint32_t HALLValue = 0;
HALLValue = HALL_GetFilterValue(); //读取经过滤波的HALL结果值
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 107 行定义.

5.23.3.6 HALL_GetFlag()

```
uint32_t HALL_GetFlag (  
    uint32_t tempFlag )
```

在文件 [lks32mc07x_hall.c](#) 第 327 行定义.

函数调用图:



5.23.3.7 HALL_GetIRQFlag()

```
uint32_t HALL_GetIRQFlag (  
    uint32_t tempFlag )
```

函数名称: `uint32_t HALL_GetIRQFlag(uint32_t tempFlag)`

功能描述: 读取HALL中断标志

参见
被引用内容: 无

参数

输入参数:	tempFlag:	
	HALL_CAPTURE_EVENT	HALL信号变化中断
	HALL_OVERFLOW_EVENT	HALL计数溢出中断
输出参数:	无	

返回

返回 值： HALL中断标志 为HALL_CAPTURE_EVENT即对应标志位置位，为0即对应标志位未置位

注解

其它说明： 无

警告

无

示例代码:

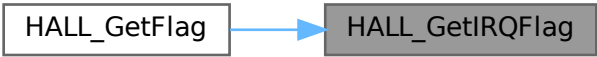
```
if (HALL_CAPTURE_EVENT == HALL_GetIRQFlag (HALL_CAPTURE_EVENT)) //读取HALL信号变化中断标志位
{
}
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 271 行定义.

这是这个函数的调用关系图:



5.23.3.8 HALL_Init()

```
void HALL_Init (
    HALL_InitTypeDef * HALL_InitStruct )
```

函数名称: void [HALL_Init\(HALL_InitTypeDef* HALL_InitStruct\)](#)

功能描述: HALL初始化函数

参见

被引用内容： 无

参数

输入参数:	HALL_InitTypeDef HALL_InitStruct
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
HALL_InitTypeDef HALL_InitStruct;

HALL_StructInit(&HALL_InitStruct);

HALL_InitStruct.FilterLen = 512;           //Hall信号数字滤波长度 512个时钟周期
HALL_InitStruct.ClockDivision = HALL_CLK_DIV8; //设置Hall模块时钟分频系数
HALL_InitStruct.Filter75_Ena = DISABLE;    // Hall信号滤波方式, 7判5模式或者全1有效模式
HALL_InitStruct.HALL_Ena = ENABLE;        // 模块使能
HALL_InitStruct.Capture_IRQ_Ena = ENABLE;  // 捕捉中断使能
HALL_InitStruct.Overflow_IRQ_Ena = ENABLE; // 超时中断使能
HALL_InitStruct.CountTH = 960000;          // Hall模块计数模值, 计数超过模值会产生超时中断
HALL_InitStruct.softIE = ENABLE;          // 软件中断失能
HALL_Init(&HALL_InitStruct);
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 lks32mc07x_hall.c 第 48 行定义.

5.23.3.9 HALL_IsCaptureEvent()

```
uint32_t HALL_IsCaptureEvent (
    void )
```

函数名称: uint32_t HALL_IsCaptureEvent(void)

功能描述: 读取是否发生捕获事件标志

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回 值: 捕获事件标志, 为HALL_CAPTURE_EVENT即对应标志位置位, 为0即对应标志位未置位

注解

其它说明: 无

警告

无

示例代码:

```
if (HALL_CAPTURE_EVENT == HALL_IsCaptureEvent()) //读取HALL信号变化标志位
{
}
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 183 行定义.

5.23.3.10 HALL_IsOverFlowEvent()

```
uint32_t HALL_IsOverFlowEvent (
    void )
```

函数名称: uint32_t [HALL_IsCaptureEvent\(void\)](#)

功能描述: 读取是否发生计时溢出事件标志

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回 值： 捕获事件标志，为HALL_OVERFLOW.EVENT即对应标志位置位，为0即对应标志位未置位

注解

其它说明： 无

警告

无

示例代码:

```
if ( HALL_OVERFLOW_EVENT == HALL_IsOverflowEvent () )//读取计时溢出标志位
{
}
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 209 行定义.

5.23.3.11 HALL.SoftTrigger()

```
void HALL_SoftTrigger (
    void )
```

函数名称: uint32_t [HALL_SoftTrigger\(void\)](#)

功能描述: 软件触发HALL信号变化中断

参见

被引用内容： 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
HALL_SoftTrigger(); // 软件触发HALL信号变化中断
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 296 行定义.

5.23.3.12 HALL_StructInit()

```
void HALL_StructInit (  
    HALL_InitTypeDef * HALL_InitStruct )
```

函数名称: void [HALL_StructInit\(HALL_InitTypeDef* HALL_InitStruct\)](#)

功能描述: HALL结构体初始化

参见

被引用内容: 无

参数

输入参数:	HALL_InitTypeDef HALL_InitStruct
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
HALLEInitTypeDef HALLEInitStruct;  
HALLEStructInit(&HALLEInitStruct); //初始化结构体
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 75 行定义.

5.24 lks32mc07x_hall.h

[浏览该文件的文档.](#)

```
00001  
00014 #ifndef __lks32mc07x_HALL_H  
00015 #define __lks32mc07x_HALL_H  
00016  
00017 /* Includes -----*/  
00018 #include "lks32mc07x.h"  
00019 #include "basic.h"  
00020  
00024 typedef struct  
00025 {  
00026     __IO uint32_t CFG;  
00027     __IO uint32_t INFO;  
00028     __IO uint32_t WIDTH;  
00029     __IO uint32_t TH;  
00030     __IO uint32_t CNT;  
00031 } HALLETypeDef;  
00035 #ifndef HALL  
00036 #define HALL ((HALLETypeDef *)HALLE0_BASE)  
00037 #endif  
00041 typedef struct  
00042 {  
00043     uint32_t CountTH;  
00044     uint16_t FilterLen;  
00045     uint8_t ClockDivision;  
00046     uint8_t Filter75.Ena;  
00047     uint8_t HALL.Ena;  
00048     uint8_t Capture.IRQ.Ena;  
00049     uint8_t OverFlow.IRQ.Ena;  
00050     uint8_t softIE;  
00051 } HALLEInitTypeDef;  
00052
```

```

00053 #define HALL_CLK_DIV1      ((uint32_t)0x00)
00054 #define HALL_CLK_DIV2      ((uint32_t)0x01)
00055 #define HALL_CLK_DIV4      ((uint32_t)0x02)
00056 #define HALL_CLK_DIV8      ((uint32_t)0x03)
00058 #define HALL_CAPTURE_EVENT  ((uint32_t)0x00010000)
00059 #define HALL_OVERFLOW_EVENT ((uint32_t)0x00020000)
00061 void HALL_StructInit (HALL_InitTypeDef *HALL_InitStruct); /*HALL结构体初始化*/
00062 void HALL_Init (HALL_InitTypeDef *HALL_InitStruct); /*HALL初始化*/
00063 void HALL_Cmd (FuncState state); /*HALL使能*/
00064
00065 uint32_t HALL_GetFilterValue (void); /*读取滤波的HALL值*/
00066 uint32_t HALL_GetCaptureValue (void); /*读取未滤波的HALL值*/
00067 uint32_t HALL_GetCount (void); /*读取HALL计数值*/
00068
00069 uint32_t HALL_IsCaptureEvent (void); /*判断是否捕获事件*/
00070 uint32_t HALL_IsOverFlowEvent (void); /*判断是否溢出事件*/
00071 uint32_t HALL_GetFlag (uint32_t tempFlag); /*取得HALL事件标志*/
00072
00073 void HALL_Clear_IRQ (uint32_t tempFlag); /*清除HALL中断标志位*/
00074 uint32_t HALL_GetIRQFlag (uint32_t tempFlag); /*读取HALL中断标志位*/
00075
00076 void HALL_SoftTrigger (void); /*软件触发HALL信号变化中断*/
00077
00078 #endif /*_lks32mc07x_HALL_H */
00079
00080 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/

```

5.25 include/lks32mc07x_i2c.h 文件参考

I2C库函数头文件

```

#include "lks32mc07x.h"
#include "lks32mc07x_dma.h"
#include "basic.h"

```

lks32mc07x_i2c.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct I2C_TypeDef
- struct I2C_InitTypeDef

宏定义

- #define I2C_BUS_STATE_STT_ERR BIT7
 < Includes -----
- #define I2C_BUS_STATE_LOST_ARB BIT6
 总线仲裁丢失
- #define I2C_BUS_STATE_STOP_EVT BIT5
 STOP事件记录
- #define I2C_BUS_STATE_ADDR_DATA BIT3
 正在传输地址数据
- #define I2C_BUS_STATE_RX_ACK BIT1
 接收到ACK响应
- #define I2C_BUS_STATE_Done BIT0
 传输完成
- #define I2C_IF_TC 0
 传输完成中断
- #define I2C_IF_BUS_ERR 1
 总线错误中断
- #define I2C_IF_STOP 2
 STOP事件中断
- #define I2C_IF_NACK 3
 NACK事件中断
- #define I2C_IF_ADDR_CMP 4
 硬件地址匹配中断

函数

- void I2C_Init (I2C_TypeDef *I2Cx, I2C_InitTypeDef *this)
 I2C初始化
- void I2C_StructInit (I2C_InitTypeDef *)
 I2C配置结构体初始化
- void I2C_ClearIRQFlag (I2C_TypeDef *I2Cx, u16 I2C_IF_x)
 I2C清除中断 操作的表: 输出参数:
- u8 I2C_GetIRQFlag (I2C_TypeDef *I2Cx, u16 I2C_IF_x)
 I2C获取当前中断状态 操作的表: 输出参数:
- u8 I2C_GetFlag (I2C_TypeDef *I2Cx, u16 I2C_IF_x)
 I2C清除中断
- void I2C_Enable (I2C_TypeDef *I2Cx)
 I2C模块使能
- void I2C_Disable (I2C_TypeDef *I2Cx)
 I2C模块关闭
- void I2C_SetBaudRate (u32 BaudRate)
 设置I2C模块的波特率
- u8 I2C_ReadBusState (I2C_TypeDef *I2Cx, u16)
 读I2C总线状态
- void I2C_ClearBusState (I2C_TypeDef *I2Cx, u16)
 I2C总线状态复位
- void I2C_SendData (I2C_TypeDef *I2Cx, u8 n)
 I2C发送数据
- u8 I2C_ReadData (I2C_TypeDef *I2Cx)
 I2C读缓冲区数据

5.25.1 详细描述

I2C库函数头文件

作者

YangZJ

日期

2023-01-02

在文件 [lks32mc07x_i2c.h](#) 中定义.

5.25.2 宏定义说明

5.25.2.1 I2C_BUS_STATE_ADDR_DATA

```
#define I2C_BUS_STATE_ADDR_DATA BIT3
```

正在传输地址数据

在文件 [lks32mc07x_i2c.h](#) 第 19 行定义.

5.25.2.2 I2C_BUS_STATE_Done

```
#define I2C_BUS_STATE_Done BIT0
```

传输完成

在文件 [lks32mc07x_i2c.h](#) 第 22 行定义.

5.25.2.3 I2C_BUS_STATE_LOST_ARB

```
#define I2C_BUS_STATE_LOST_ARB BIT6
```

总线仲裁丢失

在文件 [lks32mc07x_i2c.h](#) 第 16 行定义.

5.25.2.4 I2C_BUS_STATE_RX_ACK

```
#define I2C_BUS_STATE_RX_ACK BIT1
```

接收到ACK响应

在文件 [lks32mc07x_i2c.h](#) 第 21 行定义.

5.25.2.5 I2C_BUS_STATE_STOP_EVT

```
#define I2C_BUS_STATE_STOP_EVT BIT5
```

STOP事件记录

在文件 [lks32mc07x_i2c.h](#) 第 17 行定义.

5.25.2.6 I2C_BUS_STATE_STT_ERR

```
#define I2C_BUS_STATE_STT_ERR BIT7
```

< Includes -----

总线错误

在文件 [lks32mc07x_i2c.h](#) 第 15 行定义.

5.25.2.7 I2C_IF_ADDR_CMP

```
#define I2C_IF_ADDR_CMP 4
```

硬件地址匹配中断

在文件 [lks32mc07x_i2c.h](#) 第 28 行定义.

5.25.2.8 I2C_IF_BUS_ERR

```
#define I2C_IF_BUS_ERR 1
```

总线错误中断

在文件 [lks32mc07x_i2c.h](#) 第 25 行定义.

5.25.2.9 I2C_IF_NACK

```
#define I2C_IF_NACK 3
```

NACK事件中断

在文件 [lks32mc07x_i2c.h](#) 第 27 行定义.

5.25.2.10 I2C_IF_STOP

```
#define I2C_IF_STOP 2
```

STOP事件中断

在文件 [lks32mc07x_i2c.h](#) 第 26 行定义.

5.25.2.11 I2C_IF_TC

```
#define I2C_IF_TC 0
```

传输完成中断

在文件 [lks32mc07x_i2c.h](#) 第 24 行定义.

5.26 lks32mc07x_i2c.h

[浏览该文件的文档.](#)

```
00001
00007 #ifndef __lks32mc45x_I2C_H
00008 #define __lks32mc45x_I2C_H
00010 #include "lks32mc07x.h"
00011 #include "lks32mc07x_dma.h"
00012 #include "basic.h"
00013
00014 // I2C_BUS_STATE I2C总线状态
00015 #define I2C_BUS_STATE_STT_ERR BIT7
00016 #define I2C_BUS_STATE_LOST_ARB BIT6
00017 #define I2C_BUS_STATE_STOP_EVT BIT5
00018
00019 #define I2C_BUS_STATE_ADDR_DATA BIT3
00020
00021 #define I2C_BUS_STATE_RX_ACK BIT1
00022 #define I2C_BUS_STATE_Done BIT0
00023
00024 #define I2C_IF_TC 0
00025 #define I2C_IF_BUS_ERR 1
00026 #define I2C_IF_STOP 2
00027 #define I2C_IF_NACK 3
00028 #define I2C_IF_ADDR_CMP 4
00029
00030 typedef struct
00031 {
00032     __IO u32 ADDR;
00033     __IO u32 CFG;
00034     __IO u32 SCR;
00035     __IO u32 DATA;
00036     __IO u32 MSCR;
00037     __IO u32 BCR;
00038     __IO u32 BSIZE;
00039 } I2C_TypeDef;
00040
00041 #ifndef I2C0
00042 #define I2C0 ((I2C_TypeDef *)I2C0_BASE)
00043 #endif
00044 typedef struct
00045 {
00046     u32 ADRCMP;
00047     u32 MST_MODE;
00048     u32 SLV_MODE;
00049     u32 BUSRT_EN;
00050     u32 BaudRate;
00051     u32 IE;
00052     u32 TC_IE;
00053     u32 BUS_ERR_IE;
00054     u32 STOP_IE;
00055     u32 BURST_NACK;
00056     u32 BURST_ADDR_CMP;
00057 } I2C_InitTypeDef;
00058
00059 void I2C_Init(I2C_TypeDef *I2Cx, I2C_InitTypeDef *this);
00060 void I2C_StructInit(I2C_InitTypeDef *);
00061 void I2C_ClearIRQFlag(I2C_TypeDef *I2Cx, u16 I2C_IF_x);
00062 u8 I2C_GetIRQFlag(I2C_TypeDef *I2Cx, u16 I2C_IF_x);
00063 u8 I2C_GetFlag(I2C_TypeDef *I2Cx, u16 I2C_IF_x);
00064
00065 void I2C_Enable(I2C_TypeDef *I2Cx);
00066 void I2C_Disable(I2C_TypeDef *I2Cx);
00067 void I2C_SetBaudRate(u32 BaudRate);
00068
00069 u8 I2C_ReadBusState(I2C_TypeDef *I2Cx, u16);
00070 void I2C_ClearBusState(I2C_TypeDef *I2Cx, u16);
00071 void I2C_SendData(I2C_TypeDef *I2Cx, u8 n);
00072 u8 I2C_ReadData(I2C_TypeDef *I2Cx);
00073 #endif /*__lks32mc03x_I2C_H*/
```


5.27 include/lks32mc07x.lib.h 文件参考

文件名称: [lks32mc07x.lib.h](#)

文件标识: 无

内容摘要: 库函数头文件

其它说明: 无

```
#include "lks32mc07x.h"
#include "lks32mc07x_user_manual.h"
#include "lks32mc07x_adc.h"
#include "lks32mc07x_cmp.h"
#include "lks32mc07x_clu.h"
#include "lks32mc07x_crc.h"
#include "lks32mc07x_can.h"
#include "lks32mc07x_dma.h"
#include "lks32mc07x_dac.h"
#include "lks32mc07x_dsp.h"
#include "lks32mc07x_exti.h"
#include "lks32mc07x_flash.h"
#include "lks32mc07x_gpio.h"
#include "lks32mc07x_hall.h"
#include "lks32mc07x_i2c.h"
#include "lks32mc07x_mcpwm.h"
#include "lks32mc07x_nvr.h"
#include "lks32mc07x_opa.h"
#include "lks32mc07x_qep.h"
#include "lks32mc07x_sif.h"
#include "lks32mc07x_spi.h"
#include "lks32mc07x_sys.h"
#include "lks32mc07x_tim.h"
#include "lks32mc07x_tmp.h"
#include "lks32mc07x_uart.h"
#include "lks32mc07x_wake.h"
```

[lks32mc07x.lib.h](#) 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



宏定义

- #define [REG.WRITE](#)(reg, mask) reg = (mask)
- #define [REG.READ](#)(reg) (reg)
- #define [REG.WRITE_BIT](#)(reg, mask, val) reg = ((reg) & ~(mask)) | (val)
- #define [REG.READ_BIT](#)(reg, mask) ((reg) & (mask))
- #define [REG.IF_WRITE_BIT](#)(reg, mask, condition) reg = (condition) ? ((reg) | (mask)) : ((reg) & ~(mask))
- #define [REG.READ_BIT_POS_MASK](#)(reg, name) (((reg) & (name##_MASK)) >> name##_POS)
- #define [REG.WRITE_BIT_POS_MASK](#)(reg, name, val) [REG.WRITE_BIT](#)(reg, name##_MASK, name(val))
- #define [REG.SET](#)(reg, mask) reg |= (mask)
- #define [REG.RESET](#)(reg, mask) reg &= ~(mask)

5.27.1 详细描述

文件名称: [lks32mc07x.lib.h](#)

文件标识: 无

内容摘要: 库函数头文件

其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月12日	1.1	HuangMG	修改

修改日志:

在文件 [lks32mc07x.lib.h](#) 中定义.

5.27.2 宏定义说明

5.27.2.1 REG_IF_WRITE_BIT

```
#define REG_IF_WRITE_BIT(  
    reg,  
    mask,  
    condition ) reg = (condition) ? ((reg) | (mask)) : ((reg) & ~(mask))
```

在文件 [lks32mc07x.lib.h](#) 第 51 行定义.

5.27.2.2 REG_READ

```
#define REG_READ(  
    reg ) (reg)
```

在文件 [lks32mc07x.lib.h](#) 第 48 行定义.

5.27.2.3 REG_READ_BIT

```
#define REG_READ_BIT(  
    reg,  
    mask ) ((reg) & (mask))
```

在文件 [lks32mc07x.lib.h](#) 第 50 行定义.

5.27.2.4 REG_READ_BIT_POS_MASK

```
#define REG_READ_BIT_POS_MASK(  
    reg,  
    name ) (((reg) & (name##_MASK)) >> name##_POS)
```

在文件 [lks32mc07x.lib.h](#) 第 52 行定义.

5.27.2.5 REG_RESET

```
#define REG_RESET(  
    reg,  
    mask ) reg &= ~(mask)
```

在文件 [lks32mc07x.lib.h](#) 第 55 行定义.

5.27.2.6 REG_SET

```
#define REG_SET(  
    reg,  
    mask ) reg |= (mask)
```

在文件 [lks32mc07x.lib.h](#) 第 54 行定义.

5.27.2.7 REG_WRITE

```
#define REG_WRITE(  
    reg,  
    mask ) reg = (mask)
```

在文件 [lks32mc07x.lib.h](#) 第 47 行定义.

5.27.2.8 REG_WRITE_BIT

```
#define REG_WRITE_BIT(  
    reg,  
    mask,  
    val ) reg = ((reg) & ~(mask)) | (val)
```

在文件 [lks32mc07x.lib.h](#) 第 49 行定义.

5.27.2.9 REG_WRITE_BIT_POS_MASK

```
#define REG_WRITE_BIT_POS_MASK(  
    reg,  
    name,  
    val ) REG_WRITE_BIT(reg, name##_MASK, name(val))
```

在文件 [lks32mc07x.lib.h](#) 第 53 行定义.

5.28 lks32mc07x_lib.h

浏览该文件的文档。

```

00001
00014 #ifndef __lks32mc07x_LIB_H
00015 #define __lks32mc07x_LIB_H
00016
00017 /* Includes -----*/
00018 // typedef enum {DISABLE = 0, ENABLE = 1} FuncState;
00019
00020 #include "lks32mc07x.h"
00021 #include "lks32mc07x_user_manual.h"
00022 #include "lks32mc07x_adc.h"
00023 #include "lks32mc07x_cmp.h"
00024 #include "lks32mc07x_clu.h"
00025 #include "lks32mc07x_crc.h"
00026 #include "lks32mc07x_can.h"
00027 #include "lks32mc07x_dma.h"
00028 #include "lks32mc07x_dac.h"
00029 #include "lks32mc07x_dsp.h"
00030 #include "lks32mc07x_exti.h"
00031 #include "lks32mc07x_flash.h"
00032 #include "lks32mc07x_gpio.h"
00033 #include "lks32mc07x_hall.h"
00034 #include "lks32mc07x_i2c.h"
00035 #include "lks32mc07x_mcpwm.h"
00036 #include "lks32mc07x_nvr.h"
00037 #include "lks32mc07x_opa.h"
00038 #include "lks32mc07x_qep.h"
00039 #include "lks32mc07x_sif.h"
00040 #include "lks32mc07x_spi.h"
00041 #include "lks32mc07x_sys.h"
00042 #include "lks32mc07x_tim.h"
00043 #include "lks32mc07x_tmp.h"
00044 #include "lks32mc07x_uart.h"
00045 #include "lks32mc07x_wake.h"
00046
00047 #define REG.WRITE(reg, mask)          reg = (mask)
00048 // 写寄存器
00049 #define REG.READ(reg)                (reg)
00050 // 读寄存器
00049 #define REG.WRITE_BIT(reg, mask, val) reg = ((reg) & ~(mask)) | (val)
00050 // 按位写
00050 #define REG.READ_BIT(reg, mask)      ((reg) & (mask))
00051 // 按位读
00051 #define REG.IF_WRITE_BIT(reg, mask, condition) reg = (condition) ? ((reg) | (mask)) : ((reg) & ~(mask))
00052 // 条件写
00052 #define REG.READ_BIT_POS_MASK(reg, name) (((reg) & (name##_MASK)) >> name##_POS)
00053 // 读取寄存器中指定掩码位的值
00053 #define REG.WRITE_BIT_POS_MASK(reg, name, val) REG.WRITE_BIT(reg, name##_MASK, name(val))
00054 // 写寄存器中指定掩码位的值
00054 #define REG.SET(reg, mask)           reg |= (mask)
00055 // 写1
00055 #define REG.RESET(reg, mask)        reg &= ~(mask)
00056 // 写0
00056
00057 #endif /*__lks32mc07x_UART_H */
00058
00059 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/

```

5.29 include/lks32mc07x_mcpwm.h 文件参考

文件名称: LKS32MC07x_mcpwm.h

文件标识: 无

内容摘要: MCPWM外设驱动程序

其它说明: 无

#include "lks32mc07x_lib.h"

lks32mc07x_mcpwm.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了：



结构体

- struct [MCPWM_TypeDef](#)
- struct [MCPWM_InitTypeDef](#)

宏定义

- #define [MCPWM0](#) ((MCPWM_TypeDef *)MCPWM0_BASE)
- #define [MCPWM0_CENTRAL_PWM_MODE](#) 0
中心对齐PWM模式
- #define [MCPWM0_EDGE_PWM_MODE](#) 1
边沿对齐PWM模式
- #define [MCPWM0_HIGH_LEVEL](#) 1
高电平
- #define [MCPWM0_LOW_LEVEL](#) 0
高电平
- #define [MCPWM0_HIGH_LEVEL_ACTIVE](#) 0
高电平有效
- #define [MCPWM0_LOW_LEVEL_ACTIVE](#) 1
低电平有效
- #define [MCPWM0_FAIL_SEL_CMP](#) 1
Fail事件来源比较器
- #define [MCPWM0_FAIL_SEL_IO](#) 0
Fail事件来源比较器
- #define [MCPWM0_TH00_AUPDATE](#) BIT0
MCPWM.TH00 自动加载使能
- #define [MCPWM0_TH01_AUPDATE](#) BIT1
MCPWM.TH01 自动加载使能
- #define [MCPWM0_TH10_AUPDATE](#) BIT2
MCPWM.TH10 自动加载使能
- #define [MCPWM0_TH11_AUPDATE](#) BIT3
MCPWM.TH11 自动加载使能
- #define [MCPWM0_TH20_AUPDATE](#) BIT4
MCPWM.TH20 自动加载使能
- #define [MCPWM0_TH21_AUPDATE](#) BIT5
MCPWM.TH21 自动加载使能
- #define [MCPWM0_TMR0_AUPDATE](#) BIT8
MCPWM.TMR0 自动加载使能
- #define [MCPWM0_TMR1_AUPDATE](#) BIT9
MCPWM.TMR1 自动加载使能
- #define [MCPWM0_TH0_AUPDATE](#) BIT10
MCPWM.TH0 自动加载使能

- #define MCPWM0_CNT0_AUPDAT BIT11
MCPWM.CNT0 自动加载使能
- #define MCPWM0_TH30_AUPDATE BIT16
MCPWM.TH00 自动加载使能
- #define MCPWM0_TH31_AUPDATE BIT17
MCPWM.TH01 自动加载使能
- #define MCPWM0_TH40_AUPDATE BIT18
MCPWM.TH10 自动加载使能
- #define MCPWM0_TH41_AUPDATE BIT19
MCPWM.TH11 自动加载使能
- #define MCPWM0_TH50_AUPDATE BIT20
MCPWM.TH20 自动加载使能
- #define MCPWM0_TH51_AUPDATE BIT20
MCPWM.TH21 自动加载使能
- #define MCPWM0_TMR2_AUPDATE BIT24
MCPWM.TMR0 自动加载使能
- #define MCPWM0_TMR3_AUPDATE BIT25
MCPWM.TMR1 自动加载使能
- #define MCPWM0_TH1_AUPDATE BIT26
MCPWM.TH0 自动加载使能

- #define MCPWM0_CNT1_AUPDAT BIT27
MCPWM.CNT0 自动加载使能
- #define MCPWM0_ALL_AUPDAT 0xffffffff
全部自动加载使能
- #define MCPWM0CNT0_T0_IRQ_IF BIT0
计数器0计数值回到MCPWM.TH中断标志位
- #define MCPWM0CNT0_T1_IRQ_IF BIT1
计数器0计数值回到0中断标志位
- #define MCPWM0CNT0_TH00_IRQ_IF BIT2
计数器0计数值回到MCPWM.TH00中断标志位
- #define MCPWM0CNT0_TH01_IRQ_IF BIT3
计数器0计数值回到MCPWM.TH01中断标志位
- #define MCPWM0CNT0_TH10_IRQ_IF BIT4
计数器0计数值回到MCPWM.TH10中断标志位
- #define MCPWM0CNT0_TH11_IRQ_IF BIT5
计数器0计数值回到MCPWM.TH11中断标志位
- #define MCPWM0CNT0_TH20_IRQ_IF BIT6
计数器0计数值回到MCPWM.TH20中断标志位
- #define MCPWM0CNT0_TH21_IRQ_IF BIT7
计数器0计数值回到MCPWM.TH21中断标志位
- #define MCPWM0CNT0_TMR0_IRQ_IF BIT10
计数器0计数值回到MCPWM.TMR0中断标志位
- #define MCPWM0CNT0_TMR1_IRQ_IF BIT11
计数器0计数值回到MCPWM.TMR1中断标志位
- #define MCPWM0CNT0_TMR2_IRQ_IF BIT12
计数器0计数值回到MCPWM.TMR2中断标志位
- #define MCPWM0CNT0_TMR3_IRQ_IF BIT13
计数器0计数值回到MCPWM.TMR3中断标志位
- #define MCPWM0CNT0_UP_IRQ_IF BIT14
计数器0更新中断标志位

- **#define MCPWM0CNT1_T0_IRQ_IF** BIT0
计数器0计数值回到-MCPWM.TH中断标志位
- **#define MCPWM0CNT1_T1_IRQ_IF** BIT1
计数器0计数值回到0中断标志位
- **#define MCPWM0CNT1_TH30_IRQ_IF** BIT2
计数器0计数值回到MCPWM.TH00中断标志位
- **#define MCPWM0CNT1_TH31_IRQ_IF** BIT3
计数器0计数值回到MCPWM.TH01中断标志位
- **#define MCPWM0CNT1_TH40_IRQ_IF** BIT4
计数器0计数值回到MCPWM.TH10中断标志位
- **#define MCPWM0CNT1_TH41_IRQ_IF** BIT5
计数器0计数值回到MCPWM.TH11中断标志位
- **#define MCPWM0CNT1_TH50_IRQ_IF** BIT6
计数器0计数值回到MCPWM.TH20中断标志位
- **#define MCPWM0CNT1_TH51_IRQ_IF** BIT7
计数器0计数值回到MCPWM.TH21中断标志位
- **#define MCPWM0CNT1_TMR2_IRQ_IF** BIT12
计数器0计数值回到MCPWM.TMR2中断标志位
- **#define MCPWM0CNT1_TMR3_IRQ_IF** BIT13
计数器0计数值回到MCPWM.TMR3中断标志位
- **#define MCPWM0CNT1_UP_IRQ_IF** BIT14
计数器0更新中断标志位
- **#define MCPWM0_FAIL0_IRQ_IF** BIT19
MCPWMCH0.P和CH0N同时有效中断标志位
- **#define MCPWM0_FAIL1_IRQ_IF** BIT20
MCPWMCH0.P和CH0N同时有效中断标志位
- **#define MCPWM0_FAIL2_IRQ_IF** BIT21
MCPWMCH0.P和CH0N同时有效中断标志位
- **#define MCPWM0_FAIL3_IRQ_IF** BIT22
MCPWMCH0.P和CH0N同时有效中断标志位
- **#define MCPWM0_ZCS0_IRQ_IF** BIT23
MCPWMCH0.P和CH0N同时有效中断标志位
- **#define MCPWM0_ZCS1_IRQ_IF** BIT24
MCPWMCH0.P和CH0N同时有效中断标志位
- **#define MCPWM_IF0_UP_IF** BIT14
时基0更新事件MCPWM0.TH0/MCPWM0.TH00~MCPWM0.TH21/MCPWM0.TMR等寄存器更新到MCPWM实际运行系统的中断事件。
- **#define MCPWM_IF0_TMR3_IF** BIT13
MCPWM0.CNT0等于MCPWM0.TMR3中断事件
- **#define MCPWM_IF0_TMR2_IF** BIT12
MCPWM0.CNT0等于MCPWM0.TMR2中断事件
- **#define MCPWM_IF0_TMR1_IF** BIT11
MCPWM0.CNT0等于MCPWM0.TMR1中断事件
- **#define MCPWM_IF0_TMR0_IF** BIT10
MCPWM0.CNT0等于MCPWM0.TMR0中断事件
- **#define MCPWM_IF0_TH21_IF** BIT7
MCPWM0.CNT0等于MCPWM0.TH21中断事件
- **#define MCPWM_IF0_TH20_IF** BIT6
MCPWM0.CNT0等于MCPWM0.TH20中断事件
- **#define MCPWM_IF0_TH11_IF** BIT5
MCPWM0.CNT0等于MCPWM0.TH11中断事件

- #define **MCPWM_IF0_TH10_IF** BIT4
MCPWM0_CNT0等于MCPWM0_TH10中断事件
- #define **MCPWM_IF0_TH01_IF** BIT3
MCPWM0_CNT0等于MCPWM0_TH01中断事件
- #define **MCPWM_IF0_TH00_IF** BIT2
MCPWM0_CNT0等于MCPWM0_TH00中断事件
- #define **MCPWM_IF0_T1_IF** BIT1
t1事件, MCPWM0_CNT0等于0中断事件
- #define **MCPWM_IF0_T0_IF** BIT0
t0事件, MCPWM0_CNT0等于MCPWM0_TH中断事件
- #define **MCPWM_IF1_UP_IF** BIT14
时基1更新事件MCPWM0_TH0/MCPWM0_TH00~MCPWM0_TH21/MCPWM0_TMR等寄存器更新到MCPWM实际运行系统的中断事件。
- #define **MCPWM_IF1_TMR3_IF** BIT13
MCPWM0_CNT1等于MCPWM0_TMR3中断事件
- #define **MCPWM_IF1_TMR2_IF** BIT12
MCPWM0_CNT1等于MCPWM0_TMR2中断事件
- #define **MCPWM_IF1_TMR1_IF** BIT11
MCPWM0_CNT1等于MCPWM0_TMR1中断事件
- #define **MCPWM_IF1_TMR0_IF** BIT10
MCPWM0_CNT1等于MCPWM0_TMR0中断事件
- #define **MCPWM_IF1_TH51_IF** BIT7
MCPWM0_CNT1等于MCPWM0_TH51中断事件
- #define **MCPWM_IF1_TH50_IF** BIT6
MCPWM0_CNT1等于MCPWM0_TH50中断事件
- #define **MCPWM_IF1_TH41_IF** BIT5
MCPWM0_CNT1等于MCPWM0_TH41中断事件
- #define **MCPWM_IF1_TH40_IF** BIT4
MCPWM0_CNT1等于MCPWM0_TH40中断事件
- #define **MCPWM_IF1_TH31_IF** BIT3
MCPWM0_CNT1等于MCPWM0_TH31中断事件
- #define **MCPWM_IF1_TH30_IF** BIT2
MCPWM0_CNT1等于MCPWM0_TH30中断事件
- #define **MCPWM_IF1_T1_IF** BIT1
t1事件, MCPWM0_CNT1等于0中断事件
- #define **MCPWM_IF1_T0_IF** BIT0
t0事件, MCPWM0_CNT1等于MCPWM0_TH中断事件
- #define **MCPWM{EIF_ZCS1}** BIT9
时基1 ZCS中断源事件。
- #define **MCPWM{EIF_ZCS0}** BIT8
时基0 ZCS中断源事件。
- #define **MCPWM{EIF_FAIL3}** BIT7
FAIL3中断源事件。
- #define **MCPWM{EIF_FAIL2}** BIT6
FAIL2中断源事件。
- #define **MCPWM{EIF_FAIL1}** BIT5
FAIL1中断源事件。
- #define **MCPWM{EIF_FAIL0}** BIT4
FAIL0中断源事件。

枚举

- enum MCPWM0CNTx { MCPWM0CNT0 = 0 , MCPWM0CNT1 = 1 }

函数

- void MCPWM_Init (MCPWM_InitTypeDef *MCPWM_InitStruct)
函数名称: void MCPWM_Init(MCPWM_InitTypeDef *MCPWM_InitStruct)
- void MCPWM_StructInit (MCPWM_InitTypeDef *MCPWM_InitStruct)
函数名称: void MCPWM_StructInit(MCPWM_InitTypeDef* MCPWM_InitStruct)
- void PWMOutputs (FuncState t.state)
函数名称: void PWMOutputs(FuncState t.state)
- void MCPWM_ClearIRQFlag (MCPWM0CNTx CNTx, uint32_t INT_flag)
函数名称: void MCPWM_ClearIRQFlag(uint32_t INT_flag)
- u16 MCPWM_GetIRQFlag (MCPWM0CNTx CNTx, uint32_t INT_flag)
函数名称: u16 MCPWM_GetIRQFlag(MCPWM0CNTx CNTx,uint32_t INT_flag)
- void MCPWM_Clk_Enable (u8 Clk_Enable)
- void MCPWM_DIV_Set (u8 ClockDiv)
- void MCPWM_CNTx_Enable (MCPWM0CNTx CNTx, FuncState state)
- void MCPWM_CH012_Init (MCPWM_InitTypeDef *MCPWM_InitStruct)
- void MCPWM_CH345_Init (MCPWM_InitTypeDef *MCPWM_InitStruct)
- void MCPWM_CH012_IESet (MCPWM_InitTypeDef *MCPWM_InitStruct)
- void MCPWM_CH345_IESet (MCPWM_InitTypeDef *MCPWM_InitStruct)
- void MCPWM_ModuleClockCmd (FuncState state)

5.29.1 详细描述

文件名称: LKS32MC07x_mcpwm.h
文件标识: 无
内容摘要: MCPWM外设驱动程序
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年3月06日	1.0	Zhujie	创建

修改日志:

在文件 lks32mc07x_mcpwm.h 中定义.

5.29.2 宏定义说明

5.29.2.1 MCPWM0

```
#define MCPWM0 ((MCPWM_TypeDef *)MCPWM0_BASE)
```

在文件 [lks32mc07x_mcpwm.h](#) 第 88 行定义.

5.29.2.2 MCPWM0_ALL_AUPDAT

```
#define MCPWM0_ALL_AUPDAT 0xffffffff
```

全部自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 356 行定义.

5.29.2.3 MCPWM0_CENTRAL_PWM_MODE

```
#define MCPWM0_CENTRAL_PWM_MODE 0
```

中心对齐PWM模式

在文件 [lks32mc07x_mcpwm.h](#) 第 322 行定义.

5.29.2.4 MCPWM0_CNT0_AUPDAT

```
#define MCPWM0_CNT0_AUPDAT BIT11
```

MCPWM_CNT0 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 344 行定义.

5.29.2.5 MCPWM0_CNT1_AUPDAT

```
#define MCPWM0_CNT1_AUPDAT BIT27
```

MCPWM_CNT0 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 355 行定义.

5.29.2.6 MCPWM0_EDGE_PWM_MODE

```
#define MCPWM0_EDGE_PWM_MODE 1
```

边沿对齐PWM模式

在文件 [lks32mc07x_mcpwm.h](#) 第 323 行定义.

5.29.2.7 MCPWM0_FAIL0_IRQ_IF

```
#define MCPWM0_FAIL0_IRQ_IF BIT19
```

MCPWMCH0_P和CH0N同时有效中断标志位
在文件 [lks32mc07x_mcpwm.h](#) 第 386 行定义.

5.29.2.8 MCPWM0_FAIL1_IRQ_IF

```
#define MCPWM0_FAIL1_IRQ_IF BIT20
```

MCPWMCH0_P和CH0N同时有效中断标志位
在文件 [lks32mc07x_mcpwm.h](#) 第 387 行定义.

5.29.2.9 MCPWM0_FAIL2_IRQ_IF

```
#define MCPWM0_FAIL2_IRQ_IF BIT21
```

MCPWMCH0_P和CH0N同时有效中断标志位
在文件 [lks32mc07x_mcpwm.h](#) 第 388 行定义.

5.29.2.10 MCPWM0_FAIL3_IRQ_IF

```
#define MCPWM0_FAIL3_IRQ_IF BIT22
```

MCPWMCH0_P和CH0N同时有效中断标志位
在文件 [lks32mc07x_mcpwm.h](#) 第 389 行定义.

5.29.2.11 MCPWM0_FAIL_SEL_CMP

```
#define MCPWM0_FAIL_SEL_CMP 1
```

Fail事件来源比较器

在文件 [lks32mc07x_mcpwm.h](#) 第 331 行定义.

5.29.2.12 MCPWM0_FAIL_SEL_IO

```
#define MCPWM0_FAIL_SEL_IO 0
```

Fail事件来源比较器

在文件 [lks32mc07x_mcpwm.h](#) 第 332 行定义.

5.29.2.13 MCPWM0_HIGH_LEVEL

```
#define MCPWM0_HIGH_LEVEL 1
```

高电平

在文件 [lks32mc07x_mcpwm.h](#) 第 325 行定义.

5.29.2.14 MCPWM0_HIGH_LEVEL_ACTIVE

```
#define MCPWM0_HIGH_LEVEL_ACTIVE 0
```

高电平有效

在文件 [lks32mc07x_mcpwm.h](#) 第 328 行定义.

5.29.2.15 MCPWM0_LOW_LEVEL

```
#define MCPWM0_LOW_LEVEL 0
```

高电平

在文件 [lks32mc07x_mcpwm.h](#) 第 326 行定义.

5.29.2.16 MCPWM0_LOW_LEVEL_ACTIVE

```
#define MCPWM0_LOW_LEVEL_ACTIVE 1
```

低电平有效

在文件 [lks32mc07x_mcpwm.h](#) 第 329 行定义.

5.29.2.17 MCPWM0_TH00_AUPDATE

```
#define MCPWM0_TH00_AUPDATE BIT0
```

MCPWM_TH00 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 335 行定义.

5.29.2.18 MCPWM0_TH01_AUPDATE

```
#define MCPWM0_TH01_AUPDATE BIT1
```

MCPWM_TH01 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 336 行定义.

5.29.2.19 MCPWM0_TH0_AUPDATE

```
#define MCPWM0_TH0_AUPDATE BIT10
```

MCPWM_TH0 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 343 行定义.

5.29.2.20 MCPWM0_TH10_AUPDATE

```
#define MCPWM0_TH10_AUPDATE BIT2
```

MCPWM_TH10 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 337 行定义.

5.29.2.21 MCPWM0_TH11_AUPDATE

```
#define MCPWM0_TH11_AUPDATE BIT3
```

MCPWM_TH11 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 338 行定义.

5.29.2.22 MCPWM0_TH1_AUPDATE

```
#define MCPWM0_TH1_AUPDATE BIT26
```

MCPWM_TH0 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 354 行定义.

5.29.2.23 MCPWM0_TH20_AUPDATE

```
#define MCPWM0_TH20_AUPDATE BIT4
```

MCPWM_TH20 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 339 行定义.

5.29.2.24 MCPWM0_TH21_AUPDATE

```
#define MCPWM0_TH21_AUPDATE BIT5
```

MCPWM_TH21 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 340 行定义.

5.29.2.25 MCPWM0_TH30_AUPDATE

```
#define MCPWM0_TH30_AUPDATE BIT16
```

MCPWM_TH00 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 346 行定义.

5.29.2.26 MCPWM0_TH31_AUPDATE

```
#define MCPWM0_TH31_AUPDATE BIT17
```

MCPWM_TH01 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 347 行定义.

5.29.2.27 MCPWM0_TH40_AUPDATE

```
#define MCPWM0_TH40_AUPDATE BIT18
```

MCPWM_TH10 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 348 行定义.

5.29.2.28 MCPWM0_TH41_AUPDATE

```
#define MCPWM0_TH41_AUPDATE BIT19
```

MCPWM_TH11 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 349 行定义.

5.29.2.29 MCPWM0_TH50_AUPDATE

```
#define MCPWM0_TH50_AUPDATE BIT20
```

MCPWM_TH20 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 350 行定义.

5.29.2.30 MCPWM0_TH51_AUPDATE

```
#define MCPWM0_TH51_AUPDATE BIT20
```

MCPWM_TH21 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 351 行定义.

5.29.2.31 MCPWM0_TMR0_AUPDATE

```
#define MCPWM0_TMR0_AUPDATE BIT8
```

MCPWM.TMR0 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 341 行定义.

5.29.2.32 MCPWM0_TMR1_AUPDATE

```
#define MCPWM0_TMR1_AUPDATE BIT9
```

MCPWM.TMR1 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 342 行定义.

5.29.2.33 MCPWM0_TMR2_AUPDATE

```
#define MCPWM0_TMR2_AUPDATE BIT24
```

MCPWM.TMR0 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 352 行定义.

5.29.2.34 MCPWM0_TMR3_AUPDATE

```
#define MCPWM0_TMR3_AUPDATE BIT25
```

MCPWM.TMR1 自动加载使能

在文件 [lks32mc07x_mcpwm.h](#) 第 353 行定义.

5.29.2.35 MCPWM0_ZCS0_IRQ_IF

```
#define MCPWM0_ZCS0_IRQ_IF BIT23
```

MCPWMCH0_P和CH0N同时有效中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 390 行定义.

5.29.2.36 MCPWM0_ZCS1_IRQ_IF

```
#define MCPWM0_ZCS1_IRQ_IF BIT24
```

MCPWMCH0_P和CH0N同时有效中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 391 行定义.

5.29.2.37 MCPWM0CNT0_T0_IRQ_IF

```
#define MCPWM0CNT0_T0_IRQ_IF BIT0
```

计数器0计数值回到-MCPWM_TH中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 358 行定义.

5.29.2.38 MCPWM0CNT0_T1_IRQ_IF

```
#define MCPWM0CNT0_T1_IRQ_IF BIT1
```

计数器0计数值回到0中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 359 行定义.

5.29.2.39 MCPWM0CNT0_TH00_IRQ_IF

```
#define MCPWM0CNT0_TH00_IRQ_IF BIT2
```

计数器0计数值回到MCPWM_TH00中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 360 行定义.

5.29.2.40 MCPWM0CNT0_TH01_IRQ_IF

```
#define MCPWM0CNT0_TH01_IRQ_IF BIT3
```

计数器0计数值回到MCPWM_TH01中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 361 行定义.

5.29.2.41 MCPWM0CNT0_TH10_IRQ_IF

```
#define MCPWM0CNT0_TH10_IRQ_IF BIT4
```

计数器0计数值回到MCPWM_TH10中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 362 行定义.

5.29.2.42 MCPWM0CNT0_TH11_IRQ_IF

```
#define MCPWM0CNT0_TH11_IRQ_IF BIT5
```

计数器0计数值回到MCPWM_TH11中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 363 行定义.

5.29.2.43 MCPWM0CNT0_TH20_IRQ_IF

```
#define MCPWM0CNT0_TH20_IRQ_IF BIT6
```

计数器0计数值回到MCPWM_TH20中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 364 行定义.

5.29.2.44 MCPWM0CNT0_TH21_IRQ_IF

```
#define MCPWM0CNT0_TH21_IRQ_IF BIT7
```

计数器0计数值回到MCPWM_TH21中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 365 行定义.

5.29.2.45 MCPWM0CNT0_TMR0_IRQ_IF

```
#define MCPWM0CNT0_TMR0_IRQ_IF BIT10
```

计数器0计数值回到MCPWM_TMR0中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 367 行定义.

5.29.2.46 MCPWM0CNT0_TMR1_IRQ_IF

```
#define MCPWM0CNT0_TMR1_IRQ_IF BIT11
```

计数器0计数值回到MCPWM_TMR1中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 368 行定义.

5.29.2.47 MCPWM0CNT0_TMR2_IRQ_IF

```
#define MCPWM0CNT0_TMR2_IRQ_IF BIT12
```

计数器0计数值回到MCPWM_TMR2中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 369 行定义.

5.29.2.48 MCPWM0CNT0_TMR3_IRQ_IF

```
#define MCPWM0CNT0_TMR3_IRQ_IF BIT13
```

计数器0计数值回到MCPWM_TMR3中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 370 行定义.

5.29.2.49 MCPWM0CNT0_UP_IRQ_IF

```
#define MCPWM0CNT0_UP_IRQ_IF BIT14
```

计数器0更新中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 371 行定义.

5.29.2.50 MCPWM0CNT1_T0_IRQ_IF

```
#define MCPWM0CNT1_T0_IRQ_IF BIT0
```

计数器0计数值回到-MCPWM_TH中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 373 行定义.

5.29.2.51 MCPWM0CNT1_T1_IRQ_IF

```
#define MCPWM0CNT1_T1_IRQ_IF BIT1
```

计数器0计数值回到0中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 374 行定义.

5.29.2.52 MCPWM0CNT1_TH30_IRQ_IF

```
#define MCPWM0CNT1_TH30_IRQ_IF BIT2
```

计数器0计数值回到MCPWM_TH00中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 375 行定义.

5.29.2.53 MCPWM0CNT1_TH31_IRQ_IF

```
#define MCPWM0CNT1_TH31_IRQ_IF BIT3
```

计数器0计数值回到MCPWM_TH01中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 376 行定义.

5.29.2.54 MCPWM0CNT1_TH40_IRQ_IF

```
#define MCPWM0CNT1_TH40_IRQ_IF BIT4
```

计数器0计数值回到MCPWM_TH10中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 377 行定义.

5.29.2.55 MCPWM0CNT1_TH41_IRQ_IF

```
#define MCPWM0CNT1_TH41_IRQ_IF BIT5
```

计数器0计数值回到MCPWM_TH11中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 378 行定义.

5.29.2.56 MCPWM0CNT1_TH50_IRQ_IF

```
#define MCPWM0CNT1_TH50_IRQ_IF BIT6
```

计数器0计数值回到MCPWM_TH20中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 379 行定义.

5.29.2.57 MCPWM0CNT1_TH51_IRQ_IF

```
#define MCPWM0CNT1_TH51_IRQ_IF BIT7
```

计数器0计数值回到MCPWM_TH21中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 380 行定义.

5.29.2.58 MCPWM0CNT1_TMR2_IRQ_IF

```
#define MCPWM0CNT1_TMR2_IRQ_IF BIT12
```

计数器0计数值回到MCPWM_TMR2中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 382 行定义.

5.29.2.59 MCPWM0CNT1_TMR3_IRQ_IF

```
#define MCPWM0CNT1_TMR3_IRQ_IF BIT13
```

计数器0计数值回到MCPWM_TMR3中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 383 行定义.

5.29.2.60 MCPWM0CNT1_UP_IRQ_IF

```
#define MCPWM0CNT1_UP_IRQ_IF BIT14
```

计数器0更新中断标志位

在文件 [lks32mc07x_mcpwm.h](#) 第 384 行定义.

5.29.2.61 MCPWM_EIF_FAIL0

```
#define MCPWM_EIF_FAIL0 BIT4
```

FAIL0中断源事件。

定义MCPWM定时器序号

在文件 [lks32mc07x_mcpwm.h](#) 第 431 行定义.

5.29.2.62 MCPWM_EIF_FAIL1

```
#define MCPWM_EIF_FAIL1 BIT5
```

FAIL1中断源事件。

在文件 [lks32mc07x_mcpwm.h](#) 第 428 行定义.

5.29.2.63 MCPWM_EIF_FAIL2

```
#define MCPWM_EIF_FAIL2 BIT6
```

FAIL2中断源事件。

在文件 [lks32mc07x_mcpwm.h](#) 第 427 行定义.

5.29.2.64 MCPWM_EIF_FAIL3

```
#define MCPWM_EIF_FAIL3 BIT7
```

FAIL3中断源事件。

在文件 [lks32mc07x_mcpwm.h](#) 第 426 行定义.

5.29.2.65 MCPWM_EIF_ZCS0

```
#define MCPWM_EIF_ZCS0 BIT8
```

时基0 ZCS中断源事件。

在文件 [lks32mc07x_mcpwm.h](#) 第 425 行定义.

5.29.2.66 MCPWM_EIF_ZCS1

```
#define MCPWM_EIF_ZCS1 BIT9
```

时基1 ZCS中断源事件。

在文件 [lks32mc07x_mcpwm.h](#) 第 424 行定义.

5.29.2.67 MCPWM_IF0_T0_IF

```
#define MCPWM_IF0_T0_IF BIT0
```

t0事件，MCPWM0_CNT0等于MCPWM0_TH中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 406 行定义.

5.29.2.68 MCPWM_IF0_T1_IF

```
#define MCPWM_IF0_T1_IF BIT1
```

t1事件，MCPWM0_CNT0等于0中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 405 行定义.

5.29.2.69 MCPWM_IF0_TH00_IF

```
#define MCPWM_IF0_TH00_IF BIT2
```

MCPWM0_CNT0等于MCPWM0_TH00中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 404 行定义.

5.29.2.70 MCPWM_IF0_TH01_IF

```
#define MCPWM_IF0_TH01_IF BIT3
```

MCPWM0_CNT0等于MCPWM0_TH01中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 403 行定义.

5.29.2.71 MCPWM_IF0_TH10_IF

```
#define MCPWM_IF0_TH10_IF BIT4
```

MCPWM0_CNT0等于MCPWM0_TH10中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 402 行定义.

5.29.2.72 MCPWM_IF0_TH11_IF

```
#define MCPWM_IF0_TH11_IF BIT5
```

MCPWM0_CNT0等于MCPWM0_TH11中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 401 行定义.

5.29.2.73 MCPWM_IF0_TH20_IF

```
#define MCPWM_IF0_TH20_IF BIT6
```

MCPWM0_CNT0等于MCPWM0_TH20中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 400 行定义.

5.29.2.74 MCPWM_IF0_TH21_IF

```
#define MCPWM_IF0_TH21_IF BIT7
```

MCPWM0_CNT0等于MCPWM0_TH21中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 399 行定义.

5.29.2.75 MCPWM_IF0_TMR0_IF

```
#define MCPWM_IF0_TMR0_IF BIT10
```

MCPWM0_CNT0等于MCPWM0_TMR0中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 398 行定义.

5.29.2.76 MCPWM_IF0_TMR1_IF

```
#define MCPWM_IF0_TMR1_IF BIT11
```

MCPWM0_CNT0等于MCPWM0_TMR1中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 397 行定义.

5.29.2.77 MCPWM_IF0_TMR2_IF

```
#define MCPWM_IF0_TMR2_IF BIT12
```

MCPWM0_CNT0等于MCPWM0_TMR2中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 396 行定义.

5.29.2.78 MCPWM_IF0_TMR3_IF

```
#define MCPWM_IF0_TMR3_IF BIT13
```

MCPWM0_CNT0等于MCPWM0_TMR3中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 395 行定义.

5.29.2.79 MCPWM_IF0_UP_IF

```
#define MCPWM_IF0_UP_IF BIT14
```

时基0更新事件MCPWM0_TH0/MCPWM0_TH00~MCPWM0_TH21/MCPWM0_TMR等寄存器更新到MCPWM实际运行系统的中断事件。

在文件 [lks32mc07x_mcpwm.h](#) 第 394 行定义。

5.29.2.80 MCPWM_IF1_T0_IF

```
#define MCPWM_IF1_T0_IF BIT0
```

t0事件，MCPWM0_CNT1等于MCPWM0_TH中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 421 行定义。

5.29.2.81 MCPWM_IF1_T1_IF

```
#define MCPWM_IF1_T1_IF BIT1
```

t1事件，MCPWM0_CNT1等于0中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 420 行定义。

5.29.2.82 MCPWM_IF1_TH30_IF

```
#define MCPWM_IF1_TH30_IF BIT2
```

MCPWM0_CNT1等于MCPWM0_TH30中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 419 行定义。

5.29.2.83 MCPWM_IF1_TH31_IF

```
#define MCPWM_IF1_TH31_IF BIT3
```

MCPWM0_CNT1等于MCPWM0_TH31中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 418 行定义。

5.29.2.84 MCPWM_IF1_TH40_IF

```
#define MCPWM_IF1_TH40_IF BIT4
```

MCPWM0_CNT1等于MCPWM0_TH40中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 417 行定义。

5.29.2.85 MCPWM_IF1_TH41_IF

```
#define MCPWM_IF1_TH41_IF BIT5
```

MCPWM0_CNT1等于MCPWM0_TH41中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 416 行定义.

5.29.2.86 MCPWM_IF1_TH50_IF

```
#define MCPWM_IF1_TH50_IF BIT6
```

MCPWM0_CNT1等于MCPWM0_TH50中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 415 行定义.

5.29.2.87 MCPWM_IF1_TH51_IF

```
#define MCPWM_IF1_TH51_IF BIT7
```

MCPWM0_CNT1等于MCPWM0_TH51中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 414 行定义.

5.29.2.88 MCPWM_IF1_TMR0_IF

```
#define MCPWM_IF1_TMR0_IF BIT10
```

MCPWM0_CNT1等于MCPWM0_TMR0中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 413 行定义.

5.29.2.89 MCPWM_IF1_TMR1_IF

```
#define MCPWM_IF1_TMR1_IF BIT11
```

MCPWM0_CNT1等于MCPWM0_TMR1中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 412 行定义.

5.29.2.90 MCPWM_IF1_TMR2_IF

```
#define MCPWM_IF1_TMR2_IF BIT12
```

MCPWM0_CNT1等于MCPWM0_TMR2中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 411 行定义.

5.29.2.91 MCPWM_IF1_TMR3_IF

```
#define MCPWM_IF1_TMR3_IF BIT13
```

MCPWM0.CNT1等于MCPWM0.TMR3中断事件

在文件 [lks32mc07x_mcpwm.h](#) 第 410 行定义.

5.29.2.92 MCPWM_IF1_UP_IF

```
#define MCPWM_IF1_UP_IF BIT14
```

时基1更新事件MCPWM0.TH0/MCPWM0.TH00~MCPWM0.TH21/MCPWM0.TMR等寄存器更新到MCPWM实际运行系统的中断事件。

在文件 [lks32mc07x_mcpwm.h](#) 第 409 行定义.

5.29.3 枚举类型说明

5.29.3.1 MCPWM0CNTx

```
enum MCPWM0CNTx
```

枚举值

MCPWM0CNT0	MCPWM0计数器0
MCPWM0CNT1	MCPWM0计数器1

在文件 [lks32mc07x_mcpwm.h](#) 第 432 行定义.

5.29.4 函数说明

5.29.4.1 MCPWM_CH012.IESet()

```
void MCPWM_CH012.IESet (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 583 行定义.

5.29.4.2 MCPWM_CH012.Init()

```
void MCPWM_CH012.Init (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 432 行定义.

5.29.4.3 MCPWM_CH345.IESet()

```
void MCPWM_CH345_IETSet (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 590 行定义.

5.29.4.4 MCPWM_CH345.Init()

```
void MCPWM_CH345_Init (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 508 行定义.

5.29.4.5 MCPWM_ClearIRQFlag()

```
void MCPWM_ClearIRQFlag (
    MCPWM0CNTx CNTx,
    uint32_t INT_flag )
```

函数名称: void MCPWM_ClearIRQFlag(uint32_t INT_flag)

功能描述: 清除MCPWM中断标志

参见

被引用内容: 无

参数

参数

输入参数:	MCPWM0CNTx	MCPWM0CNT0 计数器0	
	MCPWM0CNTx	MCPWM0CNT1 计数器1	
	MCPWM0CNT0.T0_IRQ_IF	BIT0	计数器0计数值回到-MCPWM.TH中断标志位
	MCPWM0CNT0.T1_IRQ_IF	BIT1	计数器0计数值回到0中断标志位
	MCPWM0CNT0.TH00↔ IRQ_IF	BIT2	计数器0计数值回到MCPWM.TH00中断标志位
	MCPWM0CNT0.TH01↔ IRQ_IF	BIT3	计数器0计数值回到MCPWM.TH01中断标志位
	MCPWM0CNT0.TH10↔ IRQ_IF	BIT4	计数器0计数值回到MCPWM.TH10中断标志位
	MCPWM0CNT0.TH11↔ IRQ_IF	BIT5	计数器0计数值回到MCPWM.TH11中断标志位
	MCPWM0CNT0.TH20↔ IRQ_IF	BIT6	计数器0计数值回到MCPWM.TH20中断标志位
	MCPWM0CNT0.TH21↔ IRQ_IF	BIT7	计数器0计数值回到MCPWM.TH21中断标志位
	MCPWM0CNT0.TMR0↔ IRQ_IF	BIT10	计数器0计数值回到MCPWM.TMR0中断标志位
	MCPWM0CNT0.TMR1↔ IRQ_IF	BIT11	计数器0计数值回到MCPWM.TMR1中断标志位
	MCPWM0CNT0.TMR2↔ IRQ_IF	BIT12	计数器0计数值回到MCPWM.TMR2中断标志位
	MCPWM0CNT0.TMR3↔ IRQ_IF	BIT13	计数器0计数值回到MCPWM.TMR3中断标志位
	MCPWM0CNT0.UP_IRQ_IF	BIT14	计数器0更新中断标志位
	MCPWM0CNT1.T0_IRQ_IF	BIT0	计数器1计数值回到-MCPWM.TH中断标志位
	MCPWM0CNT1.T1_IRQ_IF	BIT1	计数器1计数值回到0中断标志位
	MCPWM0CNT1.TH30↔ IRQ_IF	BIT2	计数器1计数值回到MCPWM.TH30中断标志位
	MCPWM0CNT1.TH31↔ IRQ_IF	BIT3	计数器1计数值回到MCPWM.TH31中断标志位
	MCPWM0CNT1.TH40↔ IRQ_IF	BIT4	计数器1计数值回到MCPWM.TH40中断标志位
	MCPWM0CNT1.TH41↔ IRQ_IF	BIT5	计数器1计数值回到MCPWM.TH41中断标志位
	MCPWM0CNT1.TH50↔ IRQ_IF	BIT6	计数器1计数值回到MCPWM.TH50中断标志位
	MCPWM0CNT1.TH51↔ IRQ_IF	BIT7	计数器1计数值回到MCPWM.TH51中断标志位

参数

输出参数:	无
-------	---

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (MCPWM_GetIRQFlag(MCPWM0CNT0, MCPWM0CNT0.T0_IRQ_IF)) //获取计数器0计数值回到-MCPWM_TH中断标志位
{
    MCPWM_ClearIRQFlag(MCPWM0CNT0, MCPWM0CNT0.T0_IRQ_IF) //清除计数器0计数值回到-MCPWM_TH中断标志位
}
```

Date	Version	Author	Description
2023年03月06日	1.0	Zhujie	创建

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 267 行定义.

5.29.4.6 MCPWM_Clk.Enable()

```
void MCPWM_Clk.Enable (
    u8 Clk_Enable )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 375 行定义.

5.29.4.7 MCPWM_CNTx.Enable()

```
void MCPWM_CNTx.Enable (
    MCPWM0CNTx CNTx,
    FuncState state )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 404 行定义.

5.29.4.8 MCPWM_DIV_Set()

```
void MCPWM_DIV_Set (
    u8 ClockDiv )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 390 行定义.

5.29.4.9 MCPWM_GetIRQFlag()

```
ul6 MCPWM_GetIRQFlag (
    MCPWM0CNTx CNTx,
    uint32_t INT_flag )
```

函数名称: [u16 MCPWM_GetIRQFlag\(MCPWM0CNTx CNTx,uint32.t INT_flag\)](#)

功能描述: 获取MCPWM中断标志

参见

被引用内容: 无

参数

参数

输入参数:	MCPWM0CNTx	MCPWM0CNT0 计数器0	
	MCPWM0CNTx	MCPWM0CNT1 计数器1	
	MCPWM0CNT0.T0_IRQ_IF	BIT0	计数器0计数值回到-MCPWM.TH中断标志位
	MCPWM0CNT0.T1_IRQ_IF	BIT1	计数器0计数值回到0中断标志位
	MCPWM0CNT0.TH00↔ IRQ_IF	BIT2	计数器0计数值回到MCPWM.TH00中断标志位
	MCPWM0CNT0.TH01↔ IRQ_IF	BIT3	计数器0计数值回到MCPWM.TH01中断标志位
	MCPWM0CNT0.TH10↔ IRQ_IF	BIT4	计数器0计数值回到MCPWM.TH10中断标志位
	MCPWM0CNT0.TH11↔ IRQ_IF	BIT5	计数器0计数值回到MCPWM.TH11中断标志位
	MCPWM0CNT0.TH20↔ IRQ_IF	BIT6	计数器0计数值回到MCPWM.TH20中断标志位
	MCPWM0CNT0.TH21↔ IRQ_IF	BIT7	计数器0计数值回到MCPWM.TH21中断标志位
	MCPWM0CNT0.TMR0↔ IRQ_IF	BIT10	计数器0计数值回到MCPWM.TMR0中断标志位
	MCPWM0CNT0.TMR1↔ IRQ_IF	BIT11	计数器0计数值回到MCPWM.TMR1中断标志位
	MCPWM0CNT0.TMR2↔ IRQ_IF	BIT12	计数器0计数值回到MCPWM.TMR2中断标志位
	MCPWM0CNT0.TMR3↔ IRQ_IF	BIT13	计数器0计数值回到MCPWM.TMR3中断标志位
	MCPWM0CNT0.UP_IRQ_IF	BIT14	计数器0更新中断标志位
	MCPWM0CNT1.T0_IRQ_IF	BIT0	计数器1计数值回到-MCPWM.TH中断标志位
	MCPWM0CNT1.T1_IRQ_IF	BIT1	计数器1计数值回到0中断标志位
	MCPWM0CNT1.TH30↔ IRQ_IF	BIT2	计数器1计数值回到MCPWM.TH30中断标志位
	MCPWM0CNT1.TH31↔ IRQ_IF	BIT3	计数器1计数值回到MCPWM.TH31中断标志位
	MCPWM0CNT1.TH40↔ IRQ_IF	BIT4	计数器1计数值回到MCPWM.TH40中断标志位
	MCPWM0CNT1.TH41↔ IRQ_IF	BIT5	计数器1计数值回到MCPWM.TH41中断标志位
	MCPWM0CNT1.TH50↔ IRQ_IF	BIT6	计数器1计数值回到MCPWM.TH50中断标志位
	MCPWM0CNT1.TH51↔ IRQ_IF	BIT7	计数器1计数值回到MCPWM.TH51中断标志位

参数

输出参数:	无
-------	---

返回

返回 值: 读对应标志位是否为1, 1位置位, 0为未置位

注解

其它说明: 无

警告

只有对应中断使能后, 改为才能读取, 如果对应中断未使能, 读取结果一直为0

示例代码:

```
if (MCPWM_GetIRQFlag (MCPWM0CNT0, MCPWM0CNT0_T0_IRQ_IF)) //获取计数器0计数值回到-MCPWM_TH中断标志位
{
}
```

Date	Version	Author	Description
2023年03月06日	1.0	Zhujie	创建

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 333 行定义.

5.29.4.10 MCPWM.Init()

```
void MCPWM_Init (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

函数名称: void [MCPWM_Init\(MCPWM_InitTypeDef *MCPWM_InitStruct\)](#)

功能描述: MCPWM初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	MCPWM_InitTypeDef *MCPWM_InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
MCPWM_InitTypeDef MCPWM_InitStructure;  
MCPWM_StructInit (&MCPWM_InitStructure); //初始化MCPWM相关结构体  
MCPWM_InitStructure.MCLK_EN = ENABLE;  
MCPWM_InitStructure.CLK_DIV = 0;  
MCPWM_InitStructure.CMP_FLT_CLKDIV = 12;  
MCPWM_InitStructure.BASE_CNT0_EN = ENABLE;  
MCPWM_InitStructure.TH0 = PWM_PERIOD0;  
MCPWM_InitStructure.TH00 = -(PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH01 = (PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH10 = -(PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH11 = (PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH20 = -(PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH21 = (PWM_PERIOD0 >> 2);  
  
MCPWM_InitStructure.MCPWM_WorkModeCH0 = MCPWM0_CENTRAL_PWM_MODE;  
MCPWM_InitStructure.MCPWM_WorkModeCH1 = MCPWM0_CENTRAL_PWM_MODE;  
MCPWM_InitStructure.MCPWM_WorkModeCH2 = MCPWM0_CENTRAL_PWM_MODE;  
MCPWM_InitStructure.DeadTimeCH012N = DEADTIME;  
MCPWM_InitStructure.DeadTimeCH012P = DEADTIME;  
MCPWM_InitStructure.CMP_CTRL_CNT0 = DISABLE;  
MCPWM_InitStructure.EVT_CNT0_EN = DISABLE;  
MCPWM_InitStructure.EVT0 = DISABLE;  
  
MCPWM_InitStructure.TR0_UP_INTV = DISABLE;  
MCPWM_InitStructure.TR0_T0_UpdateEN = ENABLE;  
MCPWM_InitStructure.TR0_T1_UpdateEN = DISABLE;  
MCPWM_InitStructure.TR0_AEC = DISABLE;  
  
MCPWM_InitStructure.T0_Update0_INT_EN = DISABLE;  
MCPWM_InitStructure.T1_Update0_INT_EN = ENABLE;  
MCPWM_InitStructure.Update1_INT_EN = ENABLE;  
MCPWM_InitStructure.CH0N.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH0P.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH1N.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH1P.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH2N.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH2P.Polarity_INV = DISABLE;  
  
MCPWM_InitStructure.Switch_CH0N_CH0P = DISABLE;  
MCPWM_InitStructure.Switch_CH1N_CH1P = DISABLE;  
MCPWM_InitStructure.Switch_CH2N_CH2P = DISABLE;  
MCPWM_InitStructure.CH0P.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH0N.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH1P.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH1N.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH2P.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH2N.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_Init (&MCPWM_InitStructure);
```

Date	Version	Author	Description
2023年03月06日	1.0	ZhuJie	创建

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 111 行定义.

函数调用图:



5.29.4.11 MCPWM.ModuleClockCmd()

```
void MCPWM.ModuleClockCmd (
    FuncState state )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 597 行定义.

函数调用图:



5.29.4.12 MCPWM.StructInit()

```
void MCPWM.StructInit (
    MCPWM.InitTypeDef * MCPWM.InitStruct )
```

函数名称: void [MCPWM.StructInit\(MCPWM.InitTypeDef* MCPWM.InitStruct\)](#)

功能描述: MCPWM结构体初始化

参见

被引用内容: 无

参数

输入参数:	MCPWM.InitTypeDef *MCPWM.InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
MCPWM_InitTypeDef MCPWM_InitStructure;  
MCPWM_StructInit (&MCPWM_InitStructure); //初始化MCPWM相关结构体
```

Date	Version	Author	Description
2019年05月21日	1.0	Howlet	创建

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 39 行定义.

5.29.4.13 PWMOutputs()

```
void PWMOutputs (  
    FuncState t_state )
```

函数名称: void [PWMOutputs\(FuncState t_state\)](#)

功能描述: MCPWM波形输出使能与失能函数

参见

被引用内容: 无

参数

输入参数:	FuncState t_state ENABLE: 使能, DISABLE:使能
输出参数:	无

返回

返回 值: 无

注解

其它说明： 无

警告

无

示例代码:

```
PWMOutputs (ENABLE) //使能MCPWM输出
```

Date	Version	Author	Description
2019年05月21日	1.0	Howlet	创建
2023年03月06日	2.0	Zhujie	修改

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 202 行定义.

5.30 lks32mc07x_mcpwm.h

[浏览该文件的文档.](#)

```
00001
00015 #ifndef __LKS08X_PWM_H
00016 #define __LKS08X_PWM_H
00017
00018 /* Includes -----*/
00019 #include "lks32mc07x_lib.h"
00020
00021 typedef struct
00022 {
00023     __IO u32 TH00;
00024     __IO u32 TH01;
00025     __IO u32 TH10;
00026     __IO u32 TH11;
00028     __IO u32 TH20;
00029     __IO u32 TH21;
00030     __IO u32 TH30;
00031     __IO u32 TH31;
00033     __IO u32 TH40;
00034     __IO u32 TH41;
00035     __IO u32 TH50;
00036     __IO u32 TH51;
00038     __IO u32 TMR0;
00039     __IO u32 TMR1;
00040     __IO u32 TMR2;
00041     __IO u32 TMR3;
00043     __IO u32 TH0;
00044     __IO u32 TH1;
00045     __IO u32 CNT0;
00046     __IO u32 CNT1;
00048     __IO u32 UPDATE;
00049     __IO u32 FCNT;
00050     __IO u32 EVT0;
00051     __IO u32 EVT1;
00053     __IO u32 DTH00;
00054     __IO u32 DTH01;
00055     __IO u32 DTH10;
00056     __IO u32 DTH11;
00058     __IO u32 FLT;
```

```

00059     __IO u32 SDCFG;
00060     __IO u32 AUCN;
00061     __IO u32 TCLK;
00063     __IO u32 IE0;
00064     __IO u32 IF0;
00065     __IO u32 IE1;
00066     __IO u32 IF1;
00068     __IO u32 EIE;
00069     __IO u32 EIF;
00070     __IO u32 RE;
00071     __IO u32 PP;
00073     __IO u32 IO01;
00074     __IO u32 IO23;
00075     __IO u32 IO45;
00076     __IO u32 null;
00078     __IO u32 FAIL012;
00079     __IO u32 FAIL345;
00080     __IO u32 CH_DEF;
00081     __IO u32 CH_MSK;
00083     __IO u32 PRT;
00084     __IO u32 STT_HYST;
00085     __IO u32 ZCS_DELAY;
00086 } MCPWM_TypeDef;
00087
00088 #define MCPWM0 ((MCPWM_TypeDef *)MCPWM0_BASE)
00089
00090 typedef struct
00091 {
00092     uint16_t TH0;
00093     uint16_t TH1;
00095     uint16_t TH00;
00096     uint16_t TH01;
00097     uint16_t TH10;
00098     uint16_t TH11;
00099     uint16_t TH20;
00100     uint16_t TH21;
00102     uint16_t TH30;
00103     uint16_t TH31;
00104     uint16_t TH40;
00105     uint16_t TH41;
00106     uint16_t TH50;
00107     uint16_t TH51;
00109     uint8_t CMP_CTRL_CNT1;
00110     uint8_t CMP_CTRL_CNT0;
00112     uint8_t BASE_CNT1_EN;
00113     uint8_t BASE_CNT0_EN;
00115     uint8_t EVT_CNT1_EN;
00116     uint8_t EVT_CNT0_EN;
00118     uint8_t TMR3_TB;
00119     uint8_t TMR2_TB;
00121     uint8_t ZCS_EN;
00123     uint8_t MCLK_EN;
00124     uint8_t CLK_DIV;
00126     uint16_t MCPWM_WorkModeCH0;
00127     uint16_t MCPWM_WorkModeCH1;
00128     uint16_t MCPWM_WorkModeCH2;
00129     uint16_t MCPWM_WorkModeCH3;
00130     uint16_t MCPWM_WorkModeCH4;
00131     uint16_t MCPWM_WorkModeCH5;
00133     uint16_t TMR0;
00134     uint16_t TMR1;
00135     uint16_t TMR2;
00136     uint16_t TMR3;
00138     uint16_t DeadTimeCH012N;
00139     uint16_t DeadTimeCH012P;
00140     uint16_t DeadTimeCH345N;
00141     uint16_t DeadTimeCH345P;
00143     uint16_t EVT0;
00144     uint16_t EVT1;
00146     uint8_t CH0N.Polarity.INV;
00147     uint8_t CH0P.Polarity.INV;
00148     uint8_t CH1N.Polarity.INV;
00149     uint8_t CH1P.Polarity.INV;
00150     uint8_t CH2N.Polarity.INV;
00151     uint8_t CH2P.Polarity.INV;
00152     uint8_t CH3N.Polarity.INV;
00153     uint8_t CH3P.Polarity.INV;
00154     uint8_t CH4N.Polarity.INV;
00155     uint8_t CH4P.Polarity.INV;
00156     uint8_t CH5N.Polarity.INV;
00157     uint8_t CH5P.Polarity.INV;
00159     uint8_t CH0P.SCTRLP;
00160     uint8_t CH0N.SCTRLN;
00161     uint8_t CH1P.SCTRLP;
00162     uint8_t CH1N.SCTRLN;
00163     uint8_t CH2P.SCTRLP;
00164     uint8_t CH2N.SCTRLN;

```

```
00165      uint8_t CH3P_SCTRLP;  
00166      uint8_t CH3N_SCTRLN;  
00167      uint8_t CH4P_SCTRLP;  
00168      uint8_t CH4N_SCTRLN;  
00169      uint8_t CH5P_SCTRLP;  
00170      uint8_t CH5N_SCTRLN;  
00172      uint8_t CH0_PS;  
00173      uint8_t CH0_NS;  
00174      uint8_t CH1_PS;  
00175      uint8_t CH1_NS;  
00176      uint8_t CH2_PS;  
00177      uint8_t CH2_NS;  
00178      uint8_t CH3_PS;  
00179      uint8_t CH3_NS;  
00180      uint8_t CH4_PS;  
00181      uint8_t CH4_NS;  
00182      uint8_t CH5_PS;  
00183      uint8_t CH5_NS;  
00185      uint8_t Switch_CH0N.CH0P;  
00186      uint8_t Switch_CH1N.CH1P;  
00187      uint8_t Switch_CH2N.CH2P;  
00188      uint8_t Switch_CH3N.CH3P;  
00189      uint8_t Switch_CH4N.CH4P;  
00190      uint8_t Switch_CH5N.CH5P;  
00192      uint8_t TR0_UP_INTV;  
00193      uint8_t TR0_T0_UpdateEN;  
00194      uint8_t TR0_T1_UpdateEN;  
00195      uint8_t TR0_AEC;  
00197      uint8_t TR1_UP_INTV;  
00198      uint8_t TR1_T0_UpdateEN;  
00199      uint8_t TR1_T1_UpdateEN;  
00200      uint8_t TR1_AEC;  
00202      uint8_t FAIL0.INPUT_EN;  
00203      uint8_t FAIL1.INPUT_EN;  
00204      uint8_t FAIL0.OCAP;  
00205      uint8_t FAIL0.Signal_Sel;  
00206      uint8_t FAIL1.Signal_Sel;  
00207      uint8_t FAIL0.Polarity;  
00208      uint8_t FAIL1.Polarity;  
00209      uint8_t HALT_PRT0;  
00211      uint8_t FAIL2.INPUT_EN;  
00212      uint8_t FAIL3.INPUT_EN;  
00213      uint8_t FAIL1.OCAP;  
00214      uint8_t FAIL2.Signal_Sel;  
00215      uint8_t FAIL3.Signal_Sel;  
00216      uint8_t FAIL2.Polarity;  
00217      uint8_t FAIL3.Polarity;  
00218      uint8_t HALT_PRT1;  
00220      uint8_t CH0P_default_output;  
00221      uint8_t CH0N_default_output;  
00222      uint8_t CH1P_default_output;  
00223      uint8_t CH1N_default_output;  
00224      uint8_t CH2P_default_output;  
00225      uint8_t CH2N_default_output;  
00226      uint8_t CH3P_default_output;  
00227      uint8_t CH3N_default_output;  
00228      uint8_t CH4P_default_output;  
00229      uint8_t CH4N_default_output;  
00230      uint8_t CH5P_default_output;  
00231      uint8_t CH5N_default_output;  
00233      uint8_t CH0N_FAIL_EN;  
00234      uint8_t CH0P_FAIL_EN;  
00235      uint8_t CH1N_FAIL_EN;  
00236      uint8_t CH1P_FAIL_EN;  
00237      uint8_t CH2N_FAIL_EN;  
00238      uint8_t CH2P_FAIL_EN;  
00239      uint8_t CH3N_FAIL_EN;  
00240      uint8_t CH3P_FAIL_EN;  
00241      uint8_t CH4N_FAIL_EN;  
00242      uint8_t CH4P_FAIL_EN;  
00243      uint8_t CH5N_FAIL_EN;  
00244      uint8_t CH5P_FAIL_EN;  
00246      uint8_t T0_Update0.INT_EN;  
00247      uint8_t T1_Update0.INT_EN;  
00249      uint8_t TMR0_Match0.INT_EN;  
00250      uint8_t TMR1_Match0.INT_EN;  
00251      uint8_t TMR2_Match0.INT_EN;  
00252      uint8_t TMR3_Match0.INT_EN;  
00254      uint8_t TH00_Match0.INT_EN;  
00255      uint8_t TH01_Match0.INT_EN;  
00256      uint8_t TH10_Match0.INT_EN;  
00257      uint8_t TH11_Match0.INT_EN;  
00258      uint8_t TH20_Match0.INT_EN;  
00259      uint8_t TH21_Match0.INT_EN;  
00261      uint8_t Update0.INT_EN;  
00263      uint8_t T0_Update1.INT_EN;  
00264      uint8_t T1_Update1.INT_EN;
```

```

00266     uint8_t TMR0_Match1.INT_EN;
00267     uint8_t TMR1_Match1.INT_EN;
00268     uint8_t TMR2_Match1.INT_EN;
00269     uint8_t TMR3_Match1.INT_EN;
00271     uint8_t TH00_Match1.INT_EN;
00272     uint8_t TH01_Match1.INT_EN;
00273     uint8_t TH10_Match1.INT_EN;
00274     uint8_t TH11_Match1.INT_EN;
00275     uint8_t TH20_Match1.INT_EN;
00276     uint8_t TH21_Match1.INT_EN;
00278     uint8_t Update1.INT_EN;
00280     uint8_t FAIL0.INT_EN; /* FAIL0事件中断使能 */
00281     uint8_t FAIL1.INT_EN; /* FAIL1事件中断使能 */
00282     uint8_t FAIL2.INT_EN; /* FAIL0事件中断使能 */
00283     uint8_t FAIL3.INT_EN; /* FAIL1事件中断使能 */
00284
00285     uint8_t IO_CMP_FLT_CLKDIV;
00287     uint8_t ZCS0.INT_EN;
00288     uint8_t ZCS1.INT_EN;
00290     uint8_t IO0.PPE;
00291     uint8_t IO1.PPE;
00292     uint8_t IO2.PPE;
00293     uint8_t IO3.PPE;
00294     uint8_t IO4.PPE;
00295     uint8_t IO5.PPE;
00297     uint8_t TR1.T1.RE1;
00298     uint8_t TR1.T0.RE1;
00299     uint8_t TR0.T1.RE1;
00300     uint8_t TR0.T0.RE1;
00301     uint8_t TMR3.RE1;
00302     uint8_t TMR2.RE1;
00303     uint8_t TMR1.RE1;
00304     uint8_t TMR0.RE1;
00306     uint8_t TR1.T1.RE0;
00307     uint8_t TR1.T0.RE0;
00308     uint8_t TR0.T1.RE0;
00309     uint8_t TR0.T0.RE0;
00310     uint8_t TMR3.RE0;
00311     uint8_t TMR2.RE0;
00312     uint8_t TMR1.RE0;
00313     uint8_t TMR0.RE0;
00315     uint16_t STT_HYST;
00316     uint16_t ZCS_DELAY;
00318     uint32_t AUEN;
00319 } MCPWM_InitTypeDef;
00320
00321 /* MCPWM输出 对齐方式 */
00322 #define MCPWM0_CENTRAL_PWM_MODE 0
00323 #define MCPWM0_EDGE_PWM_MODE 1
00325 #define MCPWM0_HIGH_LEVEL 1
00326 #define MCPWM0_LOW_LEVEL 0
00328 #define MCPWM0_HIGH_LEVEL_ACTIVE 0
00329 #define MCPWM0_LOW_LEVEL_ACTIVE 1
00331 #define MCPWM0_FAIL_SEL_CMP 1
00332 #define MCPWM0_FAIL_SEL_IO 0
00334 /* MCPWM0_AUEN MCPWM 自动更新使能寄存器 */
00335 #define MCPWM0_TH00_AUPDATE BIT0
00336 #define MCPWM0_TH01_AUPDATE BIT1
00337 #define MCPWM0_TH10_AUPDATE BIT2
00338 #define MCPWM0_TH11_AUPDATE BIT3
00339 #define MCPWM0_TH20_AUPDATE BIT4
00340 #define MCPWM0_TH21_AUPDATE BIT5
00341 #define MCPWM0_TMR0_AUPDATE BIT8
00342 #define MCPWM0_TMR1_AUPDATE BIT9
00343 #define MCPWM0_TH0_AUPDATE BIT10
00344 #define MCPWM0_CNT0_AUPDAT BIT11
00346 #define MCPWM0_TH30_AUPDATE BIT16
00347 #define MCPWM0_TH31_AUPDATE BIT17
00348 #define MCPWM0_TH40_AUPDATE BIT18
00349 #define MCPWM0_TH41_AUPDATE BIT19
00350 #define MCPWM0_TH50_AUPDATE BIT20
00351 #define MCPWM0_TH51_AUPDATE BIT20
00352 #define MCPWM0_TMR2_AUPDATE BIT24
00353 #define MCPWM0_TMR3_AUPDATE BIT25
00354 #define MCPWM0_TH1_AUPDATE BIT26
00355 #define MCPWM0_CNT1_AUPDAT BIT27
00356 #define MCPWM0_ALL_AUPDAT 0xffffffff
00358 #define MCPWM0CNT0_T0_IRQ_IF BIT0
00359 #define MCPWM0CNT0_T1_IRQ_IF BIT1
00360 #define MCPWM0CNT0_TH00_IRQ_IF BIT2
00361 #define MCPWM0CNT0_TH01_IRQ_IF BIT3
00362 #define MCPWM0CNT0_TH10_IRQ_IF BIT4
00363 #define MCPWM0CNT0_TH11_IRQ_IF BIT5
00364 #define MCPWM0CNT0_TH20_IRQ_IF BIT6
00365 #define MCPWM0CNT0_TH21_IRQ_IF BIT7
00367 #define MCPWM0CNT0_TMR0_IRQ_IF BIT10
00368 #define MCPWM0CNT0_TMR1_IRQ_IF BIT11

```



```

00369 #define MCPWM0CNT0.TMR2_IRQ_IF BIT12
00370 #define MCPWM0CNT0.TMR3_IRQ_IF BIT13
00371 #define MCPWM0CNT0.UP_IRQ_IF BIT14
00373 #define MCPWM0CNT1.T0_IRQ_IF BIT0
00374 #define MCPWM0CNT1.T1_IRQ_IF BIT1
00375 #define MCPWM0CNT1.TH30_IRQ_IF BIT2
00376 #define MCPWM0CNT1.TH31_IRQ_IF BIT3
00377 #define MCPWM0CNT1.TH40_IRQ_IF BIT4
00378 #define MCPWM0CNT1.TH41_IRQ_IF BIT5
00379 #define MCPWM0CNT1.TH50_IRQ_IF BIT6
00380 #define MCPWM0CNT1.TH51_IRQ_IF BIT7
00382 #define MCPWM0CNT1.TMR2_IRQ_IF BIT12
00383 #define MCPWM0CNT1.TMR3_IRQ_IF BIT13
00384 #define MCPWM0CNT1.UP_IRQ_IF BIT14
00386 #define MCPWM0.FAIL0_IRQ_IF BIT19
00387 #define MCPWM0.FAIL1_IRQ_IF BIT20
00388 #define MCPWM0.FAIL2_IRQ_IF BIT21
00389 #define MCPWM0.FAIL3_IRQ_IF BIT22
00390 #define MCPWM0.ZCS0_IRQ_IF BIT23
00391 #define MCPWM0.ZCS1_IRQ_IF BIT24
00393 // MCPWM_IF0 位定义
00394 #define MCPWM_IF0.UP_IF BIT14
00395 #define MCPWM_IF0.TMR3_IF BIT13
00396 #define MCPWM_IF0.TMR2_IF BIT12
00397 #define MCPWM_IF0.TMR1_IF BIT11
00398 #define MCPWM_IF0.TMR0_IF BIT10
00399 #define MCPWM_IF0.TH21_IF BIT7
00400 #define MCPWM_IF0.TH20_IF BIT6
00401 #define MCPWM_IF0.TH11_IF BIT5
00402 #define MCPWM_IF0.TH10_IF BIT4
00403 #define MCPWM_IF0.TH01_IF BIT3
00404 #define MCPWM_IF0.TH00_IF BIT2
00405 #define MCPWM_IF0.T1_IF BIT1
00406 #define MCPWM_IF0.T0_IF BIT0
00407
00408 // MCPWM_IF1 位定义
00409 #define MCPWM_IF1.UP_IF BIT14
00410 #define MCPWM_IF1.TMR3_IF BIT13
00411 #define MCPWM_IF1.TMR2_IF BIT12
00412 #define MCPWM_IF1.TMR1_IF BIT11
00413 #define MCPWM_IF1.TMR0_IF BIT10
00414 #define MCPWM_IF1.TH51_IF BIT7
00415 #define MCPWM_IF1.TH50_IF BIT6
00416 #define MCPWM_IF1.TH41_IF BIT5
00417 #define MCPWM_IF1.TH40_IF BIT4
00418 #define MCPWM_IF1.TH31_IF BIT3
00419 #define MCPWM_IF1.TH30_IF BIT2
00420 #define MCPWM_IF1.T1_IF BIT1
00421 #define MCPWM_IF1.T0_IF BIT0
00422
00423 // MCPWM{EIF 位定义
00424 #define MCPWM{EIF.ZCS1 BIT9
00425 #define MCPWM{EIF.ZCS0 BIT8
00426 #define MCPWM{EIF.FAIL3 BIT7
00427 #define MCPWM{EIF.FAIL2 BIT6
00428 #define MCPWM{EIF.FAIL1 BIT5
00429 #define MCPWM{EIF.FAIL0 BIT4
00430
00432 typedef enum
00433 {
00434     MCPWM0CNT0 = 0,
00435     MCPWM0CNT1 = 1,
00436 } MCPWM0CNTx;
00437
00438 void MCPWM_Init(MCPWM_InitTypeDef *MCPWM_InitStruct);
00439 void MCPWM_StructInit(MCPWM_InitTypeDef *MCPWM_InitStruct);
00440 void PWMOutputs(FuncState t.state);
00441 void MCPWM_ClearIRQFlag(MCPWM0CNTx CNTx, uint32_t INT_flag);
00442 uint16 MCPWM_GetIRQFlag(MCPWM0CNTx CNTx, uint32_t INT_flag);
00443 void MCPWM_Clk_Enable(u8 Clk.Enable);
00444 void MCPWM_DIV_Set(u8 ClockDiv);
00445 void MCPWM_CNTx_Enable(MCPWM0CNTx CNTx, FuncState state);
00446 void MCPWM_CH012_Init(MCPWM_InitTypeDef *MCPWM_InitStruct);
00447 void MCPWM_CH345_Init(MCPWM_InitTypeDef *MCPWM_InitStruct);
00448 void MCPWM_CH012_IESet(MCPWM_InitTypeDef *MCPWM_InitStruct);
00449 void MCPWM_CH345_IESet(MCPWM_InitTypeDef *MCPWM_InitStruct);
00450 void MCPWM_ModuleClockCmd(FuncState state);
00451
00452 #endif /*_CHANHOM_PWM_H */
00453
00454 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.31 include/lks32mc07x_nvr.h 文件参考

文件名称: lks32mc07x_nvr.h
文件标识: 无
内容摘要: NVR驱动头文件
其它说明: 无

#include "lks32mc07x_lib.h"
lks32mc07x_nvr.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



函数

- uint32_t Read_NVR (uint32_t adr)
 读取NVR 1KB大小的用户区函数
- uint32_t Read_Trim (uint32_t adr)
 读取芯片模块校正值函数

5.31.1 详细描述

文件名称: lks32mc07x_nvr.h
文件标识: 无
内容摘要: NVR驱动头文件
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x-nvr.h](#) 中定义.

5.31.2 函数说明

5.31.2.1 Read_NVR()

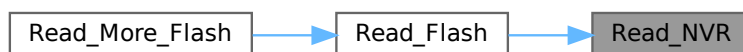
```
uint32_t Read_NVR (
    uint32_t adr )
```

读取NVR 1KB大小的用户区函数

示例代码：读取NVR用户地址0的数据

```
uint32_t LKS_NVRValue = 0;
LKS_NVRValue = Read_NVR(0x00); //范围0x000~0x400 1KB空间
```

这是这个函数的调用关系图:



5.31.2.2 Read_Trim()

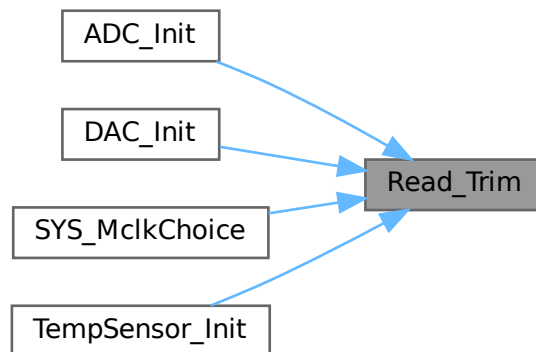
```
uint32_t Read_Trim (
    uint32_t adr )
```

读取芯片模块校正函数

示例代码：读取芯片ID

```
uint32_t LKS_ID1;
uint32_t LKS_ID2;
uint32_t LKS_ID3;
uint32_t LKS_ID4;
LKS_ID1 = Read_Trim(0x0000084C);
LKS_ID2 = Read_Trim(0x00000850);
LKS_ID3 = Read_Trim(0x00000854);
LKS_ID4 = Read_Trim(0x00000858);
```

这是这个函数的调用关系图:



5.32 lks32mc07x_nvr.h

[浏览该文件的文档.](#)

```

00001
00014 #ifndef __lks32mc07x_NVR_H
00015 #define __lks32mc07x_NVR_H
00016 #include "lks32mc07x_lib.h"
00017
00026 uint32_t Read_NVR(uint32_t adr);
00041 uint32_t Read_Trim(uint32_t adr);
00042 #endif
00043 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/
00044 /* -----END OF FILE----- */
  
```

5.33 include/lks32mc07x_opa.h 文件参考

文件名称: [lks32mc07x_opa.h](#)
 文件标识: 无
 内容摘要: OPA驱动头文件
 其它说明: 无

#include "lks32mc07x_lib.h"

lks32mc07x_opa.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct **OPA_InitTypeDef**
OPA初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

宏定义

- #define **PGA_GAIN_32** 0x00
反馈电阻320k:10k
- #define **PGA_GAIN_16** 0x01
反馈电阻160k:10k
- #define **PGA_GAIN_8** 0x02
反馈电阻80k:10k
- #define **PGA_GAIN_4** 0x03
反馈电阻40k:10k
- #define **PGA_IT_1** 0x00
opa偏置电流调节x1
- #define **PGA_IT_1P2** 0x01
opa偏置电流调节x1.2
- #define **PGA_IT_1P5** 0x02
opa偏置电流调节x1.5
- #define **PGA_IT_2** 0x03
opa偏置电流调节x2
- #define **PGA_IT_DISABLE** **PGA_IT_1**
设置为默认值

枚举

- enum **enumOPA** { **OPA0** = 0 , **OPA1** = 1 , **OPA2** = 2 , **OPA3** = 3 }

函数

- void **OPA_Init** (enumOPA opa, OPA_InitTypeDef *OPA_InitStruct)
函数名称: void *OPA_Init(enumOPA opa, OPA_InitTypeDef* OPA_InitStruct)*
- void **OPA_StructInit** (OPA_InitTypeDef *OPA_InitStruct)
函数名称: void *OPA_StructInit(OPA_InitTypeDef* OPA_InitStruct)*
- void **OPA_OUT** (enumOPA opa, FuncState state)
函数名称: void *OPA_OUT(enumOPA opa, FuncState state)*

5.33.1 详细描述

文件名称: **lks32mc07x_opa.h**
 文件标识: 无
 内容摘要: OPA驱动头文件
 其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_opa.h](#) 中定义.

5.33.2 宏定义说明

5.33.2.1 PGA_GAIN_16

```
#define PGA_GAIN_16 0x01
```

反馈电阻160k:10k

在文件 [lks32mc07x_opa.h](#) 第 54 行定义.

5.33.2.2 PGA_GAIN_32

```
#define PGA_GAIN_32 0x00
```

反馈电阻320k:10k

在文件 [lks32mc07x_opa.h](#) 第 53 行定义.

5.33.2.3 PGA_GAIN_4

```
#define PGA_GAIN_4 0x03
```

反馈电阻40k:10k

在文件 [lks32mc07x_opa.h](#) 第 56 行定义.

5.33.2.4 PGA_GAIN_8

```
#define PGA_GAIN_8 0x02
```

反馈电阻80k:10k

在文件 [lks32mc07x_opa.h](#) 第 55 行定义.

5.33.2.5 PGA_IT_1

```
#define PGA_IT_1 0x00
```

opa偏置电流调节x1

在文件 [lks32mc07x_opa.h](#) 第 58 行定义.

5.33.2.6 PGA.IT_1P2

```
#define PGA.IT_1P2 0x01
```

opa偏置电流调节x1.2

在文件 [lks32mc07x_opa.h](#) 第 59 行定义.

5.33.2.7 PGA.IT_1P5

```
#define PGA.IT_1P5 0x02
```

opa偏置电流调节x1.5

在文件 [lks32mc07x_opa.h](#) 第 60 行定义.

5.33.2.8 PGA.IT_2

```
#define PGA.IT_2 0x03
```

opa偏置电流调节x2

在文件 [lks32mc07x_opa.h](#) 第 61 行定义.

5.33.2.9 PGA.IT_DISABLE

```
#define PGA.IT_DISABLE PGA.IT_1
```

设置为默认值

在文件 [lks32mc07x_opa.h](#) 第 62 行定义.

5.33.3 枚举类型说明

5.33.3.1 enumOPA

```
enum enumOPA
```

枚举值

OPA0	OPA0模块
OPA1	OPA1模块
OPA2	OPA2模块
OPA3	OPA3模块

在文件 [lks32mc07x_opa.h](#) 第 20 行定义.

5.33.4 函数说明

5.33.4.1 OPA_Init()

```
void OPA_Init (
    enumOPA opa,
    OPA_InitTypeDef * OPA_InitStruct )
```

函数名称: void OPA_Init(enumOPA opa, OPA_InitTypeDef* OPA_InitStruct)

功能描述: OPA初始化函数

参见
被引用内容: 无

参数

输入参数:	enumOPA opa , OPA_InitTypeDef *OPA_InitStruct
输出参数:	无

返回
返回 值: 无

注解
其它说明: 无

警告
无

示例代码:

```
OPA_InitTypeDef OPA_InitStruct;
OPA_StructInit (&OPA_InitStruct);
OPA_InitStruct.OPA_IT = PGA_IT_DISABLE; //opa偏置电流调节禁止
OPA_InitStruct.OPA_CLEna = ENABLE; //使能
OPA_InitStruct.OPA_Gain = PGA_GAIN_4; //增益40k:10k
OPA_Init(OPA2 , &OPA_InitStruct);
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 lks32mc07x_opa.c 第 41 行定义.

5.33.4.2 OPA_OUT()

```
void OPA_OUT (
    enumOPA opa,
    FuncState state )
```

函数名称: void OPA_OUT(enumOPA opa, FuncState state)

功能描述: OPA外部输出，输出到外部P2.7引脚

参见

被引用内容: 无

参数

输入参数:	enumOPA opa, FuncState state
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
OPA_OUT(OPA0,ENABLE); //将OPA0输出通过外部P2.7引脚送出
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_opa.c](#) 第 162 行定义.

5.33.4.3 OPA_StructInit()

```
void OPA_StructInit (
    OPA_InitTypeDef * OPA_InitStruct )
```

函数名称: void [OPA_StructInit](#)(OPA_InitTypeDef* OPA_InitStruct)

功能描述: OPA结构体初始化

参见
被引用内容: 无

参数

输入参数:	OPA_InitTypeDef *OPA_InitStruct
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
OPA_InitTypeDef OPA_InitStruct;  
OPA\_StructInit (&OPA_InitStruct); //初始化结构体
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_opa.c](#) 第 137 行定义.

5.34 lks32mc07x_opa.h

[浏览该文件的文档.](#)

```
00001  
00014 #ifndef __LKS32MC07x_OPA_H  
00015 #define __LKS32MC07x_OPA_H  
00016  
00017 /* Includes -----*/  
00018 #include "lks32mc07x_lib.h"
```

```

00019
00020 typedef enum
00021 {
00022     OPA0 = 0,
00023     OPA1 = 1,
00024     OPA2 = 2,
00025     OPA3 = 3
00026 } enumOPA;
00027
00031 typedef struct
00032 {
00040     uint32_t OPA_IT;
00048     uint32_t OPA_Gain;
00049     uint32_t OPA_CLEna;
00050 } OPA_InitTypeDef;
00051
00052 /* -----PGA操作相关定义 ----- */
00053 #define PGA_GAIN_32    0x00
00054 #define PGA_GAIN_16    0x01
00055 #define PGA_GAIN_8     0x02
00056 #define PGA_GAIN_4     0x03
00058 #define PGA_IT_1       0x00
00059 #define PGA_IT_1P2     0x01
00060 #define PGA_IT_1P5     0x02
00061 #define PGA_IT_2       0x03
00062 #define PGA_IT_DISABLE PGA_IT_1
00064 void OPA_Init(enumOPA opa, OPA_InitTypeDef *OPA_InitStruct); /*opa初始化*/
00065 void OPA_StructInit(OPA_InitTypeDef *OPA_InitStruct); /*opa结构体初始化*/
00066 void OPA_OUT(enumOPA opa, FuncState state); /*opa输出到IO配置*/
00067
00068 #endif /*_lks32mc07x_OPA_H */
00069
00070 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.35 include/lks32mc07x_qep.h 文件参考

文件名称: [lks32mc07x_qep.h](#)

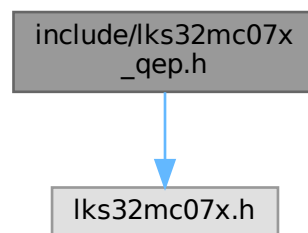
文件标识: 无

内容摘要: QEP驱动头文件

其它说明: 无

```
#include "lks32mc07x.h"
```

lks32mc07x_qep.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct **TIM_QETypeDef**
编码器寄存器结构体句柄
- struct **TIM_QEInitTypeDef**

宏定义

- #define **QEP0** ((TIM_QETypeDef *)QEP0_BASE)
QEP0结构体基地址定义
- #define **QEP1** ((TIM_QETypeDef *)QEP1_BASE)
QEP1结构体基地址定义
- #define **QEP_CLK_DIV1** 0x00
QEP计数器频率1分频
- #define **QEP_CLK_DIV2** 0x01
QEP计数器频率2分频
- #define **QEP_CLK_DIV4** 0x02
QEP计数器频率4分频
- #define **QEP_CLK_DIV8** 0x03
QEP计数器频率8分频
- #define **QEP_CLK_DIV16** 0x04
QEP计数器频率16分频
- #define **QEP_CLK_DIV32** 0x05
QEP计数器频率32分频
- #define **QEP_CLK_DIV64** 0x06
QEP计数器频率64分频
- #define **QEP_CLK_DIV128** 0x07
QEP计数器频率128分频
- #define **QEP_IRQEna_None** 0
编码器中断使能配置定义
- #define **QEP_IRQEna_UnderFlow** BIT0
编码器中断使能配置定义
- #define **QEP_IRQEna_OverFlow** BIT1
编码器中断使能配置定义
- #define **QEP_IF_UNDERFLOW** BIT0
编码器中断标志定义
- #define **QEP_IF_OVERFLOW** BIT1
编码器中断标志定义
- #define **QEP_Mode_T1** ((uint32_t)0x00) /*counting on T1*/
- #define **QEP_Mode_T1_T2** ((uint32_t)0x01) /*counting on T1&T2*/
- #define **QEP_Mode_CCWSIGN_UpDown** ((uint32_t)0x02) /*CCW+SIGN 符号加脉冲信号计数模式, 若需要计数信号的上下沿, 则QEP_FE.CNT.EN置1*/
- #define **QEP_Mode_CCWCW_UpDown** ((uint32_t)0x03) /*CCW+CW 符号加脉冲信号计数模式, 若需要计数信号的上下沿, 则QEP_FE.CNT.EN置1*/

函数

- void **TIM_QEPInit** (TIM_QEPTTypeDef *QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct)
函数名称: void TIM_QEPInit(TIM_QEPTTypeDef *QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct)
- void **TIM_QEPStructInit** (TIM_QEPInitTypeDef *TIM_QEPInitStruct)
函数名称: void TIM_QEPStructInit(TIM_QEPInitTypeDef *TIM_QEPInitStruct)
- void **TIM_QEPCmd** (TIM_QEPTTypeDef *QEPx, FuncState state)
函数名称: void TIM_QEPCmd(TIM_QEPTTypeDef *QEPx, FuncState state)
- uint16_t **TIM_QEP_GetCount** (TIM_QEPTTypeDef *QEPx)
函数名称: uint16_t TIM_QEP_GetCount(TIM_QEPTTypeDef *QEPx)
- void **QEP_ClearIRQFlag** (TIM_QEPTTypeDef *QEPx, uint32_t tempFlag)
函数名称: void QEP_ClearIRQFlag(TIM_QEPTTypeDef *QEPx,uint32_t tempFlag)
- u16 **QEP_GetIRQFlag** (TIM_QEPTTypeDef *QEPx, u32 QEP_IF_x)
函数名称: u16 QEP_GetIRQFlag(TIM_QEPTTypeDef *QEPx, u32 QEP_IF_x)
- u16 **QEP_GetFlag** (TIM_QEPTTypeDef *QEPx, u32 QEP_IF_x)
函数名称: u16 QEP_GetFlag(TIM_QEPTTypeDef *QEPx, u32 QEP_IF_x)

5.35.1 详细描述

文件名称: lks32mc07x_qep.h
文件标识: 无
内容摘要: QEP驱动头文件
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	创建

修改日志:

在文件 lks32mc07x_qep.h 中定义.

5.35.2 宏定义说明

5.35.2.1 QEP0

#define QEP0 ((TIM_QEPTTypeDef *)QEP0_BASE)

QEP0结构体基地址定义

在文件 lks32mc07x_qep.h 第 36 行定义.

5.35.2.2 QEP1

```
#define QEP1 ((TIM_QEPTypeDef *)QEP1_BASE)
```

QEP1结构体基地址定义

在文件 [lks32mc07x_qep.h](#) 第 41 行定义.

5.35.2.3 QEP_CLK_DIV1

```
#define QEP_CLK_DIV1 0x00
```

QEP计数器频率1分频

在文件 [lks32mc07x_qep.h](#) 第 81 行定义.

5.35.2.4 QEP_CLK_DIV128

```
#define QEP_CLK_DIV128 0x07
```

QEP计数器频率128分频

在文件 [lks32mc07x_qep.h](#) 第 88 行定义.

5.35.2.5 QEP_CLK_DIV16

```
#define QEP_CLK_DIV16 0x04
```

QEP计数器频率16分频

在文件 [lks32mc07x_qep.h](#) 第 85 行定义.

5.35.2.6 QEP_CLK_DIV2

```
#define QEP_CLK_DIV2 0x01
```

QEP计数器频率2分频

在文件 [lks32mc07x_qep.h](#) 第 82 行定义.

5.35.2.7 QEP_CLK_DIV32

```
#define QEP_CLK_DIV32 0x05
```

QEP计数器频率32分频

在文件 [lks32mc07x_qep.h](#) 第 86 行定义.

5.35.2.8 QEP_CLK_DIV4

```
#define QEP_CLK_DIV4 0x02
```

QEP计数器频率4分频

在文件 [lks32mc07x_qep.h](#) 第 83 行定义.

5.35.2.9 QEP_CLK_DIV64

```
#define QEP_CLK_DIV64 0x06
```

QEP计数器频率64分频

在文件 [lks32mc07x_qep.h](#) 第 87 行定义.

5.35.2.10 QEP_CLK_DIV8

```
#define QEP_CLK_DIV8 0x03
```

QEP计数器频率8分频

在文件 [lks32mc07x_qep.h](#) 第 84 行定义.

5.35.2.11 QEP_IF_OVERFLOW

```
#define QEP_IF_OVERFLOW BIT1
```

编码器中断标志定义

Encoder上溢中断

在文件 [lks32mc07x_qep.h](#) 第 110 行定义.

5.35.2.12 QEP_IF_UNDERFLOW

```
#define QEP_IF_UNDERFLOW BIT0
```

编码器中断标志定义

Encoder下溢中断

在文件 [lks32mc07x_qep.h](#) 第 106 行定义.

5.35.2.13 QEP_IRQEna_None

```
#define QEP_IRQEna_None 0
```

编码器中断使能配置定义

无使能

在文件 [lks32mc07x_qep.h](#) 第 93 行定义.

5.35.2.14 QEP_IRQEna_OverFlow

```
#define QEP_IRQEna_OverFlow BIT1
```

编码器中断使能配置定义

使能上溢中断

在文件 [lks32mc07x_qep.h](#) 第 101 行定义.

5.35.2.15 QEP_IRQEna_UnderFlow

```
#define QEP_IRQEna_UnderFlow BIT0
```

编码器中断使能配置定义

使能下溢中断

在文件 [lks32mc07x_qep.h](#) 第 97 行定义.

5.35.2.16 QEP_Mode_CCWCW_UpDown

```
#define QEP_Mode_CCWCW_UpDown ((uint32_t)0x03) /*CCW+CW 符号加脉冲信号计数模式，若需要计数信号的上下沿，  
则QEP_FE.CNT.EN置1*/
```

在文件 [lks32mc07x_qep.h](#) 第 115 行定义.

5.35.2.17 QEP_Mode_CCWSIGN_UpDown

```
#define QEP_Mode_CCWSIGN_UpDown ((uint32_t)0x02) /*CCW+SIGN 符号加脉冲信号计数模式，若需要计数信号的上  
下沿，则QEP_FE.CNT.EN置1*/
```

在文件 [lks32mc07x_qep.h](#) 第 114 行定义.

5.35.2.18 QEP_Mode_T1

```
#define QEP_Mode_T1 ((uint32_t)0x00) /*counting on T1*/
```

在文件 [lks32mc07x_qep.h](#) 第 112 行定义.

5.35.2.19 QEP_Mode_T1_T2

```
#define QEP_Mode_T1_T2 ((uint32_t)0x01) /*counting on T1&T2*/
```

在文件 [lks32mc07x_qep.h](#) 第 113 行定义.

5.35.3 函数说明

5.35.3.1 QEP_ClearIRQFlag()

```
void QEP_ClearIRQFlag (  
    TIM_QEPTypeDef * QEPx,  
    uint32_t tempFlag )
```

函数名称: void [QEP_ClearIRQFlag\(TIM_QEPTypeDef *QEPx,uint32_t tempFlag\)](#)

功能描述: 清除QEP中断标志

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1		
	tempFlag:		
	QEP_IF_UNDERFLOW	Encoder下溢中断	
	QEP_IF_OVERFLOW	Encoder上溢中断	
输出参数:	无		

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (QEP_GetIRQFlag(QEP0,QEP_IF_OVERFLOW)) //获取编码器上溢中断标志
{
    QEP_ClearIRQFlag(QEP0,QEP_IF_OVERFLOW); //清除编码器上溢中断标志
}
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x-qep.c](#) 第 216 行定义.

5.35.3.2 QEP_GetFlag()

```
u16 QEP_GetFlag (
    TIM_QEPTypeDef * QEPx,
    u32 QEP_IF_x )
```

函数名称: u16 [QEP_GetFlag\(TIM_QEPTypeDef *QEPx, u32 QEP_IF_x\)](#)

功能描述: 获取QEP中断标志

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1	
	QEP_IF_x:	
	QEP_IF_UNDERFLOW QEP_IF_OVERFLOW	Encoder下溢中断 Encoder上溢中断
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

不需要对应中断使能，也可以正常读取中断标志位

示例代码:

```
if(QEP_GetFlag(QEP0,QEP_IF_OVERFLOW)) //获取上溢中断标志
{
}
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 283 行定义.

5.35.3.3 QEP_GetIRQFlag()

```
u16 QEP_GetIRQFlag (
    TIM_QEPTypedef * QEPx,
    u32 QEP_IF_x )
```

函数名称: `u16 QEP_GetIRQFlag(TIM_QEPTypedef *QEPx, u32 QEP_IF_x)`

功能描述: 获取QEP中断标志

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1	
	QEP_IF_x:	
	QEP_IF_UNDERFLOW QEP_IF_OVERFLOW	Encoder下溢中断 Encoder上溢中断
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

只有对应中断使能后，改为才能读取，如果对应中断未使能，读取结果一直为0

示例代码:

```
if (QEP_GetIRQFlag (QEP0, QEP_IF_OVERFLOW)) //获取上溢中断标志
{
}
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 246 行定义.

5.35.3.4 TIM_QEP_GetCount()

```
uint16_t TIM_QEP_GetCount (
    TIM_QEPTypedef * QEPx )
```

函数名称: `uint16_t TIM_QEP_GetCount(TIM_QEPTypedef *QEPx)`

功能描述: 编码器计数值

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
uint32_t QEP0.Value = 0;  
QEP0.Value = TIM_QEP_GetCount(QEP0); //获取编码器0计数值
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 185 行定义.

5.35.3.5 TIM_QEPCmd()

```
void TIM_QEPCmd (  
    TIM_QEPTypedef * QEPx,  
    FuncState state )
```

函数名称: void [TIM_QEPCmd\(TIM_QEPTypedef *QEPx, FuncState state\)](#)

功能描述: 编码器QEPx使能和停止

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1 FuncState state: 使能与失能
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
TIM_QEPCmd(ECD0, ENABLE); // ECD0模块使能
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 153 行定义.

5.35.3.6 TIM_QEPInit()

```
void TIM_QEPInit (
    TIM_QEPTypedef * QEPx,
    TIM_QEPInitTypeDef * TIM_QEPInitStruct )
```

函数名称: void [TIM_QEPInit\(TIM_QEPTypedef *QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct\)](#)

功能描述: 编码器初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#);

参数

输入参数:	QEPx QEP0/QEP1 TIM_QEPInitTypeDef *TIM_QEPInitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
TIM_QEPInitTypeDef TIM_QEPInitStruct;
TIM_QEPStructInit (&TIM_QEPInitStruct);
TIM_QEPInitStruct.QEP_FE_CNT_EN = DISABLE;
TIM_QEPInitStruct.QEP_Mode = QEP_Mode_T1;
TIM_QEPInitStruct.QEP_ClockDiv = QEP_CLK_DIV1;
TIM_QEPInitStruct.QEP_Filter = 0;
TIM_QEPInitStruct.QEP_TH = 65535;
TIM_QEPInitStruct.QEP_ZEC = ENABLE;
TIM_QEPInitStruct.QEP_ZLC = DISABLE;
TIM_QEPInitStruct.QEP_ZNC = DISABLE;
TIM_QEPInitStruct.QEP_ZPC = ENABLE;
TIM_QEPInitStruct.QEP_IRQEna = DISABLE;
TIM_QEPInit (QEP0, &TIM_QEPInitStruct);
TIM_QEPCmd (QEP0, ENABLE);

// Timer结构体初始化
// 使能下降沿进行计数, ENABLE:使能 DISABLE:失能
// 编码器模式选择
// 编码器时钟分频
// 输入信号滤波系数
// 编码器计数周期
// Z 信号清零极性选择: 低电平/下降沿清零使能
// Z 信号清零极性选择: 高电平/下降沿清零使能
// Z 信号电平清零 QEP 计数器使能
// Z 信号边沿清零 QEP 计数器使能
// 中断使能
// QEP0模块使能
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 lks32mc07x_qep.c 第 63 行定义.

函数调用图:



5.35.3.7 TIM_QEPStructInit()

```
void TIM_QEPStructInit (
    TIM_QEPInitTypeDef * TIM_QEPInitStruct )
```

函数名称: void TIM_QEPStructInit(TIM_QEPInitTypeDef *TIM_QEPInitStruct)

功能描述: 编码器结构体初始化

参见

被引用内容: 无

参数

输入参数:	TIM_QEPInitTypeDef *TIM_QEPInitStruct
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
TIM_QEPInitTypeDef TIM_QEPInitStruct;
TIM_QEPStructInit (&TIM_QEPInitStruct); //编码器结构体初始化
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 119 行定义.

5.36 lks32mc07x_qep.h

[浏览该文件的文档.](#)

```

00001
00015 #ifndef __lks32mc07x_QEP_H
00016 #define __lks32mc07x_QEP_H
00017
00018 /* Includes -----*/
00019 #include "lks32mc07x.h"
00020
00024 typedef struct
00025 {
00026     __IO uint32_t CFG;
00027     __IO uint32_t TH;
00028     __IO uint32_t CNT;
00029     __IO uint32_t IE;
00030     __IO uint32_t IF;
00031 } TIM_QEPTypeDef;
00032
00036 #define QEP0 ((TIM_QEPTypeDef *)QEP0_BASE)
00041 #define QEP1 ((TIM_QEPTypeDef *)QEP1_BASE)
00042
00043 typedef struct
00044 {
00052     uint32_t QEP_Mode; // 编码器模式选择
00053     uint32_t QEP_FE_CNT_EN; // 使能下降沿计算 (上升沿始终计数), ENABLE使能 DISABLE失能
00054     uint32_t QEP_ZNC; // Z信号清零极性选择: 低电平/下降沿清零使能, ENABLE使能 DISABLE失能
00055     uint32_t QEP_ZPC; // Z信号清零极性选择: 高电平/上升沿清零使能, ENABLE使能 DISABLE失能
00056     uint32_t QEP_ZLC; // Z信号电平清零QEP计数器使能, ENABLE使能 DISABLE失能
00057     uint32_t QEP_ZEC; // Z信号边沿清零QEP计数器使能, ENABLE使能 DISABLE失能
00058     uint16_t QEP_TH; // QEP计数门限寄存器
00070     uint32_t QEP_ClockDiv; // 时钟分频
00071     uint32_t QEP_Filter; // 信号输入滤波
00078     uint32_t QEP_IRQEna; // 中断使能
00079 } TIM_QEPInitTypeDef;
00080
00081 #define QEP_CLK_DIV1 0x00
00082 #define QEP_CLK_DIV2 0x01
00083 #define QEP_CLK_DIV4 0x02
00084 #define QEP_CLK_DIV8 0x03
00085 #define QEP_CLK_DIV16 0x04
00086 #define QEP_CLK_DIV32 0x05
00087 #define QEP_CLK_DIV64 0x06
00088 #define QEP_CLK_DIV128 0x07
00093 #define QEP_IRQEna_None 0
00097 #define QEP_IRQEna_UnderFlow BIT0
00101 #define QEP_IRQEna_OverFlow BIT1
00106 #define QEP_IF_UNDERFLOW BIT0
00110 #define QEP_IF_OVERFLOW BIT1
00112 #define QEP_Mode_T1 ((uint32_t)0x00) /*counting on T1*/
00113 #define QEP_Mode_T1_T2 ((uint32_t)0x01) /*counting on T1&T2*/
00114 #define QEP_Mode_CCWSIGN_UpDown ((uint32_t)0x02) /*CCW+SIGN 符号加脉冲信号计数模式, 若需要计数信号的上下沿,
    则QEP_FE_CNT_EN置1*/
00115 #define QEP_Mode_CCWCW_UpDown ((uint32_t)0x03) /*CCW+CW 符号加脉冲信号计数模式, 若需要计数信号的上下沿,
    则QEP_FE_CNT_EN置1*/
00116
00117 /*QEP初始化*/
00118 void TIM_QEPInit(TIM_QEPTypeDef *QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct);
00119 void TIM_QEPStructInit(TIM_QEPInitTypeDef *TIM_QEPInitStruct);
00120 void TIM_QEPCmd(TIM_QEPTypeDef *QEPx, FuncState state);
00121 uint16_t TIM_QEP_GetCount(TIM_QEPTypeDef *QEPx);
00122 void QEP_ClearIRQFlag(TIM_QEPTypeDef *QEPx, uint32_t tempFlag);
00123 uint16 QEP_GetIRQFlag(TIM_QEPTypeDef *QEPx, uint32 QEP_IF_x);
00124 uint16 QEP_GetFlag(TIM_QEPTypeDef *QEPx, uint32 QEP_IF_x);
00125
00126 #endif

```

5.37 include/lks32mc07x_sif.h 文件参考

文件名称: **lks32mc07x_sif.h**

文件标识: 无

内容摘要: **SIF**驱动头文件

其它说明: 无


```
#include "lks32mc07x_lib.h"
```

lks32mc07x_sif.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [SIF_InitTypeDef](#)
- struct [SIF_RegTypeDef](#)

宏定义

- #define [SIF_IE.Enable](#) BIT0
SIF中断使能开关。默认值为0
- #define [SIF_DMA.Enable](#) BIT1
SIF DMA请求使能开关。默认值为0
- #define [SIF0](#) (([SIF_RegTypeDef](#) *) (SIF0_BASE))
- #define [SPI_IF.CLEAR](#) BIT4
SPI传输，完成事件。高电平有效，写1清除。

函数

- void [SIF_StructInit](#) ([SIF_InitTypeDef](#) *SIFInitStruct)
- void [SIF_Init](#) ([SIF_RegTypeDef](#) *SIFx, [SIF_InitTypeDef](#) *SIFInitStruct)
函数名称: [void SIF_Init\(SIF_RegTypeDef *SIFx, SIF_InitTypeDef *SIFInitStruct\)](#)
- void [SIF_Timebase.set](#) (u32 tosc)
函数名称: [void SIF_Timebase.set\(u32 tosc\)](#)
- void [SIF_TH.set](#) (u32 tsh, u32 tdh)
函数名称: [void SIF_Timebase.set\(u32 tosc\)](#)
- void [SIF_Senddata](#) (u8 Data)
函数名称: [void SIF_Senddata\(u8 Data\)](#)
- void [SIF_DMA.IsEnable](#) (u8 state)
函数名称: [void SIF_DMA.Enable\(void\)](#)
- void [SIF_Irq.IsEnable](#) (u8 state)
函数名称: [void SIF_DMA.Enable\(void\)](#)
- void [SIF_Clear.Irq](#) (void)
函数名称: [void SIF_Clear.Irq\(\)](#)

5.37.1 详细描述

文件名称: [lks32mc07x_sif.h](#)
文件标识: 无
内容摘要: SIF驱动头文件
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月12日	1.1	myx	修改

修改日志:

在文件 [lks32mc07x_sif.h](#) 中定义.

5.37.2 宏定义说明

5.37.2.1 SIF0

```
#define SIF0 ((SIF_RegTypeDef *) (SIF0_BASE))
```

在文件 [lks32mc07x_sif.h](#) 第 52 行定义.

5.37.2.2 SIF_DMA_Enable

```
#define SIF_DMA_Enable BIT1
```

SIF DMA请求使能开关。默认值为0

在文件 [lks32mc07x_sif.h](#) 第 50 行定义.

5.37.2.3 SIF_IE_Enable

```
#define SIF_IE_Enable BIT0
```

SIF中断使能开关。默认值为0

在文件 [lks32mc07x_sif.h](#) 第 49 行定义.

5.37.2.4 SPI_IF_CLEAR

```
#define SPI_IF_CLEAR BIT4
```

SPI传输，完成事件。高电平有效，写1清除。

在文件 [lks32mc07x_sif.h](#) 第 55 行定义.

5.37.3 函数说明

5.37.3.1 SIF_Clear_Irq()

```
void SIF_Clear_Irq (  
    void )
```

函数名称: void [SIF_Clear_Irq\(\)](#)

功能描述: 清除SIF模块标志位

参见

被引用内容:

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 224 行定义.

5.37.3.2 SIF_DMA_IsEnable()

```
void SIF_DMA_IsEnable (
    u8 state )
```

函数名称: void [SIF_DMA_Enable\(void\)](#)

功能描述: SIF DMA传输请求使能

参见

被引用内容:

参数

输入参数:	state : ENABLE/DIABLE
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 164 行定义.

5.37.3.3 SIF_Init()

```
void SIF_Init (
    SIF_RegTypeDef * SIFx,
    SIF_InitTypeDef * SIFInitStruct )
```

函数名称: void [SIF_Init](#)([SIF_RegTypeDef](#) *[SIFx](#), [SIF_InitTypeDef](#) *[SIFInitStruct](#))

功能描述: SIF初始化函数

参见
被引用内容:

参数

输入参数:	SIFx: SIF , SIF_InitTypeDef * SIF_InitStruct
输出参数:	无

返回
返 回 值: 无

注解
其它说明: 无

警告
无

示例代码:

```
SIF_InitTypeDef SIF_InitStruct;
SIF_StructInit (&SIF_InitStruct);

SIF_InitStruct.SIF_DONE = DISABLE;           // 传输完成后默认低电平
SIF_InitStruct.SIF_SYNC = DISABLE;           // 同步信号默认低电平
SIF_InitStruct.SIF_SYNC_PULSE = ENABLE;      // 同步信号有电平反转
SIF_InitStruct.SIF_DONE_VLD = DISABLE;       // 无结束信号
SIF_InitStruct.SIF_SYNC_VLD = ENABLE;        // 有同步信号
SIF_InitStruct.SIF_RATIO = DISABLE;          // 数据占空比 2: 1
SIF_InitStruct.SIF_MSB = ENABLE;             // 数据传送顺序 高字节在前
SIF_InitStruct.SIF_EN = ENABLE;              // SIF模块使能
SIF_InitStruct.SIF_TOSC = 46;                // 时基设置 47 * 333 = 15.67US
SIF_InitStruct.SIF_TSTH1 = 999;              // 同步时长 1000 * SIF.TOSC * 32
SIF_InitStruct.SIF_TDTH1 = 0;               // 结束信号时长 1ms
SIF_InitStruct.SIF_IRQ_IF = ENABLE;          // 有SIF中断标志位
SIF_InitStruct.SIF_DMA_EN = ENABLE;          // DMA传输使能
SIF_InitStruct.SIF_IRQ_EN = DISABLE;         // SIF中断关闭

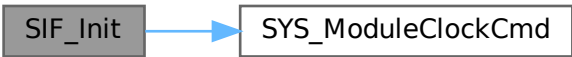
SIF_Init (SIF0, &SIF_InitStruct);
```

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 61 行定义.

函数调用图:



5.37.3.4 SIF_Irq_IsEnable()

```
void SIF_Irq_IsEnable (
    u8 state )
```

函数名称: void [SIF_DMA_Enable\(void\)](#)

功能描述: SIF DMA传输请求使能

参见

被引用内容:

参数

输入参数:	state : ENABLE/DIABLE
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 194 行定义.

5.37.3.5 SIF_Senddata()

```
void SIF_Senddata (
    u8 Data )
```

函数名称: void [SIF_Senddata\(u8 Data\)](#)

功能描述: 传输数据设置

参见

被引用内容:

参数

输入参数:	Data :传输的数据
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 141 行定义.

5.37.3.6 SIF_StructInit()

```
void SIF_StructInit (  
    SIF_InitTypeDef * SIFInitStruct )
```

在文件 [lks32mc07x_sif.c](#) 第 17 行定义.

5.37.3.7 SIF_TH.set()

```
void SIF_THset (  
    u32 tsh,  
    u32 tdh )
```

函数名称: void [SIF_Timebase_set\(u32 tosc\)](#)

功能描述: 设置信号通信方式

参见

被引用内容:

参数

输入参数:	tsh tsl : 同步信号周期 tdh tdl : 结束信号周期
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 117 行定义.

5.37.3.8 SIF_Timebase_set()

```
void SIF_Timebase_set (
    u32 tosc )
```

函数名称: void [SIF_Timebase_set\(u32 tosc\)](#)

功能描述: 设置SIF传输时基

参见

被引用内容:

参数

输入参数:	tosc : 时基系数
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 93 行定义.

5.38 lks32mc07x_sif.h

[浏览该文件的文档.](#)

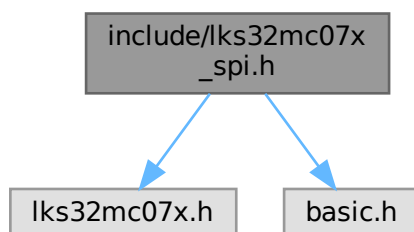
```
00001
00014 #ifndef _CONNIE_SIF_H_
00015 #define _CONNIE_SIF_H_
00016
00017 #include "lks32mc07x_lib.h"
00018
00019 typedef struct
00020 {
00021     uint8_t SIF_DONE;          /* 传输完成后默认电平 1: 高电平 0: 低电平 */
00022     uint8_t SIF_SYNC;          /* 同步信号默认电平 1: 高电平 0: 低电平 */
00023     uint8_t SIF_SYNC_PULSE;    /* 同步信号是否有电平反转 1: 有 0: 无 */
00024     uint8_t SIF_DONE_VLD;      /* 有无结束信号 1: 有 0: 无 */
00025     uint8_t SIF_SYNC_VLD;      /* 有无同步信号 1: 有 0: 无 */
00026     uint8_t SIF_RATIO;         /* 数据占空比 1: 3: 1 0: 2: 1 */
00027     uint8_t SIF_MSB;           /* 数据传送顺序 1: 高字节在前 0: 低字节在前 */
00028     uint8_t SIF_EN;            /* SIF模块使能, 高有效 */
00029     uint16_t SIF_TOSC;          /* 时基设置 SIF.TOSC+1 333ns为单位 最大 4096 * 333 = 1364US */
00030     uint16_t SIF_TSTH1;        /* 同步时长 (SIF.TSTH1 + 1) * SIF.TOSC 最大1024 * SIF.TOSC * 32 */
00031     uint8_t SIF_TDTH1;         /* 结束信号时长 (SIF.TDTH1 + 1) MS */
00032     uint8_t SIF_IRQ_IF;        /* SIF中断标志位 1: 有 0: 无 */
00033     uint8_t SIF_DMA_EN;        /* DMA 传输使能 高有效 */
00034     uint8_t SIF_IRQ_EN;        /* SIF中断使能 高有效 */
00035 } SIF_InitTypeDef;
00036
00037
00038 typedef struct
00039 {
00040     __IO uint32_t SIFx_CFG;
00041     __IO uint32_t SIFx_TOSC;
00042     __IO uint32_t SIFx_TSTH1;
00043     __IO uint32_t SIFx_TDTH1;
00044     __IO uint32_t SIFx_IRQ;
00045 } SIF_RegTypeDef;
00046
00047 /*中断使能定义*/
00048 #define SIF_IE_Enable BIT0
00049 #define SIF_DMA_Enable BIT1
00050
00051 #define SIF0 ((SIF_RegTypeDef *) (SIF0_BASE))
00052
00053 /*中断标志定义*/
00054 #define SIF_IRQ_Clear BIT4
00055
00056 void SIF_StructInit(SIF_InitTypeDef *SIFInitStruct);
00057 void SIF_Init(SIF_RegTypeDef *SIFx, SIF_InitTypeDef *SIFInitStruct);
00058 void SIF_Timebase_set(u32 tosc);
00059 void SIF_TH_set(u32 tsh, u32 tdh);
00060 void SIF_Senddata(u8 Data);
00061 void SIF_DMA_IsEnable(u8 state);
00062 void SIF_Irq_IsEnable(u8 state);
00063 void SIF_Clear_Irq(void);
00064
00065 #endif
```

5.39 include/lks32mc07x_spi.h 文件参考

```
#include "lks32mc07x.h"
```

```
#include "basic.h"
```

lks32mc07x_spi.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [SPI_TypeDef](#)
SPI寄存器结构体句柄
- struct [SPI_InitTypeDef](#)

宏定义

- #define [SPI0](#) (([SPI_TypeDef](#) *)SPI0_BASE)
- #define [SPI_FIRSTSEND_LSB](#) 1
低位先传
- #define [SPI_FIRSTSEND_MSB](#) 0
高位先传
- #define [SPI_DMA_ENABLE](#) 0
使能DMA搬运
- #define [SPI_DMA_DISABLE](#) 1
禁止DMA搬运
- #define [SPI_IRQEna_Enable](#) BIT7
中断使能定义, SPI中断使能
- #define [SPI_IRQEna_TranDone](#) BIT6
中断使能定义, 传输完成中断
- #define [SPI_IRQEna_SSErr](#) BIT5

- 中断使能定义，传输异常事件中断
- `#define SPI_IRQEna_DataOver` BIT4
中断使能定义，传输溢出事件中断
- `#define SPI_IF_TranDone` BIT2
中断标志定义，传输完成中断标志位
- `#define SPI_IF_SSErr` BIT1
中断标志定义，传输异常中断标志位
- `#define SPI_IF_DataOver` BIT0
中断标志定义，传输溢出中断标志位

枚举

- `enum SPI_Mode` { `SPI_Master` = 0x01 , `SPI_Slave` = 0x00 }
- `enum SPI_Duplex` { `SPI_Full` = 0x0 , `SPI_Half_Tx` = 0x2 , `SPI_Half_Rx` = 0x3 }

函数

- `void SPI_Init (SPI_TypeDef *SPIx, SPI_InitTypeDef *SPI_InitStruct)`
函数名称: `void SPI_Init(SPI_TypeDef* SPIx, SPI_InitTypeDef* SPI_InitStruct)`
- `void SPI_StructInit (SPI_InitTypeDef *SPI_InitStruct)`
函数名称: `void SPI_StructInit(SPI_InitTypeDef* SPI_InitStruct)`
- `void SPI_SendData (SPI_TypeDef *SPIx, uint8_t n)`
函数名称: `void SPI_SendData(SPI_TypeDef *SPIx, uint8_t n)`
- `uint8_t SPI_ReadData (SPI_TypeDef *SPIx)`
函数名称: `uint8_t SPI_ReadData(SPI_TypeDef *SPIx)`
- `uint8_t SPI_GetIRQFlag (SPI_TypeDef *SPIx, u8 IRQ_Flag)`
函数名称: `uint8_t SPI_GetIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag)`
- `void SPI_ClearIRQFlag (SPI_TypeDef *SPIx, u8 IRQ_Flag)`
函数名称: `void SPI_ClearIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag)`

5.39.1 宏定义说明

5.39.1.1 SPI0

```
#define SPI0 ((SPI_TypeDef *)SPI0_BASE)
```

在文件 `lks32mc07x_spi.h` 第 59 行定义.

5.39.1.2 SPI_DMA_DISABLE

```
#define SPI_DMA_DISABLE 1
```

禁止DMA搬运

在文件 `lks32mc07x_spi.h` 第 84 行定义.

5.39.1.3 SPI_DMA_ENABLE

```
#define SPI_DMA_ENABLE 0
```

使能DMA搬运

在文件 [lks32mc07x_spi.h](#) 第 83 行定义.

5.39.1.4 SPI_FIRSTSEND_LSB

```
#define SPI_FIRSTSEND_LSB 1
```

低位先传

在文件 [lks32mc07x_spi.h](#) 第 80 行定义.

5.39.1.5 SPI_FIRSTSEND_MSB

```
#define SPI_FIRSTSEND_MSB 0
```

高位先传

在文件 [lks32mc07x_spi.h](#) 第 81 行定义.

5.39.1.6 SPI_IF_DataOver

```
#define SPI_IF_DataOver BIT0
```

中断标志定义，传输溢出中断标志位

在文件 [lks32mc07x_spi.h](#) 第 95 行定义.

5.39.1.7 SPI_IF_SSErr

```
#define SPI_IF_SSErr BIT1
```

中断标志定义，传输异常中断标志位

在文件 [lks32mc07x_spi.h](#) 第 94 行定义.

5.39.1.8 SPI_IF_TranDone

```
#define SPI_IF_TranDone BIT2
```

中断标志定义，传输完成中断标志位

在文件 [lks32mc07x_spi.h](#) 第 93 行定义.

5.39.1.9 SPI_IRQEna_DataOver

```
#define SPI_IRQEna_DataOver BIT4
```

中断使能定义，传输溢出事件中断

在文件 [lks32mc07x_spi.h](#) 第 90 行定义.

5.39.1.10 SPI_IRQEna_Enable

```
#define SPI_IRQEna_Enable BIT7
```

中断使能定义，SPI中断使能

在文件 [lks32mc07x_spi.h](#) 第 87 行定义.

5.39.1.11 SPI_IRQEna_SSErr

```
#define SPI_IRQEna_SSErr BIT5
```

中断使能定义，传输异常事件中断

在文件 [lks32mc07x_spi.h](#) 第 89 行定义.

5.39.1.12 SPI_IRQEna-TranDone

```
#define SPI_IRQEna-TranDone BIT6
```

中断使能定义，传输完成中断

在文件 [lks32mc07x_spi.h](#) 第 88 行定义.

5.39.2 枚举类型说明

5.39.2.1 SPI_Duplex

```
enum SPI_Duplex
```

枚举值

SPI_Full	
SPI_Half_Tx	
SPI_Half_Rx	

在文件 [lks32mc07x_spi.h](#) 第 39 行定义.

5.39.2.2 SPI_Mode

enum SPI_Mode

枚举值

SPI_Master	
SPI_Slave	

在文件 lks32mc07x_spi.h 第 33 行定义.

5.39.3 函数说明

5.39.3.1 SPI_ClearIRQFlag()

```
void SPI_ClearIRQFlag (
    SPI_TypeDef * SPIx,
    u8 IRQ_Flag )
```

函数名称: void SPI_ClearIRQFlag(SPI_TypeDef *SPIx,u8 IRQ_Flag)

功能描述: 清除SPI中断标志位

参见

被引用内容: 无

参数

输入参数:	SPIx: SPI_IRQ_Flag ^{1/45}	
	SPI_IF_TranDone	传输完成中断标志位
	SPI_IF_SSErr	传输异常中断标志位
	SPI_IF_DataOver	传输溢出中断标志位
输出参数:	无	

返回

返回值: 无

注解

其它说明: 无

警告



示例代码:

```
if(SPI_GetIRQFlag(SPI,SPI_IF_TranDone))//获取传输完成中断标志位
{
    SPI_ClearIRQFlag(SPI,SPI_IF_TranDone); //清除传输完成中断标志位
}
```

Date	Version	Author	Description
2022年04月20日	1.0	Zhu Jie	创建

修改日志:

在文件 lks32mc07x_spi.c 第 237 行定义.

5.39.3.2 SPI_GetIRQFlag()

```
uint8_t SPI_GetIRQFlag (
    SPI_TypeDef * SPIx,
    u8 IRQ_Flag )
```

函数名称: uint8_t SPI_GetIRQFlag(SPI_TypeDef *SPIx,u8 IRQ_Flag)

功能描述: 获取SPI中断标志位

参见

被引用内容: 无

参数

输入参数:	SPIx: SPI_IRQ_Flagï¼4š	
	SPI_IF_TranDone	传输完成中断标志位
	SPI_IF_SSErr	传输异常中断标志位
	SPI_IF_DataOver	传输溢出中断标志位
输出参数:	无	

返回

返回 值: 无

注解

其它说明： 无

警告

只有对应中断使能后，改为才能读取，如果对应中断未使能，读取结果一直为0

示例代码:

```
if (SPI_GetIRQFlag (SPI, SPI_IF_TranDone)) //获取传输完成中断标志位
{
}
```

Date	Version	Author	Description
2022年04月20日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 196 行定义.

5.39.3.3 SPI_Init()

```
void SPI_Init (
    SPI_TypeDef * SPIx,
    SPI_InitTypeDef * SPI_InitStruct )
```

函数名称: void [SPI_Init\(SPI_TypeDef* SPIx, SPI_InitTypeDef* SPI_InitStruct\)](#)

功能描述: SPI初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	SPIx: SPI , SPI_InitTypeDef *SPI_InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明： 无

警告

⚠

示例代码:

```
SPI_InitTypeDef SPI_InitStructure;
SPI_StructInit(&SPI_InitStructure);           //SPI结构体初始化
SPI_InitStructure.Duplex = SPI_Full;          //双工模式设置
SPI_InitStructure.Mode = SPI_Master;         //master主机模式
SPI_InitStructure.EN = ENABLE;                //使能SPI模块
SPI_InitStructure.DMA = SPI_DMA_DISABLE;     // 选择SPI搬移方式，由MCU搬运数据到SPI
SPI_InitStructure.Trig = 0;                   //内部自动触发传输
SPI_InitStructure.DataOrder = SPI_FIRSTSEND_MSB; //该字节先发送 MSB
SPI_InitStructure.CPHA = 0;                   //第一个沿为发送数据时刻
SPI_InitStructure.CPOL = 0;                   //CLK默认高电平
SPI_InitStructure.BaudRate = 30;              //传输速度96MHZ/(2*(30+1))= 1.548MHZ
SPI_InitStructure.ByteLength = 0;             //发送零个字节
SPI_InitStructure.IRQEna = DISABLE;           //关闭SPI各中断
SPI_Init(SPI, &SPI_InitStructure);           //SPI初始化程序
```

Date	Version	Author	Description
2016年05月21日	1.0	cfwu	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 59 行定义.

函数调用图:



5.39.3.4 SPI_ReadData()

```
uint8_t SPI_ReadData (
    SPI_TypeDef * SPIx )
```

函数名称: `uint8_t SPI_ReadData(SPI_TypeDef *SPIx)`

功能描述: SPI读缓冲区数据

参见

被引用内容： 无

参数

输入参数:	SPIx: SPI
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

æ—

示例代码:

```
u8 SPI_RXValue = 0;
SPI_RXValue = SPI_ReadData(SPI); //SPI接收一字节数据
```

Date	Version	Author	Description
2016年05月21日	1.0	cfwu	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 165 行定义.

5.39.3.5 SPI_SendData()

```
void SPI_SendData (
    SPI_TypeDef * SPIx,
    uint8_t n )
```

函数名称: void [SPI_SendData\(SPI_TypeDef *SPIx, uint8_t n\)](#)

功能描述: SPI发送数据

参见

被引用内容: 无

参数

输入参数:	SPIx: SPI , n: 发送一字节数据
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

⚠

示例代码:

```
SPI_SendData(SPI,0x12); //SPI发送0x12一字节数据
```

Date	Version	Author	Description
2016年05月21日	1.0	cfwu	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 135 行定义.

5.39.3.6 SPI_StructInit()

```
void SPI_StructInit (
    SPI_InitTypeDef * SPI_InitStruct )
```

函数名称: void [SPI_StructInit\(SPI_InitTypeDef* SPI_InitStruct\)](#)

功能描述: SPI结构体初始化

参见

被引用内容: 无

参数

输入参数:	SPI_InitTypeDef *SPI_InitStruct
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

⚠

示例代码:

```
SPI_InitTypeDef SPI_InitStruct;
SPI_StructInit(&SPI_InitStruct);           //SPI结构体初始化
```

Date	Version	Author	Description
2016年05月21日	1.0	cfwu	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 97 行定义.

5.40 lks32mc07x_spi.h

[浏览该文件的文档.](#)

```
00001 /*****
00002  * 版权所有 (C)2015, LINKO SEMICONDUCTOR Co.,ltd
00003  *
00004  * 文件名称lks32mc07x8x_spi.h
00005  * 文件标识:
00006  * 内容摘要: SPI驱动头文件
00007  * 其它说明: 无
00008  * 当前版本: V 1.0
00009  * 作 者:
00010  * 完成日期:
00011  *
00012  * 修改记录1:
00013  * 修改日期:
00014  * 版 本 号: V 1.0
00015  * 修 改 人:
00016  * 修改内容: 创建
00017  *
00018  * 修改记录2:
00019  * 修改日期:
00020  * 版 本 号:
00021  * 修 改 人:
00022  * 修改内容:
00023  *
00024  *****/
00025
00026 #ifndef __lks32mc07x_SPI_H
00027 #define __lks32mc07x_SPI_H
00028
00029 /* Includes -----*/
00030 #include "lks32mc07x.h"
00031 #include "basic.h"
00032
```

```

00033 typedef enum
00034 {
00035     SPI_Master = 0x01, /* Master Mode */
00036     SPI_Slave = 0x00 /* Slave Mode */
00037 } SPI_Mode;
00038
00039 typedef enum
00040 {
00041     SPI_Full = 0x0, /* 开启全双工模式 */
00042     SPI_Half_Tx = 0x2, /* 开启半双工模式 仅发送 */
00043     SPI_Half_Rx = 0x3 /* 开启半双工模式 仅接受 */
00044 } SPI_Duplex;
00045
00049 typedef struct
00050 {
00051     __IO uint32_t CFG;
00052     __IO uint32_t IE;
00053     __IO uint32_t BAUD;
00054     __IO uint32_t TX_DATA;
00055     __IO uint32_t RX_DATA;
00056     __IO uint32_t SIZE;
00057 } SPI_TypeDef;
00058
00059 #define SPI0 ((SPI_TypeDef *)SPI0_BASE)
00060
00061 typedef struct
00062 {
00063     SPI_Duplex Duplex;
00064     uint8_t CS;
00065     SPI_Mode Mode;
00066     uint8_t CPHA;
00067     uint8_t CPOL;
00068     uint8_t ENDIAN;
00069     uint8_t EN;
00071     uint8_t IRQEna;
00072     uint8_t Trig;
00074     uint8_t TRANS_MODE;
00075     uint16_t BaudRate;
00077     uint8_t ByteLength;
00078 } SPI_InitTypeDef;
00079
00080 #define SPI_FIRSTSEND_LSB 1
00081 #define SPI_FIRSTSEND_MSB 0
00083 #define SPI_DMA_ENABLE 0
00084 #define SPI_DMA_DISABLE 1
00086 /*中断使能定义*/
00087 #define SPI_IRQEna_Enable BIT7
00088 #define SPI_IRQEna_TranDone BIT6
00089 #define SPI_IRQEna_SSErr BIT5
00090 #define SPI_IRQEna_DataOver BIT4
00092 /*中断标志定义*/
00093 #define SPI_IF_TranDone BIT2
00094 #define SPI_IF_SSErr BIT1
00095 #define SPI_IF_DataOver BIT0
00097 void SPI_Init(SPI_TypeDef *SPIx, SPI_InitTypeDef *SPI_InitStruct);
00098 void SPI_StructInit(SPI_InitTypeDef *SPI_InitStruct);
00099
00100 void SPI_SendData(SPI_TypeDef *SPIx, uint8_t n);
00101 uint8_t SPI_ReadData(SPI_TypeDef *SPIx);
00102
00103 uint8_t SPI_GetIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag);
00104 void SPI_ClearIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag);
00105
00106 #endif /*__lks32mc07x_SPI_H */
00107
00108 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.41 include/lks32mc07x_sys.h 文件参考

文件名称: [lks32mc07x_sys.h](#)

文件标识: 无

内容摘要: sys驱动头文件

其它说明: 无

```
#include "lks32mc07x_lib.h"
```

lks32mc07x_sys.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct **AFE_TypeDef**
模拟模块系统寄存器对象定义
- struct **SYS_TypeDef**
数字模块系统配置寄存器对象定义
- struct **SYS_InitTypeDef**
系统初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参
- struct **IWDG_TypeDef**
IWDG寄存器结构体句柄
- struct **IWDG_InitTypeDef**
IWDG结构体基地址定义

宏定义

- #define **AFE** ((AFE_TypeDef *) (SYS_BASE))
- #define **SYS** ((SYS_TypeDef *) (SYS_BASE + 0x80))
- #define **SYS_PLLSRSEL_RCH** 0x00 /*使用4MHz RC时钟*/
- #define **SYS_PLLSRSEL_CRYSTAL** 0x01 /*使用晶体时钟*/
- #define **CLK_SEL_HS** 0x00
- #define **CLK_SEL_PLL** 0x01
- #define **SYS_MCLK_96M_RC** 0x01
- #define **SYS_MCLK_48M_RC** 0x02
- #define **SYS_MCLK_24M_RC** 0x03
- #define **SYS_MCLK_12M_RC** 0x04
- #define **SYS_MCLK_32K_RC** 0x05
- #define **SYS_MCLK_96M_XTAL** 0x11
- #define **SYS_MCLK_48M_XTAL** 0x12
- #define **SYS_MCLK_24M_XTAL** 0x13
- #define **SYS_MCLK_12M_XTAL** 0x14
- #define **SYS_Clk_SPIDiv1** 0
- #define **SYS_Clk_SPIDiv2** 1
- #define **SYS_Clk_SPIDiv4** 2
- #define **SYS_Clk_SPIDiv8** 3
- #define **SYS_Clk_UARTDiv1** 0
- #define **SYS_Clk_UARTDiv2** 1

- #define SYS_Clk_UARTDiv4 2
- #define SYS_Clk_UARTDiv8 3
- #define SYS_Module_SPI0 BIT0
- #define SYS_Module_I2C0 BIT1
- #define SYS_Module_CMP BIT2
- #define SYS_Module_HALL0 BIT3
- #define SYS_Module_TIMER0 BIT4
- #define SYS_Module_TIMER1 BIT5
- #define SYS_Module_TIMER2 BIT6
- #define SYS_Module_TIMER3 BIT7
- #define SYS_Module_QEP0 BIT8
- #define SYS_Module_QEP1 BIT9
- #define SYS_Module_MCPWM0 BIT10
- #define SYS_Module_GPIO BIT11
- #define SYS_Module_UART0 BIT14
- #define SYS_Module_UART1 BIT15
- #define SYS_Module_CRC0 BIT16
- #define SYS_Module_DSP0 BIT17
- #define SYS_Module_CAN0 BIT19
- #define SYS_Module_SIF0 BIT20
- #define SYS_Module_CL0 BIT21
- #define SYS_AnalogModule_ADC BIT0
- #define SYS_AnalogModule_OPA0 BIT2
- #define SYS_AnalogModule_OPA1 BIT3
- #define SYS_AnalogModule_OPA2 BIT4
- #define SYS_AnalogModule_OPA3 BIT5
- #define SYS_AnalogModule_CMP0 BIT6
- #define SYS_AnalogModule_CMP1 BIT7
- #define SYS_AnalogModule_BGP BIT8
- #define SYS_AnalogModule_DAC1 BIT9
- #define SYS_AnalogModule_RCH BIT10
- #define SYS_AnalogModule_CMP2 BIT11
- #define SYS_AnalogModule_DAC0 BIT12
- #define SYS_AnalogModule_TMP BIT13
- #define SYS_AnalogModule_XTAL BIT14
- #define SYS_AnalogModule_PLL BIT15
- #define SYS_MCU_POWER_3v3 1
- #define SYS_MCU_POWER_5v0 0
- #define SYS_POWER_4V0 0x00 /*电源掉电监测阈值为4.0V*/
- #define SYS_POWER_3V75 0x01 /*电源掉电监测阈值为3.75V*/
- #define SYS_POWER_3V5 0x02 /*电源掉电监测阈值为3.5V*/
- #define SYS_POWER_3V25 0x03 /* 仅在芯片版本0123时有效 电源掉电监测阈值为3.25V*/
- #define SYS_POWER_2V5 0x03 /* 仅在芯片版本>=4时有效 电源掉电监测阈值为2.5V*/
- #define SECOND2IWDGCNT(x) ((u32)(32000 * (float)x + 0x800) & 0x1ff000)

看门狗时间计算

函数

- void **IWDG_Init** (IWDG_InitTypeDef *)
函数名称: *void IWDG_Init(IWDG_InitTypeDef *this)*
- void **IWDG_StructInit** (IWDG_InitTypeDef *)
函数名称: *void IWDG_StructInit(IWDG_InitTypeDef *this)*
- void **IWDG_DISABLE** (void)
函数名称: *void IWDG_DISABLE(void)*
- void **IWDG_ENABLE** (void)
函数名称: *void IWDG_ENABLE(void)*
- void **IWDG_Feed** (void)
函数名称: *void IWDG_Feed(void)*
- void **SYS_ModuleClockCmd** (uint32_t nModule, FuncState state)
函数名称: *void SYS_ModuleClockCmd(uint32_t nModule, FuncState state)*
- void **SYS_AnalogModuleClockCmd** (uint32_t nModule, FuncState state)
函数名称: *void SYS_AnalogModuleClockCmd(uint32_t nModule, FuncState state)*
- void **SYS_SoftResetModule** (uint32_t nModule)
函数名称: *void SYS_SoftResetModule(uint32_t nModule)*
- void **SYS_SoftReset_all** (uint32_t state)
函数名称: *void SYS_SoftReset_all(uint32_t state)*
- void **SYS_MclkChoice** (u8 SYS_MCLK_X)
函数名称: *void SYS_MclkChoice(u8 SYS_MCLK_X)*
- u32 **SYS_ReadMcuClk** (void)
函数名称: *u32 SYS_ReadMcuClk(void)*
- void **SYS_UartClkDiv** (u16 div)
函数名称: *void SYS_UartClkDiv(u16 div)*
- void **SYS_I2CClkDiv** (u16 div)
函数名称: *void SYS_I2CClkDiv(u16 div)*
- void **SYS_SPIClkDiv** (u16 div)
函数名称: *void SYS_SPIClkDiv(u16 div)*
- void **SYS_ClearRst** (void)
函数名称: *void SYS_ClearRst(void)*
- void **SYS_Init** (SYS_InitTypeDef *SYS_InitStruct)
函数名称: *void SYS_Init(SYS_InitTypeDef* SYS_InitStruct)*
- void **SYS_StructInit** (SYS_InitTypeDef *SYS_InitStruct)
函数名称: *void SYS_StructInit(SYS_InitTypeDef* SYS_InitStruct)*
- void **SYS_VolSelModule** (uint32_t Vol)
设置芯片工作电压 (同时打开电源掉电检测中断)
- void **SYS_VolSelModuleIRQ** (void)
关闭- void **SYS_VolSelModuleEnableIRQ** (uint32_t Vol)
使能掉电检测中断, 需要先配置芯片工作电压(函数: *SYS_VolSelModule*)

5.41.1 详细描述

文件名称: **lks32mc07x.sys.h**

文件标识: 无

内容摘要: **sys**驱动头文件

其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.h](#) 中定义.

5.41.2 宏定义说明

5.41.2.1 AFE

```
#define AFE ((AFE_TypeDef *) (SYS_BASE))
```

在文件 [lks32mc07x_sys.h](#) 第 57 行定义.

5.41.2.2 CLK_SEL_HS

```
#define CLK_SEL_HS 0x00
```

在文件 [lks32mc07x_sys.h](#) 第 109 行定义.

5.41.2.3 CLK_SEL_PLL

```
#define CLK_SEL_PLL 0x01
```

在文件 [lks32mc07x_sys.h](#) 第 110 行定义.

5.41.2.4 SECOND2IWDGCNT

```
#define SECOND2IWDGCNT(  
    x ) ((u32) (32000 * (float)x + 0x800) & 0x1ff000)
```

看门狗时间计算

在文件 [lks32mc07x_sys.h](#) 第 245 行定义.

5.41.2.5 SYS

```
#define SYS ((SYS_TypeDef *) (SYS_BASE + 0x80))
```

在文件 [lks32mc07x_sys.h](#) 第 85 行定义.

5.41.2.6 SYS_AnalogModule_ADC

```
#define SYS_AnalogModule_ADC BIT0
```

在文件 [lks32mc07x_sys.h](#) 第 157 行定义.

5.41.2.7 SYS_AnalogModule_BGP

```
#define SYS_AnalogModule_BGP BIT8
```

在文件 [lks32mc07x_sys.h](#) 第 164 行定义.

5.41.2.8 SYS_AnalogModule_CMP0

```
#define SYS_AnalogModule_CMP0 BIT6
```

在文件 [lks32mc07x_sys.h](#) 第 162 行定义.

5.41.2.9 SYS_AnalogModule_CMP1

```
#define SYS_AnalogModule_CMP1 BIT7
```

在文件 [lks32mc07x_sys.h](#) 第 163 行定义.

5.41.2.10 SYS_AnalogModule_CMP2

```
#define SYS_AnalogModule_CMP2 BIT11
```

在文件 [lks32mc07x_sys.h](#) 第 167 行定义.

5.41.2.11 SYS_AnalogModule_DAC0

```
#define SYS_AnalogModule_DAC0 BIT12
```

在文件 [lks32mc07x_sys.h](#) 第 168 行定义.

5.41.2.12 SYS_AnalogModule_DAC1

```
#define SYS_AnalogModule_DAC1 BIT9
```

在文件 [lks32mc07x_sys.h](#) 第 165 行定义.

5.41.2.13 SYS_AnalogModule_OPA0

```
#define SYS_AnalogModule_OPA0 BIT2
```

在文件 [lks32mc07x_sys.h](#) 第 158 行定义.

5.41.2.14 SYS_AnalogModule.OPA1

```
#define SYS_AnalogModule_OPA1 BIT3
```

在文件 [lks32mc07x_sys.h](#) 第 159 行定义.

5.41.2.15 SYS_AnalogModule.OPA2

```
#define SYS_AnalogModule_OPA2 BIT4
```

在文件 [lks32mc07x_sys.h](#) 第 160 行定义.

5.41.2.16 SYS_AnalogModule.OPA3

```
#define SYS_AnalogModule_OPA3 BIT5
```

在文件 [lks32mc07x_sys.h](#) 第 161 行定义.

5.41.2.17 SYS_AnalogModule.PLL

```
#define SYS_AnalogModule_PLL BIT15
```

在文件 [lks32mc07x_sys.h](#) 第 171 行定义.

5.41.2.18 SYS_AnalogModule.RCH

```
#define SYS_AnalogModule_RCH BIT10
```

在文件 [lks32mc07x_sys.h](#) 第 166 行定义.

5.41.2.19 SYS_AnalogModule.TMP

```
#define SYS_AnalogModule_TMP BIT13
```

在文件 [lks32mc07x_sys.h](#) 第 169 行定义.

5.41.2.20 SYS_AnalogModule.XTAL

```
#define SYS_AnalogModule_XTAL BIT14
```

在文件 [lks32mc07x_sys.h](#) 第 170 行定义.

5.41.2.21 SYS_Clk.SPIDiv1

```
#define SYS_Clk_SPIDiv1 0
```

在文件 [lks32mc07x_sys.h](#) 第 124 行定义.

5.41.2.22 SYS.Clk.SPIDiv2

```
#define SYS.Clk.SPIDiv2 1
```

在文件 [lks32mc07x.sys.h](#) 第 125 行定义.

5.41.2.23 SYS.Clk.SPIDiv4

```
#define SYS.Clk.SPIDiv4 2
```

在文件 [lks32mc07x.sys.h](#) 第 126 行定义.

5.41.2.24 SYS.Clk.SPIDiv8

```
#define SYS.Clk.SPIDiv8 3
```

在文件 [lks32mc07x.sys.h](#) 第 127 行定义.

5.41.2.25 SYS.Clk.UARTDiv1

```
#define SYS.Clk.UARTDiv1 0
```

在文件 [lks32mc07x.sys.h](#) 第 130 行定义.

5.41.2.26 SYS.Clk.UARTDiv2

```
#define SYS.Clk.UARTDiv2 1
```

在文件 [lks32mc07x.sys.h](#) 第 131 行定义.

5.41.2.27 SYS.Clk.UARTDiv4

```
#define SYS.Clk.UARTDiv4 2
```

在文件 [lks32mc07x.sys.h](#) 第 132 行定义.

5.41.2.28 SYS.Clk.UARTDiv8

```
#define SYS.Clk.UARTDiv8 3
```

在文件 [lks32mc07x.sys.h](#) 第 133 行定义.

5.41.2.29 SYS.MCLK_12M_RC

```
#define SYS.MCLK_12M_RC 0x04
```

在文件 [lks32mc07x.sys.h](#) 第 115 行定义.

5.41.2.30 SYS_MCLK_12M_XTAL

```
#define SYS_MCLK_12M_XTAL 0x14
```

在文件 [lks32mc07x_sys.h](#) 第 121 行定义.

5.41.2.31 SYS_MCLK_24M_RC

```
#define SYS_MCLK_24M_RC 0x03
```

在文件 [lks32mc07x_sys.h](#) 第 114 行定义.

5.41.2.32 SYS_MCLK_24M_XTAL

```
#define SYS_MCLK_24M_XTAL 0x13
```

在文件 [lks32mc07x_sys.h](#) 第 120 行定义.

5.41.2.33 SYS_MCLK_32K_RC

```
#define SYS_MCLK_32K_RC 0x05
```

在文件 [lks32mc07x_sys.h](#) 第 116 行定义.

5.41.2.34 SYS_MCLK_48M_RC

```
#define SYS_MCLK_48M_RC 0x02
```

在文件 [lks32mc07x_sys.h](#) 第 113 行定义.

5.41.2.35 SYS_MCLK_48M_XTAL

```
#define SYS_MCLK_48M_XTAL 0x12
```

在文件 [lks32mc07x_sys.h](#) 第 119 行定义.

5.41.2.36 SYS_MCLK_96M_RC

```
#define SYS_MCLK_96M_RC 0x01
```

在文件 [lks32mc07x_sys.h](#) 第 112 行定义.

5.41.2.37 SYS_MCLK_96M_XTAL

```
#define SYS_MCLK_96M_XTAL 0x11
```

在文件 [lks32mc07x_sys.h](#) 第 118 行定义.

5.41.2.38 SYS.MCU_POWER_3v3

```
#define SYS_MCU_POWER_3v3 1
```

在文件 [lks32mc07x.sys.h](#) 第 173 行定义.

5.41.2.39 SYS.MCU_POWER_5v0

```
#define SYS_MCU_POWER_5v0 0
```

在文件 [lks32mc07x.sys.h](#) 第 174 行定义.

5.41.2.40 SYS.Module_CAN0

```
#define SYS_Module_CAN0 BIT19
```

在文件 [lks32mc07x.sys.h](#) 第 152 行定义.

5.41.2.41 SYS.Module_CL0

```
#define SYS_Module_CL0 BIT21
```

在文件 [lks32mc07x.sys.h](#) 第 154 行定义.

5.41.2.42 SYS.Module_CMP

```
#define SYS_Module_CMP BIT2
```

在文件 [lks32mc07x.sys.h](#) 第 138 行定义.

5.41.2.43 SYS.Module_CRC0

```
#define SYS_Module_CRC0 BIT16
```

在文件 [lks32mc07x.sys.h](#) 第 150 行定义.

5.41.2.44 SYS.Module_DSP0

```
#define SYS_Module_DSP0 BIT17
```

在文件 [lks32mc07x.sys.h](#) 第 151 行定义.

5.41.2.45 SYS.Module_GPIO

```
#define SYS_Module_GPIO BIT11
```

在文件 [lks32mc07x.sys.h](#) 第 147 行定义.

5.41.2.46 SYS.Module_HALL0

```
#define SYS.Module_HALL0 BIT3
```

在文件 [lks32mc07x.sys.h](#) 第 139 行定义.

5.41.2.47 SYS.Module_I2C0

```
#define SYS.Module_I2C0 BIT1
```

在文件 [lks32mc07x.sys.h](#) 第 137 行定义.

5.41.2.48 SYS.Module_MCPWM0

```
#define SYS.Module_MCPWM0 BIT10
```

在文件 [lks32mc07x.sys.h](#) 第 146 行定义.

5.41.2.49 SYS.Module_QEP0

```
#define SYS.Module_QEP0 BIT8
```

在文件 [lks32mc07x.sys.h](#) 第 144 行定义.

5.41.2.50 SYS.Module_QEP1

```
#define SYS.Module_QEP1 BIT9
```

在文件 [lks32mc07x.sys.h](#) 第 145 行定义.

5.41.2.51 SYS.Module_SIF0

```
#define SYS.Module_SIF0 BIT20
```

在文件 [lks32mc07x.sys.h](#) 第 153 行定义.

5.41.2.52 SYS.Module_SPI0

```
#define SYS.Module_SPI0 BIT0
```

在文件 [lks32mc07x.sys.h](#) 第 136 行定义.

5.41.2.53 SYS.Module_TIMER0

```
#define SYS.Module_TIMER0 BIT4
```

在文件 [lks32mc07x.sys.h](#) 第 140 行定义.

5.41.2.54 SYS.Module.TIMER1

```
#define SYS.Module.TIMER1 BIT5
```

在文件 [lks32mc07x.sys.h](#) 第 141 行定义.

5.41.2.55 SYS.Module.TIMER2

```
#define SYS.Module.TIMER2 BIT6
```

在文件 [lks32mc07x.sys.h](#) 第 142 行定义.

5.41.2.56 SYS.Module.TIMER3

```
#define SYS.Module.TIMER3 BIT7
```

在文件 [lks32mc07x.sys.h](#) 第 143 行定义.

5.41.2.57 SYS.Module.UART0

```
#define SYS.Module.UART0 BIT14
```

在文件 [lks32mc07x.sys.h](#) 第 148 行定义.

5.41.2.58 SYS.Module.UART1

```
#define SYS.Module.UART1 BIT15
```

在文件 [lks32mc07x.sys.h](#) 第 149 行定义.

5.41.2.59 SYS.PLLSRSEL_CRYSTAL

```
#define SYS.PLLSRSEL_CRYSTAL 0x01 /*使用晶体时钟*/
```

在文件 [lks32mc07x.sys.h](#) 第 106 行定义.

5.41.2.60 SYS.PLLSRSEL_RCH

```
#define SYS.PLLSRSEL_RCH 0x00 /*使用4MHz RC时钟*/
```

在文件 [lks32mc07x.sys.h](#) 第 105 行定义.

5.41.2.61 SYS.POWER_2V5

```
#define SYS.POWER_2V5 0x03 /* 仅在芯片版本>=4时有效 电源掉电监测阈值为2.5V*/
```

在文件 [lks32mc07x.sys.h](#) 第 185 行定义.

5.41.2.62 SYS_POWER_3V25

```
#define SYS_POWER_3V25 0x03 /* 仅在芯片版本0123时有效 电源掉电监测阈值为3.25V*/
```

在文件 [lks32mc07x.sys.h](#) 第 184 行定义.

5.41.2.63 SYS_POWER_3V5

```
#define SYS_POWER_3V5 0x02 /*电源掉电监测阈值为3.5V*/
```

在文件 [lks32mc07x.sys.h](#) 第 183 行定义.

5.41.2.64 SYS_POWER_3V75

```
#define SYS_POWER_3V75 0x01 /*电源掉电监测阈值为3.75V*/
```

在文件 [lks32mc07x.sys.h](#) 第 182 行定义.

5.41.2.65 SYS_POWER_4V0

```
#define SYS_POWER_4V0 0x00 /*电源掉电监测阈值为4.0V*/
```

在文件 [lks32mc07x.sys.h](#) 第 181 行定义.

5.41.3 函数说明

5.41.3.1 IWDG_DISABLE()

```
void IWDG_DISABLE (
    void )
```

函数名称: void [IWDG_DISABLE\(void\)](#)

功能描述: 关闭看门狗模块

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
IWDG_DISABLE(); //关闭看门狗模块
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 692 行定义.

5.41.3.2 IWDG_ENABLE()

```
void IWDG_ENABLE (  
    void )
```

函数名称: void [IWDG_ENABLE\(void\)](#)

功能描述: 打开看门狗模块

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明： 无

警告

无

示例代码:

```
IWDG_ENABLE();    //打开看门狗模块
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 716 行定义.

5.41.3.3 IWDG_Feed()

```
void IWDG_Feed (
    void )
```

函数名称: [void IWDG_Feed\(void\)](#)

功能描述: 看门狗喂狗

参见

被引用内容： 无

参数

输入参数:	无
输出参数:	无

返回

返回值： 无

注解

其它说明： 无

警告

无

示例代码:

```
IWDG_Feed(); //看门狗喂狗
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 740 行定义.

5.41.3.4 IWDG_Init()

```
void IWDG_Init (
    IWDG_InitTypeDef * this )
```

函数名称: void [IWDG_Init\(IWDG_InitTypeDef *this\)](#)

功能描述: 看门狗初始化

参见

被引用内容: 无

参数

输入参数:	IWDG_InitTypeDef *this 看门狗配置结构体
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
IWDG_InitTypeDef IWDG_InitStructure;  
IWDG_StructInit(&IWDG_InitStructure);  
IWDG_InitStructure.WDG_EN = ENABLE;           //使能看门狗  
IWDG_InitStructure.RTH = SECOND2IWDGCNT(2.0); //设置看门狗  
IWDG_InitStructure.DWK_EN = DISABLE;          // 深度休眠定时唤醒关闭  
IWDG_InitStructure.WTH = 0;                   // 看门狗定时唤醒时间（21位计数器，但低12恒位0）  
IWDG_Init(&IWDG_InitStructure);
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 625 行定义.

5.41.3.5 IWDG_StructInit()

```
void IWDG_StructInit (  
    IWDG_InitTypeDef * this )
```

函数名称: void [IWDG_StructInit\(IWDG_InitTypeDef *this\)](#)

功能描述: 看门狗配置结构体初始化

参见

被引用内容: 无

参数

输入参数:	IWDG_InitTypeDef *this 看门狗配置结构体
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
IWDG_InitTypeDef IWDG_InitStruct;  
IWDG_StrutInit (&IWDG_InitStruct); //初始化结构体
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 664 行定义.

5.41.3.6 SYS_AnalogModuleClockCmd()

```
void SYS_AnalogModuleClockCmd (  
    uint32_t nModule,  
    FuncState state )
```

函数名称: void [SYS_AnalogModuleClockCmd\(uint32_t nModule, FuncState state\)](#)

功能描述: 模拟模块使能和停止

参见

被调用枚举: FuncState

参数

输入参数:	state: ENABLE使能 , DISABLE关闭 nModule: 参数可选:		
	宏定义	说明	
	SYS_AnalogModule_ADC	BIT0	ADC模块
	SYS_AnalogModule_OPA0	BIT2	OPA0模块
	SYS_AnalogModule_OPA1	BIT3	OPA1模块
	SYS_AnalogModule_OPA2	BIT4	OPA2模块
	SYS_AnalogModule_OPA3	BIT5	OPA3模块
	SYS_AnalogModule_CMP0	BIT6	CMP0模块
	SYS_AnalogModule_CMP1	BIT7	CMP1模块
	SYS_AnalogModule_BGP	BIT8	BGP模块
	SYS_AnalogModule_RCH	BIT10	RCH模块
	SYS_AnalogModule_DAC	BIT12	DAC模块
	SYS_AnalogModule_TMP	BIT13	温度传感器
	SYS_AnalogModule_XTAL	BIT14	晶振
	SYS_AnalogModule_PLL	BIT15	PLL模块
输出参数:	无		

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

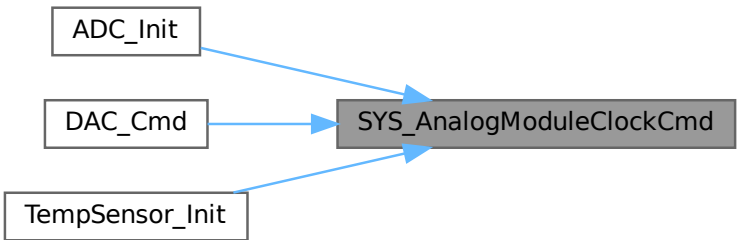
```
SYS_AnalogModuleClockCmd(SYS_AnalogModule_PLL,ENABLE); //使能PLL模块
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 236 行定义.

这是这个函数的调用关系图:



5.41.3.7 SYS_ClearRst()

```
void SYS_ClearRst (
    void )
```

函数名称: void [SYS.ClearRst\(void\)](#)

功能描述: SYS清除复位标志记录

参见

被调用函数： 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 请注意由于复位记录工作于低速时钟域, 清除执行完成需要一定时间, 不应清除后立即读记录状态

警告

无

示例代码:

```
u32 RSET_VALUE = 0;
SYS_ClearRst(); //清除复位标志记录
delay_ms(1);
RSET_VALUE = SYS_GetRstSource(); //读取复位源记录值
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 144 行定义.

5.41.3.8 SYS_I2CClkDiv()

```
void SYS_I2CClkDiv (
    u16 div )
```

函数名称: void [SYS_I2CClkDiv\(u16 div\)](#)

功能描述: I2C分频设置

参见

被调用枚举: 无

参数

输入参数:	div: 分频系数
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志

在文件 [lks32mc07x_sys.c](#) 第 567 行定义.

5.41.3.9 SYS.Init()

```
void SYS_Init (
    SYS_InitTypeDef * SYS_InitStruct )
```

函数名称: void [SYS.Init\(SYS_InitTypeDef* SYS_InitStruct\)](#)

功能描述: SYS模块初始化函数

参见

被调用函数: 无

参数

输入参数:	SYS_InitTypeDef* SYS_InitStruct
输出参数:	无

返回

返回值：无

注解

其它说明：一般不使用该函数，因为涉及模块较多，比较杂乱，对应里面的配置在各模块 初始化单独配置。

警告

无

示例代码:

```
SYS_InitTypeDef SYS_InitStruct;  
SYS_StructInit(&SYS_InitStruct);  
SYS_InitStruct.PLL_SrcSel = SYS_PLLSRSEL_RCH;    // 使用内部4MHz晶振作为时钟输入源  
SYS_Init(&SYS_InitStruct);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x-sys.c](#) 第 47 行定义.

5.41.3.10 SYS_MclkChoice()

```
void SYS_MclkChoice (  
    u8 SYS_MCLK_x )
```

函数名称: void [SYS_MclkChoice\(u8 SYS.MCLK_x\)](#)

功能描述: 主时钟选择

参见

被调用枚举：无

参数

输入参数:	nModule: 参数可选:		
	宏定义	原始值	说明
	SYS_MCLK_96M_RC 0x01		
	96MHz_PLL_RC		
	SYS_MCLK_48M_RC 0x02		
	48MHz_PLL_RC		
	SYS_MCLK_24M_RC 0x03		
	24MHz_PLL_RC		
	SYS_MCLK_12M_RC 0x04		
	12MHz_PLL_RC		
	SYS_MCLK_32K_RC 0x05		
	32kHz_LSI		
	SYS_MCLK_96M_XTAL		
	0x11 96MHz_PLL_XTAL		
	SYS_MCLK_48M_XTAL		
	0x12 48MHz_PLL_XTAL		
	SYS_MCLK_24M_XTAL		
	0x13 24MHz_PLL_XTAL		
	SYS_MCLK_12M_XTAL		
	0x14 12MHz_PLL_XTAL		
输出参数:	无		

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

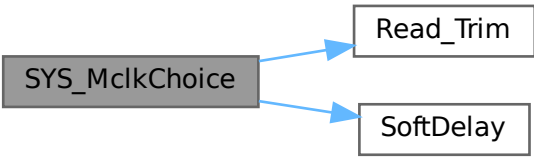
```
//软复位SPI模块，清除SPI寄存器配置，PC指针复位
SYS_MclkChoice (SYS_MCLK_96M_RC);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:增加延时函数，等待寄存器复位完毕与释放复位操作。

在文件 [lks32mc07x_sys.c](#) 第 391 行定义.

函数调用图:



5.41.3.11 SYS.ModuleClockCmd()

```
void SYS_ModuleClockCmd (
    uint32_t nModule,
    FuncState state )
```

函数名称: void [SYS.ModuleClockCmd\(uint32_t nModule, FuncState state\)](#)

功能描述: 数字模块时钟使能和停止

参见

被调用枚举: FuncState

参数

输入参数:	state: ENABLE使能 , DISABLE关闭 nModule: 参数可选:		
	宏定义	原始值	说明
	SYS.Module.I2C	BIT0	I2C模块
	SYS.Module.HALL	BIT1	HALL模块
	SYS.Module.TIMER	BIT2	UTIMER模块
	SYS.Module.MCPWM	BIT3	MCPWM模块
	SYS.Module.UART0	BIT4	UART0模块
	SYS.Module.UART1	BIT5	UART1模块
	SYS.Module.DSP	BIT6	DSP模块
	SYS.Module.CAN	BIT7	CAN模块
	SYS.Module.SPI	BIT8	SPI模块
输出参数:	无		

返回

返回值: 无

注解

其它说明： 无

警告

无

示例代码:

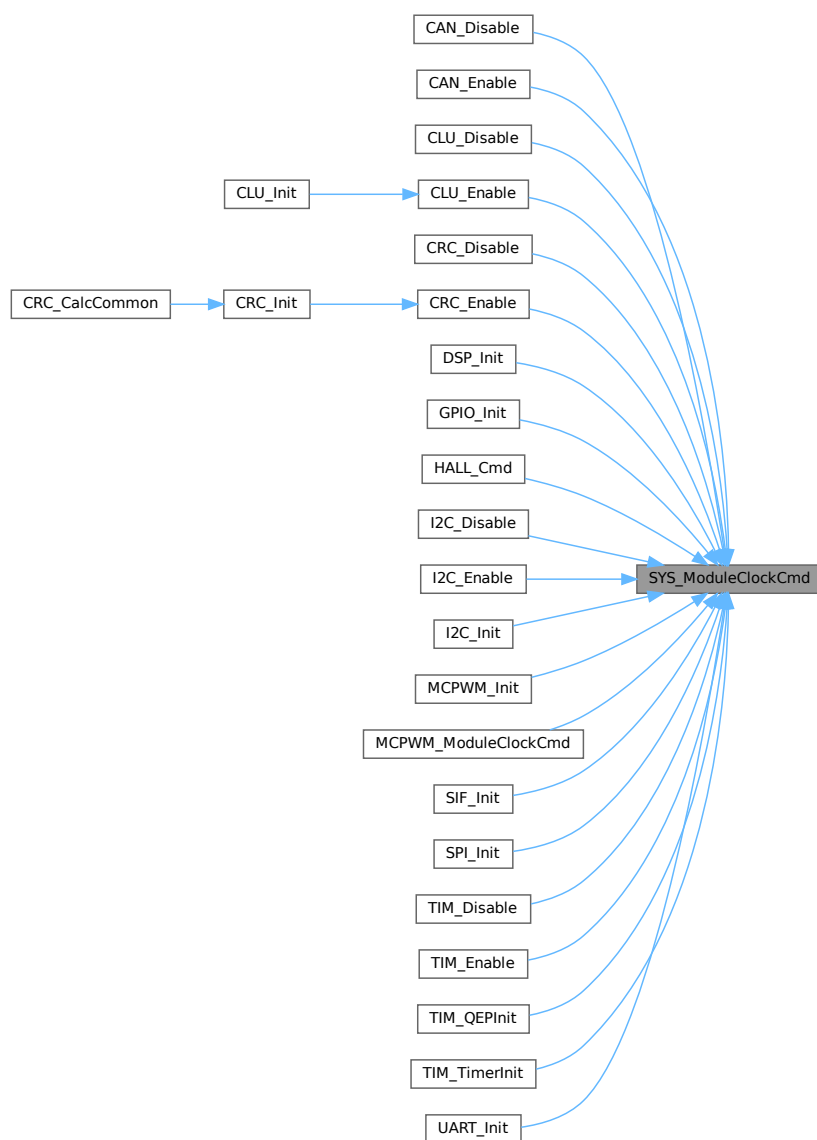
```
SYS_ModuleClockCmd(SYS_Module_SPI, ENABLE); //使能SPI时钟
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 184 行定义.

这是这个函数的调用关系图:



5.41.3.12 SYS_ReadMcuClk()

```
u32 SYS_ReadMcuClk (
    void )
```

函数名称: u32 [SYS_ReadMcuClk\(void\)](#)

功能描述: 获取当前时钟

参见

被调用枚举: 无

参数

输入参数:	
输出参数:	无

返回

返回 值: 时钟频率

注解

其它说明: 无

警告

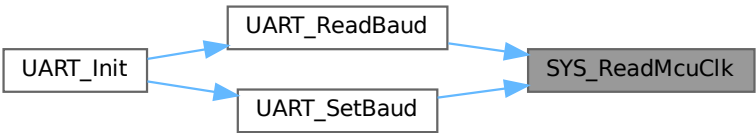
无

示例代码:

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:增加延时函数，等待寄存器复位完毕与释放复位操作。

在文件 [lks32mc07x.sys.c](#) 第 517 行定义.
这是这个函数的调用关系图:



5.41.3.13 SYS.SoftReset_all()

```
void SYS_SoftReset_all (
    uint32_t state )
```

函数名称: void [SYS.SoftReset_all\(uint32_t state\)](#)
功能描述: 数字模块软复位

参见

被调用枚举： 无

参数

输入参数:	state: ENABLE使能 , DISABLE关闭 nModule: 参数可选:
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
//软复位所有模块，清除相关外设寄存器配置，PC指针复位  
SYS_SoftResetModule(ENABLE);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:增加延时函数，等待寄存器复位完毕与释放复位操作。

在文件 [lks32mc07x_sys.c](#) 第 313 行定义.

5.41.3.14 SYS_SoftResetModule()

```
void SYS_SoftResetModule (  
    uint32_t nModule )
```

函数名称: void [SYS_SoftResetModule\(uint32_t nModule\)](#)

功能描述: 数字模块软复位

参见

被调用枚举： 无

参数

输入参数:	state: ENABLE使能, DISABLE关闭 nModule: 参数可选:		
	宏定义	原始值	说明
	SYS_Module_I2C	BIT0	I2C模块
	SYS_Module_HALL	BIT1	HALL模块
	SYS_Module_TIMER	BIT2	UTIMER模块
	SYS_Module_MCPWM	BIT3	MCPWM模块
	SYS_Module_UART0	BIT4	UART0模块
	SYS_Module_UART1	BIT5	UART1模块
	SYS_Module_DSP	BIT6	DSP模块
	SYS_Module_CAN	BIT7	CAN模块
	SYS_Module_SPI	BIT8	SPI模块
输出参数:	无		

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

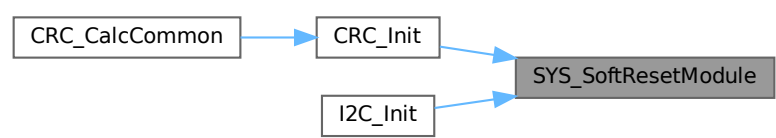
```
//软复位SPI模块,清除SPI寄存器配置,PC指针复位
SYS_SoftResetModule(SYS_Module_SPI);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:增加延时函数,等待寄存器复位完毕与释放复位操作。

在文件 lks32mc07x_sys.c 第 284 行定义.

这是这个函数的调用关系图:



5.41.3.15 SYS.SPIClkDiv()

```
void SYS_SPIClkDiv (
    u16 div )
```

函数名称: void **SYS.SPIClkDiv(u16 div)**

功能描述: SPI分频设置

参见
被调用枚举: 无

参数

输入参数:	div: 分频系数
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志

在文件 [lks32mc07x_sys.c](#) 第 593 行定义.

5.41.3.16 SYS.StructInit()

```
void SYS_StructInit (  
    SYS_InitTypeDef * SYS_InitStruct )
```

函数名称: void [SYS.StructInit\(SYS_InitTypeDef* SYS_InitStruct\)](#)

功能描述: **SYS**结构体初始化

参见

被调用函数: 无

参数

输入参数:	SYS_InitTypeDef* SYS_InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
SYS_InitTypeDef SYS_InitStruct;  
SYS_StructInit (&SYS_InitStruct);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 107 行定义.

5.41.3.17 SYS.UartClkDiv()

```
void SYS_UartClkDiv (
    u16 div )
```

函数名称: void [SYS.UartClkDiv\(u16 div\)](#)

功能描述: 串口分频设置

参见

被调用枚举: 无

参数

输入参数:	div: 分频系数
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志

在文件 [lks32mc07x-sys.c](#) 第 541 行定义.

5.41.3.18 SYS.VolSelModule()

```
void SYS_VolSelModule (
    uint32_t SYS_MCU_POWER_x )
```

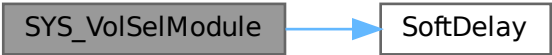
设置芯片工作电压（同时打开电源掉电检测中断）

参数

<i>SYS_MCU_POWER_x</i>	芯片供电电压 SYS.MCU.POWER_3v3 或 SYS.MCU.POWER_5v0
------------------------	--

在文件 [lks32mc07x-sys.c](#) 第 771 行定义.

函数调用图:



5.41.3.19 SYS.VolSelModuleEnableIRQ()

```
void SYS_VolSelModuleEnableIRQ (
    uint32_t Vol )
```

使能掉电检测中断，需要先配置芯片工作电压(函数: SYS.VolSelModule)

在文件 [lks32mc07x-sys.c](#) 第 832 行定义.

5.41.3.20 SYS.VolSelModuleIRQ()

```
void SYS_VolSelModuleIRQ (
    void )
```

关闭pwm输出并复位mcu（用于掉电检测中断）

在文件 [lks32mc07x-sys.c](#) 第 809 行定义.

5.42 lks32mc07x_sys.h

[浏览该文件的文档.](#)

```

00001
00015 #ifndef __lks32mc07x_SYS_H
00016 #define __lks32mc07x_SYS_H
00017 #include "lks32mc07x.lib.h"
00018
00023 typedef struct
00024 {
00025     __IO u32 ADC;    // 0x00)
00026     __IO u32 INFO;   // 0x04)
00027     __IO u32 DBG;    // 0x08)
00028
00029     __IO u32 null7;   // 0x0C)
00030     __IO u32 REG0;    // 0x10)
00031     __IO u32 REG1;    // 0x14)
00032     __IO u32 REG2;    // 0x18)
00033     __IO u32 REG3;    // 0x1C)
00034     __IO u32 REG4;    // 0x20)
00035     __IO u32 REG5;    // 0x24)
00036     __IO u32 REG6;    // 0x28)
00037     __IO u32 REG7;    // 0x2C)
00038     __IO u32 REG8;    // 0x30)
00039     __IO u32 REG9;    // 0x34)
00040     __IO u32 REGA;    // 0x38)
00041     __IO u32 REGB;    // 0x3C)
00042     __IO u32 REGC;    // 0x40)
00043     __IO u32 null11;   // 0x44)
00044     __IO u32 null12;   // 0x48)
00045     __IO u32 null13;   // 0x4C)
00046     __IO u32 null14;   // 0x50)
00047     __IO u32 null15;   // 0x54)
00048     __IO u32 null16;   // 0x58)
00049     __IO u32 DAC_CTRL; // 0x5C)
00050     __IO u32 DAC0;     // 0x60)
00051     __IO u32 DAC1;     // 0x64)
00052     __IO u32 DAC0_AMC; // 0x68)
00053     __IO u32 DAC0_DC;  // 0x6C)
00054     __IO u32 DAC1_AMC; // 0x70)
00055     __IO u32 DAC1_DC;  // 0x74)
00056 } AFE_TypeDef;
00057 #define AFE ((AFE_TypeDef *) (SYS_BASE))
00058
00062 typedef struct
00063 {
00064     __IO u32 CLK_CFG;    // 0x80)
00065     __IO u32 IO_CFG;     // 0x84)
00066     __IO u32 DBG_CFG;    // 0x88)
00067     __IO u32 null;       // 0x8c)
00068     __IO u32 CLK_DIV0;   // 0x90) I2C clock div
00069     __IO u32 CLK_DIV1;   // 0x94) 保留
00070     __IO u32 CLK_DIV2;   // 0x98) UART0/1 clock div
00071     __IO u32 CLK_FEN;    // 0x9C) peripheral clock enable
00072     __IO u32 TRIM;       // 0xA0)
00073     __IO u32 SFT_RST;    // 0xA4)
00074     __IO u32 WR_PROTECT; // 0xA8)
00075     __IO u32 MBIST;      // 0xC0)
00076     __IO u32 null7;      // 0xC4)
00077     __IO u32 null18;     // 0xC8)
00078     __IO u32 null19;     // 0xCC)
00079     __IO u32 FLSE;       // 0xD0)
00080     __IO u32 FLSP;       // 0xD4)
00081     __IO u32 FLST;       // 0xD8)
00082     __IO u32 null10;     // 0xDC)
00083     __IO u32 SBIST;      // 0xE0)
00084 } SYS_TypeDef;
00085 #define SYS ((SYS_TypeDef *) (SYS_BASE + 0x80))
00086
00090 typedef struct
00091 {
00092     uint32_t PLL_SrcSel; /*PLL信号输入源选择, RC时钟或外部晶体*/
00093     uint32_t PLL_DivSel; /*选择8个时钟周期中, 哪个周期输出时钟*/
00094     uint32_t Clk_Sel;
00096     uint32_t Clk_DivSPI; /*SPI I2C时钟分频*/
00097     uint32_t Clk_DivUART; /*UART时钟分频*/
00098     uint32_t Clk_FEN;    /*模块时钟使能*/
00099
00100     uint32_t WDT_Ena;     /*使能看门狗*/
00101     uint32_t POR_Filter_Ena; /*使能POR滤波*/
00102 } SYS_InitTypeDef;
00103
00104 /*PLL信号输入源选择*/
00105 #define SYS_PLLSRSEL_RCH 0x00 /*使用4MHz RC时钟*/
00106 #define SYS_PLLSRSEL_CRYSTAL 0x01 /*使用晶体时钟*/

```



```

00107
00108 /* PLL 或者 HS 时钟选择*/
00109 #define CLK_SEL_HS      0x00
00110 #define CLK_SEL_PLL     0x01
00111
00112 #define SYS_MCLK_96M_RC  0x01
00113 #define SYS_MCLK_48M_RC  0x02
00114 #define SYS_MCLK_24M_RC  0x03
00115 #define SYS_MCLK_12M_RC  0x04
00116 #define SYS_MCLK_32K_RC  0x05
00117
00118 #define SYS_MCLK_96M_XTAL 0x11
00119 #define SYS_MCLK_48M_XTAL 0x12
00120 #define SYS_MCLK_24M_XTAL 0x13
00121 #define SYS_MCLK_12M_XTAL 0x14
00122
00123 /*SPI时钟分频*/
00124 #define SYS_Clk_SPIDiv1  0
00125 #define SYS_Clk_SPIDiv2  1
00126 #define SYS_Clk_SPIDiv4  2
00127 #define SYS_Clk_SPIDiv8  3
00128
00129 /*UART时钟分频*/
00130 #define SYS_Clk_UARTDiv1  0
00131 #define SYS_Clk_UARTDiv2  1
00132 #define SYS_Clk_UARTDiv4  2
00133 #define SYS_Clk_UARTDiv8  3
00134
00135 /*数字模块位定义*/
00136 #define SYS_Module_SPI0   BIT0
00137 #define SYS_Module_I2C0   BIT1
00138 #define SYS_Module_CMP    BIT2
00139 #define SYS_Module_HALL0  BIT3
00140 #define SYS_Module_TIMER0 BIT4
00141 #define SYS_Module_TIMER1 BIT5
00142 #define SYS_Module_TIMER2 BIT6
00143 #define SYS_Module_TIMER3 BIT7
00144 #define SYS_Module_QEP0   BIT8
00145 #define SYS_Module_QEP1   BIT9
00146 #define SYS_Module_MCPWM0 BIT10
00147 #define SYS_Module_GPIO   BIT11
00148 #define SYS_Module_UART0  BIT14
00149 #define SYS_Module_UART1  BIT15
00150 #define SYS_Module_CRC0   BIT16
00151 #define SYS_Module_DSP0   BIT17
00152 #define SYS_Module_CAN0   BIT19
00153 #define SYS_Module_SIF0   BIT20
00154 #define SYS_Module_CL0    BIT21
00155
00156 /*模拟模块定义*/
00157 #define SYS_AnalogModule_ADC BIT0 // ADC
00158 #define SYS_AnalogModule_OPA0 BIT2 // OPA0
00159 #define SYS_AnalogModule_OPA1 BIT3 // OPA1
00160 #define SYS_AnalogModule_OPA2 BIT4 // OPA2
00161 #define SYS_AnalogModule_OPA3 BIT5 // OPA3
00162 #define SYS_AnalogModule_CMP0 BIT6 // CMP0
00163 #define SYS_AnalogModule_CMP1 BIT7 // CMP1
00164 #define SYS_AnalogModule_BGP BIT8 // BGP
00165 #define SYS_AnalogModule_DAC1 BIT9 // BGP
00166 #define SYS_AnalogModule_RCH BIT10 // RCH
00167 #define SYS_AnalogModule_CMP2 BIT11 // CMP2
00168 #define SYS_AnalogModule_DAC0 BIT12 // DAC
00169 #define SYS_AnalogModule_TMP BIT13 // TEMPsss
00170 #define SYS_AnalogModule_XTAL BIT14 // XTAL
00171 #define SYS_AnalogModule_PLL BIT15 // PLL
00172
00173 #define SYS_MCU_POWER_3v3 1 // MCU 3.3V供电
00174 #define SYS_MCU_POWER_5v0 0 // MCU 5V供电
00175
00176 // 在没有全局配置POWER_MODE时, 使用5V供电作为默认参数
00177 #ifndef POWER_MODE
00178 #define POWER_MODE SYS_MCU_POWER_5v0
00179 #endif
00180
00181 #define SYS_POWER_4V0 0x00 /*电源掉电监测阈值为4.0V*/
00182 #define SYS_POWER_3V75 0x01 /*电源掉电监测阈值为3.75V*/
00183 #define SYS_POWER_3V5 0x02 /*电源掉电监测阈值为3.5V*/
00184 #define SYS_POWER_3V25 0x03 /* 仅在芯片版本0123时有效 电源掉电监测阈值为3.25V*/
00185 #define SYS_POWER_2V5 0x03 /* 仅在芯片版本>=4时有效 电源掉电监测阈值为2.5V*/
00186
00187 // 兼容之前的写法
00188 #ifndef MCU_POWER_3v3
00189 #define MCU_POWER_3v3 SYS_MCU_POWER_3v3 // MCU 3.3V供电
00190 #endif
00191
00192 #ifndef MCU_POWER_5v0
00193 #define MCU_POWER_5v0 SYS_MCU_POWER_5v0 // MCU 5V供电

```

```

00194 #endif
00195
00196 #ifndef POWER_4V0
00197 #define POWER_4V0 SYS_POWER_4V0 /*电源掉电监测阈值为4.0V*/
00198 #endif
00199
00200 #ifndef POWER_3V75
00201 #define POWER_3V75 SYS_POWER_3V75 /*电源掉电监测阈值为3.75V*/
00202 #endif
00203
00204 #ifndef POWER_3V5
00205 #define POWER_3V5 SYS_POWER_3V5 /*电源掉电监测阈值为3.5V*/
00206 #endif
00207
00208 #ifndef POWER_3V25
00209 #define POWER_3V25 SYS_POWER_3V25 /* 仅在芯片版本0123时有效 电源掉电监测阈值为3.25V*/
00210 #endif
00211
00212 #ifndef POWER_2V5
00213 #define POWER_2V5 SYS_POWER_2V5 /* 仅在芯片版本>=4时有效 电源掉电监测阈值为2.5V*/
00214 #endif
00215
00219 typedef struct
00220 {
00221     __IO uint32_t PSW;
00222     __IO uint32_t CFG;
00223     __IO uint32_t CLR;
00224     __IO uint32_t WTH;
00225     __IO uint32_t RTH;
00226     __IO uint32_t CNT;
00227 } IWDG_TypeDef;
00231 #ifndef IWDG
00232 #define IWDG ((IWDG_TypeDef *) IWDG_BASE)
00233 #endif
00237 typedef struct
00238 {
00239     u32 DWK_EN;
00240     u32 WDG_EN;
00241     u32 WTH;
00242     u32 RTH;
00243 } IWDG_InitTypeDef;
00244
00245 #define SECOND2IWDGCNT(x) ((u32)(32000 * (float)x + 0x800) & 0x1fff000)
00247 void IWDG_Init(IWDG_InitTypeDef *); // 看门狗初始化
00248 void IWDG_StructInit(IWDG_InitTypeDef *); // 看门狗配置结构体初始化
00249 void IWDG_DISABLE(void); // 关闭看门狗
00250 void IWDG_ENABLE(void); // 看门狗使能
00251 void IWDG_Feed(void); // 看门狗喂狗
00252
00253 /*模块使能*/
00254 void SYS_ModuleClockCmd(uint32_t nModule, FuncState state);
00255 void SYS_AnalogModuleClockCmd(uint32_t nModule, FuncState state);
00256
00257 /*模块软复位*/
00258 void SYS_SoftResetModule(uint32_t nModule);
00259 void SYS_SoftReset_all(uint32_t state);
00260
00261 /*系统时钟配置*/
00262 void SYS_MclkChoice(u8 SYS_MCLK_x);
00263
00264 /*获取当前时钟*/
00265 u32 SYS_ReadMcuClk(void);
00266
00267 /*外设时钟分频设置*/
00268 void SYS_UartClkDiv(u16 div);
00269 void SYS_I2CClkDiv(u16 div);
00270 void SYS_SPIClkDiv(u16 div);
00271
00272 /*SYS清零复位*/
00273 void SYS_ClearRst(void);
00274
00275 /*SYS时钟初始化*/
00276 void SYS_Init(SYS_InitTypeDef *SYS_InitStruct);
00277
00278 /*SYS时钟结构体初始化*/
00279 void SYS_StructInit(SYS_InitTypeDef *SYS_InitStruct);
00280
00281 void SYS_VolSelModule(uint32_t Vol);
00282 void SYS_VolSelModuleIRQ(void);
00283 void SYS_VolSelModuleEnableIRQ(uint32_t Vol);
00284 #endif /*__LKS32mc07x_SYS_H */
00285
00286 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.43 include/lks32mc07x_tim.h 文件参考

文件名称: [lks32mc07x_tim.h](#)

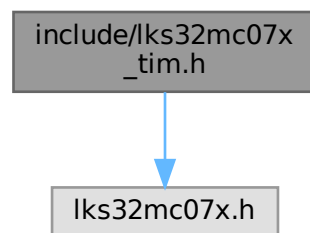
文件标识: 无

内容摘要: TIM驱动头文件

其它说明: 无

```
#include "lks32mc07x.h"
```

lks32mc07x_tim.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [TIM_TimerTypeDef](#)
- struct [TIM_TimerInitTypeDef](#)

宏定义

- #define [UTIMER0](#) (([TIM_TimerTypeDef](#) *)TIMER0_BASE)
TIMER0结构体基地址定义
- #define [UTIMER1](#) (([TIM_TimerTypeDef](#) *)TIMER1_BASE)
TIMER1结构体基地址定义
- #define [UTIMER2](#) (([TIM_TimerTypeDef](#) *)TIMER2_BASE)
TIMER2结构体基地址定义
- #define [UTIMER3](#) (([TIM_TimerTypeDef](#) *)TIMER3_BASE)
TIMER3结构体基地址定义
- #define [TIMER_IF_CH0](#) BIT0
Timer0 CH0中断, 比较或捕获中断
- #define [TIMER_IF_CH1](#) BIT1

- Timer0 CH1*中断，比较或捕获中断
- #define **TIMER_IF_ZERO** BIT2
Timer0 过零中断
- #define **TIMER_IF_FAIL** BIT3
Fail 事件中断标志
- #define **TIMER_CLK_DIV1** 0x00
*Timer*计数器频率1分频
- #define **TIMER_CLK_DIV2** 0x01
*Timer*计数器频率2分频
- #define **TIMER_CLK_DIV4** 0x02
*Timer*计数器频率4分频
- #define **TIMER_CLK_DIV8** 0x03
*Timer*计数器频率8分频
- #define **TIMER_CLK_DIV16** 0x04
*Timer*计数器频率16分频
- #define **TIMER_CLK_DIV32** 0x05
*Timer*计数器频率32分频
- #define **TIMER_CLK_DIV64** 0x06
*Timer*计数器频率64分频
- #define **TIMER_CLK_DIV128** 0x07
*Timer*计数器频率128分频
- #define **CAP_SRC_TCH0** 0X00
捕获定时器对应通道0输入信号
- #define **CAP_SRC_TCH1** 0X01
捕获定时器对应通道1输入信号
- #define **CAP_SRC_CLU0** 0X02
捕获*CLU0*输出信号
- #define **CAP_SRC_CLU1** 0X03
捕获*CLU1*输出信号
- #define **CAP_SRC_CLU2** 0X04
捕获*CLU2*输出信号
- #define **CAP_SRC_CLU3** 0X05
捕获*CLU3*输出信号
- #define **CAP_SRC_CMP0** 0X06
捕获比较器0输出信号
- #define **CAP_SRC_CMP1** 0X07
捕获比较器1输出信号
- #define **CAP_SRC_CMP2** 0X08
捕获比较器2输出信号
- #define **CAP_SRC_CHXOR** 0X09
捕获*Timer*通道0和1的异或信号
- #define **TIMER_POL_HIGH** 0x01
比较模式下的输出高电平
- #define **TIMER_POL_LOW** 0x00
比较模式下的输出低电平
- #define **TIMER_OPMode_CAP** ((uint32_t)0x0000001)
捕获模式
- #define **TIMER_OPMode_CMP** ((uint32_t)0x0000000)
比较模式
- #define **TIMER0_EVT_CH0** 0x00
*TIMER0*通道0

- #define **TIMER0.EVT_CH1** 0x01
TIMER0通道1
- #define **TIMER1.EVT_CH0** 0x02
TIMER1通道0
- #define **TIMER1.EVT_CH1** 0x03
TIMER1通道1
- #define **TIMER2.EVT_CH0** 0x04
TIMER2通道0
- #define **TIMER2.EVT_CH1** 0x05
TIMER2通道1
- #define **TIMER3.EVT_CH0** 0x06
TIMER3通道0
- #define **TIMER3.EVT_CH1** 0x07
TIMER3通道1
- #define **TIMER.EVT_CLU0** 0x08
CLU0 输出信号
- #define **TIMER.EVT_CLU1** 0x09
CLU1 输出信号
- #define **TIMER.EVT_CLU2** 0x0A
CLU2 输出信号
- #define **TIMER.EVT_CLU3** 0x0B
CLU3 输出信号
- #define **TIMER.EVT_TADC0** 0x0C
MCPWM TADC[0]比较事件
- #define **TIMER.EVT_TADC1** 0x0D
MCPWM TADC[1]比较事件
- #define **TIMER.EVT_TADC2** 0x0E
MCPWM TADC[2]比较事件
- #define **TIMER.EVT_TADC3** 0x0F
MCPWM TADC[3]比较事件
- #define **TIMER.CH0.IF** BIT0
Timer通道0比较/捕获中断标志
- #define **TIMER.CH1.IF** BIT1
Timer通道1比较/捕获中断标志
- #define **TIMER.ZC.IF** BIT2
Timer计数器过0中断标志
- #define **TIMER.FAIL.IF** BIT3
Timer Fail事件中断标志

枚举

- enum **Timer_IRQEnaDef** {
Timer_IRQEna_None = 0 , **Timer_IRQEna_CH0** = BIT0 , **Timer_IRQEna_CH1** = BIT1 , **Timer_IRQEna_ZC** = BIT2 ,
Timer0_IRQEna_FAIL = BIT3 , **Timer_IRQEna_CH0_RE** = BIT8 , **Timer_IRQEna_CH1_RE** = BIT9 ,
Timer_IRQEna_ZC_RE = BIT10 ,
Timer0_IRQEna_FAIL_RE = BIT11 , **Timer_IRQEna_All** = 0xf }
 中断使能配置定义

函数

- void **TIM_TimerInit** (TIM_TimerTypeDef *TIMERx, TIM_TimerInitTypeDef *this)
函数名称: void TIM_TimerInit(TIM_TimerTypeDef *TIMERx, TIM_TimerInitTypeDef *this)
- void **TIM_TimerStrutInit** (TIM_TimerInitTypeDef *this)
函数名称: void TIM_TimerStrutInit(TIM_TimerInitTypeDef *this)
- void **TIM_TimerCmd** (TIM_TimerTypeDef *TIMERx, FuncState state)
函数名称: void TIM_TimerCmd(TIM_TimerTypeDef *TIMERx, FuncState state)
- void **TIM_CMP0** (TIM_TimerTypeDef *TIMERx, uint32_t temp)
函数名称: void TIM_CMP0(TIM_TimerTypeDef *TIMERx, uint32_t temp)
- void **TIM_CMP1** (TIM_TimerTypeDef *TIMERx, uint32_t temp)
函数名称: void TIM_CMP1(TIM_TimerTypeDef *TIMERx, uint32_t temp)
- u32 **TIM_Timer_GetCMPT0** (TIM_TimerTypeDef *TIMERx)
函数名称: u32 TIM_Timer_GetCMPT0(TIM_TimerTypeDef *TIMERx)
- u32 **TIM_Timer_GetCMPT1** (TIM_TimerTypeDef *TIMERx)
函数名称: u32 TIM_Timer_GetCMPT1(TIM_TimerTypeDef *TIMERx)
- void **TIM_ClearIRQFlag** (TIM_TimerTypeDef *TIMERx, uint32_t tempFlag)
函数名称: void TIM_ClearIRQFlag(TIM_TimerTypeDef *TIMERx ,uint32_t tempFlag)
- u16 **TIM_GetIRQFlag** (TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x)
函数名称: u16 TIM_GetIRQFlag(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x)
- u16 **TIM_GetFlag** (TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x)
函数名称: u16 TIM_GetFlag(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x)
- uint32_t **TIM_Timer_GetCount** (TIM_TimerTypeDef *TIMERx)
函数名称: uint32_t TIM_Timer_GetCount(TIM_TimerTypeDef *TIMERx)
- void **UTIMER_Enable** (TIM_TimerTypeDef *UTIMERx)
- void **UTIMER_Disable** (TIM_TimerTypeDef *UTIMERx)

5.43.1 详细描述

文件名称: lks32mc07x_tim.h
文件标识: 无
内容摘要: TIM驱动头文件
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 lks32mc07x_tim.h 中定义.

5.43.2 宏定义说明

5.43.2.1 CAP_SRC.CHXOR

```
#define CAP_SRC.CHXOR 0X09
```

捕获Timer通道0和1的异或信号

在文件 [lks32mc07x_tim.h](#) 第 94 行定义.

5.43.2.2 CAP_SRC.CLU0

```
#define CAP_SRC.CLU0 0X02
```

捕获CLU0输出信号

在文件 [lks32mc07x_tim.h](#) 第 87 行定义.

5.43.2.3 CAP_SRC.CLU1

```
#define CAP_SRC.CLU1 0X03
```

捕获CLU1输出信号

在文件 [lks32mc07x_tim.h](#) 第 88 行定义.

5.43.2.4 CAP_SRC.CLU2

```
#define CAP_SRC.CLU2 0X04
```

捕获CLU2输出信号

在文件 [lks32mc07x_tim.h](#) 第 89 行定义.

5.43.2.5 CAP_SRC.CLU3

```
#define CAP_SRC.CLU3 0X05
```

捕获CLU3输出信号

在文件 [lks32mc07x_tim.h](#) 第 90 行定义.

5.43.2.6 CAP_SRC.CMP0

```
#define CAP_SRC.CMP0 0X06
```

捕获比较器0输出信号

在文件 [lks32mc07x_tim.h](#) 第 91 行定义.

5.43.2.7 CAP_SRC_CMP1

```
#define CAP_SRC_CMP1 0X07
```

捕获比较器1输出信号

在文件 [lks32mc07x_tim.h](#) 第 92 行定义.

5.43.2.8 CAP_SRC_CMP2

```
#define CAP_SRC_CMP2 0X08
```

捕获比较器2输出信号

在文件 [lks32mc07x_tim.h](#) 第 93 行定义.

5.43.2.9 CAP_SRC_TCH0

```
#define CAP_SRC_TCH0 0X00
```

捕获定时器对应通道0输入信号

在文件 [lks32mc07x_tim.h](#) 第 85 行定义.

5.43.2.10 CAP_SRC_TCH1

```
#define CAP_SRC_TCH1 0X01
```

捕获定时器对应通道1输入信号

在文件 [lks32mc07x_tim.h](#) 第 86 行定义.

5.43.2.11 TIMER0_EVT_CH0

```
#define TIMER0_EVT_CH0 0x00
```

TIMER0通道0

在文件 [lks32mc07x_tim.h](#) 第 102 行定义.

5.43.2.12 TIMER0_EVT_CH1

```
#define TIMER0_EVT_CH1 0x01
```

TIMER0通道1

在文件 [lks32mc07x_tim.h](#) 第 103 行定义.

5.43.2.13 TIMER1_EVT_CH0

```
#define TIMER1_EVT_CH0 0x02
```

TIMER1通道0

在文件 [lks32mc07x.tim.h](#) 第 104 行定义.

5.43.2.14 TIMER1_EVT_CH1

```
#define TIMER1_EVT_CH1 0x03
```

TIMER1通道1

在文件 [lks32mc07x.tim.h](#) 第 105 行定义.

5.43.2.15 TIMER2_EVT_CH0

```
#define TIMER2_EVT_CH0 0x04
```

TIMER2通道0

在文件 [lks32mc07x.tim.h](#) 第 106 行定义.

5.43.2.16 TIMER2_EVT_CH1

```
#define TIMER2_EVT_CH1 0x05
```

TIMER2通道1

在文件 [lks32mc07x.tim.h](#) 第 107 行定义.

5.43.2.17 TIMER3_EVT_CH0

```
#define TIMER3_EVT_CH0 0x06
```

TIMER3通道0

在文件 [lks32mc07x.tim.h](#) 第 108 行定义.

5.43.2.18 TIMER3_EVT_CH1

```
#define TIMER3_EVT_CH1 0x07
```

TIMER3通道1

在文件 [lks32mc07x.tim.h](#) 第 109 行定义.

5.43.2.19 TIMER_CH0_IF

```
#define TIMER_CH0_IF BIT0
```

Timer通道0比较/捕获中断标志

在文件 [lks32mc07x.tim.h](#) 第 119 行定义.

5.43.2.20 TIMER_CH1_IF

```
#define TIMER_CH1_IF BIT1
```

Timer通道1比较/捕获中断标志

在文件 [lks32mc07x.tim.h](#) 第 120 行定义.

5.43.2.21 TIMER_CLK_DIV1

```
#define TIMER_CLK_DIV1 0x00
```

Timer计数器频率1分频

在文件 [lks32mc07x.tim.h](#) 第 76 行定义.

5.43.2.22 TIMER_CLK_DIV128

```
#define TIMER_CLK_DIV128 0x07
```

Timer计数器频率128分频

在文件 [lks32mc07x.tim.h](#) 第 83 行定义.

5.43.2.23 TIMER_CLK_DIV16

```
#define TIMER_CLK_DIV16 0x04
```

Timer计数器频率16分频

在文件 [lks32mc07x.tim.h](#) 第 80 行定义.

5.43.2.24 TIMER_CLK_DIV2

```
#define TIMER_CLK_DIV2 0x01
```

Timer计数器频率2分频

在文件 [lks32mc07x.tim.h](#) 第 77 行定义.

5.43.2.25 TIMER_CLK_DIV32

```
#define TIMER_CLK_DIV32 0x05
```

Timer计数器频率32分频

在文件 [lks32mc07x.tim.h](#) 第 81 行定义.

5.43.2.26 TIMER_CLK_DIV4

```
#define TIMER_CLK_DIV4 0x02
```

Timer计数器频率4分频

在文件 [lks32mc07x.tim.h](#) 第 78 行定义.

5.43.2.27 TIMER_CLK_DIV64

```
#define TIMER_CLK_DIV64 0x06
```

Timer计数器频率64分频

在文件 [lks32mc07x.tim.h](#) 第 82 行定义.

5.43.2.28 TIMER_CLK_DIV8

```
#define TIMER_CLK_DIV8 0x03
```

Timer计数器频率8分频

在文件 [lks32mc07x.tim.h](#) 第 79 行定义.

5.43.2.29 TIMER_EVT_CLU0

```
#define TIMER_EVT_CLU0 0x08
```

CLU0 输出信号

在文件 [lks32mc07x.tim.h](#) 第 110 行定义.

5.43.2.30 TIMER_EVT_CLU1

```
#define TIMER_EVT_CLU1 0x09
```

CLU1 输出信号

在文件 [lks32mc07x.tim.h](#) 第 111 行定义.

5.43.2.31 TIMER_EVT_CLU2

```
#define TIMER_EVT_CLU2 0x0A
```

CLU2 输出信号

在文件 [lks32mc07x.tim.h](#) 第 112 行定义.

5.43.2.32 TIMER_EVT_CLU3

```
#define TIMER_EVT_CLU3 0x0B
```

CLU3 输出信号

在文件 [lks32mc07x.tim.h](#) 第 113 行定义.

5.43.2.33 TIMER_EVT_TADC0

```
#define TIMER_EVT_TADC0 0x0C
```

MCPWM TADC[0]比较事件

在文件 [lks32mc07x.tim.h](#) 第 114 行定义.

5.43.2.34 TIMER_EVT_TADC1

```
#define TIMER_EVT_TADC1 0x0D
```

MCPWM TADC[1]比较事件

在文件 [lks32mc07x.tim.h](#) 第 115 行定义.

5.43.2.35 TIMER_EVT_TADC2

```
#define TIMER_EVT_TADC2 0x0E
```

MCPWM TADC[2]比较事件

在文件 [lks32mc07x.tim.h](#) 第 116 行定义.

5.43.2.36 TIMER_EVT_TADC3

```
#define TIMER_EVT_TADC3 0x0F
```

MCPWM TADC[3]比较事件

在文件 [lks32mc07x.tim.h](#) 第 117 行定义.

5.43.2.37 TIMER_FAIL_IF

```
#define TIMER_FAIL_IF BIT3
```

Timer Fail事件中断标志

在文件 [lks32mc07x.tim.h](#) 第 122 行定义.

5.43.2.38 TIMER_IF_CH0

```
#define TIMER_IF_CH0 BIT0
```

Timer0 CH0中断, 比较或捕获中断

在文件 [lks32mc07x.tim.h](#) 第 71 行定义.

5.43.2.39 TIMER_IF_CH1

```
#define TIMER_IF_CH1 BIT1
```

Timer0 CH1中断, 比较或捕获中断

在文件 [lks32mc07x.tim.h](#) 第 72 行定义.

5.43.2.40 TIMER_IF_FAIL

```
#define TIMER_IF_FAIL BIT3
```

Fail 事件中断标志

在文件 [lks32mc07x.tim.h](#) 第 74 行定义.

5.43.2.41 TIMER_IF_ZERO

```
#define TIMER_IF_ZERO BIT2
```

Timer0 过零中断

在文件 [lks32mc07x.tim.h](#) 第 73 行定义.

5.43.2.42 TIMER_OPMODE_CAP

```
#define TIMER_OPMODE_CAP ((uint32_t)0x00000001)
```

捕获模式

在文件 [lks32mc07x.tim.h](#) 第 99 行定义.

5.43.2.43 TIMER_OPMode_CMP

```
#define TIMER_OPMode_CMP ((uint32_t)0x00000000)
```

比较模式

在文件 [lks32mc07x_tim.h](#) 第 100 行定义.

5.43.2.44 TIMER_POL_HIGH

```
#define TIMER_POL_HIGH 0x01
```

比较模式下的输出高电平

在文件 [lks32mc07x_tim.h](#) 第 96 行定义.

5.43.2.45 TIMER_POL_LOW

```
#define TIMER_POL_LOW 0x00
```

比较模式下的输出低电平

在文件 [lks32mc07x_tim.h](#) 第 97 行定义.

5.43.2.46 TIMER_ZC_IF

```
#define TIMER_ZC_IF BIT2
```

Timer计数器过0中断标志

在文件 [lks32mc07x_tim.h](#) 第 121 行定义.

5.43.2.47 UTIMER0

```
#define UTIMER0 ((TIM_TimeTypeDef *)TIMER0_BASE)
```

TIMER0结构体基地址定义

在文件 [lks32mc07x_tim.h](#) 第 37 行定义.

5.43.2.48 UTIMER1

```
#define UTIMER1 ((TIM_TimeTypeDef *)TIMER1_BASE)
```

TIMER1结构体基地址定义

在文件 [lks32mc07x_tim.h](#) 第 41 行定义.

5.43.2.49 UTIMER2

```
#define UTIMER2 ((TIM_TypeDef *)TIMER2_BASE)
```

TIMER2结构体基地址定义

在文件 lks32mc07x.tim.h 第 45 行定义.

5.43.2.50 UTIMER3

```
#define UTIMER3 ((TIM_TypeDef *)TIMER3_BASE)
```

TIMER3结构体基地址定义

在文件 lks32mc07x.tim.h 第 49 行定义.

5.43.3 枚举类型说明

5.43.3.1 Timer_IRQEnaDef

```
enum Timer_IRQEnaDef
```

中断使能配置定义

枚举值

Timer_IRQEna_None	关闭所有中断
Timer_IRQEna_CH0	通道 0 中断
Timer_IRQEna_CH1	通道 1 中断
Timer_IRQEna_ZC	计数器过 0 中断
Timer0_IRQEna_FAIL	定时器0, FAIL事件中断
Timer_IRQEna_CH0_RE	通道0比较/捕获DMA请求使能
Timer_IRQEna_CH1_RE	通道1比较/捕获DMA请求使能
Timer_IRQEna_ZC_RE	计数器过0 DMA请求使能
Timer0_IRQEna_FAIL_RE	定时器0, FAIL事件DMA请求使能
Timer_IRQEna_All	开启所有中断

在文件 lks32mc07x.tim.h 第 54 行定义.

5.43.4 函数说明

5.43.4.1 TIM_ClearIRQFlag()

```
void TIM_ClearIRQFlag (
    TIM_TypeDef * TIMeRx,
    uint32_t tempFlag )
```

函数名称: void TIM_ClearIRQFlag(TIM_TypeDef *TIMeRx ,uint32_t tempFlag)

功能描述: 清除TIM中断标志

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0	
	timer_if:	
	Timer_IRQEna_CH0_RE	通道0比较/捕获DMA请求使能
	Timer_IRQEna_CH1_RE	通道1比较/捕获DMA请求使能
	Timer_IRQEna_ZC_RE	计数器过0 DMA请求使能
	Timer0_IRQEna_FAIL_RE	定时器0, FAIL事件DMA请求使能
	Timer_IRQEna_CH1	Timer CH1中断, 比较或捕获中断
	Timer_IRQEna_CH0	Timer CH0中断, 比较或捕获中断
	Timer_IRQEna_ZC	Timer 过零中断
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if(TIM_GetIRQFlag(UTIMER0,Timer_IRQEna_CH0)) //判断UTimer0的CH0是否发生比较中断
{
    TIM_ClearIRQFlag(UTIMER0,Timer_IRQEna_CH0); //清除UTimer0通道0比较中断标志位
}
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 338 行定义.

5.43.4.2 TIM_CMP0()

```
void TIM_CMP0 (
```



```
TIM_TimerTypeDef * TIMeRx,
uint32_t temp )
```

函数名称: void TIM_CMP0(TIM_TimerTypeDef *TIMeRx, uint32_t temp)

功能描述: 写入定时器通道0比较值

参见
被引用内容: 无

参数

输入参数:	TIMeRx: UTIMER0/UTIMER0/UTIMER0/UTIMER0 temp: 比较数值
输出参数:	无

返回
返回 值: 无

注解
其它说明: 无

警告
无

示例代码:

```
TIM_CMP0 (UTIMER0, 2000); //设定定时器0通道0比较值为2000
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 lks32mc07x_tim.c 第 226 行定义.

5.43.4.3 TIM_CMP1()

```
void TIM_CMP1 (
TIM_TimerTypeDef * TIMeRx,
uint32_t temp )
```

函数名称: void TIM_CMP1(TIM_TimerTypeDef *TIMERx, uint32_t temp)

功能描述: 写入定时器通道1比较值

参见

被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0 temp: 比较数值
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
TIM_CMP1(UTIMER0 , 2000); //设定定时器0通道1比较值为2000
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 lks32mc07x_tim.c 第 251 行定义.

5.43.4.4 TIM_GetFlag()

```
u16 TIM_GetFlag (  
    TIM_TimerTypeDef * UTIMERx,  
    u32 TIMER_IF_x )
```

函数名称: u16 TIM_GetFlag(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x)

功能描述: 获取TIM中断标志

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0	
	TIMER_IF_x:	
	TIMER_IF_CH1	Timer CH1中断, 比较或捕获中断
	TIMER_IF_CH0	Timer CH0中断, 比较或捕获中断
	TIMER_IF_ZERO	Timer 过零中断
输出参数:	无	

返回

返 回 值： 0或1， 对应中断标志置位返回1， 未置位返回0

注解

其它说明： 无

警告

不需要对应中断使能后， 就可以读取对应中断标志位

示例代码:

```
if(TIM_GetFlag(UTIMER0,TIMER_IF_CH0)) //判断UTimer0的CH0是否发生比较中断
{
}
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 407 行定义.

5.43.4.5 TIM_GetIRQFlag()

```
u16 TIM_GetIRQFlag (
    TIM_TimerTypeDef * UTIMERx,
    u32 TIMER_IF_x )
```

函数名称: u16 TIM_GetIRQFlag(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x)

功能描述: 获取TIM中断标志

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0	
	TIMER_IF_x:	
	TIMER_IF_CH1	Timer CH1中断, 比较或捕获中断
	TIMER_IF_CH0	Timer CH0中断, 比较或捕获中断
	TIMER_IF_ZERO	Timer 过零中断
输出参数:	无	

返回

返 回 值： 0或1， 对应中断标志置位返回1， 未置位返回0

注解

其它说明： 无

警告

只有对应中断使能后， 改为才能读取， 如果对应中断未使能， 读取结果一直为0

示例代码:

```
if(TIM_GetIRQFlag(UTIMER0,TIMER_IF_CH0)) //判断UTimer0的CH0是否发生比较中断
{
}
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 369 行定义.

5.43.4.6 TIM_Timer_GetCMPT0()

```
u32 TIM_Timer_GetCMPT0 (
    TIM_TimerTypeDef * TIMERx )
```

函数名称: u32 [TIM_Timer_GetCMPT0\(TIM_TimerTypeDef *TIMERx\)](#)

功能描述: 获取定时器通道0捕获值

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0
输出参数:	无

返回

返回 值: 定时器通道0捕获值

注解

其它说明: 当定时器发生捕获事件时，将捕获时刻的cnt值存储到该寄存器中

警告

无

示例代码:

```
uint32_t TIMER0_CAPValue = 0;
TIMER0_CAPValue = TIM_Timer_GetCMPT0 (UTIMER0); //获取定时器0捕获值
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 277 行定义.

5.43.4.7 TIM_Timer_GetCMPT1()

```
u32 TIM_Timer_GetCMPT1 (
    TIM_TimerTypeDef * TIMERx )
```

函数名称: u32 [TIM_Timer_GetCMPT1\(TIM_TimerTypeDef *TIMERx\)](#)

功能描述: 获取定时器通道1捕获值

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0
输出参数:	无

返回

返回 值: 定时器通道1捕获值

注解

其它说明: 当定时器发生捕获事件时, 将捕获时刻的cnt值存储到该寄存器中

警告

无

示例代码:

```
uint32_t TIMER0_CAPValue = 0;
TIMER0_CAPValue = TIM_Timer_GetCMPT1(UTIMER0); //获取定时器0捕获值
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 302 行定义.

5.43.4.8 TIM_Timer_GetCount()

```
uint32_t TIM_Timer_GetCount (
    TIM_TimerTypeDef * TIMERx )
```

函数名称: uint32_t [TIM_Timer_GetCount\(TIM_TimerTypeDef *TIMERx\)](#)

功能描述: 获取Timer计数值

参见

被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
uint32_t TIMER0_Value = 0;
TIMER0_Value = TIM_Timer_GetCount(UTIMER0); //获取定时器0计数值
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x.tim.c](#) 第 439 行定义.

5.43.4.9 TIM_TimerCmd()

```
void TIM_TimerCmd (
    TIM_TimerTypeDef * TIMERx,
    FuncState state )
```

函数名称: void [TIM_TimerCmd\(TIM_TimerTypeDef *TIMERx, FuncState state\)](#)

功能描述: 定时器TIMERx使能和停止

参见

被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0 FuncState state: 使能与失能
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

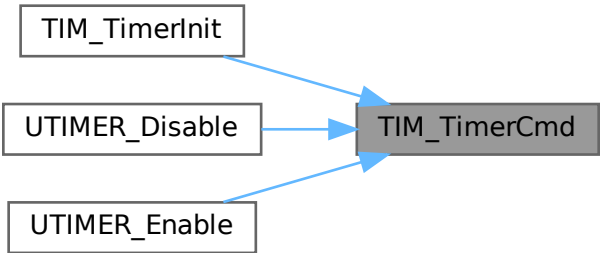
```
TIM_TimerCmd(UTIMER0, ENABLE); // Timer0模块使能
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 194 行定义.

这是这个函数的调用关系图:



5.43.4.10 TIM_TimerInit()

```
void TIM_TimerInit (
    TIM_TimerTypeDef * TIMERx,
    TIM_TimerInitTypeDef * this )
```

函数名称: void [TIM_TimerInit\(TIM_TimerTypeDef *TIMERx, TIM_TimerInitTypeDef *this\)](#)

功能描述: 定时器初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0 TIM_TimerInitTypeDef *TIM_TimerInitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

事例1比较模式配置:

```
TIM_TimerInitTypeDef TIM_InitStructure;
TIM_TimerStrutInit(&TIM_InitStructure); // Timer结构体初始化
TIM_InitStructure.Timer_CH0WorkMode = TIMER_OPMode_CMP; // 设置Timer CH0 为比较模式
TIM_InitStructure.Timer_CH0Output = 0; // 计数器回零时, 比较模式输出极性控制
TIM_InitStructure.Timer_CH1WorkMode = TIMER_OPMode_CMP; // 设置Timer CH1 为比较模式
TIM_InitStructure.Timer_CH1Output = 0; // 计数器回零时, 比较模式输出极性控制
TIM_InitStructure.Timer_TH = 48000; // 定时器计数门限初始值48000
TIM_InitStructure.Timer_CMP0 = 24000; // 设置比较模式的CH0比较初始值24000
TIM_InitStructure.Timer_CMP1 = 24000; // 设置比较模式的CH1比较初始值24000
TIM_InitStructure.Timer_ClockDiv = TIMER_CLK_DIV1; // 设置Timer模块时钟2分频系数
TIM_InitStructure.Timer_IRQEna = Timer_IRQEna_CH0 | Timer_IRQEna_CH1 | Timer_IRQEna_ZC; // 开启Timer模块比
较中断和过零中断
TIM_TimerInit(UTIMER0, &TIM_InitStructure);
TIM_TimerCmd(UTIMER0, ENABLE); // 定时器2模块使能
```

事例2捕获模式配置:

```
TIM_TimerInitTypeDef TIM_InitStructure;
TIM_TimerStrutInit(&TIM_InitStructure); // Timer结构体初始化
TIM_InitStructure.Timer_SRC0 = CAP_SRC_TCH1; // 定时器0通道0捕获定时器0通道1的输入信号
TIM_InitStructure.Timer_CH0WorkMode = TIMER_OPMode_CAP; // 设置Timer CH0为捕获模式
TIM_InitStructure.Timer_CAP0_CLR_EN = ENABLE; // 当发生CAP0捕获事件时, 清零Timer计数器
TIM_InitStructure.Timer_CH0_RE_CAP_EN = ENABLE; // 使能Timer通道0上升沿捕获
TIM_InitStructure.Timer_CH0_FE_CAP_EN = DISABLE; // 关闭Timer通道0下降沿捕获

TIM_InitStructure.Timer_SRC1 = CAP_SRC_TCH1; // 定时器0通道1捕获定时器0通道1的输入信号
TIM_InitStructure.Timer_CH1WorkMode = TIMER_OPMode_CAP; // 设置Timer CH1为捕获模式
TIM_InitStructure.Timer_CAP1_CLR_EN = DISABLE; // 当发生CAP1捕获事件时, 清零Timer计数器
TIM_InitStructure.Timer_CH1_RE_CAP_EN = DISABLE; // 使能Timer通道1上升沿捕获
TIM_InitStructure.Timer_CH1_FE_CAP_EN = ENABLE; // 关闭Timer通道1下降沿捕获

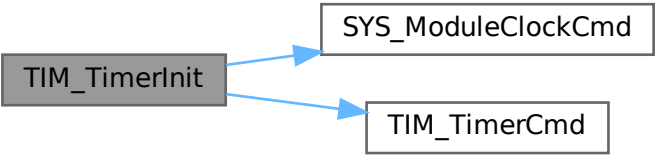
TIM_InitStructure.Timer_TH = 96000000; // 定时器计数门限初始值96000000
TIM_InitStructure.Timer_FLT = 0; // 设置捕捉模式或编码器模式下对应通道的数字滤波值
TIM_InitStructure.Timer_ClockDiv = TIMER_CLK_DIV1; // 设置Timer模块时钟2分频系数
TIM_InitStructure.Timer_IRQEna = Timer_IRQEna_ZC; // 开启Timer模块过零中断
TIM_TimerInit(UTIMER3, &TIM_InitStructure);
TIM_TimerCmd(UTIMER3, ENABLE); // Timer3 模块使能
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志: 去除定时器使能接口

在文件 [lks32mc07x_tim.c](#) 第 71 行定义.

函数调用图:



5.43.4.11 TIM_TimerStrutInit()

```
void TIM_TimerStrutInit (
    TIM_TimerInitTypeDef * this )
```

函数名称: void [TIM_TimerStrutInit\(TIM_TimerInitTypeDef *this\)](#)

功能描述: 定时器结构体初始化

参见

被引用内容: 无

参数

输入参数:	TIM_TimerInitTypeDef *this
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
TIM_TimerInitTypeDef TIM_InitStructure;  
TIM_TimrStrutInit (&TIM_InitStructure); // Timer结构体初始化
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 lks32mc07x_tim.c 第 124 行定义.

5.43.4.12 UTIMER_Disable()

```
void UTIMER_Disable (  
    TIM_TimerTypeDef * UTIMERx )
```

在文件 lks32mc07x_tim.c 第 525 行定义.

函数调用图:



5.43.4.13 UTIMER_Enable()

```
void UTIMER_Enable (  
    TIM_TimerTypeDef * UTIMERx )
```

在文件 lks32mc07x_tim.c 第 521 行定义.

函数调用图:



5.44 lks32mc07x_tim.h

[浏览该文件的文档.](#)

```

00001
00014 #ifndef __lks32mc07x_TIM_H
00015 #define __lks32mc07x_TIM_H
00016
00017 /* Includes -----*/
00018 #include "lks32mc07x.h"
00019
00020 typedef struct
00021 {
00022     __IO uint32_t CFG;
00023     __IO uint32_t TH;
00024     __IO uint32_t CNT;
00025     __IO uint32_t CMP0;
00026     __IO uint32_t CMP1;
00027     __IO uint32_t EVT;
00028     __IO uint32_t FLT;
00029     __IO uint32_t IE;
00030     __IO uint32_t IF;
00031     __IO uint32_t IO;
00032 } TIM_TimeTypeDef;
00033
00037 #define UTIMER0 ((TIM_TimeTypeDef *)TIMER0_BASE)
00041 #define UTIMER1 ((TIM_TimeTypeDef *)TIMER1_BASE)
00045 #define UTIMER2 ((TIM_TimeTypeDef *)TIMER2_BASE)
00049 #define UTIMER3 ((TIM_TimeTypeDef *)TIMER3_BASE)
00050
00054 typedef enum
00055 {
00056     Timer_IRQEna_None = 0,
00057     Timer_IRQEna_CH0 = BIT0,
00058     Timer_IRQEna_CH1 = BIT1,
00059     Timer_IRQEna_ZC = BIT2,
00060     Timer0_IRQEna_FAIL = BIT3,
00062     Timer_IRQEna_CH0_RE = BIT8,
00063     Timer_IRQEna_CH1_RE = BIT9,
00064     Timer_IRQEna_ZC_RE = BIT10,
00065     Timer0_IRQEna_FAIL_RE = BIT11,
00067     Timer_IRQEna_All = 0xf
00068 } Timer_IRQEnaDef;
00069
00070 /*中断标志定义*/
00071 #define TIMER_IF_CH0 BIT0
00072 #define TIMER_IF_CH1 BIT1
00073 #define TIMER_IF_ZERO BIT2
00074 #define TIMER_IF_FAIL BIT3
00076 #define TIMER_CLK_DIV1 0x00
00077 #define TIMER_CLK_DIV2 0x01
00078 #define TIMER_CLK_DIV4 0x02
00079 #define TIMER_CLK_DIV8 0x03
00080 #define TIMER_CLK_DIV16 0x04
00081 #define TIMER_CLK_DIV32 0x05
00082 #define TIMER_CLK_DIV64 0x06
00083 #define TIMER_CLK_DIV128 0x07
00085 #define CAP_SRC_TCH0 0x00
00086 #define CAP_SRC_TCH1 0x01
00087 #define CAP_SRC_CLU0 0x02
00088 #define CAP_SRC_CLU1 0x03
00089 #define CAP_SRC_CLU2 0x04
00090 #define CAP_SRC_CLU3 0x05
00091 #define CAP_SRC_CMP0 0x06
00092 #define CAP_SRC_CMP1 0x07
00093 #define CAP_SRC_CMP2 0x08
00094 #define CAP_SRC_CHXOR 0x09
00096 #define TIMER_POL_HIGH 0x01
00097 #define TIMER_POL_LOW 0x00
00099 #define TIMER_OPMODE_CAP ((uint32_t)0x00000001)
00100 #define TIMER_OPMODE_CMP ((uint32_t)0x00000000)
00102 #define TIMER0_EVT_CH0 0x00
00103 #define TIMER0_EVT_CH1 0x01
00104 #define TIMER1_EVT_CH0 0x02
00105 #define TIMER1_EVT_CH1 0x03
00106 #define TIMER2_EVT_CH0 0x04
00107 #define TIMER2_EVT_CH1 0x05
00108 #define TIMER3_EVT_CH0 0x06
00109 #define TIMER3_EVT_CH1 0x07
00110 #define TIMER_EVT_CLU0 0x08
00111 #define TIMER_EVT_CLU1 0x09
00112 #define TIMER_EVT_CLU2 0x0A
00113 #define TIMER_EVT_CLU3 0x0B
00114 #define TIMER_EVT_TADC0 0x0C
00115 #define TIMER_EVT_TADC1 0x0D
00116 #define TIMER_EVT_TADC2 0x0E

```

```

00117 #define TIMER_EVT.TADC3 0x0F
00119 #define TIMER_CH0_IF BIT0
00120 #define TIMER_CH1_IF BIT1
00121 #define TIMER_ZC_IF BIT2
00122 #define TIMER_FAIL_IF BIT3
00124 typedef struct
00125 {
00126     FuncState Timer_EN;
00127     uint32_t Timer_CAP1_CLR_EN;
00128     uint32_t Timer_CAP0_CLR_EN;
00130     uint32_t Timer_ONE_TRIG;
00132     uint32_t Timer_CENTER;
00134     uint32_t Timer_DIR;
00146     uint32_t Timer_ClockDiv;
00147     uint32_t Timer_ETON;
00148     uint32_t Timer_GATE_EN;
00150     uint32_t Timer_RL_EN;
00151     uint32_t Timer_XCLK_EN;
00165     uint32_t Timer_SRC1;
00166     uint32_t Timer_CH1Output;
00167     uint32_t Timer_CH1_WorkMode;
00168     uint32_t Timer_CH1_FE_CAP_EN;
00169     uint32_t Timer_CH1_RE_CAP_EN;
00183     uint32_t Timer_SRC0;
00184     uint32_t Timer_CH0Output;
00185     uint32_t Timer_CH0_WorkMode;
00186     uint32_t Timer_CH0_FE_CAP_EN;
00187     uint32_t Timer_CH0_RE_CAP_EN;
00189     uint32_t Timer_TH;
00190     uint32_t Timer_CNT;
00191     uint32_t Timer_CMP0;
00192     uint32_t Timer_CMP1;
00201     uint32_t Timer_EVT;
00202     uint32_t Timer_FLT;
00214     uint32_t Timer_IRQEna;
00216     uint32_t Timer0_UPDATE;
00218     uint32_t Timer0_SHADOW;
00221     uint32_t Timer0_CH1_DEFAULT;
00222     uint32_t Timer0_CH0_DEFAULT;
00223     uint32_t Timer0_HALT_PRT;
00224     uint32_t Timer0_FAIL_SEL;
00225     uint32_t Timer0_FAIL_POL;
00226     uint32_t Timer0_FAIL_EN;
00227 } TIM_TimerInitTypeDef;
00228
00229 /*Timer初始化*/
00230 void TIM_TimerInit(TIM_TimerTypeDef *TIMERx, TIM_TimerInitTypeDef *this);
00231 void TIM_TimerStrutInit(TIM_TimerInitTypeDef *this);
00232 void TIM_TimerCmd(TIM_TimerTypeDef *TIMERx, FuncState state);
00233 void TIM_CMP0(TIM_TimerTypeDef *TIMERx, uint32_t temp);
00234 void TIM_CMP1(TIM_TimerTypeDef *TIMERx, uint32_t temp);
00235 u32 TIM_Timer_GetCMPT0(TIM_TimerTypeDef *TIMERx);
00236 u32 TIM_Timer_GetCMPT1(TIM_TimerTypeDef *TIMERx);
00237 void TIM_ClearIRQFlag(TIM_TimerTypeDef *TIMERx, uint32_t tempFlag);
00238 u16 TIM_GetIRQFlag(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x);
00239 u16 TIM_GetFlag(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x);
00240 uint32_t TIM_Timer_GetCount(TIM_TimerTypeDef *TIMERx);
00241 void UTIMER_Enable(TIM_TimerTypeDef *UTIMERx);
00242 void UTIMER_Disable(TIM_TimerTypeDef *UTIMERx);
00243
00244 #endif /*_lks32mc07x_TIM_H */
00245
00246 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.45 include/lks32mc07x_tmp.h 文件参考

文件名称: lks32mc08x_tmp.h

文件标识: 无

内容摘要: 芯片内置温度传感器驱动头文件

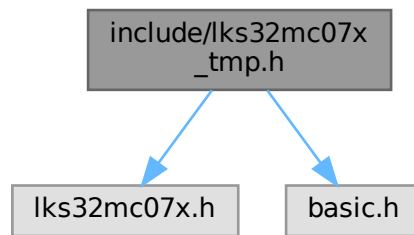
其它说明: 无

```

#include "lks32mc07x.h"
#include "basic.h"

```

lks32mc07x_tmp.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct [Stru_TempertureCof_Def](#)

芯片内置温度传感器初始化结构体句柄，初始化时请定义该句柄，并用其它地址来传参

函数

- void [TempSensor_Init](#) (void)
函数名称: [void TempSensor_Init\(void\)](#)
- s16 [GetCurrentTemperature](#) (s16 ADC_value)
函数名称: [s16 GetCurrentTemperature\(s16 ADC_value\)](#)

5.45.1 详细描述

文件名称: lks32mc08x_tmp.h

文件标识: 无

内容摘要: 芯片内置温度传感器驱动头文件

其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年3月9日	1.0	Zhu Jie	修改

修改日志:

在文件 [lks32mc07x.tmp.h](#) 中定义.

5.45.2 函数说明

5.45.2.1 GetCurrentTemperature()

```
s16 GetCurrentTemperature (
    s16 ADC_value ) [extern]
```

函数名称: **s16** [GetCurrentTemperature\(s16 ADC_value\)](#)

功能描述: 获取当前温度值

参见

被引用内容: 无

参数

输入参数:	ADC_value: ADC通道14为温度传感器, ADC采样结果值
输出参数:	无

返回

返回 值: **t.Temperture**: 当前温度值, 单位: 1个Lsb代表0.1度

注解

其它说明: **ADC_value**输入的ADC值不区分左右对齐, 函数内已经进行ADC结果左右对齐判断

警告

无

示例代码:

```
s16 temp_value = 0;
temp_value = GetCurrentTemperature (ADC0_DATA0); //获取当前温度值, 单位0.1度
```

Date	Version	Author	Description
2015年11月5日	1.0	Howlet	创建
2023年3月9日	2.0	Zhu Jie	修改

修改日志:

在文件 [lks32mc07x_tmp.c](#) 第 70 行定义.

5.45.2.2 TempSensor.Init()

```
void TempSensor.Init (  
    void ) [extern]
```

函数名称: void [TempSensor.Init\(void\)](#)

功能描述: 温度传感器初始化

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

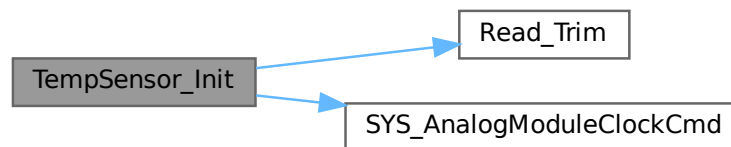
```
TempSensor.Init\(\); //温度传感器初始化
```

Date	Version	Author	Description
2015年11月5日	1.0	Howlet	创建
2023年3月9日	2.0	Zhu Jie	修改

修改日志:

在文件 [lks32mc07x_tmp.c](#) 第 39 行定义.

函数调用图:



5.46 lks32mc07x_tmp.h

[浏览该文件的文档.](#)

```

00001
00014 #ifndef _LKS32MC07X_TMP_H_
00015 #define _LKS32MC07X_TMP_H_
00016
00017 #include "lks32mc07x.h"
00018 #include "basic.h"
00019
00023 typedef struct
00024 {
00025     u16 nCofA;
00026     u16 nOffsetB;
00027 } Stru.TempertureCof_Def;
00028
00029 extern void TempSensor_Init(void);
00030 extern s16 GetCurrentTemperature(s16 ADC_value);
00031
00032 #endif /*_CONNIE_TMP_H_*/
00033
00034 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/
  
```

5.47 include/lks32mc07x_uart.h 文件参考

文件名称: [lks32mc07x_uart.h](#)

文件标识: 无

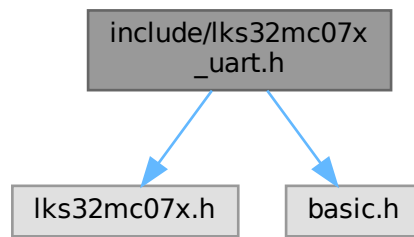
内容摘要: UART驱动头文件

其它说明: 无

```

#include "lks32mc07x.h"
#include "basic.h"
  
```

lks32mc07x_uart.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct **UART_TypeDef**
UART寄存器结构体句柄
- struct **UART_InitTypeDef**

宏定义

- #define **UART0** ((UART_TypeDef *)UART0_BASE)
串口0结构体基地址定义
- #define **UART1** ((UART_TypeDef *)UART1_BASE)
串口1结构体基地址定义
- #define **UART_WORDLENGTH_8b** 0
8bit数据位
- #define **UART_WORDLENGTH_9b** 1
9bit数据位
- #define **UART_STOPBITS_1b** 0
无停止位
- #define **UART_STOPBITS_2b** 1
1bit停止位
- #define **UART_FIRSTSEND_LSB** 0
低端先发送
- #define **UART_FIRSTSEND_MSB** 1
高端先发送
- #define **UART_IRQEna_SendOver** BIT0
使能发送完成中断

- `#define UART_IRQEna_RcvOver` BIT1
使能接收完成中断
- `#define UART_IRQEna_SendBuffEmpty` BIT2
使能发送缓冲区空中断
- `#define UART_IRQEna_StopError` BIT3
使能停止位错误
- `#define UART_IRQEna_CheckError` BIT4
使能校验错误
- `#define UART_IRQEna_TX_OV` BIT5
发送缓冲区溢出中断使能
- `#define UART_IRQEna_RX_OV` BIT6
接收缓冲区溢出中断使能
- `#define UART_IRQEna_LBD` BIT7
LIN break character 检测中断使能
- `#define UART_IRQEna_IDLE` BIT8
空闲帧中断使能
- `#define TX_DMA_RE` BIT0 /*发送完成DMA请求使能*/
- `#define RX_DMA_RE` BIT1 /*接收完成DMA请求使能*/
- `#define TX_BUF_DMA_RE` BIT2 /*发送缓冲区空DMA请求使能*/
- `#define UART_IF_SendOver` BIT0
发送完成中断标志
- `#define UART_IF_RcvOver` BIT1
接收完成中断标志
- `#define UART_IF_SendBufEmpty` BIT2
发送缓冲区空中断标志
- `#define UART_IF_StopError` BIT3
停止位错误中断标志
- `#define UART_IF_CheckError` BIT4
校验错误中断标志
- `#define UART_IF_TX_OV` BIT5
发送缓冲区溢出中断标志
- `#define UART_IF_RX_OV` BIT6
接收缓冲区溢出中断标志
- `#define UART_IF_LBD` BIT7
LIN break character 检测中断标志
- `#define UART_IF_IDLE` BIT8
空闲帧中断标志

枚举

- `enum UART_ParityMode` { `UART_Parity_NO` = 0x00 , `UART_Parity_EVEN` = 0x01 , `UART_Parity_ODD` = 0x02
}
串口奇偶校验配置枚举成员

函数

- void **UART_Init** (UART_TypeDef *UARTx, UART_InitTypeDef *UART_InitStruct)
函数名称: void *UART_Init(UART_TypeDef* UARTx, UART_InitTypeDef* UART_InitStruct)*
- void **UART_StructInit** (UART_InitTypeDef *UART_InitStruct)
函数名称: void *UART_StructInit(UART_InitTypeDef* UART_InitStruct)*
- void **UART_SendData** (UART_TypeDef *UARTx, uint32_t n)
函数名称: void *UART_SENDDATA(UART_TypeDef* UARTx, uint32_t n)*
- uint32_t **UART_ReadData** (UART_TypeDef *UARTx)
函数名称: uint32_t *UART_ReadData(UART_TypeDef* UARTx)*
- void **UART_ClearIRQFlag** (UART_TypeDef *UARTx, uint32_t tempFlag)
函数名称: void *UART_ClearIRQFlag(UART_TypeDef* UARTx, uint32_t tempFlag)*
- uint32_t **UART_GetIRQFlag** (UART_TypeDef *UARTx, uint32_t tempFlag)
函数名称: uint32_t *UART_GetIRQFlag(UART_TypeDef* UARTx, uint32_t tempFlag)*
- uint8_t **UART_SendAddr_485** (UART_TypeDef *UARTx, uint8_t n)
函数名称: uint8_t *UART_SendAddr_485(UART_TypeDef* UARTx, uint8_t n)*
- uint8_t **UART_SendData_485** (UART_TypeDef *UARTx, uint8_t n)
函数名称: uint8_t *UART_SendData_485(UART_TypeDef* UARTx, uint8_t n)*

5.47.1 详细描述

文件名称: lks32mc07x_uart.h

文件标识: 无

内容摘要: UART驱动头文件

其它说明: 无

版权所有

(C)2021, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 lks32mc07x_uart.h 中定义.

5.47.2 宏定义说明

5.47.2.1 RX_DMA_RE

```
#define RX_DMA_RE BIT1 /*接收完成DMA请求使能*/
```

在文件 lks32mc07x_uart.h 第 111 行定义.

5.47.2.2 TX_BUF_DMA_RE

```
#define TX_BUF_DMA_RE BIT2 /*发送缓冲区空DMA请求使能*/
```

在文件 [lks32mc07x_uart.h](#) 第 112 行定义.

5.47.2.3 TX_DMA_RE

```
#define TX_DMA_RE BIT0 /*发送完成DMA请求使能*/
```

在文件 [lks32mc07x_uart.h](#) 第 110 行定义.

5.47.2.4 UART0

```
#define UART0 ((UART_TypeDef *)UART0_BASE)
```

串口0结构体基地址定义

在文件 [lks32mc07x_uart.h](#) 第 49 行定义.

5.47.2.5 UART1

```
#define UART1 ((UART_TypeDef *)UART1_BASE)
```

串口1结构体基地址定义

在文件 [lks32mc07x_uart.h](#) 第 54 行定义.

5.47.2.6 UART_FIRSTSEND_LSB

```
#define UART_FIRSTSEND_LSB 0
```

低端先发送

在文件 [lks32mc07x_uart.h](#) 第 96 行定义.

5.47.2.7 UART_FIRSTSEND_MSB

```
#define UART_FIRSTSEND_MSB 1
```

高端先发送

在文件 [lks32mc07x_uart.h](#) 第 97 行定义.

5.47.2.8 UART_IF_CheckError

```
#define UART_IF_CheckError BIT4
```

校验错误中断标志

在文件 [lks32mc07x_uart.h](#) 第 119 行定义.

5.47.2.9 UART_IF_IDLE

```
#define UART_IF_IDLE BIT8
```

空闲帧中断标志

在文件 [lks32mc07x_uart.h](#) 第 123 行定义.

5.47.2.10 UART_IF_LBD

```
#define UART_IF_LBD BIT7
```

LIN break character 检测中断标志

在文件 [lks32mc07x_uart.h](#) 第 122 行定义.

5.47.2.11 UART_IF_RcvOver

```
#define UART_IF_RcvOver BIT1
```

接收完成中断标志

在文件 [lks32mc07x_uart.h](#) 第 116 行定义.

5.47.2.12 UART_IF_RX_OV

```
#define UART_IF_RX_OV BIT6
```

接收缓冲区溢出中断标志

在文件 [lks32mc07x_uart.h](#) 第 121 行定义.

5.47.2.13 UART_IF_SendBufEmpty

```
#define UART_IF_SendBufEmpty BIT2
```

发送缓冲区空中断标志

在文件 [lks32mc07x_uart.h](#) 第 117 行定义.

5.47.2.14 UART_IF_SendOver

```
#define UART_IF_SendOver BIT0
```

发送完成中断标志

在文件 [lks32mc07x_uart.h](#) 第 115 行定义.

5.47.2.15 UART_IF_StopError

```
#define UART_IF_StopError BIT3
```

停止位错误中断标志

在文件 [lks32mc07x_uart.h](#) 第 118 行定义.

5.47.2.16 UART_IF_TX_OV

```
#define UART_IF_TX_OV BIT5
```

发送缓冲区溢出中断标志

在文件 [lks32mc07x_uart.h](#) 第 120 行定义.

5.47.2.17 UART_IRQEna_CheckError

```
#define UART_IRQEna_CheckError BIT4
```

使能校验错误

在文件 [lks32mc07x_uart.h](#) 第 104 行定义.

5.47.2.18 UART_IRQEna_IDLE

```
#define UART_IRQEna_IDLE BIT8
```

空闲帧中断使能

在文件 [lks32mc07x_uart.h](#) 第 108 行定义.

5.47.2.19 UART_IRQEna_LBD

```
#define UART_IRQEna_LBD BIT7
```

LIN break character 检测中断使能

在文件 [lks32mc07x_uart.h](#) 第 107 行定义.

5.47.2.20 UART_IRQEna_RcvOver

```
#define UART_IRQEna_RcvOver BIT1
```

使能接收完成中断

在文件 [lks32mc07x_uart.h](#) 第 101 行定义.

5.47.2.21 UART_IRQEna_RX_OV

```
#define UART_IRQEna_RX_OV BIT6
```

接收缓冲区溢出中断使能

在文件 [lks32mc07x_uart.h](#) 第 106 行定义.

5.47.2.22 UART_IRQEna_SendBuffEmpty

```
#define UART_IRQEna_SendBuffEmpty BIT2
```

使能发送缓冲区空中断

在文件 [lks32mc07x_uart.h](#) 第 102 行定义.

5.47.2.23 UART_IRQEna_SendOver

```
#define UART_IRQEna_SendOver BIT0
```

使能发送完成中断

在文件 [lks32mc07x_uart.h](#) 第 100 行定义.

5.47.2.24 UART_IRQEna_StopError

```
#define UART_IRQEna_StopError BIT3
```

使能停止位错误

在文件 [lks32mc07x_uart.h](#) 第 103 行定义.

5.47.2.25 UART_IRQEna_TX_OV

```
#define UART_IRQEna_TX_OV BIT5
```

发送缓冲区溢出中断使能

在文件 [lks32mc07x_uart.h](#) 第 105 行定义.

5.47.2.26 UART_STOPBITS_1b

```
#define UART_STOPBITS_1b 0
```

无停止位

在文件 [lks32mc07x_uart.h](#) 第 93 行定义.

5.47.2.27 UART_STOPBITS_2b

```
#define UART_STOPBITS_2b 1
```

1bit停止位

在文件 [lks32mc07x_uart.h](#) 第 94 行定义.

5.47.2.28 UART_WORDLENGTH_8b

```
#define UART_WORDLENGTH_8b 0
```

8bit数据位

在文件 [lks32mc07x_uart.h](#) 第 90 行定义.

5.47.2.29 UART_WORDLENGTH_9b

```
#define UART_WORDLENGTH_9b 1
```

9bit数据位

在文件 [lks32mc07x_uart.h](#) 第 91 行定义.

5.47.3 枚举类型说明

5.47.3.1 UART_ParityMode

```
enum UART_ParityMode
```

串口奇偶校验配置枚举成员

枚举值

UART_Parity_NO	
UART_Parity_EVEN	
UART_Parity_ODD	

在文件 [lks32mc07x_uart.h](#) 第 22 行定义.

5.47.4 函数说明

5.47.4.1 UART_ClearIRQFlag()

```
void UART_ClearIRQFlag (
    UART_TypeDef * UARTx,
    uint32_t tempFlag )
```

函数名称: void **UART_ClearIRQFlag**(UART_TypeDef *UARTx, uint32_t nFlag)

功能描述: 清除UART中断标志

参见

被引用内容: [UART0](#)

参数

输入参数:	UART_TypeDef: UART0	
	tempFlag参数可选:	
	宏定义	说明
	UART_IF_SendOver	发送完成中断
	UART_IF_RcvOver	接收完成中断
	UART_IF_SendBufEmpty	发送缓冲区空中断
	UART_IF_StopError	停止位错误
	UART_IF_CheckError	校验错误
	UART_IF_TX_OV	发送缓冲区溢出中断
	UART_IF_RX_OV	接收缓冲区溢出中断
	UART_IF_LBD	LIN break character 检测中断
	UART_IF_IDLE	空闲帧中断
输出参数:	无	

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (UART_GetIRQFlag (UART0, UART_IF_SendOver)) //获取发送完成中断标志位
{
    UART_ClearIRQFlag (UART0, UART_IF_SendOver) //清除发送完成中断标志位
}
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 254 行定义.

5.47.4.2 UART_GetIRQFlag()

```
uint32_t UART_GetIRQFlag (  
    UART_TypeDef * UARTx,  
    uint32_t tempFlag )
```

函数名称: `uint32_t UART_GetIRQFlag(UART_TypeDef *UARTx , uint32_t tempFlag)`

功能描述: 获取UART中断标志

参见

被引用内容: UARTx

参数

输入参数:	UART_TypeDef: UARTx	
	tempFlag参数可选:	
	宏定义	说明
	UART_IF_SendOver	发送完成中断
	UART_IF_RcvOver	接收完成中断
	UART_IF_SendBufEmpty	发送缓冲区空中断
	UART_IF_StopError	停止位错误
	UART_IF_CheckError	校验错误
	UART_IF_TX_OV	发送缓冲区溢出中断
	UART_IF_RX_OV	接收缓冲区溢出中断
输出参数:	UART_IF_LBD	LIN break character 检测中断
	UART_IF_IDLE	空闲帧中断
	无	

返回

返回值: 无

注解

其它说明: 无

警告

只有对应中断使能后，改为才能读取，如果对应中断未使能，读取结果一直为0

示例代码:

```
if (UART_GetIRQFlag (UART0, UART_IF_SendOver)) //获取发送完成中断标志位
{
    UART_ClearIRQFlag (UART0, UART_IF_SendOver) //清除发送完成中断标志位
}
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 212 行定义.

5.47.4.3 UART_Init()

```
void UART_Init (
    UART_TypeDef * UARTx,
    UART_InitTypeDef * UART_InitStruct )
```

函数名称: void UART_Init(UART_TypeDef* UARTx, EUART_InitTypeDef* UART_InitStruct)

功能描述: UART初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	UART_TypeDef: UARTx , UART_InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明: 串口波特率计算使用UART.MCU.MCLK系统时钟进行计算，使用前需要确认系统时钟频率

警告

无

示例代码:

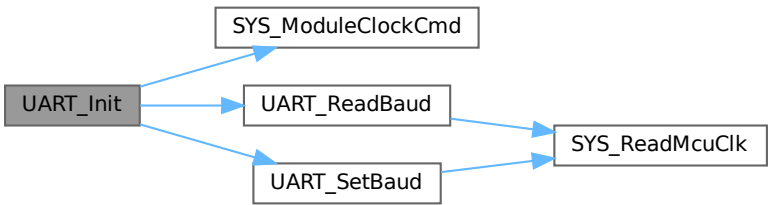
```
UART_InitTypeDef UART_InitStructure;
UART_StructInit(&UART_InitStructure);
UART_InitStructure.BaudRate = 9600; // 设置波特率9600
UART_InitStructure.WordLength= UART_WORDLENGTH_8b; // 发送数据长度8位
UART_InitStructure.StopBits = UART_STOPBITS_1b; // 停止位长度1位
UART_InitStructure.FirstSend = UART_FIRSTSEND_LSB; // 先发送LSB
UART_InitStructure.ParityMode= UART_Parity_NO; // 无奇偶校验
UART_InitStructure.RXD_INV = DISABLE; // RXD电平正常输出
UART_InitStructure.TXD_INV = DISABLE; // TXD电平正常输出
UART_InitStructure.IRQEna = UART_IRQEna_SendOver | UART_IRQEna_RcvOver;
UART_Init(UART0, &UART_InitStructure);
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 51 行定义.

函数调用图:



5.47.4.4 UART_ReadData()

```
uint32_t UART_ReadData (
    UART_TypeDef * UARTx )
```

函数名称: `uint32_t UART_ReadData(UART_TypeDef *UARTx)`

功能描述: UART读缓冲区数据

参见

被引用内容: [UART0](#)

参数

输入参数:	UART_TypeDef: UART0
输出参数:	无

返回

返回 值: 返回串口接收缓冲区的1Byte数据

注解

其它说明: 无

警告

无

示例代码:

```
u8 UART_BUFF = 0;
UART_BUFF = UART_ReadData (UART0); //读取串口0接收一字节数据
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 173 行定义.

5.47.4.5 UART_SendAddr_485()

```
uint8_t UART_SendAddr_485 (
    UART_TypeDef * UARTx,
    uint8_t n )
```

函数名称: `uint8_t UART_SendAddr_485(UART_TypeDef *UARTx, uint8_t n)`

功能描述: 发送485通讯中需要接收数据的从机地址

参见

被引用内容: UARTx

参数

输入参数:	UART_TypeDef: UARTx, n: 要发生1Byte地址
输出参数:	无

返回

返回值: 1: 发送失败, 0: 发送成功

注解

其它说明: cnt作用是延时防止程序卡死。在发送数据后立即发送地址的情况下,会判断等待发送缓冲区空在进行地址发送,如果长时间还未发送完成(cnt计数时间),则跳出while循环。如果串口波特率较低cnt计数已满,会出现数据还未发送完成进行下一个数据发送,为了避免此类问题发送可以增加cnt值。

警告

无

示例代码:

```
UART_SendAddr_485(UART0, 0x55); // 串口0发送485从机接收地址为0x55
```

Date	Version	Author	Description
2021年10月14日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 281 行定义.

5.47.4.6 UART_SendData()

```
void UART_SendData (
    UART_TypeDef * UARTx,
    uint32_t n )
```

函数名称: void UART_SENDDATA(UART_TypeDef *UARTx, uint32_t n)

功能描述: UART发送数据

参见

被引用内容: UARTx

参数

输入参数:	UART_TypeDef: UARTx, n: 要发生1Byte数据
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
UART_SendData(UART0, 0x12); //串口0发送0x12一字节数据
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 148 行定义.

5.47.4.7 UART_SendData_485()

```
uint8_t UART_SendData_485 (  
    UART_TypeDef * UARTx,  
    uint8_t n )
```

函数名称: `uint8_t UART_SendData_485(UART_TypeDef *UARTx, uint8_t n)`

功能描述: 发送485通讯中的数据信息

参见

被引用内容: UARTx

参数

输入参数:	UART_TypeDef: UARTx, n: 要发生1Byte数据
输出参数:	无

返回

返回 值: 1: 发送失败, 0: 发送成功

注解

其它说明: cnt作用是延时防止程序卡死。在发送数据后立即发送地址的情况下,会判断等待发送缓冲区空在进行地址发送,如果长时间还未发送完成(cnt计数时间),则跳出while循环。如果串口波特率较低cnt计数已满,会出现数据还未发送完成进行下一个数据发送,为了避免此类问题发送可以增加cnt值。

警告

无

示例代码:

```
UART_SendData_485(UART0,0x12); //串口0发送485数据为0x12
```

Date	Version	Author	Description
2021年10月14日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 324 行定义.

5.47.4.8 UART_StructInit()

```
void UART_StructInit (  
    UART_InitTypeDef * UART_InitStruct )
```

函数名称: void [UART_StructInit\(UART_InitTypeDef* UART_InitStruct\)](#)

功能描述: UART结构体初始化函数

参见

被引用内容: 无

参数

输入参数:	UART_InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
UART_InitTypeDef UART_InitStructure;  
UART_StructInit(&UART_InitStructure); //初始化UART相关配置结构体变量
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 111 行定义.

5.48 lks32mc07x_uart.h

[浏览该文件的文档.](#)

```
00001  
00014 #ifndef __lks32mc07x_UART_H  
00015 #define __lks32mc07x_UART_H  
00016  
00017 /* Includes -----*/  
00018 #include "lks32mc07x.h"  
00019 #include "basic.h"  
00020  
00022 typedef enum  
00023 {  
00024     UART_Parity_NO    = 0x00,  
00025     UART_Parity_EVEN  = 0x01,  
00026     UART_Parity_ODD   = 0x02  
00027 } UART_ParityMode;  
00028  
00032 typedef struct  
00033 {  
00034     __IO uint32_t CTRL;  
00035     __IO uint32_t DIVH;  
00036     __IO uint32_t DIVL;
```

```

00037     __IO uint32_t  BUFF;
00038     __IO uint32_t  ADP;
00039     __IO uint32_t  STT;
00040     __IO uint32_t  RE;
00041     __IO uint32_t  IE;
00042     __IO uint32_t  IF;
00043     __IO uint32_t  IOC;
00044 } UART_TypeDef;
00045
00049 #define UART0 ((UART_TypeDef *)UART0_BASE)
00050
00054 #define UART1 ((UART_TypeDef *)UART1_BASE)
00055
00056 typedef struct
00057 {
00058     uint32_t BaudRate;
00063     uint8_t WordLength;
00068     uint8_t StopBits;
00073     uint8_t FirstSend;
00077     UART_ParityMode ParityMode;
00078
00079     uint8_t MultiDropEna;
00080     uint8_t Bit9Value;
00081     uint8_t DUPLEX;
00083     uint16_t Match485Addr;
00084     uint16_t IRQEna;
00085     uint8_t DMARE;
00086     uint8_t RXD_INV;
00087     uint8_t TXD_INV;
00088 } UART_InitTypeDef;
00089
00090 #define UART_WORDLENGTH_8b 0
00091 #define UART_WORDLENGTH_9b 1
00093 #define UART_STOPBITS_1b 0
00094 #define UART_STOPBITS_2b 1
00096 #define UART_FIRSTSEND_LSB 0
00097 #define UART_FIRSTSEND_MSB 1
00099 /*中断使能定义*/
00100 #define UART_IRQEna_SendOver      BIT0
00101 #define UART_IRQEna_RcvOver      BIT1
00102 #define UART_IRQEna_SendBufEmpty BIT2
00103 #define UART_IRQEna_StopError    BIT3
00104 #define UART_IRQEna_CheckError   BIT4
00105 #define UART_IRQEna_TX_OV        BIT5
00106 #define UART_IRQEna_RX_OV        BIT6
00107 #define UART_IRQEna_LBD          BIT7
00108 #define UART_IRQEna_IDLE         BIT8
00110 #define TX_DMA_RE                 BIT0 /*发送完成DMA请求使能*/
00111 #define RX_DMA_RE                 BIT1 /*接收完成DMA请求使能*/
00112 #define TX_BUF_DMA_RE             BIT2 /*发送缓冲区空DMA请求使能*/
00113
00114 /*中断标志定义*/
00115 #define UART_IF_SendOver          BIT0
00116 #define UART_IF_RcvOver          BIT1
00117 #define UART_IF_SendBufEmpty     BIT2
00118 #define UART_IF_StopError        BIT3
00119 #define UART_IF_CheckError       BIT4
00120 #define UART_IF_TX_OV            BIT5
00121 #define UART_IF_RX_OV            BIT6
00122 #define UART_IF_LBD              BIT7
00123 #define UART_IF_IDLE             BIT8
00125 void UART_Init(UART_TypeDef *UARTx, UART_InitTypeDef *UART_InitStruct);
00126 void UART_StructInit(UART_InitTypeDef *UART_InitStruct);
00127
00128 void UART_SendData(UART_TypeDef *UARTx, uint32_t n);
00129 uint32_t UART_ReadData(UART_TypeDef *UARTx);
00130
00131 void UART_ClearIRQFlag(UART_TypeDef *UARTx, uint32_t tempFlag);
00132
00133 uint32_t UART_GetIRQFlag(UART_TypeDef *UARTx, uint32_t tempFlag);
00134
00135 uint8_t UART_SendAddr_485(UART_TypeDef *UARTx, uint8_t n);
00136 uint8_t UART_SendData_485(UART_TypeDef *UARTx, uint8_t n);
00137
00138 #endif /*_lks32mc07x_UART_H */
00139
00140 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/

```

5.49 include/lks32mc07x_wake.h 文件参考

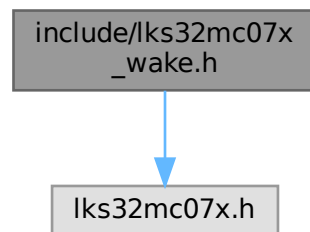
文件名称: [lks32mc07x_wake.h](#)

文件标识: 无

内容摘要: **wake**驱动头文件
其它说明: 无

```
#include "lks32mc07x.h"
```

lks32mc07x_wake.h 的引用(Include)关系图:



此图展示该文件直接或间接的被哪些文件引用了:



结构体

- struct **WAKE_InitTypeDef**
低功耗时钟配置中间变量

宏定义

- #define **SYS_WakeInterval_025s** 0x00
休眠唤醒间隔时间0.25s
- #define **SYS_WakeInterval_05s** 0x01
休眠唤醒间隔时间0.5s
- #define **SYS_WakeInterval_1s** 0x02
休眠唤醒间隔时间1s
- #define **SYS_WakeInterval_2s** 0x03
休眠唤醒间隔时间2s
- #define **SYS_WakeInterval_4s** 0x04
休眠唤醒间隔时间4s
- #define **SYS_WakeInterval_8s** 0x05
休眠唤醒间隔时间8s
- #define **SYS_WakeInterval_16s** 0x06
休眠唤醒间隔时间16s
- #define **SYS_WakeInterval_32s** 0x07

- 休眠唤醒间隔时间32s
- #define **IOWK_FLT_EN** 0x02
使能IO滤波
- #define **IOWK_FLT_DI** 0x00
关闭IO滤波
- #define **WAKEIO_P0.0** BIT0
P0.0 作为唤醒IO
- #define **WAKEIO_P0.2** BIT1
P0.2 作为唤醒IO
- #define **WAKEIO_P0.6** BIT2
P0.6 作为唤醒IO
- #define **WAKEIO_P0.11** BIT3
P0.11作为唤醒IO
- #define **WAKEIO_P0.14** BIT4
P0.14作为唤醒IO
- #define **WAKEIO_P2.4** BIT5
P2.4 作为唤醒IO
- #define **WAKEIO_P2.7** BIT6
P2.7 作为唤醒IO
- #define **WAKEIO_P2.15** BIT7
P2.15 作为唤醒IO
- #define **WAKEIO_CLUOUT0** BIT8
CLUOUT0 作为唤醒使能
- #define **WAKEIO_CLUOUT1** BIT9
CLUOUT1 作为唤醒使能
- #define **WAKEIO_CLUOUT2** BIT10
CLUOUT2 作为唤醒使能
- #define **WAKEIO_CLUOUT3** BIT11
CLUOUT3 作为唤醒使能

枚举

- enum **enumWakeTrigLevel** { **WAKETRIG_LOW** , **WAKETRIG_HIGH** }
低功耗IO唤醒电平枚举成员

函数

- void **SYS_EVTCLRSleep** (void)
函数名称: *void SYS_EVTCLRSleep(void)*
- void **SYS_FallSleep** (void)
函数名称: *void SYS_FallSleep(void)*
- void **SYS_SetWakeInterval** (uint32_t rth, uint32_t wth, uint8_t ena)
函数名称: *void SYS_SetWakeInterval(uint32_t rth , uint32_t wth , uint8_t ena)*
- void **Switch2PLL** (void)
函数名称: *void Switch2PLL(void)*
- void **Switch2HRC** (void)
函数名称: *void Switch2HRC(void)*
- void **SetWakeIO** (uint32_t wakeIO, enumWakeTrigLevel trigLevel, uint32_t flt, uint8_t ena)
函数名称: *void SetWakeIO(uint32_t wakeIO, enumWakeTrigLevel trigLevel, uint32_t flt , uint8_t ena)*
- void **SYS_EnterSleep** (void)
系统休眠，需要配置唤醒源（io唤醒或定时唤醒），以及定义 **WAKE_IRQHandler** 函数

5.49.1 详细描述

文件名称: [lks32mc07x_wake.h](#)

文件标识: 无

内容摘要: wake驱动头文件

其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.h](#) 中定义.

5.49.2 宏定义说明

5.49.2.1 IOWK_FLT_DI

```
#define IOWK_FLT_DI 0x00
```

关闭IO滤波

在文件 [lks32mc07x_wake.h](#) 第 50 行定义.

5.49.2.2 IOWK_FLT_EN

```
#define IOWK_FLT_EN 0x02
```

使能IO滤波

在文件 [lks32mc07x_wake.h](#) 第 49 行定义.

5.49.2.3 SYS_WakeInterval_025s

```
#define SYS_WakeInterval_025s 0x00
```

休眠唤醒间隔时间0.25s

在文件 [lks32mc07x_wake.h](#) 第 39 行定义.

5.49.2.4 SYS_WakeInterval_05s

```
#define SYS_WakeInterval_05s 0x01
```

休眠唤醒间隔时间0.5s

在文件 [lks32mc07x_wake.h](#) 第 40 行定义.

5.49.2.5 SYS_WakeInterval_16s

```
#define SYS_WakeInterval_16s 0x06
```

休眠唤醒间隔时间16s

在文件 [lks32mc07x_wake.h](#) 第 45 行定义.

5.49.2.6 SYS_WakeInterval_1s

```
#define SYS_WakeInterval_1s 0x02
```

休眠唤醒间隔时间1s

在文件 [lks32mc07x_wake.h](#) 第 41 行定义.

5.49.2.7 SYS_WakeInterval_2s

```
#define SYS_WakeInterval_2s 0x03
```

休眠唤醒间隔时间2s

在文件 [lks32mc07x_wake.h](#) 第 42 行定义.

5.49.2.8 SYS_WakeInterval_32s

```
#define SYS_WakeInterval_32s 0x07
```

休眠唤醒间隔时间32s

在文件 [lks32mc07x_wake.h](#) 第 46 行定义.

5.49.2.9 SYS_WakeInterval_4s

```
#define SYS_WakeInterval_4s 0x04
```

休眠唤醒间隔时间4s

在文件 [lks32mc07x_wake.h](#) 第 43 行定义.

5.49.2.10 SYS.WakeInterval_8s

```
#define SYS.WakeInterval_8s 0x05
```

休眠唤醒间隔时间8s

在文件 [lks32mc07x_wake.h](#) 第 44 行定义.

5.49.2.11 WAKEIO.CLUOUT0

```
#define WAKEIO.CLUOUT0 BIT8
```

CLUOUT0 作为唤醒使能

在文件 [lks32mc07x_wake.h](#) 第 61 行定义.

5.49.2.12 WAKEIO.CLUOUT1

```
#define WAKEIO.CLUOUT1 BIT9
```

CLUOUT1 作为唤醒使能

在文件 [lks32mc07x_wake.h](#) 第 62 行定义.

5.49.2.13 WAKEIO.CLUOUT2

```
#define WAKEIO.CLUOUT2 BIT10
```

CLUOUT2 作为唤醒使能

在文件 [lks32mc07x_wake.h](#) 第 63 行定义.

5.49.2.14 WAKEIO.CLUOUT3

```
#define WAKEIO.CLUOUT3 BIT11
```

CLUOUT3 作为唤醒使能

在文件 [lks32mc07x_wake.h](#) 第 64 行定义.

5.49.2.15 WAKEIO.P0_0

```
#define WAKEIO.P0_0 BIT0
```

P0.0 作为唤醒IO

在文件 [lks32mc07x_wake.h](#) 第 53 行定义.

5.49.2.16 WAKEIO_P0_11

```
#define WAKEIO_P0_11 BIT3
```

P0.11作为唤醒IO

在文件 [lks32mc07x_wake.h](#) 第 56 行定义.

5.49.2.17 WAKEIO_P0_14

```
#define WAKEIO_P0_14 BIT4
```

P0.14作为唤醒IO

在文件 [lks32mc07x_wake.h](#) 第 57 行定义.

5.49.2.18 WAKEIO_P0_2

```
#define WAKEIO_P0_2 BIT1
```

P0.2 作为唤醒IO

在文件 [lks32mc07x_wake.h](#) 第 54 行定义.

5.49.2.19 WAKEIO_P0_6

```
#define WAKEIO_P0_6 BIT2
```

P0.6 作为唤醒IO

在文件 [lks32mc07x_wake.h](#) 第 55 行定义.

5.49.2.20 WAKEIO_P2_15

```
#define WAKEIO_P2_15 BIT7
```

P2.15 作为唤醒IO

在文件 [lks32mc07x_wake.h](#) 第 60 行定义.

5.49.2.21 WAKEIO_P2_4

```
#define WAKEIO_P2_4 BIT5
```

P2.4 作为唤醒IO

在文件 [lks32mc07x_wake.h](#) 第 58 行定义.

5.49.2.22 WAKEIO_P2_7

```
#define WAKEIO_P2_7 BIT6
```

P2.7 作为唤醒IO

在文件 [lks32mc07x_wake.h](#) 第 59 行定义.

5.49.3 枚举类型说明

5.49.3.1 enumWakeTrigLevel

```
enum enumWakeTrigLevel
```

低功耗IO唤醒电平枚举成员

枚举值

WAKETRIG_LOW	
WAKETRIG_HIGH	输入IO低电平唤醒

在文件 [lks32mc07x_wake.h](#) 第 32 行定义.

5.49.4 函数说明

5.49.4.1 SetWakeup()

```
void SetWakeup (
    uint32_t wakeup,
    enumWakeTrigLevel trigLevel,
    uint32_t flt,
    uint8_t ena )
```

函数名称: void [SetWakeup\(uint32_t wakeup, enumWakeTrigLevel trigLevel, uint32_t flt , uint8_t ena\)](#)

功能描述: 设置休眠唤醒IO

参见

唤醒电平选择枚举: [enumWakeTrigLevel](#)

参数

输入参数:	wakeup = BIT11: CLUOUT3_EN CLUOUT3 作为唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT10: CLUOUT2_EN CLUOUT2 作为唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT9: CLUOUT1_EN CLUOUT1 作为唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT8: CLUOUT0_EN CLUOUT0 作为唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT7: GPIO 2[15] / P2[4] 外部唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT6: GPIO 2[7] / P2[4] 外部唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT5: GPIO 2[4] / P2[4] 外部唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT4: GPIO 0[14] / P0[14] 外部唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT3: GPIO 0[11] / P0[11] 外部唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT2: GPIO 0[6] / P0[6] 外部唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT1: GPIO 0[2] / P0[2] 外部唤醒使能。1: 使能; 0: 禁用。 wakeup = BIT0: GPIO 0[0] / P0[0] 外部唤醒使能。1: 使能; 0: 禁用。 enumWakeTrigLevel : 唤醒电平选择: 0:低电平唤醒 1:高电平唤醒 flt: ENABLE: 开启IO滤波 DISABLE: 关闭IO滤波 ena: ENABLE: 开启IO唤醒 DISABLE: 关闭IO唤醒
输出参数:	无

返回

返回 值： 无

注解

其它说明： 低功耗IO可以使用多个， 但所有IO唤醒信号使用同一个极性选择。

警告

无

示例代码:

```
SetWakeIO(WAKEIO.P0.0,WAKETRIG.LOW,ENABLE,ENABLE); //系统休眠时,使能P0.0作为低功耗唤醒IO.
低电平唤醒
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 161 行定义.

5.49.4.2 Switch2HRC()

```
void Switch2HRC (
    void )
```

函数名称: void [Switch2HRC\(void\)](#)

功能描述: 休眠唤醒前一刻， 关闭高速时钟与BGP和掉电检测电路函数

参见

被调用函数： 无

参数

输入参数:	无
输出参数:	无

返回

返回 值： 无

注解

其它说明： 系统低功耗休眠使用

警告

无

示例代码:

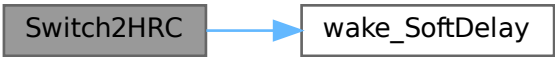
```
Switch2HRC();
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 103 行定义.

函数调用图:



5.49.4.3 Switch2PLL()

```
void Switch2PLL (
    void )
```

函数名称: void [Switch2PLL\(void\)](#)

功能描述: 休眠唤醒后，系统时钟切换到PLL时钟函数

参见

被调用函数： 无

参数

输入参数:	无
输出参数:	无

返回

返回值：无

注解

其它说明： 系统低功耗唤醒使用

警告

无

示例代码:

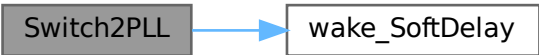
```
Switch2PLL();
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 71 行定义.

函数调用图:



5.49.4.4 SYS_EnterSleep()

```
void SYS_EnterSleep (
    void )
```

系统休眠，需要配置唤醒源（io唤醒或定时唤醒），以及定义WAKE_IRQHandler函数

在文件 [lks32mc07x_wake.c](#) 第 307 行定义.

5.49.4.5 SYS_EVTCLRSleep()

```
void SYS_EVTCLRSleep (  
    void )
```

函数名称: void [SYS_EVTCLRSleep\(void\)](#)

功能描述: 清除休眠全部事件记录

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
SYS\_EVTCLRSleep\(\) ;
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 248 行定义.

5.49.4.6 SYS_FallSleep()

```
void SYS_FallSleep (  
    void )
```

函数名称: void [SYS.FallSleep\(void\)](#)

功能描述: 触发系统进入休眠状态，必须定义WAKE_IRQHandler否则芯片无法正常唤醒

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
SYS.FallSleep\(\) ;
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 210 行定义.

5.49.4.7 [SYS.SetWakeInterval\(\)](#)

```
void SYS_SetWakeInterval (
    uint32_t rth,
    uint32_t wth,
    uint8_t ena )
```

函数名称: void [SYS.SetWakeInterval\(uint32_t rth , uint32_t wth , uint8_t ena\)](#)

功能描述: 设置休眠唤醒周期

参见

被调用函数： 无

参数

输入参数:	rth	看门狗超时复位门限值，也是重新装载值(最大值0x1FF000)
	wth	看门狗定时唤醒门限值(最大值小于rth)
	ena	ENABLE: 开启定时唤醒 DISABLE: 关闭定时唤醒
输出参数:	无	

返回

返回 值： 无

注解

其它说明： 唤醒时间计算公式： $WakeupIWDG = tLRC * (rth - lwth)$ tLRC 为 LRC时钟周期， $1/32kHz=31.25\mu S$

$rth - wth * (tLRC) =$

唤醒时间

$8000 * 31.25\mu S =$

$0.25S$

$16000 * 31.25\mu S =$

$0.5S$

$32000 * 31.25\mu S =$

$1S$

$64000 * 31.25\mu S =$

$2S$

$128000 * 31.25\mu S =$

$4S$

警告

无

示例代码:

```
SetWakeIO(WAKEIO_P0_0,WAKETRIG_LOW,ENABLE,ENABLE); //系统休眠时,使能P0.0作为低功耗唤醒IO.
低电平唤醒
```


Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 289 行定义.

5.50 lks32mc07x_wake.h

浏览该文件的文档.

```

00001
00015 #ifndef __lks32mc07x_WAKE__
00016 #define __lks32mc07x_WAKE__
00017 #include "lks32mc07x.h"
00018
00022 typedef struct
00023 {
00024     uint32_t AFE_REG5_RECORD;
00025     uint32_t CLK_CFG_RECORD;
00026
00027 } WAKE_InitTypeDef;
00028
00032 typedef enum
00033 {
00034     WAKETRIG_LOW,
00035     WAKETRIG_HIGH,
00036 } enumWakeTrigLevel;
00037
00038 /*休眠唤醒间隔时间*/
00039 #define SYS.WakeInterval.025s 0x00
00040 #define SYS.WakeInterval.05s 0x01
00041 #define SYS.WakeInterval.1s 0x02
00042 #define SYS.WakeInterval.2s 0x03
00043 #define SYS.WakeInterval.4s 0x04
00044 #define SYS.WakeInterval.8s 0x05
00045 #define SYS.WakeInterval.16s 0x06
00046 #define SYS.WakeInterval.32s 0x07
00048 /*低功耗唤醒IO滤波*/
00049 #define IOWK_FLT_EN 0x02
00050 #define IOWK_FLT_DI 0x00
00052 /*低功耗唤醒IO选择*/
00053 #define WAKEIO_P0_0 BIT0
00054 #define WAKEIO_P0_2 BIT1
00055 #define WAKEIO_P0_6 BIT2
00056 #define WAKEIO_P0_11 BIT3
00057 #define WAKEIO_P0_14 BIT4
00058 #define WAKEIO_P2_4 BIT5
00059 #define WAKEIO_P2_7 BIT6
00060 #define WAKEIO_P2_15 BIT7
00061 #define WAKEIO_CLUOUT0 BIT8
00062 #define WAKEIO_CLUOUT1 BIT9
00063 #define WAKEIO_CLUOUT2 BIT10
00064 #define WAKEIO_CLUOUT3 BIT11
00066 /*休眠、唤醒*/
00067 void SYS.EVTCLRSleep(void);
00068 void SYS.FallSleep(void);
00069 void SYS.SetWakeInterval(uint32_t rth, uint32_t wth, uint8_t ena);
00070 void Switch2PLL(void);
00071 void Switch2HRC(void);
00072 void SetWakeIO(uint32_t wakeIO, enumWakeTrigLevel trigLevel, uint32_t flt, uint8_t ena);
00073 void SYS.EnterSleep(void);
00074
00075 #endif

```

5.51 source/lks32mc07x_adc.c 文件参考

文件名称: [lks32mc07x_adc.c](#)

文件标识: 无

内容摘要: ADC外设驱动程序
其它说明: 无

```
#include "lks32mc07x_adc.h"
#include "lks32mc07x_sys.h"
lks32mc07x_adc.c 的引用(Include)关系图:
```



函数

- void **ADC_Init** (ADC_TypeDef *ADCx, ADC_InitTypeDef *this)
 函数名称: void *ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct)*
- void **ADC_StructInit** (ADC_InitTypeDef *this)
 函数名称: void *ADC_StructInit(ADC_InitTypeDef* ADC_InitStruct)*
- void **ADC_ClearIRQFlag** (ADC_TypeDef *ADCx, uint16_t INT_flag)
 函数名称: void *ADC_ClearIRQFlag(ADC_TypeDef* ADCx, uint16_t INT_flag)*
- u16 **ADC_GetIRQFlag** (ADC_TypeDef *ADCx, uint16_t INT_flag)
 函数名称: uint16_t *ADC_GetIRQFlag(ADC_TypeDef* ADCx, uint16_t INT_flag)*
- void **ADC_CHN_GAIN_CFG** (ADC_TypeDef *ADCx, CHNx CHNum, u8 nChannel, ADC_GAINx GAIN3V6_↔ or_7V2)
 函数名称: void *ADC_CHN_GAIN_CFG(ADC_TypeDef* ADCx, CHNx CHNum, u8 passageway, ADC_GAINx GAIN3V6_or_7V2)*
- s16 **ADC_GetConversionValue** (ADC_TypeDef *ADCx, DATx DATNum)
 函数名称: s16 *ADC_GetConversionValue(ADC_TypeDef* ADCx, DATx DATNum)*
- void **ADC_SoftTrgEN** (ADC_TypeDef *ADCx, FuncState state)
 函数名称: void *ADC_SoftTrgEN(ADC_TypeDef* ADCx, FuncState state)*

5.51.1 详细描述

文件名称: lks32mc07x_adc.c
文件标识: 无
内容摘要: ADC外设驱动程序
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月27日	1.0	ZhuJie	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 中定义.

5.51.2 函数说明

5.51.2.1 ADC_CHN_GAIN_CFG()

```
void ADC_CHN_GAIN_CFG (
    ADC_TypeDef * ADCx,
    CHNx CHNum,
    u8 nChannel,
    ADC_GAINx GAIN3V6_or_7V2 )
```

函数名称: void [ADC.CHN_GAIN_CFG](#)([ADC_TypeDef](#) *[ADCx](#),[CHNx](#) [CHNum](#),[u8](#) [passageway](#), [ADC_GAINx](#) [GAIN3V6_or_7V2](#))

功能描述: ADC采样通道设置

参见

被引用内容: [CHNx](#)

参数

输入参数:	ADCx , 07系列有ADC0和ADC1 CHNum 为枚举类型, 对应0-15; Channel.0 为采样通道序号对应0 nChannel : ADC.CHANNEL.0 ~ ADC.CHANNEL.15 通道选择 ADC_GAINx : ADC_GAIN3V6 , ADC_GAIN7V2 量程选择
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
ADC\_CHN\_GAIN\_CFG (ADC0, CHN0, ADC.CHANNEL.9, ADC\_GAIN3V6) ; //配置ADC0第0次采样ADC通道9的信号, 选择量程为3.6V
ADC\_CHN\_GAIN\_CFG (ADC1, CHN1, ADC.CHANNEL.9, ADC\_GAIN7V2) ; //配置ADC1第1次采样ADC通道9的信号, 选择量程为7.2V
```

Date	Version	Author	Description
2023年03月9日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 222 行定义.

5.51.2.2 ADC_ClearIRQFlag()

```
void ADC_ClearIRQFlag (
    ADC_TypeDef * ADCx,
    uint16_t INT_flag )
```

函数名称: void [ADC_ClearIRQFlag\(ADC_TypeDef* ADCx, uint16_t INT_flag\)](#)

功能描述: ADC中断标志位清零

参见
被引用内容: 无

参数

输入参数:	ADCx:	ADC0
	INT_flag:	ADC中断标志
	ADC_ISR_IF	空闲采样完成中断标志
	ADC_HERR_IF	硬件触发发生在非空闲状态中断标志
	ADC_SERR_IF	软件触发发生在非空闲状态中断标志
	ADC_AWD0_IF	阈值监测 0 超限中断标志
	ADC_SF2_IF	第二段扫描结束中断标志
	ADC_SF1_IF	第一段扫描结束中断标志
	ADC_ALL_IF	所有ADC中断标志
输出参数:	无	

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
if (ADC_GetIRQFlag (ADC0, ADC_EOS0_IRQ_IF) ) // 获取第一段扫描结束中断标志
{
    ADC_ClearIRQFlag (ADC0, ADC_EOS0_IRQ_IF) // 第一段扫描结束中断标志
}
```

Date	Version	Author	Description
2023年02月27日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 148 行定义.

5.51.2.3 ADC_GetConversionValue()

```
s16 ADC_GetConversionValue (
    ADC_TypeDef * ADCx,
    DATx DATNum )
```

函数名称: s16 [ADC_GetConversionValue\(ADC_TypeDef *ADCx,DATx DATNum\)](#)

功能描述: 读ADC采样值

参见
被引用内容: ADCx: ADC0或ADC1

参数

输入参数:	DATx: 读取第几次采样结果值，取值为0-15
输出参数:	无

返回

返回值: 对应x次采样结果值

注解

其它说明: 无

警告

无

示例代码:

```
s16 ADC_samp0 = 0;
ADC_samp0 = ADC_GetConversionValue(ADC0,DAT3); //读取ADC0第3次采样结果
```

Date	Version	Author	Description
2023年02月27日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 260 行定义.

5.51.2.4 ADC_GetIRQFlag()

```
u16 ADC_GetIRQFlag (
    ADC_TypeDef * ADCx,
    uint16_t INT_flag )
```

函数名称: `uint16_t ADC_GetIRQFlag(ADC_TypeDef* ADCx, uint16_t INT_flag)`

功能描述: 获取ADC中断标志

参见

被引用内容: 无

参数

输入参数:	ADCx:	ADC0
	INT_flag:	ADC中断标志
	ADC_ISF_IF	空闲采样完成中断标志
	ADC_HERR_IF	硬件触发发生在非空闲状态中断标志
	ADC_SERR_IF	软件触发发生在非空闲状态中断标志
	ADC_AWD0_IF	阈值监测 0 超限中断标志
	ADC_SF2_IF	第二段扫描结束中断标志
	ADC_SF1_IF	第一段扫描结束中断标志
输出参数:	无	

返回

返回 值： 读对应标志位是否为1，1位置位，0为未置位

注解

其它说明： 无

警告

只有对应中断使能后，改为才能读取，如果对应中断未使能，读取结果一直为0

示例代码:

```
if (ADC_GetIRQFlag (ADC0,ADC_SF1_IF)) //获取第一段扫描结束中断标志
{
    ADC_ClearIRQFlag (ADC0,ADC_SF1_IF) //第一段扫描结束中断标志
}
```

Date	Version	Author	Description
2023年02月27日	1.0	Zhu Jie	创建

修改日志:

在文件 lks32mc07x_adc.c 第 184 行定义.

5.51.2.5 ADC_Init()

```
void ADC_Init (
    ADC_TypeDef * ADCx,
    ADC_InitTypeDef * this )
```

函数名称: void ADC_Init(ADC_TypeDef* ADCx, ADC_InitTypeDef* ADC_InitStruct)

功能描述: ADC初始化函数

参见

被引用内容: SYS_AnalogModuleClockCmd()

参数

输入参数:	ADCx: ADC0 , ADC_InitTypeDef *ADC_InitTypeDef
输出参数:	无

返回

返回值： 无

注解

其它说明： 无

警告

无

示例代码:

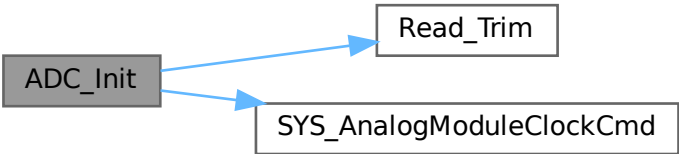
```
ADC_InitTypeDef ADC_InitStructure;
ADC_StructInit(&ADC_InitStructure); // 初始化结构体
ADC_InitStructure.ADC_CLK_DIV = ADC_Clock_48M; // ADC时钟选择48MHz
ADC_InitStructure.Align = ADC_LEFTALIGN; // ADC数据输出左对齐
ADC_InitStructure.Trigger_Mode = ADC_2SEG_TRG; // 设置ADC转换模式为双段式采样
ADC_InitStructure.FirSeg_Ch = ADC_2TIMES_SAMPLE; // 第一段共采样2个通道
ADC_InitStructure.SecSeg_Ch = ADC_2TIMES_SAMPLE; // 第二段共采样2个通道
ADC_InitStructure.ThrSeg_Ch = 0; // 第三段共采样0个通道
ADC_InitStructure.FourSeg_Ch = 0; // 第四段共采样0个通道
ADC_InitStructure.RefVol = RefVol_2V4; // ADC基准电压2.4V
ADC_InitStructure.Gain0 = 0x0; // ADC_DAT0-ADC_DAT9 (BIT0~BIT9) 采样保持电
路增益设置2/3
ADC_InitStructure.Gain1 = 0x0; // ADC_DAT10-ADC_DAT19 (BIT10~BIT19) 采样保
持电路增益设置2/3
ADC_InitStructure.Con_Sample = DISABLE; // 关闭连续采样
// 打开 UTIMER T0 T1 硬件触发采样
ADC_InitStructure.MCPWM_Trigger_En = ADC_MCPWM_T0_TRG|ADC_MCPWM_T1_TRG; // 开启UTIMER硬件第一段和第二段触发采样
ADC_InitStructure.UTIMER_Trigger_En = DISABLE; // 关闭UTIMER硬件触发采样
ADC_InitStructure.Trigger_Cnt = 0; // 发生 (Trigger_Cnt+1) 次触发事件执行一段转换
动作
ADC_InitStructure.IE = ADC_EOS0_IRQ_EN | ADC_EOS1_IRQ_EN; // ADC第一、二段采样结束中断使能
ADC_Init(ADC0, &ADC_InitStructure);
```

Date	Version	Author	Description
2015年11月26日	1.0	Howlet Li	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 54 行定义.

函数调用图:



5.51.2.6 ADC_SoftTrgEN()

```
void ADC_SoftTrgEN (
    ADC_TypeDef * ADCx,
    FuncState state )
```

函数名称: void ADC_SoftTrgEN(ADC_TypeDef* ADCx, FuncState state)

功能描述: ADC采样软件触发使能

参见

被引用内容: FuncState

参数

输入参数:	ADCx, 08系列固定选择ADC0 state: ENABLE触发, DISABLE不触发
输出参数:	无

返回

返回值: 无

注解

其它说明: ADC0_SWT写0x5AA5仅触发一次, 完成后自动清零, 等待下次软件触发

警告

无

示例代码:

```
ADC_SoftTrgEN(ADC0, ENABLE); //软件触发一次ADC采样
```

Date	Version	Author	Description
2023年02月27日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 353 行定义.

5.51.2.7 ADC_StructInit()

```
void ADC_StructInit (
    ADC_InitTypeDef * this )
```

函数名称: void [ADC_StructInit](#)(ADC_InitTypeDef* ADC_InitStruct)

功能描述: ADC结构体初始化

参见
被引用内容: 无

参数

输入参数:	ADC_InitTypeDef *ADC_InitStruct
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
ADC_InitTypeDef ADC_InitStructure;  
ADC_StructInit (&ADC_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2015年11月26日	1.0	Howlet	修改

修改日志:

在文件 [lks32mc07x_adc.c](#) 第 99 行定义.

5.52 lks32mc07x_adc.c

[浏览该文件的文档.](#)

```
00001  
00014 #include "lks32mc07x_adc.h"  
00015 #include "lks32mc07x-sys.h"  
00016  
00054 void ADC_Init (ADC_TypeDef *ADCx, ADC_InitTypeDef *this)  
00055 {
```

```

00056     SYS_AnalogModuleClockCmd(SYS_AnalogModule_ADC, ENABLE); // ADC模块使能
00057
00058     ADCx->IE = this->IE | this->RE;
00059     ADCx->CHNT = (this->IS1 << 8) | (this->S2 << 4) | (this->S1 << 0);
00060     ADCx->CFG = (this->NSMP << 12) | (this->DATA_ALIGN << 10) | (this->CSMP << 8) | (this->TCNT << 4)
| (this->TROVS << 3) | (this->OVSR << 0);
00061
00062     ADCx->LTH = this->LTH;
00063     ADCx->HTH = this->HTH;
00064     ADCx->GEN = this->GEN;
00065
00066     ADC0->DC0 = Read.Trim(0x00001420); /* 3.6V量程对应DC offset */
00067     ADC0->AMC0 = Read.Trim(0x00001424); /* 3.6V量程对应增益校正系数 */
00068     ADC0->DC1 = Read.Trim(0x00001428); /* 7.2V量程对应DC offset */
00069     ADC0->AMC1 = Read.Trim(0x0000142c); /* 7.2V量程对应增益校正系数 */
00070
00071     ADC1->DC0 = Read.Trim(0x00001430); /* 3.6V量程对应DC offset */
00072     ADC1->AMC0 = Read.Trim(0x00001434); /* 3.6V量程对应增益校正系数 */
00073     ADC1->DC1 = Read.Trim(0x00001438); /* 7.2V量程对应DC offset */
00074     ADC1->AMC1 = Read.Trim(0x0000143c); /* 7.2V量程对应增益校正系数 */
00075
00076     ADCx->TRIG = this->TRIG;
00077 }
00078
00099 void ADC_StructInit(ADC_InitTypeDef *this)
00100 {
00101     this->IE = 0; // 中断使能
00102     this->RE = 0; // DMA请求使能
00103     this->NSMP = 0; // 两段采样使能
00104     this->DATA_ALIGN = 0; // DAT对齐方式
00105     this->CSMP = 0; // 连续采样使能
00106     this->TCNT = 0; // 触发一次采样所需的事件数
00107     this->TROVS = 0; // 过采样触发模式
00108     this->OVSR = 0; // 过采样率
00109     this->TRIG = 0; // 触发信号
00110     this->S1 = 1; // 第一段常规采样次数
00111     this->S2 = 1; // 第二段常规采样次数
00112     this->IS1 = 1; // 空闲采样次数
00113     this->GAIN = 0; // 量程默认3.6V
00114 }
00115
00148 void ADC_ClearIRQFlag(ADC_TypeDef *ADCx, uint16_t INT_flag)
00149 {
00150     ADCx->IF = INT_flag;
00151 }
00152
00184 u16 ADC_GetIRQFlag(ADC_TypeDef *ADCx, uint16_t INT_flag)
00185 {
00186     u8 ADCIRQFlag = 0;
00187     if (ADCx->IF & INT_flag & ADCx->IE)
00188     {
00189         ADCIRQFlag = 1;
00190     }
00191     else
00192     {
00193         ADCIRQFlag = 0;
00194     }
00195     return ADCIRQFlag;
00196 }
00197
00222 void ADC_CHN_GAIN_CFG(ADC_TypeDef *ADCx, CHNx CHNum, u8 nChannel, ADC_GAINx GAIN3V6_or_7V2)
00223 {
00224     u8 val;
00225     u32 reg;
00226
00227     if (GAIN3V6_or_7V2 == 1)
00228     {
00229         ADCx->GAIN |= (GAIN3V6_or_7V2 << CHNum);
00230     }
00231
00232     val = CHNum >> 2;
00233     val &= 3;
00234     CHNum &= 3;
00235     CHNum *= 4;
00236     reg = (&ADCx->CHN0)[val] & ~(0xf << CHNum);
00237     (&ADCx->CHN0)[val] = reg | (nChannel << CHNum);
00238 }
00239
00260 s16 ADC_GetConversionValue(ADC_TypeDef *ADCx, DATx DATNum)
00261 {
00262     if (DATNum == DAT0)
00263     {
00264         return ADCx->DAT0;
00265     }
00266     else if (DATNum == DAT1)
00267     {
00268         return ADCx->DAT1;

```

```

00269     }
00270     else if (DATNum == DAT2)
00271     {
00272         return ADCx->DAT2;
00273     }
00274     else if (DATNum == DAT3)
00275     {
00276         return ADCx->DAT3;
00277     };
00278     }
00279     else if (DATNum == DAT4)
00280     {
00281         return ADCx->DAT4;
00282     }
00283     else if (DATNum == DAT5)
00284     {
00285         return ADCx->DAT5;
00286     }
00287     else if (DATNum == DAT6)
00288     {
00289         return ADCx->DAT6;
00290     }
00291     else if (DATNum == DAT7)
00292     {
00293         return ADCx->DAT7;
00294     }
00295     else if (DATNum == DAT8)
00296     {
00297         return ADCx->DAT8;
00298     }
00299     else if (DATNum == DAT9)
00300     {
00301         return ADCx->DAT9;
00302     }
00303     else if (DATNum == DAT10)
00304     {
00305         return ADCx->DAT10;
00306     }
00307     else if (DATNum == DAT11)
00308     {
00309         return ADCx->DAT11;
00310     }
00311     else if (DATNum == DAT12)
00312     {
00313         return ADCx->DAT12;
00314     }
00315     else if (DATNum == DAT13)
00316     {
00317         return ADCx->DAT13;
00318     }
00319     else if (DATNum == DAT14)
00320     {
00321         return ADCx->IDAT0;
00322     }
00323     else if (DATNum == DAT15)
00324     {
00325         return ADCx->IDAT1;
00326     }
00327     else
00328     {
00329         return 0;
00330     }
00331 }
00332
00353 void ADC_SoftTrgEN(ADC_TypeDef *ADCx, FuncState state)
00354 {
00355     if (state == ENABLE)
00356     {
00357         ADCx->SWT = 0x5AA5;
00358     }
00359 }
00360
00361 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.53 source/lks32mc07x_can.c 文件参考

文件名称: [lks32mc07x_can.c](#)
 文件标识: 无
 内容摘要: CAN外设驱动程序

其它说明： 无

```
#include "lks32mc07x_can.h"
#include "lks32mc07x_sys.h"
#include "string.h"
```

lks32mc07x_can.c 的引用(Include)关系图:



函数

- static bool **GetBaudParam** (u32 baud, u8 *presc, u8 *sjw, u8 *seg2, u8 *seg1)
- static void **CAN_Write_TBUF** (u32 id, u8 ide, u8 rtr, u8 *msg, u8 len)
- static void **Set_CFG_STAT** (u32 bitValue)
CAN_CFG_STAT按bit置位
- static void **Reset_CFG_STAT** (u32 bitValue)
CAN_CFG_STAT按bit复位
- static void **Delay** (u32 cnt)
- void **CAN_Init1** (CAN_InitTypeDef *this)
CAN初始化函数,采用2.0B/FD协议 输入参数: CAN_InitTypeDef*: 初始化参数
- void **CAN_StructInit** (CAN_InitTypeDef *this)
CAN配置结构体初始化, 并设置默认参数
- void **CAN_Init** (u8 BTR3, u8 BTR2, u8 BTR1, u8 BTR0)
函数名称: void CAN_Init(u8 BTR3,u8 BTR2,u8 BTR1,u8 BTR0)
- CanSendRet **CAN_Send** (CAN_TxMsg *tx_msg)
- CanSendRet **My_CAN_Send_Msg** (u32 id, u8 ide, u8 rtr, u8 *msg, u8 len)
CANFD数据发送函数, 采用CAN-FD协议, PTB发送 输入参数: id: 发送id ide: 标准/扩展帧 rtr: 远程帧/数据帧 fdf: CAN2.0/CANFD brs: 切换波特率 *msg: 发送的数据 len: 发送字节长度 输出参数: 无 返回值: 0: 发送成功 !0: 发送失败 其它说明: id: 标准ID(11位)/扩展ID(11位+18位) ide: 0, 标准帧; 1, 扩展帧 rtr: 0, 数据帧; 1, 遥控帧 fdf: 0, CAN2.0; 1, CANFD brs: 0, 不切换波特率; 1, 切换波特率 len: 要发送的数据长度, CAN2.0最大8字节, CANFD最大64字节
- CanSendRet **CAN_Send_Trig** (STBTransOption stb.trans)
CANFD数据发送函数, 采用CAN-FD协议, STB发送。默认STB为FIFO模式。 输入参数: id: 发送id ide: 标准/扩展帧 rtr: 远程帧/数据帧 fdf: CAN2.0/CANFD brs: 切换波特率 *msg: 发送的数据 len: 发送字节长度 输出参数: 无 返回值: 0: 发送成功 !0: 发送失败 其它说明: id: 标准ID(11位)/扩展ID(11位+18位) ide: 0, 标准帧; 1, 扩展帧 rtr: 0, 数据帧; 1, 遥控帧 fdf: 0, CAN2.0; 1, CANFD brs: 0, 不切换波特率; 1, 切换波特率 option: 0, 本次不发送; 1, 发送一帧; 2, 发送所有帧
- void **CAN_Send_Cancel** (TBSelOption tbsel)
- CanRecvRet **CAN_Recv** (CAN_RxMsg *rx_msg)
CAN接收数据 输入参数: *rx_msg: 接收数据存储对象 返回值: CanRecvRet
- CanRecvRet **CAN_Receive_Msg** (u32 *id, u8 *len, u8 *ide, u8 *rtr, u8 *buf)
函数名称: u8 CAN_Receive_Msg(u32* id,u8* len,u8* ide,u8* rtr,u8* buf)
- void **IDx_Filter** (u8 x, u32 id, u32 mask, IDEFilterOption ide_filter)
CAN IDx接收滤波函数 输入参数: x: 滤波通道号。 id: ID值。 mask: 屏蔽位。 bit=0表示需要位匹配, bit=1表示不需要匹配。 ide_filter: IDEFilter_St.Ex / IDEFilter_St / IDEFilter.Ex
- void **IDx_FilterCmd** (u8 x, FuncState state)
CAN接收过滤器使能与禁止
- void **CAN_LBMICmd** (FuncState state)
内部回环模式使能/禁止

- void [CAN_LBMECmd](#) (FuncState state)
外部回环模式使能/禁止
- void [CAN_SS.Enable](#) (TBSelOption tbsel, FuncState state)
单次传输模式使能/禁止
- void [CAN_Enable](#) (void)
打开CAN模块时钟
- void [CAN_Disable](#) (void)
关闭CAN模块时钟
- void [CAN_Reset](#) (void)
CAN模块进入复位模式
- void [CAN_SetSTBMode](#) (STBTransMode tsMode)
- u8 [CAN_STBFullGet](#) (void)
- u8 [CAN_RxErrorCntGet](#) (void)
获取can接收错误计数
- u8 [CAN_TxErrorCntGet](#) (void)
获取can发送错误计数
- u8 [CAN_ErrorPassiveGet](#) (void)
获取can的error passive state
- u8 [CAN_BusoffGet](#) (void)
获取can的bus off state
- [CANErrorKind](#) [CAN_KindOfErrorGet](#) (void)
获取can的kind of error，错误类别读取
- u8 [CAN_ArbitrationLostCap](#) (void)
获取can仲裁失败位置
- u8 [CAN_ROverflowGet](#) (void)
获取can的Rx Buffer是否溢出

变量

- [CANRxMsg g_rxMsg](#)

5.53.1 详细描述

文件名称: [lks32mc07x.can.c](#)
文件标识: 无
内容摘要: CAN外设驱动程序
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月23日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x.can.c](#) 中定义.

5.53.2 函数说明

5.53.2.1 CAN_ArbitrationLostCap()

```
u8 CAN_ArbitrationLostCap (
    void )
```

获取can仲裁失败位置

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 870 行定义.

5.53.2.2 CAN_BusoffGet()

```
u8 CAN_BusoffGet (
    void )
```

获取can的bus off state

更新记录

在文件 [lks32mc07x.can.c](#) 第 851 行定义.

5.53.2.3 CAN_Disable()

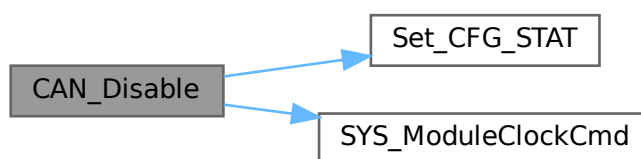
```
void CAN_Disable (
    void )
```

关闭CAN模块时钟

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 777 行定义.

函数调用图:



5.53.2.4 CAN_Enable()

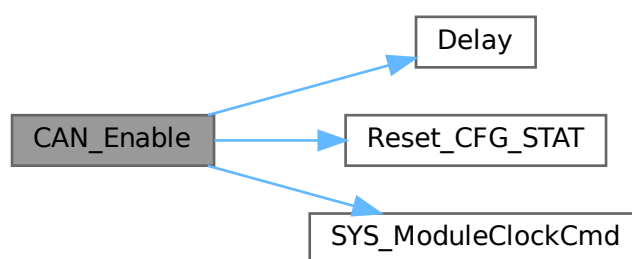
```
void CAN_Enable (
    void )
```

打开CAN模块时钟

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x_can.c](#) 第 767 行定义.

函数调用图:



5.53.2.5 CAN_ErrorPassiveGet()

```
u8 CAN_ErrorPassiveGet (
    void )
```

获取can的error passive state

更新记录

在文件 [lks32mc07x_can.c](#) 第 842 行定义.

5.53.2.6 CAN_Init()

```
void CAN_Init (
    u8 BTR3,
    u8 BTR2,
    u8 BTR1,
    u8 BTR0 )
```

函数名称: `void CAN_Init(u8 BTR3,u8 BTR2,u8 BTR1,u8 BTR0)`

功能描述: CAN初始化函数,采用2.0B协议

参见

被调用函数: 无

参数

输入参数:	波特率 AN_SBAUD.PRESC[7:0] CAN_SBAUD.SJW[6:0] CAN_SBAUD.SEG2[6:0] CAN_SBAUD.SEG1[7:0] Sample Poin 1Mbps 0x05 0x02 0x05 0x08 63% 800Kbps 0x0B 0x01 0x03 0x04 60% 666Kbps 0x0B 0x01 0x03 0x06 67% 500Kbps 0x0B 0x02 0x05 0x08 63% 400Kbps 0x0B 0x02 0x06 0x0B 65% 250Kbps 0x17 0x02 0x05 0x08 63% 200Kbps 0x17 0x02 0x06 0x0B 65% 125Kbps 0x2F 0x02 0x05 0x08 63% 100Kbps 0x2F 0x02 0x06 0x0B 65% 80Kbps 0x35 0x02 0x06 0x0B 65% 50Kbps 0x5F 0x02 0x06 0x0B 65% 40Kbps 0x77 0x02 0x06 0x0B 65% 25Kbps 0xBF 0x02 0x06 0x0B 65% 20Kbps 0xEF 0x02 0x06 0x0B 65% 10Kbps 0xEF 0x06 0x0D 0x18 65% 05Kbps 0xEF 0x0D 0x1B 0x32 65%
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
CAN_Init (0x0B, 0x02, 0x05, 0x08); //CAN波特率500Khz
```

Date	Version	Author	Description
2023年2月23日	1.0	HuangMG	创建

修改日志:

在文件 lks32mc07x_can.c 第 130 行定义.

5.53.2.7 CAN_Init1()

```
void CAN_Init1 (
```

```
CAN_InitTypeDef * this )
```

CAN初始化函数,采用2.0B/FD协议 输入参数: CAN_InitTypeDef*: 初始化参数

更新记录 V1.0 2024/07/03 YangZJ 创建

在文件 [lks32mc07x-can.c](#) 第 41 行定义.

函数调用图:



5.53.2.8 CAN_KindOfErrorGet()

```
CANErrorKind CAN_KindOfErrorGet (
    void )
```

获取can的kind of error, 错误类别读取

更新记录

在文件 [lks32mc07x-can.c](#) 第 860 行定义.

5.53.2.9 CAN_LBMECmd()

```
void CAN_LBMECmd (
    FuncState state )
```

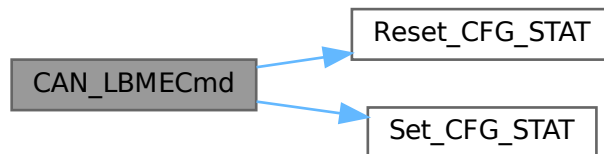
外部回环模式使能/禁止

更新记录

自发自收，自应答，需将芯片RX和TX接入CAN收发器，收发器可以不挂入总线。应在CAN初始化和ID滤波设置之后再使能。

在文件 [lks32mc07x_can.c](#) 第 717 行定义.

函数调用图:



5.53.2.10 CAN_LBMICmd()

```
void CAN_LBMICmd (
    FuncState state )
```

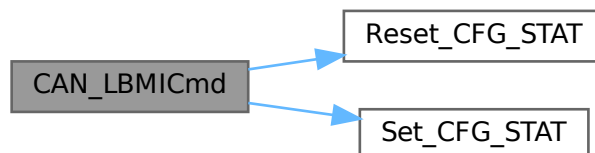
内部回环模式使能/禁止

更新记录

自发自收，无需自应答，芯片RX和TX引脚可悬空。应在CAN初始化和ID滤波设置之后再使能。

在文件 [lks32mc07x_can.c](#) 第 699 行定义.

函数调用图:



5.53.2.11 CAN_Receive_Msg()

```
CanRecvRet CAN_Receive_Msg (
    u32 * id,
    u8 * len,
    u8 * ide,
    u8 * rtr,
    u8 * buf )
```

函数名称: u8 CAN_Receive_Msg(u32* id,u8* len ,u8* ide,u8* rtr,u8 *buf)

功能描述: CAN轮询接收数据程序

参见
被调用函数: 无

参数

输入参数:	无
输出参数:	id: 接收帧id len: 接收数据长度 ide: 接收帧的类型, 标准/扩展帧 rtr: 接收帧的类型, 远程帧/数据帧 *buf: 接收帧的数据存储缓存区

返回
返 回 值: 0: 接收完成 !0: 接收失败

注解
其它说明: 1、id:标准ID(11位)/扩展ID(11位+18位)
2、ide:0,标准帧;1,扩展帧
3、rtr:0,数据帧;1,远程帧。

警告
无

示例代码:

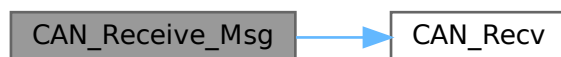
```
//can_par.id存储接收帧ID, can_par.len接收数据个数
can_par.ide存储接收帧类型标准/扩展帧,
can_par.rtr存储接收帧类型远程帧/数据帧, can_par.RX存储接收帧数据
CAN_Receive_Msg (&( can_par.id), &(can_par.len), &(can_par.ide), &(can_par.rtr), can_par.RX);
```

Date	Version	Author	Description
2023年2月23日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_can.c](#) 第 405 行定义.

函数调用图:



5.53.2.12 CAN_Recv()

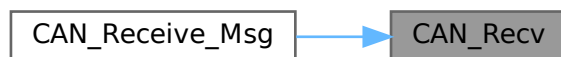
```
CanRecvRet CAN_Recv (  
    CANRxMsg * rx_msg )
```

CAN接收数据 输入参数: *rx_msg: 接收数据存储对象 返回值: CanRecvRet

更新记录

在文件 [lks32mc07x_can.c](#) 第 314 行定义.

这是这个函数的调用关系图:



5.53.2.13 CAN_Reset()

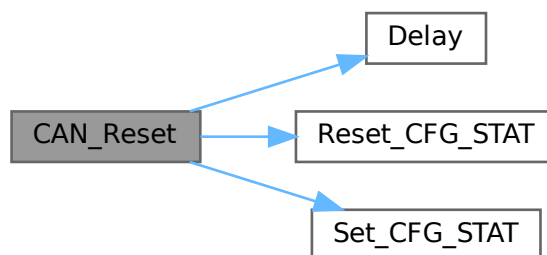
```
void CAN_Reset (  
    void )
```

CAN模块进入复位模式

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 786 行定义.

函数调用图:



5.53.2.14 CAN_ROverflowGet()

```
u8 CAN_ROverflowGet (  
    void )
```

获取can的Rx Buffer是否溢出

更新记录

在文件 [lks32mc07x.can.c](#) 第 879 行定义.

5.53.2.15 CAN_RxErrorCntGet()

```
u8 CAN_RxErrorCntGet (  
    void )
```

获取can接收错误计数

更新记录 V1.0 2022/09/19 YangZJ 创建

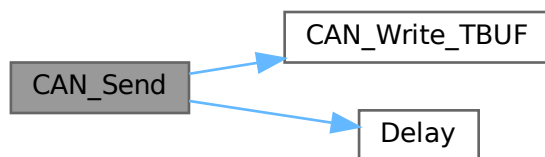
在文件 [lks32mc07x.can.c](#) 第 824 行定义.

5.53.2.16 CAN_Send()

```
CanSendRet CAN_Send (
    CANTxMsg * tx_msg )
```

在文件 [lks32mc07x.can.c](#) 第 171 行定义.

函数调用图:



这是这个函数的调用关系图:



5.53.2.17 CAN_Send_Cancel()

```
void CAN_Send_Cancel (
    TBSELoption tbsel )
```

在文件 [lks32mc07x.can.c](#) 第 296 行定义.

5.53.2.18 CAN_Send_Trig()

```
CanSendRet CAN_Send_Trig (
    STBTransOption stb_trans )
```

CANFD数据发送函数，采用CAN-FD协议，STB发送。默认STB为FIFO模式。输入参数: id:发送id ide:0:标准/扩展帧 rtr:远程帧/数据帧 fdf:CAN2.0/CANFD brs:切换波特率 *msg:发送的数据 len:发送字节长度 输出参数: 无 返回值: 0: 发送成功 !0: 发送失败 其它说明: id:标准ID(11位)/扩展ID(11位+18位) ide:0,标准帧;1,扩展帧 rtr:0,数据帧;1,遥控帧 fdf:0,CAN2.0;1,CANFD brs:0,不切换波特率;1,切换波特率 option:0,本次不发送;1,发送一帧;2,发送所有帧

更新记录 V1.0 2023/02/13 HuangMG 创建

在文件 [lks32mc07x_can.c](#) 第 257 行定义.

函数调用图:



5.53.2.19 CAN_SetSTBMode()

```
void CAN_SetSTBMode (
    STBTransMode tsMode )
```

在文件 [lks32mc07x_can.c](#) 第 797 行定义.

5.53.2.20 CAN_SS_Enable()

```
void CAN_SS_Enable (
    TBSelOption tbsel,
    FuncState state )
```

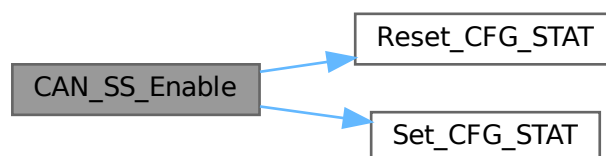
单次传输模式使能/禁止

更新记录

PTB/STB单次传输模式使能，使能后错误不重发。应在CAN初始化和ID滤波设置之后再使能。

在文件 [lks32mc07x_can.c](#) 第 737 行定义.

函数调用图:



5.53.2.21 CAN_STBFullGet()

```
u8 CAN_STBFullGet (
    void )
```

在文件 [lks32mc07x.can.c](#) 第 815 行定义.

5.53.2.22 CAN_StructInit()

```
void CAN_StructInit (
    CAN_InitTypeDef * this )
```

CAN配置结构体初始化，并设置默认参数

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x.can.c](#) 第 72 行定义.

5.53.2.23 CAN_TxErrorCntGet()

```
u8 CAN_TxErrorCntGet (
    void )
```

获取can发送错误计数

更新记录 V1.0 2022/09/19 YangZJ 创建

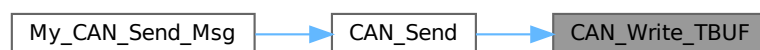
在文件 [lks32mc07x.can.c](#) 第 833 行定义.

5.53.2.24 CAN_Write_TBUF()

```
static void CAN_Write_TBUF (
    u32 id,
    u8 ide,
    u8 rtr,
    u8 * msg,
    u8 len ) [static]
```

在文件 [lks32mc07x.can.c](#) 第 620 行定义.

这是这个函数的调用关系图:

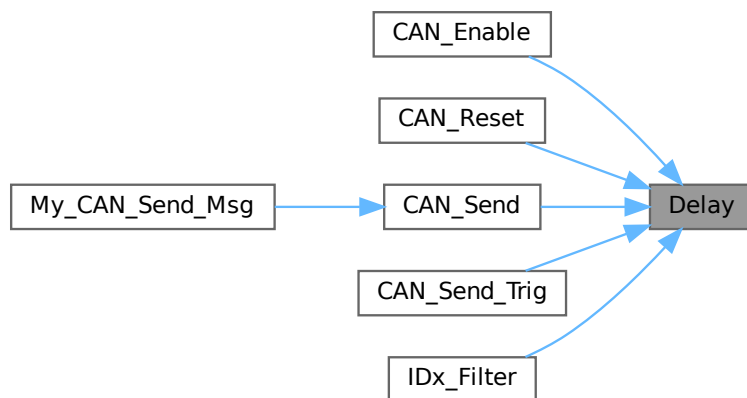


5.53.2.25 Delay()

```
static void Delay (  
    u32 cnt ) [static]
```

在文件 [lks32mc07x.can.c](#) 第 161 行定义.

这是这个函数的调用关系图:



5.53.2.26 GetBaudParam()

```
static bool GetBaudParam (  
    u32 baud,  
    u8 * presc,  
    u8 * sjw,  
    u8 * seg2,  
    u8 * seg1 ) [static]
```

在文件 [lks32mc07x.can.c](#) 第 511 行定义.

这是这个函数的调用关系图:



5.53.2.27 IDx_Filter()

```
void IDx_Filter (
    u8 x,
    u32 id,
    u32 mask,
    IDEFilterOption ide_filter )
```

CAN IDx接收滤波函数 输入参数: x: 滤波通道号。id: ID值。mask: 屏蔽位。bit=0表示需要位匹配, bit=1表示不需要匹配。ide_filter: IDEFilter_St_Ex / IDEFilter_St / IDEFilter_Ex

更新记录

在文件 [lks32mc07x_can.c](#) 第 442 行定义.

函数调用图:



5.53.2.28 IDx_FilterCmd()

```
void IDx_FilterCmd (
    u8 x,
    FuncState state )
```

CAN接收过滤器使能与禁止

更新记录 V1.0 2022/09/19 YangZJ 创建

在文件 [lks32mc07x_can.c](#) 第 483 行定义.

5.53.2.29 My_CAN_Send_Msg()

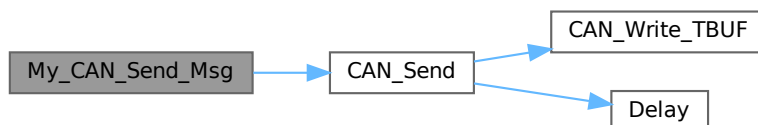
```
CanSendRet My_CAN_Send_Msg (
    u32 id,
    u8 ide,
    u8 rtr,
    u8 * msg,
    u8 len )
```

CANFD数据发送函数，采用CAN-FD协议，PTB发送 输入参数: id: 发送id ide: 标准/扩展帧 rtr: 远程帧/数据帧 fdf:CAN2.0/CANFD brs:切换波特率 *msg: 发送的数据 len:发送字节长度 输出参数: 无 返回值: 0: 发送成功 !0: 发送失败 其它说明: id:标准ID(11位)/扩展ID(11位+18位) ide:0,标准帧;1,扩展帧 rtr:0,数据帧;1,遥控帧 fdf:0,CAN2.0;1,CANFD brs:0,不切换波特率;1,切换波特率 len:要发送的数据长度，CAN2.0最大8字节，↔CANFD最大64字节

更新记录

在文件 [lks32mc07x_can.c](#) 第 230 行定义.

函数调用图:



5.53.2.30 Reset_CFG_STAT()

```
void Reset_CFG_STAT (
    u32 bitValue ) [static]
```

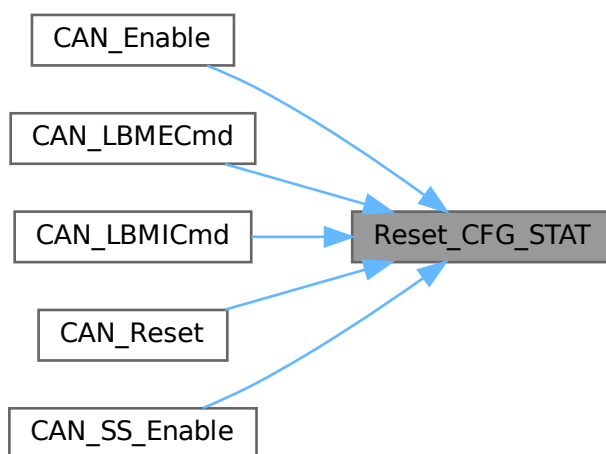
CAN_CFG_STAT按bit复位

更新记录

CAN_CFG_STAT按bit复位方法，避免向CAN_CFG_STAT.Busoff位写1

在文件 [lks32mc07x_can.c](#) 第 682 行定义.

这是这个函数的调用关系图:



5.53.2.31 Set_CFG_STAT()

```
void Set_CFG_STAT (
    u32 bitValue ) [static]
```

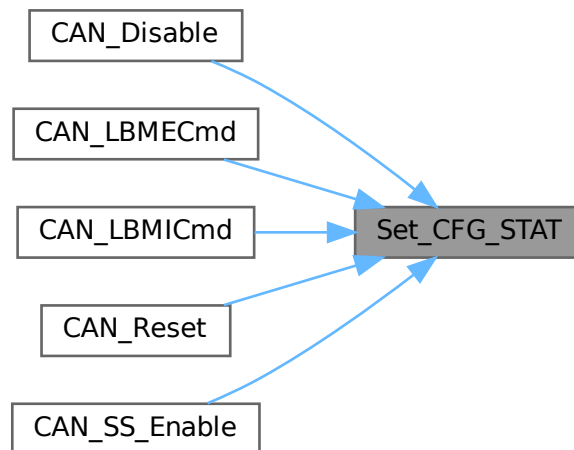
CAN_CFG_STAT按bit置位

更新记录

CAN_CFG_STAT按bit置位方法，避免向CAN_CFG_STAT.Busoff位写1

在文件 [lks32mc07x_can.c](#) 第 666 行定义.

这是这个函数的调用关系图:



5.53.3 变量说明

5.53.3.1 g_rxMsg

`CANRxMsg g_rxMsg`

初始值:

```

= {
    0,
    0,
    0,
    0,

    0,
    {0},
}
  
```

在文件 [lks32mc07x_can.c](#) 第 18 行定义.

5.54 lks32mc07x_can.c

[浏览该文件的文档.](#)

```

00001
00014 #include "lks32mc07x_can.h"
00015 #include "lks32mc07x_sys.h"
00016 #include "string.h"
00017
00018 CANRxMsg g_rxMsg = {
  
```

```

00019     0,
00020     0,
00021     0,
00022     0,
00023
00024     0,
00025     {0},
00026 };
00027
00028 static bool GetBaudParam(u32 baud, u8 *presc, u8 *sjw, u8 *seg2, u8 *seg1);
00029 static void CAN_Write_TBUF(u32 id, u8 ide, u8 rtr, u8 *msg, u8 len);
00030
00031 static void Set_CFG_STAT(u32 bitValue);
00032 static void Reset_CFG_STAT(u32 bitValue);
00033
00034 static void Delay(u32 cnt);
00035
00041 void CAN_Init1(CAN_InitTypeDef *this)
00042 {
00043     u8 s_presc, s_sjw, s_seg2, s_seg1;
00044
00045     SYS_WR_PROTECT = 0x7a83; // 关闭写保护
00046
00047     SYS_SFT_RST |= BIT19; // 复位CAN
00048     SYS_SFT_RST &= ~BIT19; // 释放CAN复位
00049
00050     SYS_CLK_FEN |= BIT19; // 使能CAN时钟
00051
00052     CAN_CFG_STAT |= BIT7; // CAN复位模式
00053     CAN_ACFEN = 0x00; // 关闭ID滤波功能
00054
00055     if (this->SBaud > 0 && GetBaudParam(this->SBaud, &s_presc, &s_sjw, &s_seg2, &s_seg1))
00056     {
00057         CAN_SBAUD = (s_presc << 24) | (s_sjw << 16) | (s_seg2 << 8) | s_seg1; // 慢速波特率设置
00058     }
00059
00060     CAN_CFG_STAT &= ~BIT7; // 正常输出模式
00061     CAN_TCMD = 0x00;
00062     CAN_RTIE = this->IE; // 默认开启 错误中断使能+接收中断使能
00063     CAN_ERRINT = this->ERRINT_IE;
00064
00065     SYS_WR_PROTECT = 0x0; // 打开写保护
00066 }
00067
00072 void CAN_StructInit(CAN_InitTypeDef *this)
00073 {
00074     this->SBaud = 500;
00075     this->IE = 0;
00076     this->ERRINT_IE = 0;
00077 }
00078
00079 /*****
00080 函数名称: void CAN_Init1(u8 BTR3, u8 BTR2, u8 BTR1, u8 BTR0)
00081 功能描述: CAN初始化函数, 采用2.0B协议
00082 操作的表:
00083
00084     输入参数: BTR0      S_SEG1: 段时间段 1 时间单元设定
00085                BTR1      S_SEG2: 段时间段 2 时间单元设定
00086                BTR2      S_SJW : 再同步补偿宽度时间设定
00087                BTR3      S_PRESC: TQ 预分频率设定值
00088 输出参数: 无
00089 返回值: 无
00090 其它说明:
00091 修改日期      版本号      修改人      修改内容
00092 -----
00093 *****/
00130 void CAN_Init(u8 BTR3, u8 BTR2, u8 BTR1, u8 BTR0)
00131 {
00132     SYS_WR_PROTECT = 0x7a83; // 关闭写保护
00133
00134     SYS_SFT_RST |= BIT19; // 复位CAN
00135     SYS_SFT_RST &= ~BIT19; // 释放CAN复位
00136
00137     SYS_CLK_FEN |= BIT19; // 使能CAN时钟
00138
00139     CAN_CFG_STAT |= 1 << 7; // 复位模式
00140
00141     CAN_ACFEN = 0x00; // 关闭ID滤波功能
00142     CAN_SBAUD = BTR0 | (BTR1 << 8) | (BTR2 << 16) | (BTR3 << 24); // 波特率设置
00143     CAN_CFG_STAT &= ~(1 << 7); // 正常输出模式
00144     CAN_TCMD = 0x00;
00145     CAN_RTIE |= 1 << 7; // RFIFO有新的数据被接收到, 中断源使能
00146     CAN_RTIE |= 1 << 1; // 错误中断使能
00147     SYS_WR_PROTECT = 0x0; // 打开写保护
00148 }
00149
00150 /*****

```

```

00151 函数名称:      void SoftDelay(void)
00152 功能描述:      软件延时函数
00153 输入参数:      无
00154 输出参数:      无
00155 返回值:        无
00156 其它说明:
00157 修改日期      版本号      修改人      修改内容
00158 -----
00159 2020/8/5      V1.0        Howlet Li      创建
00160 *****/
00161 static void Delay(u32 cnt)
00162 {
00163     volatile u32 t_cnt;
00164
00165     for (t_cnt = 0; t_cnt < cnt; t_cnt++)
00166     {
00167         __NOP();
00168     }
00169 }
00170
00171 CanSendRet CAN_Send(CANTxMsg *tx_msg)
00172 {
00173     u16 i = 0;
00174
00175     if (tx_msg->len > 8)
00176     {
00177         return CanSend_DataTooLong;
00178     }
00179
00180     if (tx_msg->tbsel == TBSel_PTB)
00181     {
00182         // PTB发送
00183         if (CAN->CFG_STAT & BIT1) // 本设备正在发送CAN帧
00184         {
00185             return CanSend_NotIdle; // 发送失败
00186         }
00187
00188         CAN_TCMD &= ~BIT7; // PTB发送。CAN_TCMD.TBSEL, 0-PTB 1-STB
00189         CAN_Write_TBUF(tx_msg->id, tx_msg->ide, tx_msg->rtr, tx_msg->buf, tx_msg->len);
00190         CAN_TCMD |= BIT4; // CAN 发送传输请求,触发PTB发送
00191         while (CAN_TCMD & BIT4)
00192         {
00193             Delay(100);
00194             if ((i++) >= 0x0fff)
00195             {
00196                 return CanSend_Timeout; // 发送失败
00197             }
00198         }
00199         return CanSend_Succ; // 发送完成
00200     }
00201     else
00202     {
00203         // STB发送
00204         if ((CAN->TCTRL & 0x03) == 0x03)
00205         {
00206             return CanSend_STBFull; // 发送失败, STB满
00207         }
00208
00209         CAN->TCMD |= BIT7; // STB发送。CAN_TCMD.TBSEL, 0-PTB 1-STB
00210         CAN_Write_TBUF(tx_msg->id, tx_msg->ide, tx_msg->rtr, tx_msg->buf, tx_msg->len);
00211         CAN->TCTRL |= BIT6; // 置位TSNEXT, 更新STB slot位置
00212
00213         return CanSend_Succ; // STB添加成功即返回, 还需调用CAN_Send_Trig触发发送
00214     }
00215 }
00216
00230 CanSendRet My_CAN_Send_Msg(u32 id, u8 ide, u8 rtr, u8 *msg, u8 len)
00231 {
00232     CANTxMsg tx_msg;
00233     tx_msg.id = id;
00234     tx_msg.ide = ide;
00235     tx_msg.rtr = rtr;
00236     tx_msg.buf = msg;
00237     tx_msg.len = len;
00238
00239     tx_msg.tbsel = TBSel_PTB;
00240
00241     return CAN_Send(&tx_msg);
00242 }
00243
00257 CanSendRet CAN_Send_Trig(STBTransOption stb_trans)
00258 {
00259     u32 i = 0;
00260
00261     if (stb_trans == STB_TransAll)
00262     {
00263         CAN->TCMD |= BIT1; // 触发所有STB发送

```



```

00264         while (CAN->TCMD & BIT1) // 最近一次是否传输完成
00265         {
00266             Delay(100);
00267             if ((i++) >= 0x0ff)
00268             {
00269                 return CanSend.Timeout; // 发送失败
00270             }
00271         }
00272     }
00273     else if (stb.trans == STB.TransOne)
00274     {
00275         CAN->TCMD |= BIT2; // 触发一帧STB发送
00276         while (CAN->TCMD & BIT2) // 最近一次是否传输完成
00277         {
00278             Delay(100);
00279             if ((i++) >= 0x0ff)
00280             {
00281                 return CanSend.Timeout; // 发送失败
00282             }
00283         }
00284     }
00285     else
00286     {
00287         return CanSend.Succ; // 不需要发送, 返回成功
00288     }
00289     return CanSend.Succ; // 发送完成
00290 }
00291 }
00292
00293 /*
00294 * 取消发送
00295 */
00296 void CAN_Send_Cancel(TBSelOption tbsel)
00297 {
00298     if (tbsel == TBSel.PTB)
00299     {
00300         CAN->TCMD |= BIT3;
00301     }
00302     else if (tbsel == TBSel.STB)
00303     {
00304         CAN->TCMD |= BIT0;
00305     }
00306 }
00307
00314 CanRecvRet CAN_Recv(CANRxMsg *rx_msg)
00315 {
00316     u8 reg_inf;
00317     u8 dlc = 0;
00318     u16 rbufx;
00319     u8 x;
00320
00321     if ((CAN_RCTRL & 0x03) > 0) /*Rx Buffer存有一帧或多帧数据, 可以通过 RFIFO 寄存器读取*/
00322     {
00323         reg_inf = CAN_RBUF_01;
00324
00325         // read ide,id
00326         if (reg_inf & BIT7) /*扩展帧*/
00327         {
00328             rx_msg->ide = 1;
00329             rx_msg->id = CAN_RBUF_00 & 0x1FFFFFFF;
00330         }
00331         else /*标准帧*/
00332         {
00333             rx_msg->ide = 0;
00334             rx_msg->id = CAN_RBUF_00 & 0x7ff;
00335         }
00336
00337         // read rtr
00338         if (reg_inf & BIT6)
00339         {
00340             rx_msg->rtr = 1;
00341         }
00342         else
00343         {
00344             rx_msg->rtr = 0;
00345         }
00346
00347         // read fdf,len
00348         dlc = reg_inf & 0x0f;
00349         rx_msg->len = dlc;
00350         if (rx_msg->rtr)
00351         {
00352             rx_msg->len = 0;
00353         }
00354
00355         // read buf
00356         x = 0;

```

```

00357         rbufx = 2;
00358         while (x < rx_msg->len)
00359         {
00360             rx_msg->buf[x] = CAN->RBUF[rbufx];
00361             rx_msg->buf[x + 1] = CAN->RBUF[rbufx] >> 8;
00362             rx_msg->buf[x + 2] = CAN->RBUF[rbufx] >> 16;
00363             rx_msg->buf[x + 3] = CAN->RBUF[rbufx] >> 24;
00364             x += 4;
00365             rbufx++;
00366         }
00367
00368         CAN_RCTRL |= BIT4; /*释放 Rx Buffer Slot*/
00369         rx_msg->valid = 1;
00370         return CanRecvSucc;
00371     }
00372
00373     return CanRecvNoData;
00374 }
00375
00405 CanRecvRet CAN_Receive_Msg(u32 *id, u8 *len, u8 *ide, u8 *rtr, u8 *buf)
00406 {
00407     CANRxMsg rx_msg;
00408     CanRecvRet ret;
00409     int i;
00410
00411     ret = CAN_Recv(&rx_msg);
00412     if (ret == CanRecvSucc)
00413     {
00414         *id = rx_msg.id;
00415         *ide = rx_msg.ide;
00416         *rtr = rx_msg.rtr;
00417         for (i = 0; i < rx_msg.len; i++)
00418         {
00419             buf[i] = rx_msg.buf[i];
00420         }
00421         *len = rx_msg.len;
00422     }
00423     else
00424     {
00425         *id = 0;
00426         *ide = 0;
00427         *rtr = 0;
00428         *len = 0;
00429     }
00430
00431     return ret;
00432 }
00433
00442 void IDx_Filter(u8 x, u32 id, u32 mask, IDEFilterOption ide_filter)
00443 {
00444     CAN_CFG_STAT |= BIT7; // 复位模式
00445
00446     CAN_ACFEN |= (1 << x); // ACFx is Enabled
00447     CAN_ACFCTRL = (x & 0x0F); // ACFADR [3:0]
00448
00449     // 1.set MASK
00450     CAN_ACFCTRL |= BIT5; // CAN_ACFCTRL.SELMASK==1 MASK
00451     if (ide_filter == IDEFilter_St_Ex)
00452     {
00453         CAN_ACF &= ~BIT30;
00454     }
00455     else if (ide_filter == IDEFilter_St)
00456     {
00457         CAN_ACF |= BIT30;
00458         CAN_ACF &= ~BIT29; // 只接收标准帧
00459     }
00460     else if (ide_filter == IDEFilter_Ex)
00461     {
00462         CAN_ACF |= BIT30;
00463         CAN_ACF |= BIT29; // 只接收扩展帧
00464     }
00465
00466     CAN_ACF &= ~0x1FFFFFFF;
00467     CAN_ACF |= (mask & 0x1FFFFFFF); // set ACFx MASK
00468
00469     // 2.set ACR
00470     CAN_ACFCTRL &= ~BIT5; // CAN_ACFCTRL.SELMASK==0 ACR
00471     CAN_ACF &= ~0x1FFFFFFF;
00472     CAN_ACF |= (id & 0x1FFFFFFF); // set ACFx ID
00473
00474     Delay(100);
00475     CAN_CFG_STAT &= ~BIT7; // 正常输出模式
00476     Delay(100);
00477 }
00478
00483 void IDx_FilterCmd(u8 x, FuncState state)
00484 {

```

```

00485     if (state == ENABLE)
00486     {
00487         CAN->ACFEN |= (1 << x);
00488     }
00489     else
00490     {
00491         CAN->ACFEN &= ~(1 << x);
00492     }
00493 }
00494
00495 /*
00496 波特率计算:
00497  $TQ = (PRESC + 1) / F_{clock}$ 
00498  $Tseg1 = TQ * (SEG1 + 2)$ 
00499  $Tseg2 = TQ * (SEG2 + 1)$ 
00500  $CAN\_BAUD = 1 / (Tseg1 + Tseg2) = 1 / (TQ * (SEG1 + 2 + SEG2 + 1)) = 1 / (TQ * (SEG1 + SEG2 + 3)) =$ 
 $F_{clock} / (PRESC + 1) * (SEG1 + SEG2 + 3)$ 
00501  $SEG1 + SEG2 = F_{clock} / ((PRESC + 1) * CAN\_BAUD) - 3$ 
00502 只要SEG1 + SEG2的总和固定, 波特率就正确。
00503 另外:  $SEG1 > SEG2$ , SEG1略大于SEG2
00504  $SEG2 \geq SJW$ 
00505
00506 example:
00507  $F_{clock} = 96000000$ ;  $PRESC = 11$ ;  $CAN\_BAUD = 500000$ ;
00508  $SEG = 96000000 / ((11+1) * 500000) - 3 = 13$ 
00509  $SEG1 + SEG2 = 12 + 1 = 11 + 2 = \dots = 8 + 5$ 
00510 */
00511 static bool GetBaudParam(u32 baud, u8 *presc, u8 *sjw, u8 *seg2, u8 *seg1)
00512 {
00513     bool ret = true;
00514     switch (baud)
00515     {
00516     case 1000:
00517         *presc = 0x05;
00518         *sjw = 0x02;
00519         *seg2 = 0x05;
00520         *seg1 = 0x08;
00521         break;
00522     case 800:
00523         *presc = 0x0B;
00524         *sjw = 0x01;
00525         *seg2 = 0x03;
00526         *seg1 = 0x04;
00527         break;
00528     case 666:
00529         *presc = 0x0B;
00530         *sjw = 0x01;
00531         *seg2 = 0x03;
00532         *seg1 = 0x06;
00533         break;
00534     case 500:
00535         *presc = 0x0B;
00536         *sjw = 0x02;
00537         *seg2 = 0x05;
00538         *seg1 = 0x08;
00539         break;
00540     case 400:
00541         *presc = 0x0B;
00542         *sjw = 0x02;
00543         *seg2 = 0x06;
00544         *seg1 = 0x0B;
00545         break;
00546     case 250:
00547         *presc = 0x17;
00548         *sjw = 0x02;
00549         *seg2 = 0x05;
00550         *seg1 = 0x08;
00551         break;
00552     case 200:
00553         *presc = 0x17;
00554         *sjw = 0x02;
00555         *seg2 = 0x06;
00556         *seg1 = 0x0B;
00557         break;
00558     case 125:
00559         *presc = 0x2F;
00560         *sjw = 0x02;
00561         *seg2 = 0x05;
00562         *seg1 = 0x08;
00563         break;
00564     case 100:
00565         *presc = 0x2F;
00566         *sjw = 0x02;
00567         *seg2 = 0x06;
00568         *seg1 = 0x0B;
00569         break;
00570     case 80:

```

```

00571         *presc = 0x35;
00572         *sjw = 0x02;
00573         *seg2 = 0x06;
00574         *seg1 = 0x0B;
00575         break;
00576     case 50:
00577         *presc = 0x5F;
00578         *sjw = 0x02;
00579         *seg2 = 0x06;
00580         *seg1 = 0x0B;
00581         break;
00582     case 40:
00583         *presc = 0x77;
00584         *sjw = 0x02;
00585         *seg2 = 0x06;
00586         *seg1 = 0x0B;
00587         break;
00588     case 25:
00589         *presc = 0xBF;
00590         *sjw = 0x02;
00591         *seg2 = 0x06;
00592         *seg1 = 0x0B;
00593         break;
00594     case 20:
00595         *presc = 0xEF;
00596         *sjw = 0x02;
00597         *seg2 = 0x06;
00598         *seg1 = 0x0B;
00599         break;
00600     case 10:
00601         *presc = 0xEF;
00602         *sjw = 0x06;
00603         *seg2 = 0x0D;
00604         *seg1 = 0x18;
00605         break;
00606     case 5:
00607         *presc = 0xEF;
00608         *sjw = 0x0D;
00609         *seg2 = 0x1B;
00610         *seg1 = 0x32;
00611         break;
00612     default:
00613         ret = false;
00614         break;
00615     }
00616
00617     return ret;
00618 }
00619
00620 static void CANWriteTBUF(u32 id, u8 ide, u8 rtr, u8 *msg, u8 len)
00621 {
00622     u8 frame_inf = 0;
00623     u8 dlc = 0;
00624     u16 tbufx = 0;
00625     u32 tbufv = 0;
00626     u16 x = 0;
00627
00628     if (ide == 0)
00629     {
00630         id &= 0x7FF; // 标准帧, 11位ID
00631     }
00632     else
00633     {
00634         id &= 0x1FFFFFFF; // 扩展帧, 29位ID
00635     }
00636     CAN_TBUF_00 = id;
00637
00638     dlc = len;
00639
00640     frame_inf |= (ide << 7); // 0标准/1扩展帧
00641     frame_inf |= (rtr << 6); // 0,数据帧;1,遥控帧
00642     frame_inf |= (dlc << 0); // dlc,发送数据长度
00643     CAN_TBUF_01 = frame_inf;
00644
00645     // set CAN_TBUF_02 ~ CAN_TBUF_03
00646     x = 0;
00647     tbufx = 2; // TBUF赋值从CAN_TBUF_02开始
00648     while (x < len)
00649     {
00650         tbufv = msg[x];
00651         if ((x + 1) < len) tbufv += (msg[x + 1] << 8);
00652         if ((x + 2) < len) tbufv += (msg[x + 2] << 16);
00653         if ((x + 3) < len) tbufv += (msg[x + 3] << 24);
00654         CAN->TBUF[tbufx] = tbufv;
00655
00656         x += 4;
00657         tbufx++;

```

```

00658     }
00659 }
00660
00666 void Set_CFG_STAT(u32 bitValue)
00667 {
00668     u32 tmp_cfg_stat;
00669     tmp_cfg_stat = CAN->CFG_STAT;
00671     tmp_cfg_stat &= ~BIT0;
00672     tmp_cfg_stat |= bitValue;
00673
00674     CAN->CFG_STAT = tmp_cfg_stat;
00675 }
00676
00682 void Reset_CFG_STAT(u32 bitValue)
00683 {
00684     u32 tmp_cfg_stat;
00685     tmp_cfg_stat = CAN->CFG_STAT;
00687     tmp_cfg_stat &= ~BIT0;
00688     tmp_cfg_stat &= ~bitValue;
00689
00690     CAN->CFG_STAT = tmp_cfg_stat;
00691 }
00692
00699 void CAN_LBMCmd(FuncState state)
00700 {
00701     if (state == ENABLE)
00702     {
00703         Set_CFG_STAT(BIT5); // 开启内回环
00704     }
00705     else
00706     {
00707         Reset_CFG_STAT(BIT5); // 关闭内回环
00708     }
00709 }
00710
00717 void CAN_LBMECmd(FuncState state)
00718 {
00719     if (state == ENABLE)
00720     {
00721         Set_CFG_STAT(BIT6); // 开启外回环
00722         CAN->RCTRL |= BIT7; // 使能自应答
00723     }
00724     else
00725     {
00726         Reset_CFG_STAT(BIT6); // 关闭外回环
00727         CAN->RCTRL &= ~BIT7; // 关闭自应答
00728     }
00729 }
00730
00737 void CAN_SS_Enable(TBSelOption tbsel, FuncState state)
00738 {
00739     if (tbsel == TBSel_PTB)
00740     {
00741         if (state == ENABLE)
00742         {
00743             Set_CFG_STAT(BIT4);
00744         }
00745         else
00746         {
00747             Reset_CFG_STAT(BIT4);
00748         }
00749     }
00750     else if (tbsel == TBSel_STB)
00751     {
00752         if (state == ENABLE)
00753         {
00754             Set_CFG_STAT(BIT3);
00755         }
00756         else
00757         {
00758             Reset_CFG_STAT(BIT3);
00759         }
00760     }
00761 }
00762
00767 void CAN_Enable(void)
00768 {
00769     SYS_ModuleClockCmd(SYS_Module_CAN0, ENABLE);
00770     Reset_CFG_STAT(BIT7);
00771     Delay(100);
00772 }
00777 void CAN_Disable(void)
00778 {
00779     SYS_ModuleClockCmd(SYS_Module_CAN0, DISABLE);
00780     Set_CFG_STAT(BIT7);

```

```

00781 }
00786 void CAN_Reset(void)
00787 {
00788     Set_CFG_STAT(BIT7);
00789     Delay(100);
00790     Reset_CFG_STAT(BIT7);
00791     Delay(100);
00792 }
00793
00794 /*
00795  * 发送缓冲区模式设置, 对应CAN_TCTRL.TSMODE位
00796  */
00797 void CAN_SetSTBMode(STBTransMode tsMode)
00798 {
00799     /* 只有STB为空时, 才能切换TSMODE */
00800     if ((CAN_TCTRL & 0x03) > 0)
00801     {
00802         return;
00803     }
00804
00805     if (tsMode == STB_FIFOMode)
00806     {
00807         CAN_TCTRL &= ~BIT5;
00808     }
00809     else
00810     {
00811         CAN_TCTRL |= BIT5;
00812     }
00813 }
00814
00815 u8 CAN_STBFullGet(void)
00816 {
00817     return (CAN->TCTRL & 0x03) == 0x03;
00818 }
00819
00824 u8 CAN_RxErrorCntGet(void)
00825 {
00826     return CAN->RECNTR & 0xFF;
00827 }
00828
00833 u8 CAN_TxErrorCntGet(void)
00834 {
00835     return CAN->TECNTR & 0xFF;
00836 }
00837
00842 u8 CAN_ErrorPassiveGet(void)
00843 {
00844     return (CAN->ERRINT >> 6) & 0x01;
00845 }
00846
00851 u8 CAN_BusoffGet(void)
00852 {
00853     return CAN->CFG_STAT & 0x01;
00854 }
00855
00860 CANErrorKind CAN_KindOfErrorGet(void)
00861 {
00862     u8 koer = (CAN->EALCAP >> 5) & 0x07;
00863     return (CANErrorKind)koer;
00864 }
00865
00870 u8 CAN_ArbitrationLostCap(void)
00871 {
00872     return (CAN->EALCAP & 0x1f);
00873 }
00874
00879 u8 CAN_ROverflowGet(void)
00880 {
00881     return (CAN->RCTRL >> 5) & 0x01;
00882 }
00883

```

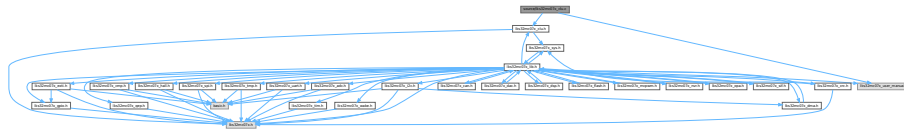
5.55 source/lks32mc07x_clu.c 文件参考

```

#include "lks32mc07x_clu.h"
#include "lks32mc07x_user_manual.h"

```

lks32mc07x.clu.c 的引用(Include)关系图:



函数

- void **CLU_Init** (CLU_TypeDef *CLUx, u8 CLU_MODEL_CLUx, CLU_InitTypeDef *this)
CLU计算初始化
- void **CLU_StructInit** (CLU_InitTypeDef *this)
CLU初始化结构体初始化
- void **CLU_Enable** (CLU_TypeDef *CLUx)
CLU模块时钟使能
- void **CLU_Disable** (CLU_TypeDef *CLUx)
CLU模块时钟关闭
- u8 **CLU_ReadOutValue** (CLU_TypeDef *CLUx, u8 CLU_MODEL_CLUx)
获取CLU的输出值

5.56 lks32mc07x.clu.c

[浏览该文件的文档.](#)

```

00001
00009 #include "lks32mc07x.clu.h"
00010 #include "lks32mc07x.user_manual.h"
00014 void CLU_Init(CLU_TypeDef *CLUx, u8 CLU_MODEL_CLUx, CLU_InitTypeDef *this)
00015 {
00016     CLU_Enable(CLUx);
00017     if ((CLU_MODEL_CLUx & CLU_MODEL_CLU0) != 0)
00018     {
00019         REG_IF_WRITE_BIT(CLUx->EN, CL0_EN0_C0EN_BIT, this->EN);           // CLUx 使能。默认关闭。 0：关闭
00020         1：使能
00021         REG_IF_WRITE_BIT(CLUx->IE, CL0_IE0_C0FIE_BIT, this->FIE);           // CLUx 下降沿触发中断使能。默认关
00022         闭。 0：关闭 1：使能
00023         REG_IF_WRITE_BIT(CLUx->IE, CL0_IE0_C0RIE_BIT, this->RIE);           // CLUx 上升沿触发中断使能。默认关
00024         闭。 0：关闭 1：使能
00025         REG_WRITE_BIT_POS_MASK(CLUx->MX, CL0_MX0_C0MXA, this->MXA);         // CLUx A输入多路复用选择
00026         REG_WRITE_BIT_POS_MASK(CLUx->MX, CL0_MX0_C0MXB, this->MXB);         // CLUx B输入多路复用选择
00027         REG_WRITE_BIT_POS_MASK(CLUx->FN, CL0_FN0_C0FN, this->FN);           // CLUx 查找表功能选择。
00028         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C0OUTSEL_BIT, this->OUTSEL);     // CLUx 输出选取 0：D触发器输出
00029         1：LUT输出
00030         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C0OEN_BIT, this->OEN);           // CLUx 输出使能。默认关闭。 0：关
00031         闭 1：使能
00032         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C0CLKINV_BIT, this->CLKINV);     // CLUx D触发器时钟电平反向。 0：
00033         关闭 1：使能
00034         REG_WRITE_BIT_POS_MASK(CLUx->CF, CL0_CF0_C0CLKSEL, this->CLKSEL);   // CLUx D触发器时钟选择。 0x0：
00035         进位输入 0x1：MXA输入 0x2：ALTCCLK0 0x3：ALTCCLK1
00036         REG_SET(CLUx->CF, CL0_CF0_C0RST_BIT);                               // CLUx D触发器复位。该位写1复
00037         位。读取恒为0。
00038     }
00039     if ((CLU_MODEL_CLUx & CLU_MODEL_CLU1) != 0)
00040     {
00041         REG_IF_WRITE_BIT(CLUx->EN, CL0_EN0_C1EN_BIT, this->EN);           // CLUx 使能。默认关闭。 0：关闭
00042         1：使能
00043         REG_IF_WRITE_BIT(CLUx->IE, CL0_IE0_C1FIE_BIT, this->FIE);           // CLUx 下降沿触发中断使能。默认关
00044         闭。 0：关闭 1：使能
00045         REG_IF_WRITE_BIT(CLUx->IE, CL0_IE0_C1RIE_BIT, this->RIE);           // CLUx 上升沿触发中断使能。默认关
00046         闭。 0：关闭 1：使能
00047         REG_WRITE_BIT_POS_MASK(CLUx->MX, CL0_MX0_C1MXA, this->MXA);         // CLUx A输入多路复用选择
00048         REG_WRITE_BIT_POS_MASK(CLUx->MX, CL0_MX0_C1MXB, this->MXB);         // CLUx B输入多路复用选择
00049         REG_WRITE_BIT_POS_MASK(CLUx->FN, CL0_FN0_C1FN, this->FN);           // CLUx 查找表功能选择。
00050         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C1OUTSEL_BIT, this->OUTSEL);     // CLUx 输出选取 0：D触发器输出
00051         1：LUT输出
00052         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C1OEN_BIT, this->OEN);           // CLUx 输出使能。默认关闭。 0：关
00053         闭 1：使能
    
```

```

00041     REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C1CLKINV_BIT, this->CLKINV); // CLUx D触发器时钟电平反向。 0:
      关闭 1: 使能
00042     REG_WRITE_BIT_POS_MASK(CLUx->CF, CL0_CF0_C1CLKSEL, this->CLKSEL); // CLUx D触发器时钟选择。 0x0:
      进位输入 0x1: MXA输入 0x2: ALTCLK0 0x3: ALTCLK1
00043     REG_SET(CLUx->CF, CL0_CF0_C1RST_BIT); // CLUx D触发器复位。该位写1复
      位。读取恒为0。
00044     }
00045     if ((CLU_MODEL_CLUx & CLU_MODEL_CLU2) != 0)
00046     {
00047         REG_IF_WRITE_BIT(CLUx->EN, CL0_EN0_C2EN_BIT, this->EN); // CLUx 使能。默认关闭。 0: 关闭
      1: 使能
00048         REG_IF_WRITE_BIT(CLUx->IE, CL0_IE0_C2FIE_BIT, this->FIE); // CLUx 下降沿触发中断使能。默认关
      闭。 0: 关闭 1: 使能
00049         REG_IF_WRITE_BIT(CLUx->IE, CL0_IE0_C2RIE_BIT, this->RIE); // CLUx 上升沿触发中断使能。默认关
      闭。 0: 关闭 1: 使能
00050         REG_WRITE_BIT_POS_MASK(CLUx->MX, CL0_MX0_C2MXA, this->MXA); // CLUx A输入多路复用选择
00051         REG_WRITE_BIT_POS_MASK(CLUx->MX, CL0_MX0_C2MXB, this->MXB); // CLUx B输入多路复用选择
00052         REG_WRITE_BIT_POS_MASK(CLUx->FN, CL0_FN0_C2FN, this->FN); // CLUx 查找表功能选择。
00053         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C2OUTSEL_BIT, this->OUTSEL); // CLUx 输出选取 0: D触发器输出
      1: LUT输出
00054         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C2OEN_BIT, this->OEN); // CLUx 输出使能。默认关闭。 0: 关
      闭 1: 使能
00055         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C2CLKINV_BIT, this->CLKINV); // CLUx D触发器时钟电平反向。 0:
      关闭 1: 使能
00056         REG_WRITE_BIT_POS_MASK(CLUx->CF, CL0_CF0_C2CLKSEL, this->CLKSEL); // CLUx D触发器时钟选择。 0x0:
      进位输入 0x1: MXA输入 0x2: ALTCLK0 0x3: ALTCLK1
00057         REG_SET(CLUx->CF, CL0_CF0_C2RST_BIT); // CLUx D触发器复位。该位写1复
      位。读取恒为0。
00058     }
00059     if ((CLU_MODEL_CLUx & CLU_MODEL_CLU3) != 0)
00060     {
00061         REG_IF_WRITE_BIT(CLUx->EN, CL0_EN0_C3EN_BIT, this->EN); // CLUx 使能。默认关闭。 0: 关闭
      1: 使能
00062         REG_IF_WRITE_BIT(CLUx->IE, CL0_IE0_C3FIE_BIT, this->FIE); // CLUx 下降沿触发中断使能。默认关
      闭。 0: 关闭 1: 使能
00063         REG_IF_WRITE_BIT(CLUx->IE, CL0_IE0_C3RIE_BIT, this->RIE); // CLUx 上升沿触发中断使能。默认关
      闭。 0: 关闭 1: 使能
00064         REG_WRITE_BIT_POS_MASK(CLUx->MX, CL0_MX0_C3MXA, this->MXA); // CLUx A输入多路复用选择
00065         REG_WRITE_BIT_POS_MASK(CLUx->MX, CL0_MX0_C3MXB, this->MXB); // CLUx B输入多路复用选择
00066         REG_WRITE_BIT_POS_MASK(CLUx->FN, CL0_FN0_C3FN, this->FN); // CLUx 查找表功能选择。
00067         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C3OUTSEL_BIT, this->OUTSEL); // CLUx 输出选取 0: D触发器输出
      1: LUT输出
00068         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C3OEN_BIT, this->OEN); // CLUx 输出使能。默认关闭。 0: 关
      闭 1: 使能
00069         REG_IF_WRITE_BIT(CLUx->CF, CL0_CF0_C3CLKINV_BIT, this->CLKINV); // CLUx D触发器时钟电平反向。 0:
      关闭 1: 使能
00070         REG_WRITE_BIT_POS_MASK(CLUx->CF, CL0_CF0_C3CLKSEL, this->CLKSEL); // CLUx D触发器时钟选择。 0x0:
      进位输入 0x1: MXA输入 0x2: ALTCLK0 0x3: ALTCLK1
00071         REG_SET(CLUx->CF, CL0_CF0_C3RST_BIT); // CLUx D触发器复位。该位写1复
      位。读取恒为0。
00072     }
00073 }
00077 void CLU_StructInit(CLU_InitTypeDef *this)
00078 {
00079     this->FIE = 0;
00080     this->RIE = 0;
00081     this->EN = 0;
00082     this->MXA = 0;
00083     this->MXB = 0;
00084     this->FN = 0;
00085     this->OUTSEL = 0;
00086     this->OEN = 0;
00087     this->CLKINV = 0;
00088     this->CLKSEL = 0;
00089 }
00093 void CLU_Enable(CLU_TypeDef *CLUx)
00094 {
00095     SYS_ModuleClockCmd(SYS_Module.CL0, ENABLE);
00096 }
00100 void CLU_Disable(CLU_TypeDef *CLUx)
00101 {
00102     SYS_ModuleClockCmd(SYS_Module.CL0, DISABLE);
00103 }
00107 u8 CLU_ReadOutValue(CLU_TypeDef *CLUx, u8 CLU_MODEL_CLUx)
00108 {
00109     u8 OutValue;
00110     if (CLUx->OUT & CLU_MODEL_CLUx)
00111     {
00112         OutValue = 1;
00113     }
00114     else
00115     {
00116         OutValue = 0;
00117     }
00118     return OutValue;
00119 }
00120

```


5.57 source/lks32mc07x_cmp.c 文件参考

文件名称: lks32mc07x_cmp.c
文件标识: 无
内容摘要: cmp外设驱动程序
其它说明: 无

```
#include "lks32mc07x_cmp.h"
#include "lks32mc07x.h"
#include "string.h"
#include "lks32mc07x_sys.h"
lks32mc07x_cmp.c 的引用(Include)关系图:
```



函数

- void **CMP_Init** (CMP_InitTypeDef *this)
函数名称: *CMP_Init(CMP_InitTypeDef* this)*
- void **CMP_StructInit** (CMP_InitTypeDef *CMP_InitStruct)
函数名称: *void CMP_StructInit(CMP_InitTypeDef* CMP_InitStruct)*
- u8 **CMP_GetIRQFlag** (u8 CMP_CHN_x)
函数名称: *void CMP_Cmd(u8 CMP_CHN_x,FuncState state)*
- void **CMP_ClearIRQFlag** (u8 CMP_CHN_x)
函数名称: *void CMP_ClearIRQFlag(u8 CMP_CHN_x)*
- uint8_t **CMP_GetFilterValue** (void)
函数名称: *uint8_t CMP_GetFilterValue(void)*
- uint8_t **CMP_GetValue** (void)
函数名称: *uint8_t CMP_GetValue(void)*

5.57.1 详细描述

文件名称: lks32mc07x_cmp.c
文件标识: 无
内容摘要: cmp外设驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 中定义.

5.57.2 函数说明

5.57.2.1 CMP_ClearIRQFlag()

```
void CMP_ClearIRQFlag (
    u8 CMP_CHN_x )
```

函数名称: `void CMP_ClearIRQFlag(u8 CMP_CHN_x)`

功能描述: 清除TIM中断标志

参见

被调用函数: 无

参数

输入参数:	CMP_CHN_0, CMP_CHN_1即获取CMP0和CMP1中断标志位
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (CMP_GetIRQFlag (CMP_CHN_1)) //获取CMP1中断标志位
{
    CMP_ClearIRQFlag (CMP_CHN_1); //清除CMP1中断标志位
}
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 214 行定义.

5.57.2.2 CMP_GetFilterValue()

```
uint8_t CMP_GetFilterValue (  
    void )
```

函数名称: `uint8_t CMP_GetFilterValue(void)`

功能描述: 读取滤波后的CMP值

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
u8 cmpVal;  
cmpVal = CMP_GetFilterValue();
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 239 行定义.

5.57.2.3 CMP_GetIRQFlag()

```
u8 CMP_GetIRQFlag (
    u8 CMP_CHN_x )
```

函数名称: void [CMP_Cmd\(u8 CMP_CHN_x,FuncState state\)](#)

功能描述: CMP时钟使能

参见
被调用函数: 无

参数

输入参数:	CMP_CHN_x: CMP_CHN_0/CMP_CHN_1/CMP_CHN_2,state: ENABLE/DISABLE
输出参数:	无

返回
返回 值: 无

注解
其它说明: 无

警告
无

示例代码:

```
        CMP\_Cmd\(CMP\_CHN\_0,ENABLE\); //CMP0使能
@par 修改日志:


| Date        | Version | Author     | Description |
|-------------|---------|------------|-------------|
| 2023年02月22日 | 1.0     | Olive Wang | 创建          |


/  

void CMP\_Cmd(u8 CMP_CHN_x, FuncState state)
{
    if (CMP_CHN_0 == CMP_CHN_x)
    {
        SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP0, state); // CMP0使能
    }
    else if (CMP_CHN_1 == CMP_CHN_x)
    {
        SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP1, state); // CMP1使能
    }
    else if (CMP_CHN_2 == CMP_CHN_x)
    {
        SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP2, state); // CMP2使能
    }
    SYS\_ModuleClockCmd\(SYS\_Module\_CMP, state\); // 数字电路时钟使能
}
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 180 行定义.

5.57.2.4 CMP_GetValue()

```
uint8_t CMP_GetValue (
    void )
```

函数名称: `uint8_t CMP_GetValue(void)`

功能描述: 读取未滤波的CMP值

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
u8 cmpVal;
cmpVal = CMP_GetValue();
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 263 行定义.

5.57.2.5 CMP_Init()

```
void CMP_Init (
    CMP_InitTypeDef * this )
```

函数名称: [CMP_Init\(CMP_InitTypeDef* this\)](#)

功能描述: [CMP](#)初始化

参见

被调用函数: [SYS_AnalogModuleClockCmd\(\)](#)

参数

输入参数:	CMP_InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
CMP_InitTypeDef CMP_InitStructure;
CMP_StructInit (&CMP_InitStructure);

CMP_InitStructure.CLK_COM_DIV = 0; // 比较器共用滤波时钟分频
CMP_InitStructure.FT = DISABLE; // 比较器快速比较
CMP_InitStructure.HYS = CMP_HYS_20mV; // 比较器滞回电压

CMP_InitStructure.CMP0_SELP = CMP0_SELP_IP1; // 比较器0正端信号选择
CMP_InitStructure.CMP0_SELN = CMP_SELN_REF; // 比较器0负端信号选择
CMP_InitStructure.CMP0_RE = DISABLE; // 比较器0DMA失能
CMP_InitStructure.CMP0_POL= CMP_HIGH_LEVEL; // 比较器0高电平有效
CMP_InitStructure.CMP0_IRQ_TRIG = IRQ_LEVEL_TRIG_MODE; // 比较器0电平触发中断模式
CMP_InitStructure.CMP0_IN_EN = ENABLE; // 比较器0信号输入使能
CMP_InitStructure.CMP0_IE = ENABLE; // 比较器0信号中断使能
CMP_InitStructure.CMP0.FIL_CLK_DIV16 = 2; // 即滤波宽度=tclk 周期*16*CMP_FltCnt (CMP_FltCnt分频系数,0~15)
CMP_InitStructure.CMP0.FIL_CLK_DIV2 = 2; // 比较器 2/1/0 滤波时钟使能
CMP_InitStructure.CMP0.CLK_EN = ENABLE; // 比较器时钟使能
CMP_InitStructure.CMP0.EN = ENABLE; // 比较器0开关 操作SYS_AFE_REGS

CMP_Init (&CMP_InitStructure); // 比较器初始化
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_cmp.c](#) 第 57 行定义.

5.57.2.6 CMP_StructInit()

```
void CMP_StructInit (
    CMP\_InitTypeDef * CMP_InitStruct )
```

函数名称: void [CMP_StructInit](#)([CMP_InitTypeDef](#)* *CMP_InitStruct*)

功能描述: ADC结构体初始化

参见

被调用函数: 无

参数

输入参数:	CMP_InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
CMP\_StructInit CMP_InitStructure;
CMP\_StructInit (&CMP_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 `lks32mc07x_cmp.c` 第 117 行定义.

5.58 lks32mc07x_cmp.c

[浏览该文件的文档.](#)

```

00001
00014 #include "lks32mc07x_cmp.h"
00015 #include "lks32mc07x.h"
00016 #include "string.h"
00017 #include "lks32mc07x_sys.h"
00018
00057 void CMP_Init(CMP_InitTypeDef *this)
00058 {
00059     SYS_WR_PROTECT = 0x7A83;
00060
00061     SYS_AFE_REG1 |= this->FT; /* 快速比较, Enable后 30nS */
00062
00063     SYS_AFE_REG2 &= ~(1 << 14) & ~(1 << 15);
00064     SYS_AFE_REG2 |= (this->CMP2_SEL << 14);
00065
00066     SYS_AFE_REG3 |= (this->CMP1_SEL << 12) | (this->CMP1_SELN << 4) | (this->CMP0_SEL << 8) |
    (this->CMP0_SELN << 2) |
00067         (this->HYS << 7) | (this->CMP2_SELN);
00068
00069     SYS_AFE_REG5 |= (this->CMP2_EN << 11) | (this->CMP1_EN << 7) | (this->CMP0_EN << 6);
00070
00071     SYS_WR_PROTECT = 0x0000;
00072
00073     CMP->IE = (this->CMP2_RE << 10) | (this->CMP1_RE << 9) | (this->CMP0_RE << 8) |
    (this->CMP2_IE << 2) | (this->CMP1_IE << 1) | (this->CMP0_IE); // 中断
00074
00075     CMP->TCLK = (this->CLK_COM_DIV << 24) |
    (this->CMP2_FIL_CLK_DIV16 << 20) | (this->CMP2_FIL_CLK_DIV2 << 16) |
    (this->CMP1_FIL_CLK_DIV16 << 12) | (this->CMP1_FIL_CLK_DIV2 << 8) |
    (this->CMP0_FIL_CLK_DIV16 << 4) | (this->CMP0_FIL_CLK_DIV2);
00076
00077     (this->CMP2_CLK_EN << 19) | (this->CMP1_CLK_EN << 11) | (this->CMP0_CLK_EN << 3);
00078
00079     CMP->CFG = (this->CMP2_IRQ_TRIG << 10) |
    (this->CMP2_IN_EN << 9) | (this->CMP2_POL << 8) |
    (this->CMP1_W_PWM_POL << 7) | (this->CMP1_IRQ_TRIG << 6) |
    (this->CMP1_IN_EN << 5) | (this->CMP1_POL << 4) |
    (this->CMP0_W_PWM_POL << 3) | (this->CMP0_IRQ_TRIG << 2) |
    (this->CMP0_IN_EN << 1) | (this->CMP0_POL);
00080
00081     CMP->BLCWIN = (this->CMP1_CHN3P_PWM_EN << 7) | (this->CMP1_CHN2P_PWM_EN << 6) |
    (this->CMP1_CHN1P_PWM_EN << 5) | (this->CMP1_CHN0P_PWM_EN << 4) |
    (this->CMP0_CHN3P_PWM_EN << 3) | (this->CMP0_CHN2P_PWM_EN << 2) |
    (this->CMP0_CHN1P_PWM_EN << 1) | (this->CMP0_CHN0P_PWM_EN);
00082
00083     CMP->IF = 3; // 清零标志位
00084 }
00085
00117 void CMP_StructInit(CMP_InitTypeDef *CMP_InitStruct)
00118 {
00119     memset(CMP_InitStruct, 0, sizeof(CMP_InitTypeDef));
00120 }
00121
00122 void CMP_Cmd(u8 CMP_CHN_x, FuncState state)
00123 {
00124     if (CMP_CHN_0 == CMP_CHN_x)
00125     {
00126         SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP0, state); // CMP0使能
00127     }
00128     else if (CMP_CHN_1 == CMP_CHN_x)
00129     {
00130         SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP1, state); // CMP1使能
00131     }
00132     else if (CMP_CHN_2 == CMP_CHN_x)
00133     {
00134         SYS_AnalogModuleClockCmd(SYS_AnalogModule_CMP2, state); // CMP2使能
00135     }
00136     SYS_ModuleClockCmd(SYS_Module_CMP, state); // 数字电路时钟使能
00137 }

```



```

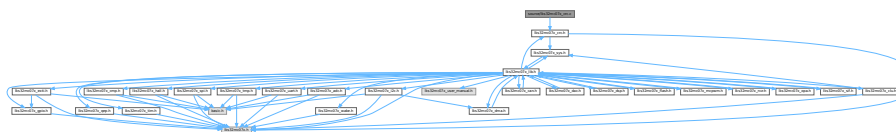
00180 u8 CMP_GetIRQFlag(u8 CMP_CHN_x)
00181 {
00182     if (CMP->IF & (BIT0 << CMP_CHN_x))
00183     {
00184         return 1;
00185     }
00186     else
00187     {
00188         return 0;
00189     }
00190 }
00214 void CMP_ClearIRQFlag(u8 CMP_CHN_x)
00215 {
00216     CMP->IF = (BIT0 << CMP_CHN_x);
00217 }
00218
00239 uint8_t CMP_GetFilterValue(void)
00240 {
00241     return ((CMP_DATA >> 8) & 0x0007);
00242 }
00263 uint8_t CMP_GetValue(void)
00264 {
00265     return (CMP_DATA & 0x0007);
00266 }
00267 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/
00268 /* -----END OF FILE----- */

```

5.59 source/lks32mc07x_crc.c 文件参考

```
#include "lks32mc07x_crc.h"
```

lks32mc07x_crc.c 的引用(Include)关系图:



函数

- void **CRC_Init** (CRC_TypeDef *CRCx, CRC_InitTypeDef *this)
CRC计算初始化
- void **CRC_StructInit** (CRC_InitTypeDef *this)
CRC初始化结构体初始化
- u32 **CRC_Calc** (CRC_TypeDef *CRCx, u8 *data, u32 len)
CRC计算
- u32 **CRC_CalcCommon** (CRC_TypeDef *CRCx, u8 *data, u32 len, u16 CRC_MODEL_x)
CRC计算接口
- void **CRC_Enable** (CRC_TypeDef *CRCx)
CRC模块时钟使能
- void **CRC_Disable** (CRC_TypeDef *CRCx)
CRC模块时钟关闭

变量

- static u32 **CRC_XOROUT** = 0

5.60 lks32mc07x_crc.c

[浏览该文件的文档。](#)

```

00001
00009 #include "lks32mc07x_crc.h"
00010 static u32 CRC_XOROUT = 0;
00011
00015 void CRC_Init(CRC_TypeDef *CRCx, CRC_InitTypeDef *this)
00016 {
00017     if (this->WIDTH > 3)
00018     {
00019         switch (this->WIDTH)
00020         {
00021             case 7:
00022                 this->WIDTH = CRC_WIDTH_7;
00023                 break;
00024             case 8:
00025                 this->WIDTH = CRC_WIDTH_8;
00026                 break;
00027             case 16:
00028                 this->WIDTH = CRC_WIDTH_16;
00029                 break;
00030             case 32:
00031                 this->WIDTH = CRC_WIDTH_32;
00032                 break;
00033             default:
00034                 this->WIDTH = CRC_WIDTH_8;
00035                 break;
00036         }
00037     }
00038     CRC_Enable(CRCx);
00039     SYS_SoftResetModule(SYS_Module_CRC0);
00040     CRC0_CR = (u32)(this->REFOUT << 12) | (u32)(this->REFIN << 8) | (u32)(this->WIDTH << 4);
00041     CRC0_CR |= BIT0; // 复位
00042     CRC0_POL = this->POLY;
00043     CRC0_INIT = this->INIT;
00044     if (this->WIDTH == CRC_WIDTH_16)
00045     {
00046         CRC_XOROUT = this->XOROUT << 16;
00047     }
00048     else
00049     {
00050         CRC_XOROUT = this->XOROUT;
00051     }
00052 }
00056 void CRC_StructInit(CRC_InitTypeDef *this)
00057 {
00058     this->POLY = 7; // 多项式
00059     this->INIT = 0; // 初始值
00060     this->XOROUT = 0; // 结果异或值
00061     this->REFIN = 0; // 输入数据反转
00062     this->REFOUT = 0; // 输出数据反转
00063     this->WIDTH = CRC_WIDTH_8; // 宽度
00064 }
00068 u32 CRC_Calc(CRC_TypeDef *CRCx, u8 *data, u32 len)
00069 {
00070     u32 val;
00071     if (len == 0)
00072     {
00073         return 0;
00074     }
00075     else
00076     {
00077         while ((len--) != 0)
00078         {
00079             REG8(&CRCx->DR) = *data;
00080             data++;
00081         }
00082     }
00083     val = (REG32(&CRCx->DR)) ^ CRC_XOROUT;
00084     return val;
00085 }
00089 u32 CRC_CalcCommon(CRC_TypeDef *CRCx, u8 *data, u32 len, u16 CRC_MODEL_x)
00090 {
00091     u32 val;
00092     CRC_InitTypeDef CRC_InitStruct;
00093     switch (CRC_MODEL_x)
00094     {
00095         case CRC_MODEL_CRC8: // 8 07 00 00 false false
00096             CRC_InitStruct.WIDTH = CRC_WIDTH_8; // 宽度
00097             CRC_InitStruct.POLY = 0x07; // 多项式
00098             CRC_InitStruct.INIT = 0x00; // 初始值
00099             CRC_InitStruct.XOROUT = 0x00; // 结果异或值
00100             CRC_InitStruct.REFIN = 0x00; // 输入数据反转
00101             CRC_InitStruct.REFOUT = DISABLE;

```

```

00102         CRC_InitStructure.REFOUT = DISABLE;          // 输出数据反转
00103         CRC_Init(CRCx, &CRC_InitStructure);
00104         break;
00105     case CRC_MODEL_CRC_8_ITU:                          // 8   07       00       55       false
false
00106         CRC_InitStructure.WIDTH = CRC_WIDTH_8; // 宽度
00107         CRC_InitStructure.POLY  = 0x07;        // 多项式
00108         CRC_InitStructure.INIT  = 0x00;        // 初始值
00109         CRC_InitStructure.XOROUT = 0x55;       // 结果异或值
00110         CRC_InitStructure.REFIN = DISABLE;     // 输入数据反转
00111         CRC_InitStructure.REFOUT = DISABLE;    // 输出数据反转
00112         CRC_Init(CRCx, &CRC_InitStructure);
00113         break;
00114     case CRC_MODEL_CRC_8_ROHC: // 8   07       FF       00       true   true
00115
00116         CRC_InitStructure.WIDTH = CRC_WIDTH_8; // 宽度
00117         CRC_InitStructure.POLY  = 0x07;        // 多项式
00118         CRC_InitStructure.INIT  = 0xff;       // 初始值
00119         CRC_InitStructure.XOROUT = 0x00;       // 结果异或值
00120         CRC_InitStructure.REFIN = ENABLE;     // 输入数据反转
00121         CRC_InitStructure.REFOUT = ENABLE;    // 输出数据反转
00122         CRC_Init(CRCx, &CRC_InitStructure);
00123         break;
00124     case CRC_MODEL_CRC_8_MAXIMx: // 8   31       00       00       true   true
00125
00126         CRC_InitStructure.WIDTH = CRC_WIDTH_8; // 宽度
00127         CRC_InitStructure.POLY  = 0x31;        // 多项式
00128         CRC_InitStructure.INIT  = 0x00;        // 初始值
00129         CRC_InitStructure.XOROUT = 0x00;       // 结果异或值
00130         CRC_InitStructure.REFIN = ENABLE;     // 输入数据反转
00131         CRC_InitStructure.REFOUT = ENABLE;    // 输出数据反转
00132         CRC_Init(CRCx, &CRC_InitStructure);
00133         break;
00134     case CRC_MODEL_CRC_16_IBMx1: // 16  8005      0000      0000      true   true
00135
00136         CRC_InitStructure.WIDTH = CRC_WIDTH_16; // 宽度
00137         CRC_InitStructure.POLY  = 0x8005;       // 多项式
00138         CRC_InitStructure.INIT  = 0x0000;      // 初始值
00139         CRC_InitStructure.XOROUT = 0x0000;     // 结果异或值
00140         CRC_InitStructure.REFIN = ENABLE;     // 输入数据反转
00141         CRC_InitStructure.REFOUT = ENABLE;    // 输出数据反转
00142         CRC_Init(CRCx, &CRC_InitStructure);
00143         break;
00144     case CRC_MODEL_CRC_16_MAXIM: // 16  8005      0000      FFFF      true   true
00145
00146         CRC_InitStructure.WIDTH = CRC_WIDTH_16; // 宽度
00147         CRC_InitStructure.POLY  = 0x8005;       // 多项式
00148         CRC_InitStructure.INIT  = 0x0000;      // 初始值
00149         CRC_InitStructure.XOROUT = 0xffff;     // 结果异或值
00150         CRC_InitStructure.REFIN = ENABLE;     // 输入数据反转
00151         CRC_InitStructure.REFOUT = ENABLE;    // 输出数据反转
00152         CRC_Init(CRCx, &CRC_InitStructure);
00153         break;
00154     case CRC_MODEL_CRC_16_USBx1: // 16  8005      FFFF      FFFF      true   true
00155
00156         CRC_InitStructure.WIDTH = CRC_WIDTH_16; // 宽度
00157         CRC_InitStructure.POLY  = 0x8005;       // 多项式
00158         CRC_InitStructure.INIT  = 0xffff;     // 初始值
00159         CRC_InitStructure.XOROUT = 0xffff;     // 结果异或值
00160         CRC_InitStructure.REFIN = ENABLE;     // 输入数据反转
00161         CRC_InitStructure.REFOUT = ENABLE;    // 输出数据反转
00162         CRC_Init(CRCx, &CRC_InitStructure);
00163         break;
00164     case CRC_MODEL_CRC_16_MODBU: // 16  8005      FFFF      0000      true   true
00165
00166         CRC_InitStructure.WIDTH = CRC_WIDTH_16; // 宽度
00167         CRC_InitStructure.POLY  = 0x8005;       // 多项式
00168         CRC_InitStructure.INIT  = 0xffff;     // 初始值
00169         CRC_InitStructure.XOROUT = 0x0000;     // 结果异或值
00170         CRC_InitStructure.REFIN = ENABLE;     // 输入数据反转
00171         CRC_InitStructure.REFOUT = ENABLE;    // 输出数据反转
00172         CRC_Init(CRCx, &CRC_InitStructure);
00173         break;
00174     case CRC_MODEL_CRC_16_CCITT: // 16  1021      0000      0000      true   true
00175
00176         CRC_InitStructure.WIDTH = CRC_WIDTH_16; // 宽度
00177         CRC_InitStructure.POLY  = 0x1021;      // 多项式
00178         CRC_InitStructure.INIT  = 0x0000;      // 初始值
00179         CRC_InitStructure.XOROUT = 0x0000;     // 结果异或值
00180         CRC_InitStructure.REFIN = ENABLE;     // 输入数据反转
00181         CRC_InitStructure.REFOUT = ENABLE;    // 输出数据反转
00182         CRC_Init(CRCx, &CRC_InitStructure);
00183         break;
00184     case CRC_MODEL_CRC_16_CCITT_FALSE: // 16  1021      FFFF      0000      false  false
00185
00186         CRC_InitStructure.WIDTH = CRC_WIDTH_16; // 宽度
00187         CRC_InitStructure.POLY  = 0x1021;      // 多项式

```

```

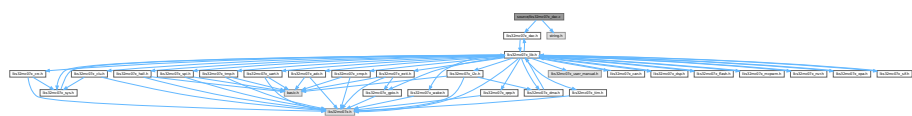
00188         CRC_InitStruct.INIT    = 0xffff;          // 初始值
00189         CRC_InitStruct.XOROUT   = 0x0000;          // 结果异或值
00190         CRC_InitStruct.REFIN    = DISABLE;         // 输入数据反转
00191         CRC_InitStruct.REFOUT   = DISABLE;         // 输出数据反转
00192         CRC_Init(CRCx, &CRC_InitStruct);
00193         break;
00194     case CRC_MODEL_CRC_16_X25: // 16 1021          FFFF          FFFF          true      true
00195
00196         CRC_InitStruct.WIDTH    = CRC_WIDTH_16;    // 宽度
00197         CRC_InitStruct.POLY     = 0x1021;          // 多项式
00198         CRC_InitStruct.INIT     = 0xffff;          // 初始值
00199         CRC_InitStruct.XOROUT   = 0xffff;          // 结果异或值
00200         CRC_InitStruct.REFIN    = ENABLE;          // 输入数据反转
00201         CRC_InitStruct.REFOUT   = ENABLE;          // 输出数据反转
00202         CRC_Init(CRCx, &CRC_InitStruct);
00203         break;
00204     case CRC_MODEL_CRC_16_XMODEM: // 16 1021          0000          0000          false     false
00205
00206         CRC_InitStruct.WIDTH    = CRC_WIDTH_16;    // 宽度
00207         CRC_InitStruct.POLY     = 0x1021;          // 多项式
00208         CRC_InitStruct.INIT     = 0x0000;          // 初始值
00209         CRC_InitStruct.XOROUT   = 0x0000;          // 结果异或值
00210         CRC_InitStruct.REFIN    = DISABLE;         // 输入数据反转
00211         CRC_InitStruct.REFOUT   = DISABLE;         // 输出数据反转
00212         CRC_Init(CRCx, &CRC_InitStruct);
00213         break;
00214     case CRC_MODEL_CRC_16_DNP: // 16 3D65          0000          FFFF          true      true
00215
00216         CRC_InitStruct.WIDTH    = CRC_WIDTH_16;    // 宽度
00217         CRC_InitStruct.POLY     = 0x3d65;          // 多项式
00218         CRC_InitStruct.INIT     = 0x0000;          // 初始值
00219         CRC_InitStruct.XOROUT   = 0xffff;          // 结果异或值
00220         CRC_InitStruct.REFIN    = ENABLE;          // 输入数据反转
00221         CRC_InitStruct.REFOUT   = ENABLE;          // 输出数据反转
00222         CRC_Init(CRCx, &CRC_InitStruct);
00223         break;
00224     case CRC_MODEL_CRC_32: // 32 04C11DB7          FFFFFFFF          FFFFFFFF          true      true
00225
00226         CRC_InitStruct.WIDTH    = CRC_WIDTH_32;    // 宽度
00227         CRC_InitStruct.POLY     = 0x04C11DB7;      // 多项式
00228         CRC_InitStruct.INIT     = 0xFFFFFFFF;      // 初始值
00229         CRC_InitStruct.XOROUT   = 0xFFFFFFFF;      // 结果异或值
00230         CRC_InitStruct.REFIN    = ENABLE;          // 输入数据反转
00231         CRC_InitStruct.REFOUT   = ENABLE;          // 输出数据反转
00232         CRC_Init(CRCx, &CRC_InitStruct);
00233         break;
00234     case CRC_MODEL_CRC_32_MPEG_2: // 32 04C11DB7          FFFFFFFF          00000000          false     false
00235
00236         CRC_InitStruct.WIDTH    = CRC_WIDTH_32;    // 宽度
00237         CRC_InitStruct.POLY     = 0x04C11DB7;      // 多项式
00238         CRC_InitStruct.INIT     = 0xFFFFFFFF;      // 初始值
00239         CRC_InitStruct.XOROUT   = 0x00000000;      // 结果异或值
00240         CRC_InitStruct.REFIN    = DISABLE;         // 输入数据反转
00241         CRC_InitStruct.REFOUT   = DISABLE;         // 输出数据反转
00242         CRC_Init(CRCx, &CRC_InitStruct);
00243         break;
00244     }
00245     val = CRC_Calc(CRCx, data, len);
00246     if (CRC_InitStruct.REFOUT)
00247     {
00248         switch (CRC_InitStruct.WIDTH)
00249         {
00250             case CRC_WIDTH_8:
00251                 val >>= 24;
00252                 break;
00253             case CRC_WIDTH_16:
00254                 val >>= 16;
00255                 break;
00256         }
00257     }
00258     return val;
00259 }
00263 void CRC_Enable(CRC_TypeDef *CRCx)
00264 {
00265     if (CRCx == CRC0)
00266     {
00267         SYS_ModuleClockCmd(SYS_Module_CRC0, ENABLE);
00268     }
00269 }
00273 void CRC_Disable(CRC_TypeDef *CRCx)
00274 {
00275     if (CRCx == CRC0)
00276     {
00277         SYS_ModuleClockCmd(SYS_Module_CRC0, DISABLE);
00278     }
00279 }
00280

```

5.61 source/lks32mc07x_dac.c 文件参考

文件名称: LKS32MC07x_dac.c
文件标识: 无
内容摘要: DAC外设驱动程序
其它说明: 无

#include "lks32mc07x_dac.h"
#include "string.h"
lks32mc07x_dac.c 的引用(Include)关系图:



函数

- void DAC_StructInit (DAC_InitTypeDef *DAC_InitStruct)
 函数名称: void DAC_StructInit(DAC_InitTypeDef* DAC_InitStruct)
- void DAC_Init (uint8_t DAC_Channel, DAC_InitTypeDef *DAC_InitStruct)
 函数名称: void DAC_init(uint8_t DAC_Channel,DAC_InitTypeDef* DAC_InitStruct)
- void DAC_OutputValue (uint8_t DAC_Channel, uint32_t DACValue)
 函数名称: void DAC_OutputValue(uint8_t DAC_Channel,uint32_t DACValue)
- void DAC_OutputVoltage (uint8_t DAC_Channel, uint16_t DACVoltage)
 函数名称: void DAC_OutputVoltage(uint8_t DAC_Channel,uint32_t DACVoltage)
- void DAC_Cmd (uint8_t DAC_Channel, FuncState state)
 函数名称: void DAC_Cmd(uint8_t DAC_Channel,FuncState state)
- void DAC_Output (uint8_t DAC_Channel, DAC_InitTypeDef *DAC_InitStruct)

5.61.1 详细描述

文件名称: LKS32MC07x_dac.c
文件标识: 无
内容摘要: DAC外设驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年3月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_dac.c](#) 中定义.

5.61.2 函数说明

5.61.2.1 DAC_Cmd()

```
void DAC_Cmd (
    uint8_t DAC_Channel,
    FuncState state )
```

函数名称: `void DAC_Cmd(uint8_t DAC_Channel,FuncState state)`

功能描述: DAC外设使能

参见

被调用函数: 无

参数

输入参数:	DAC_Channel: 通道DAC_Channel_0/DAC_Channel_1, state: ENABLE/DISABLE
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
DAC_Cmd(DAC_Channel_0, ENABLE);
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_dac.c](#) 第 201 行定义.

函数调用图:



5.61.2.2 DAC_Init()

```
void DAC_Init (
    uint8_t DAC_Channel,
    DAC_InitTypeDef * DAC_InitStruct )
```

函数名称: void DAC.init(uint8_t DAC_Channel,DAC_InitTypeDef* DAC_InitStruct)

功能描述: DAC初始化函数

参数

输入参数:	DAC_Channel:DAC_Channel_0/DAC_Channel_1, DAC_InitTypeDef
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

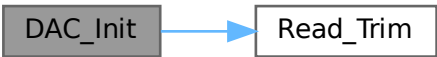
```
DAC_InitTypeDef DAC_InitStructure;
DAC_StructInit (&DAC_InitStructure);
DAC_InitStructure.DAC_GAIN = DAC_RANGE_1V2; //DAC量程为1.2V
DAC_InitStructure.DACOUT_EN = DISABLE ; //DAC通过IO P0[0]输出
DAC_InitStre.TIG.CH.EN = DISABLE; //失能UTIMER触发
DAC_InitStre.DAC.STEP = 0; //步进值为0
DAC_Init (DAC_Channel_0,&DAC_InitStructure); // DAC初始化
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x-dac.c](#) 第 67 行定义.

函数调用图:



5.61.2.3 DAC_Output()

```
void DAC_Output (
    uint8_t DAC_Channel,
    DAC_InitTypeDef * DAC_InitStruct )
```

在文件 [lks32mc07x-dac.c](#) 第 213 行定义.

5.61.2.4 DAC_OutputValue()

```
void DAC_OutputValue (
    uint8_t DAC_Channel,
    uint32_t DACValue )
```

函数名称: void [DAC_OutputValue\(uint8_t DAC_Channel,uint32_t DACValue\)](#)

功能描述: DAC输出数字量数值设置

参见
被调用函数: 无

参数

输入参数:	DAC_Channel: DAC_Channel_0/DAC_Channel_1, DACValue:DAC输出电压对应数字量
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
DAC.OutputValue(DAC.Channel_0,2048); //DAC输出2048*1.2/4096 = 0.6V
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_dac.c](#) 第 122 行定义.

5.61.2.5 DAC_OutputVoltage()

```
void DAC_OutputVoltage (
    uint8_t DAC_Channel,
    uint16_t DACVoltage )
```

函数名称: void DAC_OutputVoltage(uint8_t DAC_Channel,uint32_t DACVoltage)

功能描述: DAC输出模拟量数值设置

参见

被调用函数: 无

参数

输入参数:	DAC_Channel: DAC_Channel_0/DAC_Channel_1, DACVoltage:DAC输出电压对应数字量
输出参数:	无

返回

返回 值: 无

注解

其它说明： 输入变量DACVoltage为Q12格式，即输出1V对应DACVoltage = 1<<12 = 4096

警告

无

示例代码:

```
DAC.OutputVoltage (DAC_Channel1_0, BIT12 * 0.6); //DAC输出0.6V
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 lks32mc07x_dac.c 第 155 行定义.

5.61.2.6 DAC_StructInit()

```
void DAC_StructInit (
    DAC_InitTypeDef * DAC_InitStruct )
```

函数名称: void DAC_StructInit(DAC_InitTypeDef* DAC_InitStruct)

功能描述: DAC结构体初始化

参见

被调用函数： 无

参数

输入参数:	DAC_InitTypeDef
输出参数:	无

返回

返回值： 无

注解

其它说明： 无

警告

无

示例代码:

```
DAC_StructInit DAC_InitStructure;
DAC_StructInit(&DAC_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2023年2月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_dac.c](#) 第 37 行定义.

5.62 lks32mc07x_dac.c

浏览该文件的文档.

```
00001
00015 #include "lks32mc07x_dac.h"
00016 #include "string.h"
00037 void DAC_StructInit(DAC_InitTypeDef *DAC_InitStructure)
00038 {
00039     memset(DAC_InitStructure, 0, sizeof(DAC_InitTypeDef));
00040 }
00041
00067 void DAC_Init(uint8_t DAC_Channel, DAC_InitTypeDef *DAC_InitStructure)
00068 {
00069     SYS_WR_PROTECT = 0x7a83;
00070     SYS_AFE_REG1 &= ~(0x01 << (6 + DAC_Channel));
00071     SYS_AFE_REG1 |= DAC_InitStructure->DAC_GAIN << (6 + DAC_Channel); /* 设置DAC满量程为1.2V 或4.8V */
00072
00073     if (DAC_Channel_0 == DAC_Channel)
00074     {
00075         if ((SYS_AFE_REG1 >> 6) & 0x01) == DAC_RANGE_1V2)
00076         { /* 加载DAC 1.2V量程校正值 */
00077             SYS_AFE_DAC0_DC = Read_Trim(0x0000145C);
00078             SYS_AFE_DAC0_AMC = Read_Trim(0x00001458);
00079         }
00080         else if ((SYS_AFE_REG1 >> 6) & 0x01) == DAC_RANGE_4V85)
00081         { /* 加载DAC 4.85V量程校正值 */
00082             SYS_AFE_DAC0_DC = Read_Trim(0x00001454);
00083             SYS_AFE_DAC0_AMC = Read_Trim(0x00001450);
00084         }
00085         SYS_AFE_DAC_CTRL |= (DAC_InitStructure->DAC_STEP << 4) | (DAC_InitStructure->TIG_CH_EN);
00086     }
00087     else if (DAC_Channel_1 == DAC_Channel)
00088     {
00089         if ((SYS_AFE_REG1 >> 7) & 0x01) == DAC_RANGE_1V2)
00090         { /* 加载DAC 1.2V量程校正值 */
00091             SYS_AFE_DAC1_DC = Read_Trim(0x0000146C);
00092             SYS_AFE_DAC1_AMC = Read_Trim(0x00001468);
00093         }
00094         else if ((SYS_AFE_REG1 >> 7) & 0x01) == DAC_RANGE_4V85)
00095         { /* 加载DAC 4.85V量程校正值 */
00096             SYS_AFE_DAC1_DC = Read_Trim(0x00001464);
00097             SYS_AFE_DAC1_AMC = Read_Trim(0x00001460);
00098         }
00099         SYS_AFE_DAC_CTRL |= (DAC_InitStructure->DAC_STEP << 12) | (DAC_InitStructure->TIG_CH_EN << 8);
00100     }
00101     SYS_WR_PROTECT = 0xffff; /* 锁定寄存器写保护 */
00102 }
00122 void DAC_OutputValue(uint8_t DAC_Channel, uint32_t DAC_Value)
00123 {
00124     DAC_Value &= 0xFFF; /* 判断DAC_Value不大于4095
```

```

00125
00126     if (DAC_Channel0 == DAC_Channel)
00127     {
00128         SYS_AFE_DAC0 = DACValue;
00129     }
00130     else if (DAC_Channel1 == DAC_Channel)
00131     {
00132         SYS_AFE_DAC1 = DACValue;
00133     }
00134 }
00155 void DAC_OutputVoltage(uint8_t DAC_Channel, uint16_t DACVoltage)
00156 {
00157     u32 temp = 0;
00158     u32 range = 0;
00159
00160     if (((SYS_AFE_REG1 >> (6 + DAC_Channel)) & 0x01) == DAC_RANGE_1V2)
00161     { /* 加载DAC 1.2V量程校正值 */
00162         range = (uint16_t)((1.0 / 1.2) * BIT12);
00163     }
00164     else if (((SYS_AFE_REG1 >> (6 + DAC_Channel)) & 0x01) == DAC_RANGE_4V85)
00165     { /* 加载DAC 4.85V量程校正值 */
00166         range = (uint16_t)((1.0 / 4.85) * BIT12);
00167     }
00168
00169     temp = (DACVoltage * range + BIT11) >> 12;
00170     temp &= 0xFF; /* 最大值4095限制 */
00171
00172     if (DAC_Channel0 == DAC_Channel)
00173     {
00174         SYS_AFE_DAC0 = temp;
00175     }
00176     else if (DAC_Channel1 == DAC_Channel)
00177     {
00178         SYS_AFE_DAC1 = temp;
00179     }
00180 }
00201 void DAC_Cmd(uint8_t DAC_Channel, FuncState state)
00202 {
00203     if (DAC_Channel0 == DAC_Channel)
00204     {
00205         SYS_AnalogModuleClockCmd(SYS_AnalogModule_DAC0, state); /* DAC0 时钟使能 */
00206     }
00207     else if (DAC_Channel1 == DAC_Channel)
00208     {
00209         SYS_AnalogModuleClockCmd(SYS_AnalogModule_DAC1, state); /* DAC1 时钟使能 */
00210     }
00211 }
00212
00213 void DAC_Output(uint8_t DAC_Channel, DAC_InitTypeDef *DAC_InitStruct)
00214 {
00215     SYS_WR_PROTECT = 0x7a83; /* 解锁寄存器写保护 */
00216     if (DAC_Channel0 == DAC_Channel)
00217     {
00218         SYS_AFE_REG3 |= DAC_InitStruct->DACOUT_EN << 11; /* DAC输出配置 */
00219     }
00220     else if (DAC_Channel1 == DAC_Channel)
00221     {
00222         SYS_AFE_REG3 |= DAC_InitStruct->DACOUT_EN << 15; /* DAC输出配置 */
00223     }
00224     SYS_WR_PROTECT = 0xffff; /* 锁定寄存器写保护 */
00225 }
00226 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/

```

5.63 source/lks32mc07x_dma.c 文件参考

文件名称: LKS32MC07x_dma.c

文件标识: 无

内容摘要: DMA外设驱动程序

其它说明: 无

```

#include "lks32mc07x_dma.h"
#include "string.h"

```

lks32mc07x_dma.c 的引用(Include)关系图:



函数

- void **DMA_StructInit** (DMA_InitTypeDef *DMAInitStruct)
函数名称: void *DMA_StructInit(DMA_InitTypeDef *DMAInitStruct)*
- void **DMA_Init** (DMA_RegTypeDef *DMAx, DMA_InitTypeDef *this)
函数名称: void *DMA_Init(DMA_RegTypeDef *DMAx, DMA_InitTypeDef *DMAInitStruct)*
- u32 **DMA_GetIRQFlag** (u32 DMA_IF_x)
函数名称: uint32_t *DMA_GetIRQFlag(DMA_RegTypeDef *DMAx, u32 timer_if)*
- void **DMA_ClearIRQFlag** (u32 DMA_IF_x)
函数名称: void *DMA_ClearIRQFlag(DMA_RegTypeDef *DMAx, uint32_t tempFlag)*
- void **DMA_Enable** (void)
函数名称: void *DMA_Enable(void)*
- void **DMA_Disable** (void)
函数名称: void *DMA_Enable(void)*
- void **DMA_SetAddr** (DMA_RegTypeDef *DMA_CHx, u32 addr1, u32 addr2)
函数名称: void *DMA_SetAddr(DMA_RegTypeDef *DMA_CHx, u32 addr1, u32 addr2)*
- void **DMA_SetLength** (DMA_RegTypeDef *DMA_CHx, u32 len)
函数名称: void *DMA_SetLength(DMA_RegTypeDef *DMA_CHx, u32 len)*
- void **DMA_Trigger** (DMA_RegTypeDef *DMA_CHx)
函数名称: void *DMA_Trigger(DMA_RegTypeDef *DMA_CHx)*
- void **DMA_SetRequest** (DMA_RegTypeDef *DMA_CHx, u32 DMA_REQ_x)
函数名称: void *DMA_SetRequest(DMA_RegTypeDef *DMA_CHx, u32 DMA_REQ_x)*
- void **DMA_CHx_EN** (DMA_RegTypeDef *DMAx, u32 Channel_EN)
函数名称: void *DMA_CHx_EN(DMA_RegTypeDef *DMAx,u32 Channel.EN)*
- u32 **DMA_GetFlag** (u32 DMA_IF_x)

5.63.1 详细描述

文件名称: LKS32MC07x_dma.c
文件标识: 无
内容摘要: DMA外设驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2020年8月1日	1.0	Howlet	创建

修改日志:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:修改dma初始化相关函数

在文件 [lks32mc07x_dma.c](#) 中定义.

5.63.2 函数说明

5.63.2.1 DMA_CHx.EN()

```
void DMA_CHx_EN (
    DMA_RegTypeDef * DMAx,
    u32 Channel_EN )
```

函数名称: void [DMA_CHx_EN\(DMA_RegTypeDef *DMAx,u32 Channel_EN\)](#)

功能描述: DMA通道使能

参见

被引用内容:

参数

输入参数:	DMA.CHX :DMA 通道选择 DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3 Channel_EN : ENABLE/DISABLE
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 407 行定义.

5.63.2.2 DMA_ClearIRQFlag()

```
void DMA_ClearIRQFlag (
    u32 DMA_IF_x )
```

函数名称: void DMA_ClearIRQFlag(DMA_RegTypeDef *DMAx , uint32_t tempFlag)

功能描述: 清除DMA中断标志

参见

被引用内容: DMAx可选: DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3

参数

输入参数:	DMAx: DMA通道选择	
	timer_if参数可选:	
	宏定义	说明
	DMA_CH0_FIF	通道 0 完成中断标志
	DMA_CH1_FIF	通道 1 完成中断标志
	DMA_CH2_FIF	通道 2 完成中断标志
	DMA_CH3_FIF	通道 3 完成中断标志
	DMA_CH0{EIF	通道 0 错误中断标志
	DMA_CH1{EIF	通道 1 错误中断标志
	DMA_CH2{EIF	通道 2 错误中断标志
	DMA_CH3{EIF	通道 3 错误中断标志
输出参数:	无	

返回

返回值: 无

注解

其它说明： 无

警告

无

示例代码:

```
if(DMA_GetIRQFlag(DMA_CH0,DMA_CH0_FIF))//获取DMA通道0完成中断标志
{
    DMA_ClearIRQFlag(DMA_CH0,DMA_CH0_FIF)//清除DMA通道0完成中断标志
}
```

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 209 行定义.

5.63.2.3 DMA_Disable()

```
void DMA_Disable (
    void )
```

函数名称: void [DMA_Enable\(void\)](#)

功能描述: DMA模块 关闭函数

参见

被引用内容:

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明： 无

警告
无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 254 行定义.

5.63.2.4 DMA_Enable()

```
void DMA_Enable (
    void )
```

函数名称: void [DMA_Enable\(void\)](#)

功能描述: DMA模块 使能函数

参见

被引用内容: DMAx可选: DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

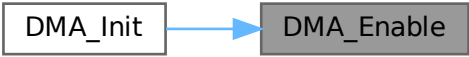
警告
无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

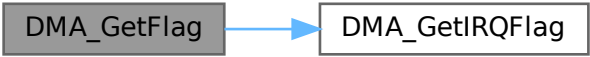
在文件 [lks32mc07x_dma.c](#) 第 232 行定义.
这是这个函数的调用关系图:



5.63.2.5 DMA_GetFlag()

```
u32 DMA_GetFlag (  
    u32 DMA_IF_x )
```

在文件 [lks32mc07x_dma.c](#) 第 418 行定义.
函数调用图:



5.63.2.6 DMA_GetIRQFlag()

```
u32 DMA_GetIRQFlag (  
    u32 DMA_IF_x )
```

函数名称: `uint32_t DMA_GetIRQFlag(DMA_RegTypeDef *DMAx , u32 timer_if)`
功能描述: 获取DMA中断标志

参见

被引用内容: DMAx可选: DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3

参数

输入参数:	DMAx: DMA通道选择	
	timer_if参数可选:	
	宏定义	说明
	DMA_CH0_FIF	通道 0 完成中断标志
	DMA_CH1_FIF	通道 1 完成中断标志
	DMA_CH2_FIF	通道 2 完成中断标志
	DMA_CH3_FIF	通道 3 完成中断标志
	DMA_CH0{EIF	通道 0 错误中断标志
	DMA_CH1{EIF	通道 1 错误中断标志
	DMA_CH2{EIF	通道 2 错误中断标志
	DMA_CH3{EIF	通道 3 错误中断标志

参见

参数

输出参数:	无
-------	---

返回

返回 值: 无

注解

其它说明: 无

警告

只有对应中断使能后, 改为才能读取, 如果对应中断未使能, 读取结果一直为0

示例代码:

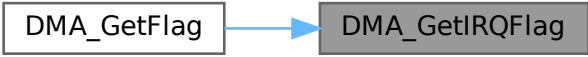
```
if (DMA_GetIRQFlag (DMA_CH0,DMA_CH0_FIF) ) //获取DMA通道0完成中断标志
{
}
```

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 166 行定义.

这是这个函数的调用关系图:



5.63.2.7 DMA_Init()

```
void DMA_Init (
    DMA_RegTypeDef * DMAx,
    DMA_InitTypeDef * this )
```

函数名称: void [DMA_Init\(DMA_RegTypeDef *DMAx, DMA_InitTypeDef *DMAInitStruct\)](#)

功能描述: DMA初始化函数

参见
被引用内容: 无

参数

输入参数:	ADCx: ADC0 , ADC_InitTypeDef
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
DMA_InitTypeDef DMA_InitStructure;
DMA_StructInit(&DMA_InitStructure);
DMA_InitStructure.EN = ENABLE; // DMA通道使能
DMA_InitStructure.IRQ_EN = ENABLE; // DMA 传输完成中断使能
DMA_InitStructure.CIRC = DISABLE; // DMA传输模式: 循环模式, 高有效
DMA_InitStructure.SINC = ENABLE; // 源地址递增, 高有效, 地址按照 SBTW
// 对应大小递增 1/2/4
DMA_InitStructure.DINC = DISABLE; // 目的地址递增, 高有效, 地址按照 DBTW
// 对应大小递增 1/2/4
DMA_InitStructure.SBTW = DMA_BYTE_TRANS; // 源访问位宽, 0:byte,
// 1:half-word, 2:word
DMA_InitStructure.DBTW = DMA_BYTE_TRANS; // 目的访问位宽, 0:byte,
// 1:half-word, 2:word
DMA_InitStructure.REN.B.SIF_EN = ENABLE; // SIF DMA请求使能, 高有效
DMA_InitStructure.TMS = 12; // DMA 通道 x 数据搬运次数 1~255
DMA_InitStructure.RMODE = 1; // 0:单轮传输, 一轮连续传输多次 /
// 1:多轮, 每轮进行一次数据传输
DMA_InitStructure.SADR = (u32)stru_yxtsenddata.byxt_send_data; // DMA 通道 x 源地址 &CAN0_TXBUF0
DMA_InitStructure.DADR = (u32)&SIF0_WDATA; // DMA 通道 x 目的地址 CAN0_TXRX0
CAN0_RFIFO00
DMA_Init(DMA_CHN0, &DMA_InitStructure);
```

Date	Version	Author	Description
2015年11月26日	1.0	Howlet Li	创建

修改日志:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:去除初始化使能DMA通道, 改为DMA.CHx_EN () 函数使能DMA通道

在文件 lks32mc07x_dma.c 第 97 行定义.

函数调用图:



5.63.2.8 DMA_SetAddr()

```
void DMA_SetAddr (
    DMA_RegTypeDef * DMA_CHx,
```

```
u32 addr1,  
u32 addr2 )
```

函数名称: void [DMA_SetAddr\(DMA_RegTypeDef *DMA_CHx, u32 addr1, u32 addr2\)](#)

功能描述: DMA传输地址设置

参见

被引用内容:

参数

输入参数:	DMA.CHX :DMA 通道选择 DMA.CH0 , DMA.CH1 , DMA.CH2 , DMA.CH3 addr1 : 源地址设置 addr2 : 目的地址设置
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 280 行定义.

5.63.2.9 DMA_SetLength()

```
void DMA_SetLength (  
    DMA_RegTypeDef * DMA_CHx,  
    u32 len )
```

函数名称: void [DMA_SetLength\(DMA_RegTypeDef *DMA_CHx, u32 len\)](#)

功能描述: DMA传输长度设置

参见

被引用内容:

参数

输入参数:	DMA.CHX :DMA 通道选择 DMA.CH0 , DMA.CH1 , DMA.CH2 , DMA.CH3 len : 传输长度设置
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 308 行定义.

5.63.2.10 DMA_SetRequest()

```
void DMA_SetRequest (
    DMA_RegTypeDef * DMA_CHx,
    u32 DMA_REQ_x )
```

函数名称: void [DMA_SetRequest\(DMA_RegTypeDef *DMA_CHx, u32 DMA_REQ_x\)](#)

功能描述: DMA 外设请求使能

参见

被引用内容:

参数

输入参数:	DMA.CHX :DMA 通道选择 DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3 DMA_REQ_x : 参数可选:		
	宏定义	原始值	说明
	DMA_REQ_UART1_TX	BIT19	
	DMA_REQ_UART1_RX	BIT18	
	DMA_REQ_UART0_TX	BIT17	
	DMA_REQ_UART0_RX	BIT16	
	DMA_REQ_I2C0_TX	BIT15	
	DMA_REQ_I2C0_RX	BIT14	
	DMA_REQ_MCPWM1	BIT13	
	DMA_REQ_MCPWM0	BIT12	
	DMA_REQ_SPI_TX	BIT11	
	DMA_REQ_SPI_RX	BIT10	
	DMA_REQ_TIMER4	BIT7	
	DMA_REQ_TIMER3	BIT6	
	DMA_REQ_TIMER2	BIT5	
	DMA_REQ_TIMER1	BIT4	
	DMA_REQ_TIMER0	BIT3	
	DMA_REQ_DSP	BIT2	
	DMA_REQ_ADC1	BIT1	
	DMA_REQ_ADC0	BIT0	
输出参数:	无		

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 382 行定义.

5.63.2.11 DMA_StructInit()

```
void DMA_StructInit (
    DMA_InitTypeDef * DMAInitStruct )
```

函数名称: void [DMA_StructInit\(DMA_InitTypeDef *DMAInitStruct\)](#)

功能描述: DMA结构体初始化

参见
被引用内容: 无

参数

输入参数:	DMA_InitTypeDef
输出参数:	无

返回
返 回 值: 无

注解
其它说明: 无

警告
无

示例代码:

```
DMA\_InitTypeDef DMA_InitStructure;
DMA_StructInit (&DMA_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 54 行定义.

5.63.2.12 DMA_Trigger()

```
void DMA_Trigger (
    DMA_RegTypeDef * DMA_CHx )
```

函数名称: void DMA_Trigger(DMA_RegTypeDef *DMA_CHx)

功能描述: DMA 软件触发设置

参见
被引用内容:

参数

输入参数:	DMA.CHX :DMA 通道选择 DMA_CH0 , DMA_CH1 , DMA_CH2 , DMA_CH3
输出参数:	无

返回
返回 值: 无

注解
其它说明: 无

警告
无

示例代码:

Date	Version	Author	Description
2023年2月11日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_dma.c](#) 第 336 行定义.

5.64 lks32mc07x_dma.c

[浏览该文件的文档.](#)

```

00001
00019 #include "lks32mc07x_dma.h"
00020 #include "string.h"
00021
00022 /*****
00023 函数名称: void DMA_StructInit(DMA_InitTypeDef *DMAInitStruct)
00024 功能描述: DMA结构体初始化
00025 操作的表: 无
00026 输入参数: 无
00027 输出参数: 无
00028 返回值: 无
00029 其它说明:
00030 修改日期 版本号 修改人 修改内容
00031 -----
00032 2020.08.1 V1.0 howlet 创建
00033 *****/
00054 void DMA_StructInit(DMA_InitTypeDef *DMAInitStruct)
00055 {
00056     memset(DMAInitStruct, 0, sizeof(DMA_InitTypeDef));
00057 }
00058
00097 void DMA_Init(DMA_RegTypeDef *DMAx, DMA_InitTypeDef *this)
00098 {
00099     // DMA0_CTRL = 0x0001; // DMA整体使能
00100     DMA_Enable();
00101     /* 通道配置寄存器 赋值 */
00102     DMAx->CCR = 0;
00103     do {
00104         DMAx->CTMS = this->TMS;
00105     } while (DMAx->CTMS != this->TMS); // 解决某些极端情况下CTMS次数和实际目标次数不一致的问题
00106
00107     DMAx->SADR = this->SADR;
00108     DMAx->DADR = this->DADR;
00109     DMAx->REN = this->REN.R;
00110     DMAx->CCR = (this->SBTW << 10) | (this->DBTW << 8) |
00111                 (this->SINC << 6) | (this->DINC << 4) |
00112                 (this->CIRC << 3) | (this->RMODE << 1) |
00113                 (this->EN);
00114     if (this->IRQ_EN)
00115         switch ((u32)DMAx)
00116         {
00117             case DMA0_BASE:
00118                 DMA0_IE |= BIT0;
00119                 break;
00120             case DMA0_BASE + 0x20:
00121                 DMA0_IE |= BIT1;
00122                 break;
00123             case DMA0_BASE + 0x40:
00124                 DMA0_IE |= BIT2;
00125                 break;
00126             case DMA0_BASE + 0x60:
00127                 DMA0_IE |= BIT3;
00128                 break;
00129         }
00130     DMA0_CTRL = 0x0001; /*enable dma, mcu has higher priorit */
00131 }
00132
00166 u32 DMA_GetIRQFlag(u32 DMA_IF_x)
00167 {
00168     if (DMA0_IF & DMA_IF_x & DMA0_IE)
00169     {
00170         return 1;
00171     }
00172     return 0;
00173 }
00174
00209 void DMA_ClearIRQFlag(u32 DMA_IF_x)
00210 {
00211     DMA0_IF = DMA_IF_x;
00212 }
00213
00232 void DMA_Enable(void)
00233 {
00234     DMA_TOP->CTRL = 1;
00235 }
00254 void DMA_Disable(void)
00255 {
00256     DMA_TOP->CTRL = 0;
00257 }
00258
00280 void DMA_SetAddr(DMA_RegTypeDef *DMA_CHx, u32 addr1, u32 addr2)
00281 {

```

```

00282     DMA_CHx->CCR = DMA_CHx->CCR & ~BIT0;
00283     DMA_CHx->SADR = addr1;
00284     DMA_CHx->DADR = addr2;
00285     DMA_CHx->CCR = DMA_CHx->CCR | BIT0;
00286 }
00287
00308 void DMA_SetLength(DMA_RegTypeDef *DMA_CHx, u32 len)
00309 {
00310     DMA_CHx->CCR = DMA_CHx->CCR & ~BIT0;
00311     do {
00312         DMA_CHx->CTMS = len;
00313     } while (DMA_CHx->CTMS != len); // 解决某些极端情况下CTMS次数和实际目标次数不一致的问题
00314     DMA_CHx->CCR = DMA_CHx->CCR | BIT0;
00315 }
00316
00336 void DMA_Trigger(DMA_RegTypeDef *DMA_CHx)
00337 {
00338     DMA_CHx->REN |= BIT31;
00339 }
00340
00382 void DMA_SetRequest(DMA_RegTypeDef *DMA_CHx, u32 DMA_REQ_x)
00383 {
00384     DMA_CHx->REN = DMA_REQ_x;
00385 }
00386
00407 void DMA_CHx_EN(DMA_RegTypeDef *DMAx, u32 Channel_EN)
00408 {
00409     if (Channel_EN)
00410     {
00411         DMAx->CCR |= BIT0; /*使能DMA搬运*/
00412     }
00413     else
00414     {
00415         DMAx->CCR &= ~BIT0; /*关闭通道使能*/
00416     }
00417 }
00418 u32 DMA_GetFlag(u32 DMA_IF_x)
00419 {
00420     return DMA_GetIRQFlag(DMA_IF_x);
00421 }

```

5.65 source/lks32mc07x_dsp.c 文件参考

文件名称: [lks32mc07x_dsp.c](#)

文件标识: 无

内容摘要: DSP驱动程序

其它说明: 无

#include "lks32mc07x_dsp.h"

lks32mc07x_dsp.c 的引用(Include)关系图:



函数

- void [DSP_Init](#) (void)
函数名称: void [DSP_Init\(void\)](#)
- s16 [DSP_sin](#) (s16 angle)
函数名称: s16 [DSP_sin\(u16 angle\)](#)
- s16 [DSP_cos](#) (s16 angle)
函数名称: s16 [DSP_cos\(u16 angle\)](#)

- s32 **DSP_rms** (s32 a, s32 b)
函数名称: *s16 u16 DSP_rms(s16 a,s16 b)*
- s16 **DSP_arctan** (s16 x, s16 y)
函数名称: *DSP_arctan(s16 x,s16 y)*
- s32 **DSP_div** (s32 a, s32 b)
函数名称: *s32 DSP_div(s32 a,s32 b)*
- u16 **DSP_mod** (s16 x, s16 y)
函数名称: *s16 DSP_mod(s32 x,s16 y)*
- u16 **DSP_sqrt** (u32 val)
函数名称: *u16 DSP_sqrt(u32 val)*
- void **DSP_SinCosMode** (void)
*sincos*模式
- void **DSP_ArctanMode** (void)
*Arctan*模式

5.65.1 详细描述

文件名称: **lks32mc07x_dsp.c**
文件标识: 无
内容摘要: DSP驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2015年11月5日	1.0	Howlet	创建

修改日志:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:DSP调用函数内增加中断打断重新复位DSP功能

在文件 **lks32mc07x_dsp.c** 中定义.

5.65.2 函数说明

5.65.2.1 DSP_arctan()

s16 DSP_arctan (

```
s16 x,  
s16 y )
```

函数名称: **DSP_arctan(s16 x,s16 y)**

功能描述: **DSP 反正切计算函数**

参见

被引用内容:

参数

输入参数:	x: 分母 y:分子
输出参数:	无

返回

返回 值: 反正切值

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 155 行定义.

5.65.2.2 DSP_ArctanMode()

```
void DSP_ArctanMode (  
    void )
```

Arctan模式

在文件 [lks32mc07x_dsp.c](#) 第 268 行定义.

5.65.2.3 DSP_cos()

```
s16 DSP_cos (
    s16 angle )
```

函数名称: s16 DSP_cos(u16 angle)

功能描述: DSP cos计算函数

参见

被引用内容:

参数

输入参数:	angle: 输入角度
输出参数:	无

返回

返回值: cos角度值

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 93 行定义.

5.65.2.4 DSP_div()

```
s32 DSP_div (
    s32 a,
    s32 b )
```

函数名称: **s32 DSP_div(s32 a,s32 b)**

功能描述: DSP 除法计算

参见

被引用内容:

参数

输入参数:	x: 被除数 y:除数
输出参数:	无

返回

返回 值: 两数的商

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 **lks32mc07x.dsp.c** 第 187 行定义.

5.65.2.5 DSP_Init()

```
void DSP_Init (
    void )
```

函数名称: **void DSP_Init(void)**

功能描述: DSP使能和关闭

参见

被引用内容: **SYS_ModuleClockCmd()**

参数

输入参数:	无
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

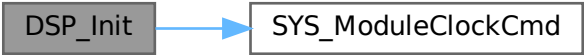
无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 40 行定义.
函数调用图:



5.65.2.6 DSP_mod()

```
ul6 DSP_mod (
    s16 x,
    s16 y )
```

函数名称: s16 DSP_mod(s32 x,s16 y)
功能描述: DSP 计算向量的模

参见

被引用内容:

参数

输入参数:	x: x轴 y: y轴
输出参数:	无

返回

返回 值: 向量的模

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 218 行定义.

5.65.2.7 DSP_rms()

```
s32 DSP_rms (
    s32 a,
    s32 b )
```

函数名称: s16 u16 DSP_rms(s16 a,s16 b)

功能描述: DSP 除法求余数计算函数

参见

被引用内容:

参数

输入参数:	a: 被除数 b:除数
输出参数:	无

返回

返回 值: 两数的余数

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 124 行定义.

5.65.2.8 DSP_sin()

```
s16 DSP_sin (
    s16 angle )
```

函数名称: s16 DSP_sin(u16 angle)

功能描述: DSP sin计算函数

参见

被引用内容:

参数

输入参数:	angle: 输入角度
输出参数:	无

返回

返回 值: `sin`角度值

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 `lks32mc07x_dsp.c` 第 63 行定义.

5.65.2.9 `DSP_SinCosMode()`

```
void DSP_SinCosMode (
    void )
```

`sincos`模式

在文件 `lks32mc07x_dsp.c` 第 261 行定义.

5.65.2.10 `DSP_sqrt()`

```
u16 DSP_sqrt (
    u32 val )
```

函数名称: `u16 DSP_sqrt(u32 val)`

功能描述: 开平方函数

参见

被引用内容:

参数

输入参数:	val：被开方数
输出参数:	无

返回

返回 值： 向量的莫

注解

其它说明： 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	2.0	ywx	修改

修改日志:

在文件 [lks32mc07x_dsp.c](#) 第 248 行定义.

5.66 lks32mc07x_dsp.c

[浏览该文件的文档.](#)

```
00001
00020 #include "lks32mc07x_dsp.h"
00021
00040 void DSP_Init(void)
00041 {
00042     SYS_ModuleClockCmd(SYS_Module_DSP0, ENABLE); /* DSP时钟使能*/
00043 }
00044
00063 s16 DSP_sin(s16 angle)
00064 {
00065     s16 answer;
00066     int32_t flg = __get_PRIMASK();
00067     __disable_irq();
00068     DSP_SIN_COS_MODE();
00069     DSP_REG->THETA = angle;
00070     answer = DSP_REG->SIN;
00071     __set_PRIMASK(flg);
00072     return answer;
00073 }
00074
```

```

00093 s16 DSP_cos(s16 angle)
00094 {
00095     s16 answer;
00096     int32_t flg = __get_PRIMASK();
00097     __disable_irq();
00098     DSP_SIN_COS_MODE();
00099     DSP_REG->THETA = angle;
00100     answer = DSP_REG->COS;
00101     __set_PRIMASK(flg);
00102     return answer;
00103 }
00104
00124 s32 DSP_rms(s32 a, s32 b)
00125 {
00126     s32 answer;
00127     int32_t flg = __get_PRIMASK();
00128     __disable_irq();
00129     DSP_REG->DID = a;
00130     DSP_REG->DIS = b;
00131     answer = DSP_REG->REM;
00132     __set_PRIMASK(flg);
00133     return answer;
00134 }
00135
00155 s16 DSP_arctan(s16 x, s16 y)
00156 {
00157     s16 answer;
00158     int32_t flg = __get_PRIMASK();
00159     __disable_irq();
00160     DSP_ARCTAN_MODE();
00161     DSP_REG->X = x;
00162     DSP_REG->Y = y;
00163     answer = DSP_REG->ARCTAN;
00164     __set_PRIMASK(flg);
00165     return answer;
00166 }
00167
00187 s32 DSP_div(s32 a, s32 b)
00188 {
00189     s32 answer;
00190     int32_t flg = __get_PRIMASK();
00191     __disable_irq();
00192     DSP_REG->DID = a;
00193     DSP_REG->DIS = b;
00194     answer = DSP_REG->QUO;
00195     __set_PRIMASK(flg);
00196     return answer;
00197 }
00198
00218 u16 DSP_mod(s16 x, s16 y)
00219 {
00220     s16 answer;
00221     int32_t flg = __get_PRIMASK();
00222     __disable_irq();
00223     DSP_REG->X = x;
00224     DSP_REG->Y = y;
00225     answer = DSP_REG->MOD;
00226     __set_PRIMASK(flg);
00227     return answer;
00228 }
00229
00248 u16 DSP_sqrt(u32 val)
00249 {
00250     u16 answer;
00251     int32_t flg = __get_PRIMASK();
00252     __disable_irq();
00253     DSP_REG->RAD = val;
00254     answer = DSP_REG->SQRT;
00255     __set_PRIMASK(flg);
00256     return answer;
00257 }
00261 void DSP_SinCosMode(void)
00262 {
00263     DSP_SIN_COS_MODE();
00264 }
00268 void DSP_ArctanMode(void)
00269 {
00270     DSP_ARCTAN_MODE();
00271 }

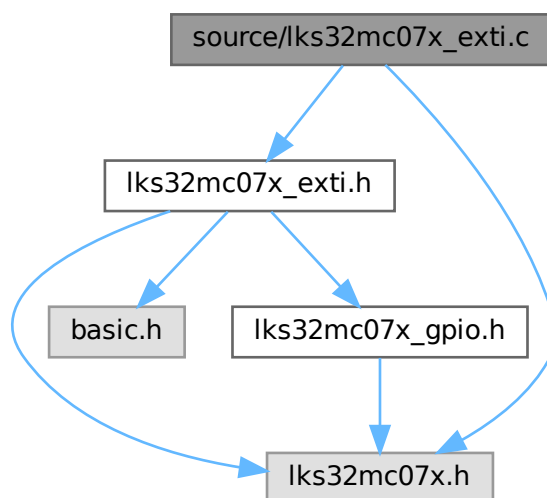
```

5.67 source/lks32mc07x_exti.c 文件参考

文件名称: [lks32mc07x_exti.c](#)
 文件标识: 无
 内容摘要: EXTI外设驱动程序
 其它说明: 无

```
#include "lks32mc07x_exti.h"
#include "lks32mc07x.h"
```

lks32mc07x_exti.c 的引用(Include)关系图:



函数

- void [EXTI_Trigger_Config](#) (GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint16_t EXTI_Trigger)
 函数名称: void [EXTI_Trigger_Config\(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint16_t EXTI_Trigger\)](#)
- u8 [EXTI_GetIRQFlag](#) (GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource)
 函数名称: u8 [EXTI_GetIRQFlag\(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource\)](#)
- void [EXTI_ClearIRQFlag](#) (GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource)
 函数名称: void [EXTI_ClearIRQFlag\(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource\)](#)
- uint32_t [EXTI_ReadIFFlag](#) (EXTI_TypeDef *EXTIx)
- void [EXTI_ResetIF](#) (EXTI_TypeDef *EXTIx, uint32_t temp)

5.67.1 详细描述

文件名称: [lks32mc07x_exti.c](#)
 文件标识: 无
 内容摘要: EXTI外设驱动程序
 其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月28日	1.0	cakezhang	创建

在文件 [lks32mc07x_exti.c](#) 中定义.

5.67.2 函数说明

5.67.2.1 EXTI_ClearIRQFlag()

```
void EXTI_ClearIRQFlag (
    GPIO_TypeDef * GPIOx,
    uint32_t GPIO_PinSource )
```

函数名称: void [EXTI_ClearIRQFlag\(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource\)](#)

功能描述: 清除外部中断标志位

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象可选(GPIO0~GPIO3) GPIO_PinSource:指定的PinSource可选(GPIO_PinSource_0~GPIO_PinSource_15)
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if(EXTI_GetIRQFlag(GPIO0,GPIO_PinSource_0))//获取P0.0外部中断标志位
{
    EXTI_ClearIRQFlag(GPIO0,GPIO_PinSource_0); //清除P0.0外部中断标志位
}
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_exti.c](#) 第 390 行定义.

5.67.2.2 EXTI_GetIRQFlag()

```
u8 EXTI_GetIRQFlag (
    GPIO_TypeDef * GPIOx,
    uint32_t GPIO_PinSource )
```

函数名称: u8 EXTI_GetIRQFlag(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource)

功能描述: 读取外部中断标志位

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象可选(GPIO0~GPIO3) GPIO_PinSource:指定的PinSource可选(GPIO_PinSource_0~GPIO_PinSource_15)
输出参数:	无

返回

返回值: 中断标志, 高有效

注解

其它说明: 无

警告

无

示例代码:

```
if(EXTI_GetIRQFlag(GPIO0,GPIO_PinSource_0))//获取P0.0外部中断标志位
{
}
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_exti.c](#) 第 159 行定义.

5.67.2.3 EXTI_ReadIFFlag()

```
uint32_t EXTI_ReadIFFlag (
    EXTI_TypeDef * EXTIx )
```

在文件 [lks32mc07x_exti.c](#) 第 468 行定义.

5.67.2.4 EXTI_ResetIF()

```
void EXTI_ResetIF (
    EXTI_TypeDef * EXTIx,
    uint32_t temp )
```

在文件 [lks32mc07x_exti.c](#) 第 472 行定义.

5.67.2.5 EXTI_Trigger_Config()

```
void EXTI_Trigger_Config (
    GPIO_TypeDef * GPIOx,
    uint32_t GPIO_PinSource,
    uint16_t EXTI_Trigger )
```

函数名称: void [EXTI_Trigger_Config\(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint16_t EXTI_Trigger\)](#)

功能描述: GPIO外部触发极性选择

参见

被引用内容: [GPIO_TypeDef](#)

参数

输入参数:	GPIOx:GPIO对象可选(GPIO0~GPIO3) GPIO_PadSource:指定的PadSource可选(GPIO_PinSource.0~GPIO_PinSource.15) EXTI_Trigger:指定的触发极性
-------	--

参见

EXTI_Trigger参数来源: EXTIMode_TypeDef

参数

输出参数:	无
-------	---

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
EXTI_Trigger_Config(GPIO0,GPIO_PinSource_0,EXTI_Trigger_Negedge); //P0.0信号下降沿触发外部中断事件
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_exti.c](#) 第 39 行定义.

5.68 lks32mc07x_exti.c

[浏览该文件的文档.](#)

```
00001
00014 #include "lks32mc07x_exti.h"
00015 #include "lks32mc07x.h"
00016
```

```

00039 void EXTI_Trigger_Config(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint16_t EXTI_Trigger)
00040 {
00041     if (GPIOx == GPIO0)
00042     {
00043         EXTI->IE = EXTI->IE | 0x01;
00044
00045         if (GPIO_PinSource == 0)
00046         {
00047             EXTI->CR0 = (EXTI->CR0 & 0xffffc) | EXTI_Trigger;
00048         }
00049         else if (GPIO_PinSource == 1)
00050         {
00051             EXTI->CR0 = (EXTI->CR0 & 0xffff3) | (EXTI_Trigger << 2);
00052         }
00053         else if (GPIO_PinSource == 2)
00054         {
00055             EXTI->CR1 = (EXTI->CR0 & 0xffcf) | (EXTI_Trigger << 4);
00056         }
00057         else if (GPIO_PinSource == 3)
00058         {
00059             EXTI->CR0 = (EXTI->CR0 & 0xff3f) | (EXTI_Trigger << 6);
00060         }
00061         else if (GPIO_PinSource == 6)
00062         {
00063             EXTI->CR0 = (EXTI->CR0 & 0xfcff) | (EXTI_Trigger << 8);
00064         }
00065         else if (GPIO_PinSource == 11)
00066         {
00067             EXTI->CR0 = (EXTI->CR0 & 0xf3ff) | (EXTI_Trigger << 10);
00068         }
00069         else if (GPIO_PinSource == 14)
00070         {
00071             EXTI->CR0 = (EXTI->CR0 & 0xcfff) | (EXTI_Trigger << 12);
00072         }
00073         else if (GPIO_PinSource == 15)
00074         {
00075             EXTI->CR0 = (EXTI->CR0 & 0x3fff) | (EXTI_Trigger << 14);
00076         }
00077         else
00078         {
00079             return;
00080         }
00081     }
00082
00083     if (GPIOx == GPIO1)
00084     {
00085         EXTI->IE = EXTI->IE | 0x02;
00086
00087         if (GPIO_PinSource == 0)
00088         {
00089             EXTI->CR1 = (EXTI->CR1 & 0xffffc) | EXTI_Trigger;
00090         }
00091         else if (GPIO_PinSource == 1)
00092         {
00093             EXTI->CR1 = (EXTI->CR1 & 0xffff3) | (EXTI_Trigger << 2);
00094         }
00095         else if (GPIO_PinSource == 10)
00096         {
00097             EXTI->CR1 = (EXTI->CR1 & 0xffcf) | (EXTI_Trigger << 4);
00098         }
00099         else
00100         {
00101             return;
00102         }
00103     }
00104
00105     if (GPIOx == GPIO2)
00106     {
00107         EXTI->IE = EXTI->IE | 0x04;
00108
00109         if (GPIO_PinSource == 3)
00110         {
00111             EXTI->CR1 = (EXTI->CR1 & 0xff3f) | (EXTI_Trigger << 6);
00112         }
00113         else if (GPIO_PinSource == 4)
00114         {
00115             EXTI->CR1 = (EXTI->CR1 & 0xfcff) | (EXTI_Trigger << 8);
00116         }
00117         else if (GPIO_PinSource == 7)
00118         {
00119             EXTI->CR1 = (EXTI->CR1 & 0xf3ff) | (EXTI_Trigger << 10);
00120         }
00121         else if (GPIO_PinSource == 12)
00122         {
00123             EXTI->CR1 = (EXTI->CR1 & 0xcfff) | (EXTI_Trigger << 12);
00124         }
00125         else if (GPIO_PinSource == 15)

```

```
00126     {
00127         EXTI->CR1 = (EXTI->CR1 & 0x3fff) | (EXTI.Trigger << 14);
00128     }
00129     else
00130     {
00131         return;
00132     }
00133 }
00134 }
00135
00159 u8 EXTI_GetIRQFlag(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource)
00160 {
00161     if (GPIOx == GPIO0)
00162     {
00163         if (GPIO_PinSource == 0)
00164         {
00165             if (EXTI_IF & 0x01)
00166             {
00167                 return 1;
00168             }
00169             else
00170             {
00171                 return 0;
00172             }
00173         }
00174         else if (GPIO_PinSource == 1)
00175         {
00176             if (EXTI_IF & 0x02)
00177             {
00178                 return 1;
00179             }
00180             else
00181             {
00182                 return 0;
00183             }
00184         }
00185         else if (GPIO_PinSource == 2)
00186         {
00187             if (EXTI_IF & 0x04)
00188             {
00189                 return 1;
00190             }
00191             else
00192             {
00193                 return 0;
00194             }
00195         }
00196         else if (GPIO_PinSource == 3)
00197         {
00198             if (EXTI_IF & 0x08)
00199             {
00200                 return 1;
00201             }
00202             else
00203             {
00204                 return 0;
00205             }
00206         }
00207         else if (GPIO_PinSource == 6)
00208         {
00209             if (EXTI_IF & 0x10)
00210             {
00211                 return 1;
00212             }
00213             else
00214             {
00215                 return 0;
00216             }
00217         }
00218         else if (GPIO_PinSource == 11)
00219         {
00220             if (EXTI_IF & 0x20)
00221             {
00222                 return 1;
00223             }
00224             else
00225             {
00226                 return 0;
00227             }
00228         }
00229         else if (GPIO_PinSource == 14)
00230         {
00231             if (EXTI_IF & 0x40)
00232             {
00233                 return 1;
00234             }
00235             else
```

```
00236         {
00237             return 0;
00238         }
00239     }
00240     else if (GPIO_PinSource == 15)
00241     {
00242         if (EXTI_IF & 0x80)
00243         {
00244             return 1;
00245         }
00246         else
00247         {
00248             return 0;
00249         }
00250     }
00251     else
00252     {
00253         return 0;
00254     }
00255 }
00256
00257 else if (GPIOx == GPIO1)
00258 {
00259     if (GPIO_PinSource == 0)
00260     {
00261         if (EXTI_IF & 0x100)
00262         {
00263             return 1;
00264         }
00265         else
00266         {
00267             return 0;
00268         }
00269     }
00270     else if (GPIO_PinSource == 1)
00271     {
00272         if (EXTI_IF & 0x200)
00273         {
00274             return 1;
00275         }
00276         else
00277         {
00278             return 0;
00279         }
00280     }
00281     else if (GPIO_PinSource == 10)
00282     {
00283         if (EXTI_IF & 0x400)
00284         {
00285             return 1;
00286         }
00287         else
00288         {
00289             return 0;
00290         }
00291     }
00292     else
00293     {
00294         return 0;
00295     }
00296 }
00297
00298 else if (GPIOx == GPIO2)
00299 {
00300     if (GPIO_PinSource == 3)
00301     {
00302         if (EXTI_IF & 0x800)
00303         {
00304             return 1;
00305         }
00306         else
00307         {
00308             return 0;
00309         }
00310     }
00311     else if (GPIO_PinSource == 4)
00312     {
00313         if (EXTI_IF & 0x1000)
00314         {
00315             return 1;
00316         }
00317         else
00318         {
00319             return 0;
00320         }
00321     }
00322     else if (GPIO_PinSource == 7)
```

```

00323     {
00324         if (EXTI_IF & 0x2000)
00325         {
00326             return 1;
00327         }
00328         else
00329         {
00330             return 0;
00331         }
00332     }
00333     else if (GPIO_PinSource == 12)
00334     {
00335         if (EXTI_IF & 0x4000)
00336         {
00337             return 1;
00338         }
00339         else
00340         {
00341             return 0;
00342         }
00343     }
00344     else if (GPIO_PinSource == 15)
00345     {
00346         if (EXTI_IF & 0x8000)
00347         {
00348             return 1;
00349         }
00350         else
00351         {
00352             return 0;
00353         }
00354     }
00355     else
00356     {
00357         return 0;
00358     }
00359 }
00360 else
00361 {
00362     return 0;
00363 }
00364 }
00365
00390 void EXTI_ClearIRQFlag(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource)
00391 {
00392     if (GPIOx == GPIO0)
00393     {
00394         if (GPIO_PinSource == 0)
00395         {
00396             EXTI_IF = 0x01;
00397         }
00398         else if (GPIO_PinSource == 1)
00399         {
00400             EXTI_IF = 0x02;
00401         }
00402         else if (GPIO_PinSource == 2)
00403         {
00404             EXTI_IF = 0x04;
00405         }
00406         else if (GPIO_PinSource == 3)
00407         {
00408             EXTI_IF = 0x08;
00409         }
00410         else if (GPIO_PinSource == 6)
00411         {
00412             EXTI_IF = 0x10;
00413         }
00414         else if (GPIO_PinSource == 11)
00415         {
00416             EXTI_IF = 0x20;
00417         }
00418         else if (GPIO_PinSource == 14)
00419         {
00420             EXTI_IF = 0x40;
00421         }
00422         else if (GPIO_PinSource == 15)
00423         {
00424             EXTI_IF = 0x80;
00425         }
00426     }
00427     if (GPIOx == GPIO1)
00428     {
00429         if (GPIO_PinSource == 0)
00430         {
00431             EXTI_IF = 0x100;
00432         }
00433     }

```

```

00434         else if (GPIO.PinSource == 1)
00435         {
00436             EXTI_IF = 0x200;
00437         }
00438         else if (GPIO.PinSource == 10)
00439         {
00440             EXTI_IF = 0x400;
00441         }
00442     }
00443
00444     if (GPIOx == GPIO2)
00445     {
00446         if (GPIO.PinSource == 3)
00447         {
00448             EXTI_IF = 0x800;
00449         }
00450         else if (GPIO.PinSource == 4)
00451         {
00452             EXTI_IF = 0x1000;
00453         }
00454         else if (GPIO.PinSource == 7)
00455         {
00456             EXTI_IF = 0x2000;
00457         }
00458         else if (GPIO.PinSource == 12)
00459         {
00460             EXTI_IF = 0x4000;
00461         }
00462         else if (GPIO.PinSource == 15)
00463         {
00464             EXTI_IF = 0x8000;
00465         }
00466     }
00467 }
00468 uint32_t EXTI_ReadIFFlag(EXTI_TypeDef *EXTIx)
00469 {
00470     return EXTIx->IF;
00471 }
00472 void EXTI_ResetIF(EXTI_TypeDef *EXTIx, uint32_t temp)
00473 {
00474     EXTIx->IF = temp;
00475 }
00476
00477 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.69 source/lks32mc07x_flash.c 文件参考

文件名称: [lks32mc07x_flash.c](#)
 文件标识: 无
 内容摘要: FLASH外设驱动程序
 其它说明: 无

#include "lks32mc07x_flash.h"
 #include "lks32mc07x_nvr.h"
 lks32mc07x_flash.c 的引用(Include)关系图:



函数

- void [EraseSector](#) (u32 adr, u16 nvr)
函数名称: void [EraseSector\(u32 adr, u16 nvr\)](#)
- int [ProgramPage](#) (u32 adr, u32 sz, u8 *buf, u16 nvr)

- 函数名称: `int ProgramPage(u32 adr, u32 sz, u8 *buf, u16 nvr)`
- `u32 Read.Flash (uint32_t adr, u16 nvr)`
函数名称: `u32 Read.Flash(uint32_t adr, u16 nvr)`
 - `void Read.More.Flash (u32 adr, u32 sz, u32 *buf, u16 nvr)`
函数名称: `void Read.More.Flash(u32 adr, u32 sz, u32 *buf, u16 nvr)`

变量

- `volatile u32 erase.flag = 0`
`flash`擦除操作启动变量
- `volatile u32 prog.m.flag = 0`
`flash`编程操作启动变量

5.69.1 详细描述

文件名称: `lks32mc07x.flash.c`
 文件标识: 无
 内容摘要: FLASH外设驱动程序
 其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 `lks32mc07x.flash.c` 中定义.

5.69.2 函数说明

5.69.2.1 EraseSector()

```
void EraseSector (
    u32 adr,
    u16 nvr )
```

函数名称: `void EraseSector(u32 adr, u16 nvr)`

功能描述: Flash按扇区擦除操作

参见

被引用内容: 无

参数

输入参数:	adr: 扇区地址 (一个扇区512字节) nvr: 为0x800的时候擦除NVR, 为0时擦除主Flash erase_flag: 调用函数前必须赋值0x9A0D361F, 否则不执行擦除, 用来防程序跑飞
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```

事例1:
erase_flag = 0x9A0D361F; //擦除解锁
EraseSector(0x00000000, Flash.MAIN); //擦除flash的main区域即存储程序区域
erase_flag = 0x00; //擦除上锁

事例2:
erase_flag = 0x9A0D361F; //擦除解锁
EraseSector(0x00000000, Flash.NVR); //擦除flash的NVR区域即独立的1KB用户存储区
erase_flag = 0x00; //擦除上锁

```

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_flash.c](#) 第 55 行定义.

5.69.2.2 ProgramPage()

```

int ProgramPage (
    u32 adr,
    u32 sz,
    u8 * buf,
    u16 nvr )

```

函数名称: int [ProgramPage\(u32 adr, u32 sz, u8 *buf, u16 nvr\)](#)

功能描述: Flash编程操作

参见

被引用内容： 无

参数

输入参数:	adr: 扇区地址 sz : 编程字节数量(0~512字节) buf: 要编程的数据数组指针 nvr: 为0x800的时候编程NVR, 为0时编程主Flash progm_flag: 调用函数前必须赋值0x9AFDA40C, 否则不执行编程, 用来防程序跑飞
输出参数:	无

返回

返回 值: 1: 编程成功; 0: 编程失败

注解

其它说明: 无

警告

无

示例代码:

```
u8 Flash_Buff[2] = {0x12,0x23};
    事例1:
    erase_flag = 0x9A0D361F; //擦除解锁
    EraseSector(0x00000000 , Flash_MAIN); //擦除flash的main区域即存储程序区域
    erase_flag = 0x00; //擦除上锁

    prog_flag = 0x9AFDA40C; //编程解锁
    ProgramPage(0x00000000 ,2,Flash_Buff, Flash_MAIN); //编程flash的main区域即存储程序区域
    erase_flag = 0x00; //编程上锁
    事例2:
    erase_flag = 0x9A0D361F; //擦除解锁
    EraseSector(0x00000000 , Flash_NVR); //擦除flash的NVR区域即独立的1KB用户存储区
    erase_flag = 0x00; //擦除上锁

    erase_flag = 0x9A0D361F; //编程解锁
    EraseSector(0x00000000,2,Flash_Buff,Flash_NVR); //编程flash的NVR区域即独立的1KB用户存储区
    erase_flag = 0x00; //编程上锁
```

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_flash.c](#) 第 137 行定义.

5.69.2.3 Read_Flash()

```
u32 Read_Flash (
    uint32_t adr,
    u16 nvr )
```

函数名称: u32 [Read_Flash](#)(uint32_t adr , u16 nvr)

功能描述: 读取FLASH数据函数

参见
被引用内容: 无

参数

输入参数:	adr: 扇区地址 (一个扇区512字节) nvr: 为0x800的时候擦除NVR, 为0时擦除主Flash
输出参数:	无

返回
返 回 值: 读取的数据值

注解
其它说明: 无

警告
无

示例代码:

事例1:

u32 Flash_main = 0;
Flash_main = [Read_Flash](#)(0x00000000 , [Flash_MAIN](#)); //获取flash的main区域0x00000000地址一个word数据

事例2:

u32 Flash_nvr = 0;
Flash_nvr = [Read_Flash](#)(0x00000000 , [Flash_NVR](#)); //获取flash的NVR区域0x00000000地址一个word数据

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_flash.c](#) 第 232 行定义.

函数调用图:



这是这个函数的调用关系图:



5.69.2.4 Read_More_Flash()

```
void ReadMore.Flash (
    u32 adr,
    u32 sz,
    u32 * buf,
    u16 nvr )
```

函数名称: void [Read_More_Flash](#)(u32 adr, u32 sz, u32 *buf, u16 nvr)

功能描述: 读取FLASH多地址数据函数

参见
被引用内容: 无

参数

输入参数:	adr: 扇区地址 (一个扇区512字节) nvr: 为0x800, 读取MAIN数据, 为0x0, 读取NVR数据 buf:读取数据存储的地址 sz :读取sz个字
输出参数:	无

返回
返 回 值: 读取的数据值

注解

其它说明： 无

警告

无

示例代码:

```
u32 Flash_Read.Buff[4] = {0};
事例1:
Read_More_Flash (0x00000000 ,4,Flash_Read.Buff,Flash.MAIN); //获取flash的main区域0x00000000地址4个word数据
事例2:
Read_More_Flash (0x00000000 ,4,Flash_Read.Buff,Flash.NVR); //获取flash的NVR区域0x00000000地址4个word数据
```

Date	Version	Author	Description
2023年2月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_flash.c](#) 第 276 行定义.

函数调用图:



5.69.3 变量说明

5.69.3.1 erase_flag

```
volatile u32 erase_flag = 0
```

flash擦除操作启动变量

在文件 [lks32mc07x_flash.c](#) 第 20 行定义.

5.69.3.2 prog_m_flag

```
volatile u32 prog_m_flag = 0
```

flash编程操作启动变量

在文件 [lks32mc07x_flash.c](#) 第 24 行定义.

5.70 lks32mc07x_flash.c

浏览该文件的文档。

```

00001
00015 #include "lks32mc07x_flash.h"
00016 #include "lks32mc07x_nvr.h"
00020 volatile u32 erase_flag = 0;
00024 volatile u32 prog_m_flag = 0;
00025
00055 void EraseSector(u32 adr, u16 nvr)
00056 {
00057     u32 key1, key2, key3;
00058     key1 = 0x0;
00059     key2 = 0x0;
00060     key3 = 0x0;
00061     SYS_PROTECT = 0x7A83;
00062     SYS_FLSE = 0x8FCA;
00063     if (erase_flag == 0x9A0D361F)
00064     { // get flash_cfg addr
00065         key1 = 0xB00C060A;
00066         key2 = 0x2A033015;
00067         key3 = erase_flag ^ key1 ^ key2;
00068         REG32(key3) &= ~0x80008800; // clear erase_en, set to sector, clear nvr bit
00069         FLASH_ADDR = adr;
00070         REG32(key3) |= (0x80000000 | nvr);
00071         key1 = 0x0;
00072         key2 = 0x0;
00073         key3 = 0x0;
00074     }
00075
00076     if (erase_flag == 0x9A0D361F)
00077     { // get flash_erase addr
00078         key1 = 0xB003A53A;
00079         key2 = 0x2A0C9335;
00080         key3 = erase_flag ^ key1 ^ key2;
00081         REG32(key3) = 0x7654DCBA; // trig sector erase, FLASH_ERASE
00082         FLASH_CFG &= ~0x80008800; // clear erase_en, set to sector, clear nvr bit
00083         erase_flag = 0x00000000;
00084         key1 = 0x0;
00085         key2 = 0x0;
00086         key3 = 0x0;
00087     }
00088     FLASH_CFG &= ~0x80008800; // clear erase_en, set to sector, clear nvr bit
00089     erase_flag = 0x00000000;
00090     SYS_FLSE = 0;
00091     SYS_PROTECT = 0;
00092     key1 = 0x0;
00093     key2 = 0x0;
00094     key3 = 0x0;
00095 }
00096
00137 int ProgramPage(u32 adr, u32 sz, u8 *buf, u16 nvr)
00138 {
00139     volatile u32 Addr, size;
00140     volatile u8 *p;
00141     volatile u8 trlt = 1;
00142
00143     u32 key1, key2, key3;
00144     /* Add your Code */
00145     key1 = 0x0;
00146     key2 = 0x0;
00147     key3 = 0x0;
00148     SYS_PROTECT = 0x7A83;
00149     SYS_FLSP = 0x8F35;
00150     if (prog_m_flag == 0x9AFDA40C)
00151     {
00152         // get flash_cfg addr
00153         key1 = 0x2A5CA00F;
00154         key2 = 0xB0A30403;
00155
00156         key3 = prog_m_flag ^ key1 ^ key2;
00157         REG32(key3) &= ~0x80008000;
00158         FLASH_ADDR = adr;
00159         REG32(key3) |= (0x08000000 | nvr);
00160         prog_m_flag = 0;
00161         key1 = 0x0;
00162         key2 = 0x0;
00163         key3 = 0x0;
00164
00165         p = buf;
00166         size = (sz + 3) & ~3; // align the word
00167         Addr = adr;
00168         while (size)
00169         {
00170             FLASH_ADDR = (Addr & ~0x00000003); // address
alignment

```



```

00171         FLASH_WDATA = *p + (*p + 1) << 8) + (*p + 2) << 16) + (*p + 3) << 24); // write data to
flash
00172
00173         p += 4;
00174         size -= 4; // Go to next word
00175         Addr += 4;
00176     }
00177
00178     p = buf;
00179     size = (sz + 3) & ~3; // align the word
00180     Addr = adr;
00181     while (size)
00182     {
00183         u32 t_RData;
00184         FLASH_ADDR = (Addr & ~0x00000003); // address alignment
00185         t_RData = *p + (*p + 1) << 8) + (*p + 2) << 16) + (*p + 3) << 24);
00186         if (t_RData != FLASH_RDATA)
00187         {
00188             t_rlt = 0;
00189         }
00190
00191         p += 4;
00192         size -= 4;
00193         Addr += 4;
00194     }
00195 }
00196 FLASH_CFG &= ~0x08000800;
00197 key1 = 0x0;
00198 key2 = 0x0;
00199 key3 = 0x0;
00200 prog_mflag = 0;
00201 SYS_FLSP = 0;
00202 SYS_PROTECT = 0;
00203 return (t_rlt);
00204 }
00205
00232 u32 Read_Flash(uint32_t adr, u16 nvr)
00233 {
00234     uint32_t data;
00235     if (nvr == Flash_MAIN) // 读取MIAN区域
00236     {
00237         FLASH_CFG &= ~BIT11;
00238         FLASH_ADDR = adr;
00239         data = FLASH_RDATA;
00240         FLASH_CFG &= ~BIT11; // 恢复MIAN操作
00241     }
00242     else
00243     {
00244         data = Read_NVR(adr);
00245     }
00246     return data;
00247 }
00248
00276 void Read_More_Flash(u32 adr, u32 sz, u32 *buf, u16 nvr)
00277 {
00278     volatile u32 Addr = 0;
00279     volatile u32 size = 0;
00280     volatile u32 *p;
00281     size = sz;
00282     Addr = adr;
00283     p = buf;
00284     while (size)
00285     {
00286         *p = Read_Flash(Addr, nvr);
00287         while (!(FLASH_READY & BIT0)) {};
00288         size -= 1;
00289         Addr += 4;
00290         p += 1;
00291     }
00292 }

```

5.71 source/lks32mc07x_gpio.c 文件参考

文件名称: [lks32mc07x_gpio.c](#)
 文件标识: 无
 内容摘要: GPIO外设驱动程序
 其它说明: 无

```
#include "lks32mc07x_lib.h"
#include "lks32mc07x_sys.h"
lks32mc07x_gpio.c 的引用(Include)关系图:
```



函数

- void **GPIO_Init** (GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *this)
函数名称: void *GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)*
- void **GPIO_StructInit** (GPIO_InitTypeDef *GPIO_InitStruct)
函数名称: void *GPIO_StructInit(GPIO_InitTypeDef* GPIO_InitStruct)*
- uint8_t **GPIO_ReadInputDataBit** (GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
函数名称: uint8_t *GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)*
- uint32_t **GPIO_ReadInputData** (GPIO_TypeDef *GPIOx)
函数名称: uint32_t *GPIO_ReadInputData(GPIO_TypeDef* GPIOx)*
- uint8_t **GPIO_ReadOutputDataBit** (GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
函数名称: uint8_t *GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)*
- uint32_t **GPIO_ReadOutputData** (GPIO_TypeDef *GPIOx)
函数名称: uint32_t *GPIO_ReadOutputData(GPIO_TypeDef* GPIOx)*
- void **GPIO_SetBits** (GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
函数名称: void *GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)*
- void **GPIO_ResetBits** (GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
函数名称: void *GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)*
- void **GPIO_WriteBit** (GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, BitAction BitVal)
函数名称: void *GPIO_WriteBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal)*
- void **GPIO_Write** (GPIO_TypeDef *GPIOx, uint32_t Val)
函数名称: void *GPIO_Write(GPIO_TypeDef* GPIOx, uint32_t Val)*
- void **GPIO_PinAFConfig** (GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint32_t GPIO_AF)
函数名称: void *GPIO_PinAFConfig(GPIO_TypeDef* GPIOx, uint32_t GPIO_PinSource, uint32_t GPIO_AF)*

5.71.1 详细描述

文件名称: lks32mc07x_gpio.c
文件标识: 无
内容摘要: GPIO外设驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 中定义.

5.71.2 函数说明

5.71.2.1 GPIO_Init()

```
void GPIO_Init (
    GPIO_TypeDef * GPIOx,
    GPIO_InitTypeDef * this )
```

函数名称: void [GPIO_Init\(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct\)](#)

功能描述: GPIO初始化函数

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_InitStruct: GPIO初始化结构体
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
GPIO_InitTypeDef GPIO_InitStruct;
//P2.11输入模式
GPIO_StructInit(&GPIO_InitStruct);           //初始化结构体
GPIO_InitStruct.GPIO_Mode = GPIO_Mode_IN;    //GPIO输入模式
GPIO_InitStruct.GPIO_Pin = GPIO_Pin_11;
GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL;
GPIO_Init(GPIO2, &GPIO_InitStruct);
//P0.2输出模式
GPIO_StructInit(&GPIO_InitStruct);           //初始化结构体
GPIO_InitStruct.GPIO_Mode = GPIO_Mode_OUT;   //GPIO输出模式
GPIO_InitStruct.GPIO_Pin = GPIO_Pin_3;
```

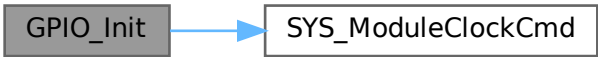
```
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
GPIO_Init(GPIO0, &GPIO_InitStructure);
//P3.2开漏输出
GPIO_StructInit(&GPIO_InitStructure); //初始化结构体
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT; //GPIO输出模式
GPIO_InitStructure.GPIO_PODEna = ENABLE; //使能开漏模式
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL; //关闭上拉，配置为高阻态
GPIO_Init(GPIO3, &GPIO_InitStructure);
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 56 行定义.

函数调用图:



5.71.2.2 GPIO_PinAFConfig()

```
void GPIO_PinAFConfig (
    GPIO_TypeDef * GPIOx,
    uint32_t GPIO_PinSource,
    uint32_t GPIO_AF )
```

函数名称: void [GPIO_PinAFConfig\(GPIO_TypeDef* GPIOx, uint32_t GPIO_PinSource, uint32_t GPIO_AF\)](#)

功能描述: GPIO复用功能选择

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3)	
	GPIO_PadSource:指定的PadSource (GPIO_PinSource_0~GPIO_PinSource_15)	
	GPIO_AF:指定复用功能	
	复用功能名称	说明
	AF0.GPIO	模拟功能
	AF1.SYSAF	SYS_AF、CMP、CLOCK等数字信号输出功能
	AF2.HALL	HALL功能
	AF3.MCPWM	MCPWM功能
	AF4.UART	UART功能
	AF5.SPI	SPI功能
	AF6.I2C	I2C功能
	AF7.TIMER01	Timer0、Timer1功能
	AF8.TIMER23	Timer2、Timer3、QEP0、QEP1功能
	AF9.DEBUG	ADC trigger debug功能
输出参数:	AF10.CAN	CAN功能
	AF11.SIF	SIF功能
	AF12.CL	CL功能
无		

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
GPIO_PinAFConfig(GPIO1,GPIO_PinSource_0,AF4_UART); //P1.0复用为串口功能
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 lks32mc07x_gpio.c 第 399 行定义.

5.71.2.3 GPIO_ReadInputData()

```
uint32_t GPIO_ReadInputData (
    GPIO_TypeDef * GPIOx )
```

函数名称: uint32_t [GPIO_ReadInputData\(GPIO_TypeDef* GPIOx\)](#)

功能描述: 读取GPIO的输入数据

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3)
输出参数:	无

返回
返回值: 输入数据

注解
其它说明: 无

警告
无

示例代码:

```
u8 GPIO_IN_VALUE = 0;
GPIO_IN_VALUE = GPIO\_ReadInputData\(GPIO0\);//读取P0.0~P0.15输入电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 181 行定义.

5.71.2.4 GPIO_ReadInputDataBit()

```
uint8_t GPIO_ReadInputDataBit (
```

```
GPIO_TypeDef * GPIOx,
uint16_t GPIO_Pin )
```

函数名称: `uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)`

功能描述: 读取GPIO的指定Pin的输入值

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15)
输出参数:	无

返回
返回 值: 指定Pin输入数据

注解
其它说明: 无

警告
无

示例代码:

```
u8 GPIO_IN_VALUE = 0;
GPIO_IN_VALUE = GPIO_ReadInputDataBit (GPIO0,GPIO_Pin_0); //读取P0.0输入电平 1为高电平, 0为低电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 `lks32mc07x_gpio.c` 第 146 行定义.

5.71.2.5 GPIO_ReadOutputData()

```
uint32_t GPIO_ReadOutputData (
    GPIO_TypeDef * GPIOx )
```

函数名称: `uint32_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx)`

功能描述: 读取GPIO的输出数据

参见
被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3)
输出参数:	无

返回
返回值: 输出数据

注解
其它说明: 无

警告
无

示例代码:

```
u8 GPIO_OUT_VALUE = 0;
GPIO_OUT_VALUE = GPIO_ReadOutputData(GPIO0); //读取P0.0~P0.15输出电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 `lks32mc07x_gpio.c` 第 243 行定义.

5.71.2.6 GPIO_ReadOutputDataBit()

```
uint8_t GPIO_ReadOutputDataBit (
    GPIO_TypeDef * GPIOx,
    uint16_t GPIO_Pin )
```

函数名称: `uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO.Pin)`

功能描述: 读取GPIO的指定Pin的输出值

参见

被引用内容： 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15)
输出参数:	无

返回

返回 值: 指定IO输出数据

注解

其它说明： 无

警告

无

示例代码:

```
u8 GPIO_OUT_VALUE = 0;
GPIO_OUT_VALUE = GPIO_ReadOutputDataBit (GPIO0,GPIO_Pin_0); //读取P0.0输出电平 1为高电平，0为低电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 208 行定义.

5.71.2.7 GPIO_ResetBits()

```
void GPIO_ResetBits (
    GPIO_TypeDef * GPIOx,
    uint16_t GPIO_Pin )
```

函数名称: void [GPIO_ResetBits\(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin\)](#)

功能描述: GPIO指定Pin置0

参见

被引用内容： 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15)
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
GPIO_ResetBits (GPIO0,GPIO_Pin_0); //P0.0输出低电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 295 行定义.

5.71.2.8 GPIO_SetBits()

```
void GPIO_SetBits (
    GPIO_TypeDef * GPIOx,
    uint16_t GPIO_Pin )
```

函数名称: void [GPIO_SetBits\(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin\)](#)

功能描述: GPIO指定Pin置1

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15)
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
GPIO_SetBits(GPIO0,GPIO_Pin_0); //P0.0输出高电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 269 行定义.

5.71.2.9 GPIO_StructInit()

```
void GPIO_StructInit (
    GPIO_InitTypeDef * GPIO_InitStruct )
```

函数名称: void [GPIO_StructInit\(GPIO_InitTypeDef* GPIO_InitStruct\)](#)

功能描述: GPIO结构体初始化

参见

被引用内容: 无

参数

输入参数:	GPIO_InitStruct: GPIO初始化结构体
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
GPIO_InitTypeDef GPIO_InitStructure;  
GPIO_StructInit(&GPIO_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x-gpio.c](#) 第 115 行定义.

5.71.2.10 GPIO_Write()

```
void GPIO_Write (  
    GPIO_TypeDef * GPIOx,  
    uint32_t Val )
```

函数名称: void [GPIO_Write\(GPIO_TypeDef* GPIOx, uint32_t Val\)](#)

功能描述: 向GPIO写入数据

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) BitVal:写入的Bit值 (0x00~0xffff)
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
GPIO_WriteBit (GPIO0,0x01); //P0.0置1,输出高电平, P0.1~P0.15值不变保持上一时刻状态
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_gpio.c](#) 第 355 行定义.

5.71.2.11 GPIO_WriteBit()

```
void GPIO_WriteBit (
    GPIO_TypeDef * GPIOx,
    uint16_t GPIO_Pin,
    BitAction BitVal )
```

函数名称: void [GPIO_WriteBit\(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, BitAction BitVal\)](#)

功能描述: 向GPIO指定的Pin写入数据

参见

被引用内容: 无

参数

输入参数:	GPIOx:GPIO对象,参数可选 (GPIO0~GPIO3) GPIO_Pin:指定的Pin (GPIO_Pin_0~GPIO_Pin_15) BitVal:写入的Bit值 (Bit_RESET 或 Bit_SET)
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
GPIO_WriteBit(GPIO0,GPIO_Pin_0,Bit_RESET); //P0.0输出低电平
```

Date	Version	Author	Description
2023年2月21日	1.0	cakezhang	创建

修改日志:

在文件 lks32mc07x_gpio.c 第 322 行定义.

5.72 lks32mc07x_gpio.c

[浏览该文件的文档.](#)

```
00001
00014 #include "lks32mc07x.lib.h"
00015 #include "lks32mc07x.sys.h"
00056 void GPIO_Init(GPIO_TypeDef *GPIOx, GPIO_InitTypeDef *this)
00057 {
00058     SYS_ModuleClockCmd(SYS_Module_GPIO, ENABLE);
00059     switch (this->GPIO_Mode)
00060     {
00061         case GPIO_Mode_IN:
00062             GPIOx->POE &= ~this->GPIO_Pin;
00063             GPIOx->PIE |= this->GPIO_Pin;
00064             break;
00065         case GPIO_Mode_OUT:
00066             GPIOx->POE |= this->GPIO_Pin;
00067             GPIOx->PIE &= ~this->GPIO_Pin;
00068             break;
00069         case GPIO_Mode_IO:
00070             GPIOx->POE |= this->GPIO_Pin;
```

```

00071         GPIOx->PIE |= this->GPIO_Pin;
00072         break;
00073
00074     default:
00075     case GPIO_Mode_ANA:
00076         GPIOx->POE &= ~this->GPIO_Pin;
00077         GPIOx->PIE &= ~this->GPIO_Pin;
00078         break;
00079     }
00080     if (this->GPIO_PuPd) // 上拉使能
00081         GPIOx->PUE |= this->GPIO_Pin;
00082     else
00083         GPIOx->PUE &= ~this->GPIO_Pin;
00084     if (this->GPIO_PODEna) // 开漏使能
00085         GPIOx->PODE |= this->GPIO_Pin;
00086     else
00087         GPIOx->PODE &= ~this->GPIO_Pin;
00088     if (this->GPIO_PFLT) // 滤波使能
00089         GPIOx->PFLT |= this->GPIO_Pin;
00090     else
00091         GPIOx->PFLT &= ~this->GPIO_Pin;
00092 }
00093
00115 void GPIO_StructInit(GPIO_InitTypeDef *GPIO_InitStruct)
00116 {
00117     GPIO_InitStruct->GPIO_Pin = GPIO_Pin_NONE;
00118     GPIO_InitStruct->GPIO_Mode = GPIO_Mode_IN;
00119     GPIO_InitStruct->GPIO_PuPd = GPIO_PuPd_NOPULL;
00120     GPIO_InitStruct->GPIO_PODEna = DISABLE;
00121     GPIO_InitStruct->GPIO_PFLT = DISABLE;
00122 }
00123
00146 uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
00147 {
00148     uint8_t bitstatus = 0x00;
00149     if ((GPIOx->PDI & GPIO_Pin) != 0)
00150     {
00151         bitstatus = (uint8_t)Bit_SET;
00152     }
00153     else
00154     {
00155         bitstatus = (uint8_t)Bit_RESET;
00156     }
00157     return bitstatus;
00158 }
00159
00181 uint32_t GPIO_ReadInputData(GPIO_TypeDef *GPIOx)
00182 {
00183     return GPIOx->PDI;
00184 }
00185
00208 uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
00209 {
00210     uint8_t bitstatus = 0x00;
00211     if ((GPIOx->PDO & GPIO_Pin) != 0)
00212     {
00213         bitstatus = (uint8_t)Bit_SET;
00214     }
00215     else
00216     {
00217         bitstatus = (uint8_t)Bit_RESET;
00218     }
00219     return bitstatus;
00220 }
00221
00243 uint32_t GPIO_ReadOutputData(GPIO_TypeDef *GPIOx)
00244 {
00245     return GPIOx->PDO;
00246 }
00247
00269 void GPIO_SetBits(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
00270 {
00271     GPIOx->BSRR = GPIO_Pin;
00272 }
00273
00295 void GPIO_ResetBits(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
00296 {
00297     GPIOx->BRR = GPIO_Pin;
00298 }
00299
00322 void GPIO_WriteBit(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin, BitAction BitVal)
00323 {
00324     if (BitVal != Bit_RESET)
00325     {
00326         GPIOx->BSRR |= GPIO_Pin;
00327     }
00328     else

```

```

00329     {
00330         GPIOx->BRR |= GPIO_Pin;
00331     }
00332 }
00333
00355 void GPIO_Write(GPIO_TypeDef *GPIOx, uint32_t Val)
00356 {
00357     GPIOx->BSRR = Val & 0xffff;
00358 }
00359
00399 void GPIO_PinAFConfig(GPIO_TypeDef *GPIOx, uint32_t GPIO_PinSource, uint32_t GPIO_AF)
00400 {
00401     uint32_t temp;
00402     switch (GPIO_PinSource)
00403     {
00404         /*F3210*/
00405         case 0:
00406         {
00407             temp = GPIOx->F3210 & 0xFFF0; /*get F321*/
00408             GPIOx->F3210 = temp + GPIO_AF; /*F321 + F0*/
00409             break;
00410         }
00411         case 1:
00412         {
00413             temp = GPIOx->F3210 & 0xFF0F;
00414             GPIOx->F3210 = temp + (GPIO_AF << 4);
00415             break;
00416         }
00417         case 2:
00418         { /* 2020.8.13 Repair bug HL */
00419             temp = GPIOx->F3210 & 0xF0FF;
00420             GPIOx->F3210 = temp + (GPIO_AF << 8);
00421             break;
00422         }
00423         case 3:
00424         {
00425             temp = GPIOx->F3210 & 0x0FFF;
00426             GPIOx->F3210 = temp + (GPIO_AF << 12);
00427             break;
00428         }
00429         /*F7654*/
00430         case 4:
00431         {
00432             temp = GPIOx->F7654 & 0xFFF0;
00433             GPIOx->F7654 = temp + GPIO_AF;
00434             break;
00435         }
00436         case 5:
00437         {
00438             temp = GPIOx->F7654 & 0xFF0F;
00439             GPIOx->F7654 = temp + (GPIO_AF << 4);
00440             break;
00441         }
00442         case 6:
00443         {
00444             temp = GPIOx->F7654 & 0xF0FF;
00445             GPIOx->F7654 = temp + (GPIO_AF << 8);
00446             break;
00447         }
00448         case 7:
00449         {
00450             temp = GPIOx->F7654 & 0x0FFF;
00451             GPIOx->F7654 = temp + (GPIO_AF << 12);
00452             break;
00453         }
00454         /*FBA98*/
00455         case 8:
00456         {
00457             temp = GPIOx->FBA98 & 0xFFF0;
00458             GPIOx->FBA98 = temp + GPIO_AF;
00459             break;
00460         }
00461         case 9:
00462         {
00463             temp = GPIOx->FBA98 & 0xFF0F;
00464             GPIOx->FBA98 = temp + (GPIO_AF << 4);
00465             break;
00466         }
00467         case 10:
00468         {
00469             temp = GPIOx->FBA98 & 0xF0FF;
00470             GPIOx->FBA98 = temp + (GPIO_AF << 8);
00471             break;
00472         }
00473         case 11:
00474         {
00475             temp = GPIOx->FBA98 & 0x0FFF;

```



```

00476         GPIOx->FBA98 = temp + (GPIO_AF << 12);
00477         break;
00478     }
00479     /*FFEDC*/
00480     case 12:
00481     {
00482         temp = GPIOx->FFEDC & 0xFFF0;
00483         GPIOx->FFEDC = temp + GPIO_AF;
00484         break;
00485     }
00486     case 13:
00487     {
00488         temp = GPIOx->FFEDC & 0xFF0F;
00489         GPIOx->FFEDC = temp + (GPIO_AF << 4);
00490         break;
00491     }
00492     case 14:
00493     {
00494         temp = GPIOx->FFEDC & 0xF0FF;
00495         GPIOx->FFEDC = temp + (GPIO_AF << 8);
00496         break;
00497     }
00498     case 15:
00499     {
00500         temp = GPIOx->FFEDC & 0x0FFF;
00501         GPIOx->FFEDC = temp + (GPIO_AF << 12);
00502         break;
00503     }
00504     default:
00505         break;
00506 }
00507 }
00508
00509 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.73 source/lks32mc07x_hall.c 文件参考

文件名称: [lks32mc07x_hall.c](#)
 文件标识: 无
 内容摘要: HALL外设驱动程序
 其它说明: 无

```

#include "lks32mc07x_hall.h"
#include "lks32mc07x_lib.h"
lks32mc07x_hall.c 的引用(Include)关系图:

```



函数

- void [HALL_Init](#) (HALL_InitTypeDef *HALL_InitStruct)
函数名称: void [HALL_Init](#)(HALL_InitTypeDef* HALL_InitStruct)
- void [HALL_StructInit](#) (HALL_InitTypeDef *HALL_InitStruct)
函数名称: void [HALL_StructInit](#)(HALL_InitTypeDef* HALL_InitStruct)
- uint32_t [HALL_GetFilterValue](#) (void)
函数名称: uint32_t [HALL_GetFilterValue](#)(void)
- uint32_t [HALL_GetCaptureValue](#) (void)
函数名称: uint32_t [HALL_GetCaptureValue](#)(void)
- uint32_t [HALL_GetCount](#) (void)
函数名称: uint32_t [HALL_GetCount](#)(void)

- uint32_t [HALL_IsCaptureEvent](#) (void)
函数名称: [uint32_t HALL_IsCaptureEvent\(void\)](#)
- uint32_t [HALL_IsOverFlowEvent](#) (void)
函数名称: [uint32_t HALL_IsCaptureEvent\(void\)](#)
- void [HALL_Clear_IRQ](#) (uint32_t tempFlag)
函数名称: [uint32_t HALL_ClearIRQFlag\(uint32_t tempFlag\)](#)
- uint32_t [HALL_GetIRQFlag](#) (uint32_t tempFlag)
函数名称: [uint32_t HALL_GetIRQFlag\(uint32_t tempFlag\)](#)
- void [HALL_SoftTrigger](#) (void)
函数名称: [uint32_t HALL_SoftTrigger\(void\)](#)
- void [HALL_Cmd](#) (FuncState state)
函数名称: [void HALL_Cmd\(FuncState state\)](#)
- uint32_t [HALL_GetFlag](#) (uint32_t tempFlag)

5.73.1 详细描述

文件名称: [lks32mc07x_hall.c](#)
文件标识: 无
内容摘要: HALL外设驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 中定义.

5.73.2 函数说明

5.73.2.1 HALL_Clear_IRQ()

```
void HALL_Clear_IRQ (
    uint32_t tempFlag )
```

函数名称: [uint32_t HALL_ClearIRQFlag\(uint32_t tempFlag\)](#)

功能描述: 清除HALL中断标志位

参见
被引用内容: 无

参数

输入参数:	tempFlag:	
	HALL_CAPTURE_EVENT	HALL信号变化中断
	HALL_OVERFLOW_EVENT	HALL计数溢出中断
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (HALL_GetIRQFlag (HALL_CAPTURE_EVENT)) //读取HALL信号变化中断标志位
{
    HALL_ClearIRQFlag (HALL_CAPTURE_EVENT); //清除HALL信号变化中断标志位
}
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 240 行定义.

5.73.2.2 HALL_Cmd()

```
void HALL_Cmd (
    FuncState state )
```

函数名称: void [HALL_Cmd\(FuncState state\)](#)

功能描述: HALL时钟使能

参见

被引用内容: 无

参数

输入参数:	state:	
	ENABLE	HALL使能
	DISABLE	HALL失能
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
HALL_Cmd (ENABLE) ; //HALL使能
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 323 行定义.

函数调用图:



5.73.2.3 HALL.GetCaptureValue()

```
uint32_t HALL.GetCaptureValue (
    void )
```

函数名称: uint32_t [HALL_GetCaptureValue\(void\)](#)

功能描述: 取得HALL值，未滤波结果，即采样的原始HALL状态值

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回 值: 未滤波的HALL信号结果值，前三位有效，其它位恒为0

注解

其它说明: 无

警告

无

示例代码:

```
uint32_t HALL.Value = 0;  
HALL.Value = HALL\_GetCaptureValue\(\); //读取未滤波的HALL结果值
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 132 行定义.

5.73.2.4 HALL_GetCount()

```
uint32_t HALL_GetCount (  
    void )
```

函数名称: uint32_t [HALL_GetCount\(void\)](#)

功能描述: 读取实时HALL计数器值

参见

被引用内容： 无

参数

输入参数:	无
输出参数:	无

返回

返回 值: 实时HALL计数器值

注解

其它说明: 无

警告

无

示例代码:

```
uint32_t HALL_Value = 0;  
HALL_Value = HALL_GetCount(); //读取实时HALL计数器值
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 157 行定义.

5.73.2.5 HALL_GetFilterValue()

```
uint32_t HALL_GetFilterValue (  
    void )
```

函数名称: uint32_t [HALL_GetFilterValue\(void\)](#)

功能描述: 取得HALL值, 滤波结果

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返 回 值： 经过滤波的HALL信号结果值，前三位有效，其它位恒为0

注解

其它说明： 无

警告

无

示例代码：

```
uint32_t HALL_Value = 0;
HALL_Value = HALL_GetFilterValue(); //读取经过滤波的HALL结果值
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志：

在文件 [lks32mc07x_hall.c](#) 第 107 行定义.

5.73.2.6 HALL_GetFlag()

```
uint32_t HALL_GetFlag (
    uint32_t tempFlag )
```

在文件 [lks32mc07x_hall.c](#) 第 327 行定义.

函数调用图:



5.73.2.7 HALL_GetIRQFlag()

```
uint32_t HALL_GetIRQFlag (
    uint32_t tempFlag )
```

函数名称: `uint32_t HALL_GetIRQFlag(uint32_t tempFlag)`

功能描述: 读取HALL中断标志

参见
被引用内容: 无

参数

输入参数:	tempFlag:	
	HALL_CAPTURE_EVENT	HALL信号变化中断
	HALL_OVERFLOW_EVENT	HALL计数溢出中断
输出参数:	无	

返回
返回值: HALL中断标志 为HALL_CAPTURE_EVENT即对应标志位置位，为0即对应标志位未置位

注解
其它说明: 无

警告
无

示例代码:

```
if (HALL_CAPTURE_EVENT == HALL_GetIRQFlag(HALL_CAPTURE_EVENT)) //读取HALL信号变化中断标志位
{
}
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 `lks32mc07x_hall.c` 第 271 行定义.

这是这个函数的调用关系图:



5.73.2.8 HALL_Init()

```
void HALL_Init (
    HALL_InitTypeDef * HALL_InitStruct )
```

函数名称: void HALL_Init(HALL_InitTypeDef* HALL_InitStruct)

功能描述: HALL初始化函数

参见
被引用内容: 无

参数

输入参数:	HALL_InitTypeDef HALL_InitStruct
输出参数:	无

返回
返回 值: 无

注解
其它说明: 无

警告
无

```

    HALL_InitTypeDef HALL_InitStruct;

HALL_StructInit (&HALL_InitStruct);

HALL_InitStruct.FilterLen = 512; //Hall信号数字滤波长度 512个时钟周期
HALL_InitStruct.ClockDivision = HALL_CLK_DIV8; //设置Hall模块时钟分频系数
HALL_InitStruct.Filter75_Ena = DISABLE; // Hall信号滤波方式, 7判5模式或者全1有效模式
HALL_InitStruct.HALL_Ena = ENABLE; // 模块使能
HALL_InitStruct.Capture_IRQ_Ena = ENABLE; // 捕捉中断使能
HALL_InitStruct.Overflow_IRQ_Ena = ENABLE; // 超时中断使能
HALL_InitStruct.CountTH = 960000; // Hall模块计数模值, 计数超过模值会产生超时中断
    HALL_InitStruct.softIE = ENABLE; // 软件中断失能
HALL_Init (&HALL_InitStruct);
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 48 行定义.

5.73.2.9 HALL_IsCaptureEvent()

```
uint32_t HALL_IsCaptureEvent (
    void )
```

函数名称: `uint32_t HALL_IsCaptureEvent(void)`

功能描述: 读取是否发生捕获事件标志

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回 值: 捕获事件标志, 为HALL_CAPTURE_EVENT即对应标志位置位, 为0即对应标志位未置位

注解

其它说明: 无

警告

无

示例代码:

```
if (HALL_CAPTURE_EVENT == HALL_IsCaptureEvent()) //读取HALL信号变化标志位
{
}
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 183 行定义.

5.73.2.10 HALL_IsOverFlowEvent()

```
uint32_t HALL_IsOverFlowEvent (
    void )
```

函数名称: `uint32_t HALL_IsCaptureEvent(void)`

功能描述: 读取是否发生计时溢出事件标志

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 捕获事件标志, 为HALL_OVERFLOW_EVENT即对应标志位置位, 为0即对应标志位未置位

注解

其它说明: 无

警告

无

示例代码:

```
if ( HALL_OVERFLOW_EVENT == HALL_IsOverFlowEvent () ) //读取计时溢出标志位
{
}
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 209 行定义.

5.73.2.11 HALL_SoftTrigger()

```
void HALL_SoftTrigger (  
    void )
```

函数名称: uint32_t [HALL_SoftTrigger\(void\)](#)

功能描述: 软件触发HALL信号变化中断

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
HALL\_SoftTrigger\(\) ; // 软件触发HALL信号变化中断
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 296 行定义.

5.73.2.12 HALL_StructInit()

```
void HALL_StructInit (  
    HALL_InitTypeDef * HALL_InitStruct )
```

函数名称: void [HALL_StructInit\(HALL_InitTypeDef* HALL_InitStruct\)](#)

功能描述: HALL结构体初始化

参见

被引用内容: 无

参数

输入参数:	HALL_InitTypeDef HALL_InitStruct
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
HALL\_InitTypeDef HALL_InitStruct;  
HALL\_StructInit(&HALL_InitStruct); //初始化结构体
```

Date	Version	Author	Description
2023年02月23日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_hall.c](#) 第 75 行定义.

5.74 lks32mc07x_hall.c

[浏览该文件的文档.](#)

```

00001
00014 #include "lks32mc07x_hall.h"
00015 #include "lks32mc07x_lib.h"
00048 void HALL_Init (HALL_InitTypeDef *HALL_InitStruct)
00049 {
00050     HALL->CFG = (HALL_InitStruct->FilterLen) | (HALL_InitStruct->ClockDivision << 16) |
(HALL_InitStruct->Filter75_Ena << 20) | (HALL_InitStruct->HALL_Ena << 24) |
(HALL_InitStruct->Capture_IRQ_Ena << 28) | (HALL_InitStruct->OverFlow_IRQ_Ena << 29) |
(HALL_InitStruct->softIE << 30);
00051     HALL->TH = HALL_InitStruct->CountTH;
00052     HALL->INFO = 0;
00053 }
00054
00075 void HALL_StructInit (HALL_InitTypeDef *HALL_InitStruct)
00076 {
00077     HALL_InitStruct->FilterLen = 1023;
00078     HALL_InitStruct->ClockDivision = HALL_CLK_DIV1;
00079     HALL_InitStruct->Filter75_Ena = ENABLE;
00080     HALL_InitStruct->HALL_Ena = ENABLE;
00081     HALL_InitStruct->Capture_IRQ_Ena = ENABLE;
00082     HALL_InitStruct->OverFlow_IRQ_Ena = DISABLE;
00083     HALL_InitStruct->CountTH = 1000;
00084     HALL_InitStruct->softIE = DISABLE;
00085 }
00086
00107 uint32_t HALL_GetFilterValue (void)
00108 {
00109     return (HALL->INFO & 0x07);
00110 }
00111
00132 uint32_t HALL_GetCaptureValue (void)
00133 {
00134     return (HALL->INFO >> 8) & 0x07;
00135 }
00136
00157 uint32_t HALL_GetCount (void)
00158 {
00159     return HALL->CNT;
00160 }
00161
00183 uint32_t HALL_IsCaptureEvent (void)
00184 {
00185     return (HALL->INFO & HALL_CAPTURE_EVENT);
00186 }
00187
00209 uint32_t HALL_IsOverFlowEvent (void)
00210 {
00211     return (HALL->INFO & HALL_OVERFLOW_EVENT);
00212 }
00213
00240 void HALL_Clear_IRQ (uint32_t tempFlag)
00241 {
00242     uint32_t temp = HALL->INFO & tempFlag;
00243     HALL->INFO = temp;
00244 }
00271 uint32_t HALL_GetIRQFlag (uint32_t tempFlag)
00272 {
00273     return (HALL->INFO & tempFlag);
00274 }
00275
00296 void HALL_SoftTrigger (void)
00297 {
00298     HALL->INFO |= BIT18;
00299 }
00323 void HALL_Cmd (FuncState state)
00324 {
00325     SYS_ModuleClockCmd(SYS_Module_HALL0, state); // HALL时钟使能
00326 }
00327 uint32_t HALL_GetFlag (uint32_t tempFlag)
00328 {
00329     return HALL_GetIRQFlag (tempFlag);
00330 }
00331
00332 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.75 source/lks32mc07x_i2c.c 文件参考

```
#include "lks32mc07x_i2c.h"
#include "lks32mc07x_sys.h"
#include "lks32mc07x_dma.h"
```

lks32mc07x_i2c.c 的引用(Include)关系图:



函数

- void **I2C_Init** (I2C_TypeDef *I2Cx, I2C_InitTypeDef *this)
I2C初始化
- void **I2C_StructInit** (I2C_InitTypeDef *this)
I2C结构体初始化
- u8 **I2c_ReadBusState** (I2C_TypeDef *I2Cx, u16 n)
读I2C总线状态
- void **I2c_ClearBusState** (I2C_TypeDef *I2Cx, u16 n)
I2C总线状态复位
- void **I2C_SendData** (I2C_TypeDef *I2Cx, u8 n)
I2C发送数据
- u8 **I2C_ReadData** (I2C_TypeDef *I2Cx)
I2C读缓冲区数据
- void **I2C_ClearIRQFlag** (I2C_TypeDef *I2Cx, u16 I2C_IF_x)
I2C清除中断 操作的表: 输出参数:
- u8 **I2C_GetIRQFlag** (I2C_TypeDef *I2Cx, u16 I2C_IF_x)
I2C获取当前中断状态 操作的表: 输出参数:
- u8 **I2C_GetFlag** (I2C_TypeDef *I2Cx, u16 I2C_IF_x)
I2C清除中断
- void **I2C_Enable** (I2C_TypeDef *I2Cx)
I2C模块使能
- void **I2C_Disable** (I2C_TypeDef *I2Cx)
I2C模块关闭
- void **I2C_SetBaudRate** (u32 BaudRate)
设置I2C模块的波特率

5.76 lks32mc07x_i2c.c

[浏览该文件的文档.](#)

```
00001
00009 #include "lks32mc07x_i2c.h"
00010 #include "lks32mc07x_sys.h"
00011 #include "lks32mc07x_dma.h"
00012
00018 void I2C_Init(I2C_TypeDef *I2Cx, I2C_InitTypeDef *this)
00019 {
00020     SYS_ModuleClockCmd(SYS_Module_I2C0, ENABLE);
00021     SYS_SoftResetModule(SYS_Module_I2C0);
```



```

00022
00023 {
00024     u32 BaudRate;
00025     BaudRate = ((u32)96000000 + ((this->BaudRate * 17) >> 1)) / (this->BaudRate * 17) - 1;
00026     SYS.WR.PROTECT = 0x7A83; // 解除系统寄存器写保护
00027     SYS.CLK.DIV0 = BaudRate;
00028     SYS.WR.PROTECT = 0; // 解除系统寄存器写保护
00029 }
00030 I2Cx->ADDR = (this->ADRCMP << 7);
00031 I2Cx->CFG = (this->IE << 7) | (this->TC.IE << 6) |
00032           (this->BUS_ERR.IE << 5) | (this->STOP.IE << 4) |
00033           (this->MST.MODE << 1) | (this->SLV.MODE);
00034 I2Cx->SCR = 0;
00035 I2Cx->DATA = 0;
00036 I2Cx->MSCR = 0;
00037 I2Cx->BCR = (this->BURST.NACK << 7) | (this->BURST.ADDR.CMP << 6) |
00038           (this->BURST.EN << 5);
00039 }
00045 void I2C.StructInit(I2C.InitTypeDef *this)
00046 {
00047     this->ADRCMP = DISABLE; // I2C 硬件地址比较使能开关, 只有在 DMA 模式下开启才有效。
00048     this->MST.MODE = DISABLE; // I2C 主模式使能
00049     this->SLV.MODE = DISABLE; // I2C 从模式使能
00050     this->BURST.EN = DISABLE; // I2C DMA传输使能
00051     this->BaudRate = 100000; // I2C 波特率
00052     this->IE = DISABLE; // I2C 中断使能
00053     this->TC.IE = DISABLE; // I2C 数据传输完成中断使能
00054     this->BUS_ERR.IE = DISABLE; // I2C 总线错误事件中断使能
00055     this->STOP.IE = DISABLE; // I2C STOP 事件中断使能
00056     this->BURST.NACK = DISABLE; // I2C 传输, NACK 事件中断使能
00057     this->BURST.ADDR.CMP = DISABLE; // I2C 传输, 硬件地址匹配中断使能
00058 }
00064 u8 I2C_ReadBusState(I2C.TypeDef *I2Cx, u16 n)
00065 {
00066     if (I2Cx->SCR & n)
00067     {
00068         return ENABLE;
00069     }
00070     else
00071     {
00072         return DISABLE;
00073     }
00074 }
00080 void I2C_ClearBusState(I2C.TypeDef *I2Cx, u16 n)
00081 {
00082     I2Cx->SCR &= ~n;
00083 }
00088 void I2C_SendData(I2C.TypeDef *I2Cx, u8 n)
00089 {
00090     I2Cx->DATA = n;
00091 }
00092
00099 u8 I2C_ReadData(I2C.TypeDef *I2Cx)
00100 {
00101     return (u8) I2Cx->DATA;
00102 }
00103
00110 void I2C_ClearIRQFlag(I2C.TypeDef *I2Cx, u16 I2C_IF_x)
00111 {
00112     switch (I2C_IF_x)
00113     {
00114         case I2C_IF_TC: // 传输完成中断
00115             I2Cx->SCR &= ~BIT0;
00116             break;
00117         case I2C_IF_BUS_ERR: // 总线错误中断
00118             I2Cx->SCR &= ~BIT7;
00119             break;
00120         case I2C_IF_STOP: // STOP事件中断
00121             I2Cx->SCR &= ~BIT5;
00122             break;
00123         case I2C_IF_NACK: // NACK事件中断
00124             I2Cx->SCR &= ~BIT1;
00125             break;
00126         case I2C_IF_ADDR_CMP: // 硬件地址比较中断
00127             break;
00128         default:
00129             break;
00130     }
00131 }
00132
00139 u8 I2C_GetIRQFlag(I2C.TypeDef *I2Cx, u16 I2C_IF_x)
00140 {
00141     switch (I2C_IF_x)
00142     {
00143         case I2C_IF_TC: // 传输完成中断
00144             if ((I2Cx->CFG & BIT7) == 0)
00145

```

```

00146         return 0;
00147     }
00148     if ((I2Cx->CFG & BIT6) == 0)
00149     {
00150         return 0;
00151     }
00152     if (I2Cx->SCR & BIT0)
00153     {
00154         return 1;
00155     }
00156     else
00157     {
00158         return 0;
00159     }
00160     case I2C_IF_BUS_ERR: // 总线错误中断
00161     if ((I2Cx->CFG & BIT7) == 0)
00162     {
00163         return 0;
00164     }
00165     if ((I2Cx->CFG & BIT5) == 0)
00166     {
00167         return 0;
00168     }
00169     if (I2Cx->SCR & BIT7)
00170     {
00171         return 1;
00172     }
00173     else
00174     {
00175         return 0;
00176     }
00177     case I2C_IF_STOP: // STOP事件中断
00178     if ((I2Cx->CFG & BIT7) == 0)
00179     {
00180         return 0;
00181     }
00182     if ((I2Cx->CFG & BIT4) == 0)
00183     {
00184         return 0;
00185     }
00186     if (I2Cx->SCR & BIT5)
00187     {
00188         return 1;
00189     }
00190     else
00191     {
00192         return 0;
00193     }
00194     case I2C_IF_NACK: // NACK事件中断
00195     if ((I2Cx->CFG & BIT7) == 0)
00196     {
00197         return 0;
00198     }
00199     if ((I2Cx->BCR & BIT7) == 0)
00200     {
00201         return 0;
00202     }
00203     if (I2Cx->SCR & BIT1)
00204     {
00205         return 1;
00206     }
00207     else
00208     {
00209         return 0;
00210     }
00211     case I2C_IF_ADDR_CMP: // 硬件地址匹配中断
00212     if ((I2Cx->CFG & BIT7) == 0)
00213     {
00214         return 0;
00215     }
00216     if ((I2Cx->BCR & BIT6) == 0)
00217     {
00218         return 0;
00219     }
00220     if (I2Cx->SCR & BIT3)
00221     {
00222         if ((u32)(I2Cx->DATA >> 1) == (I2Cx->ADDR & 0x7f))
00223         {
00224             return 1;
00225         }
00226         else
00227         {
00228             return 0;
00229         }
00230     }
00231     else
00232     {

```

```

00233         return 0;
00234     }
00235     default:
00236         return 0;
00237     }
00238 }
00244 u8 I2C_GetFlag(I2C_TypeDef *I2Cx, u16 I2C_IF_x)
00245 {
00246     switch (I2C_IF_x)
00247     {
00248         case I2C_IF_TC: // 传输完成中断
00249             if (I2Cx->SCR & BIT0)
00250             {
00251                 return 1;
00252             }
00253             else
00254             {
00255                 return 0;
00256             }
00257         case I2C_IF_BUS_ERR: // 总线错误中断
00258             if (I2Cx->SCR & BIT7)
00259             {
00260                 return 1;
00261             }
00262             else
00263             {
00264                 return 0;
00265             }
00266         case I2C_IF_STOP: // STOP事件中断
00267             if (I2Cx->SCR & BIT5)
00268             {
00269                 return 1;
00270             }
00271             else
00272             {
00273                 return 0;
00274             }
00275         case I2C_IF_NACK: // NACK事件中断
00276             if (I2Cx->SCR & BIT1)
00277             {
00278                 return 1;
00279             }
00280             else
00281             {
00282                 return 0;
00283             }
00284         case I2C_IF_ADDR_CMP: // 硬件地址匹配中断
00285             if (I2Cx->SCR & BIT3)
00286             {
00287                 if ((u32)(I2Cx->DATA >> 1) == (I2Cx->ADDR & 0x7f))
00288                 {
00289                     return 1;
00290                 }
00291                 else
00292                 {
00293                     return 0;
00294                 }
00295             }
00296             else
00297             {
00298                 return 0;
00299             }
00300         default:
00301             return 0;
00302     }
00303 }
00304
00310 void I2C_Enable(I2C_TypeDef *I2Cx)
00311 {
00312     if (I2Cx == I2C0)
00313     {
00314         SYS_ModuleClockCmd(SYS_Module_I2C0, ENABLE);
00315     }
00316 }
00322 void I2C_Disable(I2C_TypeDef *I2Cx)
00323 {
00324     if (I2Cx == I2C0)
00325     {
00326         SYS_ModuleClockCmd(SYS_Module_I2C0, DISABLE);
00327     }
00328 }
00329
00335 void I2C_SetBaudRate(u32 BaudRate)
00336 {
00337     u32 div;
00338     div = ((u32)96000000 + ((BaudRate * 17) >> 1)) / (BaudRate * 17) - 1;
00339     SYS_WR_PROTECT = 0x7A83;

```

```

00340     SYS_CLK_DIV0    = div;
00341     SYS_WR_PROTECT = 0;
00342 }
00343

```

5.77 source/lks32mc07x_mcpwm.c 文件参考

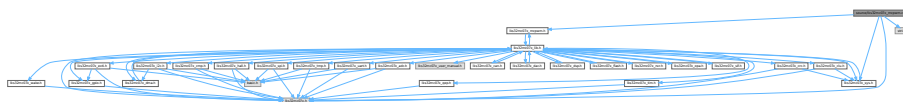
文件名称: [lks32mc07x_mcpwm.c](#)
 文件标识: 无
 内容摘要: MCPWM外设驱动程序
 其它说明: 无

```

#include "lks32mc07x_mcpwm.h"
#include "lks32mc07x.h"
#include "lks32mc07x_sys.h"
#include "string.h"

```

lks32mc07x_mcpwm.c 的引用(Include)关系图:



函数

- void [MCPWM_StructInit](#) (MCPWM_InitTypeDef *MCPWM_InitStruct)
函数名称: *void MCPWM_StructInit(MCPWM_InitTypeDef* MCPWM_InitStruct)*
- void [MCPWM_Init](#) (MCPWM_InitTypeDef *MCPWM_InitStruct)
函数名称: *void MCPWM_Init(MCPWM_InitTypeDef *MCPWM_InitStruct)*
- void [PWMOutputs](#) (FuncState t.state)
函数名称: *void PWMOutputs(FuncState t.state)*
- void [MCPWM_ClearIRQFlag](#) (MCPWM0CNTx CNTx, uint32_t INT_flag)
函数名称: *void MCPWM_ClearIRQFlag(uint32_t INT_flag)*
- u16 [MCPWM_GetIRQFlag](#) (MCPWM0CNTx CNTx, uint32_t INT_flag)
函数名称: *u16 MCPWM_GetIRQFlag(MCPWM0CNTx CNTx,uint32_t INT_flag)*
- void [MCPWM_Clk_Enable](#) (u8 Clk_Enable)
- void [MCPWM_DIV_Set](#) (u8 ClockDiv)
- void [MCPWM_CNTx_Enable](#) (MCPWM0CNTx CNTx, FuncState state)
- void [MCPWM_CH012_Init](#) (MCPWM_InitTypeDef *MCPWM_InitStruct)
- void [MCPWM_CH345_Init](#) (MCPWM_InitTypeDef *MCPWM_InitStruct)
- void [MCPWM_CH012_IESet](#) (MCPWM_InitTypeDef *MCPWM_InitStruct)
- void [MCPWM_CH345_IESet](#) (MCPWM_InitTypeDef *MCPWM_InitStruct)
- void [MCPWM_ModuleClockCmd](#) (FuncState state)

变量

- u8 [ClockDivflg](#) = 0

5.77.1 详细描述

文件名称: lks32mc07x_mcpwm.c
文件标识: 无
内容摘要: MCPWM外设驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年3月06日	1.0	Zhujie	创建

修改日志:

在文件 lks32mc07x_mcpwm.c 中定义.

5.77.2 函数说明

5.77.2.1 MCPWM.CH012.IESet()

```
void MCPWM_CH012_IESet (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

在文件 lks32mc07x_mcpwm.c 第 583 行定义.

5.77.2.2 MCPWM.CH012.Init()

```
void MCPWM_CH012_Init (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

在文件 lks32mc07x_mcpwm.c 第 432 行定义.

5.77.2.3 MCPWM.CH345.IESet()

```
void MCPWM_CH345_IESet (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

在文件 lks32mc07x_mcpwm.c 第 590 行定义.

5.77.2.4 MCPWM_CH345_Init()

```
void MCPWM_CH345_Init (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 508 行定义.

5.77.2.5 MCPWM_ClearIRQFlag()

```
void MCPWM_ClearIRQFlag (
    MCPWM0CNTx CNTx,
    uint32_t INT_flag )
```

函数名称: void MCPWM_ClearIRQFlag(uint32_t INT_flag)

功能描述: 清除MCPWM中断标志

参见

被引用内容: 无

参数

参数

输入参数:	MCPWM0CNTx	MCPWM0CNT0 计数器0	
	MCPWM0CNTx	MCPWM0CNT1 计数器1	
	MCPWM0CNT0.T0_IRQ_IF	BIT0	计数器0计数值回到-MCPWM.TH中断标志位
	MCPWM0CNT0.T1_IRQ_IF	BIT1	计数器0计数值回到0中断标志位
	MCPWM0CNT0.TH00↔ IRQ_IF	BIT2	计数器0计数值回到MCPWM.TH00中断标志位
	MCPWM0CNT0.TH01↔ IRQ_IF	BIT3	计数器0计数值回到MCPWM.TH01中断标志位
	MCPWM0CNT0.TH10↔ IRQ_IF	BIT4	计数器0计数值回到MCPWM.TH10中断标志位
	MCPWM0CNT0.TH11↔ IRQ_IF	BIT5	计数器0计数值回到MCPWM.TH11中断标志位
	MCPWM0CNT0.TH20↔ IRQ_IF	BIT6	计数器0计数值回到MCPWM.TH20中断标志位
	MCPWM0CNT0.TH21↔ IRQ_IF	BIT7	计数器0计数值回到MCPWM.TH21中断标志位
	MCPWM0CNT0.TMR0↔ IRQ_IF	BIT10	计数器0计数值回到MCPWM.TMR0中断标志位
	MCPWM0CNT0.TMR1↔ IRQ_IF	BIT11	计数器0计数值回到MCPWM.TMR1中断标志位
	MCPWM0CNT0.TMR2↔ IRQ_IF	BIT12	计数器0计数值回到MCPWM.TMR2中断标志位
	MCPWM0CNT0.TMR3↔ IRQ_IF	BIT13	计数器0计数值回到MCPWM.TMR3中断标志位
	MCPWM0CNT0.UP_IRQ_IF	BIT14	计数器0更新中断标志位
	MCPWM0CNT1.T0_IRQ_IF	BIT0	计数器1计数值回到-MCPWM.TH中断标志位
	MCPWM0CNT1.T1_IRQ_IF	BIT1	计数器1计数值回到0中断标志位
	MCPWM0CNT1.TH30↔ IRQ_IF	BIT2	计数器1计数值回到MCPWM.TH30中断标志位
	MCPWM0CNT1.TH31↔ IRQ_IF	BIT3	计数器1计数值回到MCPWM.TH31中断标志位
	MCPWM0CNT1.TH40↔ IRQ_IF	BIT4	计数器1计数值回到MCPWM.TH40中断标志位
	MCPWM0CNT1.TH41↔ IRQ_IF	BIT5	计数器1计数值回到MCPWM.TH41中断标志位
	MCPWM0CNT1.TH50↔ IRQ_IF	BIT6	计数器1计数值回到MCPWM.TH50中断标志位
	MCPWM0CNT1.TH51↔ IRQ_IF	BIT7	计数器1计数值回到MCPWM.TH51中断标志位

参数

输出参数:	无
-------	---

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (MCPWM_GetIRQFlag(MCPWM0CNT0, MCPWM0CNT0.T0_IRQ_IF)) //获取计数器0计数值回到-MCPWM_TH中断标志位
{
    MCPWM_ClearIRQFlag(MCPWM0CNT0, MCPWM0CNT0.T0_IRQ_IF) //清除计数器0计数值回到-MCPWM_TH中断标志位
}
```

Date	Version	Author	Description
2023年03月06日	1.0	Zhujie	创建

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 267 行定义.

5.77.2.6 MCPWM_Clk.Enable()

```
void MCPWM_Clk.Enable (
    u8 Clk_Enable )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 375 行定义.

5.77.2.7 MCPWM_CNTx.Enable()

```
void MCPWM_CNTx.Enable (
    MCPWM0CNTx CNTx,
    FuncState state )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 404 行定义.

5.77.2.8 MCPWM_DIV_Set()

```
void MCPWM_DIV_Set (
    u8 ClockDiv )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 390 行定义.

5.77.2.9 MCPWM_GetIRQFlag()

```
ul6 MCPWM_GetIRQFlag (
    MCPWM0CNTx CNTx,
    uint32_t INT_flag )
```

函数名称: [u16 MCPWM_GetIRQFlag\(MCPWM0CNTx CNTx,uint32.t INT_flag\)](#)

功能描述: 获取MCPWM中断标志

参见

被引用内容: 无

参数

参数

输入参数:	MCPWM0CNTx	MCPWM0CNT0 计数器0	
	MCPWM0CNTx	MCPWM0CNT1 计数器1	
	MCPWM0CNT0.T0_IRQ_IF	BIT0	计数器0计数值回到-MCPWM.TH中断标志位
	MCPWM0CNT0.T1_IRQ_IF	BIT1	计数器0计数值回到0中断标志位
	MCPWM0CNT0.TH00↔ IRQ_IF	BIT2	计数器0计数值回到MCPWM.TH00中断标志位
	MCPWM0CNT0.TH01↔ IRQ_IF	BIT3	计数器0计数值回到MCPWM.TH01中断标志位
	MCPWM0CNT0.TH10↔ IRQ_IF	BIT4	计数器0计数值回到MCPWM.TH10中断标志位
	MCPWM0CNT0.TH11↔ IRQ_IF	BIT5	计数器0计数值回到MCPWM.TH11中断标志位
	MCPWM0CNT0.TH20↔ IRQ_IF	BIT6	计数器0计数值回到MCPWM.TH20中断标志位
	MCPWM0CNT0.TH21↔ IRQ_IF	BIT7	计数器0计数值回到MCPWM.TH21中断标志位
	MCPWM0CNT0.TMR0↔ IRQ_IF	BIT10	计数器0计数值回到MCPWM.TMR0中断标志位
	MCPWM0CNT0.TMR1↔ IRQ_IF	BIT11	计数器0计数值回到MCPWM.TMR1中断标志位
	MCPWM0CNT0.TMR2↔ IRQ_IF	BIT12	计数器0计数值回到MCPWM.TMR2中断标志位
	MCPWM0CNT0.TMR3↔ IRQ_IF	BIT13	计数器0计数值回到MCPWM.TMR3中断标志位
	MCPWM0CNT0.UP_IRQ_IF	BIT14	计数器0更新中断标志位
	MCPWM0CNT1.T0_IRQ_IF	BIT0	计数器1计数值回到-MCPWM.TH中断标志位
	MCPWM0CNT1.T1_IRQ_IF	BIT1	计数器1计数值回到0中断标志位
	MCPWM0CNT1.TH30↔ IRQ_IF	BIT2	计数器1计数值回到MCPWM.TH30中断标志位
	MCPWM0CNT1.TH31↔ IRQ_IF	BIT3	计数器1计数值回到MCPWM.TH31中断标志位
	MCPWM0CNT1.TH40↔ IRQ_IF	BIT4	计数器1计数值回到MCPWM.TH40中断标志位
	MCPWM0CNT1.TH41↔ IRQ_IF	BIT5	计数器1计数值回到MCPWM.TH41中断标志位
	MCPWM0CNT1.TH50↔ IRQ_IF	BIT6	计数器1计数值回到MCPWM.TH50中断标志位
	MCPWM0CNT1.TH51↔ IRQ_IF	BIT7	计数器1计数值回到MCPWM.TH51中断标志位

参数

输出参数:	无
-------	---

返回

返回 值: 读对应标志位是否为1, 1位置位, 0为未置位

注解

其它说明: 无

警告

只有对应中断使能后, 改为才能读取, 如果对应中断未使能, 读取结果一直为0

示例代码:

```
if (MCPWM_GetIRQFlag (MCPWM0CNT0, MCPWM0CNT0_T0_IRQ_IF)) //获取计数器0计数值回到-MCPWM_TH中断标志位
{
}
```

Date	Version	Author	Description
2023年03月06日	1.0	Zhujie	创建

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 333 行定义.

5.77.2.10 MCPWM.Init()

```
void MCPWM_Init (
    MCPWM_InitTypeDef * MCPWM_InitStruct )
```

函数名称: void [MCPWM_Init\(MCPWM_InitTypeDef *MCPWM_InitStruct\)](#)

功能描述: MCPWM初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	MCPWM_InitTypeDef *MCPWM_InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
MCPWM_InitTypeDef MCPWM_InitStructure;  
MCPWM_StructInit (&MCPWM_InitStructure); //初始化MCPWM相关结构体  
MCPWM_InitStructure.MCLK_EN = ENABLE;  
MCPWM_InitStructure.CLK_DIV = 0;  
MCPWM_InitStructure.CMP_FLT_CLKDIV = 12;  
MCPWM_InitStructure.BASE_CNT0_EN = ENABLE;  
MCPWM_InitStructure.TH0 = PWM_PERIOD0;  
MCPWM_InitStructure.TH00 = -(PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH01 = (PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH10 = -(PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH11 = (PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH20 = -(PWM_PERIOD0 >> 2);  
MCPWM_InitStructure.TH21 = (PWM_PERIOD0 >> 2);  
  
MCPWM_InitStructure.MCPWM_WorkModeCH0 = MCPWM0_CENTRAL_PWM_MODE;  
MCPWM_InitStructure.MCPWM_WorkModeCH1 = MCPWM0_CENTRAL_PWM_MODE;  
MCPWM_InitStructure.MCPWM_WorkModeCH2 = MCPWM0_CENTRAL_PWM_MODE;  
MCPWM_InitStructure.DeadTimeCH012N = DEADTIME;  
MCPWM_InitStructure.DeadTimeCH012P = DEADTIME;  
MCPWM_InitStructure.CMP_CTRL_CNT0 = DISABLE;  
MCPWM_InitStructure.EVT_CNT0_EN = DISABLE;  
MCPWM_InitStructure.EVT0 = DISABLE;  
  
MCPWM_InitStructure.TR0_UP_INTV = DISABLE;  
MCPWM_InitStructure.TR0_T0_UpdateEN = ENABLE;  
MCPWM_InitStructure.TR0_T1_UpdateEN = DISABLE;  
MCPWM_InitStructure.TR0_AEC = DISABLE;  
  
MCPWM_InitStructure.T0_Update0_INT_EN = DISABLE;  
MCPWM_InitStructure.T1_Update0_INT_EN = ENABLE;  
MCPWM_InitStructure.Update1_INT_EN = ENABLE;  
MCPWM_InitStructure.CH0N.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH0P.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH1N.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH1P.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH2N.Polarity_INV = DISABLE;  
MCPWM_InitStructure.CH2P.Polarity_INV = DISABLE;  
  
MCPWM_InitStructure.Switch_CH0N_CH0P = DISABLE;  
MCPWM_InitStructure.Switch_CH1N_CH1P = DISABLE;  
MCPWM_InitStructure.Switch_CH2N_CH2P = DISABLE;  
MCPWM_InitStructure.CH0P.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH0N.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH1P.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH1N.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH2P.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_InitStructure.CH2N.default_output = MCPWM0_LOW_LEVEL;  
MCPWM_Init (&MCPWM_InitStructure);
```

Date	Version	Author	Description
2023年03月06日	1.0	Zhujie	创建

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 111 行定义.

函数调用图:



5.77.2.11 MCPWM.ModuleClockCmd()

```
void MCPWM.ModuleClockCmd (
    FuncState state )
```

在文件 [lks32mc07x_mcpwm.c](#) 第 597 行定义.

函数调用图:



5.77.2.12 MCPWM.StructInit()

```
void MCPWM.StructInit (
    MCPWM.InitTypeDef * MCPWM.InitStruct )
```

函数名称: void [MCPWM.StructInit\(MCPWM.InitTypeDef* MCPWM.InitStruct\)](#)

功能描述: MCPWM结构体初始化

参见
被引用内容: 无

参数

输入参数:	MCPWM.InitTypeDef *MCPWM.InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
MCPWM_InitTypeDef MCPWM_InitStructure;  
MCPWM_StructInit (&MCPWM_InitStructure); //初始化MCPWM相关结构体
```

Date	Version	Author	Description
2019年05月21日	1.0	Howlet	创建

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 39 行定义.

5.77.2.13 PWMOutputs()

```
void PWMOutputs (  
    FuncState t_state )
```

函数名称: void [PWMOutputs\(FuncState t_state\)](#)

功能描述: MCPWM波形输出使能与失能函数

参见

被引用内容: 无

参数

输入参数:	FuncState t_state ENABLE: 使能, DISABLE:使能
输出参数:	无

返回

返回 值: 无

注解

其它说明： 无

警告

无

示例代码:

```
PWMOutputs (ENABLE) //使能MCPWM输出
```

Date	Version	Author	Description
2019年05月21日	1.0	Howlet	创建
2023年03月06日	2.0	Zhujie	修改

修改日志:

在文件 [lks32mc07x_mcpwm.c](#) 第 202 行定义.

5.77.3 变量说明

5.77.3.1 ClockDivflg

```
u8 ClockDivflg = 0
```

在文件 [lks32mc07x_mcpwm.c](#) 第 389 行定义.

5.78 lks32mc07x_mcpwm.c

[浏览该文件的文档.](#)

```
00001
00014 #include "lks32mc07x_mcpwm.h"
00015 #include "lks32mc07x.h"
00016 #include "lks32mc07x_sys.h"
00017 #include "string.h"
00018
00039 void MCPWM_StructInit(MCPWM_InitTypeDef *MCPWM_InitStruct)
00040 {
00041     memset(MCPWM_InitStruct, 0, sizeof(MCPWM_InitTypeDef));
00042 }
00043
00111 void MCPWM_Init(MCPWM_InitTypeDef *MCPWM_InitStruct)
00112 {
00113     SYS_ModuleClockCmd(SYS_Module_MCPWM0, ENABLE); // MCPWM时钟使能
00114
00115     MCPWM0_PRT = 0x0000DEAD; /*enter password to unlock write protection */
00116
00117     MCPWM0_TCLK = MCPWM_InitStruct->CLK_DIV | (MCPWM_InitStruct->MCLK_EN << 2) |
(MCPWM_InitStruct->ZCS_EN << 3) | (MCPWM_InitStruct->TMR2_TB << 4) | (MCPWM_InitStruct->TMR3_TB << 5) |
(MCPWM_InitStruct->BASE_CNT0_EN << 6) | (MCPWM_InitStruct->BASE_CNT1_EN << 7) |
(MCPWM_InitStruct->EVT_CNT0_EN << 8) | (MCPWM_InitStruct->EVT_CNT1_EN << 9) |
(MCPWM_InitStruct->CMP_CTRL_CNT0 << 10) | (MCPWM_InitStruct->CMP_CTRL_CNT1 << 11);
00118
```

```

00119     MCPWM0.TMR0 = MCPWM.InitStruct->TMR0;
00120     MCPWM0.TMR1 = MCPWM.InitStruct->TMR1;
00121     MCPWM0.TMR2 = MCPWM.InitStruct->TMR2;
00122     MCPWM0.TMR3 = MCPWM.InitStruct->TMR3;
00123
00124     MCPWM0.TH0 = MCPWM.InitStruct->TH0;
00125     MCPWM0.TH1 = MCPWM.InitStruct->TH1;
00126
00127     MCPWM0.SDCFG = MCPWM.InitStruct->TR0.UP.INTV | (MCPWM.InitStruct->TR0.T0.UpdateEN << 4) |
(MCPWM.InitStruct->TR0.T1.UpdateEN << 5) | (MCPWM.InitStruct->TR0.AEC << 6) |
(MCPWM.InitStruct->TR1.UP.INTV << 8) | (MCPWM.InitStruct->TR1.T0.UpdateEN << 12) |
(MCPWM.InitStruct->TR1.T1.UpdateEN << 13) | (MCPWM.InitStruct->TR1.AEC << 14);
00128
00129     MCPWM0.DTH00 = MCPWM.InitStruct->DeadTimeCH012N;
00130     MCPWM0.DTH01 = MCPWM.InitStruct->DeadTimeCH012P;
00131     MCPWM0.DTH10 = MCPWM.InitStruct->DeadTimeCH345N;
00132     MCPWM0.DTH11 = MCPWM.InitStruct->DeadTimeCH345P;
00133
00134     MCPWM0.EIF = 0xffff;
00135     MCPWM0.IE0 = MCPWM.InitStruct->T0.Update0.INT.EN | (MCPWM.InitStruct->T1.Update0.INT.EN << 1) |
(MCPWM.InitStruct->TH00.Match0.INT.EN << 2) | (MCPWM.InitStruct->TH01.Match0.INT.EN << 3) |
(MCPWM.InitStruct->TH10.Match0.INT.EN << 4) | (MCPWM.InitStruct->TH11.Match0.INT.EN << 5) |
(MCPWM.InitStruct->TH20.Match0.INT.EN << 6) | (MCPWM.InitStruct->TH21.Match0.INT.EN << 7) |
(MCPWM.InitStruct->TMR0.Match0.INT.EN << 10) | (MCPWM.InitStruct->TMR1.Match0.INT.EN << 11) |
(MCPWM.InitStruct->TMR2.Match0.INT.EN << 12) | (MCPWM.InitStruct->TMR3.Match0.INT.EN << 13) |
(MCPWM.InitStruct->Update0.INT.EN << 14);
00136
00137     MCPWM0.IE1 = MCPWM.InitStruct->T0.Update1.INT.EN | (MCPWM.InitStruct->T1.Update1.INT.EN << 1) |
(MCPWM.InitStruct->TH00.Match1.INT.EN << 2) | (MCPWM.InitStruct->TH01.Match1.INT.EN << 3) |
(MCPWM.InitStruct->TH10.Match1.INT.EN << 4) | (MCPWM.InitStruct->TH11.Match1.INT.EN << 5) |
(MCPWM.InitStruct->TH20.Match1.INT.EN << 6) | (MCPWM.InitStruct->TH21.Match1.INT.EN << 7) |
(MCPWM.InitStruct->TMR0.Match1.INT.EN << 10) | (MCPWM.InitStruct->TMR1.Match1.INT.EN << 11) |
(MCPWM.InitStruct->TMR2.Match1.INT.EN << 12) | (MCPWM.InitStruct->TMR3.Match1.INT.EN << 13) |
(MCPWM.InitStruct->Update1.INT.EN << 14);
00138
00139     MCPWM0.EIE = (MCPWM.InitStruct->FAIL0.INT.EN << 4) | (MCPWM.InitStruct->FAIL1.INT.EN << 5) |
(MCPWM.InitStruct->FAIL2.INT.EN << 6) | (MCPWM.InitStruct->FAIL3.INT.EN << 7) |
(MCPWM.InitStruct->ZCS0.INT.EN << 8) | (MCPWM.InitStruct->ZCS1.INT.EN << 9);
00140
00141     MCPWM0.EVT0 = MCPWM.InitStruct->EVT0;
00142
00143     MCPWM0.EVT1 = MCPWM.InitStruct->EVT1;
00144
00145     MCPWM0.FLT = MCPWM.InitStruct->IO.CMP.FLT.CLKDIV;
00146
00147     MCPWM0.RE = MCPWM.InitStruct->TMR0.RE0 | (MCPWM.InitStruct->TMR1.RE0 << 1) |
(MCPWM.InitStruct->TMR2.RE0 << 2) | (MCPWM.InitStruct->TMR3.RE0 << 3) | (MCPWM.InitStruct->TR0.T0.RE0 <<
4) | (MCPWM.InitStruct->TR0.T1.RE0 << 5) | (MCPWM.InitStruct->TR1.T0.RE0 << 6) |
(MCPWM.InitStruct->TR1.T1.RE0 << 7) | (MCPWM.InitStruct->TMR0.RE1 << 8) | (MCPWM.InitStruct->TMR1.RE1 <<
9) | (MCPWM.InitStruct->TMR2.RE1 << 10) | (MCPWM.InitStruct->TMR3.RE1 << 11) |
(MCPWM.InitStruct->TR0.T0.RE1 << 12) | (MCPWM.InitStruct->TR0.T1.RE1 << 13) |
(MCPWM.InitStruct->TR1.T0.RE1 << 14) | (MCPWM.InitStruct->TR1.T1.RE1 << 15);
00148
00149     MCPWM0.PP = MCPWM.InitStruct->IO0.PPE | (MCPWM.InitStruct->IO1.PPE << 1) | (MCPWM.InitStruct->IO2.PPE
<< 2) | (MCPWM.InitStruct->IO3.PPE << 3) | (MCPWM.InitStruct->IO4.PPE << 4) | (MCPWM.InitStruct->IO5.PPE
<< 5);
00150
00151     MCPWM0.IO01 = MCPWM.InitStruct->CH0N.Polarity.INV | (MCPWM.InitStruct->CH0P.Polarity.INV << 1) |
(MCPWM.InitStruct->CH0.NS << 2) | (MCPWM.InitStruct->CH0.PS << 3) | (MCPWM.InitStruct->CH0N.SCTRLN << 4) |
(MCPWM.InitStruct->CH0P.SCTRLP << 5) | (MCPWM.InitStruct->Switch.CH0N.CH0P << 6) |
(MCPWM.InitStruct->MCPWM.WorkModeCH0 << 7) | (MCPWM.InitStruct->CH1N.Polarity.INV << 8) |
(MCPWM.InitStruct->CH1P.Polarity.INV << 9) | (MCPWM.InitStruct->CH1.NS << 10) | (MCPWM.InitStruct->CH1.PS
<< 11) | (MCPWM.InitStruct->CH1N.SCTRLN << 12) | (MCPWM.InitStruct->CH1P.SCTRLP << 13) |
(MCPWM.InitStruct->Switch.CH1N.CH1P << 14) | (MCPWM.InitStruct->MCPWM.WorkModeCH1 << 15);
00152
00153     MCPWM0.IO23 = MCPWM.InitStruct->CH2N.Polarity.INV | (MCPWM.InitStruct->CH2P.Polarity.INV << 1) |
(MCPWM.InitStruct->CH2.NS << 2) | (MCPWM.InitStruct->CH2.PS << 3) | (MCPWM.InitStruct->CH2N.SCTRLN << 4) |
(MCPWM.InitStruct->CH2P.SCTRLP << 5) | (MCPWM.InitStruct->Switch.CH2N.CH2P << 6) |
(MCPWM.InitStruct->MCPWM.WorkModeCH2 << 7) | (MCPWM.InitStruct->CH3N.Polarity.INV << 8) |
(MCPWM.InitStruct->CH3P.Polarity.INV << 9) | (MCPWM.InitStruct->CH3.NS << 10) | (MCPWM.InitStruct->CH3.PS
<< 11) | (MCPWM.InitStruct->CH3N.SCTRLN << 12) | (MCPWM.InitStruct->CH3P.SCTRLP << 13) |
(MCPWM.InitStruct->Switch.CH3N.CH3P << 14) | (MCPWM.InitStruct->MCPWM.WorkModeCH3 << 15);
00154
00155     MCPWM0.IO45 = MCPWM.InitStruct->CH4N.Polarity.INV | (MCPWM.InitStruct->CH4P.Polarity.INV << 1) |
(MCPWM.InitStruct->CH4.NS << 2) | (MCPWM.InitStruct->CH4.PS << 3) | (MCPWM.InitStruct->CH4N.SCTRLN << 4) |
(MCPWM.InitStruct->CH4P.SCTRLP << 5) | (MCPWM.InitStruct->Switch.CH4N.CH4P << 6) |
(MCPWM.InitStruct->MCPWM.WorkModeCH4 << 7) | (MCPWM.InitStruct->CH5N.Polarity.INV << 8) |
(MCPWM.InitStruct->CH5P.Polarity.INV << 9) | (MCPWM.InitStruct->CH5.NS << 10) | (MCPWM.InitStruct->CH5.PS
<< 11) | (MCPWM.InitStruct->CH5N.SCTRLN << 12) | (MCPWM.InitStruct->CH5P.SCTRLP << 13) |
(MCPWM.InitStruct->Switch.CH5N.CH5P << 14) | (MCPWM.InitStruct->MCPWM.WorkModeCH5 << 15);
00156
00157     MCPWM0.TH00 = MCPWM.InitStruct->TH00;
00158     MCPWM0.TH01 = MCPWM.InitStruct->TH01;
00159     MCPWM0.TH10 = MCPWM.InitStruct->TH10;
00160     MCPWM0.TH11 = MCPWM.InitStruct->TH11;
00161     MCPWM0.TH20 = MCPWM.InitStruct->TH20;
00162     MCPWM0.TH21 = MCPWM.InitStruct->TH21;

```

```

00163     MCPWM0.TH30 = MCPWM.InitStruct->TH30;
00164     MCPWM0.TH31 = MCPWM.InitStruct->TH31;
00165     MCPWM0.TH40 = MCPWM.InitStruct->TH40;
00166     MCPWM0.TH41 = MCPWM.InitStruct->TH41;
00167     MCPWM0.TH50 = MCPWM.InitStruct->TH50;
00168     MCPWM0.TH51 = MCPWM.InitStruct->TH51;
00169
00170     MCPWM0.FAIL012 = (MCPWM.InitStruct->FAIL0.Polarity << 2) | (MCPWM.InitStruct->FAIL1.Polarity << 3) |
(MCPWM.InitStruct->FAIL0.INPUT.EN << 4) | (MCPWM.InitStruct->FAIL1.INPUT.EN << 5) |
(MCPWM.InitStruct->HALT.PRT0 << 7) | (MCPWM.InitStruct->FAIL0.Signal.Sel << 8) |
(MCPWM.InitStruct->FAIL1.Signal.Sel << 10) | (MCPWM.InitStruct->FAIL0CAP << 15);
00171
00172     MCPWM0.FAIL345 = (MCPWM.InitStruct->FAIL2.Polarity << 2) | (MCPWM.InitStruct->FAIL3.Polarity << 3) |
(MCPWM.InitStruct->FAIL2.INPUT.EN << 4) | (MCPWM.InitStruct->FAIL3.INPUT.EN << 5) |
(MCPWM.InitStruct->HALT.PRT1 << 7) | (MCPWM.InitStruct->FAIL2.Signal.Sel << 8) |
(MCPWM.InitStruct->FAIL3.Signal.Sel << 10) | (MCPWM.InitStruct->FAIL1CAP << 15);
00173
00174     MCPWM0.CH_DEF = MCPWM.InitStruct->CH0N.default_output | (MCPWM.InitStruct->CH0P.default_output << 1)
| (MCPWM.InitStruct->CH1N.default_output << 2) | (MCPWM.InitStruct->CH1P.default_output << 3) |
(MCPWM.InitStruct->CH2N.default_output << 4) | (MCPWM.InitStruct->CH2P.default_output << 5) |
(MCPWM.InitStruct->CH3N.default_output << 6) | (MCPWM.InitStruct->CH3P.default_output << 7) |
(MCPWM.InitStruct->CH4N.default_output << 8) | (MCPWM.InitStruct->CH4P.default_output << 9) |
(MCPWM.InitStruct->CH5N.default_output << 10) | (MCPWM.InitStruct->CH5P.default_output << 11);
00175
00176     MCPWM0.CH_MASK = MCPWM.InitStruct->CH0N.FAIL.EN | (MCPWM.InitStruct->CH0P.FAIL.EN << 1) |
(MCPWM.InitStruct->CH1N.FAIL.EN << 2) | (MCPWM.InitStruct->CH1P.FAIL.EN << 3) |
(MCPWM.InitStruct->CH2N.FAIL.EN << 4) | (MCPWM.InitStruct->CH2P.FAIL.EN << 5) |
(MCPWM.InitStruct->CH3N.FAIL.EN << 6) | (MCPWM.InitStruct->CH3P.FAIL.EN << 7) |
(MCPWM.InitStruct->CH4N.FAIL.EN << 8) | (MCPWM.InitStruct->CH4P.FAIL.EN << 9) |
(MCPWM.InitStruct->CH5N.FAIL.EN << 10) | (MCPWM.InitStruct->CH5P.FAIL.EN << 11);
00177
00178     MCPWM0.UPDATE = 0xffff; /* write whatever value to trigger update */
00179
00180     MCPWM0.PRT = 0x0000CAFE; /* write any value other than 0xDEAD to enable write protection */
00181 }
00202 void PWMOutputs(FuncState t_state)
00203 {
00204     MCPWM0.PRT = 0xDEAD;
00205
00206     if (t_state == DISABLE)
00207     {
00208         MCPWM0.FAIL012 &= ~BIT6;
00209         MCPWM0.FAIL345 &= ~BIT6;
00210     }
00211     else
00212     {
00213         MCPWM0.FAIL012 |= BIT6;
00214         MCPWM0.FAIL345 |= BIT6;
00215     }
00216     MCPWM0.PRT = 0;
00217 }
00267 void MCPWM_ClearIRQFlag(MCPWM0CNTx CNTx, uint32_t INT_flag)
00268 {
00269     if (INT_flag <= BIT14)
00270     {
00271         if (CNTx == MCPWM0CNT0)
00272         {
00273             MCPWM0.IF0 = INT_flag;
00274         }
00275         else
00276         {
00277             MCPWM0.IF1 = INT_flag;
00278         }
00279     }
00280     else
00281     {
00282         MCPWM0.EIF = (INT_flag >> 15);
00283     }
00284 }
00333 u16 MCPWM_GetIRQFlag(MCPWM0CNTx CNTx, uint32_t INT_flag)
00334 {
00335     u8 MCPWMIRQFlag = 0;
00336     if (INT_flag <= BIT14)
00337     {
00338         if (CNTx == MCPWM0CNT0)
00339         {
00340             if (MCPWM0.IF0 & INT_flag & MCPWM0.IE0)
00341             {
00342                 MCPWMIRQFlag = 1;
00343             }
00344             else
00345             {
00346                 MCPWMIRQFlag = 0;
00347             }
00348         }
00349         else
00350         {

```

```

00351         if (MCPWM0_IF1 & INT.flag & MCPWM0_IE1)
00352         {
00353             MCPWMIRQFlag = 1;
00354         }
00355         else
00356         {
00357             MCPWMIRQFlag = 0;
00358         }
00359     }
00360 }
00361 else
00362 {
00363     if (MCPWM0{EIF & (INT.flag >> 15) & MCPWM0_EIE)
00364     {
00365         MCPWMIRQFlag = 1;
00366     }
00367     else
00368     {
00369         MCPWMIRQFlag = 0;
00370     }
00371 }
00372 return MCPWMIRQFlag;
00373 }
00374
00375 void MCPWM_Clk.Enable(u8 Clk.Enable)
00376 {
00377     MCPWM0_PRT = 0x0000DEAD; /*enter password to unlock write protection */
00378     if (Clk.Enable == 1)
00379     {
00380         MCPWM0_TCLK |= (Clk.Enable << 2);
00381     }
00382     else
00383     {
00384         MCPWM0_TCLK &= ~(1 << 2);
00385     }
00386     MCPWM0_PRT = 0x0000CAFE; /* write any value other than 0xDEAD to enable write protection */
00387 }
00388
00389 u8 ClockDivflg = 0;
00390 void MCPWM_DIV.Set(u8 ClockDiv) /*只可调用一次，多次调用第一次后续的不生效，以保证时基0和时基1分频相同*/
00391 {
00392     if (ClockDivflg == 0)
00393     {
00394         MCPWM0_PRT = 0x0000DEAD; /*enter password to unlock write protection */
00395         MCPWM0_TCLK |= ClockDiv;
00396         MCPWM0_PRT = 0x0000CAFE; /* write any value other than 0xDEAD to enable write protection */
00397         ClockDivflg = 1;
00398     }
00399     else
00400     {
00401     }
00402 }
00403
00404 void MCPWM_CNTx.Enable(MCPWM0CNTx CNTx, FuncState state)
00405 {
00406     MCPWM0_PRT = 0x0000DEAD; /*enter password to unlock write protection */
00407     if (CNTx)
00408     {
00409         if (state == 1)
00410         {
00411             MCPWM0_TCLK |= (state << 7);
00412         }
00413         else
00414         {
00415             MCPWM0_TCLK &= ~(1 << 7);
00416         }
00417     }
00418     else
00419     {
00420         if (state == 1)
00421         {
00422             MCPWM0_TCLK |= (state << 6);
00423         }
00424         else
00425         {
00426             MCPWM0_TCLK &= ~(1 << 6);
00427         }
00428     }
00429     MCPWM0_PRT = 0x0000CAFE; /* write any value other than 0xDEAD to enable write protection */
00430 }
00431
00432 void MCPWM_CH012.Init(MCPWM_InitTypeDef *MCPWM_InitStruct)
00433 {
00434     unsigned int reg_value0 = MCPWM0_TCLK;
00435     unsigned int reg_value1 = MCPWM0_SDCFG;
00436     unsigned int reg_value2 = MCPWM0_EIE;
00437     unsigned int reg_value3 = MCPWM0_RE;

```

```

00438     unsigned int reg_value4 = MCPWM0_PP;
00439     unsigned int reg_value5 = MCPWM0_IO23;
00440     unsigned int reg_value6 = MCPWM0_CH_DEF;
00441     unsigned int reg_value7 = MCPWM0_CH_MASK;
00442
00443     MCPWM0_PRT = 0x0000DEAD; /*enter password to unlock write protection */
00444
00445     reg_value0 &= ~(1 << 3) & ~(1 << 8) & ~(1 << 10); // 创建掩码
00446
00447     reg_value0 |= (MCPWM.InitStruct->ZCS_EN << 3) | (MCPWM.InitStruct->EVT_CNT0_EN << 8) |
(MCPWM.InitStruct->CMP_CTRL_CNT0 << 10);
00448     MCPWM0_TCLK = reg_value0;
00449
00450     MCPWM0_TMR0 = MCPWM.InitStruct->TMR0;
00451     MCPWM0_TMR1 = MCPWM.InitStruct->TMR1;
00452
00453     MCPWM0_TH0 = MCPWM.InitStruct->TH0;
00454
00455     reg_value1 &= ~(1) & ~(1 << 4) & ~(1 << 5) & ~(1 << 6); // 创建掩码
00456     reg_value1 |= MCPWM.InitStruct->TR0_UP_INTV | (MCPWM.InitStruct->TR0_T0_UpdateEN << 4) |
(MCPWM.InitStruct->TR0_T1_UpdateEN << 5) | (MCPWM.InitStruct->TR0_AEC << 6);
00457     MCPWM0_SDCFG = reg_value1;
00458
00459     MCPWM0_DTH00 = MCPWM.InitStruct->DeadTimeCH012N;
00460     MCPWM0_DTH01 = MCPWM.InitStruct->DeadTimeCH012P;
00461
00462     MCPWM0_EIF = 0xffff;
00463
00464     MCPWM0_FAIL012 = (MCPWM.InitStruct->FAIL0.Polarity << 2) | (MCPWM.InitStruct->FAIL1.Polarity << 3) |
(MCPWM.InitStruct->FAIL0.INPUT_EN << 4) | (MCPWM.InitStruct->FAIL1.INPUT_EN << 5) |
(MCPWM.InitStruct->HALT_PRT0 << 7) | (MCPWM.InitStruct->FAIL0.Signal_Sel << 8) |
(MCPWM.InitStruct->FAIL1.Signal_Sel << 10) | (MCPWM.InitStruct->FAIL0CAP << 15);
00465
00466     reg_value2 &= ~(1 << 4) & ~(1 << 5) & ~(1 << 8); // 创建掩码
00467     reg_value2 |= (MCPWM.InitStruct->FAIL0.INT_EN << 4) | (MCPWM.InitStruct->FAIL1.INT_EN << 5) |
(MCPWM.InitStruct->ZCS0.INT_EN << 8);
00468     MCPWM0_EIE = reg_value2;
00469
00470     MCPWM0_EVT0 = MCPWM.InitStruct->EVT0;
00471
00472     MCPWM0_FLT = MCPWM.InitStruct->IO_CMP_FLT_CLKDIV;
00473
00474     reg_value3 &= ~(1) & ~(1 << 1) & ~(1 << 4) & ~(1 << 5) & ~(1 << 8) & ~(1 << 9) & ~(1 << 12) & ~(1
<< 13); // 创建掩码
00475     reg_value3 |= MCPWM.InitStruct->TMR0_RE0 | (MCPWM.InitStruct->TMR1_RE0 << 1) |
(MCPWM.InitStruct->TR0_T0_RE0 << 4) | (MCPWM.InitStruct->TR0_T1_RE0 << 5) | (MCPWM.InitStruct->TMR0_RE1
<< 8) | (MCPWM.InitStruct->TMR1_RE1 << 9) | (MCPWM.InitStruct->TR0_T0_RE1 << 12) |
(MCPWM.InitStruct->TR0_T1_RE1 << 13);
00476     MCPWM0_RE = reg_value3;
00477
00478     reg_value4 &= ~(1) & ~(1 << 1) & ~(1 << 2); // 创建掩码
00479     reg_value4 |= MCPWM.InitStruct->IO0_PPE | (MCPWM.InitStruct->IO1_PPE << 1) |
(MCPWM.InitStruct->IO2_PPE << 2);
00480     MCPWM0_PP = reg_value4;
00481
00482     MCPWM0_IO01 = MCPWM.InitStruct->CH0N.Polarity_INV | (MCPWM.InitStruct->CH0P.Polarity_INV << 1) |
(MCPWM.InitStruct->CH0_NS << 2) | (MCPWM.InitStruct->CH0_PS << 3) | (MCPWM.InitStruct->CH0N.SCTRLN << 4)
| (MCPWM.InitStruct->CH0P.SCTRLP << 5) | (MCPWM.InitStruct->Switch.CH0N.CH0P << 6) |
(MCPWM.InitStruct->MCPWM.WorkModeCH0 << 7) | (MCPWM.InitStruct->CH1N.Polarity_INV << 8) |
(MCPWM.InitStruct->CH1P.Polarity_INV << 9) | (MCPWM.InitStruct->CH1_NS << 10) | (MCPWM.InitStruct->CH1_PS
<< 11) | (MCPWM.InitStruct->CH1N.SCTRLN << 12) | (MCPWM.InitStruct->CH1P.SCTRLP << 13) |
(MCPWM.InitStruct->Switch.CH1N.CH1P << 14) | (MCPWM.InitStruct->MCPWM.WorkModeCH1 << 15);
00483
00484     reg_value5 &= ~(1) & ~(1 << 1) & ~(1 << 2) & ~(1 << 3) & ~(1 << 4) & ~(1 << 5) & ~(1 << 6) & ~(1
<< 7); // 创建掩码
00485     reg_value5 |= MCPWM.InitStruct->CH2N.Polarity_INV | (MCPWM.InitStruct->CH2P.Polarity_INV << 1) |
(MCPWM.InitStruct->CH2_NS << 2) | (MCPWM.InitStruct->CH2_PS << 3) | (MCPWM.InitStruct->CH2N.SCTRLN << 4)
| (MCPWM.InitStruct->CH2P.SCTRLP << 5) | (MCPWM.InitStruct->Switch.CH2N.CH2P << 6) |
(MCPWM.InitStruct->MCPWM.WorkModeCH2 << 7);
00486     MCPWM0_IO23 = reg_value5;
00487
00488     MCPWM0_TH00 = MCPWM.InitStruct->TH00;
00489     MCPWM0_TH01 = MCPWM.InitStruct->TH01;
00490     MCPWM0_TH10 = MCPWM.InitStruct->TH10;
00491     MCPWM0_TH11 = MCPWM.InitStruct->TH11;
00492     MCPWM0_TH20 = MCPWM.InitStruct->TH20;
00493     MCPWM0_TH21 = MCPWM.InitStruct->TH21;
00494
00495     reg_value6 &= ~(1) & ~(1 << 1) & ~(1 << 2) & ~(1 << 3) & ~(1 << 4) & ~(1 << 5); // 创建掩码
00496     reg_value6 |= MCPWM.InitStruct->CH0N.default.output | (MCPWM.InitStruct->CH0P.default.output << 1) |
(MCPWM.InitStruct->CH1N.default.output << 2) | (MCPWM.InitStruct->CH1P.default.output << 3) |
(MCPWM.InitStruct->CH2N.default.output << 4) | (MCPWM.InitStruct->CH2P.default.output << 5);
00497     MCPWM0_CH_DEF = reg_value6;
00498
00499     reg_value7 &= ~(1) & ~(1 << 1) & ~(1 << 2) & ~(1 << 3) & ~(1 << 4) & ~(1 << 5); // 创建掩码
00500     reg_value7 |= MCPWM.InitStruct->CH0N.FAIL_EN | (MCPWM.InitStruct->CH0P.FAIL_EN << 1) |
(MCPWM.InitStruct->CH1N.FAIL_EN << 2) | (MCPWM.InitStruct->CH1P.FAIL_EN << 3) |

```

```

(MCPWM.InitStruct->CH2N.FAIL_EN << 4) | (MCPWM.InitStruct->CH2P.FAIL_EN << 5);
00501     MCPWM0.CH_MASK = reg.value7;
00502
00503     MCPWM0.UPDATE = 0xffff; /* write whatever value to trigger update */
00504
00505     MCPWM0.PRT = 0x0000CAFE; /* write any value other than 0xDEAD to enable write protection */
00506 }
00507
00508 void MCPWM.CH345.Init(MCPWM_InitTypeDef *MCPWM_InitStruct)
00509 {
00510     unsigned int reg.value0 = MCPWM0.TCLK;
00511     unsigned int reg.value1 = MCPWM0.SDCFG;
00512     unsigned int reg.value2 = MCPWM0.EIE;
00513     unsigned int reg.value3 = MCPWM0.RE;
00514     unsigned int reg.value4 = MCPWM0.PP;
00515     unsigned int reg.value5 = MCPWM0.IO23;
00516     unsigned int reg.value6 = MCPWM0.CH.DEF;
00517     unsigned int reg.value7 = MCPWM0.CH_MASK;
00518
00519     MCPWM0.PRT = 0x0000DEAD; /*enter password to unlock write protection */
00520
00521     reg.value0 &= ~(1 << 3) & ~(1 << 4) & ~(1 << 5) & ~(1 << 9) & ~(1 << 11); // 创建掩码
00522     reg.value0 |= (MCPWM.InitStruct->ZCS_EN << 3) | (MCPWM.InitStruct->TMR2.TB << 4) |
(MCPWM.InitStruct->TMR3.TB << 5) | (MCPWM.InitStruct->EVT.CNT1_EN << 9) |
(MCPWM.InitStruct->CMP_CTRL.CNT1 << 11);
00523     MCPWM0.TCLK = reg.value0;
00524
00525     MCPWM0.TMR2 = MCPWM.InitStruct->TMR2;
00526     MCPWM0.TMR3 = MCPWM.InitStruct->TMR3;
00527
00528     MCPWM0.TH1 = MCPWM.InitStruct->TH1;
00529
00530     reg.value1 &= ~(1 << 8) & ~(1 << 12) & ~(1 << 13) & ~(1 << 14); // 创建掩码
00531     reg.value1 |= (MCPWM.InitStruct->TR1.UP_INTV << 8) | (MCPWM.InitStruct->TR1.T0.UpdateEN << 12) |
(MCPWM.InitStruct->TR1.T1.UpdateEN << 13) | (MCPWM.InitStruct->TR1.AEC << 14);
00532     MCPWM0.SDCFG = reg.value1;
00533
00534     MCPWM0.DTH10 = MCPWM.InitStruct->DeadTimeCH345N;
00535     MCPWM0.DTH11 = MCPWM.InitStruct->DeadTimeCH345P;
00536
00537     MCPWM0.EIF = 0xffff;
00538
00539     MCPWM0.FAIL345 = (MCPWM.InitStruct->FAIL2.Polarity << 2) | (MCPWM.InitStruct->FAIL3.Polarity << 3) |
(MCPWM.InitStruct->FAIL2.INPUT_EN << 4) | (MCPWM.InitStruct->FAIL3.INPUT_EN << 5) |
(MCPWM.InitStruct->HALT_PRT1 << 7) | (MCPWM.InitStruct->FAIL2.Signal_Sel << 8) |
(MCPWM.InitStruct->FAIL3.Signal_Sel << 10) | (MCPWM.InitStruct->FAIL1.CAP << 15);
00540
00541     reg.value2 &= ~(1 << 6) & ~(1 << 7) & ~(1 << 9); // 创建掩码
00542     reg.value2 |= (MCPWM.InitStruct->FAIL2.INT_EN << 6) | (MCPWM.InitStruct->FAIL3.INT_EN << 7) |
(MCPWM.InitStruct->ZCS1.INT_EN << 9);
00543     MCPWM0.EIE = reg.value2;
00544
00545     MCPWM0.EVT1 = MCPWM.InitStruct->EVT1;
00546
00547     MCPWM0.FLT = MCPWM.InitStruct->IO_CMP.FLT.CLKDIV;
00548
00549     reg.value3 &= ~(1 << 2) & ~(1 << 3) & ~(1 << 6) & ~(1 << 7) & ~(1 << 10) & ~(1 << 11) & ~(1 << 14)
& ~(1 << 15); // 创建掩码
00550     reg.value3 |= (MCPWM.InitStruct->TMR2.RE0 << 2) | (MCPWM.InitStruct->TMR3.RE0 << 3) |
(MCPWM.InitStruct->TR1.T0.RE0 << 6) | (MCPWM.InitStruct->TR1.T1.RE0 << 7) | (MCPWM.InitStruct->TMR2.RE1
<< 10) | (MCPWM.InitStruct->TMR3.RE1 << 11) | (MCPWM.InitStruct->TR1.T0.RE1 << 14) |
(MCPWM.InitStruct->TR1.T1.RE1 << 15);
00551     MCPWM0.RE = reg.value3;
00552
00553     reg.value4 &= ~(1 << 3) & ~(1 << 4) & ~(1 << 5); // 创建掩码
00554     reg.value4 |= (MCPWM.InitStruct->IO3.PPE << 3) | (MCPWM.InitStruct->IO4.PPE << 4) |
(MCPWM.InitStruct->IO5.PPE << 5);
00555     MCPWM0.PP = reg.value4;
00556
00557     reg.value5 &= ~(1 << 8) & ~(1 << 9) & ~(1 << 10) & ~(1 << 11) & ~(1 << 12) & ~(1 << 13) & ~(1 <<
14) & ~(1 << 15); // 创建掩码
00558     reg.value5 |= (MCPWM.InitStruct->CH3N.Polarity_INV << 8) | (MCPWM.InitStruct->CH3P.Polarity_INV << 9)
| (MCPWM.InitStruct->CH3.NS << 10) | (MCPWM.InitStruct->CH3.PS << 11) | (MCPWM.InitStruct->CH3N.SCTRLN
<< 12) | (MCPWM.InitStruct->CH3P.SCTRLP << 13) | (MCPWM.InitStruct->Switch.CH3N.CH3P << 14) |
(MCPWM.InitStruct->MCPWM.WorkModeCH3 << 15);
00559     MCPWM0.IO23 = reg.value5;
00560
00561     MCPWM0.IO45 = MCPWM.InitStruct->CH4N.Polarity_INV | (MCPWM.InitStruct->CH4P.Polarity_INV << 1) |
(MCPWM.InitStruct->CH4.NS << 2) | (MCPWM.InitStruct->CH4.PS << 3) | (MCPWM.InitStruct->CH4N.SCTRLN << 4)
| (MCPWM.InitStruct->CH4P.SCTRLP << 5) | (MCPWM.InitStruct->Switch.CH4N.CH4P << 6) |
(MCPWM.InitStruct->MCPWM.WorkModeCH4 << 7) | (MCPWM.InitStruct->CH5N.Polarity_INV << 8) |
(MCPWM.InitStruct->CH5P.Polarity_INV << 9) | (MCPWM.InitStruct->CH5.NS << 10) | (MCPWM.InitStruct->CH5.PS
<< 11) | (MCPWM.InitStruct->CH5N.SCTRLN << 12) | (MCPWM.InitStruct->CH5P.SCTRLP << 13) |
(MCPWM.InitStruct->Switch.CH5N.CH5P << 14) | (MCPWM.InitStruct->MCPWM.WorkModeCH5 << 15);
00562
00563     MCPWM0.TH30 = MCPWM.InitStruct->TH30;
00564     MCPWM0.TH31 = MCPWM.InitStruct->TH31;

```

```

00565 MCPWM0.TH40 = MCPWM.InitStruct->TH40;
00566 MCPWM0.TH41 = MCPWM.InitStruct->TH41;
00567 MCPWM0.TH50 = MCPWM.InitStruct->TH50;
00568 MCPWM0.TH51 = MCPWM.InitStruct->TH51;
00569
00570 reg_value6 &= ~(1 << 6) & ~(1 << 7) & ~(1 << 8) & ~(1 << 9) & ~(1 << 10) & ~(1 << 11); // 创建掩码
00571 reg_value6 |= (MCPWM.InitStruct->CH3N.default_output << 6) | (MCPWM.InitStruct->CH3P.default_output
<< 7) | (MCPWM.InitStruct->CH4N.default_output << 8) | (MCPWM.InitStruct->CH4P.default_output << 9) |
(MCPWM.InitStruct->CH5N.default_output << 10) | (MCPWM.InitStruct->CH5P.default_output << 11);
00572 MCPWM0.CH.DEF = reg_value6;
00573
00574 reg_value7 &= ~(1 << 6) & ~(1 << 7) & ~(1 << 8) & ~(1 << 9) & ~(1 << 10) & ~(1 << 11); // 创建掩码
00575 reg_value7 |= (MCPWM.InitStruct->CH3N.FAIL_EN << 6) | (MCPWM.InitStruct->CH3P.FAIL_EN << 7) |
(MCPWM.InitStruct->CH4N.FAIL_EN << 8) | (MCPWM.InitStruct->CH4P.FAIL_EN << 9) |
(MCPWM.InitStruct->CH5N.FAIL_EN << 10) | (MCPWM.InitStruct->CH5P.FAIL_EN << 11);
00576 MCPWM0.CH.MASK = reg_value7;
00577
00578 MCPWM0.UPDATE = 0xffff; /* write whatever value to trigger update */
00579
00580 MCPWM0.PRT = 0x0000CAFE; /* write any value other than 0xDEAD to enable write protection */
00581 }
00582
00583 void MCPWM.CH012.IESet(MCPWM_InitTypeDef *MCPWM_InitStruct)
00584 {
00585     MCPWM0.PRT = 0x0000DEAD; /*enter password to unlock write protection */
00586     MCPWM0.IE0 = MCPWM.InitStruct->T0.Update0.INT_EN | (MCPWM.InitStruct->T1.Update0.INT_EN << 1) |
(MCPWM.InitStruct->TH00.Match0.INT_EN << 2) | (MCPWM.InitStruct->TH01.Match0.INT_EN << 3) |
(MCPWM.InitStruct->TH10.Match0.INT_EN << 4) | (MCPWM.InitStruct->TH11.Match0.INT_EN << 5) |
(MCPWM.InitStruct->TH20.Match0.INT_EN << 6) | (MCPWM.InitStruct->TH21.Match0.INT_EN << 7) |
(MCPWM.InitStruct->TMR0.Match0.INT_EN << 10) | (MCPWM.InitStruct->TMR1.Match0.INT_EN << 11) |
(MCPWM.InitStruct->TMR2.Match0.INT_EN << 12) | (MCPWM.InitStruct->TMR3.Match0.INT_EN << 13) |
(MCPWM.InitStruct->Update0.INT_EN << 14);
00587     MCPWM0.PRT = 0x0000CAFE; /* write any value other than 0xDEAD to enable write protection */
00588 }
00589
00590 void MCPWM.CH345.IESet(MCPWM_InitTypeDef *MCPWM_InitStruct)
00591 {
00592     MCPWM0.PRT = 0x0000DEAD; /*enter password to unlock write protection */
00593     MCPWM0.IE1 = MCPWM.InitStruct->T0.Update1.INT_EN | (MCPWM.InitStruct->T1.Update1.INT_EN << 1) |
(MCPWM.InitStruct->TH00.Match1.INT_EN << 2) | (MCPWM.InitStruct->TH01.Match1.INT_EN << 3) |
(MCPWM.InitStruct->TH10.Match1.INT_EN << 4) | (MCPWM.InitStruct->TH11.Match1.INT_EN << 5) |
(MCPWM.InitStruct->TH20.Match1.INT_EN << 6) | (MCPWM.InitStruct->TH21.Match1.INT_EN << 7) |
(MCPWM.InitStruct->TMR0.Match1.INT_EN << 10) | (MCPWM.InitStruct->TMR1.Match1.INT_EN << 11) |
(MCPWM.InitStruct->TMR2.Match1.INT_EN << 12) | (MCPWM.InitStruct->TMR3.Match1.INT_EN << 13) |
(MCPWM.InitStruct->Update1.INT_EN << 14);
00594     MCPWM0.PRT = 0x0000CAFE; /* write any value other than 0xDEAD to enable write protection */
00595 }
00596
00597 void MCPWM.ModuleClockCmd(FuncState state)
00598 {
00599     if (state)
00600     {
00601         SYS.ModuleClockCmd(SYS.Module_MCPWM0, ENABLE); // MCPWM时钟使能
00602     }
00603     else
00604     {
00605         SYS.ModuleClockCmd(SYS.Module_MCPWM0, DISABLE); // MCPWM时钟取消使能
00606     }
00607 }
00608 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.79 source/lks32mc07x_opa.c 文件参考

文件名称: lks32mc07x_opa.c
文件标识: 无
内容摘要: OPA运放外设驱动程序
其它说明: 无

#include "lks32mc07x_opa.h"
lks32mc07x_opa.c 的引用(Include)关系图:



函数

- void **OPA_Init** (enumOPA opa, OPA_InitTypeDef *OPA_InitStruct)
函数名称: void **OPA_Init**(enumOPA opa, OPA_InitTypeDef* OPA_InitStruct)
- void **OPA_StructInit** (OPA_InitTypeDef *OPA_InitStruct)
函数名称: void **OPA_StructInit**(OPA_InitTypeDef* OPA_InitStruct)
- void **OPA_OUT** (enumOPA opa, FuncState state)
函数名称: void **OPA_OUT**(enumOPA opa, FuncState state)

变量

- u8 **maxmagflg**

5.79.1 详细描述

文件名称: **lks32mc07x_opa.c**
文件标识: 无
内容摘要: **OPA**运放外设驱动程序
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 **lks32mc07x_opa.c** 中定义.

5.79.2 函数说明

5.79.2.1 OPA_Init()

```
void OPA_Init (  
    enumOPA opa,  
    OPA_InitTypeDef * OPA_InitStruct )
```

函数名称: void **OPA_Init**(enumOPA opa, OPA_InitTypeDef* OPA_InitStruct)

功能描述: **OPA**初始化函数

参见

被引用内容: 无

参数

输入参数:	enumOPA opa , OPA_InitTypeDef *OPA_InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
OPA_InitTypeDef OPA_InitStruct;  
OPA_StructInit (&OPA_InitStruct);  
OPA_InitStruct.OPA_IT = PGA_IT_DISABLE; //opa偏置电流调节禁止  
OPA_InitStruct.OPA_CLEna = ENABLE; //使能  
OPA_InitStruct.OPA_Gain = PGA_GAIN_4; //增益40k:10k  
OPA_Init(OPA2 ,&OPA_InitStruct);
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_opa.c](#) 第 41 行定义.

5.79.2.2 OPA_OUT()

```
void OPA_OUT (  
    enumOPA opa,  
    FuncState state )
```

函数名称: void [OPA_OUT](#)(enumOPA opa, FuncState state)

功能描述: OPA外部输出，输出到外部P2.7引脚

参见

被引用内容: 无

参数

输入参数:	enumOPA opa, FuncState state
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
OPA_OUT(OPA0, ENABLE); //将OPA0输出通过外部P2.7引脚送出
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_opa.c](#) 第 162 行定义.

5.79.2.3 OPA_StructInit()

```
void OPA_StructInit (  
    OPA_InitTypeDef * OPA_InitStruct )
```

函数名称: void [OPA_StructInit\(OPA_InitTypeDef* OPA_InitStruct\)](#)

功能描述: OPA结构体初始化

参见

被引用内容: 无

参数

输入参数:	OPA_InitTypeDef *OPA_InitStruct
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
OPA_InitTypeDef OPA_InitStructure;  
OPA_StructInit (&OPA_InitStructure);           //初始化结构体
```

Date	Version	Author	Description
2023年02月22日	1.0	Olive Wang	创建

修改日志:

在文件 [lks32mc07x_opa.c](#) 第 137 行定义.

5.79.3 变量说明

5.79.3.1 maxmagflg

u8 maxmagflg

在文件 [lks32mc07x_opa.c](#) 第 15 行定义.

5.80 lks32mc07x_opa.c

[浏览该文件的文档.](#)

```
00001  
00014 #include "lks32mc07x_opa.h"  
00015 u8 maxmagflg;  
00016  
00041 void OPA_Init(enumOPA opa, OPA_InitTypeDef *OPA_InitStructure)  
00042 {  
00043     uint32_t tmp1, tmp2;  
00044     SYS_WR_PROTECT = 0x7a83; /* 解除系统寄存器写保护 */  
00045     tmp1           = SYS_AFE_REG0;  
00046     tmp2           = SYS_AFE_REG5;  
00047  
00048     if (opa == OPA0)  
00049     {  
00050         if (OPA_InitStructure->OPA_Gain == PGA_GAIN_32)  
00051         {  
00052             maxmagflg = 1;
```

```

00053     }
00054
00055     tmp1 &= ~(BIT0 | BIT1);
00056     tmp1 |= OPA_InitStructure->OPA_Gain;
00057
00058     tmp2 &= ~BIT2;
00059     tmp2 |= (OPA_InitStructure->OPA_CLEna << 2);
00060 }
00061 else if (opa == OPA1)
00062 {
00063     if (OPA_InitStructure->OPA_Gain == PGA_GAIN_32)
00064     {
00065         maxmagflg = 1;
00066     }
00067
00068     tmp1 &= ~(BIT2 | BIT3);
00069     tmp1 |= (OPA_InitStructure->OPA_Gain << 2);
00070
00071     tmp2 &= ~BIT3;
00072     tmp2 |= (OPA_InitStructure->OPA_CLEna << 3);
00073 }
00074 else if (opa == OPA2)
00075 {
00076     if (OPA_InitStructure->OPA_Gain == PGA_GAIN_32)
00077     {
00078         maxmagflg = 1;
00079     }
00080
00081     tmp1 &= ~(BIT4 | BIT5);
00082     tmp1 |= (OPA_InitStructure->OPA_Gain << 4);
00083
00084     tmp2 &= ~BIT4;
00085     tmp2 |= (OPA_InitStructure->OPA_CLEna << 4);
00086 }
00087 else if (opa == OPA3)
00088 {
00089     if (OPA_InitStructure->OPA_Gain == PGA_GAIN_32)
00090     {
00091         maxmagflg = 1;
00092     }
00093
00094     tmp1 &= ~(BIT6 | BIT7);
00095     tmp1 |= (OPA_InitStructure->OPA_Gain << 6);
00096
00097     tmp2 &= ~BIT5;
00098     tmp2 |= (OPA_InitStructure->OPA_CLEna << 5);
00099 }
00100
00101 tmp1 &= ~(BIT8 | BIT9);
00102 if (maxmagflg == 1)
00103 {
00104     OPA_InitStructure->OPA_IT = PGA_IT_1P5;
00105 }
00106 else
00107 {
00108     OPA_InitStructure->OPA_IT = PGA_IT_1;
00109 }
00110 tmp1 |= (OPA_InitStructure->OPA_IT << 8);
00111
00112 SYS_AFE_REG0 = tmp1;
00113 SYS_AFE_REG5 = tmp2;
00114 SYS_WR_PROTECT = 0x0; /* 开启系统寄存器写保护 */
00115 }
00116
00137 void OPA_StructInit(OPA_InitTypeDef *OPA_InitStructure)
00138 {
00139     OPA_InitStructure->OPA_IT = PGA_IT_DISABLE;
00140     OPA_InitStructure->OPA_Gain = PGA_GAIN_4;
00141     OPA_InitStructure->OPA_CLEna = DISABLE;
00142 }
00162 void OPA_OUT(enumOPA opa, FuncState state)
00163 {
00164     SYS_WR_PROTECT = 0x7a83; /* 解除系统寄存器写保护 */
00165     if (state != DISABLE)
00166     {
00167         if (opa == OPA0)
00168         {
00169             SYS_AFE_REG2 = 0x01;
00170         }
00171         else if (opa == OPA1)
00172         {
00173             SYS_AFE_REG2 = 0x02;
00174         }
00175         else if (opa == OPA2)
00176         {
00177             SYS_AFE_REG2 = 0x03;
00178         }
00179     }

```

```

00179         else if (opa == OPA3)
00180         {
00181             SYS_AFE_REG2 = 0x04;
00182         }
00183     }
00184     else
00185     {
00186         SYS_AFE_REG2 = 0x00;
00187     }
00188     SYS_WR_PROTECT = 0x0; /* 开启系统寄存器写保护 */
00189 }
00190
00191 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.81 source/lks32mc07x_qep.c 文件参考

文件名称: [lks32mc07x_qep.c](#)
 文件标识: 无
 内容摘要: 编码器外设驱动程序
 其它说明: 无

#include "lks32mc07x_lib.h"
 lks32mc07x_qep.c 的引用(Include)关系图:



函数

- void [TIM_QEPInit](#) (TIM_QEPTypedef *QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct)
函数名称: void [TIM_QEPInit\(TIM_QEPTypedef *QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct\)](#)
- void [TIM_QEPStructInit](#) (TIM_QEPInitTypeDef *TIM_QEPInitStruct)
函数名称: void [TIM_QEPStructInit\(TIM_QEPInitTypeDef *TIM_QEPInitStruct\)](#)
- void [TIM_QEPCmd](#) (TIM_QEPTypedef *QEPx, FuncState state)
函数名称: void [TIM_QEPCmd\(TIM_QEPTypedef *QEPx, FuncState state\)](#)
- uint16_t [TIM_QEP_GetCount](#) (TIM_QEPTypedef *QEPx)
函数名称: uint16_t [TIM_QEP_GetCount\(TIM_QEPTypedef *QEPx\)](#)
- void [QEP_ClearIRQFlag](#) (TIM_QEPTypedef *QEPx, uint32_t tempFlag)
函数名称: void [QEP_ClearIRQFlag\(TIM_QEPTypedef *QEPx, uint32_t tempFlag\)](#)
- u16 [QEP_GetIRQFlag](#) (TIM_QEPTypedef *QEPx, u32 QEP_IF_x)
函数名称: u16 [QEP_GetIRQFlag\(TIM_QEPTypedef *QEPx, u32 QEP_IF_x\)](#)
- u16 [QEP_GetFlag](#) (TIM_QEPTypedef *QEPx, u32 QEP_IF_x)
函数名称: u16 [QEP_GetFlag\(TIM_QEPTypedef *QEPx, u32 QEP_IF_x\)](#)

5.81.1 详细描述

文件名称: [lks32mc07x_qep.c](#)
 文件标识: 无
 内容摘要: 编码器外设驱动程序
 其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_qep.c](#) 中定义.

5.81.2 函数说明

5.81.2.1 QEP_ClearIRQFlag()

```
void QEP_ClearIRQFlag (
    TIM_QEPTypeDef * QEPx,
    uint32_t tempFlag )
```

函数名称: void [QEP_ClearIRQFlag\(TIM_QEPTypeDef *QEPx,uint32_t tempFlag\)](#)

功能描述: 清除QEP中断标志

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1	
	tempFlag:	
	QEP_IF_UNDERFLOW QEP_IF_OVERFLOW	Encoder下溢中断 Encoder上溢中断
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
    if (QEP_GetIRQFlag (QEP0,QEP_IF_OVERFLOW)) //获取编码器上溢中断标志
{
    QEP_ClearIRQFlag (QEP0,QEP_IF_OVERFLOW); //清除编码器上溢中断标志
}
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x-qep.c](#) 第 216 行定义.

5.81.2.2 QEP_GetFlag()

```
u16 QEP_GetFlag (
    TIM_QEPTypeDef * QEPx,
    u32 QEP_IF_x )
```

函数名称: u16 [QEP_GetFlag\(TIM_QEPTypeDef *QEPx, u32 QEP_IF_x\)](#)

功能描述: 获取QEP中断标志

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1	
	QEP_IF_x:	
	QEP_IF_UNDERFLOW	Encoder下溢中断
	QEP_IF_OVERFLOW	Encoder上溢中断
输出参数:	无	

返回

返回值: 无

注解

其它说明: 无

警告

不需要对应中断使能，也可以正常读取中断标志位

示例代码:

```
{
    if(QEP_GetFlag(QEP0,QEP_IF_OVERFLOW)) //获取上溢中断标志
}
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 283 行定义.

5.81.2.3 QEP_GetIRQFlag()

```
u16 QEP_GetIRQFlag (
    TIM_QEPTypedef * QEPx,
    u32 QEP_IF_x )
```

函数名称: [u16 QEP_GetIRQFlag\(TIM_QEPTypedef *QEPx, u32 QEP_IF_x\)](#)

功能描述: 获取QEP中断标志

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1	
	QEP_IF_x:	
	QEP_IF_UNDERFLOW QEP_IF_OVERFLOW	Encoder下溢中断 Encoder上溢中断
输出参数:	无	

返回

返回值: 无

注解

其它说明: 无

警告

只有对应中断使能后，改为才能读取，如果对应中断未使能，读取结果一直为0

示例代码:

```
        if(QEP_GetIRQFlag(QEP0,QEP_IF_OVERFLOW)) //获取上溢中断标志
    {
    }
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 [246](#) 行定义.

5.81.2.4 TIM_QEP_GetCount()

```
uint16_t TIM_QEP_GetCount (
    TIM_QEPTypedef * QEPx )
```

函数名称: uint16_t [TIM_QEP_GetCount\(TIM_QEPTypedef *QEPx\)](#)

功能描述: 编码器计数值

参见

被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告
无

示例代码:

```
uint32_t QEP0.Value = 0;
QEP0.Value = TIM_QEP_GetCount(QEP0); //获取编码器0计数值
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 185 行定义.

5.81.2.5 TIM_QEPCmd()

```
void TIM_QEPCmd (
    TIM_QEPTTypeDef * QEPx,
    FuncState state )
```

函数名称: void [TIM_QEPCmd\(TIM_QEPTTypeDef *QEPx, FuncState state\)](#)

功能描述: 编码器QEPx使能和停止

参见
被引用内容: 无

参数

输入参数:	QEPx: QEP0/QEP1 FuncState state: 使能与失能
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
TIM_QEPCmd(ECD0, ENABLE); // ECD0模块使能
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 153 行定义.

5.81.2.6 TIM_QEPInit()

```
void TIM_QEPInit (
    TIM_QEPTypedef * QEPx,
    TIM_QEPInitTypeDef * TIM_QEPInitStruct )
```

函数名称: void [TIM_QEPInit\(TIM_QEPTypedef *QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct\)](#)

功能描述: 编码器初始化函数

参见
被引用内容: [SYS_ModuleClockCmd\(\)](#);

参数

输入参数:	QEPx QEP0/QEP1 TIM_QEPInitTypeDef *TIM_QEPInitStruct
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
TIM_QEPInitTypeDef TIM_QEPInitStruct;
TIM_QEPStructInit(&TIM_QEPInitStruct);
TIM_QEPInitStruct.QEP_FE_CNT_EN = DISABLE;
TIM_QEPInitStruct.QEP_Mode = QEP_Mode_T1;
TIM_QEPInitStruct.QEP_ClockDiv = QEP_CLK_DIV1;
TIM_QEPInitStruct.QEP_Filter = 0;
TIM_QEPInitStruct.QEP_TH = 65535;
TIM_QEPInitStruct.QEP_ZEC = ENABLE;
TIM_QEPInitStruct.QEP_ZLC = DISABLE;
TIM_QEPInitStruct.QEP_ZNC = DISABLE;
TIM_QEPInitStruct.QEP_ZPC = ENABLE;
TIM_QEPInitStruct.QEP_IRQEna = DISABLE;
TIM_QEPInit(QEP0, &TIM_QEPInitStruct);
TIM_QEPCmd(QEP0, ENABLE);

// Timer结构体初始化
// 使能下降沿进行计数, ENABLE:使能 DISABLE:失能
// 编码器模式选择
// 编码器时钟分频
// 输入信号滤波系数
// 编码器计数周期
// Z 信号清零极性选择: 低电平/下降沿清零使能
// Z 信号清零极性选择: 高电平/下降沿清零使能
// Z 信号电平清零 QEP 计数器使能
// Z 信号边沿清零 QEP 计数器使能
// 中断使能
// QEP0模块使能
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 lks32mc07x_qep.c 第 63 行定义.

函数调用图:



5.81.2.7 TIM_QEPStructInit()

```
void TIM_QEPStructInit (
    TIM_QEPInitTypeDef * TIM_QEPInitStruct )
```

函数名称: void TIM_QEPStructInit(TIM_QEPInitTypeDef *TIM_QEPInitStruct)

功能描述: 编码器结构体初始化

参见

被引用内容: 无

参数

输入参数:	TIM_QEPInitTypeDef *TIM_QEPInitStruct
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
TIM_QEPInitTypeDef TIM_QEPInitStruct;  
TIM_QEPStructInit (&TIM_QEPInitStruct); //编码器结构体初始化
```

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	修改

修改日志:

在文件 [lks32mc07x_qep.c](#) 第 119 行定义.

5.82 lks32mc07x_qep.c

[浏览该文件的文档.](#)

```
00001  
00015 #include "lks32mc07x.lib.h"  
00016  
00017 /*****  
00018 函数名称: void TIM_QEPInit(TIM_QEPTypeDef *TIM_QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct)  
00019 功能描述: QEP初始化  
00020 操作的表: 无  
00021 输入参数: 无  
00022 输出参数: 无  
00023 返回值: 无  
00024 其它说明:  
00025 修改日期      版本号      修改人      修改内容  
00026 -----  
00027 2016/06/06      V1.0      cfwu      创建  
00028 *****/  
00029  
00063 void TIM_QEPInit(TIM_QEPTypeDef *QEPx, TIM_QEPInitTypeDef *TIM_QEPInitStruct)  
00064 {  
00065     uint32_t tempTimerCfg;  
00066  
00067     QEPx->CFG = TIM_QEPInitStruct->QEP_Mode << 8;  
00068     QEPx->TH  = TIM_QEPInitStruct->QEP_TH;  
00069  
00070     if (QEPx == QEP0)  
00071     {  
00072         SYS.ModuleClockCmd(SYS.Module_TIMER2, ENABLE);  
00073         SYS.ModuleClockCmd(SYS.Module_QEP0, ENABLE);  
00074  
00075         UTIMER2_FLT = TIM_QEPInitStruct->QEP_Filter & 0x000F; /*滤波参数只使用低四位*/  
00076         tempTimerCfg = UTIMER2_CFG;  
00077         tempTimerCfg &= ~0x700000;
```

```

00078     tempTimerCfg |= (TIM_QEPInitStruct->QEP_ClockDiv << 20);
00079     UTIMER2.CFG = tempTimerCfg;
00080 }
00081 else if (QEPx == QEP1)
00082 {
00083     SYS_ModuleClockCmd(SYS_Module.TIMER3, ENABLE);
00084     SYS_ModuleClockCmd(SYS_Module.QEP1, ENABLE);
00085
00086     UTIMER3.FLT = TIM_QEPInitStruct->QEP_Filter & 0x000F; /*滤波参数只使用低四位*/
00087     tempTimerCfg = UTIMER3.CFG;
00088     tempTimerCfg &= ~0x700000;
00089     tempTimerCfg |= (TIM_QEPInitStruct->QEP_ClockDiv << 20);
00090     UTIMER3.CFG = tempTimerCfg;
00091 }
00092
00093 QEPx->CFG = (TIM_QEPInitStruct->QEP_FE_CNT_EN) << 10 | (TIM_QEPInitStruct->QEP_Mode) << 8 |
(TIM_QEPInitStruct->QEP_ZNC) << 3 | (TIM_QEPInitStruct->QEP_ZPC) << 2 | (TIM_QEPInitStruct->QEP_ZLC) <<
1 | (TIM_QEPInitStruct->QEP_ZEC);
00094 QEPx->TH = TIM_QEPInitStruct->QEP_TH;
00095
00096 QEPx->IE = TIM_QEPInitStruct->QEP_IRQEna;
00097 }
00098
00119 void TIM_QEPStructInit(TIM_QEPInitTypeDef *TIM_QEPInitStruct)
00120 {
00121     TIM_QEPInitStruct->QEP_Mode = QEP_Mode_T1;
00122     TIM_QEPInitStruct->QEP_ClockDiv = 0;
00123     TIM_QEPInitStruct->QEP_FE_CNT_EN = 0;
00124     TIM_QEPInitStruct->QEP_TH = 0;
00125     TIM_QEPInitStruct->QEP_ZEC = 0;
00126     TIM_QEPInitStruct->QEP_ZLC = 0;
00127     TIM_QEPInitStruct->QEP_ZNC = 0;
00128     TIM_QEPInitStruct->QEP_ZPC = 0;
00129     TIM_QEPInitStruct->QEP_Filter = 0;
00130     TIM_QEPInitStruct->QEP_IRQEna = 0;
00131 }
00132
00153 void TIM_QEPCmd(TIM_QEPTTypeDef *QEPx, FuncState state)
00154 {
00155     if (state != DISABLE)
00156     {
00157         QEPx->CFG |= BIT15;
00158     }
00159     else
00160     {
00161         QEPx->CFG &= ~BIT15;
00162     }
00163 }
00164
00185 uint16_t TIM_QEP_GetCount(TIM_QEPTTypeDef *QEPx)
00186 {
00187     return QEPx->CNT;
00188 }
00189
00216 void QEP_ClearIRQFlag(TIM_QEPTTypeDef *QEPx, uint32_t tempFlag)
00217 {
00218     QEPx->IF = tempFlag;
00219 }
00220
00246 u16 QEP_GetIRQFlag(TIM_QEPTTypeDef *QEPx, u32 QEP_IF_x)
00247 {
00248     if (QEPx->IF & QEPx->IE & QEP_IF_x)
00249     {
00250         return 1;
00251     }
00252     else
00253     {
00254         return 0;
00255     }
00256 }
00257
00283 u16 QEP_GetFlag(TIM_QEPTTypeDef *QEPx, u32 QEP_IF_x)
00284 {
00285     if (QEPx->IF & QEP_IF_x)
00286     {
00287         return 1;
00288     }
00289     else
00290     {
00291         return 0;
00292     }
00293 }

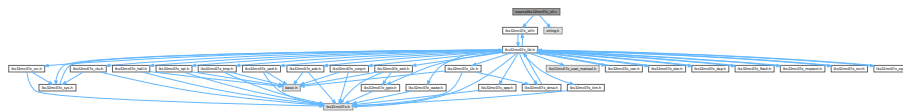
```

5.83 source/lks32mc07x_sif.c 文件参考

文件名称: LKS32MC07x_sif.c
 文件标识: 无
 内容摘要: SIF外设驱动程序
 其它说明: 无

```
#include "lks32mc07x_sif.h"
#include "string.h"
```

lks32mc07x_sif.c 的引用(Include)关系图:



函数

- void **SIF_StructInit** (SIF_InitTypeDef *SIFInitStruct)
- void **SIF_Init** (SIF_RegTypeDef *SIFx, SIF_InitTypeDef *SIFInitStruct)
 函数名称: void *SIF_Init(SIF_RegTypeDef *SIFx, SIF_InitTypeDef *SIFInitStruct)*
- void **SIF_Timebase.set** (u32 tosc)
 函数名称: void *SIF_Timebase.set(u32 tosc)*
- void **SIF_TH.set** (u32 tsh, u32 tdh)
 函数名称: void *SIF_Timebase.set(u32 tosc)*
- void **SIF_Senddata** (u8 Data)
 函数名称: void *SIF_Senddata(u8 Data)*
- void **SIF_DMA.IsEnable** (u8 state)
 函数名称: void *SIF_DMA.Enable(void)*
- void **SIF_Irq.IsEnable** (u8 state)
 函数名称: void *SIF_DMA.Enable(void)*
- void **SIF_Clear.Irq** (void)
 函数名称: void *SIF_Clear.Irq()*

5.83.1 详细描述

文件名称: LKS32MC07x_sif.c
 文件标识: 无
 内容摘要: SIF外设驱动程序
 其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月12日	1.1	myx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 中定义.

5.83.2 函数说明

5.83.2.1 SIF_Clear_Irq()

```
void SIF_Clear_Irq (
    void )
```

函数名称: `void SIF_Clear_Irq()`

功能描述: 清除SIF模块标志位

参见

被引用内容:

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 224 行定义.

5.83.2.2 SIF_DMA_IsEnable()

```
void SIF_DMA_IsEnable (
    u8 state )
```

函数名称: void [SIF_DMA_Enable\(void\)](#)

功能描述: SIF DMA传输请求使能

参见

被引用内容:

参数

输入参数:	state : ENABLE/DIABLE
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 164 行定义.

5.83.2.3 SIF_Init()

```
void SIF_Init (
    SIF_RegTypeDef * SIFx,
    SIF_InitTypeDef * SIFInitStruct )
```

函数名称: void [SIF_Init](#)([SIF_RegTypeDef](#) *[SIFx](#), [SIF_InitTypeDef](#) *[SIFInitStruct](#))

功能描述: SIF初始化函数

参见
被引用内容:

参数

输入参数:	SIFx: SIF , SIF_InitTypeDef * SIF_InitStruct
输出参数:	无

返回
返回 值: 无

注解
其它说明: 无

警告
无

```
示例代码:

SIF\_InitTypeDef SIF_InitStruct;
SIF\_StructInit (&SIF_InitStruct);

SIF_InitStruct.SIF_DONE = DISABLE;           // 传输完成后默认低电平
SIF_InitStruct.SIF_SYNC = DISABLE;           // 同步信号默认低电平
SIF_InitStruct.SIF_SYNC.PULSE = ENABLE;      // 同步信号有电平反转
SIF_InitStruct.SIF_DONE.VLD = DISABLE;       // 无结束信号
SIF_InitStruct.SIF_SYNC.VLD = ENABLE;        // 有同步信号
SIF_InitStruct.SIF_RATIO = DISABLE;          // 数据占比 2: 1
SIF_InitStruct.SIF_MSB = ENABLE;             // 数据传送顺序 高字节在前
SIF_InitStruct.SIF_EN = ENABLE;              // SIF模块使能
SIF_InitStruct.SIF_TOSC = 46;                // 时基设置 47 * 333 = 15.67US
SIF_InitStruct.SIF_TSTH1 = 999;              // 同步时长 1000 * SIF_TOSC * 32
SIF_InitStruct.SIF_TDTH1 = 0;                // 结束信号时长 1MS
SIF_InitStruct.SIF_IRQ_IF = ENABLE;          // 有SIF中断标志位
SIF_InitStruct.SIF_DMA_EN = ENABLE;          // DMA传输使能
SIF_InitStruct.SIF_IRQ_EN = DISABLE;         // SIF中断关闭

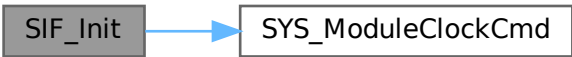
SIF_Init (SIF0, &SIF_InitStruct);
```

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 61 行定义.

函数调用图:



5.83.2.4 SIF_Irq_IsEnable()

```
void SIF_Irq_IsEnable (
    u8 state )
```

函数名称: void [SIF_DMA_Enable\(void\)](#)

功能描述: SIF DMA传输请求使能

参见

被引用内容:

参数

输入参数:	state : ENABLE/DIABLE
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 194 行定义.

5.83.2.5 SIF_Senddata()

```
void SIF_Senddata (
    u8 Data )
```

函数名称: void [SIF_Senddata\(u8 Data\)](#)

功能描述: 传输数据设置

参见

被引用内容:

参数

输入参数:	Data :传输的数据
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 141 行定义.

5.83.2.6 SIF_StructInit()

```
void SIF_StructInit (  
    SIF_InitTypeDef * SIFInitStruct )
```

在文件 [lks32mc07x_sif.c](#) 第 17 行定义.

5.83.2.7 SIF_TH.set()

```
void SIF_THset (  
    u32 tsh,  
    u32 tdh )
```

函数名称: void [SIF_Timebase_set\(u32 tosc\)](#)

功能描述: 设置信号通信方式

参见

被引用内容:

参数

输入参数:	tsh tsl : 同步信号周期 tdh tdl : 结束信号周期
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 117 行定义.

5.83.2.8 SIF_Timebase_set()

```
void SIF_Timebase_set (  
    u32 tosc )
```

函数名称: void [SIF_Timebase_set\(u32 tosc\)](#)

功能描述: 设置SIF传输时基

参见

被引用内容:

参数

输入参数:	tosc : 时基系数
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年2月12日	1.1	ywx	修改

修改日志:

在文件 [lks32mc07x_sif.c](#) 第 93 行定义.

5.84 lks32mc07x_sif.c

[浏览该文件的文档.](#)

```

00001
00014 #include "lks32mc07x_sif.h"
00015 #include "string.h"
00016
00017 void SIF_StructInit(SIF_InitTypeDef *SIFInitStruct)
00018 {
00019     memset(SIFInitStruct, 0, sizeof(SIF_InitTypeDef));
00020 }
00021
00061 void SIF_Init(SIF_RegTypeDef *SIFx, SIF_InitTypeDef *SIFInitStruct)
00062 {
00063     SIFx->SIFx_TOSC = SIFInitStruct->SIF_TOSC;
00064     SIFx->SIFx_TSTH1 = SIFInitStruct->SIF_TSTH1;
00065     SIFx->SIFx_TDTH1 = SIFInitStruct->SIF_TDTH1;
00066     SIFx->SIFx_IRQ = (SIFInitStruct->SIF_IRQ_EN << 0) | (SIFInitStruct->SIF_DMA_EN << 1) |
(SIFInitStruct->SIF_IRQ_IF << 4);
00067
00068     SIFx->SIFx_CFG = (SIFInitStruct->SIF_EN << 0) | (SIFInitStruct->SIF_MSB << 1) |
(SIFInitStruct->SIF_RATIO << 2) |
00069     (SIFInitStruct->SIF_SYNC_VLD << 3) | (SIFInitStruct->SIF_DONE_VLD << 4) |
00070     (SIFInitStruct->SIF_SYNC_PULSE << 5) | (SIFInitStruct->SIF_SYNC << 6) |
(SIFInitStruct->SIF_DONE << 7);
00071
00072     SYS_ModuleClockCmd(SYS_Module.SIF0, ENABLE); // HAL1时钟使能
00073 }
00074
00093 void SIF_Timebase_set(u32 tosc)
00094 {
00095     SIF0->SIFx_TOSC = tosc;
00096 }
00097
00117 void SIF_TH_set(u32 tsh, u32 tdh)
00118 {
00119     SIF0->SIFx_TSTH1 = tsh;
00120     SIF0->SIFx_TDTH1 = tdh;
00121 }
00122
00141 void SIF_Senddata(u8 Data)
00142 {
00143     REG32(SIF0_BASE + 0x14) = Data;
00144 }
00145
00164 void SIF_DMA_IsEnable(u8 state)
00165 {
00166     if (state == ENABLE)
00167     {
00168         SIF0->SIFx_IRQ |= SIF_DMA_Enable;
00169     }
00170     else
00171     {
00172         SIF0->SIFx_IRQ &= ~SIF_DMA_Enable;
00173     }
00174 }
00175
00194 void SIF_Irq_IsEnable(u8 state)

```

```

00195 {
00196     if (state == ENABLE)
00197     {
00198         SIF0->SIFx_IRQ |= SIF_IE_Enable;
00199     }
00200     else
00201     {
00202         SIF0->SIFx_IRQ &= ~SIF_IE_Enable;
00203     }
00204 }
00205
00224 void SIF_Clear_Irq(void)
00225 {
00226     SIF0->SIFx_IRQ |= BIT4;
00227 }

```

5.85 source/lks32mc07x_spi.c 文件参考

#include "lks32mc07x_lib.h"

lks32mc07x_spi.c 的引用(Include)关系图:



函数

- void **SPI_Init** (SPI_TypeDef *SPIx, SPI_InitTypeDef *SPI_InitStruct)
函数名称: void *SPI_Init(SPI_TypeDef* SPIx, SPI_InitTypeDef* SPI_InitStruct)*
- void **SPI_StructInit** (SPI_InitTypeDef *SPI_InitStruct)
函数名称: void *SPI_StructInit(SPI_InitTypeDef* SPI_InitStruct)*
- void **SPI_SendData** (SPI_TypeDef *SPIx, uint8_t n)
函数名称: void *SPI_SendData(SPI_TypeDef *SPIx, uint8_t n)*
- uint8_t **SPI_ReadData** (SPI_TypeDef *SPIx)
函数名称: uint8_t *SPI_ReadData(SPI_TypeDef *SPIx)*
- uint8_t **SPI_GetIRQFlag** (SPI_TypeDef *SPIx, u8 IRQ_Flag)
函数名称: uint8_t *SPI_GetIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag)*
- void **SPI_ClearIRQFlag** (SPI_TypeDef *SPIx, u8 IRQ_Flag)
函数名称: void *SPI_ClearIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag)*

5.85.1 函数说明

5.85.1.1 SPI_ClearIRQFlag()

```

void SPI_ClearIRQFlag (
    SPI_TypeDef * SPIx,
    u8 IRQ_Flag )

```

函数名称: void *SPI_ClearIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag)*

功能描述: 清除SPI中断标志位

参见

被引用内容: 无

参数

输入参数:	SPIx: SPI_IRQ_Flag ħš	
	SPI_IF_TranDone	传输完成中断标志位
	SPI_IF_SSErr	传输异常中断标志位
	SPI_IF_DataOver	传输溢出中断标志位
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

æ—

示例代码:

```
if (SPI_GetIRQFlag (SPI, SPI_IF_TranDone)) //获取传输完成中断标志位
{
    SPI_ClearIRQFlag (SPI, SPI_IF_TranDone); //清除传输完成中断标志位
}
```

Date	Version	Author	Description
2022年04月20日	1.0	Zhu Jie	创建

修改日志:

在文件 lks32mc07x_spi.c 第 237 行定义.

5.85.1.2 SPI_GetIRQFlag()

```
uint8_t SPI_GetIRQFlag (
    SPI_TypeDef * SPIx,
    u8 IRQ_Flag )
```

函数名称: uint8_t SPI_GetIRQFlag(SPI_TypeDef *SPIx,u8 IRQ_Flag)

功能描述: 获取SPI中断标志位

参见

被引用内容： 无

参数

输入参数:	SPIx: SPI_IRQ_Flag ^{1/4}	
	SPI_IF_TranDone	传输完成中断标志位
	SPI_IF_SSErr	传输异常中断标志位
	SPI_IF_DataOver	传输溢出中断标志位
输出参数:	无	

返回

返回 值: 无

注解

其它说明: 无

警告

只有对应中断使能后，改为才能读取，如果对应中断未使能，读取结果一直为0

示例代码:

```
if(SPI_GetIRQFlag(SPI,SPI_IF_TranDone))//获取传输完成中断标志位
{
}
```

Date	Version	Author	Description
2022年04月20日	1.0	Zhu Jie	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 196 行定义.

5.85.1.3 SPI_Init()

void SPI_Init (

 SPI_TypeDef * SPIx,

 SPI_InitTypeDef * SPI_InitStruct)

函数名称: void [SPI_Init\(SPI_TypeDef* SPIx, SPI_InitTypeDef* SPI_InitStruct\)](#)

功能描述: SPI初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	SPIx: SPI , SPI_InitTypeDef *SPI_InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

⚠

示例代码:

```
SPI_InitTypeDef SPI_InitStruct;
SPI_StructInit (&SPI_InitStruct);           //SPI结构体初始化
SPI_InitStruct.Duplex = SPI_Full;            //双工模式设置
SPI_InitStruct.Mode = SPI_Master;            //master主机模式
SPI_InitStruct.EN = ENABLE;                  //使能SPI模块
SPI_InitStruct.DMA = SPI_DMA_DISABLE;        // 选择SPI搬移方式, 由MCU搬运数据到SPI
SPI_InitStruct.Trig = 0;                     //内部自动触发传输
SPI_InitStruct.DataOrder = SPI_FIRSTSEND_MSB; //该字节先发送 MSB
SPI_InitStruct.CPHA = 0;                     //第一个沿为发送数据时刻
SPI_InitStruct.CPOL = 0;                     //CLK默认高电平
SPI_InitStruct.BaudRate = 30;                 //传输速度96MHZ/(2*(30+1))= 1.548MHZ
SPI_InitStruct.ByteLength = 0;                //发送零个字节
SPI_InitStruct.IRQEna = DISABLE;              //关闭SPI各中断
SPI_Init (SPI, &SPI_InitStruct);             //SPI初始化程序
```

Date	Version	Author	Description
2016年05月21日	1.0	cfwu	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 59 行定义.

函数调用图:



5.85.1.4 SPI_ReadData()

```
uint8_t SPI_ReadData (
    SPI_TypeDef * SPIx )
```

函数名称: uint8_t **SPI_ReadData**(SPI_TypeDef *SPIx)

功能描述: SPI读缓冲区数据

参见

被引用内容: 无

参数

输入参数:	SPIx: SPI
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

⚠

示例代码:

```
u8 SPI_RXValue = 0;
SPI_RXValue = SPI_ReadData(SPI); //SPI接收一字节数据
```

Date	Version	Author	Description
2016年05月21日	1.0	cfwu	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 165 行定义.

5.85.1.5 SPI_SendData()

```
void SPI_SendData (
    SPI_TypeDef * SPIx,
    uint8_t n )
```

函数名称: void [SPI_SendData\(SPI_TypeDef *SPIx, uint8_t n\)](#)

功能描述: SPI发送数据

参见

被引用内容: 无

参数

输入参数:	SPIx: SPI, n: 发送一字节数据
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

æ—

示例代码:

```
SPI\_SendData(SPI,0x12); //SPI发送0x12一字节数据
```

Date	Version	Author	Description
2016年05月21日	1.0	cfwu	创建

修改日志:

在文件 [lks32mc07x_spi.c](#) 第 135 行定义.

5.85.1.6 [SPI_StructInit\(\)](#)

```
void SPI_StructInit (  
    SPI\_InitTypeDef * SPI_InitStruct )
```

函数名称: void [SPI_StructInit\(SPI_InitTypeDef* SPI_InitStruct\)](#)

功能描述: SPI结构体初始化

参见

被引用内容: 无

参数

输入参数:	<code>SPI_InitTypeDef *SPI_InitStruct</code>
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

⚠

示例代码:

```
SPI_InitTypeDef SPI_InitStruct;  
SPI_StructInit(&SPI_InitStruct);           //SPI结构体初始化
```

Date	Version	Author	Description
2016年05月21日	1.0	cfwu	创建

修改日志:

在文件 `lks32mc07x_spi.c` 第 97 行定义.

5.86 lks32mc07x_spi.c

浏览该文件的文档.

```
00001 /*****  
00002  * 版权所有 (C)2015, LINKO SEMICONDUCTOR Co.ltd  
00003  *  
00004  * 文件名称: lks32mc07x_spi.c  
00005  * 文件标识:  
00006  * 内容摘要: SPI外设驱动程序  
00007  * 其它说明: 无  
00008  * 当前版本: V 1.0  
00009  * 作 者:  
00010  * 完成日期:  
00011  *  
00012  * 修改记录1:  
00013  * 修改日期:  
00014  * 版 本 号: V 1.0  
00015  * 修 改 人:  
00016  * 修改内容: 创建  
00017  *  
00018  * 修改记录2:  
00019  * 修改日期:  
00020  * 版 本 号:
```

```

00021 * 修改人:
00022 * 修改内容:
00023 *
00024 *****/
00025 #include "lks32mc07x.lib.h"
00026
00059 void SPI_Init(SPI_TypeDef *SPIx, SPI_InitTypeDef *SPI_InitStruct)
00060 {
00061     SYS_WR_PROTECT = 0x7a83; /* 使能系统寄存器写操作*/
00062     SYS_SFT_RST |= BIT0;      // 复位SPI
00063     SYS_SFT_RST &= ~BIT0;     // 释放SPI
00064     SYS_WR_PROTECT = 0x0;     /* 关闭系统寄存器写操作*/
00065
00066     SYS_ModuleClockCmd(SYS_Module.SPI0, ENABLE);
00067
00068     SPIx->IE = SPI_InitStruct->Trig << 3 | (SPI_InitStruct->IRQEna);
00069
00070     SPIx->BAUD = SPI_InitStruct->BaudRate | (SPI_InitStruct->TRANS_MODE << 15);
00071
00072     SPIx->SIZE = SPI_InitStruct->ByteLength;
00073
00074     SPIx->CFG = SPI_InitStruct->EN | (SPI_InitStruct->ENDIAN << 1) | (SPI_InitStruct->CPOL << 2) |
        (SPI_InitStruct->CPHA << 3) | (SPI_InitStruct->Mode << 4) | (SPI_InitStruct->CS << 5) |
        (SPI_InitStruct->Duplex << 6);
00075 }
00076
00097 void SPI_StructInit(SPI_InitTypeDef *SPI_InitStruct)
00098 {
00099     SPI_InitStruct->Duplex = SPI_Full;
00100     SPI_InitStruct->CS = 0;
00101     SPI_InitStruct->Mode = SPI_Slave;
00102     SPI_InitStruct->CPOL = 0;
00103     SPI_InitStruct->CPHA = 0;
00104     SPI_InitStruct->ENDIAN = DISABLE;
00105     SPI_InitStruct->EN = 0;
00106
00107     SPI_InitStruct->IRQEna = DISABLE;
00108     SPI_InitStruct->Trig = 0;
00109
00110     SPI_InitStruct->TRANS_MODE = 0;
00111     SPI_InitStruct->BaudRate = 3;
00112
00113     SPI_InitStruct->ByteLength = 0;
00114 }
00115
00135 void SPI_SendData(SPI_TypeDef *SPIx, uint8_t n)
00136 {
00137     if ((SPIx->CFG & (1 << 5)) == 1)
00138     {
00139         SPIx->TX_DATA = n;
00140     }
00141     else
00142     {
00143     }
00144 }
00165 uint8_t SPI_ReadData(SPI_TypeDef *SPIx)
00166 {
00167     return SPIx->RX_DATA;
00168 }
00169
00196 uint8_t SPI_GetIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag)
00197 {
00198     u8 SPI_IRQFlag = 0;
00199     if (SPIx->IE & IRQ_Flag)
00200     {
00201         SPI_IRQFlag = 1;
00202     }
00203     else
00204     {
00205         SPI_IRQFlag = 0;
00206     }
00207     return SPI_IRQFlag;
00208 }
00209
00237 void SPI_ClearIRQFlag(SPI_TypeDef *SPIx, u8 IRQ_Flag)
00238 {
00239     SPIx->IE |= IRQ_Flag;
00240 }

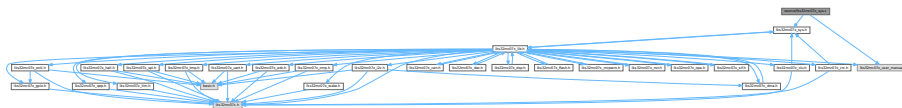
```


5.87 source/lks32mc07x_sys.c 文件参考

文件名称: [lks32mc07x_sys.c](#)
 文件标识: 无
 内容摘要: SYS外设驱动程序
 其它说明: 无

```
#include "lks32mc07x_sys.h"
#include "lks32mc07x_user_manual.h"
```

[lks32mc07x_sys.c](#) 的引用(Include)关系图:



函数

- void [SYS_Init](#) (SYS_InitTypeDef *SYS_InitStruct)
 函数名称: [void SYS_Init\(SYS_InitTypeDef* SYS_InitStruct\)](#)
- void [SYS_StructInit](#) (SYS_InitTypeDef *SYS_InitStruct)
 函数名称: [void SYS_StructInit\(SYS_InitTypeDef* SYS_InitStruct\)](#)
- void [SYS_ClearRst](#) (void)
 函数名称: [void SYS_ClearRst\(void\)](#)
- void [SYS_ModuleClockCmd](#) (uint32_t nModule, FuncState state)
 函数名称: [void SYS_ModuleClockCmd\(uint32_t nModule, FuncState state\)](#)
- void [SYS_AnalogModuleClockCmd](#) (uint32_t nModule, FuncState state)
 函数名称: [void SYS_AnalogModuleClockCmd\(uint32_t nModule, FuncState state\)](#)
- void [SYS_SoftResetModule](#) (uint32_t nModule)
 函数名称: [void SYS_SoftResetModule\(uint32_t nModule\)](#)
- void [SYS_SoftReset_all](#) (uint32_t state)
 函数名称: [void SYS_SoftReset_all\(uint32_t state\)](#)
- static void [SoftDelay](#) (u32 cnt)
 函数名称: [static void SoftDelay\(u32 cnt\)](#)
- void [SYS_MclkChoice](#) (u8 SYS_MCLK_X)
 函数名称: [void SYS_MclkChoice\(u8 SYS_MCLK_X\)](#)
- u32 [SYS_ReadMcuClk](#) (void)
 函数名称: [u32 SYS_ReadMcuClk\(void\)](#)
- void [SYS_UartClkDiv](#) (u16 div)
 函数名称: [void SYS_UartClkDiv\(u16 div\)](#)
- void [SYS_I2CClkDiv](#) (u16 div)
 函数名称: [void SYS_I2CClkDiv\(u16 div\)](#)
- void [SYS_SPIClkDiv](#) (u16 div)
 函数名称: [void SYS_SPIClkDiv\(u16 div\)](#)
- void [IWDG_Init](#) (IWDG_InitTypeDef *this)
 函数名称: [void IWDG_Init\(IWDG_InitTypeDef *this\)](#)
- void [IWDG_StructInit](#) (IWDG_InitTypeDef *this)
 函数名称: [void IWDG_StructInit\(IWDG_InitTypeDef *this\)](#)
- void [IWDG_DISABLE](#) (void)

- 函数名称: void IWDG_DISABLE(void)

• void IWDG_ENABLE (void)

函数名称: void IWDG_ENABLE(void)

• void IWDG_Feed (void)

函数名称: void IWDG_Feed(void)

• void SYS_VolSelModule (uint32_t SYS_MCU_POWER_x)

设置芯片工作电压（同时打开电源掉电检测中断）

• void SYS_VolSelModuleIRQ (void)

关闭pwm输出并复位mcu（用于掉电检测中断）

• void SYS_VolSelModuleEnableIRQ (uint32_t SYS_MCU_POWER_x)

使能掉电检测中断，需要先配置芯片工作电压(函数: SYS_VolSelModule)

变量

- static u32 sys_clk = 96000000
- 全局变量存储主时钟频率值

5.87.1 详细描述

文件名称: lks32mc07x.sys.c
文件标识: 无
内容摘要: SYS外设驱动程序
其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 lks32mc07x.sys.c 中定义.

5.87.2 函数说明

5.87.2.1 IWDG_DISABLE()

```
void IWDG_DISABLE (
    void )
```

函数名称: void IWDG_DISABLE(void)

功能描述: 关闭看门狗模块

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
IWDG_DISABLE(); //关闭看门狗模块
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x-sys.c](#) 第 692 行定义.

5.87.2.2 IWDG_ENABLE()

```
void IWDG_ENABLE (  
    void )
```

函数名称: void [IWDG_ENABLE\(void\)](#)

功能描述: 打开看门狗模块

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
IWDG_ENABLE(); //打开看门狗模块
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 716 行定义.

5.87.2.3 IWDG_Feed()

```
void IWDG_Feed (
    void )
```

函数名称: [void IWDG_Feed\(void\)](#)

功能描述: 看门狗喂狗

参见

被引用内容: 无

参数

输入参数:	无
输出参数:	无

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

```
IWDG_Feed(); //看门狗喂狗
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 740 行定义.

5.87.2.4 IWDG_Init()

```
void IWDG_Init (
    IWDG_InitTypeDef * this )
```

函数名称: void [IWDG_Init\(IWDG_InitTypeDef *this\)](#)

功能描述: 看门狗初始化

参见

被引用内容： 无

参数

输入参数:	IWDG_InitTypeDef *this 看门狗配置结构体
输出参数:	无

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

```
IWDG_InitTypeDef IWDG_InitStruct;  
IWDG_StrutInit (&IWDG_InitStruct);  
IWDG_InitStruct.WDG_EN = ENABLE;           //使能看门狗  
IWDG_InitStruct.RTH      = SECOND2IWDGCNT(2.0); //设置看门狗  
IWDG_InitStruct.DWK_EN   = DISABLE;         // 深度休眠定时唤醒关闭  
IWDG_InitStruct.WTH      = 0;               // 看门狗定时唤醒时间（21位计数器，但低12恒位0）  
IWDG_Init (&IWDG_InitStruct);
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 625 行定义.

5.87.2.5 IWDG_StrutInit()

```
void IWDG_StrutInit (  
    IWDG_InitTypeDef * this )
```

函数名称: void [IWDG_StrutInit\(IWDG_InitTypeDef *this\)](#)

功能描述: 看门狗配置结构体初始化

参见

被引用内容： 无

参数

输入参数:	IWDG_InitTypeDef *this 看门狗配置结构体
输出参数:	无

返回

返回 值: 无

注解

其它说明： 无

警告

无

示例代码:

```
IWDG_InitTypeDef IWDG_InitStructure;  
IWDG_StructInit (&IWDG_InitStructure); //初始化结构体
```

Date	Version	Author	Description
2023年02月28日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 664 行定义.

5.87.2.6 SoftDelay()

```
static void SoftDelay (  
    u32 cnt ) [static]
```

函数名称: static void [SoftDelay\(u32 cnt\)](#)

功能描述: 延时函数

参见

被调用函数： 无

参数

输入参数:	cnt: 延时cnt的空指令NOP()
输出参数:	无

返回

返回值： 无

注解

其它说明： 改函数只在此文件使用，主要用在Switch2PLL函数内

警告

无

示例代码:

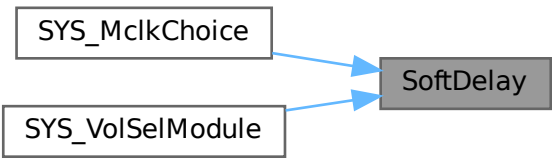
```
SoftDelay(0xff); //延时255个_NOP()时间1个_NOP大约按一个系统时钟周期计算
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 348 行定义.

这是这个函数的调用关系图:



5.87.2.7 SYS_AnalogModuleClockCmd()

```
void SYS_AnalogModuleClockCmd (
    uint32_t nModule,
    FuncState state )
```

函数名称: void [SYS_AnalogModuleClockCmd\(uint32_t nModule, FuncState state\)](#)

功能描述: 模拟模块使能和停止

参见

被调用枚举: FuncState

参数

输入参数:	state: ENABLE使能, DISABLE关闭 nModule: 参数可选:		
	宏定义	说明	
	SYS_AnalogModule_ADC	BIT0	ADC模块
	SYS_AnalogModule_OPA0	BIT2	OPA0模块
	SYS_AnalogModule_OPA1	BIT3	OPA1模块
	SYS_AnalogModule_OPA2	BIT4	OPA2模块
	SYS_AnalogModule_OPA3	BIT5	OPA3模块
	SYS_AnalogModule_CMP0	BIT6	CMP0模块
	SYS_AnalogModule_CMP1	BIT7	CMP1模块
	SYS_AnalogModule_BGP	BIT8	BGP模块
	SYS_AnalogModule_RCH	BIT10	RCH模块
	SYS_AnalogModule_DAC	BIT12	DAC模块
	SYS_AnalogModule_TMP	BIT13	温度传感器
	SYS_AnalogModule_XTAL	BIT14	晶振
	SYS_AnalogModule_PLL	BIT15	PLL模块
输出参数:	无		

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

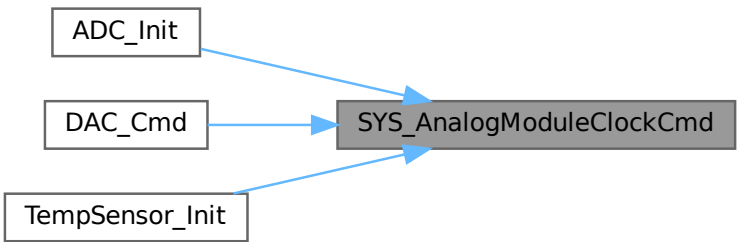
```
SYS_AnalogModuleClockCmd(SYS_AnalogModule_PLL,ENABLE); //使能PLL模块
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 236 行定义.

这是这个函数的调用关系图:



5.87.2.8 SYS.ClearRst()

```
void SYS_ClearRst (
    void )
```

函数名称: void [SYS.ClearRst\(void\)](#)

功能描述: SYS清除复位标志记录

参见
被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回
返回 值: 无

注解
其它说明: 请注意由于复位记录工作于低速时钟域,清除执行完成需要一定时间,不应清 除后立
即读记录状态

警告
无

示例代码:

```
u32 RSET_VALUE = 0;
SYS_ClearRst(); //清除复位标志记录
delay_ms(1);
RSET_VALUE = SYS_GetRstSource(); //读取复位源记录值
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x-sys.c](#) 第 144 行定义.

5.87.2.9 SYS_I2CClkDiv()

```
void SYS_I2CClkDiv (
    u16 div )
```

函数名称: void [SYS_I2CClkDiv\(u16 div\)](#)

功能描述: I2C分频设置

参见

被调用枚举: 无

参数

输入参数:	div: 分频系数
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志

在文件 [lks32mc07x_sys.c](#) 第 567 行定义.

5.87.2.10 SYS.Init()

```
void SYS_Init (
    SYS_InitTypeDef * SYS_InitStruct )
```

函数名称: void [SYS.Init\(SYS.InitTypeDef* SYS_InitStruct\)](#)

功能描述: **SYS**模块初始化函数

参见

被调用函数: 无

参数

输入参数:	SYS_InitTypeDef* SYS_InitStruct
输出参数:	无

返回

返回值: 无

注解

其它说明: 一般不使用该函数，因为涉及模块较多，比较杂乱，对应里面的配置在各模块 初始化单独配置。

警告

无

示例代码:

```
SYS_InitTypeDef SYS_InitStruct;
SYS_StructInit (&SYS_InitStruct);
SYS_InitStruct.PLL_SrcSel = SYS_PLLSRSEL_RCH; // 使用内部4MHz晶振作为时钟输入源
SYS_Init (&SYS_InitStruct);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 47 行定义.

5.87.2.11 SYS.MclkChoice()

```
void SYS_MclkChoice (
    u8 SYS_MCLK_x )
```

函数名称: void [SYS.MclkChoice\(u8 SYS.MCLK_x\)](#)

功能描述: 主时钟选择

参见

被调用枚举: 无

参数

输入参数:	nModule: 参数可选:		
	宏定义	原始值	说明
	SYS_MCLK_96M_RC 0x01		
	96MHz_PLL_RC		
	SYS_MCLK_48M_RC 0x02		
	48MHz_PLL_RC		
	SYS_MCLK_24M_RC 0x03		
	24MHz_PLL_RC		
	SYS_MCLK_12M_RC 0x04		
	12MHz_PLL_RC		
	SYS_MCLK_32K_RC 0x05		
	32kHz_LSI		
	SYS_MCLK_96M_XTAL		
	0x11 96MHz_PLL_XTAL		
	SYS_MCLK_48M_XTAL		
	0x12 48MHz_PLL_XTAL		
	SYS_MCLK_24M_XTAL		
	0x13 24MHz_PLL_XTAL		
	SYS_MCLK_12M_XTAL		
	0x14 12MHz_PLL_XTAL		
输出参数:	无		

返回

返回值: 无

注解

其它说明： 无

警告

无

示例代码:

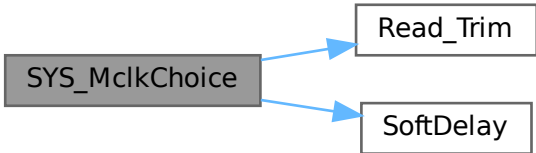
```
//软复位SPI模块，清除SPI寄存器配置，PC指针复位
SYS_MclkChoice (SYS_MCLK_96M_RC);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:增加延时函数，等待寄存器复位完毕与释放复位操作。

在文件 [lks32mc07x_sys.c](#) 第 391 行定义.

函数调用图:



5.87.2.12 SYS.ModuleClockCmd()

```
void SYS.ModuleClockCmd (
    uint32_t nModule,
    FuncState state )
```

函数名称: void [SYS.ModuleClockCmd\(uint32_t nModule, FuncState state\)](#)

功能描述: 数字模块时钟使能和停止

参见

被调用枚举: FuncState

参数

输入参数:	state: ENABLE使能, DISABLE关闭 nModule: 参数可选:		
	宏定义	原始值	说明
	SYS_Module_I2C	BIT0	I2C模块
	SYS_Module_HALL	BIT1	HALL模块
	SYS_Module_TIMER	BIT2	UTIMER模块
	SYS_Module_MCPWM	BIT3	MCPWM模块
	SYS_Module_UART0	BIT4	UART0模块
	SYS_Module_UART1	BIT5	UART1模块
	SYS_Module_DSP	BIT6	DSP模块
	SYS_Module_CAN	BIT7	CAN模块
	SYS_Module_SPI	BIT8	SPI模块
输出参数:	无		

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

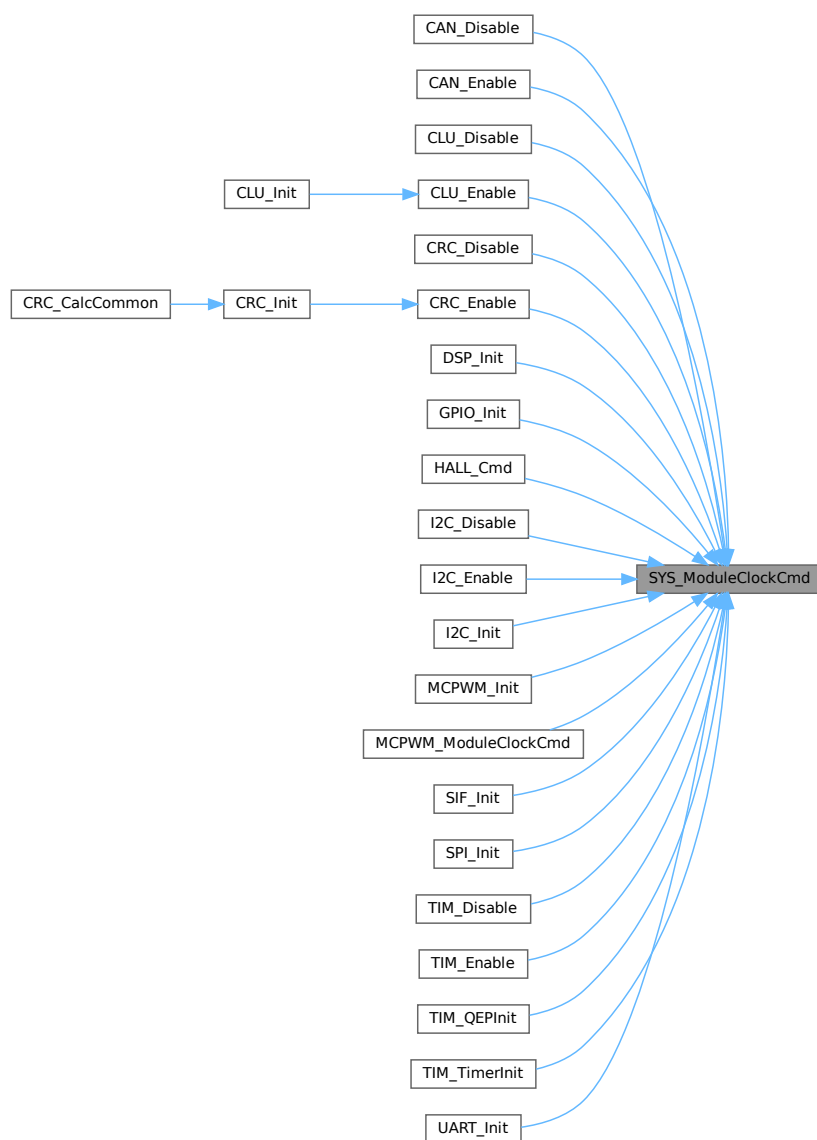
```
SYS_ModuleClockCmd(SYS_Module_SPI,ENABLE); //使能SPI时钟
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 184 行定义.

这是这个函数的调用关系图:



5.87.2.13 SYS_ReadMcuClk()

```
u32 SYS_ReadMcuClk (
    void )
```

函数名称: u32 [SYS_ReadMcuClk\(void\)](#)

功能描述: 获取当前时钟

参见

被调用枚举: 无

参数

输入参数:	
输出参数:	无

返回

返回 值: 时钟频率

注解

其它说明: 无

警告

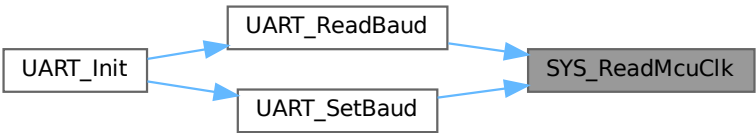
无

示例代码:

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:增加延时函数，等待寄存器复位完毕与释放复位操作。

在文件 [lks32mc07x.sys.c](#) 第 517 行定义.
这是这个函数的调用关系图:



5.87.2.14 SYS.SoftReset_all()

```
void SYS_SoftReset_all (
    uint32_t state )
```

函数名称: void [SYS.SoftReset_all\(uint32_t state\)](#)
功能描述: 数字模块软复位

参见

被调用枚举： 无

参数

输入参数:	state: ENABLE使能 , DISABLE关闭 nModule: 参数可选:
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
//软复位所有模块，清除相关外设寄存器配置，PC指针复位  
SYS_SoftResetModule(ENABLE);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:增加延时函数，等待寄存器复位完毕与释放复位操作。

在文件 [lks32mc07x_sys.c](#) 第 313 行定义.

5.87.2.15 SYS_SoftResetModule()

```
void SYS_SoftResetModule (  
    uint32_t nModule )
```

函数名称: void [SYS_SoftResetModule\(uint32_t nModule\)](#)

功能描述: 数字模块软复位

参见

被调用枚举： 无

参数

输入参数:	state: ENABLE使能, DISABLE关闭 nModule: 参数可选:		
	宏定义	原始值	说明
	SYS_Module_I2C	BIT0	I2C模块
	SYS_Module_HALL	BIT1	HALL模块
	SYS_Module_TIMER	BIT2	UTIMER模块
	SYS_Module_MCPWM	BIT3	MCPWM模块
	SYS_Module_UART0	BIT4	UART0模块
	SYS_Module_UART1	BIT5	UART1模块
	SYS_Module_DSP	BIT6	DSP模块
	SYS_Module_CAN	BIT7	CAN模块
	SYS_Module_SPI	BIT8	SPI模块
输出参数:	无		

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

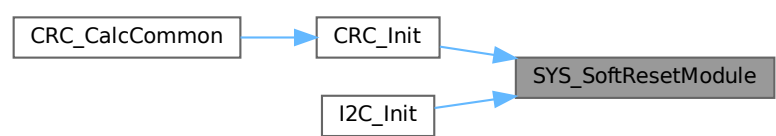
```
//软复位SPI模块,清除SPI寄存器配置,PC指针复位
SYS_SoftResetModule(SYS_Module_SPI);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:增加延时函数,等待寄存器复位完毕与释放复位操作。

在文件 lks32mc07x_sys.c 第 284 行定义.

这是这个函数的调用关系图:



5.87.2.16 SYS.SPIClkDiv()

```
void SYS_SPIClkDiv (
    u16 div )
```

函数名称: void **SYS.SPIClkDiv(u16 div)**

功能描述: SPI分频设置

参见
被调用枚举: 无

参数

输入参数:	div: 分频系数
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志

在文件 [lks32mc07x_sys.c](#) 第 593 行定义.

5.87.2.17 SYS.StructInit()

```
void SYS_StructInit (  
    SYS_InitTypeDef * SYS_InitStruct )
```

函数名称: void [SYS.StructInit\(SYS_InitTypeDef* SYS_InitStruct\)](#)

功能描述: **SYS**结构体初始化

参见

被调用函数: 无

参数

输入参数:	SYS_InitTypeDef* SYS_InitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
SYS_InitTypeDef SYS_InitStruct;  
SYS_StructInit (&SYS_InitStruct);
```

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志:

在文件 [lks32mc07x_sys.c](#) 第 107 行定义.

5.87.2.18 SYS.UartClkDiv()

```
void SYS_UartClkDiv (
    u16 div )
```

函数名称: void [SYS.UartClkDiv\(u16 div\)](#)

功能描述: 串口分频设置

参见

被调用枚举: 无

参数

输入参数:	div: 分频系数
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

Date	Version	Author	Description
2023年02月12日	1.0	ywx	创建

修改日志

在文件 [lks32mc07x-sys.c](#) 第 541 行定义.

5.87.2.19 SYS.VolSelModule()

```
void SYS_VolSelModule (
    uint32_t SYS_MCU_POWER_x )
```

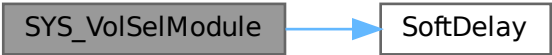
设置芯片工作电压（同时打开电源掉电检测中断）

参数

<i>SYS_MCU_POWER_x</i>	芯片供电电压 SYS.MCU.POWER_3v3 或 SYS.MCU.POWER_5v0
------------------------	--

在文件 [lks32mc07x-sys.c](#) 第 771 行定义.

函数调用图:



5.87.2.20 SYS.VolSelModuleEnableIRQ()

```
void SYS_VolSelModuleEnableIRQ (
    uint32_t SYS_MCU_POWER_x )
```

使能掉电检测中断，需要先配置芯片工作电压(函数: SYS.VolSelModule)

在文件 [lks32mc07x-sys.c](#) 第 832 行定义.

5.87.2.21 SYS.VolSelModuleIRQ()

```
void SYS_VolSelModuleIRQ (
    void )
```

关闭pwm输出并复位mcu（用于掉电检测中断）

在文件 [lks32mc07x-sys.c](#) 第 809 行定义.

5.87.3 变量说明

5.87.3.1 sys_clk

```
u32 sys_clk = 96000000 [static]
```

全局变量存储主时钟频率值

在文件 `lks32mc07x_sys.c` 第 22 行定义。

5.88 lks32mc07x_sys.c

[浏览该文件的文档.](#)

```
00001
00015 #include "lks32mc07x_sys.h"
00016 #include "lks32mc07x_user_manual.h"
00017
00022 static u32 sys_clk = 96000000;
00023
00047 void SYS_Init(SYS_InitTypeDef *SYS_InitStruct)
00048 {
00049     uint32_t RCHPd, XTALPdn;
00050     uint32_t tmp1, tmp2;
00051     uint32_t cnt;
00052
00053     SYS_WR_PROTECT = 0x7A83; // 解除系统寄存器写保护
00054
00055     tmp1 = AFE->REG6;
00056     tmp1 &= 0x7FFF;
00057     tmp1 |= (SYS_InitStruct->PLL_SrcSel << 15);
00058     AFE->REG6 = tmp1;
00059
00060     if (SYS_InitStruct->PLL_SrcSel == SYS_PLLSRSEL_CRYSTAL)
00061     {
00062         RCHPd = DISABLE; /*RCH(4MHz)时钟关闭*/
00063         XTALPdn = ENABLE; /*晶体起振电路开启*/
00064     }
00065     else
00066     {
00067         RCHPd = ENABLE; /*RCH(4MHz)时钟打开*/
00068         XTALPdn = DISABLE; /*晶体起振电路关闭*/
00069     }
00070
00071     tmp2 = AFE->REG5;
00072     tmp2 &= 0xBBFF; // 1010 1010 1111 1111
00073     tmp2 |= (RCHPd << 10);
00074     tmp2 |= (XTALPdn << 14);
00075     AFE->REG5 = tmp2;
00076
00077     SYS_CLK_CFG = SYS_InitStruct->PLL_DivSel + (SYS_InitStruct->Clk_Sel << 8);
00078     for (cnt = 0; cnt < 4000; cnt++)
00079     {
00080         __NOP();
00081     }
00082     SYS->CLK_DIV0 = SYS_InitStruct->Clk_DivSPI;
00083     SYS->CLK_DIV2 = SYS_InitStruct->Clk_DivUART;
00084     SYS->CLK_FEN = SYS_InitStruct->Clk_FEN;
00085 }
00086
00107 void SYS_StructInit(SYS_InitTypeDef *SYS_InitStruct)
00108 {
00109     SYS_InitStruct->PLL_SrcSel = SYS_PLLSRSEL_RCH;
00110     SYS_InitStruct->PLL_DivSel = 0xFF;
00111     SYS_InitStruct->Clk_Sel = CLK_SEL_PLL;
00112
00113     SYS_InitStruct->Clk_DivSPI = SYS_Clk_SPIDiv1;
00114     SYS_InitStruct->Clk_DivUART = SYS_Clk_UARTDiv1;
00115     SYS_InitStruct->Clk_FEN = 0;
00116
00117     SYS_InitStruct->WDT_Ena = DISABLE;
00118     SYS_InitStruct->PORFilter_Ena = DISABLE;
00119 }
00120
00144 void SYS_ClearRst(void)
00145 {
00146     SYS->WR_PROTECT = 0x7a83;
```



```

00147     SYS->SFT_RST      = 0x0;
00148     SYS->WR_PROTECT = 0;
00149 }
00150
00184 void SYS.ModuleClockCmd(uint32_t nModule, FuncState state)
00185 {
00186     SYS->WR_PROTECT = 0x7A83; // 解除系统寄存器写保护
00187
00188     if (state == ENABLE)
00189     {
00190         SYS->CLK_FEN |= nModule;
00191     }
00192     else
00193     {
00194         SYS->CLK_FEN &= ~nModule;
00195     }
00196     SYS->WR_PROTECT = 0x0; // 系统寄存器写保护
00197 }
00198
00236 void SYS.AnalogModuleClockCmd(uint32_t nModule, FuncState state)
00237 {
00238     SYS->WR_PROTECT = 0x7A83; // 解除系统寄存器写保护
00239
00240     if (state != DISABLE)
00241     {
00242         AFE->REG5 |= nModule;
00243     }
00244     else
00245     {
00246         AFE->REG5 &= ~nModule;
00247     }
00248 }
00249
00284 void SYS.SoftResetModule(uint32_t nModule)
00285 {
00286     SYS->WR_PROTECT = 0x7A83;
00287     SYS->SFT_RST      = nModule;
00288     SYS->WR_PROTECT = 1;
00289 }
00290
00313 void SYS.SoftReset_all(uint32_t state)
00314 {
00315     if (state == 1)
00316     {
00317         SYS->WR_PROTECT = 0x7A83;
00318         SYS->SFT_RST      = 0xffffffff;
00319         SYS->WR_PROTECT = 1;
00320     }
00321     else
00322     {
00323         SYS->WR_PROTECT = 0x7A83;
00324         SYS->SFT_RST      = 0x0;
00325         SYS->WR_PROTECT = 1;
00326     }
00327 }
00328
00348 static void SoftDelay(u32 cnt)
00349 {
00350     volatile u32 t_cnt;
00351     for (t_cnt = 0; t_cnt < cnt; t_cnt++)
00352     {
00353         __NOP();
00354     }
00355 }
00356
00391 void SYS.MclkChoice(u8 SYS_MCLK_x)
00392 {
00393     u32 reg = 0;
00394     u16 versionnum;
00395     SYS->WR_PROTECT = 0x7A83;
00396     versionnum      = Read.Trim(0x000014E4);
00397     if (versionnum >= 307)
00398     {
00399         #if POWER_MODE
00400             REG32(0x40000040) = Read.Trim(0x1410);
00401             for (reg = 0; reg < 0x20; reg++)
00402                 __ASM("NOP");
00403             reg = 0;
00404         #endif
00405     }
00406     SYS->CLK_CFG = 0; // 确保MCU工作在HRC
00407     SoftDelay(0x264F);
00408     switch (SYS_MCLK_x)
00409     {
00410         case SYS_MCLK_96M_RC:
00411         {
00412             AFE->REG5 |= (BIT15); // 打开PLL RCH default open

```

```

00413         AFE->REG6 &= ~BIT15; // PLL来源于内部8Mhz
00414         reg      = (1 << 8) | (0xff);
00415         sys_clk   = 96000000;
00416         break;
00417     }
00418     case SYS_MCLK_48M_RC:
00419     {
00420         AFE->REG5 |= (BIT15); // 打开PLL RCH default open
00421         AFE->REG6 &= ~BIT15; // PLL来源于内部8Mhz
00422         reg      = (1 << 8) | (0x55);
00423         sys_clk   = 48000000;
00424         break;
00425     }
00426     case SYS_MCLK_24M_RC:
00427     {
00428         AFE->REG5 |= (BIT15); // 打开PLL RCH default open
00429         AFE->REG6 &= ~BIT15; // PLL来源于内部8Mhz
00430         reg      = (1 << 8) | (0x11);
00431         sys_clk   = 24000000;
00432         break;
00433     }
00434     case SYS_MCLK_12M_RC:
00435     {
00436         AFE->REG5 |= (BIT15); // 打开PLL RCH default open
00437         AFE->REG6 &= ~BIT15; // PLL来源于内部8Mhz
00438         reg      = (1 << 8) | (0x01); // main clk from LSI
00439         sys_clk   = 12000000;
00440         break;
00441     }
00442     case SYS_MCLK_32K_RC:
00443     {
00444         AFE->REG5 |= (BIT15); // 打开PLL RCH default open
00445         AFE->REG6 &= ~BIT15; // PLL来源于内部8Mhz
00446         reg      = (2 << 8) | (0xFF);
00447         sys_clk   = 32000;
00448         break;
00449     }
00450     case SYS_MCLK_96M_XTAL:
00451     {
00452         // 1.使用晶体时钟的时候, 首先将主时钟切为HRC
00453         SYS->CLK_CFG &= 0xfCff; // 将第8 9 位为置0, 主时钟来源HRC
00454         // 2.开启XTAL PLL
00455         AFE->REG5 |= (BIT15 | BIT14); // 打开PLL 晶振开关
00456         AFE->REG6 |= BIT15; // PLL来源于外部晶振
00457         reg      = (1 << 8) | (0xFF);
00458         sys_clk   = 96000000;
00459         break;
00460     }
00461     case SYS_MCLK_48M_XTAL:
00462     {
00463         // 1.使用晶体时钟的时候, 首先将主时钟切为HRC
00464         SYS->CLK_CFG &= 0xfCff; // 将第8 9 位为置0, 主时钟来源HRC
00465         // 2.开启XTAL PLL
00466         AFE->REG5 |= (BIT15 | BIT14); // 打开PLL 晶振开关
00467         reg      = (1 << 8) | (0x55);
00468         sys_clk   = 48000000;
00469         break;
00470     }
00471     case SYS_MCLK_24M_XTAL:
00472     {
00473         // 1.使用晶体时钟的时候, 首先将主时钟切为HRC
00474         SYS->CLK_CFG &= 0xfCff; // 将第8 9 位为置0, 主时钟来源HRC
00475         // 2.开启XTAL PLL
00476         AFE->REG5 |= (BIT15 | BIT14); // 打开PLL 晶振开关
00477         reg      = (1 << 8) | (0x11);
00478         sys_clk   = 24000000;
00479         break;
00480     }
00481     case SYS_MCLK_12M_XTAL:
00482     {
00483         // 1.使用晶体时钟的时候, 首先将主时钟切为HRC
00484         SYS->CLK_CFG &= 0xfCff; // 将第8 9 位为置0, 主时钟来源HRC
00485         // 2.开启XTAL PLL
00486         AFE->REG5 |= (BIT15 | BIT14); // 打开PLL 晶振开关
00487         reg      = (1 << 8) | (0x01);
00488         sys_clk   = 12000000;
00489         break;
00490     }
00491 }
00492 // 主时钟切换位PLL时, 延时100us
00493 SoftDelay(0x264F); // 1个_NOP大约按一个系统时钟周期计算
00494 SYS->CLK_CFG = reg; // 选择时钟来源和分频
00495 SoftDelay(0x264F);
00496 SYS->WR_PROTECT = 0x0;
00497 }
00498
00517 u32 SYS_ReadMcuClk(void)

```

```

00518 {
00519     return sys_clk;
00520 }
00521
00541 void SYS_UartClkDiv(u16 div)
00542 {
00543     SYS->WR.PROTECT = 0x7a83;
00544     SYS->CLK_DIV2 = div;
00545     SYS->WR.PROTECT = 0x0;
00546 }
00547
00567 void SYS_I2CClkDiv(u16 div)
00568 {
00569     SYS->WR.PROTECT = 0x7a83;
00570     SYS->CLK_DIV0 = div;
00571     SYS->WR.PROTECT = 0x0;
00572 }
00573
00593 void SYS_SPIClkDiv(u16 div)
00594 {
00595     SYS->WR.PROTECT = 0x7a83;
00596     SYS->CLK_DIV0 = div;
00597     SYS->WR.PROTECT = 0x0;
00598 }
00599
00625 void IWDG_Init(IWDG_InitTypeDef *this)
00626 {
00627     this->RTH = (this->RTH > BIT21) ? BIT21 - 1 : this->RTH;
00628     this->WTH = (this->WTH > BIT21) ? BIT21 - 1 : this->WTH;
00629     IWDG->CFG = (this->DWK_EN << 4) | (this->WDG_EN);
00630     IWDG->PSW = PSW_IWDG_PRE;
00631     IWDG->RTH = this->RTH;
00632     if (this->RTH > this->WTH)
00633     {
00634         IWDG->WTH = this->RTH - this->WTH;
00635     }
00636     else
00637     {
00638         IWDG->WTH = 0x001000;
00639     }
00640     IWDG->PSW = PSW_IWDG_PRE;
00641     IWDG->CLR = PSW_IWDG_CLR;
00642 }
00643
00664 void IWDG_StructInit(IWDG_InitTypeDef *this)
00665 {
00666     this->DWK_EN = DISABLE;           // 深度休眠定时唤醒使能
00667     this->WDG_EN = DISABLE;           // 独立看门狗使能
00668     this->WTH = SECOND2IWDGCNT(1);    // 看门狗定时唤醒时间, 大于复位时间后无效
00669     this->RTH = SECOND2IWDGCNT(2);    // 看门狗超时复位时间 (21位计数器, 但低12恒位0)
00670                                     // SECOND2IWDGCNT输入参数, 秒
00671 }
00672
00692 void IWDG_DISABLE(void)
00693 {
00694     IWDG->CFG = 0x3c00;
00695 }
00696
00716 void IWDG_ENABLE(void)
00717 {
00718     IWDG->CFG = 0x1;
00719 }
00720
00740 void IWDG_Feed(void)
00741 {
00742     if ((SYS_CLK_CFG & (SYS_CLK_CFG_CLK_SEL_MASK | SYS_CLK_CFG_CLK_DIV_MASK)) == 0x1ff)
00743     {
00744         // 全速运行时直接喂狗
00745         IWDG->PSW = PSW_IWDG_PRE;
00746         IWDG->CLR = PSW_IWDG_CLR;
00747     }
00748     else
00749     {
00750         int32_t flg = __get_PRIMASK();
00751         uint32_t clkcfg;
00752         __disable_irq();
00753         SYS_WR_PROTECT = 0x7a83;
00754
00755         clkcfg = SYS_CLK_CFG;
00756
00757         SYS_CLK_CFG |= 0xff;
00758         IWDG->PSW = PSW_IWDG_PRE;
00759         IWDG->CLR = PSW_IWDG_CLR;
00760         SYS_CLK_CFG = clkcfg;
00761
00762         SYS_WR_PROTECT = 0;
00763     }

```

```

00764     __set_PRIMASK(flag);
00765 }
00766 }
00771 void SYS_VolSelModule(uint32_t SYS_MCU_POWER_x)
00772 {
00773
00774     if (SYS_MCU_POWER_x == MCU_POWER_5v0) // 5.0 Voltage
00775     {
00776         SYS_WR_PROTECT = 0x7a83; // 解除系统寄存器写保护 */
00777         SYS_AFE_REG6 &= ~(BIT8 | BIT9); // 恢复初始配置
00778         SoftDelay(100); // 延时100us*/
00779         SYS_AFE_REG6 |= SYS_POWER_3V75 << 8; // 配置为3.75V监控, 可根据实际配置其它档
            位: 3.25V/3.5V/3.75V/4.0V
00780         SoftDelay(100); // 延时100us, 等待电源检测配置完毕*/
00781         while (SYS_AFE_DBG & BIT15) // 等待外部5VOK
00782         {
00783         };
00784         SYS_AFE_REG6 &= ~(BIT8 | BIT9); // 恢复初始配置
00785         SoftDelay(100); // 延时100us*/
00786         SYS_AFE_REG6 |= SYS_POWER_3V25 << 8; // 配置为3.25V, 防止电源和欠压比较的抖动
00787         SoftDelay(100); // 延时100us*/
00788         NVIC_ClearPendingIRQ(PWRDN_IRQn);
00789         SYS_WR_PROTECT = 0x0; // 关闭系统寄存器写操作*/
00790     }
00791     else if ((SYS_AFE_INFO & 0xf) >= 4) // 芯片版本大于等于4时, 可以用电源掉电检测电路监测3.3v的电压做欠压保护, 小
            于该版本无法检测3.3v电压
00792     {
00793         SYS_WR_PROTECT = 0x7a83; // 解除系统寄存器写保护
00794         SYS_AFE_REG6 &= ~(BIT8 | BIT9); // 恢复初始配置
00795         SYS_AFE_REG6 |= SYS_POWER_2V5 << 8; // 配置为2.5V欠压保护
00796         SoftDelay(100); // 延时100us
00797         while (SYS_AFE_DBG & BIT15) // 等待外部2.5VOK
00798         {
00799         };
00800         SoftDelay(100); // 延时100us
00801         NVIC_ClearPendingIRQ(PWRDN_IRQn);
00802         SYS_WR_PROTECT = 0x0; // 关闭系统寄存器写操作*/
00803     }
00804 }
00805
00809 void SYS_VolSelModuleIRQ(void)
00810 {
00811     uint32_t cnt;
00812     NVIC_DisableIRQ(PWRDN_IRQn); // 关闭电源检测中断 */
00813     NVIC_ClearPendingIRQ(PWRDN_IRQn);
00814     // turn off pwm
00815     SYS_WR_PROTECT = 0x7a83; // 时钟选HRC */
00816     SYS_CLK_CFG = 0x0000;
00817     for (cnt = 0; cnt < 4000; cnt++)
00818     {
00819         __NOP();
00820     }
00821     SYS_AFE_REG6 &= ~(BIT8 | BIT9); // this clause reset to 4.0v
00822     SYS_WR_PROTECT = 0x0000;
00823     MCPWM0_PRT = 0xDEAD; // 关电机 */
00824     MCPWM0_FAIL012 &= (~0x0040);
00825     MCPWM0_FAIL345 &= (~0x0040);
00826     MCPWM0_PRT = 0x0;
00827     NVIC_SystemReset(); // 软复位
00828 }
00832 void SYS_VolSelModuleEnableIRQ(uint32_t SYS_MCU_POWER_x)
00833 {
00834     if (SYS_MCU_POWER_x == MCU_POWER_5v0)
00835     {
00836         NVIC_SetPriority(PWRDN_IRQn, 0); // PWRDN_IRQn中断优先级设置为0
00837         NVIC_EnableIRQ(PWRDN_IRQn); // 打开电源检测中断
00838     }
00839     else if ((SYS_AFE_INFO & 0xf) >= 4) // 芯片版本大于等于4时, 可以用电源掉电检测电路监测3.3v的电压做欠压保护, 小
            于该版本无法检测3.3v电压
00840     {
00841         NVIC_SetPriority(PWRDN_IRQn, 0); // PWRDN_IRQn中断优先级设置为0
00842         NVIC_EnableIRQ(PWRDN_IRQn); // 打开电源检测中断
00843     }
00844     else
00845     {
00846         NVIC_DisableIRQ(PWRDN_IRQn); // 关闭电源检测中断
00847     }
00848 }
00849 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/

```

5.89 source/lks32mc07x_tim.c 文件参考

文件名称: [lks32mc07x_tim.c](#)

文件标识: 无

内容摘要: 定时器外设驱动程序

其它说明: 无

```
#include "lks32mc07x_tim.h"
```

```
#include "lks32mc07x_sys.h"
```

[lks32mc07x_tim.c](#) 的引用(Include)关系图:



函数

- void [TIM.TimerInit](#) (TIM.TimerTypeDef *TIMERx, TIM.TimerInitTypeDef *this)
函数名称: void [TIM.TimerInit\(TIM.TimerTypeDef *TIMERx, TIM.TimerInitTypeDef *this\)](#)
- void [TIM.TimerStrutInit](#) (TIM.TimerInitTypeDef *this)
函数名称: void [TIM.TimerStrutInit\(TIM.TimerInitTypeDef *this\)](#)
- void [TIM.TimerCmd](#) (TIM.TimerTypeDef *TIMERx, FuncState state)
函数名称: void [TIM.TimerCmd\(TIM.TimerTypeDef *TIMERx, FuncState state\)](#)
- void [TIM.CMP0](#) (TIM.TimerTypeDef *TIMERx, uint32_t temp)
函数名称: void [TIM.CMP0\(TIM.TimerTypeDef *TIMERx, uint32_t temp\)](#)
- void [TIM.CMP1](#) (TIM.TimerTypeDef *TIMERx, uint32_t temp)
函数名称: void [TIM.CMP1\(TIM.TimerTypeDef *TIMERx, uint32_t temp\)](#)
- u32 [TIM.Timer_GetCMPT0](#) (TIM.TimerTypeDef *TIMERx)
函数名称: u32 [TIM.Timer_GetCMPT0\(TIM.TimerTypeDef *TIMERx\)](#)
- u32 [TIM.Timer_GetCMPT1](#) (TIM.TimerTypeDef *TIMERx)
函数名称: u32 [TIM.Timer_GetCMPT1\(TIM.TimerTypeDef *TIMERx\)](#)
- void [TIM.ClearIRQFlag](#) (TIM.TimerTypeDef *TIMERx, uint32_t tempFlag)
函数名称: void [TIM.ClearIRQFlag\(TIM.TimerTypeDef *TIMERx, uint32_t tempFlag\)](#)
- u16 [TIM.GetIRQFlag](#) (TIM.TimerTypeDef *UTIMERx, u32 TIMER_IF_x)
函数名称: u16 [TIM.GetIRQFlag\(TIM.TimerTypeDef *UTIMERx, u32 TIMER_IF_x\)](#)
- u16 [TIM.GetFlag](#) (TIM.TimerTypeDef *UTIMERx, u32 TIMER_IF_x)
函数名称: u16 [TIM.GetFlag\(TIM.TimerTypeDef *UTIMERx, u32 TIMER_IF_x\)](#)
- uint32_t [TIM.Timer_GetCount](#) (TIM.TimerTypeDef *TIMERx)
函数名称: uint32_t [TIM.Timer_GetCount\(TIM.TimerTypeDef *TIMERx\)](#)
- void [TIM.Enable](#) (TIM.TimerTypeDef *UTIMERx)
函数名称: void [TIM.Enable\(TIM.TimerTypeDef *UTIMERx\)](#)
- void [TIM.Disable](#) (TIM.TimerTypeDef *UTIMERx)
函数名称: void [TIM.Disable\(TIM.TimerTypeDef *UTIMERx\)](#)
- void [UTIMER.Enable](#) (TIM.TimerTypeDef *UTIMERx)
- void [UTIMER.Disable](#) (TIM.TimerTypeDef *UTIMERx)

5.89.1 详细描述

文件名称: [lks32mc07x_tim.c](#)
文件标识: 无
内容摘要: 定时器外设驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 中定义.

5.89.2 函数说明

5.89.2.1 TIM_ClearIRQFlag()

```
void TIM_ClearIRQFlag (
    TIM_TimerTypeDef * TIMeRx,
    uint32_t tempFlag )
```

函数名称: void [TIM_ClearIRQFlag\(TIM_TimerTypeDef *TIMeRx ,uint32_t tempFlag\)](#)

功能描述: 清除TIM中断标志

参见
被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0	
	timer_if:	
	Timer_IRQEna_CH0_RE	通道0比较/捕获DMA请求使能
	Timer_IRQEna_CH1_RE	通道1比较/捕获DMA请求使能
	Timer_IRQEna_ZC_RE	计数器过0 DMA请求使能
	Timer0_IRQEna_FAIL_RE	定时器0, FAIL事件DMA请求使能
	Timer_IRQEna_CH1	Timer CH1中断, 比较或捕获中断
	Timer_IRQEna_CH0	Timer CH0中断, 比较或捕获中断
	Timer_IRQEna_ZC	Timer 过零中断
输出参数:	无	

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

```
if(TIM_GetIRQFlag(UTIMER0,Timer_IRQEna_CH0)) //判断UTimer0的CH0是否发生比较中断
{
    TIM_ClearIRQFlag(UTIMER0,Timer_IRQEna_CH0); //清除UTimer0通道0比较中断标志位
}
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 338 行定义.

5.89.2.2 TIM_CMP0()

```
void TIM_CMP0 (
    TIM_TimerTypeDef *TIMERx,
    uint32_t temp )
```

函数名称: void [TIM_CMP0\(TIM_TimerTypeDef *TIMERx, uint32_t temp\)](#)

功能描述: 写入定时器通道0比较值

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0 temp: 比较数值
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
TIM_CMP0(UTIMER0, 2000); //设定定时器0通道0比较值为2000
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 226 行定义.

5.89.2.3 TIM_CMP1()

```
void TIM_CMP1 (
    TIM_TimerTypeDef * TIMERx,
    uint32_t temp )
```

函数名称: void [TIM_CMP1\(TIM_TimerTypeDef *TIMERx, uint32_t temp\)](#)

功能描述: 写入定时器通道1比较值

参见

被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0 temp: 比较数值
输出参数:	无

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

```
TIM_CMP1(UTIMER0 , 2000); //设定定时器0通道1比较值为2000
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 251 行定义.

5.89.2.4 TIM_Disable()

```
void TIM_Disable (
    TIM_TimerTypeDef * UTIMERx )
```

函数名称: void [TIM_Disable\(TIM_TimerTypeDef *UTIMERx\)](#)

功能描述: 定时器时钟失能函数

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0
输出参数:	无

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

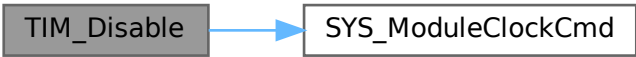
```
TIM_Disable(UTIMER0); //定时器0时钟失能
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 lks32mc07x_tim.c 第 501 行定义.

函数调用图:



5.89.2.5 TIM_Enable()

```
void TIM_Enable (
    TIM_TimerTypeDef * UTIMERx )
```

函数名称: void TIM_Enable(TIM_TimerTypeDef *UTIMERx)

功能描述: 定时器时钟使能函数

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
TIM_Enable(UTIMER0); //定时器0时钟使能
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 463 行定义.

函数调用图:



5.89.2.6 TIM_GetFlag()

```
u16 TIM_GetFlag (
    TIM_TimerTypeDef * UTIMERx,
    u32 TIMER_IF_x )
```

函数名称: u16 [TIM_GetFlag\(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x\)](#)

功能描述: 获取TIM中断标志

参见

被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0		
	TIMER_IF_x:		
	TIMER_IF_CH1	Timer CH1中断, 比较或捕获中断	
	TIMER_IF_CH0	Timer CH0中断, 比较或捕获中断	
	TIMER_IF_ZERO	Timer 过零中断	
输出参数:	无		

返回

返回 值: 0或1，对应中断标志置位返回1，未置位返回0

注解

其它说明: 无

警告

不需要对应中断使能后，就可以读取对应中断标志位

示例代码:

```
if(TIM_GetFlag(UTIMER0,TIMER_IF_CH0)) //判断UTimer0的CH0是否发生比较中断
{
}
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 407 行定义.

5.89.2.7 TIM_GetIRQFlag()

```
u16 TIM_GetIRQFlag (
    TIM_TimerTypeDef * UTIMERx,
    u32 TIMER_IF_x )
```

函数名称: u16 [TIM_GetIRQFlag\(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x\)](#)

功能描述: 获取TIM中断标志

参见

被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0		
	TIMER_IF_x:		
	TIMER_IF_CH1	Timer CH1中断, 比较或捕获中断	
	TIMER_IF_CH0	Timer CH0中断, 比较或捕获中断	
	TIMER_IF_ZERO	Timer 过零中断	
输出参数:	无		

返回

返回 值: 0或1, 对应中断标志置位返回1, 未置位返回0

注解

其它说明: 无

警告

只有对应中断使能后, 改为才能读取, 如果对应中断未使能, 读取结果一直为0

示例代码:

```
if(TIM_GetIRQFlag(UTIMER0,TIMER_IF_CH0)) //判断UTimer0的CH0是否发生比较中断
{
}
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 369 行定义.

5.89.2.8 TIM_Timer_GetCMPT0()

```
u32 TIM_Timer_GetCMPT0 (
    TIM_TimerTypeDef * TIMERx )
```

函数名称: u32 [TIM_Timer_GetCMPT0](#)([TIM_TimerTypeDef](#) *[TIMERx](#))

功能描述: 获取定时器通道0捕获值

参见

被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0
输出参数:	无

返回

返回 值: 定时器通道0捕获值

注解

其它说明: 当定时器发生捕获事件时, 将捕获时刻的cnt值存储到该寄存器中

警告

无

示例代码:

```
uint32_t TIMER0_CAPValue = 0;  
TIMER0_CAPValue = TIM_Timer_GetCMPT0(UTIMER0); //获取定时器0捕获值
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 277 行定义.

5.89.2.9 TIM_Timer_GetCMPT1()

```
u32 TIM_Timer_GetCMPT1 (  
    TIM_TimerTypeDef * TIMERx )
```

函数名称: u32 [TIM_Timer_GetCMPT1\(TIM_TimerTypeDef *TIMERx\)](#)

功能描述: 获取定时器通道1捕获值

参见

被引用内容: 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0
输出参数:	无

返回

返回 值： 定时器通道1捕获值

注解

其它说明： 当定时器发生捕获事件时，将捕获时刻的cnt值存储到该寄存器中

警告

无

示例代码:

```
uint32_t TIMeR0_CAPValue = 0;
TIMeR0_CAPValue = TIM.Timer_GetCMPt1(UTIMeR0); //获取定时器0捕获值
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 302 行定义.

5.89.2.10 TIM_Timer_GetCount()

```
uint32_t TIM_Timer_GetCount (
    TIM_TimerTypeDef * TIMeRx )
```

函数名称: uint32_t [TIM_Timer_GetCount\(TIM_TimerTypeDef *TIMeRx\)](#)

功能描述: 获取Timer计数值

参见

被引用内容： 无

参数

输入参数:	TIMeRx: UTIMeR0/UTIMeR0/UTIMeR0/UTIMeR0
输出参数:	无

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

```
uint32_t TIMER0.Value = 0;
TIMER0.Value = TIM_Timer_GetCount (UTIMER0); //获取定时器0计数值
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 439 行定义.

5.89.2.11 TIM_TimerCmd()

```
void TIM_TimerCmd (
    TIM_TimerTypeDef * TIMERx,
    FuncState state )
```

函数名称: void [TIM_TimerCmd\(TIM_TimerTypeDef *TIMERx, FuncState state\)](#)

功能描述: 定时器TIMERx使能和停止

参见

被引用内容： 无

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0 FuncState state: 使能与失能
输出参数:	无

返回

返回值： 无

注解

其它说明： 无

警告

无

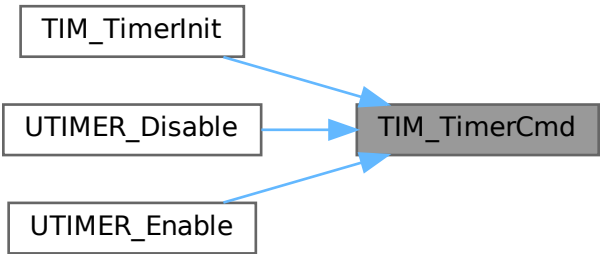
示例代码:

```
TIM_TimerCmd(UTIMER0, ENABLE); // Timer0模块使能
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 [lks32mc07x_tim.c](#) 第 194 行定义.
这是这个函数的调用关系图:



5.89.2.12 TIM_TimerInit()

```
void TIM_TimerInit (
    TIM_TimerTypeDef * TIMERx,
    TIM_TimerInitTypeDef * this )
```

函数名称: void [TIM_TimerInit\(TIM_TimerTypeDef *TIMERx, TIM_TimerInitTypeDef *this\)](#)

功能描述: 定时器初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	TIMERx: UTIMER0/UTIMER0/UTIMER0/UTIMER0 TIM_TimerInitTypeDef *TIM_TimerInitStruct
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
事例1比较模式配置:
TIM_TimerInitTypeDef TIM_InitStruct;
TIM_TimerStrutInit(&TIM_InitStruct); // Timer结构体初始化
TIM_InitStruct.Timer_CH0WorkMode = TIMER_OPMODE_CMP; // 设置Timer CH0 为比较模式
TIM_InitStruct.Timer_CH0Output = 0; // 计数器回零时, 比较模式输出极性控制
TIM_InitStruct.Timer_CH1WorkMode = TIMER_OPMODE_CMP; // 设置Timer CH1 为比较模式
TIM_InitStruct.Timer_CH1Output = 0; // 计数器回零时, 比较模式输出极性控制
TIM_InitStruct.Timer_TH = 48000; // 定时器计数门限初始值48000
TIM_InitStruct.Timer_CMP0 = 24000; // 设置比较模式的CH0比较初始值24000
TIM_InitStruct.Timer_CMP1 = 24000; // 设置比较模式的CH1比较初始值24000
TIM_InitStruct.Timer_ClockDiv = TIMER_CLK_DIV1; // 设置Timer模块时钟2分频系数
TIM_InitStruct.Timer_IRQEna = Timer_IRQEna_CH0 | Timer_IRQEna_CH1 | Timer_IRQEna_ZC; // 开启Timer模块比
较中断和过零中断
TIM_TimerInit(UTIMER0, &TIM_InitStruct);
TIM_TimerCmd(UTIMER0, ENABLE); // 定时器2模块使能

事例2捕获模式配置:
TIM_TimerInitTypeDef TIM_InitStruct;
TIM_TimerStrutInit(&TIM_InitStruct); // Timer结构体初始化
TIM_InitStruct.Timer_SRC0 = CAP_SRC_TCH1; // 定时器0通道0捕获定时器0通道1的输入信号
TIM_InitStruct.Timer_CH0WorkMode = TIMER_OPMODE_CAP; // 设置Timer CH0为捕获模式
TIM_InitStruct.Timer_CAP0_CLR_EN = ENABLE; // 当发生CAP0捕获事件时, 清零Timer计数器
TIM_InitStruct.Timer_CH0_RE_CAP_EN = ENABLE; // 使能Timer通道0上升沿捕获
TIM_InitStruct.Timer_CH0_FE_CAP_EN = DISABLE; // 关闭Timer通道0下降沿捕获

TIM_InitStruct.Timer_SRC1 = CAP_SRC_TCH1; // 定时器0通道1捕获定时器0通道1的输入信号
TIM_InitStruct.Timer_CH1WorkMode = TIMER_OPMODE_CAP; // 设置Timer CH1为捕获模式
TIM_InitStruct.Timer_CAP1_CLR_EN = DISABLE; // 当发生CAP1捕获事件时, 清零Timer计数器
TIM_InitStruct.Timer_CH1_RE_CAP_EN = DISABLE; // 使能Timer通道1上升沿捕获
TIM_InitStruct.Timer_CH1_FE_CAP_EN = ENABLE; // 关闭Timer通道1下降沿捕获

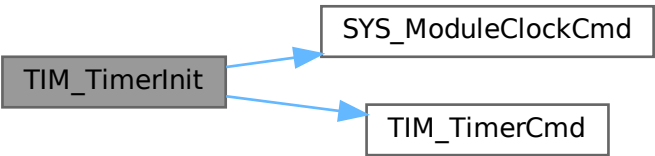
TIM_InitStruct.Timer_TH = 96000000; // 定时器计数门限初始值96000000
TIM_InitStruct.Timer_FLT = 0; // 设置捕捉模式或编码器模式下对应通道的数字滤波值
TIM_InitStruct.Timer_ClockDiv = TIMER_CLK_DIV1; // 设置Timer模块时钟2分频系数
TIM_InitStruct.Timer_IRQEna = Timer_IRQEna_ZC; // 开启Timer模块过零中断
TIM_TimerInit(UTIMER3, &TIM_InitStruct);
TIM_TimerCmd(UTIMER3, ENABLE); // Timer3 模块使能
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志: 去除定时器使能接口

在文件 [lks32mc07x_tim.c](#) 第 71 行定义.

函数调用图:



5.89.2.13 TIM_TimerStrutInit()

```
void TIM_TimerStrutInit (
    TIM_TimerInitTypeDef * this )
```

函数名称: void [TIM_TimerStrutInit\(TIM_TimerInitTypeDef *this\)](#)

功能描述: 定时器结构体初始化

参见
被引用内容: 无

参数

输入参数:	TIM_TimerInitTypeDef *this
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
TIM_TimerInitTypeDef TIM_InitStructure;  
TIM_TimrStrutInit(&TIM_InitStructure); // Timer结构体初始化
```

Date	Version	Author	Description
2023年2月22日	1.0	HungMG	创建

修改日志:

在文件 lks32mc07x_tim.c 第 124 行定义.

5.89.2.14 UTIMER_Disable()

```
void UTIMER_Disable (  
    TIM_TimerTypeDef * UTIMERx )
```

在文件 lks32mc07x_tim.c 第 525 行定义.

函数调用图:



5.89.2.15 UTIMER_Enable()

```
void UTIMER_Enable (  
    TIM_TimerTypeDef * UTIMERx )
```

在文件 lks32mc07x_tim.c 第 521 行定义.

函数调用图:



5.90 lks32mc07x_tim.c

[浏览该文件的文档.](#)

```

00001
00014 #include "lks32mc07x_tim.h"
00015 #include "lks32mc07x_sys.h"
00016
00071 void TIM.TimerInit(TIM.TimerTypeDef *TIMERx, TIM.TimerInitTypeDef *this)
00072 {
00073     if (TIMERx == UTIMER0)
00074     {
00075         SYS.ModuleClockCmd(SYS.Module_TIMER0, ENABLE);
00076     }
00077     else if (TIMERx == UTIMER1)
00078     {
00079         SYS.ModuleClockCmd(SYS.Module_TIMER1, ENABLE);
00080     }
00081     else if (TIMERx == UTIMER2)
00082     {
00083         SYS.ModuleClockCmd(SYS.Module_TIMER2, ENABLE);
00084     }
00085     else if (TIMERx == UTIMER3)
00086     {
00087         SYS.ModuleClockCmd(SYS.Module_TIMER3, ENABLE);
00088     }
00089
00090     TIMERx->TH = this->Timer.TH;
00091     TIMERx->CMP0 = this->Timer.CMP0;
00092     TIMERx->CMP1 = this->Timer.CMP1;
00093     TIMERx->EVT = this->Timer.EVT;
00094     TIMERx->FLT = this->Timer.FLT;
00095     TIMERx->IE = this->Timer.IRQEna;
00096
00097     TIMERx->IO = this->Timer.CH1.DEFAULT << 9 | this->Timer.CH0.DEFAULT << 8 | this->Timer.HALT_PRT
<< 7 | this->Timer0.FAIL_SEL << 2 | this->Timer0.FAIL_POL << 1 | this->Timer0.FAIL_EN;
00098
00099     TIMERx->CFG = (u32)(this->Timer.CAP1.CLR_EN << 29) | (u32)(this->Timer.CAP0.CLR_EN << 28) |
(u32)(this->Timer0.UPDATE << 27) | (u32)(this->Timer0.SHADOW << 26) | (u32)(this->Timer.ONE_TRIG << 25)
| (u32)(this->Timer.CENTER << 24) | (u32)(this->Timer.DIR << 23) | (u32)(this->Timer.ClockDiv << 20) |
(u32)(this->Timer.ETON << 19) | (u32)(this->Timer.GATE_EN << 18) | (u32)(this->Timer.RL_EN << 17) |
(u32)(this->Timer.XCLK_EN << 16) | (u32)(this->Timer.SRC1 << 12) | (u32)(this->Timer.CH1Output << 11) |
(u32)(this->Timer.CH1.WorkMode << 10) | (u32)(this->Timer.CH1.FE.CAP_EN << 9) |
(u32)(this->Timer.CH1.RE.CAP_EN << 8) | (u32)(this->Timer.SRC0 << 4) | (u32)(this->Timer.CH0Output << 3)
| (u32)(this->Timer.CH0.WorkMode << 2) | (u32)(this->Timer.CH0.FE.CAP_EN << 1) |
(u32)(this->Timer.CH0.RE.CAP_EN);
00100
00101     TIM.TimerCmd(TIMERx, this->Timer.EN);
00102 }
00103
00124 void TIM.TimerStrutInit(TIM.TimerInitTypeDef *this)
00125 {
00126     this->Timer.CAP1.CLR_EN = DISABLE;
00127     this->Timer.CAP0.CLR_EN = DISABLE;
00128
00129     this->Timer.ONE_TRIG = DISABLE;
00130
00131     this->Timer.CENTER = 0;
00132
00133     this->Timer.DIR = 0;
00134
00135     this->Timer.ClockDiv = TIMER.CLK_DIV1;
00136     this->Timer.ETON = 0;
00137     this->Timer.GATE_EN = 0;
00138     this->Timer.RL_EN = 0;
00139     this->Timer.XCLK_EN = 0;
00140
00141     this->Timer.SRC1 = 0;
00142     this->Timer.CH1Output = 0;
00143     this->Timer.CH1.WorkMode = 0;
00144     this->Timer.CH1.FE.CAP_EN = 0;
00145     this->Timer.CH1.RE.CAP_EN = 0;
00146
00147     this->Timer.SRC0 = 0;
00148     this->Timer.CH0Output = 0;
00149     this->Timer.CH0.WorkMode = 0;
00150     this->Timer.CH0.FE.CAP_EN = 0;
00151     this->Timer.CH0.RE.CAP_EN = 0;
00152
00153     this->Timer.TH = 0;
00154     this->Timer.CNT = 0;
00155     this->Timer.CMP0 = 0;
00156     this->Timer.CMP1 = 0;
00157
00158     this->Timer.EVT = 0;
00159     this->Timer.FLT = 0;

```

```

00160
00161     this->Timer.IRQEna = 0;
00162
00163     this->Timer0.UPDATE = 0;
00164     this->Timer0.SHADOW = 0;
00165
00166     this->Timer0.CH0.DEFAULT = 0;
00167     this->Timer0.CH1.DEFAULT = 0;
00168     this->Timer0.HALT_PRT = 0;
00169     this->Timer0.FAILSEL = 0;
00170     this->Timer0.FAILPOL = 0;
00171     this->Timer0.FAILLEN = 0;
00172 }
00173
00194 void TIM.TimerCmd(TIM_TimerTypeDef *TIMERx, FuncState state)
00195 {
00196     if (state != DISABLE)
00197     {
00198         TIMERx->CFG |= BIT31;
00199     }
00200     else
00201     {
00202         TIMERx->CFG &= ~BIT31;
00203     }
00204 }
00205
00226 void TIM.CMP0(TIM_TimerTypeDef *TIMERx, uint32_t temp)
00227 {
00228     TIMERx->CMP0 = temp;
00229 }
00230
00251 void TIM.CMP1(TIM_TimerTypeDef *TIMERx, uint32_t temp)
00252 {
00253
00254     TIMERx->CMP1 = temp;
00255 }
00256
00277 u32 TIM.Timer.GetCMPT0(TIM_TimerTypeDef *TIMERx)
00278 {
00279     return TIMERx->CMP0;
00280 }
00281
00302 u32 TIM.Timer.GetCMPT1(TIM_TimerTypeDef *TIMERx)
00303 {
00304     return TIMERx->CMP1;
00305 }
00306
00338 void TIM.ClearIRQFlag(TIM_TimerTypeDef *TIMERx, uint32_t tempFlag)
00339 {
00340     TIMERx->IF = tempFlag;
00341 }
00342
00369 u16 TIM.GetIRQFlag(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x)
00370 {
00371     if (UTIMERx->IF & UTIMERx->IE & TIMER_IF_x)
00372     {
00373         return 1;
00374     }
00375     else
00376     {
00377         return 0;
00378     }
00379 }
00380
00407 u16 TIM.GetFlag(TIM_TimerTypeDef *UTIMERx, u32 TIMER_IF_x)
00408 {
00409     if (UTIMERx->IF & TIMER_IF_x)
00410     {
00411         return 1;
00412     }
00413     else
00414     {
00415         return 0;
00416     }
00417 }
00418
00439 uint32_t TIM.Timer.GetCount(TIM_TimerTypeDef *TIMERx)
00440 {
00441     return TIMERx->CNT;
00442 }
00443
00463 void TIM.Enable(TIM_TimerTypeDef *UTIMERx)
00464 {
00465     if (UTIMERx == UTIMER0)
00466     {
00467         SYS.ModuleClockCmd(SYS.Module-TIMER0, ENABLE);
00468     }

```

```

00469     else if (UTIMERx == UTIMER1)
00470     {
00471         SYS.ModuleClockCmd(SYS.Module_TIMER1, ENABLE);
00472     }
00473     else if (UTIMERx == UTIMER2)
00474     {
00475         SYS.ModuleClockCmd(SYS.Module_TIMER2, ENABLE);
00476     }
00477     else if (UTIMERx == UTIMER3)
00478     {
00479         SYS.ModuleClockCmd(SYS.Module_TIMER3, ENABLE);
00480     }
00481 }
00501 void TIM_Disable(TIM_TimerTypeDef *UTIMERx)
00502 {
00503     if (UTIMERx == UTIMER0)
00504     {
00505         SYS.ModuleClockCmd(SYS.Module_TIMER0, DISABLE);
00506     }
00507     else if (UTIMERx == UTIMER1)
00508     {
00509         SYS.ModuleClockCmd(SYS.Module_TIMER1, DISABLE);
00510     }
00511     else if (UTIMERx == UTIMER2)
00512     {
00513         SYS.ModuleClockCmd(SYS.Module_TIMER2, DISABLE);
00514     }
00515     else if (UTIMERx == UTIMER3)
00516     {
00517         SYS.ModuleClockCmd(SYS.Module_TIMER3, DISABLE);
00518     }
00519 }
00520
00521 void UTIMER_Enable(TIM_TimerTypeDef *UTIMERx)
00522 {
00523     TIM_TimerCmd(UTIMERx, ENABLE);
00524 }
00525 void UTIMER_Disable(TIM_TimerTypeDef *UTIMERx)
00526 {
00527     TIM_TimerCmd(UTIMERx, DISABLE);
00528 }
00529 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.91 source/lks32mc07x_tmp.c 文件参考

文件名称: lks32mc08x_tmp.c

文件标识: 无

内容摘要: 芯片内置温度传感器外设驱动程序

其它说明: 无

```
#include "lks32mc07x_tmp.h"
```

```
#include "lks32mc07x_sys.h"
```

lks32mc07x_tmp.c 的引用(Include)关系图:



函数

- void [TempSensor_Init](#) (void)
函数名称: void [TempSensor_Init\(void\)](#)
- s16 [GetCurrentTemperature](#) (s16 ADC_value)
函数名称: s16 [GetCurrentTemperature\(s16 ADC_value\)](#)

变量

- [Stru_TempertureCof_Def](#) [m_TempertureCof](#)
温度传感器系数

5.91.1 详细描述

文件名称: [lks32mc08x_tmp.c](#)
文件标识: 无
内容摘要: 芯片内置温度传感器外设驱动程序
其它说明: 无

版权所有
(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年3月9日	1.0	Zhu Jie	修改

修改日志:

在文件 [lks32mc07x_tmp.c](#) 中定义.

5.91.2 函数说明

5.91.2.1 [GetCurrentTemperature\(\)](#)

```
s16 GetCurrentTemperature (
    s16 ADC_value )
```

函数名称: [s16 GetCurrentTemperature\(s16 ADC_value\)](#)

功能描述: 获取当前温度值

参见
被引用内容: 无

参数

输入参数:	ADC_value : ADC通道14为温度传感器，ADC采样结果值
输出参数:	无

返回

返回 值： t.Temperture： 当前温度值， 单位： 1个Lsb代表0.1度

注解

其它说明： ADC.value输入的ADC值不区分左右对齐， 函数内已经进行ADC结果左右对齐判断

警告

无

示例代码:

```
s16 temp_value = 0;
temp_value = GetCurrentTemperature(ADC0.DATA0); //获取当前温度值,单位0.1度
```

Date	Version	Author	Description
2015年11月5日	1.0	Howlet	创建
2023年3月9日	2.0	Zhu Jie	修改

修改日志:

在文件 [lks32mc07x_tmp.c](#) 第 70 行定义.

5.91.2.2 TempSensor.Init()

```
void TempSensor.Init (
    void )
```

函数名称: void [TempSensor.Init\(void\)](#)

功能描述: 温度传感器初始化

参见

被引用内容： 无

参数

输入参数:	无
输出参数:	无

返回

返回 值： 无

注解

其它说明： 无

警告

无

示例代码:

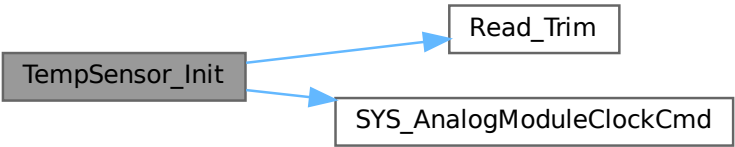
```
TempSensor.Init(); //温度传感器初始化
```

Date	Version	Author	Description
2015年11月5日	1.0	Howlet	创建
2023年3月9日	2.0	Zhu Jie	修改

修改日志:

在文件 lks32mc07x_tmp.c 第 39 行定义.

函数调用图:



5.91.3 变量说明

5.91.3.1 m_TempertureCof

```
Stru.TempertureCof.Def m_TempertureCof
```

温度传感器系数

在文件 lks32mc07x_tmp.c 第 17 行定义.

5.92 lks32mc07x_tmp.c

浏览该文件的文档.

```

00001
00015 #include "lks32mc07x_tmp.h"
00016 #include "lks32mc07x_sys.h"
00017 Stru.TemperatureCof_Def m_TemperatureCof;
00039 void TempSensor.Init(void)
00040 {
00041     SYS_WR_PROTECT = 0x7a83; /* 解除系统寄存器写保护 */
00042     SYS_AnalogModuleClockCmd(SYS_AnalogModule.TMP, ENABLE);
00043
00044     m_TemperatureCof.nCofA = Read.Trim(0x000014D0);
00045     m_TemperatureCof.nOffsetB = Read.Trim(0x000014D4);
00046     SYS_WR_PROTECT = 0x7a93; /* 开启系统寄存器写保护 */
00047 }
00048
00070 s16 GetCurrentTemperature(s16 ADC_value)
00071 {
00072     s16 t_Temperature;
00073     if (ADC1->CFG & BIT10)
00074     {
00075         t_Temperature = (m_TemperatureCof.nOffsetB - ((s32)m_TemperatureCof.nCofA * ADC_value) / 1000);
00076     }
00077     else
00078     {
00079         t_Temperature = (m_TemperatureCof.nOffsetB - ((s32)m_TemperatureCof.nCofA * (ADC_value >> 4)) /
00080         1000);
00081     }
00082     return t_Temperature;
00083 }
00084 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****/

```

5.93 source/lks32mc07x_uart.c 文件参考

文件名称: [lks32mc07x_uart.c](#)
 文件标识: 无
 内容摘要: UART外设驱动程序
 其它说明: 无

#include "lks32mc07x_lib.h"
 lks32mc07x_uart.c 的引用(Include)关系图:



宏定义

- #define MCU_MCLK 96000000

函数

- static void [UART_SetBaud](#) (uint32_t Uart0Baud, uint32_t Uart1Baud)
- static uint32_t [UART_ReadBaud](#) (UART_TypeDef *UARTx)
- void [UART_Init](#) (UART_TypeDef *UARTx, UART_InitTypeDef *UART_InitStruct)
 函数名称: void [UART_Init](#)(UART_TypeDef* UARTx, UART_InitTypeDef* UART_InitStruct)
- void [UART_StructInit](#) (UART_InitTypeDef *UART_InitStruct)

- 函数名称: void UART_StructInit(UART_InitTypeDef* UART_InitStruct)
- void UART_SendData (UART_TypeDef *UARTx, uint32_t n)

函数名称: void UART_SENDDATA(UART_TypeDef *UARTx, uint32_t n)
 - uint32_t UART_ReadData (UART_TypeDef *UARTx)

函数名称: uint32_t UART_ReadData(UART_TypeDef *UARTx)
 - uint32_t UART_GetIRQFlag (UART_TypeDef *UARTx, uint32_t tempFlag)

函数名称: uint32_t UART_GetIRQFlag(UART_TypeDef *UARTx , uint32_t tempFlag)
 - void UART_ClearIRQFlag (UART_TypeDef *UARTx, uint32_t tempFlag)

函数名称: void UART_ClearIRQFlag(UART_TypeDef *UARTx, uint32_t nFlag)
 - uint8_t UART_SendAddr_485 (UART_TypeDef *UARTx, uint8_t n)

函数名称: uint8_t UART_SendAddr_485(UART_TypeDef *UARTx, uint8_t n)
 - uint8_t UART_SendData_485 (UART_TypeDef *UARTx, uint8_t n)

函数名称: uint8_t UART_SendData_485(UART_TypeDef *UARTx, uint8_t n)

5.93.1 详细描述

文件名称: lks32mc07x_uart.c
文件标识: 无
内容摘要: UART外设驱动程序
其它说明: 无

版权所有
(C)2021, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 lks32mc07x_uart.c 中定义.

5.93.2 宏定义说明

5.93.2.1 MCU_MCLK

#define MCU_MCLK 96000000

在文件 lks32mc07x_uart.c 第 19 行定义.

5.93.3 函数说明

5.93.3.1 UART_ClearIRQFlag()

```
void UART_ClearIRQFlag (
    UART_TypeDef * UARTx,
    uint32_t tempFlag )
```

函数名称: void **UART_ClearIRQFlag**(UART_TypeDef *UARTx, uint32_t nFlag)

功能描述: 清除UART中断标志

参见

被引用内容: [UART0](#)

参数

输入参数:	UART_TypeDef: UART0	
	tempFlag参数可选:	
	宏定义	说明
	UART_IF_SendOver	发送完成中断
	UART_IF_RcvOver	接收完成中断
	UART_IF_SendBufEmpty	发送缓冲区空中断
	UART_IF_StopError	停止位错误
	UART_IF_CheckError	校验错误
	UART_IF_TX_OV	发送缓冲区溢出中断
	UART_IF_RX_OV	接收缓冲区溢出中断
	UART_IF_LBD	LIN break character 检测中断
	UART_IF_IDLE	空闲帧中断
输出参数:	无	

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
if (UART_GetIRQFlag (UART0, UART_IF_SendOver)) //获取发送完成中断标志位
{
    UART_ClearIRQFlag (UART0, UART_IF_SendOver) //清除发送完成中断标志位
}
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 254 行定义.

5.93.3.2 UART_GetIRQFlag()

```
uint32_t UART_GetIRQFlag (  
    UART_TypeDef * UARTx,  
    uint32_t tempFlag )
```

函数名称: `uint32_t UART_GetIRQFlag(UART_TypeDef *UARTx , uint32_t tempFlag)`

功能描述: 获取UART中断标志

参见

被引用内容: UARTx

参数

输入参数:	UART_TypeDef: UARTx	
	tempFlag参数可选:	
	宏定义	说明
	UART_IF_SendOver	发送完成中断
	UART_IF_RcvOver	接收完成中断
	UART_IF_SendBufEmpty	发送缓冲区空中断
	UART_IF_StopError	停止位错误
	UART_IF_CheckError	校验错误
	UART_IF_TX_OV	发送缓冲区溢出中断
	UART_IF_RX_OV	接收缓冲区溢出中断
输出参数:	UART_IF_LBD	LIN break character 检测中断
	UART_IF_IDLE	空闲帧中断
	无	

返回

返回值: 无

注解

其它说明: 无

警告

只有对应中断使能后，改为才能读取，如果对应中断未使能，读取结果一直为0

示例代码:

```
if (UART_GetIRQFlag (UART0, UART_IF_SendOver)) //获取发送完成中断标志位
{
    UART_ClearIRQFlag (UART0, UART_IF_SendOver) //清除发送完成中断标志位
}
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 212 行定义.

5.93.3.3 UART_Init()

```
void UART_Init (
    UART_TypeDef * UARTx,
    UART_InitTypeDef * UART_InitStruct )
```

函数名称: void UART_Init(UART_TypeDef* UARTx, EUART_InitTypeDef* UART_InitStruct)

功能描述: UART初始化函数

参见

被引用内容: [SYS_ModuleClockCmd\(\)](#)

参数

输入参数:	UART_TypeDef: UARTx , UART_InitTypeDef
输出参数:	无

返回

返回值: 无

注解

其它说明: 串口波特率计算使用UART.MCU.MCLK系统时钟进行计算，使用前需要确认系统时钟频率

警告

无

示例代码:

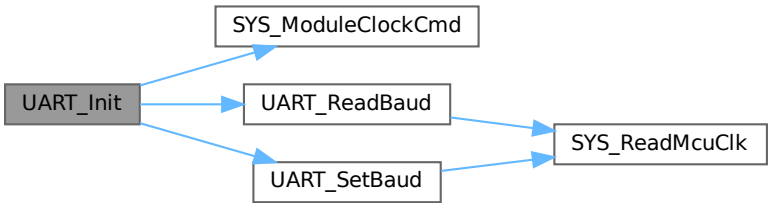
```
UART_InitTypeDef UART_InitStructure;  
UART_StructInit(&UART_InitStructure);  
UART_InitStructure.BaudRate = 9600; // 设置波特率9600  
UART_InitStructure.WordLength= UART_WORDLENGTH_8b; // 发送数据长度8位  
UART_InitStructure.StopBits = UART_STOPBITS_1b; // 停止位长度1位  
UART_InitStructure.FirstSend = UART_FIRSTSEND_LSB; // 先发送LSB  
UART_InitStructure.ParityMode= UART_Parity_NO; // 无奇偶校验  
UART_InitStructure.RXD_INV = DISABLE; // RXD电平正常输出  
UART_InitStructure.TXD_INV = DISABLE; // TXD电平正常输出  
UART_InitStructure.IRQEna = UART_IRQEna_SendOver | UART_IRQEna_RcvOver;  
UART_Init(UART0, &UART_InitStructure);
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 51 行定义.

函数调用图:

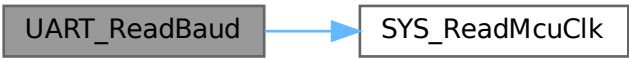


5.93.3.4 UART_ReadBaud()

```
static uint32_t UART_ReadBaud (  
    UART_TypeDef * UARTx ) [static]
```

在文件 [lks32mc07x_uart.c](#) 第 362 行定义.

函数调用图:



这是这个函数的调用关系图:



5.93.3.5 UART_ReadData()

```
uint32_t UART_ReadData (
    UART_TypeDef * UARTx )
```

函数名称: `uint32_t UART_ReadData(UART_TypeDef *UARTx)`

功能描述: UART读缓冲区数据

参见

被引用内容: [UART0](#)

参数

输入参数:	UART_TypeDef: UART0
输出参数:	无

返回

返回 值: 返回串口接收缓冲区的1Byte数据

注解

其它说明: 无

警告

无

示例代码:

```
u8 UART_BUFF = 0;
UART_BUFF = UART_ReadData (UART0); //读取串口0接收一字节数据
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 173 行定义.

5.93.3.6 UART_SendAddr_485()

```
uint8_t UART_SendAddr_485 (
    UART_TypeDef * UARTx,
    uint8_t n )
```

函数名称: `uint8_t UART_SendAddr_485(UART_TypeDef *UARTx, uint8_t n)`

功能描述: 发送485通讯中需要接收数据的从机地址

参见

被引用内容: UARTx

参数

输入参数:	UART_TypeDef: UARTx, n: 要发生1Byte地址
输出参数:	无

返回

返回 值: 1: 发送失败, 0: 发送成功

注解

其它说明: cnt作用是延时防止程序卡死。在发送数据后立即发送地址的情况下,会判断等待发送缓冲区空在进行地址发送,如果长时间还未发送完成(cnt计数时间),则跳出while循环。如果串口波特率较低cnt计数已满,会出现数据还未发送完成进行下一个数据发送,为了避免此类问题发送可以增加cnt值。

警告
无

示例代码:

```
UART_SendAddr_485 (UART0, 0x55); // 串口0发送485从机接收地址为0x55
```

Date	Version	Author	Description
2021年10月14日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 281 行定义.

5.93.3.7 UART_SendData()

```
void UART_SendData (
    UART_TypeDef * UARTx,
    uint32_t n )
```

函数名称: void UART_SENDDATA(UART_TypeDef *UARTx, uint32_t n)

功能描述: UART发送数据

参见
被引用内容: UARTx

参数

输入参数:	UART_TypeDef: UARTx, n: 要发生1Byte数据
输出参数:	无

返回
返回值: 无

注解
其它说明: 无

警告
无

示例代码:

```
UART_SendData (UART0, 0x12); //串口0发送0x12一字节数据
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 lks32mc07x_uart.c 第 148 行定义.

5.93.3.8 UART_SendData_485()

```
uint8_t UART_SendData_485 (
    UART_TypeDef * UARTx,
    uint8_t n )
```

函数名称: uint8_t UART_SendData_485(UART_TypeDef *UARTx, uint8_t n)

功能描述: 发送485通讯中的数据信息

参见

被引用内容: UARTx

参数

输入参数:	UART_TypeDef: UARTx, n: 要发生1Byte数据
输出参数:	无

返回

返回值: 1: 发送失败, 0: 发送成功

注解

其它说明: cnt作用是延时防止程序卡死。在发送数据后立即发送地址的情况下,会判断等待发送缓冲区空在进行地址发送,如果长时间还未发送完成(cnt计数时间),则跳出while循环。如果串口波特率较低cnt计数已满,会出现数据还未发送完成进行下一个数据发送,为了避免此类问题发送可以增加cnt值。

警告

无

示例代码:

```
UART_SendData_485 (UART0,0x12); //串口0发送485数据为0x12
```

Date	Version	Author	Description
2021年10月14日	1.0	HuangMG	创建

修改日志:

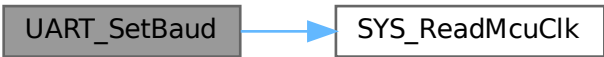
在文件 [lks32mc07x_uart.c](#) 第 324 行定义.

5.93.3.9 UART_SetBaud()

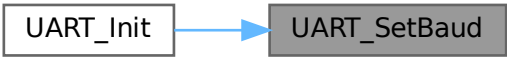
```
static void UART_SetBaud (
    uint32_t Uart0Baud,
    uint32_t Uart1Baud ) [static]
```

在文件 [lks32mc07x_uart.c](#) 第 342 行定义.

函数调用图:



这是这个函数的调用关系图:



5.93.3.10 UART_StructInit()

```
void UART_StructInit (
    UART_InitTypeDef * UART_InitStruct )
```

函数名称: void [UART_StructInit\(UART_InitTypeDef* UART_InitStruct\)](#)

功能描述: UART结构体初始化函数

参见

被引用内容: 无

参数

输入参数:	UART_InitTypeDef
输出参数:	无

返回

返回 值: 无

注解

其它说明: 无

警告

无

示例代码:

```
UART_InitTypeDef UART_InitStructure;  
UART_StructInit(&UART_InitStructure); //初始化UART相关配置结构体变量
```

Date	Version	Author	Description
2023年2月23日	1.0	cakezhang	创建

修改日志:

在文件 [lks32mc07x_uart.c](#) 第 111 行定义.

5.94 lks32mc07x_uart.c

[浏览该文件的文档.](#)

```
00001  
00014 #include "lks32mc07x_lib.h"  
00015  
00016 static void UART_SetBaud(uint32_t Uart0Baud, uint32_t Uart1Baud);  
00017 static uint32_t UART_ReadBaud(UART_TypeDef *UARTx);  
00018 #ifndef MCU_MCLK  
00019 #define MCU_MCLK 96000000  
00020 #endif  
00051 void UART_Init(UART_TypeDef *UARTx, UART_InitTypeDef *UART_InitStructure)  
00052 {  
00053     uint32_t parity;  
00054     uint32_t parityEna;  
00055  
00056     if (UARTx == UART0)  
00057     {  
00058         SYS_ModuleClockCmd(SYS_Module_UART0, ENABLE);  
00059         UART_SetBaud(UART_InitStructure->BaudRate, UART_ReadBaud(UART1));  
00060     }  
00061     else if (UARTx == UART1)  
00062     {
```

```

00063     SYS.ModuleClockCmd(SYS.Module_UART1, ENABLE);
00064     UART.SetBaud(UART.ReadBaud(UART0), UART.InitStruct->BaudRate);
00065 }
00066
00067 if (UART.InitStruct->ParityMode == UART_Parity_EVEN)
00068 {
00069     parityEna = ENABLE;
00070     parity     = 0;
00071 }
00072 else if (UART.InitStruct->ParityMode == UART_Parity_ODD)
00073 {
00074     parityEna = ENABLE;
00075     parity     = 1;
00076 }
00077 else
00078 {
00079     parityEna = DISABLE;
00080     parity     = 0;
00081 }
00082 UARTx->CTRL = UART.InitStruct->WordLength | (UART.InitStruct->StopBits << 1) |
(UART.InitStruct->FirstSend << 2) | (parity << 3) | (parityEna << 4) | (UART.InitStruct->MultiDropEna
<< 5) | (UART.InitStruct->Bit9Value << 6) | (UART.InitStruct->DUPLEX << 7);
00083
00084 UARTx->IOC = (UART.InitStruct->TXD_INV << 1) | UART.InitStruct->RXD_INV;
00085
00086 UARTx->ADR = UART.InitStruct->Match485Addr;
00087 UARTx->RE   = UART.InitStruct->DMARE;
00088 UARTx->IE   = UART.InitStruct->IRQEna;
00089 }
00090
00111 void UART_StructInit(UART_InitTypeDef *UART_InitStruct)
00112 {
00113     UART_InitStruct->BaudRate     = 9600;
00114     UART_InitStruct->WordLength   = UART_WORDLENGTH_8b;
00115     UART_InitStruct->StopBits     = UART_STOPBITS_1b;
00116     UART_InitStruct->FirstSend    = UART_FIRSTSEND_LSB;
00117     UART_InitStruct->ParityMode   = UART_Parity_NO;
00118
00119     UART_InitStruct->MultiDropEna = DISABLE;
00120     UART_InitStruct->Bit9Value    = 0;
00121     UART_InitStruct->DUPLEX       = 0;
00122
00123     UART_InitStruct->Match485Addr = 0;
00124     UART_InitStruct->IRQEna       = 0;
00125     UART_InitStruct->RXD_INV      = DISABLE;
00126     UART_InitStruct->TXD_INV      = DISABLE;
00127 }
00128
00148 void UART_SendData(UART_TypeDef *UARTx, uint32_t n)
00149 {
00150     UARTx->BUFF = n;
00151 }
00152
00173 uint32_t UART_ReadData(UART_TypeDef *UARTx)
00174 {
00175     return UARTx->BUFF;
00176 }
00177
00212 uint32_t UART_GetIRQFlag(UART_TypeDef *UARTx, uint32_t tempFlag)
00213 {
00214     if ((UARTx->IF) & tempFlag & (UARTx->IE))
00215     {
00216         return 1;
00217     }
00218     return 0;
00219 }
00254 void UART_ClearIRQFlag(UART_TypeDef *UARTx, uint32_t tempFlag)
00255 {
00256     UARTx->IF = tempFlag;
00257 }
00258
00281 uint8_t UART_SendAddr_485(UART_TypeDef *UARTx, uint8_t n)
00282 {
00283     uint32_t m   = 0x100;
00284     uint32_t cnt = 0xffffffff;
00285     m |= n;
00286     UARTx->BUFF = m;
00287     while (! (UARTx->STT & BIT0)) /*发送缓冲区非空*/
00288     {
00289         if (cnt > 0)
00290         {
00291             cnt--;
00292             __NOP();
00293         }
00294         else
00295         {
00296             return 1; /*发送失败*/

```

```

00297     }
00298 }
00299 return 0; /*发送成功*/
00300 }
00301
00324 uint8_t UART_SendData_485(UART_TypeDef *UARTx, uint8_t n)
00325 {
00326     uint32_t cnt = 0xffffffff;
00327     UARTx->BUFF = n;
00328     while (!(UARTx->STT & BIT0)) /*发送缓冲区非空*/
00329     {
00330         if (cnt > 0)
00331         {
00332             cnt--;
00333             __NOP();
00334         }
00335         else
00336         {
00337             return 1; /*发送失败*/
00338         }
00339     }
00340     return 0; /*发送成功*/
00341 }
00342 static void UART_SetBaud(uint32_t Uart0Baud, uint32_t Uart1Baud)
00343 {
00344     uint32_t d0 = SYS_ReadMcuClk() / Uart0Baud;
00345     uint32_t d1 = SYS_ReadMcuClk() / Uart1Baud;
00346     uint32_t max = d0 > d1 ? d0 : d1;
00347     uint32_t d2;
00348     d2 = max >> 16;
00349     d2 += 1;
00350     d0 /= d2;
00351     d1 /= d2;
00352     d0 -= 1;
00353     d1 -= 1;
00354     UART0->DIVL = d0 & 0xFF;
00355     UART0->DIVH = (d0 & 0xFF00) >> 8;
00356     UART1->DIVL = d1 & 0xFF;
00357     UART1->DIVH = (d1 & 0xFF00) >> 8;
00358     SYS->WR_PROTECT = 0x7A83;
00359     SYS_CLK_DIV2 = d2 - 1;
00360     SYS->WR_PROTECT = 0x0;
00361 }
00362 static uint32_t UART_ReadBaud(UART_TypeDef *UARTx)
00363 {
00364     uint32_t baud;
00365     baud = SYS_ReadMcuClk() / (SYS_CLK_DIV2 + 1) / (1 + 256 * UARTx->DIVH + UARTx->DIVL);
00366     return baud;
00367 }
00368 /***** (C) COPYRIGHT LINKO SEMICONDUCTOR *****END OF FILE*****/

```

5.95 source/lks32mc07x_wake.c 文件参考

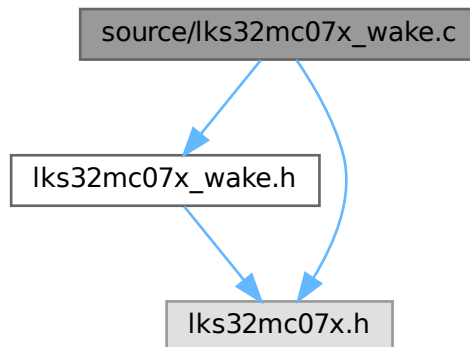
文件名称: [lks32mc07x_wake.c](#)
 文件标识: 无
 内容摘要: wake外设驱动程序
 其它说明: 无

```

#include "lks32mc07x_wake.h"
#include "lks32mc07x.h"

```


lks32mc07x_wake.c 的引用(Include)关系图:



函数

- static void **wake_SoftDelay** (u32 cnt)
函数名称: *static void SoftDelay(u32 cnt)*
- void **Switch2PLL** (void)
函数名称: *void Switch2PLL(void)*
- void **Switch2HRC** (void)
函数名称: *void Switch2HRC(void)*
- void **SetWakeIO** (uint32_t wakeIO, enumWakeTrigLevel trigLevel, uint32_t flt, uint8_t ena)
函数名称: *void SetWakeIO(uint32_t wakeIO, enumWakeTrigLevel trigLevel, uint32_t flt, uint8_t ena)*
- void **SYS_FallSleep** (void)
函数名称: *void SYS_FallSleep(void)*
- void **SYS_EVTCLRSleep** (void)
函数名称: *void SYS_EVTCLRSleep(void)*
- void **SYS_SetWakeInterval** (uint32_t rth, uint32_t with, uint8_t ena)
函数名称: *void SYS_SetWakeInterval(uint32_t rth, uint32_t with, uint8_t ena)*
- void **SYS_EnterSleep** (void)
系统休眠, 需要配置唤醒源 (io唤醒或定时唤醒), 以及定义 *WAKE_IRQHandler* 函数

变量

- **WAKE_InitTypeDef WAKE_InitStruct**
低功耗初始化结构体定义

5.95.1 详细描述

文件名称: **lks32mc07x_wake.c**
 文件标识: 无
 内容摘要: **wake** 外设驱动程序
 其它说明: 无

版权所有

(C)2015, LINKO SEMICONDUCTOR Co.ltd

Date	Version	Author	Description
2023年02月23日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 中定义.

5.95.2 函数说明

5.95.2.1 SetWakeIO()

```
void SetWakeIO (
    uint32_t wakeIO,
    enumWakeTrigLevel trigLevel,
    uint32_t flt,
    uint8_t ena )
```

函数名称: void [SetWakeIO\(uint32_t wakeIO, enumWakeTrigLevel trigLevel, uint32_t flt , uint8_t ena\)](#)

功能描述: 设置休眠唤醒IO

参见

唤醒电平选择枚举: [enumWakeTrigLevel](#)

参数

输入参数:	wakeIO = BIT11: CLUOUT3_EN CLUOUT3 作为唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT10: CLUOUT2_EN CLUOUT2 作为唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT9: CLUOUT1_EN CLUOUT1 作为唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT8: CLUOUT0_EN CLUOUT0 作为唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT7: GPIO 2[15] / P2[4] 外部唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT6: GPIO 2[7] / P2[4] 外部唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT5: GPIO 2[4] / P2[4] 外部唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT4: GPIO 0[14] / P0[14] 外部唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT3: GPIO 0[11] / P0[11] 外部唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT2: GPIO 0[6] / P0[6] 外部唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT1: GPIO 0[2] / P0[2] 外部唤醒使能。1: 使能; 0: 禁用。 wakeIO = BIT0: GPIO 0[0] / P0[0] 外部唤醒使能。1: 使能; 0: 禁用。 enumWakeTrigLevel : 唤醒电平选择: 0:低电平唤醒 1:高电平唤醒 flt: ENABLE: 开启IO滤波 DISABLE: 关闭IO滤波 ena: ENABLE: 开启IO唤醒 DISABLE: 关闭IO唤醒
输出参数:	无

返回

返回 值: 无

注解

其它说明: 低功耗IO可以使用多个, 但所有IO唤醒信号使用同一个极性选择。

警告

无

示例代码:

```
SetWakeIO(WAKEIO_P0_0,WAKETRIG_LOW,ENABLE,ENABLE); //系统休眠时,使能P0.0作为低功耗唤醒IO,低电平唤醒
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 161 行定义.

5.95.2.2 Switch2HRC()

```
void Switch2HRC (  
    void )
```

函数名称: void [Switch2HRC\(void\)](#)

功能描述: 休眠唤醒前一时刻, 关闭高速时钟与BGP和掉电检测电路函数

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 系统低功耗休眠使用

警告

无

示例代码:

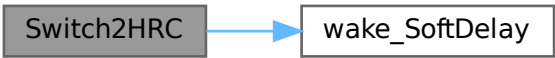
```
Switch2HRC\(\);
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 103 行定义.

函数调用图:



5.95.2.3 Switch2PLL()

```
void Switch2PLL (
    void )
```

函数名称: void Switch2PLL(void)

功能描述: 休眠唤醒后，系统时钟切换到PLL时钟函数

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 系统低功耗唤醒使用

警告

无

示例代码:

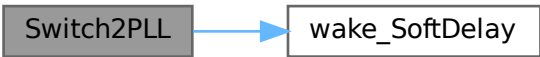
```
Switch2PLL();
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 71 行定义.

函数调用图:



5.95.2.4 SYS_EnterSleep()

```
void SYS_EnterSleep (
    void )
```

系统休眠，需要配置唤醒源（io唤醒或定时唤醒），以及定义WAKE_IRQHandler函数

在文件 [lks32mc07x_wake.c](#) 第 307 行定义.

5.95.2.5 SYS_EVTCLRSleep()

```
void SYS_EVTCLRSleep (
    void )
```

函数名称: void [SYS_EVTCLRSleep\(void\)](#)

功能描述: 清除休眠全部事件记录

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明: 无

警告

无

示例代码:

```
SYS_EVTCLRSleep();
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 248 行定义.

5.95.2.6 SYS_FallSleep()

```
void SYS_FallSleep (
    void )
```

函数名称: void [SYS_FallSleep\(void\)](#)

功能描述: 触发系统进入休眠状态，必须定义WAKE_IRQHandler否则芯片无法正常唤醒

参见

被调用函数: 无

参数

输入参数:	无
输出参数:	无

返回

返回值: 无

注解

其它说明： 无

警告

无

示例代码:

```
SYS_FallSleep();
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 210 行定义.

5.95.2.7 SYS_SetWakeInterval()

```
void SYS_SetWakeInterval (
    uint32_t rth,
    uint32_t wth,
    uint8_t ena )
```

函数名称: void [SYS_SetWakeInterval\(uint32_t rth , uint32_t wth , uint8_t ena\)](#)

功能描述: 设置休眠唤醒周期

参见

被调用函数： 无

参数

输入参数:	rth	看门狗超时复位门限值，也是重新装载值(最大值0xFF000)
	wth	看门狗定时唤醒门限值(最大值小于rth)
	ena	ENABLE: 开启定时唤醒 DISABLE: 关闭定时唤醒
输出参数:	无	

返回

返回值: 无

注解

其它说明: 唤醒时间计算公式: $WakeupIWDG = tLRC * (rth - lwth)$ tLRC 为 LRC时钟周期, 1/32kHz=31.25uS

$rth - wth * (tLRC) =$

唤醒时间

$8000 * 31.25uS =$

0.25S

$16000 * 31.25uS =$

0.5S

$32000 * 31.25uS =$

1S

$64000 * 31.25uS =$

2S

$128000 * 31.25uS =$

4S

警告

无

示例代码:

```
SetWakeIO(WAKEIO_P0_0, WAKETRIG_LOW, ENABLE, ENABLE); //系统休眠时,使能P0.0作为低功耗唤醒IO.
低电平唤醒
```

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 [lks32mc07x_wake.c](#) 第 289 行定义.

5.95.2.8 wake_SoftDelay()

```
static void wake_SoftDelay (
    u32 cnt ) [static]
```

函数名称: static void SoftDelay(u32 cnt)

功能描述: 延时函数

参见
被调用函数: 无

参数

输入参数:	cnt: 延时cnt的空指令_NOP()
输出参数:	无

返回
返回值: 无

注解
其它说明: 改函数只在此文件使用，主要用在Switch2PLL函数内

警告
无

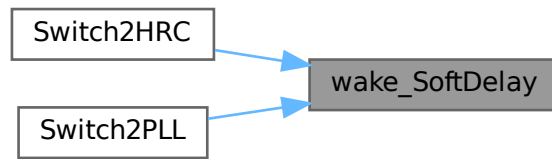
示例代码:
`SoftDelay(0xff); //延时255个_NOP() 时间1个_NOP大约按一个系统时钟周期计算`

Date	Version	Author	Description
2023年02月22日	1.0	HuangMG	创建

修改日志:

在文件 lks32mc07x_wake.c 第 43 行定义.

这是这个函数的调用关系图:



5.95.3 变量说明

5.95.3.1 WAKE_InitStruct

WAKE_InitTypeDef WAKE_InitStruct

低功耗初始化结构体定义

在文件 `lks32mc07x_wake.c` 第 22 行定义.

5.96 lks32mc07x_wake.c

[浏览该文件的文档.](#)

```

00001
00015 #include "lks32mc07x_wake.h"
00016 #include "lks32mc07x.h"
00017
00022 WAKE_InitTypeDef WAKE_InitStruct;
00023
00043 static void wake_SoftDelay(u32 cnt)
00044 {
00045     volatile u32 t_cnt;
00046     for (t_cnt = 0; t_cnt < cnt; t_cnt++)
00047     {
00048         __NOP();
00049     }
00050 }
00051
00071 void Switch2PLL(void)
00072 {
00073     int32_t flg = __get_PRIMASK();           // 保存全局中断状态
00074     __disable_irq();                         // 关闭全局中断
00075     SYS_WR_PROTECT = 0x7a83;                 // 开启寄存器写使能
00076     SYS_AFE_REG5 = WAKE_InitStruct.AFE_REG5_RECORD; // 开启 PLL, BGP
00077     wake_SoftDelay(400);                     // 等待时钟稳定
00078     SYS_CLK_CFG = WAKE_InitStruct.CLK_CFG_RECORD; // 将主时钟切换到PLL时钟
00079     wake_SoftDelay(4000);                    // 等待时钟稳定
00080     SYS_WR_PROTECT = 0x0;                   // 关闭写使能
00081     __set_PRIMASK(flg);                     // 还原全局中断状态
00082 }
00083
00103 void Switch2HRC(void)
00104 {
00105     int32_t flg = __get_PRIMASK();           // 保存全局中断状态
00106     __disable_irq();                         // 关闭全局中断
00107     SYS_WR_PROTECT = 0x7a83;                 // 开启寄存器写使能
00108     WAKE_InitStruct.AFE_REG5_RECORD = SYS_AFE_REG5; // 保存模拟模块配置
00109     WAKE_InitStruct.CLK_CFG_RECORD = SYS_CLK_CFG; // 保存时钟配置
00110     SYS_CLK_CFG = 0x0;                      // 切换到HRC时钟
00111     wake_SoftDelay(4000);                    // 等待时钟稳定
00112     SYS_AFE_REG5 = 0x0500;                  // 准备关闭 PLL, BGP, 以及其它模拟电路使能
  
```

```

00113     SYS_WR_PROTECT = 0x0;                                // 关闭写使能
00114     __set_PRIMASK(flag);                                // 还原全局中断状态
00115 }
00116
00161 void SetWakeIO(uint32_t wakeIO, enumWakeTrigLevel trigLevel, uint32_t flt, uint8_t ena)
00162 {
00163     if (ena == ENABLE)
00164     {
00165         AON_IO_WAKE_EN |= wakeIO;
00166     }
00167     else
00168     {
00169         AON_IO_WAKE_EN &= ~wakeIO;
00170     }
00171
00172     if (trigLevel == WAKETRIG_HIGH)
00173     {
00174         AON_IO_WAKE_POL = 0x01;
00175     }
00176     else
00177     {
00178         AON_IO_WAKE_POL = 0x00;
00179     }
00180
00181     if (flt == IOWK_FLT_EN)
00182     {
00183         AON_PWR_CFG = BIT1;
00184     }
00185     else
00186     {
00187         AON_PWR_CFG = 0;
00188     }
00189 }
00190
00210 void SYS_FallsSleep(void)
00211 {
00212     uint32_t nvic_iser;
00213     int32_t flg = __get_PRIMASK(); // 保存全局中断状态
00214     __disable_irq();               // 关闭全局中断, 确保执行休眠期间, 不会进入其他中断
00215     nvic_iser = NVIC->ISER[0];    // 保存当前中断配置
00216     NVIC->ICER[0] = 0xFFFFFFFF;   // 关闭所有中断
00217     NVIC_EnableIRQ(WAKE_IRQn);    // 打开唤醒中断
00218     __enable_irq();               // 打开中断使能
00219     IWDG_PSW = 0xA6B4;            // 看门狗写保护
00220     IWDG_CLR = 0x798D;            // 喂狗, 确保定时唤醒时间和设定时间一致
00221     SCB->SCR |= (1UL << 2);      // 使能深度休眠
00222     __WFI();                      // 系统休眠
00223     __disable_irq();              // 唤醒后会先执行WAKE_IRQHandler, 随后到此处
00224     NVIC_DisableIRQ(WAKE_IRQn);  // 关闭唤醒中断
00225     NVIC->ISER[0] = nvic_iser;   // 恢复中断使能
00226     __set_PRIMASK(flag);         // 还原全局中断状态
00227 }
00228
00248 void SYS_EVTCLRSleep(void)
00249 {
00250     AON_EVT_RCD = PSW_EVT_CLR; /* 清除全部事件记录*/
00251 }
00252
00289 void SYS_SetWakeInterval(uint32_t rth, uint32_t wth, uint8_t ena)
00290 {
00291     if (ena == ENABLE)
00292     {
00293         IWDG_CFG |= 0x01 << 4; /*深度休眠定时唤醒使能*/
00294     }
00295     else
00296     {
00297         IWDG_CFG &= ~(0x01 << 4); /*关闭深度休眠定时唤醒*/
00298     }
00299     IWDG_PSW = 0xA6B4; /*使能看门狗部分寄存器写操作*/
00300     IWDG_RTH = rth;
00301     IWDG_WTH = wth;
00302 }
00303
00307 void SYS_EnterSleep(void)
00308 {
00309     u32 t_cnt;
00310     uint32_t reg5;           // 模拟配置
00311     uint32_t clkcfg;         // 时钟配置
00312     uint32_t clk_fen;        // 模块使能配置
00313     uint32_t saved_enable;   // 中断配置
00314
00315     __disable_irq();         // 关闭全局中断, 确保执行休眠期间, 不会进入其他任何中断
00316     reg5 = SYS_AFE_REG5;     // 保存模拟使能配置
00317     clkcfg = SYS_CLK_CFG;    // 保存时钟配置
00318     clk_fen = SYS_CLK_FEN;   // 保存模块时钟使能配置
00319     saved_enable = NVIC->ISER[0]; // 保存当前中断配置
00320

```

```

00321     SYS_WR_PROTECT = 0x7a83;                // 开启寄存器写使能
00322     NVIC->ICER[0] = 0xFFFFFFFF;             // 关闭所有中断
00323     NVIC_EnableIRQ(WAKE_IRQn);              // 打开唤醒中断
00324     IWDG_PSW = 0xA6B4;                      // 打开看门狗使能
00325     IWDG_CLR = 0x798D;                      // 喂狗
00326     SYS_CLK_CFG = 0;                        // 切换到HRC时钟
00327     for (t_cnt = 0; t_cnt < 4000; t_cnt++) // 等待时钟稳定
00328     {
00329         __NOP();
00330     }
00331     SYS_CLK_FEN = 0;                        // 关闭数字模块时钟
00332     SYS_AFE_REG5 = BIT10 | BIT8;           // 准备关闭 PLL,BGP,以及其它模拟电路使能
00333     SYS_WR_PROTECT = 0x0;                  // 关闭系统寄存器写使能
00334     __enable_irq();                        // 打开全局中断
00335     SCB->SCR |= (1UL << 2);               // 使能深度休眠
00336
00337     __WFI();                               // 休眠 wakeak
00338     __disable_irq();                       // 唤醒后关闭全局中断
00339     NVIC_DisableIRQ(WAKE_IRQn);           // 关闭唤醒中断
00340
00341     SYS_WR_PROTECT = 0x7a83; // 开启寄存器写使能
00342     SYS_AFE_REG5 = reg5; // 开启 PLL,BGP
00343
00344     for (t_cnt = 0; t_cnt < 400; t_cnt++) // 此时使用hrc, 等待>1mS, 确保PLL稳定
00345     {
00346         __NOP();
00347     }
00348     SYS_CLK_CFG = clkcfg; // 恢复时钟配置
00349     NVIC->ISER[0] = saved_enable; // 恢复中断使能
00350     SYS_CLK_FEN = clk_fen; // 打开其他模块
00351     SYS_WR_PROTECT = 0x0; // 打开写保护
00352     __enable_irq(); // 使能全局中断
00353 }

```


Index

--pad0__
 stru.DMA_REN, 154
--pad1__
 stru.DMA_REN, 154
--pad2__
 stru.DMA_REN, 155
--pad3__
 stru.DMA_REN, 155
--pad4__
 stru.DMA_REN, 155
--pad5__
 stru.DMA_REN, 155
--pad6__
 stru.DMA_REN, 155
--pad7__
 stru.DMA_REN, 155
--pad8__
 stru.DMA_REN, 155
--pad9__
 stru.DMA_REN, 155

ACF
 CAN_TypeDef, 48
ACFCTRL
 CAN_TypeDef, 48
ACFEN
 CAN_TypeDef, 48
ADC
 AFE_TypeDef, 40
ADC0
 lks32mc07x_adc.h, 193
ADC0_EN
 stru.DMA_REN, 156
ADC1
 lks32mc07x_adc.h, 193
ADC1_EN
 stru.DMA_REN, 156
ADC_ALL_IF
 lks32mc07x_adc.h, 193
ADC_AWD0_IE
 lks32mc07x_adc.h, 193
ADC_AWD0_IF
 lks32mc07x_adc.h, 193
ADC_AWD0_RE
 lks32mc07x_adc.h, 193
ADC_CHANNEL_0
 lks32mc07x_adc.h, 194
ADC_CHANNEL_1
 lks32mc07x_adc.h, 194
ADC_CHANNEL_10
 lks32mc07x_adc.h, 194
ADC_CHANNEL_11
 lks32mc07x_adc.h, 194
ADC_CHANNEL_12
 lks32mc07x_adc.h, 194
ADC_CHANNEL_13
 lks32mc07x_adc.h, 194
ADC_CHANNEL_14
 lks32mc07x_adc.h, 195
ADC_CHANNEL_15
 lks32mc07x_adc.h, 195
ADC_CHANNEL_2
 lks32mc07x_adc.h, 195
ADC_CHANNEL_3
 lks32mc07x_adc.h, 195
ADC_CHANNEL_4
 lks32mc07x_adc.h, 195
ADC_CHANNEL_5
 lks32mc07x_adc.h, 195
ADC_CHANNEL_6
 lks32mc07x_adc.h, 196
ADC_CHANNEL_7
 lks32mc07x_adc.h, 196
ADC_CHANNEL_8
 lks32mc07x_adc.h, 196
ADC_CHANNEL_9
 lks32mc07x_adc.h, 196
ADC_CHN_GAIN_CFG
 lks32mc07x_adc.c, 593
 lks32mc07x_adc.h, 203
ADC_ClearIRQFlag
 lks32mc07x_adc.c, 594
 lks32mc07x_adc.h, 204
ADC_GAIN3V6
 lks32mc07x_adc.h, 202
ADC_GAIN7V2
 lks32mc07x_adc.h, 202
ADC_GAINx
 lks32mc07x_adc.h, 201
ADC_GetConversionValue
 lks32mc07x_adc.c, 595
 lks32mc07x_adc.h, 205
ADC_GetIRQFlag
 lks32mc07x_adc.c, 596
 lks32mc07x_adc.h, 206
ADC_HERR_IE
 lks32mc07x_adc.h, 196
ADC_HERR_IF
 lks32mc07x_adc.h, 196

ADC_HERR_RE
lks32mc07x_adc.h, 197

ADC_Init
lks32mc07x_adc.c, 597
lks32mc07x_adc.h, 208

ADC_InitTypeDef, 25
CSMP, 26
DATA_ALIGN, 26
GAIN, 26
GEN, 26
HTH, 26
IE, 26
IS1, 27
LTH, 27
NSMP, 27
OVSR, 27
RE, 27
S1, 27
S2, 28
TCNT, 28
TRIG, 28
TROVS, 28

ADC_ISF_IE
lks32mc07x_adc.h, 197

ADC_ISF_IF
lks32mc07x_adc.h, 197

ADC_ISF_RE
lks32mc07x_adc.h, 197

ADC_SERR_IE
lks32mc07x_adc.h, 197

ADC_SERR_IF
lks32mc07x_adc.h, 197

ADC_SERR_RE
lks32mc07x_adc.h, 198

ADC_SF1_IE
lks32mc07x_adc.h, 198

ADC_SF1_IF
lks32mc07x_adc.h, 198

ADC_SF1_RE
lks32mc07x_adc.h, 198

ADC_SF2_IE
lks32mc07x_adc.h, 198

ADC_SF2_IF
lks32mc07x_adc.h, 198

ADC_SF2_RE
lks32mc07x_adc.h, 199

ADC_SoftTrgEN
lks32mc07x_adc.c, 598
lks32mc07x_adc.h, 209

ADC_StructInit
lks32mc07x_adc.c, 599
lks32mc07x_adc.h, 210

ADC_TRIG_CL_IT0
lks32mc07x_adc.h, 199

ADC_TRIG_CL_IT1
lks32mc07x_adc.h, 199

ADC_TRIG_CL_IT2
lks32mc07x_adc.h, 199

ADC_TRIG_CL_IT3
lks32mc07x_adc.h, 199

ADC_TRIG_CL_NT0
lks32mc07x_adc.h, 199

ADC_TRIG_CL_NT1
lks32mc07x_adc.h, 200

ADC_TRIG_CL_NT2
lks32mc07x_adc.h, 200

ADC_TRIG_CL_NT3
lks32mc07x_adc.h, 200

ADC_TRIG_MCPWM0_T0
lks32mc07x_adc.h, 200

ADC_TRIG_MCPWM0_T1
lks32mc07x_adc.h, 200

ADC_TRIG_MCPWM0_T2
lks32mc07x_adc.h, 200

ADC_TRIG_MCPWM0_T3
lks32mc07x_adc.h, 201

ADC_TRIG_UTIMER0_CMP0
lks32mc07x_adc.h, 201

ADC_TRIG_UTIMER0_CMP1
lks32mc07x_adc.h, 201

ADC_TRIG_UTIMER2_CMP0
lks32mc07x_adc.h, 201

ADC_TRIG_UTIMER2_CMP1
lks32mc07x_adc.h, 201

ADC_TypeDef, 28
AMC0, 31
AMC1, 31
CFG, 31
CHN0, 31
CHN1, 31
CHN2, 32
CHN3, 32
CHNT, 32
DAT0, 32
DAT1, 32
DAT10, 32
DAT11, 33
DAT12, 33
DAT13, 33
DAT2, 33
DAT3, 33
DAT4, 33
DAT5, 34
DAT6, 34
DAT7, 34
DAT8, 34
DAT9, 34
DC0, 34
DC1, 35
GAIN, 35
GEN, 35
HTH, 35
ICHN, 35
IDAT0, 35
IDAT1, 36
IE, 36

IF, 36
LTH, 36
null, 36
null1, 36
null10, 37
null11, 37
null12, 37
null13, 37
null14, 37
null15, 37
null16, 38
null2, 38
null3, 38
null4, 38
null5, 38
null6, 38
null7, 39
null8, 39
null9, 39
SWT, 39
TRIG, 39
ADDR
 I2C_TypeDef, 90
ADR
 UART_TypeDef, 186
ADRCMP
 I2C_InitTypeDef, 88
AF0_GPIO
 lks32mc07x_gpio.h, 364
AF10_CAN
 lks32mc07x_gpio.h, 364
AF11_SIF
 lks32mc07x_gpio.h, 364
AF12_CL
 lks32mc07x_gpio.h, 364
AF1_SYSAF
 lks32mc07x_gpio.h, 364
AF2_HALL
 lks32mc07x_gpio.h, 364
AF3_MCPWM
 lks32mc07x_gpio.h, 364
AF4_UART
 lks32mc07x_gpio.h, 365
AF5_SPI
 lks32mc07x_gpio.h, 365
AF6_I2C
 lks32mc07x_gpio.h, 365
AF7_TIMER01
 lks32mc07x_gpio.h, 365
AF8_TIMER23
 lks32mc07x_gpio.h, 365
AF9_DEBUG
 lks32mc07x_gpio.h, 365
AFE
 lks32mc07x_sys.h, 496
AFE_REG5_RECORD
 WAKE_InitTypeDef, 188
AFE_TypeDef, 40
ADC, 40
DAC0, 40
DAC0_AMC, 41
DAC0_DC, 41
DAC1, 41
DAC1_AMC, 41
DAC1_DC, 41
DAC_CTRL, 41
DBG, 41
INFO, 41
null1, 42
null2, 42
null3, 42
null4, 42
null5, 42
null6, 42
null7, 42
REG0, 42
REG1, 43
REG2, 43
REG3, 43
REG4, 43
REG5, 43
REG6, 43
REG7, 43
REG8, 43
REG9, 44
REGA, 44
REGB, 44
REGC, 44
AMC0
 ADC_TypeDef, 31
AMC1
 ADC_TypeDef, 31
ARCTAN
 DSP_Typedef, 76
AUEN
 MCPWM_InitTypeDef, 102
 MCPWM_TypeDef, 136
B
 stru_DMA_REN, 156
BASE_CNT0_EN
 MCPWM_InitTypeDef, 102
BASE_CNT1_EN
 MCPWM_InitTypeDef, 102
BAUD
 SPI_TypeDef, 152
BaudRate
 I2C_InitTypeDef, 88
 SPI_InitTypeDef, 150
 UART_InitTypeDef, 182
BCR
 I2C_TypeDef, 90
Bit9Value
 UART_InitTypeDef, 182
Bit_RESET
 lks32mc07x_gpio.h, 372
Bit_SET

- lks32mc07x_gpio.h, 372
- BitAction
 - lks32mc07x_gpio.h, 372
- BLCWIN
 - CMP_TypeDef, 65
- BRR
 - GPIO_TypeDef, 81
- BSIZE
 - I2C_TypeDef, 90
- BSRR
 - GPIO_TypeDef, 81
- buf
 - CANRxMsg, 52
 - CANTxMsg, 53
- BUFF
 - UART_TypeDef, 186
- BURST_ADDR_CMP
 - I2C_InitTypeDef, 88
- BURST_NACK
 - I2C_InitTypeDef, 88
- BUS_ERR_IE
 - I2C_InitTypeDef, 88
- BUSRT_EN
 - I2C_InitTypeDef, 88
- ByteLength
 - SPI_InitTypeDef, 150
- CAN
 - lks32mc07x_can.h, 218
- CAN_ArbitrationLostCap
 - lks32mc07x_can.c, 605
 - lks32mc07x_can.h, 225
- CAN_BusoffGet
 - lks32mc07x_can.c, 605
 - lks32mc07x_can.h, 226
- CAN_Disable
 - lks32mc07x_can.c, 605
 - lks32mc07x_can.h, 226
- CAN_Enable
 - lks32mc07x_can.c, 605
 - lks32mc07x_can.h, 226
- CAN_ERRINT_ALIE
 - lks32mc07x_can.h, 218
- CAN_ERRINT_ALIF
 - lks32mc07x_can.h, 218
- CAN_ERRINT_BEIE
 - lks32mc07x_can.h, 218
- CAN_ERRINT_BEIF
 - lks32mc07x_can.h, 218
- CAN_ERRINT_EPASS
 - lks32mc07x_can.h, 218
- CAN_ERRINT_EPIE
 - lks32mc07x_can.h, 218
- CAN_ERRINT_EPIF
 - lks32mc07x_can.h, 219
- CAN_ERRINT_EWARN
 - lks32mc07x_can.h, 219
- CAN_ErrorPassiveGet
 - lks32mc07x_can.c, 606
- lks32mc07x_can.h, 227
- CAN_Init
 - lks32mc07x_can.c, 606
 - lks32mc07x_can.h, 227
- CAN_Init1
 - lks32mc07x_can.c, 607
 - lks32mc07x_can.h, 228
- CAN_InitTypeDef, 44
 - ERRINT_IE, 45
 - IE, 45
 - SBaud, 45
- CAN_KindOfErrorGet
 - lks32mc07x_can.c, 608
 - lks32mc07x_can.h, 229
- CAN_LBMECmd
 - lks32mc07x_can.c, 608
 - lks32mc07x_can.h, 229
- CAN_LBMICmd
 - lks32mc07x_can.c, 609
 - lks32mc07x_can.h, 230
- CAN_MODE_LBME
 - lks32mc07x_can.h, 219
- CAN_MODE_LBMI
 - lks32mc07x_can.h, 219
- CAN_MODE_LOM
 - lks32mc07x_can.h, 219
- CAN_MODE_RESET
 - lks32mc07x_can.h, 219
- CAN_MODE_TPSS
 - lks32mc07x_can.h, 220
- CAN_MODE_TSSS
 - lks32mc07x_can.h, 220
- can_parameter, 45
 - id, 46
 - ide, 46
 - len, 46
 - rtr, 46
 - RX, 46
 - rx_sta, 46
 - TX, 46
- CAN_Receive_Msg
 - lks32mc07x_can.c, 609
 - lks32mc07x_can.h, 230
- CAN_Recv
 - lks32mc07x_can.c, 611
 - lks32mc07x_can.h, 232
- CAN_Reset
 - lks32mc07x_can.c, 611
 - lks32mc07x_can.h, 232
- CAN_ROverflowGet
 - lks32mc07x_can.c, 612
 - lks32mc07x_can.h, 233
- CAN_RTIE_EIE
 - lks32mc07x_can.h, 220
- CAN_RTIE_RAFIE
 - lks32mc07x_can.h, 220
- CAN_RTIE_RFIE
 - lks32mc07x_can.h, 220

CAN_RTIE_RIE
 lks32mc07x_can.h, 220
 CAN_RTIE_ROIE
 lks32mc07x_can.h, 221
 CAN_RTIE_TPIE
 lks32mc07x_can.h, 221
 CAN_RTIE_TSFF
 lks32mc07x_can.h, 221
 CAN_RTIE_TSIE
 lks32mc07x_can.h, 221
 CAN_RTIF_AIF
 lks32mc07x_can.h, 221
 CAN_RTIF_EIF
 lks32mc07x_can.h, 221
 CAN_RTIF_RAFIF
 lks32mc07x_can.h, 222
 CAN_RTIF_RFIF
 lks32mc07x_can.h, 222
 CAN_RTIF_RIF
 lks32mc07x_can.h, 222
 CAN_RTIF_ROIF
 lks32mc07x_can.h, 222
 CAN_RTIF_TPIF
 lks32mc07x_can.h, 222
 CAN_RTIF_TSIF
 lks32mc07x_can.h, 222
 CAN_RxErrorCntGet
 lks32mc07x_can.c, 612
 lks32mc07x_can.h, 233
 CAN_Send
 lks32mc07x_can.c, 612
 lks32mc07x_can.h, 233
 CAN_Send_Cancel
 lks32mc07x_can.c, 613
 lks32mc07x_can.h, 234
 CAN_Send_Trig
 lks32mc07x_can.c, 613
 lks32mc07x_can.h, 234
 CAN_SetSTBMode
 lks32mc07x_can.c, 614
 lks32mc07x_can.h, 235
 CAN_SS_Enable
 lks32mc07x_can.c, 614
 lks32mc07x_can.h, 235
 CAN_STBFullGet
 lks32mc07x_can.c, 614
 lks32mc07x_can.h, 235
 CAN_StructInit
 lks32mc07x_can.c, 615
 lks32mc07x_can.h, 236
 CAN_TxErrorCntGet
 lks32mc07x_can.c, 615
 lks32mc07x_can.h, 236
 CAN_TypeDef, 47
 ACF, 48
 ACFCTRL, 48
 ACFEN, 48
 CFG_STAT, 48
 EALCAP, 48
 ERRINT, 49
 LIMIT, 49
 RBUF, 49
 RCTRL, 49
 RECNT, 49
 resv0, 49
 resv1, 50
 resv2, 50
 resv3, 50
 resv4, 50
 RTIE, 50
 RTIF, 50
 SBAUD, 51
 TBUF, 51
 TCMD, 51
 TCTRL, 51
 TECNT, 51
 CAN_Write_TBUF
 lks32mc07x_can.c, 615
 CanError_Ack
 lks32mc07x_can.h, 224
 CanError_Bit
 lks32mc07x_can.h, 224
 CanError_Crc
 lks32mc07x_can.h, 224
 CanError_Form
 lks32mc07x_can.h, 224
 CanError_Stuff
 lks32mc07x_can.h, 224
 CanError_Unknown
 lks32mc07x_can.h, 224
 CanError_None
 lks32mc07x_can.h, 224
 CANErrorKind
 lks32mc07x_can.h, 223
 CanRecv_NoData
 lks32mc07x_can.h, 224
 CanRecv_Succ
 lks32mc07x_can.h, 224
 CanRecvRet
 lks32mc07x_can.h, 224
 CANRxMsg, 52
 buf, 52
 id, 52
 ide, 52
 len, 52
 rtr, 52
 valid, 53
 CANRXMSG_HEADLEN
 lks32mc07x_can.h, 223
 CanSend_DataTooLong
 lks32mc07x_can.h, 224
 CanSend_NotIdle
 lks32mc07x_can.h, 224
 CanSend_STBFull
 lks32mc07x_can.h, 224
 CanSend_Succ

lks32mc07x_can.h, 224
 CanSend_Timeout
 lks32mc07x_can.h, 224
 CanSendRet
 lks32mc07x_can.h, 224
 CANTxMsg, 53
 buf, 53
 id, 53
 ide, 53
 len, 54
 rtr, 54
 ss, 54
 tbsel, 54
 CAP_SRC_CHXOR
 lks32mc07x_tim.h, 533
 CAP_SRC_CLU0
 lks32mc07x_tim.h, 533
 CAP_SRC_CLU1
 lks32mc07x_tim.h, 533
 CAP_SRC_CLU2
 lks32mc07x_tim.h, 533
 CAP_SRC_CLU3
 lks32mc07x_tim.h, 533
 CAP_SRC_CMP0
 lks32mc07x_tim.h, 533
 CAP_SRC_CMP1
 lks32mc07x_tim.h, 533
 CAP_SRC_CMP2
 lks32mc07x_tim.h, 534
 CAP_SRC_TCH0
 lks32mc07x_tim.h, 534
 CAP_SRC_TCH1
 lks32mc07x_tim.h, 534
 Capture_IRQ_Ena
 HALL_InitTypeDef, 84
 CCR
 DMA_RegTypeDef, 75
 CF
 CLU_TypeDef, 58
 CFG
 ADC_TypeDef, 31
 CMP_TypeDef, 65
 HALL_TypeDef, 86
 I2C_TypeDef, 90
 IWDG_TypeDef, 93
 SPI_TypeDef, 152
 TIM_QEPTTypeDef, 168
 TIM_TimerTypeDef, 180
 CFG_STAT
 CAN_TypeDef, 48
 CH0_NS
 MCPWM_InitTypeDef, 102
 CH0_PS
 MCPWM_InitTypeDef, 102
 CH0N_default_output
 MCPWM_InitTypeDef, 102
 CH0N_FAIL_EN
 MCPWM_InitTypeDef, 103
 CH0N_Polarity_INV
 MCPWM_InitTypeDef, 103
 CH0N_SCTRLN
 MCPWM_InitTypeDef, 103
 CH0P_default_output
 MCPWM_InitTypeDef, 103
 CH0P_FAIL_EN
 MCPWM_InitTypeDef, 103
 CH0P_Polarity_INV
 MCPWM_InitTypeDef, 103
 CH0P_SCTRLP
 MCPWM_InitTypeDef, 104
 CH1_NS
 MCPWM_InitTypeDef, 104
 CH1_PS
 MCPWM_InitTypeDef, 104
 CH1N_default_output
 MCPWM_InitTypeDef, 104
 CH1N_FAIL_EN
 MCPWM_InitTypeDef, 104
 CH1N_Polarity_INV
 MCPWM_InitTypeDef, 104
 CH1N_SCTRLN
 MCPWM_InitTypeDef, 105
 CH1P_default_output
 MCPWM_InitTypeDef, 105
 CH1P_FAIL_EN
 MCPWM_InitTypeDef, 105
 CH1P_Polarity_INV
 MCPWM_InitTypeDef, 105
 CH1P_SCTRLP
 MCPWM_InitTypeDef, 105
 CH2_NS
 MCPWM_InitTypeDef, 105
 CH2_PS
 MCPWM_InitTypeDef, 106
 CH2N_default_output
 MCPWM_InitTypeDef, 106
 CH2N_FAIL_EN
 MCPWM_InitTypeDef, 106
 CH2N_Polarity_INV
 MCPWM_InitTypeDef, 106
 CH2N_SCTRLN
 MCPWM_InitTypeDef, 106
 CH2P_default_output
 MCPWM_InitTypeDef, 106
 CH2P_FAIL_EN
 MCPWM_InitTypeDef, 107
 CH2P_Polarity_INV
 MCPWM_InitTypeDef, 107
 CH2P_SCTRLP
 MCPWM_InitTypeDef, 107
 CH3_NS
 MCPWM_InitTypeDef, 107
 CH3_PS
 MCPWM_InitTypeDef, 107
 CH3N_default_output
 MCPWM_InitTypeDef, 107

CH3N_FAIL_EN
 MCPWM_InitTypeDef, 108
 CH3N_Polarity_INV
 MCPWM_InitTypeDef, 108
 CH3N_SCTRLN
 MCPWM_InitTypeDef, 108
 CH3P_default_output
 MCPWM_InitTypeDef, 108
 CH3P_FAIL_EN
 MCPWM_InitTypeDef, 108
 CH3P_Polarity_INV
 MCPWM_InitTypeDef, 108
 CH3P_SCTRLP
 MCPWM_InitTypeDef, 109
 CH4_NS
 MCPWM_InitTypeDef, 109
 CH4_PS
 MCPWM_InitTypeDef, 109
 CH4N_default_output
 MCPWM_InitTypeDef, 109
 CH4N_FAIL_EN
 MCPWM_InitTypeDef, 109
 CH4N_Polarity_INV
 MCPWM_InitTypeDef, 109
 CH4N_SCTRLN
 MCPWM_InitTypeDef, 110
 CH4P_default_output
 MCPWM_InitTypeDef, 110
 CH4P_FAIL_EN
 MCPWM_InitTypeDef, 110
 CH4P_Polarity_INV
 MCPWM_InitTypeDef, 110
 CH4P_SCTRLP
 MCPWM_InitTypeDef, 110
 CH5_NS
 MCPWM_InitTypeDef, 110
 CH5_PS
 MCPWM_InitTypeDef, 111
 CH5N_default_output
 MCPWM_InitTypeDef, 111
 CH5N_FAIL_EN
 MCPWM_InitTypeDef, 111
 CH5N_Polarity_INV
 MCPWM_InitTypeDef, 111
 CH5N_SCTRLN
 MCPWM_InitTypeDef, 111
 CH5P_default_output
 MCPWM_InitTypeDef, 111
 CH5P_FAIL_EN
 MCPWM_InitTypeDef, 112
 CH5P_Polarity_INV
 MCPWM_InitTypeDef, 112
 CH5P_SCTRLP
 MCPWM_InitTypeDef, 112
 CH_DEF
 MCPWM_TypeDef, 136
 CH_MSK
 MCPWM_TypeDef, 136
 CHN0
 ADC_TypeDef, 31
 lks32mc07x_adc.h, 202
 CHN0P_PWM_EN
 CMP_SingleTypeDef, 62
 CHN1
 ADC_TypeDef, 31
 lks32mc07x_adc.h, 202
 CHN10
 lks32mc07x_adc.h, 202
 CHN11
 lks32mc07x_adc.h, 202
 CHN12
 lks32mc07x_adc.h, 202
 CHN13
 lks32mc07x_adc.h, 202
 CHN14
 lks32mc07x_adc.h, 202
 CHN15
 lks32mc07x_adc.h, 202
 CHN1P_PWM_EN
 CMP_SingleTypeDef, 62
 CHN2
 ADC_TypeDef, 32
 lks32mc07x_adc.h, 202
 CHN2P_PWM_EN
 CMP_SingleTypeDef, 62
 CHN3
 ADC_TypeDef, 32
 lks32mc07x_adc.h, 202
 CHN3P_PWM_EN
 CMP_SingleTypeDef, 62
 CHN4
 lks32mc07x_adc.h, 202
 CHN5
 lks32mc07x_adc.h, 202
 CHN6
 lks32mc07x_adc.h, 202
 CHN7
 lks32mc07x_adc.h, 202
 CHN8
 lks32mc07x_adc.h, 202
 CHN9
 lks32mc07x_adc.h, 202
 CHNT
 ADC_TypeDef, 32
 CHNx
 lks32mc07x_adc.h, 202
 CIRC
 DMA_InitTypeDef, 72
 CLK_CFG
 SYS_TypeDef, 162
 CLK_CFG_RECORD
 WAKE_InitTypeDef, 188
 CLK_COM_DIV
 CMP_InitTypeDef, 60
 CLK_DIV
 MCPWM_InitTypeDef, 112

CLK_DIV0
 SYS_TypeDef, 162
 CLK_DIV1
 SYS_TypeDef, 162
 CLK_DIV2
 SYS_TypeDef, 162
 Clk_DivSPI
 SYS_InitTypeDef, 160
 Clk_DivUART
 SYS_InitTypeDef, 160
 CLK_EN
 CMP_SingleTypeDef, 62
 CLK_FEN
 SYS_TypeDef, 163
 Clk_FEN
 SYS_InitTypeDef, 160
 Clk_Sel
 SYS_InitTypeDef, 160
 CLK_SEL_HS
 lks32mc07x_sys.h, 496
 CLK_SEL_PLL
 lks32mc07x_sys.h, 496
 CLKINV
 CLU_InitTypeDef, 55
 CLKSEL
 CLU_InitTypeDef, 55
 ClockDivflg
 lks32mc07x_mcpwm.c, 743
 ClockDivision
 HALL_InitTypeDef, 84
 CLR
 IWDG_TypeDef, 93
 Clu, 11
 CLU_Disable, 12
 CLU_Enable, 12
 CLU_Init, 12
 CLU_ReadOutValue, 13
 CLU_StructInit, 13
 CLU_Disable
 Clu, 12
 CLU_Enable
 Clu, 12
 CLU_FN_0
 lks32mc07x_clu.h, 248
 CLU_FN_1
 lks32mc07x_clu.h, 248
 CLU_FN_A
 lks32mc07x_clu.h, 248
 CLU_FN_A_B
 lks32mc07x_clu.h, 248
 CLU_FN_A_B_C
 lks32mc07x_clu.h, 248
 CLU_FN_A_B_NOTC
 lks32mc07x_clu.h, 249
 CLU_FN_A_C
 lks32mc07x_clu.h, 249
 CLU_FN_A_NOTB
 lks32mc07x_clu.h, 249
 CLU_FN_A_NOTB_C
 lks32mc07x_clu.h, 249
 CLU_FN_A_NOTB_NOTC
 lks32mc07x_clu.h, 249
 CLU_FN_A_NOTC
 lks32mc07x_clu.h, 249
 CLU_FN_B
 lks32mc07x_clu.h, 250
 CLU_FN_B_C
 lks32mc07x_clu.h, 250
 CLU_FN_B_NOTC
 lks32mc07x_clu.h, 250
 CLU_FN_C
 lks32mc07x_clu.h, 250
 CLU_FN_NOTA
 lks32mc07x_clu.h, 250
 CLU_FN_NOTA_B
 lks32mc07x_clu.h, 250
 CLU_FN_NOTA_B_C
 lks32mc07x_clu.h, 251
 CLU_FN_NOTA_B_NOTC
 lks32mc07x_clu.h, 251
 CLU_FN_NOTA_C
 lks32mc07x_clu.h, 251
 CLU_FN_NOTA_NOTB
 lks32mc07x_clu.h, 251
 CLU_FN_NOTA_NOTB_C
 lks32mc07x_clu.h, 251
 CLU_FN_NOTA_NOTB_NOTC
 lks32mc07x_clu.h, 251
 CLU_FN_NOTA_NOTC
 lks32mc07x_clu.h, 252
 CLU_FN_NOTB
 lks32mc07x_clu.h, 252
 CLU_FN_NOTB_C
 lks32mc07x_clu.h, 252
 CLU_FN_NOTB_NOTC
 lks32mc07x_clu.h, 252
 CLU_FN_NOTC
 lks32mc07x_clu.h, 252
 CLU_Init
 Clu, 12
 CLU_InitTypeDef, 54
 CLKINV, 55
 CLKSEL, 55
 EN, 55
 FIE, 56
 FN, 56
 MXA, 56
 MXB, 56
 OEN, 56
 OUTSEL, 56
 RIE, 57
 CLU_MODEL_CLU0
 lks32mc07x_clu.h, 252
 CLU_MODEL_CLU1
 lks32mc07x_clu.h, 253
 CLU_MODEL_CLU2

lks32mc07x.clu.h, 253
 CLU_MODEL_CLU3
 lks32mc07x.clu.h, 253
 CLU_MXA_CLU0_CLU0OUT
 lks32mc07x.clu.h, 253
 CLU_MXA_CLU0_CLU1OUT
 lks32mc07x.clu.h, 253
 CLU_MXA_CLU0_CLU2OUT
 lks32mc07x.clu.h, 253
 CLU_MXA_CLU0_CLU3OUT
 lks32mc07x.clu.h, 254
 CLU_MXA_CLU0_P0_0
 lks32mc07x.clu.h, 254
 CLU_MXA_CLU0_P0_12
 lks32mc07x.clu.h, 254
 CLU_MXA_CLU0_P0_14
 lks32mc07x.clu.h, 254
 CLU_MXA_CLU0_P0_3
 lks32mc07x.clu.h, 254
 CLU_MXA_CLU0_P0_5
 lks32mc07x.clu.h, 254
 CLU_MXA_CLU0_P0_7
 lks32mc07x.clu.h, 255
 CLU_MXA_CLU0_P1_0
 lks32mc07x.clu.h, 255
 CLU_MXA_CLU0_P2_12
 lks32mc07x.clu.h, 255
 CLU_MXA_CLU0_PWM0
 lks32mc07x.clu.h, 255
 CLU_MXA_CLU0_PWM1
 lks32mc07x.clu.h, 255
 CLU_MXA_CLU0_Timer0ch0ovfl
 lks32mc07x.clu.h, 255
 CLU_MXA_CLU0_Timer1ch0ovfl
 lks32mc07x.clu.h, 256
 CLU_MXA_CLU1_CLU0OUT
 lks32mc07x.clu.h, 256
 CLU_MXA_CLU1_CLU1OUT
 lks32mc07x.clu.h, 256
 CLU_MXA_CLU1_CLU2OUT
 lks32mc07x.clu.h, 256
 CLU_MXA_CLU1_CLU3OUT
 lks32mc07x.clu.h, 256
 CLU_MXA_CLU1_P0_0
 lks32mc07x.clu.h, 256
 CLU_MXA_CLU1_P0_11
 lks32mc07x.clu.h, 257
 CLU_MXA_CLU1_P0_14
 lks32mc07x.clu.h, 257
 CLU_MXA_CLU1_P0_15
 lks32mc07x.clu.h, 257
 CLU_MXA_CLU1_P0_2
 lks32mc07x.clu.h, 257
 CLU_MXA_CLU1_P0_5
 lks32mc07x.clu.h, 257
 CLU_MXA_CLU1_P0_6
 lks32mc07x.clu.h, 257
 CLU_MXA_CLU1_P2_12
 lks32mc07x.clu.h, 258
 CLU_MXA_CLU1_PWM2
 lks32mc07x.clu.h, 258
 CLU_MXA_CLU1_PWM3
 lks32mc07x.clu.h, 258
 CLU_MXA_CLU1_Timer0ch1ovfl
 lks32mc07x.clu.h, 258
 CLU_MXA_CLU1_Timer1ch1ovfl
 lks32mc07x.clu.h, 258
 CLU_MXA_CLU2_CLU0OUT
 lks32mc07x.clu.h, 258
 CLU_MXA_CLU2_CLU1OUT
 lks32mc07x.clu.h, 259
 CLU_MXA_CLU2_CLU2OUT
 lks32mc07x.clu.h, 259
 CLU_MXA_CLU2_CLU3OUT
 lks32mc07x.clu.h, 259
 CLU_MXA_CLU2_P1_10
 lks32mc07x.clu.h, 259
 CLU_MXA_CLU2_P1_13
 lks32mc07x.clu.h, 259
 CLU_MXA_CLU2_P1_15
 lks32mc07x.clu.h, 259
 CLU_MXA_CLU2_P2_1
 lks32mc07x.clu.h, 260
 CLU_MXA_CLU2_P2_14
 lks32mc07x.clu.h, 260
 CLU_MXA_CLU2_P2_4
 lks32mc07x.clu.h, 260
 CLU_MXA_CLU2_P2_6
 lks32mc07x.clu.h, 260
 CLU_MXA_CLU2_P2_8
 lks32mc07x.clu.h, 260
 CLU_MXA_CLU2_PWM0
 lks32mc07x.clu.h, 260
 CLU_MXA_CLU2_PWM2
 lks32mc07x.clu.h, 261
 CLU_MXA_CLU2_Timer2ch0ovfl
 lks32mc07x.clu.h, 261
 CLU_MXA_CLU2_Timer3ch0ovfl
 lks32mc07x.clu.h, 261
 CLU_MXA_CLU3_CLU0OUT
 lks32mc07x.clu.h, 261
 CLU_MXA_CLU3_CLU1OUT
 lks32mc07x.clu.h, 261
 CLU_MXA_CLU3_CLU2OUT
 lks32mc07x.clu.h, 261
 CLU_MXA_CLU3_CLU3OUT
 lks32mc07x.clu.h, 262
 CLU_MXA_CLU3_P1_15
 lks32mc07x.clu.h, 262
 CLU_MXA_CLU3_P2_0
 lks32mc07x.clu.h, 262
 CLU_MXA_CLU3_P2_1
 lks32mc07x.clu.h, 262
 CLU_MXA_CLU3_P2_13
 lks32mc07x.clu.h, 262
 CLU_MXA_CLU3_P2_3

lks32mc07x.clu.h, [262](#)
 CLU_MXA_CLU3_P2_6
 lks32mc07x.clu.h, [263](#)
 CLU_MXA_CLU3_P2_8
 lks32mc07x.clu.h, [263](#)
 CLU_MXA_CLU3_P3_9
 lks32mc07x.clu.h, [263](#)
 CLU_MXA_CLU3_PWM1
 lks32mc07x.clu.h, [263](#)
 CLU_MXA_CLU3_PWM3
 lks32mc07x.clu.h, [263](#)
 CLU_MXA_CLU3_Timer2ch1ovfl
 lks32mc07x.clu.h, [263](#)
 CLU_MXA_CLU3_Timer3ch1ovfl
 lks32mc07x.clu.h, [264](#)
 CLU_MXB_CLU0_ADBUSY0
 lks32mc07x.clu.h, [264](#)
 CLU_MXB_CLU0_ADBUSY1
 lks32mc07x.clu.h, [264](#)
 CLU_MXB_CLU0_CLU0OUT
 lks32mc07x.clu.h, [264](#)
 CLU_MXB_CLU0_CLU1OUT
 lks32mc07x.clu.h, [264](#)
 CLU_MXB_CLU0_CLU2OUT
 lks32mc07x.clu.h, [264](#)
 CLU_MXB_CLU0_CLU3OUT
 lks32mc07x.clu.h, [265](#)
 CLU_MXB_CLU0_P0_11
 lks32mc07x.clu.h, [265](#)
 CLU_MXB_CLU0_P0_13
 lks32mc07x.clu.h, [265](#)
 CLU_MXB_CLU0_P0_15
 lks32mc07x.clu.h, [265](#)
 CLU_MXB_CLU0_P0_2
 lks32mc07x.clu.h, [265](#)
 CLU_MXB_CLU0_P0_4
 lks32mc07x.clu.h, [265](#)
 CLU_MXB_CLU0_P0_6
 lks32mc07x.clu.h, [266](#)
 CLU_MXB_CLU0_P2_11
 lks32mc07x.clu.h, [266](#)
 CLU_MXB_CLU0_P2_7
 lks32mc07x.clu.h, [266](#)
 CLU_MXB_CLU0_PWM3
 lks32mc07x.clu.h, [266](#)
 CLU_MXB_CLU0_SPIMOSI
 lks32mc07x.clu.h, [266](#)
 CLU_MXB_CLU1_ADBUSY0
 lks32mc07x.clu.h, [266](#)
 CLU_MXB_CLU1_ADBUSY1
 lks32mc07x.clu.h, [267](#)
 CLU_MXB_CLU1_CLU0OUT
 lks32mc07x.clu.h, [267](#)
 CLU_MXB_CLU1_CLU1OUT
 lks32mc07x.clu.h, [267](#)
 CLU_MXB_CLU1_CLU2OUT
 lks32mc07x.clu.h, [267](#)
 CLU_MXB_CLU1_CLU3OUT
 lks32mc07x.clu.h, [267](#)
 CLU_MXB_CLU1_P0_12
 lks32mc07x.clu.h, [267](#)
 CLU_MXB_CLU1_P0_13
 lks32mc07x.clu.h, [268](#)
 CLU_MXB_CLU1_P0_3
 lks32mc07x.clu.h, [268](#)
 CLU_MXB_CLU1_P0_4
 lks32mc07x.clu.h, [268](#)
 CLU_MXB_CLU1_P0_7
 lks32mc07x.clu.h, [268](#)
 CLU_MXB_CLU1_P1_0
 lks32mc07x.clu.h, [268](#)
 CLU_MXB_CLU1_P2_11
 lks32mc07x.clu.h, [268](#)
 CLU_MXB_CLU1_P2_7
 lks32mc07x.clu.h, [269](#)
 CLU_MXB_CLU1_PWM1
 lks32mc07x.clu.h, [269](#)
 CLU_MXB_CLU1_SPIMISO
 lks32mc07x.clu.h, [269](#)
 CLU_MXB_CLU2_CLU0OUT
 lks32mc07x.clu.h, [269](#)
 CLU_MXB_CLU2_CLU1OUT
 lks32mc07x.clu.h, [269](#)
 CLU_MXB_CLU2_CLU2OUT
 lks32mc07x.clu.h, [269](#)
 CLU_MXB_CLU2_CLU3OUT
 lks32mc07x.clu.h, [270](#)
 CLU_MXB_CLU2_I2CSDA
 lks32mc07x.clu.h, [270](#)
 CLU_MXB_CLU2_P1_11
 lks32mc07x.clu.h, [270](#)
 CLU_MXB_CLU2_P1_14
 lks32mc07x.clu.h, [270](#)
 CLU_MXB_CLU2_P2_0
 lks32mc07x.clu.h, [270](#)
 CLU_MXB_CLU2_P2_13
 lks32mc07x.clu.h, [270](#)
 CLU_MXB_CLU2_P2_15
 lks32mc07x.clu.h, [271](#)
 CLU_MXB_CLU2_P2_3
 lks32mc07x.clu.h, [271](#)
 CLU_MXB_CLU2_P2_5
 lks32mc07x.clu.h, [271](#)
 CLU_MXB_CLU2_P3_9
 lks32mc07x.clu.h, [271](#)
 CLU_MXB_CLU2_PWM1
 lks32mc07x.clu.h, [271](#)
 CLU_MXB_CLU2_PWM3
 lks32mc07x.clu.h, [271](#)
 CLU_MXB_CLU2_SPICLK
 lks32mc07x.clu.h, [272](#)
 CLU_MXB_CLU3_CLU0OUT
 lks32mc07x.clu.h, [272](#)
 CLU_MXB_CLU3_CLU1OUT
 lks32mc07x.clu.h, [272](#)
 CLU_MXB_CLU3_CLU2OUT

lks32mc07x_clu.h, 272
 CLU_MXB_CLU3_CLU3OUT
 lks32mc07x_clu.h, 272
 CLU_MXB_CLU3_I2CSCL
 lks32mc07x_clu.h, 272
 CLU_MXB_CLU3_P1_10
 lks32mc07x_clu.h, 273
 CLU_MXB_CLU3_P1_11
 lks32mc07x_clu.h, 273
 CLU_MXB_CLU3_P1_13
 lks32mc07x_clu.h, 273
 CLU_MXB_CLU3_P1_14
 lks32mc07x_clu.h, 273
 CLU_MXB_CLU3_P2_14
 lks32mc07x_clu.h, 273
 CLU_MXB_CLU3_P2_15
 lks32mc07x_clu.h, 273
 CLU_MXB_CLU3_P2_4
 lks32mc07x_clu.h, 274
 CLU_MXB_CLU3_P2_5
 lks32mc07x_clu.h, 274
 CLU_MXB_CLU3_PWM0
 lks32mc07x_clu.h, 274
 CLU_MXB_CLU3_PWM2
 lks32mc07x_clu.h, 274
 CLU_MXB_CLU3_SPICSN
 lks32mc07x_clu.h, 274
 CLU_ReadOutVolve
 Clu, 13
 CLU_StructInit
 Clu, 13
 CLU_TypeDef, 57
 CF, 58
 EN, 58
 FN, 58
 IE, 58
 IF, 58
 MX, 58
 OUT, 58
 CMP
 lks32mc07x_cmp.h, 280
 CMP0
 CMP_InitTypeDef, 60
 TIM_TimerTypeDef, 180
 CMP0_SELN_HALL0_MID
 lks32mc07x_cmp.h, 280
 CMP0_SEL_P0
 lks32mc07x_cmp.h, 280
 CMP0_SEL_P1
 lks32mc07x_cmp.h, 281
 CMP0_SEL_P2
 lks32mc07x_cmp.h, 281
 CMP0_SEL_P3
 lks32mc07x_cmp.h, 281
 CMP0_SEL_P4
 lks32mc07x_cmp.h, 281
 CMP0_SEL_OPA0_IP
 lks32mc07x_cmp.h, 281
 CMP0_SEL_P_OPA0_OUT
 lks32mc07x_cmp.h, 281
 CMP0_SEL_P_OPA1_OUT
 lks32mc07x_cmp.h, 282
 CMP1
 CMP_InitTypeDef, 60
 TIM_TimerTypeDef, 180
 CMP1_SELN_HALL1_MID
 lks32mc07x_cmp.h, 282
 CMP1_SEL_P0
 lks32mc07x_cmp.h, 282
 CMP1_SEL_P0
 lks32mc07x_cmp.h, 282
 CMP1_SEL_P1
 lks32mc07x_cmp.h, 282
 CMP1_SEL_P2
 lks32mc07x_cmp.h, 282
 CMP1_SEL_P3
 lks32mc07x_cmp.h, 283
 CMP1_SEL_P_OPA2_OUT
 lks32mc07x_cmp.h, 283
 CMP1_SEL_P_OPA3_IP
 lks32mc07x_cmp.h, 283
 CMP1_SEL_P_OPA3_OUT
 lks32mc07x_cmp.h, 283
 CMP2
 CMP_InitTypeDef, 60
 CMP2_SELN_DAC1
 lks32mc07x_cmp.h, 283
 CMP2_SEL_P0
 lks32mc07x_cmp.h, 283
 CMP2_SEL_P1
 lks32mc07x_cmp.h, 284
 CMP2_SEL_P_OPA2_OUT
 lks32mc07x_cmp.h, 284
 CMP2_SEL_P_OPA3_OUT
 lks32mc07x_cmp.h, 284
 CMP_CHN_0
 lks32mc07x_cmp.h, 284
 CMP_CHN_1
 lks32mc07x_cmp.h, 284
 CMP_CHN_2
 lks32mc07x_cmp.h, 284
 CMP_ClearIRQFlag
 lks32mc07x_cmp.c, 632
 lks32mc07x_cmp.h, 286
 CMP_Cmd
 lks32mc07x_cmp.h, 287
 CMP_CTRL_CNT0
 MCPWM_InitTypeDef, 112
 CMP_CTRL_CNT1
 MCPWM_InitTypeDef, 112
 CMP_EN
 stru_DMA_REN, 156
 CMP_GetFilterValue
 lks32mc07x_cmp.c, 633
 lks32mc07x_cmp.h, 287
 CMP_GetIRQFlag

- lks32mc07x_cmp.c, 634
- lks32mc07x_cmp.h, 288
- CMP_GetValue
 - lks32mc07x_cmp.c, 635
 - lks32mc07x_cmp.h, 289
- CMP_HIGH_LEVEL
 - lks32mc07x_cmp.h, 285
- CMP_HYS_0mV
 - lks32mc07x_cmp.h, 285
- CMP_HYS_20mV
 - lks32mc07x_cmp.h, 285
- CMP_Init
 - lks32mc07x_cmp.c, 636
 - lks32mc07x_cmp.h, 290
- CMP_InitTypeDef, 59
 - CLK_COM_DIV, 60
 - CMP0, 60
 - CMP1, 60
 - CMP2, 60
 - FT, 60
 - HYS, 60
- CMP_LOW_LEVEL
 - lks32mc07x_cmp.h, 285
- CMP_SELN_DAC0
 - lks32mc07x_cmp.h, 285
- CMP_SELN_IN
 - lks32mc07x_cmp.h, 285
- CMP_SELN_REF
 - lks32mc07x_cmp.h, 286
- CMP_SingleTypeDef, 61
 - CHN0P_PWM_EN, 62
 - CHN1P_PWM_EN, 62
 - CHN2P_PWM_EN, 62
 - CHN3P_PWM_EN, 62
 - CLK_EN, 62
 - EN, 62
 - FIL_CLK_DIV16, 62
 - FIL_CLK_DIV2, 63
 - IE, 63
 - IN_EN, 63
 - IRQ_TRIG, 63
 - POL, 63
 - RE, 64
 - SELN, 64
 - SELP, 64
 - W_PWM_POL, 64
- CMP_StructInit
 - lks32mc07x_cmp.c, 637
 - lks32mc07x_cmp.h, 291
- CMP_TypeDef, 65
 - BLCWIN, 65
 - CFG, 65
 - DATA, 66
 - IE, 66
 - IF, 66
 - TCLK, 66
- CNT
 - HALL_TypeDef, 86
 - IWDG_TypeDef, 93
 - TIM_QETypeDef, 168
 - TIM_TimerTypeDef, 180
- CNT0
 - MCPWM_TypeDef, 136
- CNT1
 - MCPWM_TypeDef, 136
- COS
 - DSP_Typedef, 76
- CountTH
 - HALL_InitTypeDef, 84
- CPHA
 - SPI_InitTypeDef, 150
- CPOL
 - SPI_InitTypeDef, 150
- CR
 - CRC_TypeDef, 69
- CR0
 - EXTI_TypeDef, 79
- CR1
 - EXTI_TypeDef, 79
- Crc, 14
 - CRC_Calc, 14
 - CRC_CalcCommon, 15
 - CRC_Disable, 15
 - CRC_Enable, 15
 - CRC_Init, 16
 - CRC_StructInit, 17
 - CRC_XOROUT, 17
- CRC_Calc
 - Crc, 14
- CRC_CalcCommon
 - Crc, 15
- CRC_Disable
 - Crc, 15
- CRC_Enable
 - Crc, 15
- CRC_Init
 - Crc, 16
- CRC_InitTypeDef, 66
 - INIT, 67
 - null, 67
 - POLY, 67
 - REFIN, 67
 - REFOUT, 68
 - WIDTH, 68
 - XOROUT, 68
- CRC_MODEL_CRC_16_CCITT
 - lks32mc07x_crc.h, 296
- CRC_MODEL_CRC_16_CCITT_FALSE
 - lks32mc07x_crc.h, 296
- CRC_MODEL_CRC_16_DNP
 - lks32mc07x_crc.h, 296
- CRC_MODEL_CRC_16_IBMx1
 - lks32mc07x_crc.h, 296
- CRC_MODEL_CRC_16_MAXIM
 - lks32mc07x_crc.h, 296
- CRC_MODEL_CRC_16_MODBU

lks32mc07x_crc.h, [296](#)
 CRC_MODEL_CRC_16_USBx1
 lks32mc07x_crc.h, [296](#)
 CRC_MODEL_CRC_16_X25
 lks32mc07x_crc.h, [297](#)
 CRC_MODEL_CRC_16_XMODEM
 lks32mc07x_crc.h, [297](#)
 CRC_MODEL_CRC_32
 lks32mc07x_crc.h, [297](#)
 CRC_MODEL_CRC_32_MPEG_2
 lks32mc07x_crc.h, [297](#)
 CRC_MODEL_CRC_8
 lks32mc07x_crc.h, [297](#)
 CRC_MODEL_CRC_8_ITU
 lks32mc07x_crc.h, [297](#)
 CRC_MODEL_CRC_8_MAXIMx
 lks32mc07x_crc.h, [298](#)
 CRC_MODEL_CRC_8_ROHC
 lks32mc07x_crc.h, [298](#)
 CRC_StructInit
 Crc, [17](#)
 CRC_TypeDef, [68](#)
 CR, [69](#)
 DR, [69](#)
 INIT, [69](#)
 POL, [69](#)
 CRC_WIDTH_16
 lks32mc07x_crc.h, [298](#)
 CRC_WIDTH_32
 lks32mc07x_crc.h, [298](#)
 CRC_WIDTH_7
 lks32mc07x_crc.h, [298](#)
 CRC_WIDTH_8
 lks32mc07x_crc.h, [298](#)
 CRC_XOROUT
 Crc, [17](#)
 CS
 SPI_InitTypeDef, [150](#)
 CSMP
 ADC_InitTypeDef, [26](#)
 CTMS
 DMA_RegTypeDef, [75](#)
 CTRL
 DMA_MODULERegTypeDef, [74](#)
 UART_TypeDef, [186](#)
 DAC0
 AFE_TypeDef, [40](#)
 DAC0_AMC
 AFE_TypeDef, [41](#)
 DAC0_DC
 AFE_TypeDef, [41](#)
 DAC0_UTIMER0_TRG_EN
 lks32mc07x_dac.h, [301](#)
 DAC0_UTIMER1_TRG_EN
 lks32mc07x_dac.h, [301](#)
 DAC0_UTIMER2_TRG_EN
 lks32mc07x_dac.h, [302](#)
 DAC0_UTIMER3_TRG_EN
 lks32mc07x_dac.h, [302](#)
 DAC1
 AFE_TypeDef, [41](#)
 DAC1_AMC
 AFE_TypeDef, [41](#)
 DAC1_DC
 AFE_TypeDef, [41](#)
 DAC1_UTIMER0_TRG_EN
 lks32mc07x_dac.h, [302](#)
 DAC1_UTIMER1_TRG_EN
 lks32mc07x_dac.h, [302](#)
 DAC1_UTIMER2_TRG_EN
 lks32mc07x_dac.h, [302](#)
 DAC1_UTIMER3_TRG_EN
 lks32mc07x_dac.h, [302](#)
 DAC_Channel_0
 lks32mc07x_dac.h, [303](#)
 DAC_Channel_1
 lks32mc07x_dac.h, [303](#)
 DAC_Cmd
 lks32mc07x_dac.c, [644](#)
 lks32mc07x_dac.h, [303](#)
 DAC_CTRL
 AFE_TypeDef, [41](#)
 DAC_GAIN
 DAC_InitTypeDef, [70](#)
 DAC_Init
 lks32mc07x_dac.c, [645](#)
 lks32mc07x_dac.h, [304](#)
 DAC_InitTypeDef, [69](#)
 DAC_GAIN, [70](#)
 DAC_STEP, [70](#)
 DACOUT_EN, [70](#)
 TIG_CH_EN, [70](#)
 DAC_Output
 lks32mc07x_dac.c, [646](#)
 lks32mc07x_dac.h, [306](#)
 DAC_OutputValue
 lks32mc07x_dac.c, [646](#)
 lks32mc07x_dac.h, [307](#)
 DAC_OutputVoltage
 lks32mc07x_dac.c, [647](#)
 lks32mc07x_dac.h, [308](#)
 DAC_RANGE_1V2
 lks32mc07x_dac.h, [303](#)
 DAC_RANGE_4V85
 lks32mc07x_dac.h, [303](#)
 DAC_STEP
 DAC_InitTypeDef, [70](#)
 DAC_StructInit
 lks32mc07x_dac.c, [648](#)
 lks32mc07x_dac.h, [309](#)
 DACOUT_EN
 DAC_InitTypeDef, [70](#)
 DADR
 DMA_InitTypeDef, [72](#)
 DMA_RegTypeDef, [75](#)
 DAT0

- ADC_TypeDef, [32](#)
- lks32mc07x_adc.h, [202](#)
- DAT1
 - ADC_TypeDef, [32](#)
 - lks32mc07x_adc.h, [202](#)
- DAT10
 - ADC_TypeDef, [32](#)
 - lks32mc07x_adc.h, [203](#)
- DAT11
 - ADC_TypeDef, [33](#)
 - lks32mc07x_adc.h, [203](#)
- DAT12
 - ADC_TypeDef, [33](#)
 - lks32mc07x_adc.h, [203](#)
- DAT13
 - ADC_TypeDef, [33](#)
 - lks32mc07x_adc.h, [203](#)
- DAT14
 - lks32mc07x_adc.h, [203](#)
- DAT15
 - lks32mc07x_adc.h, [203](#)
- DAT2
 - ADC_TypeDef, [33](#)
 - lks32mc07x_adc.h, [202](#)
- DAT3
 - ADC_TypeDef, [33](#)
 - lks32mc07x_adc.h, [202](#)
- DAT4
 - ADC_TypeDef, [33](#)
 - lks32mc07x_adc.h, [203](#)
- DAT5
 - ADC_TypeDef, [34](#)
 - lks32mc07x_adc.h, [203](#)
- DAT6
 - ADC_TypeDef, [34](#)
 - lks32mc07x_adc.h, [203](#)
- DAT7
 - ADC_TypeDef, [34](#)
 - lks32mc07x_adc.h, [203](#)
- DAT8
 - ADC_TypeDef, [34](#)
 - lks32mc07x_adc.h, [203](#)
- DAT9
 - ADC_TypeDef, [34](#)
 - lks32mc07x_adc.h, [203](#)
- DATA
 - CMP_TypeDef, [66](#)
 - I2C_TypeDef, [91](#)
- DATA_ALIGN
 - ADC_InitTypeDef, [26](#)
- DATx
 - lks32mc07x_adc.h, [202](#)
- DBG
 - AFE_TypeDef, [41](#)
- DBG_CFG
 - SYS_TypeDef, [163](#)
- DBTW
 - DMA_InitTypeDef, [72](#)
- DC0
 - ADC_TypeDef, [34](#)
- DC1
 - ADC_TypeDef, [35](#)
- DeadTimeCH012N
 - MCPWM_InitTypeDef, [113](#)
- DeadTimeCH012P
 - MCPWM_InitTypeDef, [113](#)
- DeadTimeCH345N
 - MCPWM_InitTypeDef, [113](#)
- DeadTimeCH345P
 - MCPWM_InitTypeDef, [113](#)
- Delay
 - lks32mc07x_can.c, [615](#)
- DID
 - DSP_Typedef, [76](#)
- DINC
 - DMA_InitTypeDef, [72](#)
- DIS
 - DSP_Typedef, [76](#)
- DIVH
 - UART_TypeDef, [186](#)
- DIVL
 - UART_TypeDef, [186](#)
- DMA_BYTE_TRANS
 - lks32mc07x_dma.h, [313](#)
- DMA_CHN0
 - lks32mc07x_dma.h, [313](#)
- DMA_CHN1
 - lks32mc07x_dma.h, [313](#)
- DMA_CHN2
 - lks32mc07x_dma.h, [313](#)
- DMA_CHN3
 - lks32mc07x_dma.h, [313](#)
- DMA_CHx_EN
 - lks32mc07x_dma.c, [652](#)
 - lks32mc07x_dma.h, [318](#)
- DMA_ClearIRQFlag
 - lks32mc07x_dma.c, [653](#)
 - lks32mc07x_dma.h, [319](#)
- DMA_Disable
 - lks32mc07x_dma.c, [654](#)
 - lks32mc07x_dma.h, [320](#)
- DMA_Enable
 - lks32mc07x_dma.c, [655](#)
 - lks32mc07x_dma.h, [321](#)
- DMA_GetFlag
 - lks32mc07x_dma.c, [656](#)
 - lks32mc07x_dma.h, [322](#)
- DMA_GetIRQFlag
 - lks32mc07x_dma.c, [656](#)
 - lks32mc07x_dma.h, [323](#)
- DMA_HALFWORD_TRANS
 - lks32mc07x_dma.h, [313](#)
- DMA_IE_CH0
 - lks32mc07x_dma.h, [313](#)
- DMA_IE_CH1
 - lks32mc07x_dma.h, [314](#)

DMA_IE_CH2
 lks32mc07x_dma.h, 314
 DMA_IE_CH3
 lks32mc07x_dma.h, 314
 DMA_IF_CH0
 lks32mc07x_dma.h, 314
 DMA_IF_CH1
 lks32mc07x_dma.h, 314
 DMA_IF_CH2
 lks32mc07x_dma.h, 314
 DMA_IF_CH3
 lks32mc07x_dma.h, 314
 DMA_Init
 lks32mc07x_dma.c, 658
 lks32mc07x_dma.h, 324
 DMA_InitTypeDef, 71
 CIRC, 72
 DADR, 72
 DBTW, 72
 DINC, 72
 EN, 72
 IRQ_EN, 72
 REN, 72
 RMODE, 72
 SADR, 73
 SBTW, 73
 SINC, 73
 TMS, 73
 DMA_MODULERegTypeDef, 73
 CTRL, 74
 IE, 74
 IF, 74
 DMA_RegTypeDef, 74
 CCR, 75
 CTMS, 75
 DADR, 75
 REN, 75
 SADR, 75
 DMA_REQ_ADC0
 lks32mc07x_dma.h, 315
 DMA_REQ_ADC1
 lks32mc07x_dma.h, 315
 DMA_REQ_CMP
 lks32mc07x_dma.h, 315
 DMA_REQ_GPIO
 lks32mc07x_dma.h, 315
 DMA_REQ_HALL
 lks32mc07x_dma.h, 315
 DMA_REQ_I2C0_RX
 lks32mc07x_dma.h, 315
 DMA_REQ_I2C0_TX
 lks32mc07x_dma.h, 315
 DMA_REQ_MCPWM0
 lks32mc07x_dma.h, 315
 DMA_REQ_MCPWM1
 lks32mc07x_dma.h, 316
 DMA_REQ_SIF
 lks32mc07x_dma.h, 316
 DMA_REQ_SPI_RX
 lks32mc07x_dma.h, 316
 DMA_REQ_SPI_TX
 lks32mc07x_dma.h, 316
 DMA_REQ_SW
 lks32mc07x_dma.h, 316
 DMA_REQ_TIMER0
 lks32mc07x_dma.h, 316
 DMA_REQ_TIMER1
 lks32mc07x_dma.h, 316
 DMA_REQ_TIMER2
 lks32mc07x_dma.h, 317
 DMA_REQ_TIMER3
 lks32mc07x_dma.h, 317
 DMA_REQ_TIMER4
 lks32mc07x_dma.h, 317
 DMA_REQ_UART0_RX
 lks32mc07x_dma.h, 317
 DMA_REQ_UART0_TX
 lks32mc07x_dma.h, 317
 DMA_REQ_UART1_RX
 lks32mc07x_dma.h, 317
 DMA_REQ_UART1_TX
 lks32mc07x_dma.h, 317
 DMA_SetAddr
 lks32mc07x_dma.c, 659
 lks32mc07x_dma.h, 326
 DMA_SetLength
 lks32mc07x_dma.c, 660
 lks32mc07x_dma.h, 327
 DMA_SetRequest
 lks32mc07x_dma.c, 661
 lks32mc07x_dma.h, 328
 DMA_StructInit
 lks32mc07x_dma.c, 663
 lks32mc07x_dma.h, 329
 DMA_TCIE
 lks32mc07x_dma.h, 317
 DMA_TEIE
 lks32mc07x_dma.h, 318
 DMA_TOP
 lks32mc07x_dma.h, 318
 DMA_Trigger
 lks32mc07x_dma.c, 664
 lks32mc07x_dma.h, 330
 DMA_WORD_TRANS
 lks32mc07x_dma.h, 318
 DMARE
 UART_InitTypeDef, 182
 DR
 CRC_TypeDef, 69
 DSP0_arctan
 lks32mc07x_dsp.h, 334
 DSP0_cos
 lks32mc07x_dsp.h, 335
 DSP0_div
 lks32mc07x_dsp.h, 335
 DSP0_mod

- lks32mc07x_dsp.h, 335
- DSP0_rms
 - lks32mc07x_dsp.h, 335
- DSP0_sin
 - lks32mc07x_dsp.h, 335
- DSP0_sqrt
 - lks32mc07x_dsp.h, 335
- DSP_arctan
 - lks32mc07x_dsp.c, 667
 - lks32mc07x_dsp.h, 338
- DSP_ARCTAN_MODE
 - lks32mc07x_dsp.h, 335
- DSP_ArctanMode
 - lks32mc07x_dsp.c, 668
 - lks32mc07x_dsp.h, 339
- DSP_CALC_ARCTAN
 - lks32mc07x_dsp.h, 335
- DSP_CALC_COS
 - lks32mc07x_dsp.h, 336
- DSP_CALC_DIV
 - lks32mc07x_dsp.h, 336
- DSP_CALC_MOD
 - lks32mc07x_dsp.h, 336
- DSP_CALC_RMS
 - lks32mc07x_dsp.h, 337
- DSP_CALC_SIN
 - lks32mc07x_dsp.h, 337
- DSP_CALC_SQRT
 - lks32mc07x_dsp.h, 337
- DSP_cos
 - lks32mc07x_dsp.c, 668
 - lks32mc07x_dsp.h, 339
- DSP_div
 - lks32mc07x_dsp.c, 669
 - lks32mc07x_dsp.h, 340
- DSP_EN
 - stru_DMA_REN, 156
- DSP_Init
 - lks32mc07x_dsp.c, 670
 - lks32mc07x_dsp.h, 341
- DSP_mod
 - lks32mc07x_dsp.c, 671
 - lks32mc07x_dsp.h, 342
- DSP_REG
 - lks32mc07x_dsp.h, 338
- DSP_rms
 - lks32mc07x_dsp.c, 672
 - lks32mc07x_dsp.h, 343
- DSP_sin
 - lks32mc07x_dsp.c, 673
 - lks32mc07x_dsp.h, 344
- DSP_SIN_COS_MODE
 - lks32mc07x_dsp.h, 338
- DSP_SinCosMode
 - lks32mc07x_dsp.c, 674
 - lks32mc07x_dsp.h, 345
- DSP_sqrt
 - lks32mc07x_dsp.c, 674
- lks32mc07x_dsp.h, 345
- DSP_Typedef, 76
 - ARCTAN, 76
 - COS, 76
 - DID, 76
 - DIS, 76
 - MOD, 77
 - PC, 77
 - QUO, 77
 - RAD, 77
 - REM, 77
 - SC, 77
 - SIN, 77
 - SQRT, 77
 - THETA, 78
 - X, 78
 - Y, 78
- DTH00
 - MCPWM_TypeDef, 136
- DTH01
 - MCPWM_TypeDef, 137
- DTH10
 - MCPWM_TypeDef, 137
- DTH11
 - MCPWM_TypeDef, 137
- DUPLEX
 - UART_InitTypeDef, 183
- Duplex
 - SPI_InitTypeDef, 150
- DWK_EN
 - IWDG_InitTypeDef, 92
- EALCAP
 - CAN_TypeDef, 48
- EIE
 - MCPWM_TypeDef, 137
- EIF
 - MCPWM_TypeDef, 137
- EN
 - CLU_InitTypeDef, 55
 - CLU_TypeDef, 58
 - CMP_SingleTypeDef, 62
 - DMA_InitTypeDef, 72
 - SPI_InitTypeDef, 151
- ENDIAN
 - SPI_InitTypeDef, 151
- enumOPA
 - lks32mc07x_opa.h, 453
- enumWakeTrigLevel
 - lks32mc07x_wake.h, 583
- erase_flag
 - lks32mc07x_flash.c, 693
 - lks32mc07x_flash.h, 360
- EraseSector
 - lks32mc07x_flash.c, 687
 - lks32mc07x_flash.h, 356
- ERRINT
 - CAN_TypeDef, 49
- ERRINT_IE

CAN_InitTypeDef, 45
 EVT
 TIM_TimerTypeDef, 180
 EVT0
 MCPWM_InitTypeDef, 113
 MCPWM_TypeDef, 137
 EVT1
 MCPWM_InitTypeDef, 113
 MCPWM_TypeDef, 138
 EVT_CNT0_EN
 MCPWM_InitTypeDef, 114
 EVT_CNT1_EN
 MCPWM_InitTypeDef, 114
 EXTI
 Iks32mc07x_exti.h, 350
 EXTI_ClearIRQFlag
 Iks32mc07x_exti.c, 678
 Iks32mc07x_exti.h, 350
 EXTI_GetIRQFlag
 Iks32mc07x_exti.c, 679
 Iks32mc07x_exti.h, 351
 EXTI_ReadIFFlag
 Iks32mc07x_exti.c, 680
 Iks32mc07x_exti.h, 352
 EXTI_ResetIF
 Iks32mc07x_exti.c, 680
 Iks32mc07x_exti.h, 352
 EXTI_Trigger_Config
 Iks32mc07x_exti.c, 680
 Iks32mc07x_exti.h, 352
 EXTI_Trigger_Edge
 Iks32mc07x_exti.h, 350
 EXTI_Trigger_Negedge
 Iks32mc07x_exti.h, 350
 EXTI_Trigger_None
 Iks32mc07x_exti.h, 350
 EXTI_Trigger_Posedge
 Iks32mc07x_exti.h, 350
 EXTI_TypeDef, 78
 CR0, 79
 CR1, 79
 IE, 79
 IF, 79
 EXTI_Mode_TypeDef
 Iks32mc07x_exti.h, 350
 F3210
 GPIO_TypeDef, 81
 F7654
 GPIO_TypeDef, 82
 FAIL012
 MCPWM_TypeDef, 138
 FAIL0_INPUT_EN
 MCPWM_InitTypeDef, 114
 FAIL0_INT_EN
 MCPWM_InitTypeDef, 114
 FAIL0_Polarity
 MCPWM_InitTypeDef, 114
 FAIL0_Signal_Sel
 MCPWM_InitTypeDef, 114
 FAIL1_INPUT_EN
 MCPWM_InitTypeDef, 115
 FAIL1_INT_EN
 MCPWM_InitTypeDef, 115
 FAIL1_Polarity
 MCPWM_InitTypeDef, 115
 FAIL1_Signal_Sel
 MCPWM_InitTypeDef, 115
 FAIL2_INPUT_EN
 MCPWM_InitTypeDef, 115
 FAIL2_INT_EN
 MCPWM_InitTypeDef, 115
 FAIL2_Polarity
 MCPWM_InitTypeDef, 116
 FAIL2_Signal_Sel
 MCPWM_InitTypeDef, 116
 FAIL345
 MCPWM_TypeDef, 138
 FAIL3_INPUT_EN
 MCPWM_InitTypeDef, 116
 FAIL3_INT_EN
 MCPWM_InitTypeDef, 116
 FAIL3_Polarity
 MCPWM_InitTypeDef, 116
 FAIL3_Signal_Sel
 MCPWM_InitTypeDef, 116
 FAIL_OCAP
 MCPWM_InitTypeDef, 117
 FAIL_1CAP
 MCPWM_InitTypeDef, 117
 FBA98
 GPIO_TypeDef, 82
 FCNT
 MCPWM_TypeDef, 138
 FFEDC
 GPIO_TypeDef, 82
 FIE
 CLU_InitTypeDef, 56
 FIL_CLK_DIV16
 CMP_SingleTypeDef, 62
 FIL_CLK_DIV2
 CMP_SingleTypeDef, 63
 Filter75_Ena
 HALL_InitTypeDef, 85
 FilterLen
 HALL_InitTypeDef, 85
 FirstSend
 UART_InitTypeDef, 183
 Flash_MAIN
 Iks32mc07x_flash.h, 356
 Flash_NVR
 Iks32mc07x_flash.h, 356
 FLSE
 SYS_TypeDef, 163
 FLSP
 SYS_TypeDef, 163
 FLST

- SYS_TypeDef, 163
- FLT
 - MCPWM_TypeDef, 138
 - TIM_TimerTypeDef, 180
- FN
 - CLU_InitTypeDef, 56
 - CLU_TypeDef, 58
- FT
 - CMP_InitTypeDef, 60
- g_rxMsg
 - lks32mc07x_can.c, 620
 - lks32mc07x_can.h, 237
- GAIN
 - ADC_InitTypeDef, 26
 - ADC_TypeDef, 35
- GEN
 - ADC_InitTypeDef, 26
 - ADC_TypeDef, 35
- GetBaudParam
 - lks32mc07x_can.c, 616
- GetCurrentTemperature
 - lks32mc07x_tmp.c, 830
 - lks32mc07x_tmp.h, 557
- GPIO0
 - lks32mc07x_gpio.h, 366
- GPIO1
 - lks32mc07x_gpio.h, 366
- GPIO2
 - lks32mc07x_gpio.h, 366
- GPIO3
 - lks32mc07x_gpio.h, 366
- GPIO_EN
 - stru_DMA_REN, 156
- GPIO_Init
 - lks32mc07x_gpio.c, 697
 - lks32mc07x_gpio.h, 373
- GPIO_InitTypeDef, 79
 - GPIO_Mode, 80
 - GPIO_PFLT, 80
 - GPIO_Pin, 80
 - GPIO_PODEna, 80
 - GPIO_PuPd, 80
- GPIO_Mode
 - GPIO_InitTypeDef, 80
- GPIO_Mode_ANA
 - lks32mc07x_gpio.h, 373
- GPIO_Mode_IN
 - lks32mc07x_gpio.h, 373
- GPIO_Mode_IO
 - lks32mc07x_gpio.h, 373
- GPIO_Mode_OUT
 - lks32mc07x_gpio.h, 373
- GPIO_PFLT
 - GPIO_InitTypeDef, 80
- GPIO_Pin
 - GPIO_InitTypeDef, 80
- GPIO_Pin_0
 - lks32mc07x_gpio.h, 366
- GPIO_Pin_1
 - lks32mc07x_gpio.h, 366
- GPIO_Pin_10
 - lks32mc07x_gpio.h, 367
- GPIO_Pin_11
 - lks32mc07x_gpio.h, 367
- GPIO_Pin_12
 - lks32mc07x_gpio.h, 367
- GPIO_Pin_13
 - lks32mc07x_gpio.h, 367
- GPIO_Pin_14
 - lks32mc07x_gpio.h, 367
- GPIO_Pin_15
 - lks32mc07x_gpio.h, 367
- GPIO_Pin_2
 - lks32mc07x_gpio.h, 368
- GPIO_Pin_3
 - lks32mc07x_gpio.h, 368
- GPIO_Pin_4
 - lks32mc07x_gpio.h, 368
- GPIO_Pin_5
 - lks32mc07x_gpio.h, 368
- GPIO_Pin_6
 - lks32mc07x_gpio.h, 368
- GPIO_Pin_7
 - lks32mc07x_gpio.h, 368
- GPIO_Pin_8
 - lks32mc07x_gpio.h, 369
- GPIO_Pin_9
 - lks32mc07x_gpio.h, 369
- GPIO_Pin_ALL
 - lks32mc07x_gpio.h, 369
- GPIO_Pin_NONE
 - lks32mc07x_gpio.h, 369
- GPIO_PinAFConfig
 - lks32mc07x_gpio.c, 698
 - lks32mc07x_gpio.h, 374
- GPIO_PinSource_0
 - lks32mc07x_gpio.h, 369
- GPIO_PinSource_1
 - lks32mc07x_gpio.h, 369
- GPIO_PinSource_10
 - lks32mc07x_gpio.h, 370
- GPIO_PinSource_11
 - lks32mc07x_gpio.h, 370
- GPIO_PinSource_12
 - lks32mc07x_gpio.h, 370
- GPIO_PinSource_13
 - lks32mc07x_gpio.h, 370
- GPIO_PinSource_14
 - lks32mc07x_gpio.h, 370
- GPIO_PinSource_15
 - lks32mc07x_gpio.h, 370
- GPIO_PinSource_2
 - lks32mc07x_gpio.h, 371
- GPIO_PinSource_3
 - lks32mc07x_gpio.h, 371
- GPIO_PinSource_4

- lks32mc07x_gpio.h, 371
- GPIO_PinSource_5
 - lks32mc07x_gpio.h, 371
- GPIO_PinSource_6
 - lks32mc07x_gpio.h, 371
- GPIO_PinSource_7
 - lks32mc07x_gpio.h, 371
- GPIO_PinSource_8
 - lks32mc07x_gpio.h, 372
- GPIO_PinSource_9
 - lks32mc07x_gpio.h, 372
- GPIO_PODEna
 - GPIO_InitTypeDef, 80
- GPIO_PuPd
 - GPIO_InitTypeDef, 80
- GPIO_PuPd_NOPULL
 - lks32mc07x_gpio.h, 373
- GPIO_PuPd_UP
 - lks32mc07x_gpio.h, 373
- GPIO_ReadInputData
 - lks32mc07x_gpio.c, 699
 - lks32mc07x_gpio.h, 376
- GPIO_ReadInputDataBit
 - lks32mc07x_gpio.c, 700
 - lks32mc07x_gpio.h, 377
- GPIO_ReadOutputData
 - lks32mc07x_gpio.c, 701
 - lks32mc07x_gpio.h, 378
- GPIO_ReadOutputDataBit
 - lks32mc07x_gpio.c, 702
 - lks32mc07x_gpio.h, 379
- GPIO_ResetBits
 - lks32mc07x_gpio.c, 703
 - lks32mc07x_gpio.h, 380
- GPIO_SetBits
 - lks32mc07x_gpio.c, 704
 - lks32mc07x_gpio.h, 381
- GPIO_StructInit
 - lks32mc07x_gpio.c, 705
 - lks32mc07x_gpio.h, 382
- GPIO_TypeDef, 81
 - BRR, 81
 - BSRR, 81
 - F3210, 81
 - F7654, 82
 - FBA98, 82
 - FFEDC, 82
 - null, 82
 - PDI, 82
 - PDO, 82
 - PFLT, 83
 - PIE, 83
 - PODE, 83
 - POE, 83
 - PUE, 83
- GPIO_Write
 - lks32mc07x_gpio.c, 706
 - lks32mc07x_gpio.h, 383
- GPIO_WriteBit
 - lks32mc07x_gpio.c, 707
 - lks32mc07x_gpio.h, 383
- GPIO_Mode_TypeDef
 - lks32mc07x_gpio.h, 372
- GPIO_PuPd_TypeDef
 - lks32mc07x_gpio.h, 373
- HALL
 - lks32mc07x_hall.h, 389
- HALL0_EN
 - stru_DMA_REN, 156
- HALL_CAPTURE_EVENT
 - lks32mc07x_hall.h, 389
- HALL_Clear_IRQ
 - lks32mc07x_hall.c, 712
 - lks32mc07x_hall.h, 390
- HALL_CLK_DIV1
 - lks32mc07x_hall.h, 389
- HALL_CLK_DIV2
 - lks32mc07x_hall.h, 389
- HALL_CLK_DIV4
 - lks32mc07x_hall.h, 389
- HALL_CLK_DIV8
 - lks32mc07x_hall.h, 389
- HALL_Cmd
 - lks32mc07x_hall.c, 713
 - lks32mc07x_hall.h, 391
- HALL_Ena
 - HALL_InitTypeDef, 85
- HALL_GetCaptureValue
 - lks32mc07x_hall.c, 714
 - lks32mc07x_hall.h, 392
- HALL_GetCount
 - lks32mc07x_hall.c, 715
 - lks32mc07x_hall.h, 393
- HALL_GetFilterValue
 - lks32mc07x_hall.c, 717
 - lks32mc07x_hall.h, 394
- HALL_GetFlag
 - lks32mc07x_hall.c, 718
 - lks32mc07x_hall.h, 395
- HALL_GetIRQFlag
 - lks32mc07x_hall.c, 718
 - lks32mc07x_hall.h, 395
- HALL_Init
 - lks32mc07x_hall.c, 720
 - lks32mc07x_hall.h, 396
- HALL_InitTypeDef, 84
 - Capture_IRQ_Ena, 84
 - ClockDivision, 84
 - CountTH, 84
 - Filter75_Ena, 85
 - FilterLen, 85
 - HALL_Ena, 85
 - OverFlow_IRQ_Ena, 85
 - softIE, 85
- HALL_IsCaptureEvent
 - lks32mc07x_hall.c, 721

lks32mc07x_hall.h, 397
 HALL_IsOverflowEvent
 lks32mc07x_hall.c, 722
 lks32mc07x_hall.h, 398
 HALL_OVERFLOW_EVENT
 lks32mc07x_hall.h, 389
 HALL_SoftTrigger
 lks32mc07x_hall.c, 723
 lks32mc07x_hall.h, 399
 HALL_StructInit
 lks32mc07x_hall.c, 724
 lks32mc07x_hall.h, 400
 HALL_TypeDef, 86
 CFG, 86
 CNT, 86
 INFO, 86
 TH, 86
 WIDTH, 87
 HALT_PRT0
 MCPWM_InitTypeDef, 117
 HALT_PRT1
 MCPWM_InitTypeDef, 117
 HTH
 ADC_InitTypeDef, 26
 ADC_TypeDef, 35
 HYS
 CMP_InitTypeDef, 60
 I2c, 17
 I2c_ClearBusState, 18
 I2C_ClearIRQFlag, 18
 I2C_Disable, 19
 I2C_Enable, 19
 I2C_GetFlag, 20
 I2C_GetIRQFlag, 20
 I2C_Init, 20
 I2c_ReadBusState, 21
 I2C_ReadData, 21
 I2C_SendData, 22
 I2C_SetBaudRate, 22
 I2C_StructInit, 23
 I2C0_RX_EN
 stru_DMA_REN, 156
 I2C0_TX_EN
 stru_DMA_REN, 157
 I2C_BUS_STATE_ADDR_DATA
 lks32mc07x_i2c.h, 404
 I2C_BUS_STATE_Done
 lks32mc07x_i2c.h, 404
 I2C_BUS_STATE_LOST_ARB
 lks32mc07x_i2c.h, 404
 I2C_BUS_STATE_RX_ACK
 lks32mc07x_i2c.h, 404
 I2C_BUS_STATE_STOP_EVT
 lks32mc07x_i2c.h, 404
 I2C_BUS_STATE_STT_ERR
 lks32mc07x_i2c.h, 405
 I2c_ClearBusState
 I2c, 18
 I2C_ClearIRQFlag
 I2c, 18
 I2C_Disable
 I2c, 19
 I2C_Enable
 I2c, 19
 I2C_GetFlag
 I2c, 20
 I2C_GetIRQFlag
 I2c, 20
 I2C_IF_ADDR_CMP
 lks32mc07x_i2c.h, 405
 I2C_IF_BUS_ERR
 lks32mc07x_i2c.h, 405
 I2C_IF_NACK
 lks32mc07x_i2c.h, 405
 I2C_IF_STOP
 lks32mc07x_i2c.h, 405
 I2C_IF_TC
 lks32mc07x_i2c.h, 405
 I2C_Init
 I2c, 20
 I2C_InitTypeDef, 87
 ADRCMP, 88
 BaudRate, 88
 BURST_ADDR_CMP, 88
 BURST_NACK, 88
 BUS_ERR_IE, 88
 BUSRT_EN, 88
 IE, 88
 MST_MODE, 89
 SLV_MODE, 89
 STOP_IE, 89
 TC_IE, 89
 I2c_ReadBusState
 I2c, 21
 I2C_ReadData
 I2c, 21
 I2C_SendData
 I2c, 22
 I2C_SetBaudRate
 I2c, 22
 I2C_StructInit
 I2c, 23
 I2C_TypeDef, 89
 ADDR, 90
 BCR, 90
 BSIZE, 90
 CFG, 90
 DATA, 91
 MSCR, 91
 SCR, 91
 ICHN
 ADC_TypeDef, 35
 id
 can_parameter, 46
 CANRxMsg, 52
 CANTxMsg, 53

ID0_Filter
 lks32mc07x_can.h, 223
 ID1_Filter
 lks32mc07x_can.h, 223
 ID2_Filter
 lks32mc07x_can.h, 223
 ID3_Filter
 lks32mc07x_can.h, 223
 IDAT0
 ADC_TypeDef, 35
 IDAT1
 ADC_TypeDef, 36
 ide
 can_parameter, 46
 CANRxMsg, 52
 CANTxMsg, 53
 IDEFilter_Ex
 lks32mc07x_can.h, 224
 IDEFilter_St
 lks32mc07x_can.h, 224
 IDEFilter_St_Ex
 lks32mc07x_can.h, 224
 IDEFilterOption
 lks32mc07x_can.h, 224
 IDx_Filter
 lks32mc07x_can.c, 616
 lks32mc07x_can.h, 236
 IDx_FilterCmd
 lks32mc07x_can.c, 617
 lks32mc07x_can.h, 236
 IE
 ADC_InitTypeDef, 26
 ADC_TypeDef, 36
 CAN_InitTypeDef, 45
 CLU_TypeDef, 58
 CMP_SingleTypeDef, 63
 CMP_TypeDef, 66
 DMA_MODULERegTypeDef, 74
 EXTI_TypeDef, 79
 I2C_InitTypeDef, 88
 SPI_TypeDef, 153
 TIM_QEPTypDef, 168
 TIM_TimerTypeDef, 180
 UART_TypeDef, 186
 IE0
 MCPWM_TypeDef, 138
 IE1
 MCPWM_TypeDef, 139
 IF
 ADC_TypeDef, 36
 CLU_TypeDef, 58
 CMP_TypeDef, 66
 DMA_MODULERegTypeDef, 74
 EXTI_TypeDef, 79
 TIM_QEPTypDef, 168
 TIM_TimerTypeDef, 181
 UART_TypeDef, 186
 IF0
 MCPWM_TypeDef, 139
 IF1
 MCPWM_TypeDef, 139
 IN_EN
 CMP_SingleTypeDef, 63
 include/lks32mc07x_adc.h, 189, 211
 include/lks32mc07x_can.h, 213, 238
 include/lks32mc07x_clu.h, 240, 275
 include/lks32mc07x_cmp.h, 277, 292
 include/lks32mc07x_crc.h, 294, 299
 include/lks32mc07x_dac.h, 300, 310
 include/lks32mc07x_dma.h, 310, 331
 include/lks32mc07x_dsp.h, 333, 346
 include/lks32mc07x_exti.h, 348, 354
 include/lks32mc07x_flash.h, 354, 360
 include/lks32mc07x_gpio.h, 361, 385
 include/lks32mc07x_hall.h, 386, 401
 include/lks32mc07x_i2c.h, 402, 406
 include/lks32mc07x_lib.h, 407, 410
 include/lks32mc07x_mcpwm.h, 410, 443
 include/lks32mc07x_nvr.h, 448, 450
 include/lks32mc07x_opa.h, 450, 456
 include/lks32mc07x_qep.h, 457, 469
 include/lks32mc07x_sif.h, 470, 480
 include/lks32mc07x_spi.h, 481, 491
 include/lks32mc07x_sys.h, 492, 526
 include/lks32mc07x_tim.h, 529, 554
 include/lks32mc07x_tmp.h, 555, 559
 include/lks32mc07x_uart.h, 559, 576
 include/lks32mc07x_wake.h, 577, 591
 INFO
 AFE_TypeDef, 41
 HALL_TypeDef, 86
 INIT
 CRC_InitTypeDef, 67
 CRC_TypeDef, 69
 IO
 TIM_TimerTypeDef, 181
 IO01
 MCPWM_TypeDef, 139
 IO0_PPE
 MCPWM_InitTypeDef, 117
 IO1_PPE
 MCPWM_InitTypeDef, 117
 IO23
 MCPWM_TypeDef, 139
 IO2_PPE
 MCPWM_InitTypeDef, 118
 IO3_PPE
 MCPWM_InitTypeDef, 118
 IO45
 MCPWM_TypeDef, 139
 IO4_PPE
 MCPWM_InitTypeDef, 118
 IO5_PPE
 MCPWM_InitTypeDef, 118
 IO_CFG
 SYS_TypeDef, 163

IO_CMP_FLT_CLKDIV
 MCPWM_InitTypeDef, 118
 IOC
 UART_TypeDef, 187
 IOWK_FLT_DI
 lks32mc07x_wake.h, 580
 IOWK_FLT_EN
 lks32mc07x_wake.h, 580
 IRQ_EDGE_TRIG_MODE
 lks32mc07x_cmp.h, 286
 IRQ_EN
 DMA_InitTypeDef, 72
 IRQ_LEVEL_TRIG_MODE
 lks32mc07x_cmp.h, 286
 IRQ_TRIG
 CMP_SingleTypeDef, 63
 IRQEna
 SPI_InitTypeDef, 151
 UART_InitTypeDef, 183
 IS1
 ADC_InitTypeDef, 27
 IWDG_DISABLE
 lks32mc07x_sys.c, 784
 lks32mc07x_sys.h, 504
 IWDG_ENABLE
 lks32mc07x_sys.c, 785
 lks32mc07x_sys.h, 505
 IWDG_Feed
 lks32mc07x_sys.c, 786
 lks32mc07x_sys.h, 506
 IWDG_Init
 lks32mc07x_sys.c, 787
 lks32mc07x_sys.h, 507
 IWDG_InitTypeDef, 91
 DWK_EN, 92
 RTH, 92
 WDG_EN, 92
 WTH, 92
 IWDG_StructInit
 lks32mc07x_sys.c, 788
 lks32mc07x_sys.h, 508
 IWDG_TypeDef, 92
 CFG, 93
 CLR, 93
 CNT, 93
 PSW, 93
 RTH, 94
 WTH, 94
 len
 can_parameter, 46
 CANRxMsg, 52
 CANTxMsg, 54
 LIMIT
 CAN_TypeDef, 49
 lks32mc07x_adc.c
 ADC_CHN_GAIN_CFG, 593
 ADC_ClearIRQFlag, 594
 ADC_GetConversionValue, 595
 ADC_GetIRQFlag, 596
 ADC_Init, 597
 ADC_SoftTrgEN, 598
 ADC_StructInit, 599
 lks32mc07x_adc.h
 ADC0, 193
 ADC1, 193
 ADC_ALL_IF, 193
 ADC_AWD0_IE, 193
 ADC_AWD0_IF, 193
 ADC_AWD0_RE, 193
 ADC_CHANNEL_0, 194
 ADC_CHANNEL_1, 194
 ADC_CHANNEL_10, 194
 ADC_CHANNEL_11, 194
 ADC_CHANNEL_12, 194
 ADC_CHANNEL_13, 194
 ADC_CHANNEL_14, 195
 ADC_CHANNEL_15, 195
 ADC_CHANNEL_2, 195
 ADC_CHANNEL_3, 195
 ADC_CHANNEL_4, 195
 ADC_CHANNEL_5, 195
 ADC_CHANNEL_6, 196
 ADC_CHANNEL_7, 196
 ADC_CHANNEL_8, 196
 ADC_CHANNEL_9, 196
 ADC_CHN_GAIN_CFG, 203
 ADC_ClearIRQFlag, 204
 ADC_GAIN3V6, 202
 ADC_GAIN7V2, 202
 ADC_GAINx, 201
 ADC_GetConversionValue, 205
 ADC_GetIRQFlag, 206
 ADC_HERR_IE, 196
 ADC_HERR_IF, 196
 ADC_HERR_RE, 197
 ADC_Init, 208
 ADC_ISF_IE, 197
 ADC_ISF_IF, 197
 ADC_ISF_RE, 197
 ADC_SERR_IE, 197
 ADC_SERR_IF, 197
 ADC_SERR_RE, 198
 ADC_SF1_IE, 198
 ADC_SF1_IF, 198
 ADC_SF1_RE, 198
 ADC_SF2_IE, 198
 ADC_SF2_IF, 198
 ADC_SF2_RE, 199
 ADC_SoftTrgEN, 209
 ADC_StructInit, 210
 ADC_TRIG_CL_IT0, 199
 ADC_TRIG_CL_IT1, 199
 ADC_TRIG_CL_IT2, 199
 ADC_TRIG_CL_IT3, 199
 ADC_TRIG_CL_NT0, 199
 ADC_TRIG_CL_NT1, 200

- ADC_TRIG_CL_NT2, 200
- ADC_TRIG_CL_NT3, 200
- ADC_TRIG_MCPWM0_T0, 200
- ADC_TRIG_MCPWM0_T1, 200
- ADC_TRIG_MCPWM0_T2, 200
- ADC_TRIG_MCPWM0_T3, 201
- ADC_TRIG_UTIMER0_CMP0, 201
- ADC_TRIG_UTIMER0_CMP1, 201
- ADC_TRIG_UTIMER2_CMP0, 201
- ADC_TRIG_UTIMER2_CMP1, 201
- CHN0, 202
- CHN1, 202
- CHN10, 202
- CHN11, 202
- CHN12, 202
- CHN13, 202
- CHN14, 202
- CHN15, 202
- CHN2, 202
- CHN3, 202
- CHN4, 202
- CHN5, 202
- CHN6, 202
- CHN7, 202
- CHN8, 202
- CHN9, 202
- CHNx, 202
- DAT0, 202
- DAT1, 202
- DAT10, 203
- DAT11, 203
- DAT12, 203
- DAT13, 203
- DAT14, 203
- DAT15, 203
- DAT2, 202
- DAT3, 202
- DAT4, 203
- DAT5, 203
- DAT6, 203
- DAT7, 203
- DAT8, 203
- DAT9, 203
- DATx, 202
- lks32mc07x.can.c
 - CAN_ArbitrationLostCap, 605
 - CAN_BusoffGet, 605
 - CAN_Disable, 605
 - CAN_Enable, 605
 - CAN_ErrorPassiveGet, 606
 - CAN_Init, 606
 - CAN_Init1, 607
 - CAN_KindOfErrorGet, 608
 - CAN_LBMECmd, 608
 - CAN_LBMICmd, 609
 - CAN_Receive_Msg, 609
 - CAN_Recv, 611
 - CAN_Reset, 611
- CAN_ROverflowGet, 612
- CAN_RxErrorCntGet, 612
- CAN_Send, 612
- CAN_Send_Cancel, 613
- CAN_Send_Trig, 613
- CAN_SetSTBMode, 614
- CAN_SS_Enable, 614
- CAN_STBFullGet, 614
- CAN_StructInit, 615
- CAN_TxErrorCntGet, 615
- CAN_Write_TBUF, 615
- Delay, 615
- g_rxMsg, 620
- GetBaudParam, 616
- IDx_Filter, 616
- IDx_FilterCmd, 617
- My_CAN_Send_Msg, 617
- Reset_CFG_STAT, 618
- Set_CFG_STAT, 619
- lks32mc07x.can.h
 - CAN, 218
 - CAN_ArbitrationLostCap, 225
 - CAN_BusoffGet, 226
 - CAN_Disable, 226
 - CAN_Enable, 226
 - CAN_ERRINT_ALIE, 218
 - CAN_ERRINT_ALIF, 218
 - CAN_ERRINT_BEIE, 218
 - CAN_ERRINT_BEIF, 218
 - CAN_ERRINT_EPASS, 218
 - CAN_ERRINT_EPIE, 218
 - CAN_ERRINT_EPIF, 219
 - CAN_ERRINT_EWARN, 219
 - CAN_ErrorPassiveGet, 227
 - CAN_Init, 227
 - CAN_Init1, 228
 - CAN_KindOfErrorGet, 229
 - CAN_LBMECmd, 229
 - CAN_LBMICmd, 230
 - CAN_MODE_LBME, 219
 - CAN_MODE_LBMI, 219
 - CAN_MODE_LOM, 219
 - CAN_MODE_RESET, 219
 - CAN_MODE_TPSS, 220
 - CAN_MODE_TSSS, 220
 - CAN_Receive_Msg, 230
 - CAN_Recv, 232
 - CAN_Reset, 232
 - CAN_ROverflowGet, 233
 - CAN_RTIE_EIE, 220
 - CAN_RTIE_RAFIE, 220
 - CAN_RTIE_RFIE, 220
 - CAN_RTIE_RIE, 220
 - CAN_RTIE_ROIE, 221
 - CAN_RTIE_TPIE, 221
 - CAN_RTIE_TSFF, 221
 - CAN_RTIE_TSIE, 221
 - CAN_RTIF_AIF, 221

CAN_RTIF_EIF, 221
CAN_RTIF_RAFIF, 222
CAN_RTIF_RFIF, 222
CAN_RTIF_RIF, 222
CAN_RTIF_ROIF, 222
CAN_RTIF_TPIF, 222
CAN_RTIF_TSIF, 222
CAN_RxErrorCntGet, 233
CAN_Send, 233
CAN_Send_Cancel, 234
CAN_Send_Trig, 234
CAN_SetSTBMode, 235
CAN_SS_Enable, 235
CAN_STBFullGet, 235
CAN_StructInit, 236
CAN_TxErrorCntGet, 236
CanErrorr_Ack, 224
CanErrorr_Bit, 224
CanErrorr_Crc, 224
CanErrorr_Form, 224
CanErrorr_Stuff, 224
CanErrorr_Unknown, 224
CanError_None, 224
CANErrorKind, 223
CanRecv_NoData, 224
CanRecv_Succ, 224
CanRecvRet, 224
CANRXMSG_HEADLEN, 223
CanSend_DataTooLong, 224
CanSend_NotIdle, 224
CanSend_STBFull, 224
CanSend_Succ, 224
CanSend_Timeout, 224
CanSendRet, 224
g_rxMsg, 237
ID0_Filter, 223
ID1_Filter, 223
ID2_Filter, 223
ID3_Filter, 223
IDFilter_Ex, 224
IDFilter_St, 224
IDFilter_St_Ex, 224
IDFilterOption, 224
IDx_Filter, 236
IDx_FilterCmd, 236
My_CAN_Send_Msg, 237
STB_FIFOmode, 225
STB_PriorityMode, 225
STB_TransAll, 225
STB_TransNo, 225
STB_TransOne, 225
STBTransMode, 225
STBTransOption, 225
TBSel_PTb, 225
TBSel_STB, 225
TBSelOption, 225
lks32mc07x_clu.h
CLU_FN_0, 248
CLU_FN_1, 248
CLU_FN_A, 248
CLU_FN_A_B, 248
CLU_FN_A_B_C, 248
CLU_FN_A_B_NOTC, 249
CLU_FN_A_C, 249
CLU_FN_A_NOTB, 249
CLU_FN_A_NOTB_C, 249
CLU_FN_A_NOTB_NOTC, 249
CLU_FN_A_NOTC, 249
CLU_FN_B, 250
CLU_FN_B_C, 250
CLU_FN_B_NOTC, 250
CLU_FN_C, 250
CLU_FN_NOTA, 250
CLU_FN_NOTA_B, 250
CLU_FN_NOTA_B_C, 251
CLU_FN_NOTA_B_NOTC, 251
CLU_FN_NOTA_C, 251
CLU_FN_NOTA_NOTB, 251
CLU_FN_NOTA_NOTB_C, 251
CLU_FN_NOTA_NOTB_NOTC, 251
CLU_FN_NOTA_NOTC, 252
CLU_FN_NOTB, 252
CLU_FN_NOTB_C, 252
CLU_FN_NOTB_NOTC, 252
CLU_FN_NOTC, 252
CLU_MODEL_CLU0, 252
CLU_MODEL_CLU1, 253
CLU_MODEL_CLU2, 253
CLU_MODEL_CLU3, 253
CLU_MXA_CLU0_CLU0OUT, 253
CLU_MXA_CLU0_CLU1OUT, 253
CLU_MXA_CLU0_CLU2OUT, 253
CLU_MXA_CLU0_CLU3OUT, 254
CLU_MXA_CLU0_P0_0, 254
CLU_MXA_CLU0_P0_12, 254
CLU_MXA_CLU0_P0_14, 254
CLU_MXA_CLU0_P0_3, 254
CLU_MXA_CLU0_P0_5, 254
CLU_MXA_CLU0_P0_7, 255
CLU_MXA_CLU0_P1_0, 255
CLU_MXA_CLU0_P2_12, 255
CLU_MXA_CLU0_PWM0, 255
CLU_MXA_CLU0_PWM1, 255
CLU_MXA_CLU0_Timer0ch0ovfl, 255
CLU_MXA_CLU0_Timer1ch0ovfl, 256
CLU_MXA_CLU1_CLU0OUT, 256
CLU_MXA_CLU1_CLU1OUT, 256
CLU_MXA_CLU1_CLU2OUT, 256
CLU_MXA_CLU1_CLU3OUT, 256
CLU_MXA_CLU1_P0_0, 256
CLU_MXA_CLU1_P0_11, 257
CLU_MXA_CLU1_P0_14, 257
CLU_MXA_CLU1_P0_15, 257
CLU_MXA_CLU1_P0_2, 257
CLU_MXA_CLU1_P0_5, 257
CLU_MXA_CLU1_P0_6, 257

CLU_MXA_CLU1_P2.12, 258
 CLU_MXA_CLU1_PWM2, 258
 CLU_MXA_CLU1_PWM3, 258
 CLU_MXA_CLU1_Timer0ch1ovfl, 258
 CLU_MXA_CLU1_Timer1ch1ovfl, 258
 CLU_MXA_CLU2_CLU0OUT, 258
 CLU_MXA_CLU2_CLU1OUT, 259
 CLU_MXA_CLU2_CLU2OUT, 259
 CLU_MXA_CLU2_CLU3OUT, 259
 CLU_MXA_CLU2_P1.10, 259
 CLU_MXA_CLU2_P1.13, 259
 CLU_MXA_CLU2_P1.15, 259
 CLU_MXA_CLU2_P2.1, 260
 CLU_MXA_CLU2_P2.14, 260
 CLU_MXA_CLU2_P2.4, 260
 CLU_MXA_CLU2_P2.6, 260
 CLU_MXA_CLU2_P2.8, 260
 CLU_MXA_CLU2_PWM0, 260
 CLU_MXA_CLU2_PWM2, 261
 CLU_MXA_CLU2_Timer2ch0ovfl, 261
 CLU_MXA_CLU2_Timer3ch0ovfl, 261
 CLU_MXA_CLU3_CLU0OUT, 261
 CLU_MXA_CLU3_CLU1OUT, 261
 CLU_MXA_CLU3_CLU2OUT, 261
 CLU_MXA_CLU3_CLU3OUT, 262
 CLU_MXA_CLU3_P1.15, 262
 CLU_MXA_CLU3_P2.0, 262
 CLU_MXA_CLU3_P2.1, 262
 CLU_MXA_CLU3_P2.13, 262
 CLU_MXA_CLU3_P2.3, 262
 CLU_MXA_CLU3_P2.6, 263
 CLU_MXA_CLU3_P2.8, 263
 CLU_MXA_CLU3_P3.9, 263
 CLU_MXA_CLU3_PWM1, 263
 CLU_MXA_CLU3_PWM3, 263
 CLU_MXA_CLU3_Timer2ch1ovfl, 263
 CLU_MXA_CLU3_Timer3ch1ovfl, 264
 CLU_MXB_CLU0_ADBUSY0, 264
 CLU_MXB_CLU0_ADBUSY1, 264
 CLU_MXB_CLU0_CLU0OUT, 264
 CLU_MXB_CLU0_CLU1OUT, 264
 CLU_MXB_CLU0_CLU2OUT, 264
 CLU_MXB_CLU0_CLU3OUT, 265
 CLU_MXB_CLU0_P0.11, 265
 CLU_MXB_CLU0_P0.13, 265
 CLU_MXB_CLU0_P0.15, 265
 CLU_MXB_CLU0_P0.2, 265
 CLU_MXB_CLU0_P0.4, 265
 CLU_MXB_CLU0_P0.6, 266
 CLU_MXB_CLU0_P2.11, 266
 CLU_MXB_CLU0_P2.7, 266
 CLU_MXB_CLU0_PWM3, 266
 CLU_MXB_CLU0_SPIMOSI, 266
 CLU_MXB_CLU1_ADBUSY0, 266
 CLU_MXB_CLU1_ADBUSY1, 267
 CLU_MXB_CLU1_CLU0OUT, 267
 CLU_MXB_CLU1_CLU1OUT, 267
 CLU_MXB_CLU1_CLU2OUT, 267
 CLU_MXB_CLU1_CLU3OUT, 267
 CLU_MXB_CLU1_P0.12, 267
 CLU_MXB_CLU1_P0.13, 268
 CLU_MXB_CLU1_P0.3, 268
 CLU_MXB_CLU1_P0.4, 268
 CLU_MXB_CLU1_P0.7, 268
 CLU_MXB_CLU1_P1.0, 268
 CLU_MXB_CLU1_P2.11, 268
 CLU_MXB_CLU1_P2.7, 269
 CLU_MXB_CLU1_PWM1, 269
 CLU_MXB_CLU1_SPIMISO, 269
 CLU_MXB_CLU2_CLU0OUT, 269
 CLU_MXB_CLU2_CLU1OUT, 269
 CLU_MXB_CLU2_CLU2OUT, 269
 CLU_MXB_CLU2_CLU3OUT, 270
 CLU_MXB_CLU2_I2CSDA, 270
 CLU_MXB_CLU2_P1.11, 270
 CLU_MXB_CLU2_P1.14, 270
 CLU_MXB_CLU2_P2.0, 270
 CLU_MXB_CLU2_P2.13, 270
 CLU_MXB_CLU2_P2.15, 271
 CLU_MXB_CLU2_P2.3, 271
 CLU_MXB_CLU2_P2.5, 271
 CLU_MXB_CLU2_P3.9, 271
 CLU_MXB_CLU2_PWM1, 271
 CLU_MXB_CLU2_PWM3, 271
 CLU_MXB_CLU2_SPICLK, 272
 CLU_MXB_CLU3_CLU0OUT, 272
 CLU_MXB_CLU3_CLU1OUT, 272
 CLU_MXB_CLU3_CLU2OUT, 272
 CLU_MXB_CLU3_CLU3OUT, 272
 CLU_MXB_CLU3_I2CSCL, 272
 CLU_MXB_CLU3_P1.10, 273
 CLU_MXB_CLU3_P1.11, 273
 CLU_MXB_CLU3_P1.13, 273
 CLU_MXB_CLU3_P1.14, 273
 CLU_MXB_CLU3_P2.14, 273
 CLU_MXB_CLU3_P2.15, 273
 CLU_MXB_CLU3_P2.4, 274
 CLU_MXB_CLU3_P2.5, 274
 CLU_MXB_CLU3_PWM0, 274
 CLU_MXB_CLU3_PWM2, 274
 CLU_MXB_CLU3_SPICSN, 274
 lks32mc07x_cmp.c
 CMP_ClearIRQFlag, 632
 CMP_GetFilterValue, 633
 CMP_GetIRQFlag, 634
 CMP_GetValue, 635
 CMP_Init, 636
 CMP_StructInit, 637
 lks32mc07x_cmp.h
 CMP, 280
 CMP0_SELN_HALL0_MID, 280
 CMP0_SELP_IP0, 280
 CMP0_SELP_IP1, 281
 CMP0_SELP_IP2, 281
 CMP0_SELP_IP3, 281
 CMP0_SELP_IP4, 281

- CMP0_SEL_P_OPA0_IP, [281](#)
- CMP0_SEL_P_OPA0_OUT, [281](#)
- CMP0_SEL_P_OPA1_OUT, [282](#)
- CMP1_SEL_N_HALL1_MID, [282](#)
- CMP1_SEL_P_DAC0, [282](#)
- CMP1_SEL_P_IP0, [282](#)
- CMP1_SEL_P_IP1, [282](#)
- CMP1_SEL_P_IP2, [282](#)
- CMP1_SEL_P_IP3, [283](#)
- CMP1_SEL_P_OPA2_OUT, [283](#)
- CMP1_SEL_P_OPA3_IP, [283](#)
- CMP1_SEL_P_OPA3_OUT, [283](#)
- CMP2_SEL_N_DAC1, [283](#)
- CMP2_SEL_P_IP0, [283](#)
- CMP2_SEL_P_IP1, [284](#)
- CMP2_SEL_P_OPA2_OUT, [284](#)
- CMP2_SEL_P_OPA3_OUT, [284](#)
- CMP_CHN_0, [284](#)
- CMP_CHN_1, [284](#)
- CMP_CHN_2, [284](#)
- CMP_ClearIRQFlag, [286](#)
- CMP_Cmd, [287](#)
- CMP_GetFilterValue, [287](#)
- CMP_GetIRQFlag, [288](#)
- CMP_GetValue, [289](#)
- CMP_HIGH_LEVEL, [285](#)
- CMP_HYS_0mV, [285](#)
- CMP_HYS_20mV, [285](#)
- CMP_Init, [290](#)
- CMP_LOW_LEVEL, [285](#)
- CMP_SEL_N_DAC0, [285](#)
- CMP_SEL_N_IN, [285](#)
- CMP_SEL_N_REF, [286](#)
- CMP_StructInit, [291](#)
- IRQ_EDGE_TRIG_MODE, [286](#)
- IRQ_LEVEL_TRIG_MODE, [286](#)
- W_PWM_HIGH_LEVEL, [286](#)
- W_PWM_LOW_LEVEL, [286](#)
- lks32mc07x_crc.h
 - CRC_MODEL_CRC_16_CCITT, [296](#)
 - CRC_MODEL_CRC_16_CCITT_FALSE, [296](#)
 - CRC_MODEL_CRC_16_DNP, [296](#)
 - CRC_MODEL_CRC_16_IBMx1, [296](#)
 - CRC_MODEL_CRC_16_MAXIM, [296](#)
 - CRC_MODEL_CRC_16_MODBU, [296](#)
 - CRC_MODEL_CRC_16_USBx1, [296](#)
 - CRC_MODEL_CRC_16_X25, [297](#)
 - CRC_MODEL_CRC_16_XMODEM, [297](#)
 - CRC_MODEL_CRC_32, [297](#)
 - CRC_MODEL_CRC_32_MPEG_2, [297](#)
 - CRC_MODEL_CRC_8, [297](#)
 - CRC_MODEL_CRC_8_ITU, [297](#)
 - CRC_MODEL_CRC_8_MAXIMx, [298](#)
 - CRC_MODEL_CRC_8_ROHC, [298](#)
 - CRC_WIDTH_16, [298](#)
 - CRC_WIDTH_32, [298](#)
 - CRC_WIDTH_7, [298](#)
 - CRC_WIDTH_8, [298](#)
- lks32mc07x_dac.c
 - DAC_Cmd, [644](#)
 - DAC_Init, [645](#)
 - DAC_Output, [646](#)
 - DAC_OutputValue, [646](#)
 - DAC_OutputVoltage, [647](#)
 - DAC_StructInit, [648](#)
- lks32mc07x_dac.h
 - DAC0_UTIMER0_TRG_EN, [301](#)
 - DAC0_UTIMER1_TRG_EN, [301](#)
 - DAC0_UTIMER2_TRG_EN, [302](#)
 - DAC0_UTIMER3_TRG_EN, [302](#)
 - DAC1_UTIMER0_TRG_EN, [302](#)
 - DAC1_UTIMER1_TRG_EN, [302](#)
 - DAC1_UTIMER2_TRG_EN, [302](#)
 - DAC1_UTIMER3_TRG_EN, [302](#)
 - DAC_Channel_0, [303](#)
 - DAC_Channel_1, [303](#)
 - DAC_Cmd, [303](#)
 - DAC_Init, [304](#)
 - DAC_Output, [306](#)
 - DAC_OutputValue, [307](#)
 - DAC_OutputVoltage, [308](#)
 - DAC_RANGE_1V2, [303](#)
 - DAC_RANGE_4V85, [303](#)
 - DAC_StructInit, [309](#)
- lks32mc07x_dma.c
 - DMA_CHx_EN, [652](#)
 - DMA_ClearIRQFlag, [653](#)
 - DMA_Disable, [654](#)
 - DMA_Enable, [655](#)
 - DMA_GetFlag, [656](#)
 - DMA_GetIRQFlag, [656](#)
 - DMA_Init, [658](#)
 - DMA_SetAddr, [659](#)
 - DMA_SetLength, [660](#)
 - DMA_SetRequest, [661](#)
 - DMA_StructInit, [663](#)
 - DMA_Trigger, [664](#)
- lks32mc07x_dma.h
 - DMA_BYTE_TRANS, [313](#)
 - DMA_CHN0, [313](#)
 - DMA_CHN1, [313](#)
 - DMA_CHN2, [313](#)
 - DMA_CHN3, [313](#)
 - DMA_CHx_EN, [318](#)
 - DMA_ClearIRQFlag, [319](#)
 - DMA_Disable, [320](#)
 - DMA_Enable, [321](#)
 - DMA_GetFlag, [322](#)
 - DMA_GetIRQFlag, [323](#)
 - DMA_HALFWORD_TRANS, [313](#)
 - DMA_IE_CH0, [313](#)
 - DMA_IE_CH1, [314](#)
 - DMA_IE_CH2, [314](#)
 - DMA_IE_CH3, [314](#)
 - DMA_IF_CH0, [314](#)
 - DMA_IF_CH1, [314](#)

- DMA_IF_CH2, [314](#)
- DMA_IF_CH3, [314](#)
- DMA_Init, [324](#)
- DMA_REQ_ADC0, [315](#)
- DMA_REQ_ADC1, [315](#)
- DMA_REQ_CMP, [315](#)
- DMA_REQ_GPIO, [315](#)
- DMA_REQ_HALL, [315](#)
- DMA_REQ_I2C0_RX, [315](#)
- DMA_REQ_I2C0_TX, [315](#)
- DMA_REQ_MCPWM0, [315](#)
- DMA_REQ_MCPWM1, [316](#)
- DMA_REQ_SIF, [316](#)
- DMA_REQ_SPI_RX, [316](#)
- DMA_REQ_SPI_TX, [316](#)
- DMA_REQ_SW, [316](#)
- DMA_REQ_TIMER0, [316](#)
- DMA_REQ_TIMER1, [316](#)
- DMA_REQ_TIMER2, [317](#)
- DMA_REQ_TIMER3, [317](#)
- DMA_REQ_TIMER4, [317](#)
- DMA_REQ_UART0_RX, [317](#)
- DMA_REQ_UART0_TX, [317](#)
- DMA_REQ_UART1_RX, [317](#)
- DMA_REQ_UART1_TX, [317](#)
- DMA_SetAddr, [326](#)
- DMA_SetLength, [327](#)
- DMA_SetRequest, [328](#)
- DMA_StructInit, [329](#)
- DMA_TCIE, [317](#)
- DMA_TEIE, [318](#)
- DMA_TOP, [318](#)
- DMA_Trigger, [330](#)
- DMA_WORD_TRANS, [318](#)
- MEMORY2PERI, [318](#)
- PERI2MEMORY, [318](#)
- lks32mc07x_dsp.c
 - DSP_arctan, [667](#)
 - DSP_ArctanMode, [668](#)
 - DSP_cos, [668](#)
 - DSP_div, [669](#)
 - DSP_Init, [670](#)
 - DSP_mod, [671](#)
 - DSP_rms, [672](#)
 - DSP_sin, [673](#)
 - DSP_SinCosMode, [674](#)
 - DSP_sqrt, [674](#)
- lks32mc07x_dsp.h
 - DSP0_arctan, [334](#)
 - DSP0_cos, [335](#)
 - DSP0_div, [335](#)
 - DSP0_mod, [335](#)
 - DSP0_rms, [335](#)
 - DSP0_sin, [335](#)
 - DSP0_sqrt, [335](#)
 - DSP_arctan, [338](#)
 - DSP_ARCTAN_MODE, [335](#)
 - DSP_ArctanMode, [339](#)
 - DSP_CALC_ARCTAN, [335](#)
 - DSP_CALC_COS, [336](#)
 - DSP_CALC_DIV, [336](#)
 - DSP_CALC_MOD, [336](#)
 - DSP_CALC_RMS, [337](#)
 - DSP_CALC_SIN, [337](#)
 - DSP_CALC_SQRT, [337](#)
 - DSP_cos, [339](#)
 - DSP_div, [340](#)
 - DSP_Init, [341](#)
 - DSP_mod, [342](#)
 - DSP_REG, [338](#)
 - DSP_rms, [343](#)
 - DSP_sin, [344](#)
 - DSP_SIN_COS_MODE, [338](#)
 - DSP_SinCosMode, [345](#)
 - DSP_sqrt, [345](#)
- lks32mc07x_exti.c
 - EXTI_ClearIRQFlag, [678](#)
 - EXTI_GetIRQFlag, [679](#)
 - EXTI_ReadIFFlag, [680](#)
 - EXTI_ResetIF, [680](#)
 - EXTI_Trigger_Config, [680](#)
- lks32mc07x_exti.h
 - EXTI, [350](#)
 - EXTI_ClearIRQFlag, [350](#)
 - EXTI_GetIRQFlag, [351](#)
 - EXTI_ReadIFFlag, [352](#)
 - EXTI_ResetIF, [352](#)
 - EXTI_Trigger_Config, [352](#)
 - EXTI_Trigger_Edge, [350](#)
 - EXTI_Trigger_Negedge, [350](#)
 - EXTI_Trigger_None, [350](#)
 - EXTI_Trigger_Posedge, [350](#)
 - EXTIMode_TypeDef, [350](#)
- lks32mc07x_flash.c
 - erase_flag, [693](#)
 - EraseSector, [687](#)
 - progmem_flag, [693](#)
 - ProgramPage, [688](#)
 - Read_Flash, [690](#)
 - Read_More_Flash, [692](#)
- lks32mc07x_flash.h
 - erase_flag, [360](#)
 - EraseSector, [356](#)
 - Flash_MAIN, [356](#)
 - Flash_NVR, [356](#)
 - progmem_flag, [360](#)
 - ProgramPage, [357](#)
 - Read_More_Flash, [358](#)
- lks32mc07x_gpio.c
 - GPIO_Init, [697](#)
 - GPIO_PinAFConfig, [698](#)
 - GPIO_ReadInputData, [699](#)
 - GPIO_ReadInputDataBit, [700](#)
 - GPIO_ReadOutputData, [701](#)
 - GPIO_ReadOutputDataBit, [702](#)
 - GPIO_ResetBits, [703](#)

- GPIO_SetBits, 704
- GPIO_StructInit, 705
- GPIO_Write, 706
- GPIO_WriteBit, 707
- lks32mc07x_gpio.h
 - AF0_GPIO, 364
 - AF10_CAN, 364
 - AF11_SIF, 364
 - AF12_CL, 364
 - AF1_SYSAF, 364
 - AF2_HALL, 364
 - AF3_MCPWM, 364
 - AF4_UART, 365
 - AF5_SPI, 365
 - AF6_I2C, 365
 - AF7_TIMER01, 365
 - AF8_TIMER23, 365
 - AF9_DEBUG, 365
 - Bit_RESET, 372
 - Bit_SET, 372
 - BitAction, 372
 - GPIO0, 366
 - GPIO1, 366
 - GPIO2, 366
 - GPIO3, 366
 - GPIO_Init, 373
 - GPIO_Mode_ANA, 373
 - GPIO_Mode_IN, 373
 - GPIO_Mode_IO, 373
 - GPIO_Mode_OUT, 373
 - GPIO_Pin_0, 366
 - GPIO_Pin_1, 366
 - GPIO_Pin_10, 367
 - GPIO_Pin_11, 367
 - GPIO_Pin_12, 367
 - GPIO_Pin_13, 367
 - GPIO_Pin_14, 367
 - GPIO_Pin_15, 367
 - GPIO_Pin_2, 368
 - GPIO_Pin_3, 368
 - GPIO_Pin_4, 368
 - GPIO_Pin_5, 368
 - GPIO_Pin_6, 368
 - GPIO_Pin_7, 368
 - GPIO_Pin_8, 369
 - GPIO_Pin_9, 369
 - GPIO_Pin_ALL, 369
 - GPIO_Pin_NONE, 369
 - GPIO_PinAFConfig, 374
 - GPIO_PinSource_0, 369
 - GPIO_PinSource_1, 369
 - GPIO_PinSource_10, 370
 - GPIO_PinSource_11, 370
 - GPIO_PinSource_12, 370
 - GPIO_PinSource_13, 370
 - GPIO_PinSource_14, 370
 - GPIO_PinSource_15, 370
 - GPIO_PinSource_2, 371
 - GPIO_PinSource_3, 371
 - GPIO_PinSource_4, 371
 - GPIO_PinSource_5, 371
 - GPIO_PinSource_6, 371
 - GPIO_PinSource_7, 371
 - GPIO_PinSource_8, 372
 - GPIO_PinSource_9, 372
 - GPIO_PuPd_NOPULL, 373
 - GPIO_PuPd_UP, 373
 - GPIO_ReadInputData, 376
 - GPIO_ReadInputDataBit, 377
 - GPIO_ReadOutputData, 378
 - GPIO_ReadOutputDataBit, 379
 - GPIO_ResetBits, 380
 - GPIO_SetBits, 381
 - GPIO_StructInit, 382
 - GPIO_Write, 383
 - GPIO_WriteBit, 383
 - GPIO_Mode_TypeDef, 372
 - GPIO_PuPd_TypeDef, 373
- lks32mc07x_hall.c
 - HALL_Clear_IRQ, 712
 - HALL_Cmd, 713
 - HALL_GetCaptureValue, 714
 - HALL_GetCount, 715
 - HALL_GetFilterValue, 717
 - HALL_GetFlag, 718
 - HALL_GetIRQFlag, 718
 - HALL_Init, 720
 - HALL_IsCaptureEvent, 721
 - HALL_IsOverFlowEvent, 722
 - HALL_SoftTrigger, 723
 - HALL_StructInit, 724
- lks32mc07x_hall.h
 - HALL, 389
 - HALL_CAPTURE_EVENT, 389
 - HALL_Clear_IRQ, 390
 - HALL_CLK_DIV1, 389
 - HALL_CLK_DIV2, 389
 - HALL_CLK_DIV4, 389
 - HALL_CLK_DIV8, 389
 - HALL_Cmd, 391
 - HALL_GetCaptureValue, 392
 - HALL_GetCount, 393
 - HALL_GetFilterValue, 394
 - HALL_GetFlag, 395
 - HALL_GetIRQFlag, 395
 - HALL_Init, 396
 - HALL_IsCaptureEvent, 397
 - HALL_IsOverFlowEvent, 398
 - HALL_OVERFLOW_EVENT, 389
 - HALL_SoftTrigger, 399
 - HALL_StructInit, 400
- lks32mc07x_i2c.h
 - I2C_BUS_STATE_ADDR_DATA, 404
 - I2C_BUS_STATE_Done, 404
 - I2C_BUS_STATE_LOST_ARB, 404
 - I2C_BUS_STATE_RX_ACK, 404

- I2C_BUS_STATE_STOP_EVT, 404
- I2C_BUS_STATE_STT_ERR, 405
- I2C_IF_ADDR_CMP, 405
- I2C_IF_BUS_ERR, 405
- I2C_IF_NACK, 405
- I2C_IF_STOP, 405
- I2C_IF_TC, 405
- lks32mc07x.lib.h
 - REG_IF_WRITE_BIT, 408
 - REG_READ, 408
 - REG_READ_BIT, 408
 - REG_READ_BIT_POS_MASK, 408
 - REG_RESET, 409
 - REG_SET, 409
 - REG_WRITE, 409
 - REG_WRITE_BIT, 409
 - REG_WRITE_BIT_POS_MASK, 409
- lks32mc07x.mcpwm.c
 - ClockDivflg, 743
 - MCPWM_CH012_IESet, 731
 - MCPWM_CH012_Init, 731
 - MCPWM_CH345_IESet, 731
 - MCPWM_CH345_Init, 731
 - MCPWM_ClearIRQFlag, 732
 - MCPWM_Clk_Enable, 735
 - MCPWM_CNTx_Enable, 735
 - MCPWM_DIV_Set, 735
 - MCPWM_GetIRQFlag, 736
 - MCPWM_Init, 739
 - MCPWM_ModuleClockCmd, 741
 - MCPWM_StructInit, 741
 - PWMOutputs, 742
- lks32mc07x.mcpwm.h
 - MCPWM0, 416
 - MCPWM0_ALL_AUPDAT, 416
 - MCPWM0_CENTRAL_PWM_MODE, 416
 - MCPWM0_CNT0_AUPDAT, 416
 - MCPWM0_CNT1_AUPDAT, 416
 - MCPWM0_EDGE_PWM_MODE, 416
 - MCPWM0_FAIL0_IRQ_IF, 416
 - MCPWM0_FAIL1_IRQ_IF, 417
 - MCPWM0_FAIL2_IRQ_IF, 417
 - MCPWM0_FAIL3_IRQ_IF, 417
 - MCPWM0_FAIL_SEL_CMP, 417
 - MCPWM0_FAIL_SEL_IO, 417
 - MCPWM0_HIGH_LEVEL, 417
 - MCPWM0_HIGH_LEVEL_ACTIVE, 418
 - MCPWM0_LOW_LEVEL, 418
 - MCPWM0_LOW_LEVEL_ACTIVE, 418
 - MCPWM0_TH00_AUPDATE, 418
 - MCPWM0_TH01_AUPDATE, 418
 - MCPWM0_TH0_AUPDATE, 418
 - MCPWM0_TH10_AUPDATE, 419
 - MCPWM0_TH11_AUPDATE, 419
 - MCPWM0_TH1_AUPDATE, 419
 - MCPWM0_TH20_AUPDATE, 419
 - MCPWM0_TH21_AUPDATE, 419
 - MCPWM0_TH30_AUPDATE, 419
 - MCPWM0_TH31_AUPDATE, 420
 - MCPWM0_TH40_AUPDATE, 420
 - MCPWM0_TH41_AUPDATE, 420
 - MCPWM0_TH50_AUPDATE, 420
 - MCPWM0_TH51_AUPDATE, 420
 - MCPWM0_TMR0_AUPDATE, 420
 - MCPWM0_TMR1_AUPDATE, 421
 - MCPWM0_TMR2_AUPDATE, 421
 - MCPWM0_TMR3_AUPDATE, 421
 - MCPWM0_ZCS0_IRQ_IF, 421
 - MCPWM0_ZCS1_IRQ_IF, 421
 - MCPWM0CNT0, 431
 - MCPWM0CNT0_T0_IRQ_IF, 421
 - MCPWM0CNT0_T1_IRQ_IF, 422
 - MCPWM0CNT0_TH00_IRQ_IF, 422
 - MCPWM0CNT0_TH01_IRQ_IF, 422
 - MCPWM0CNT0_TH10_IRQ_IF, 422
 - MCPWM0CNT0_TH11_IRQ_IF, 422
 - MCPWM0CNT0_TH20_IRQ_IF, 422
 - MCPWM0CNT0_TH21_IRQ_IF, 423
 - MCPWM0CNT0_TMR0_IRQ_IF, 423
 - MCPWM0CNT0_TMR1_IRQ_IF, 423
 - MCPWM0CNT0_TMR2_IRQ_IF, 423
 - MCPWM0CNT0_TMR3_IRQ_IF, 423
 - MCPWM0CNT0_UP_IRQ_IF, 423
 - MCPWM0CNT1, 431
 - MCPWM0CNT1_T0_IRQ_IF, 424
 - MCPWM0CNT1_T1_IRQ_IF, 424
 - MCPWM0CNT1_TH30_IRQ_IF, 424
 - MCPWM0CNT1_TH31_IRQ_IF, 424
 - MCPWM0CNT1_TH40_IRQ_IF, 424
 - MCPWM0CNT1_TH41_IRQ_IF, 424
 - MCPWM0CNT1_TH50_IRQ_IF, 425
 - MCPWM0CNT1_TH51_IRQ_IF, 425
 - MCPWM0CNT1_TMR2_IRQ_IF, 425
 - MCPWM0CNT1_TMR3_IRQ_IF, 425
 - MCPWM0CNT1_UP_IRQ_IF, 425
 - MCPWM0CNTx, 431
 - MCPWM_CH012_IESet, 431
 - MCPWM_CH012_Init, 431
 - MCPWM_CH345_IESet, 431
 - MCPWM_CH345_Init, 432
 - MCPWM_ClearIRQFlag, 432
 - MCPWM_Clk_Enable, 435
 - MCPWM_CNTx_Enable, 435
 - MCPWM_DIV_Set, 435
 - MCPWM_EIF_FAIL0, 425
 - MCPWM_EIF_FAIL1, 426
 - MCPWM_EIF_FAIL2, 426
 - MCPWM_EIF_FAIL3, 426
 - MCPWM_EIF_ZCS0, 426
 - MCPWM_EIF_ZCS1, 426
 - MCPWM_GetIRQFlag, 436
 - MCPWM_IF0_T0_IF, 426
 - MCPWM_IF0_T1_IF, 427
 - MCPWM_IF0_TH00_IF, 427
 - MCPWM_IF0_TH01_IF, 427
 - MCPWM_IF0_TH10_IF, 427

- MCPWM_IF0_TH11_IF, [427](#)
- MCPWM_IF0_TH20_IF, [427](#)
- MCPWM_IF0_TH21_IF, [428](#)
- MCPWM_IF0_TMR0_IF, [428](#)
- MCPWM_IF0_TMR1_IF, [428](#)
- MCPWM_IF0_TMR2_IF, [428](#)
- MCPWM_IF0_TMR3_IF, [428](#)
- MCPWM_IF0_UP_IF, [428](#)
- MCPWM_IF1_T0_IF, [429](#)
- MCPWM_IF1_T1_IF, [429](#)
- MCPWM_IF1_TH30_IF, [429](#)
- MCPWM_IF1_TH31_IF, [429](#)
- MCPWM_IF1_TH40_IF, [429](#)
- MCPWM_IF1_TH41_IF, [429](#)
- MCPWM_IF1_TH50_IF, [430](#)
- MCPWM_IF1_TH51_IF, [430](#)
- MCPWM_IF1_TMR0_IF, [430](#)
- MCPWM_IF1_TMR1_IF, [430](#)
- MCPWM_IF1_TMR2_IF, [430](#)
- MCPWM_IF1_TMR3_IF, [430](#)
- MCPWM_IF1_UP_IF, [431](#)
- MCPWM_Init, [439](#)
- MCPWM_ModuleClockCmd, [441](#)
- MCPWM_StructInit, [441](#)
- PWMOutputs, [442](#)
- lks32mc07x_nvr.h
 - Read_NVR, [449](#)
 - Read_Trim, [449](#)
- lks32mc07x_opa.c
 - maxmagflg, [753](#)
 - OPA_Init, [750](#)
 - OPA_OUT, [751](#)
 - OPA_StructInit, [752](#)
- lks32mc07x_opa.h
 - enumOPA, [453](#)
 - OPA0, [453](#)
 - OPA1, [453](#)
 - OPA2, [453](#)
 - OPA3, [453](#)
 - OPA_Init, [454](#)
 - OPA_OUT, [454](#)
 - OPA_StructInit, [455](#)
 - PGA_GAIN_16, [452](#)
 - PGA_GAIN_32, [452](#)
 - PGA_GAIN_4, [452](#)
 - PGA_GAIN_8, [452](#)
 - PGA_IT_1, [452](#)
 - PGA_IT_1P2, [452](#)
 - PGA_IT_1P5, [453](#)
 - PGA_IT_2, [453](#)
 - PGA_IT_DISABLE, [453](#)
- lks32mc07x_qep.c
 - QEP_ClearIRQFlag, [756](#)
 - QEP_GetFlag, [757](#)
 - QEP_GetIRQFlag, [758](#)
 - TIM_QEP_GetCount, [759](#)
 - TIM_QEPCmd, [760](#)
 - TIM_QEPInit, [761](#)
 - TIM_QEPStructInit, [762](#)
- lks32mc07x_qep.h
 - QEP0, [459](#)
 - QEP1, [459](#)
 - QEP_ClearIRQFlag, [462](#)
 - QEP_CLK_DIV1, [460](#)
 - QEP_CLK_DIV128, [460](#)
 - QEP_CLK_DIV16, [460](#)
 - QEP_CLK_DIV2, [460](#)
 - QEP_CLK_DIV32, [460](#)
 - QEP_CLK_DIV4, [460](#)
 - QEP_CLK_DIV64, [461](#)
 - QEP_CLK_DIV8, [461](#)
 - QEP_GetFlag, [463](#)
 - QEP_GetIRQFlag, [464](#)
 - QEP_IF_OVERFLOW, [461](#)
 - QEP_IF_UNDERFLOW, [461](#)
 - QEP_IRQEna_None, [461](#)
 - QEP_IRQEna_OverFlow, [461](#)
 - QEP_IRQEna_UnderFlow, [462](#)
 - QEP_Mode_CCWCW_UpDown, [462](#)
 - QEP_Mode_CCWSIGN_UpDown, [462](#)
 - QEP_Mode_T1, [462](#)
 - QEP_Mode_T1_T2, [462](#)
 - TIM_QEP_GetCount, [465](#)
 - TIM_QEPCmd, [466](#)
 - TIM_QEPInit, [467](#)
 - TIM_QEPStructInit, [468](#)
- lks32mc07x_sif.c
 - SIF_Clear_Irq, [766](#)
 - SIF_DMA_IsEnable, [767](#)
 - SIF_Init, [768](#)
 - SIF_Irq_IsEnable, [769](#)
 - SIF_Senddata, [770](#)
 - SIF_StructInit, [771](#)
 - SIF_TH_set, [771](#)
 - SIF_Timebase_set, [772](#)
- lks32mc07x_sif.h
 - SIF0, [472](#)
 - SIF_Clear_Irq, [473](#)
 - SIF_DMA_Enable, [472](#)
 - SIF_DMA_IsEnable, [474](#)
 - SIF_IE_Enable, [472](#)
 - SIF_Init, [475](#)
 - SIF_Irq_IsEnable, [476](#)
 - SIF_Senddata, [477](#)
 - SIF_StructInit, [478](#)
 - SIF_TH_set, [478](#)
 - SIF_Timebase_set, [479](#)
 - SPI_IF_CLEAR, [472](#)
- lks32mc07x_spi.c
 - SPI_ClearIRQFlag, [774](#)
 - SPI_GetIRQFlag, [775](#)
 - SPI_Init, [777](#)
 - SPI_ReadData, [778](#)
 - SPI_SendData, [779](#)
 - SPI_StructInit, [780](#)
- lks32mc07x_spi.h

- SPI0, [482](#)
- SPI_ClearIRQFlag, [485](#)
- SPI_DMA_DISABLE, [482](#)
- SPI_DMA_ENABLE, [482](#)
- SPI_Duplex, [484](#)
- SPI_FIRSTSEND_LSB, [483](#)
- SPI_FIRSTSEND_MSB, [483](#)
- SPI_Full, [484](#)
- SPI_GetIRQFlag, [486](#)
- SPI_Half_Rx, [484](#)
- SPI_Half_Tx, [484](#)
- SPI_IF_DataOver, [483](#)
- SPI_IF_SSErr, [483](#)
- SPI_IF_TranDone, [483](#)
- SPI_Init, [487](#)
- SPI_IRQEna_DataOver, [483](#)
- SPI_IRQEna_Enable, [484](#)
- SPI_IRQEna_SSErr, [484](#)
- SPI_IRQEna_TranDone, [484](#)
- SPI_Master, [485](#)
- SPI_Mode, [484](#)
- SPI_ReadData, [488](#)
- SPI_SendData, [489](#)
- SPI_Slave, [485](#)
- SPI_StructInit, [490](#)
- lks32mc07x_sys.c
 - IWDG_DISABLE, [784](#)
 - IWDG_ENABLE, [785](#)
 - IWDG_Feed, [786](#)
 - IWDG_Init, [787](#)
 - IWDG_StructInit, [788](#)
 - SoftDelay, [789](#)
 - SYS_AnalogModuleClockCmd, [790](#)
 - SYS_ClearRst, [792](#)
 - sys_clk, [806](#)
 - SYS_I2CClkDiv, [793](#)
 - SYS_Init, [794](#)
 - SYS_MclkChoice, [795](#)
 - SYS_ModuleClockCmd, [796](#)
 - SYS_ReadMcuClk, [798](#)
 - SYS_SoftReset_all, [799](#)
 - SYS_SoftResetModule, [800](#)
 - SYS_SPIClkDiv, [802](#)
 - SYS_StructInit, [803](#)
 - SYS_UartClkDiv, [804](#)
 - SYS_VolSelModule, [805](#)
 - SYS_VolSelModuleEnableIRQ, [805](#)
 - SYS_VolSelModuleIRQ, [805](#)
- lks32mc07x_sys.h
 - AFE, [496](#)
 - CLK_SEL_HS, [496](#)
 - CLK_SEL_PLL, [496](#)
 - IWDG_DISABLE, [504](#)
 - IWDG_ENABLE, [505](#)
 - IWDG_Feed, [506](#)
 - IWDG_Init, [507](#)
 - IWDG_StructInit, [508](#)
 - SECOND2IWDGCNT, [496](#)
 - SYS, [496](#)
 - SYS_AnalogModule_ADC, [496](#)
 - SYS_AnalogModule_BGP, [497](#)
 - SYS_AnalogModule_CMP0, [497](#)
 - SYS_AnalogModule_CMP1, [497](#)
 - SYS_AnalogModule_CMP2, [497](#)
 - SYS_AnalogModule_DAC0, [497](#)
 - SYS_AnalogModule_DAC1, [497](#)
 - SYS_AnalogModule_OPA0, [497](#)
 - SYS_AnalogModule_OPA1, [497](#)
 - SYS_AnalogModule_OPA2, [498](#)
 - SYS_AnalogModule_OPA3, [498](#)
 - SYS_AnalogModule_PLL, [498](#)
 - SYS_AnalogModule_RCH, [498](#)
 - SYS_AnalogModule_TMP, [498](#)
 - SYS_AnalogModule_XTAL, [498](#)
 - SYS_AnalogModuleClockCmd, [509](#)
 - SYS_ClearRst, [510](#)
 - SYS_Clk_SPIDiv1, [498](#)
 - SYS_Clk_SPIDiv2, [498](#)
 - SYS_Clk_SPIDiv4, [499](#)
 - SYS_Clk_SPIDiv8, [499](#)
 - SYS_Clk_UARTDiv1, [499](#)
 - SYS_Clk_UARTDiv2, [499](#)
 - SYS_Clk_UARTDiv4, [499](#)
 - SYS_Clk_UARTDiv8, [499](#)
 - SYS_I2CClkDiv, [512](#)
 - SYS_Init, [513](#)
 - SYS_MCLK_12M_RC, [499](#)
 - SYS_MCLK_12M_XTAL, [499](#)
 - SYS_MCLK_24M_RC, [500](#)
 - SYS_MCLK_24M_XTAL, [500](#)
 - SYS_MCLK_32K_RC, [500](#)
 - SYS_MCLK_48M_RC, [500](#)
 - SYS_MCLK_48M_XTAL, [500](#)
 - SYS_MCLK_96M_RC, [500](#)
 - SYS_MCLK_96M_XTAL, [500](#)
 - SYS_MclkChoice, [514](#)
 - SYS_MCU_POWER_3v3, [500](#)
 - SYS_MCU_POWER_5v0, [501](#)
 - SYS_Module_CAN0, [501](#)
 - SYS_Module_CL0, [501](#)
 - SYS_Module_CMP, [501](#)
 - SYS_Module_CRC0, [501](#)
 - SYS_Module_DSP0, [501](#)
 - SYS_Module_GPIO, [501](#)
 - SYS_Module_HALL0, [501](#)
 - SYS_Module_I2C0, [502](#)
 - SYS_Module_MCPWM0, [502](#)
 - SYS_Module_QEP0, [502](#)
 - SYS_Module_QEP1, [502](#)
 - SYS_Module_SIF0, [502](#)
 - SYS_Module_SPI0, [502](#)
 - SYS_Module_TIMER0, [502](#)
 - SYS_Module_TIMER1, [502](#)
 - SYS_Module_TIMER2, [503](#)
 - SYS_Module_TIMER3, [503](#)
 - SYS_Module_UART0, [503](#)

- SYS_Module_UART1, 503
- SYS_ModuleClockCmd, 516
- SYS_PLLSRSEL_CRYSTAL, 503
- SYS_PLLSRSEL_RCH, 503
- SYS_POWER_2V5, 503
- SYS_POWER_3V25, 503
- SYS_POWER_3V5, 504
- SYS_POWER_3V75, 504
- SYS_POWER_4V0, 504
- SYS_ReadMcuClk, 518
- SYS_SoftReset_all, 519
- SYS_SoftResetModule, 520
- SYS_SPIClkDiv, 522
- SYS_StructInit, 523
- SYS_UartClkDiv, 524
- SYS_VolSelModule, 525
- SYS_VolSelModuleEnableIRQ, 525
- SYS_VolSelModuleIRQ, 525
- lks32mc07x_tim.c
 - TIM_ClearIRQFlag, 812
 - TIM_CMP0, 813
 - TIM_CMP1, 814
 - TIM_Disable, 815
 - TIM_Enable, 816
 - TIM_GetFlag, 817
 - TIM_GetIRQFlag, 818
 - TIM_Timer_GetCMPT0, 819
 - TIM_Timer_GetCMPT1, 820
 - TIM_Timer_GetCount, 821
 - TIM_TimerCmd, 822
 - TIM_TimerInit, 823
 - TIM_TimerStrutInit, 825
 - UTIMER_Disable, 826
 - UTIMER_Enable, 826
- lks32mc07x_tim.h
 - CAP_SRC_CHXOR, 533
 - CAP_SRC_CLU0, 533
 - CAP_SRC_CLU1, 533
 - CAP_SRC_CLU2, 533
 - CAP_SRC_CLU3, 533
 - CAP_SRC_CMP0, 533
 - CAP_SRC_CMP1, 533
 - CAP_SRC_CMP2, 534
 - CAP_SRC_TCH0, 534
 - CAP_SRC_TCH1, 534
 - TIM_ClearIRQFlag, 541
 - TIM_CMP0, 542
 - TIM_CMP1, 543
 - TIM_GetFlag, 544
 - TIM_GetIRQFlag, 545
 - TIM_Timer_GetCMPT0, 546
 - TIM_Timer_GetCMPT1, 547
 - TIM_Timer_GetCount, 548
 - TIM_TimerCmd, 549
 - TIM_TimerInit, 550
 - TIM_TimerStrutInit, 552
 - TIMER0_EVT_CH0, 534
 - TIMER0_EVT_CH1, 534
 - Timer0_IRQEna_FAIL, 541
 - Timer0_IRQEna_FAIL_RE, 541
 - TIMER1_EVT_CH0, 534
 - TIMER1_EVT_CH1, 535
 - TIMER2_EVT_CH0, 535
 - TIMER2_EVT_CH1, 535
 - TIMER3_EVT_CH0, 535
 - TIMER3_EVT_CH1, 535
 - TIMER_CH0_IF, 535
 - TIMER_CH1_IF, 536
 - TIMER_CLK_DIV1, 536
 - TIMER_CLK_DIV128, 536
 - TIMER_CLK_DIV16, 536
 - TIMER_CLK_DIV2, 536
 - TIMER_CLK_DIV32, 536
 - TIMER_CLK_DIV4, 537
 - TIMER_CLK_DIV64, 537
 - TIMER_CLK_DIV8, 537
 - TIMER_EVT_CLU0, 537
 - TIMER_EVT_CLU1, 537
 - TIMER_EVT_CLU2, 537
 - TIMER_EVT_CLU3, 538
 - TIMER_EVT_TADC0, 538
 - TIMER_EVT_TADC1, 538
 - TIMER_EVT_TADC2, 538
 - TIMER_EVT_TADC3, 538
 - TIMER_FAIL_IF, 538
 - TIMER_IF_CH0, 539
 - TIMER_IF_CH1, 539
 - TIMER_IF_FAIL, 539
 - TIMER_IF_ZERO, 539
 - Timer_IRQEna_All, 541
 - Timer_IRQEna_CH0, 541
 - Timer_IRQEna_CH0_RE, 541
 - Timer_IRQEna_CH1, 541
 - Timer_IRQEna_CH1_RE, 541
 - Timer_IRQEna_None, 541
 - Timer_IRQEna_ZC, 541
 - Timer_IRQEna_ZC_RE, 541
 - Timer_IRQEnaDef, 541
 - TIMER_OPMode_CAP, 539
 - TIMER_OPMode_CMP, 539
 - TIMER_POL_HIGH, 540
 - TIMER_POL_LOW, 540
 - TIMER_ZC_IF, 540
 - UTIMER0, 540
 - UTIMER1, 540
 - UTIMER2, 540
 - UTIMER3, 541
 - UTIMER_Disable, 553
 - UTIMER_Enable, 553
- lks32mc07x_tmp.c
 - GetCurrentTemperature, 830
 - m_TempertureCof, 832
 - TempSensor_Init, 831
- lks32mc07x_tmp.h
 - GetCurrentTemperature, 557
 - TempSensor_Init, 558

- lks32mc07x_uart.c
 - MCU_MCLK, 834
 - UART_ClearIRQFlag, 835
 - UART_GetIRQFlag, 836
 - UART_Init, 837
 - UART_ReadBaud, 838
 - UART_ReadData, 839
 - UART_SendAddr_485, 840
 - UART_SendData, 841
 - UART_SendData_485, 842
 - UART_SetBaud, 843
 - UART_StructInit, 843
- lks32mc07x_uart.h
 - RX_DMA_RE, 562
 - TX_BUF_DMA_RE, 562
 - TX_DMA_RE, 563
 - UART0, 563
 - UART1, 563
 - UART_ClearIRQFlag, 568
 - UART_FIRSTSEND_LSB, 563
 - UART_FIRSTSEND_MSB, 563
 - UART_GetIRQFlag, 569
 - UART_IF_CheckError, 563
 - UART_IF_IDLE, 564
 - UART_IF_LBD, 564
 - UART_IF_RcvOver, 564
 - UART_IF_RX_OV, 564
 - UART_IF_SendBufEmpty, 564
 - UART_IF_SendOver, 564
 - UART_IF_StopError, 565
 - UART_IF_TX_OV, 565
 - UART_Init, 570
 - UART_IRQEna_CheckError, 565
 - UART_IRQEna_IDLE, 565
 - UART_IRQEna_LBD, 565
 - UART_IRQEna_RcvOver, 565
 - UART_IRQEna_RX_OV, 566
 - UART_IRQEna_SendBuffEmpty, 566
 - UART_IRQEna_SendOver, 566
 - UART_IRQEna_StopError, 566
 - UART_IRQEna_TX_OV, 566
 - UART_Parity_EVEN, 567
 - UART_Parity_NO, 567
 - UART_Parity_ODD, 567
 - UART_ParityMode, 567
 - UART_ReadData, 571
 - UART_SendAddr_485, 572
 - UART_SendData, 573
 - UART_SendData_485, 574
 - UART_STOPBITS_1b, 566
 - UART_STOPBITS_2b, 567
 - UART_StructInit, 575
 - UART_WORDLENGTH_8b, 567
 - UART_WORDLENGTH_9b, 567
- lks32mc07x_wake.c
 - SetWakeIO, 848
 - Switch2HRC, 850
 - Switch2PLL, 851
 - SYS_EnterSleep, 852
 - SYS_EVTCLRSleep, 852
 - SYS_FallSleep, 853
 - SYS_SetWakeInterval, 854
 - WAKE_InitStruct, 857
 - wake_SoftDelay, 855
- lks32mc07x_wake.h
 - enumWakeTrigLevel, 583
 - IOWK_FLT_DI, 580
 - IOWK_FLT_EN, 580
 - SetWakeIO, 584
 - Switch2HRC, 585
 - Switch2PLL, 586
 - SYS_EnterSleep, 587
 - SYS_EVTCLRSleep, 587
 - SYS_FallSleep, 588
 - SYS_SetWakeInterval, 589
 - SYS_WakeInterval_025s, 580
 - SYS_WakeInterval_05s, 580
 - SYS_WakeInterval_16s, 581
 - SYS_WakeInterval_1s, 581
 - SYS_WakeInterval_2s, 581
 - SYS_WakeInterval_32s, 581
 - SYS_WakeInterval_4s, 581
 - SYS_WakeInterval_8s, 581
 - WAKEIO_CLUOUT0, 582
 - WAKEIO_CLUOUT1, 582
 - WAKEIO_CLUOUT2, 582
 - WAKEIO_CLUOUT3, 582
 - WAKEIO_P0_0, 582
 - WAKEIO_P0_11, 582
 - WAKEIO_P0_14, 583
 - WAKEIO_P0_2, 583
 - WAKEIO_P0_6, 583
 - WAKEIO_P2_15, 583
 - WAKEIO_P2_4, 583
 - WAKEIO_P2_7, 583
 - WAKETRIG_HIGH, 584
 - WAKETRIG_LOW, 584
- LTH
 - ADC_InitTypeDef, 27
 - ADC_TypeDef, 36
- m_TempertureCof
 - lks32mc07x_tmp.c, 832
- Match485Addr
 - UART_InitTypeDef, 183
- maxmagflg
 - lks32mc07x_opa.c, 753
- MBIST
 - SYS_TypeDef, 163
- MCLK_EN
 - MCPWM_InitTypeDef, 118
- MCPWM0
 - lks32mc07x_mcpwm.h, 416
- MCPWM0_ALL_AUPDAT
 - lks32mc07x_mcpwm.h, 416
- MCPWM0_CENTRAL_PWM_MODE
 - lks32mc07x_mcpwm.h, 416

MCPWM0_CNT0_AUPDAT
 lks32mc07x_mcpwm.h, 416
 MCPWM0_CNT1_AUPDAT
 lks32mc07x_mcpwm.h, 416
 MCPWM0_EDGE_PWM_MODE
 lks32mc07x_mcpwm.h, 416
 MCPWM0_FAIL0_IRQ_IF
 lks32mc07x_mcpwm.h, 416
 MCPWM0_FAIL1_IRQ_IF
 lks32mc07x_mcpwm.h, 417
 MCPWM0_FAIL2_IRQ_IF
 lks32mc07x_mcpwm.h, 417
 MCPWM0_FAIL3_IRQ_IF
 lks32mc07x_mcpwm.h, 417
 MCPWM0_FAIL_SEL_CMP
 lks32mc07x_mcpwm.h, 417
 MCPWM0_FAIL_SEL_IO
 lks32mc07x_mcpwm.h, 417
 MCPWM0_HIGH_LEVEL
 lks32mc07x_mcpwm.h, 417
 MCPWM0_HIGH_LEVEL_ACTIVE
 lks32mc07x_mcpwm.h, 418
 MCPWM0_LOW_LEVEL
 lks32mc07x_mcpwm.h, 418
 MCPWM0_LOW_LEVEL_ACTIVE
 lks32mc07x_mcpwm.h, 418
 MCPWM0_TH00_AUPDATE
 lks32mc07x_mcpwm.h, 418
 MCPWM0_TH01_AUPDATE
 lks32mc07x_mcpwm.h, 418
 MCPWM0_TH0_AUPDATE
 lks32mc07x_mcpwm.h, 418
 MCPWM0_TH10_AUPDATE
 lks32mc07x_mcpwm.h, 419
 MCPWM0_TH11_AUPDATE
 lks32mc07x_mcpwm.h, 419
 MCPWM0_TH1_AUPDATE
 lks32mc07x_mcpwm.h, 419
 MCPWM0_TH20_AUPDATE
 lks32mc07x_mcpwm.h, 419
 MCPWM0_TH21_AUPDATE
 lks32mc07x_mcpwm.h, 419
 MCPWM0_TH30_AUPDATE
 lks32mc07x_mcpwm.h, 419
 MCPWM0_TH31_AUPDATE
 lks32mc07x_mcpwm.h, 420
 MCPWM0_TH40_AUPDATE
 lks32mc07x_mcpwm.h, 420
 MCPWM0_TH41_AUPDATE
 lks32mc07x_mcpwm.h, 420
 MCPWM0_TH50_AUPDATE
 lks32mc07x_mcpwm.h, 420
 MCPWM0_TH51_AUPDATE
 lks32mc07x_mcpwm.h, 420
 MCPWM0_TMR0_AUPDATE
 lks32mc07x_mcpwm.h, 420
 MCPWM0_TMR1_AUPDATE
 lks32mc07x_mcpwm.h, 421
 MCPWM0_TMR2_AUPDATE
 lks32mc07x_mcpwm.h, 421
 MCPWM0_TMR3_AUPDATE
 lks32mc07x_mcpwm.h, 421
 MCPWM0_ZCS0_IRQ_IF
 lks32mc07x_mcpwm.h, 421
 MCPWM0_ZCS1_IRQ_IF
 lks32mc07x_mcpwm.h, 421
 MCPWM0CNT0
 lks32mc07x_mcpwm.h, 431
 MCPWM0CNT0_T0_IRQ_IF
 lks32mc07x_mcpwm.h, 421
 MCPWM0CNT0_T1_IRQ_IF
 lks32mc07x_mcpwm.h, 422
 MCPWM0CNT0_TH00_IRQ_IF
 lks32mc07x_mcpwm.h, 422
 MCPWM0CNT0_TH01_IRQ_IF
 lks32mc07x_mcpwm.h, 422
 MCPWM0CNT0_TH10_IRQ_IF
 lks32mc07x_mcpwm.h, 422
 MCPWM0CNT0_TH11_IRQ_IF
 lks32mc07x_mcpwm.h, 422
 MCPWM0CNT0_TH20_IRQ_IF
 lks32mc07x_mcpwm.h, 422
 MCPWM0CNT0_TH21_IRQ_IF
 lks32mc07x_mcpwm.h, 423
 MCPWM0CNT0_TMR0_IRQ_IF
 lks32mc07x_mcpwm.h, 423
 MCPWM0CNT0_TMR1_IRQ_IF
 lks32mc07x_mcpwm.h, 423
 MCPWM0CNT0_TMR2_IRQ_IF
 lks32mc07x_mcpwm.h, 423
 MCPWM0CNT0_TMR3_IRQ_IF
 lks32mc07x_mcpwm.h, 423
 MCPWM0CNT0_UP_IRQ_IF
 lks32mc07x_mcpwm.h, 423
 MCPWM0CNT1
 lks32mc07x_mcpwm.h, 431
 MCPWM0CNT1_T0_IRQ_IF
 lks32mc07x_mcpwm.h, 424
 MCPWM0CNT1_T1_IRQ_IF
 lks32mc07x_mcpwm.h, 424
 MCPWM0CNT1_TH30_IRQ_IF
 lks32mc07x_mcpwm.h, 424
 MCPWM0CNT1_TH31_IRQ_IF
 lks32mc07x_mcpwm.h, 424
 MCPWM0CNT1_TH40_IRQ_IF
 lks32mc07x_mcpwm.h, 424
 MCPWM0CNT1_TH41_IRQ_IF
 lks32mc07x_mcpwm.h, 424
 MCPWM0CNT1_TH50_IRQ_IF
 lks32mc07x_mcpwm.h, 425
 MCPWM0CNT1_TH51_IRQ_IF
 lks32mc07x_mcpwm.h, 425
 MCPWM0CNT1_TMR2_IRQ_IF
 lks32mc07x_mcpwm.h, 425
 MCPWM0CNT1_TMR3_IRQ_IF
 lks32mc07x_mcpwm.h, 425

MCPWM0CNT1_UP_IRQ_IF
 lks32mc07x_mcpwm.h, 425
 MCPWM0CNTx
 lks32mc07x_mcpwm.h, 431
 MCPWM_CH012_IESet
 lks32mc07x_mcpwm.c, 731
 lks32mc07x_mcpwm.h, 431
 MCPWM_CH012_Init
 lks32mc07x_mcpwm.c, 731
 lks32mc07x_mcpwm.h, 431
 MCPWM_CH345_IESet
 lks32mc07x_mcpwm.c, 731
 lks32mc07x_mcpwm.h, 431
 MCPWM_CH345_Init
 lks32mc07x_mcpwm.c, 731
 lks32mc07x_mcpwm.h, 432
 MCPWM_ClearIRQFlag
 lks32mc07x_mcpwm.c, 732
 lks32mc07x_mcpwm.h, 432
 MCPWM_Clk_Enable
 lks32mc07x_mcpwm.c, 735
 lks32mc07x_mcpwm.h, 435
 MCPWM_CNTx_Enable
 lks32mc07x_mcpwm.c, 735
 lks32mc07x_mcpwm.h, 435
 MCPWM_DIV_Set
 lks32mc07x_mcpwm.c, 735
 lks32mc07x_mcpwm.h, 435
 MCPWM_EIF_FAIL0
 lks32mc07x_mcpwm.h, 425
 MCPWM_EIF_FAIL1
 lks32mc07x_mcpwm.h, 426
 MCPWM_EIF_FAIL2
 lks32mc07x_mcpwm.h, 426
 MCPWM_EIF_FAIL3
 lks32mc07x_mcpwm.h, 426
 MCPWM_EIF_ZCS0
 lks32mc07x_mcpwm.h, 426
 MCPWM_EIF_ZCS1
 lks32mc07x_mcpwm.h, 426
 MCPWM_GetIRQFlag
 lks32mc07x_mcpwm.c, 736
 lks32mc07x_mcpwm.h, 436
 MCPWM_IF0_T0_IF
 lks32mc07x_mcpwm.h, 426
 MCPWM_IF0_T1_IF
 lks32mc07x_mcpwm.h, 427
 MCPWM_IF0_TH00_IF
 lks32mc07x_mcpwm.h, 427
 MCPWM_IF0_TH01_IF
 lks32mc07x_mcpwm.h, 427
 MCPWM_IF0_TH10_IF
 lks32mc07x_mcpwm.h, 427
 MCPWM_IF0_TH11_IF
 lks32mc07x_mcpwm.h, 427
 MCPWM_IF0_TH20_IF
 lks32mc07x_mcpwm.h, 427
 MCPWM_IF0_TH21_IF
 lks32mc07x_mcpwm.h, 428
 MCPWM_IF0_TMR0_IF
 lks32mc07x_mcpwm.h, 428
 MCPWM_IF0_TMR1_IF
 lks32mc07x_mcpwm.h, 428
 MCPWM_IF0_TMR2_IF
 lks32mc07x_mcpwm.h, 428
 MCPWM_IF0_TMR3_IF
 lks32mc07x_mcpwm.h, 428
 MCPWM_IF0_UP_IF
 lks32mc07x_mcpwm.h, 428
 MCPWM_IF1_T0_IF
 lks32mc07x_mcpwm.h, 429
 MCPWM_IF1_T1_IF
 lks32mc07x_mcpwm.h, 429
 MCPWM_IF1_TH30_IF
 lks32mc07x_mcpwm.h, 429
 MCPWM_IF1_TH31_IF
 lks32mc07x_mcpwm.h, 429
 MCPWM_IF1_TH40_IF
 lks32mc07x_mcpwm.h, 429
 MCPWM_IF1_TH41_IF
 lks32mc07x_mcpwm.h, 429
 MCPWM_IF1_TH50_IF
 lks32mc07x_mcpwm.h, 430
 MCPWM_IF1_TH51_IF
 lks32mc07x_mcpwm.h, 430
 MCPWM_IF1_TMR0_IF
 lks32mc07x_mcpwm.h, 430
 MCPWM_IF1_TMR1_IF
 lks32mc07x_mcpwm.h, 430
 MCPWM_IF1_TMR2_IF
 lks32mc07x_mcpwm.h, 430
 MCPWM_IF1_TMR3_IF
 lks32mc07x_mcpwm.h, 430
 MCPWM_IF1_UP_IF
 lks32mc07x_mcpwm.h, 431
 MCPWM_Init
 lks32mc07x_mcpwm.c, 739
 lks32mc07x_mcpwm.h, 439
 MCPWM_InitTypeDef, 94
 AUEN, 102
 BASE_CNT0_EN, 102
 BASE_CNT1_EN, 102
 CH0_NS, 102
 CH0_PS, 102
 CH0N_default_output, 102
 CH0N_FAIL_EN, 103
 CH0N_Polarity_INV, 103
 CH0N_SCTRLN, 103
 CH0P_default_output, 103
 CH0P_FAIL_EN, 103
 CH0P_Polarity_INV, 103
 CH0P_SCTRLP, 104
 CH1_NS, 104
 CH1_PS, 104
 CH1N_default_output, 104
 CH1N_FAIL_EN, 104

CH1N_Polarity_INV, 104
CH1N_SCTRLN, 105
CH1P_default_output, 105
CH1P_FAIL_EN, 105
CH1P_Polarity_INV, 105
CH1P_SCTRLP, 105
CH2_NS, 105
CH2_PS, 106
CH2N_default_output, 106
CH2N_FAIL_EN, 106
CH2N_Polarity_INV, 106
CH2N_SCTRLN, 106
CH2P_default_output, 106
CH2P_FAIL_EN, 107
CH2P_Polarity_INV, 107
CH2P_SCTRLP, 107
CH3_NS, 107
CH3_PS, 107
CH3N_default_output, 107
CH3N_FAIL_EN, 108
CH3N_Polarity_INV, 108
CH3N_SCTRLN, 108
CH3P_default_output, 108
CH3P_FAIL_EN, 108
CH3P_Polarity_INV, 108
CH3P_SCTRLP, 109
CH4_NS, 109
CH4_PS, 109
CH4N_default_output, 109
CH4N_FAIL_EN, 109
CH4N_Polarity_INV, 109
CH4N_SCTRLN, 110
CH4P_default_output, 110
CH4P_FAIL_EN, 110
CH4P_Polarity_INV, 110
CH4P_SCTRLP, 110
CH5_NS, 110
CH5_PS, 111
CH5N_default_output, 111
CH5N_FAIL_EN, 111
CH5N_Polarity_INV, 111
CH5N_SCTRLN, 111
CH5P_default_output, 111
CH5P_FAIL_EN, 112
CH5P_Polarity_INV, 112
CH5P_SCTRLP, 112
CLK_DIV, 112
CMP_CTRL_CNT0, 112
CMP_CTRL_CNT1, 112
DeadTimeCH012N, 113
DeadTimeCH012P, 113
DeadTimeCH345N, 113
DeadTimeCH345P, 113
EVT0, 113
EVT1, 113
EVT_CNT0_EN, 114
EVT_CNT1_EN, 114
FAIL0_INPUT_EN, 114
FAIL0_INT_EN, 114
FAIL0_Polarity, 114
FAIL0_Signal_Sel, 114
FAIL1_INPUT_EN, 115
FAIL1_INT_EN, 115
FAIL1_Polarity, 115
FAIL1_Signal_Sel, 115
FAIL2_INPUT_EN, 115
FAIL2_INT_EN, 115
FAIL2_Polarity, 116
FAIL2_Signal_Sel, 116
FAIL3_INPUT_EN, 116
FAIL3_INT_EN, 116
FAIL3_Polarity, 116
FAIL3_Signal_Sel, 116
FAIL0CAP, 117
FAIL1CAP, 117
HALT_PRT0, 117
HALT_PRT1, 117
IO0_PPE, 117
IO1_PPE, 117
IO2_PPE, 118
IO3_PPE, 118
IO4_PPE, 118
IO5_PPE, 118
IO_CMP_FLT_CLKDIV, 118
MCLK_EN, 118
MCPWM_WorkModeCH0, 119
MCPWM_WorkModeCH1, 119
MCPWM_WorkModeCH2, 119
MCPWM_WorkModeCH3, 119
MCPWM_WorkModeCH4, 119
MCPWM_WorkModeCH5, 119
STT_HYST, 120
Switch_CH0N_CH0P, 120
Switch_CH1N_CH1P, 120
Switch_CH2N_CH2P, 120
Switch_CH3N_CH3P, 120
Switch_CH4N_CH4P, 120
Switch_CH5N_CH5P, 121
T0_Update0_INT_EN, 121
T0_Update1_INT_EN, 121
T1_Update0_INT_EN, 121
T1_Update1_INT_EN, 121
TH0, 121
TH00, 122
TH00_Match0_INT_EN, 122
TH00_Match1_INT_EN, 122
TH01, 122
TH01_Match0_INT_EN, 122
TH01_Match1_INT_EN, 122
TH1, 123
TH10, 123
TH10_Match0_INT_EN, 123
TH10_Match1_INT_EN, 123
TH11, 123
TH11_Match0_INT_EN, 123
TH11_Match1_INT_EN, 124

TH20, [124](#)
 TH20_Match0.INT_EN, [124](#)
 TH20_Match1.INT_EN, [124](#)
 TH21, [124](#)
 TH21_Match0.INT_EN, [124](#)
 TH21_Match1.INT_EN, [125](#)
 TH30, [125](#)
 TH31, [125](#)
 TH40, [125](#)
 TH41, [125](#)
 TH50, [125](#)
 TH51, [126](#)
 TMR0, [126](#)
 TMR0_Match0.INT_EN, [126](#)
 TMR0_Match1.INT_EN, [126](#)
 TMR0_RE0, [126](#)
 TMR0_RE1, [126](#)
 TMR1, [127](#)
 TMR1_Match0.INT_EN, [127](#)
 TMR1_Match1.INT_EN, [127](#)
 TMR1_RE0, [127](#)
 TMR1_RE1, [127](#)
 TMR2, [127](#)
 TMR2_Match0.INT_EN, [128](#)
 TMR2_Match1.INT_EN, [128](#)
 TMR2_RE0, [128](#)
 TMR2_RE1, [128](#)
 TMR2_TB, [128](#)
 TMR3, [128](#)
 TMR3_Match0.INT_EN, [129](#)
 TMR3_Match1.INT_EN, [129](#)
 TMR3_RE0, [129](#)
 TMR3_RE1, [129](#)
 TMR3_TB, [129](#)
 TR0_AEC, [129](#)
 TR0_T0_RE0, [130](#)
 TR0_T0_RE1, [130](#)
 TR0_T0_UpdateEN, [130](#)
 TR0_T1_RE0, [130](#)
 TR0_T1_RE1, [130](#)
 TR0_T1_UpdateEN, [130](#)
 TR0_UP.INTV, [131](#)
 TR1_AEC, [131](#)
 TR1_T0_RE0, [131](#)
 TR1_T0_RE1, [131](#)
 TR1_T0_UpdateEN, [131](#)
 TR1_T1_RE0, [131](#)
 TR1_T1_RE1, [132](#)
 TR1_T1_UpdateEN, [132](#)
 TR1_UP.INTV, [132](#)
 Update0.INT_EN, [132](#)
 Update1.INT_EN, [132](#)
 ZCS0.INT_EN, [132](#)
 ZCS1.INT_EN, [133](#)
 ZCS_DELAY, [133](#)
 ZCS_EN, [133](#)
 MCPWM_ModuleClockCmd
 lks32mc07x_mcpwm.c, [741](#)
 lks32mc07x_mcpwm.h, [441](#)
 MCPWM_RE0_EN
 stru_DMA.REN, [157](#)
 MCPWM_RE1_EN
 stru_DMA.REN, [157](#)
 MCPWM_StructInit
 lks32mc07x_mcpwm.c, [741](#)
 lks32mc07x_mcpwm.h, [441](#)
 MCPWM_TypeDef, [133](#)
 AUEN, [136](#)
 CH_DEF, [136](#)
 CH_MSK, [136](#)
 CNT0, [136](#)
 CNT1, [136](#)
 DTH00, [136](#)
 DTH01, [137](#)
 DTH10, [137](#)
 DTH11, [137](#)
 EIE, [137](#)
 EIF, [137](#)
 EVT0, [137](#)
 EVT1, [138](#)
 FAIL012, [138](#)
 FAIL345, [138](#)
 FCNT, [138](#)
 FLT, [138](#)
 IE0, [138](#)
 IE1, [139](#)
 IF0, [139](#)
 IF1, [139](#)
 IO01, [139](#)
 IO23, [139](#)
 IO45, [139](#)
 null, [140](#)
 PP, [140](#)
 PRT, [140](#)
 RE, [140](#)
 SDCFG, [140](#)
 STT_HYST, [140](#)
 TCLK, [141](#)
 TH0, [141](#)
 TH00, [141](#)
 TH01, [141](#)
 TH1, [141](#)
 TH10, [141](#)
 TH11, [142](#)
 TH20, [142](#)
 TH21, [142](#)
 TH30, [142](#)
 TH31, [142](#)
 TH40, [142](#)
 TH41, [143](#)
 TH50, [143](#)
 TH51, [143](#)
 TMR0, [143](#)
 TMR1, [143](#)
 TMR2, [143](#)
 TMR3, [144](#)

UPDATE, [144](#)
 ZCS_DELAY, [144](#)
 MCPWM_WorkModeCH0
 MCPWM_InitTypeDef, [119](#)
 MCPWM_WorkModeCH1
 MCPWM_InitTypeDef, [119](#)
 MCPWM_WorkModeCH2
 MCPWM_InitTypeDef, [119](#)
 MCPWM_WorkModeCH3
 MCPWM_InitTypeDef, [119](#)
 MCPWM_WorkModeCH4
 MCPWM_InitTypeDef, [119](#)
 MCPWM_WorkModeCH5
 MCPWM_InitTypeDef, [119](#)
 MCU_MCLK
 lks32mc07x_uart.c, [834](#)
 MEMORY2PERI
 lks32mc07x_dma.h, [318](#)
 MOD
 DSP_Typedef, [77](#)
 Mode
 SPI_InitTypeDef, [151](#)
 MSCR
 I2C_TypeDef, [91](#)
 MST_MODE
 I2C_InitTypeDef, [89](#)
 MultiDropEna
 UART_InitTypeDef, [183](#)
 MX
 CLU_TypeDef, [58](#)
 MXA
 CLU_InitTypeDef, [56](#)
 MXB
 CLU_InitTypeDef, [56](#)
 My_CAN_Send_Msg
 lks32mc07x_can.c, [617](#)
 lks32mc07x_can.h, [237](#)
 nCofA
 Stru_TempertureCof_Def, [159](#)
 nOffsetB
 Stru_TempertureCof_Def, [159](#)
 NSMP
 ADC_InitTypeDef, [27](#)
 null
 ADC_TypeDef, [36](#)
 CRC_InitTypeDef, [67](#)
 GPIO_TypeDef, [82](#)
 MCPWM_TypeDef, [140](#)
 SYS_TypeDef, [163](#)
 null1
 ADC_TypeDef, [36](#)
 AFE_TypeDef, [42](#)
 null10
 ADC_TypeDef, [37](#)
 SYS_TypeDef, [164](#)
 null11
 ADC_TypeDef, [37](#)
 null12
 ADC_TypeDef, [37](#)
 null13
 ADC_TypeDef, [37](#)
 null14
 ADC_TypeDef, [37](#)
 null15
 ADC_TypeDef, [37](#)
 null16
 ADC_TypeDef, [38](#)
 null2
 ADC_TypeDef, [38](#)
 AFE_TypeDef, [42](#)
 null3
 ADC_TypeDef, [38](#)
 AFE_TypeDef, [42](#)
 null4
 ADC_TypeDef, [38](#)
 AFE_TypeDef, [42](#)
 null5
 ADC_TypeDef, [38](#)
 AFE_TypeDef, [42](#)
 null6
 ADC_TypeDef, [38](#)
 AFE_TypeDef, [42](#)
 null7
 ADC_TypeDef, [39](#)
 AFE_TypeDef, [42](#)
 SYS_TypeDef, [164](#)
 null8
 ADC_TypeDef, [39](#)
 SYS_TypeDef, [164](#)
 null9
 ADC_TypeDef, [39](#)
 SYS_TypeDef, [164](#)
 OEN
 CLU_InitTypeDef, [56](#)
 OPA0
 lks32mc07x_opa.h, [453](#)
 OPA1
 lks32mc07x_opa.h, [453](#)
 OPA2
 lks32mc07x_opa.h, [453](#)
 OPA3
 lks32mc07x_opa.h, [453](#)
 OPA_CLEna
 OPA_InitTypeDef, [145](#)
 OPA_Gain
 OPA_InitTypeDef, [145](#)
 OPA_Init
 lks32mc07x_opa.c, [750](#)
 lks32mc07x_opa.h, [454](#)
 OPA_InitTypeDef, [144](#)
 OPA_CLEna, [145](#)
 OPA_Gain, [145](#)
 OPA_IT, [145](#)
 OPA_IT
 OPA_InitTypeDef, [145](#)
 OPA_OUT

- lks32mc07x_opa.c, [751](#)
- lks32mc07x_opa.h, [454](#)
- OPA_StructInit
 - lks32mc07x_opa.c, [752](#)
 - lks32mc07x_opa.h, [455](#)
- OUT
 - CLU_TypeDef, [58](#)
- OUTSEL
 - CLU_InitTypeDef, [56](#)
- OverFlow_IRQ_Ena
 - HALL_InitTypeDef, [85](#)
- OVR
 - ADC_InitTypeDef, [27](#)
- ParityMode
 - UART_InitTypeDef, [184](#)
- PC
 - DSP_Typedef, [77](#)
- PDI
 - GPIO_TypeDef, [82](#)
- PDO
 - GPIO_TypeDef, [82](#)
- PERI2MEMORY
 - lks32mc07x_dma.h, [318](#)
- PFLT
 - GPIO_TypeDef, [83](#)
- PGA_GAIN_16
 - lks32mc07x_opa.h, [452](#)
- PGA_GAIN_32
 - lks32mc07x_opa.h, [452](#)
- PGA_GAIN_4
 - lks32mc07x_opa.h, [452](#)
- PGA_GAIN_8
 - lks32mc07x_opa.h, [452](#)
- PGA_IT_1
 - lks32mc07x_opa.h, [452](#)
- PGA_IT_1P2
 - lks32mc07x_opa.h, [452](#)
- PGA_IT_1P5
 - lks32mc07x_opa.h, [453](#)
- PGA_IT_2
 - lks32mc07x_opa.h, [453](#)
- PGA_IT_DISABLE
 - lks32mc07x_opa.h, [453](#)
- PIE
 - GPIO_TypeDef, [83](#)
- PLL_DivSel
 - SYS_InitTypeDef, [161](#)
- PLL_SrcSel
 - SYS_InitTypeDef, [161](#)
- PODE
 - GPIO_TypeDef, [83](#)
- POE
 - GPIO_TypeDef, [83](#)
- POL
 - CMP_SingleTypeDef, [63](#)
 - CRC_TypeDef, [69](#)
- POLY
 - CRC_InitTypeDef, [67](#)
- PORFilter_Ena
 - SYS_InitTypeDef, [161](#)
- PP
 - MCPWM_TypeDef, [140](#)
- progm_flag
 - lks32mc07x_flash.c, [693](#)
 - lks32mc07x_flash.h, [360](#)
- ProgramPage
 - lks32mc07x_flash.c, [688](#)
 - lks32mc07x_flash.h, [357](#)
- PRT
 - MCPWM_TypeDef, [140](#)
- PSW
 - IWDG_TypeDef, [93](#)
- PUE
 - GPIO_TypeDef, [83](#)
- PWMOutputs
 - lks32mc07x_mcpwm.c, [742](#)
 - lks32mc07x_mcpwm.h, [442](#)
- QEP0
 - lks32mc07x_qep.h, [459](#)
- QEP1
 - lks32mc07x_qep.h, [459](#)
- QEP_ClearIRQFlag
 - lks32mc07x_qep.c, [756](#)
 - lks32mc07x_qep.h, [462](#)
- QEP_CLK_DIV1
 - lks32mc07x_qep.h, [460](#)
- QEP_CLK_DIV128
 - lks32mc07x_qep.h, [460](#)
- QEP_CLK_DIV16
 - lks32mc07x_qep.h, [460](#)
- QEP_CLK_DIV2
 - lks32mc07x_qep.h, [460](#)
- QEP_CLK_DIV32
 - lks32mc07x_qep.h, [460](#)
- QEP_CLK_DIV4
 - lks32mc07x_qep.h, [460](#)
- QEP_CLK_DIV64
 - lks32mc07x_qep.h, [461](#)
- QEP_CLK_DIV8
 - lks32mc07x_qep.h, [461](#)
- QEP_ClockDiv
 - TIM_QEPInitTypeDef, [165](#)
- QEP_FE_CNT_EN
 - TIM_QEPInitTypeDef, [166](#)
- QEP_Filter
 - TIM_QEPInitTypeDef, [166](#)
- QEP_GetFlag
 - lks32mc07x_qep.c, [757](#)
 - lks32mc07x_qep.h, [463](#)
- QEP_GetIRQFlag
 - lks32mc07x_qep.c, [758](#)
 - lks32mc07x_qep.h, [464](#)
- QEP_IF_OVERFLOW
 - lks32mc07x_qep.h, [461](#)
- QEP_IF_UNDERFLOW
 - lks32mc07x_qep.h, [461](#)

QEP_IRQEna
 TIM_QEPInitTypeDef, 166
 QEP_IRQEna_None
 lks32mc07x_qep.h, 461
 QEP_IRQEna_Overflow
 lks32mc07x_qep.h, 461
 QEP_IRQEna_UnderFlow
 lks32mc07x_qep.h, 462
 QEP_Mode
 TIM_QEPInitTypeDef, 166
 QEP_Mode_CCWCW_UpDown
 lks32mc07x_qep.h, 462
 QEP_Mode_CCWSIGN_UpDown
 lks32mc07x_qep.h, 462
 QEP_Mode_T1
 lks32mc07x_qep.h, 462
 QEP_Mode_T1_T2
 lks32mc07x_qep.h, 462
 QEP_TH
 TIM_QEPInitTypeDef, 167
 QEP_ZEC
 TIM_QEPInitTypeDef, 167
 QEP_ZLC
 TIM_QEPInitTypeDef, 167
 QEP_ZNC
 TIM_QEPInitTypeDef, 167
 QEP_ZPC
 TIM_QEPInitTypeDef, 167
 QUO
 DSP_Typedef, 77

 R
 stru_DMA_REN, 157
 RAD
 DSP_Typedef, 77
 RBUF
 CAN_TypeDef, 49
 RCTRL
 CAN_TypeDef, 49
 RE
 ADC_InitTypeDef, 27
 CMP_SingleTypeDef, 64
 MCPWM_TypeDef, 140
 UART_TypeDef, 187
 Read_Flash
 lks32mc07x_flash.c, 690
 Read_More_Flash
 lks32mc07x_flash.c, 692
 lks32mc07x_flash.h, 358
 Read_NVR
 lks32mc07x_nvr.h, 449
 Read_Trim
 lks32mc07x_nvr.h, 449
 RECNT
 CAN_TypeDef, 49
 REFIN
 CRC_InitTypeDef, 67
 REFOUT
 CRC_InitTypeDef, 68

 REG0
 AFE_TypeDef, 42
 REG1
 AFE_TypeDef, 43
 REG2
 AFE_TypeDef, 43
 REG3
 AFE_TypeDef, 43
 REG4
 AFE_TypeDef, 43
 REG5
 AFE_TypeDef, 43
 REG6
 AFE_TypeDef, 43
 REG7
 AFE_TypeDef, 43
 REG8
 AFE_TypeDef, 43
 REG9
 AFE_TypeDef, 44
 REG_IF_WRITE_BIT
 lks32mc07x_lib.h, 408
 REG_READ
 lks32mc07x_lib.h, 408
 REG_READ_BIT
 lks32mc07x_lib.h, 408
 REG_READ_BIT_POS_MASK
 lks32mc07x_lib.h, 408
 REG_RESET
 lks32mc07x_lib.h, 409
 REG_SET
 lks32mc07x_lib.h, 409
 REG_WRITE
 lks32mc07x_lib.h, 409
 REG_WRITE_BIT
 lks32mc07x_lib.h, 409
 REG_WRITE_BIT_POS_MASK
 lks32mc07x_lib.h, 409
 REGA
 AFE_TypeDef, 44
 REGB
 AFE_TypeDef, 44
 REGC
 AFE_TypeDef, 44
 REM
 DSP_Typedef, 77
 REN
 DMA_InitTypeDef, 72
 DMA_RegTypeDef, 75
 Reset_CFG_STAT
 lks32mc07x_can.c, 618
 resv0
 CAN_TypeDef, 49
 resv1
 CAN_TypeDef, 50
 resv2
 CAN_TypeDef, 50
 resv3

CAN_TypeDef, 50
 resv4
 CAN_TypeDef, 50
 RIE
 CLU_InitTypeDef, 57
 RMODE
 DMA_InitTypeDef, 72
 RTH
 IWDG_InitTypeDef, 92
 IWDG_TypeDef, 94
 RTIE
 CAN_TypeDef, 50
 RTIF
 CAN_TypeDef, 50
 rtr
 can_parameter, 46
 CANRxMsg, 52
 CANTxMsg, 54
 RX
 can_parameter, 46
 RX_DATA
 SPI_TypeDef, 153
 RX_DMA_RE
 lks32mc07x_uart.h, 562
 rx_sta
 can_parameter, 46
 RXD_INV
 UART_InitTypeDef, 184

 S1
 ADC_InitTypeDef, 27
 S2
 ADC_InitTypeDef, 28
 SADR
 DMA_InitTypeDef, 73
 DMA_RegTypeDef, 75
 SBAUD
 CAN_TypeDef, 51
 SBaud
 CAN_InitTypeDef, 45
 SBIST
 SYS_TypeDef, 164
 SBTW
 DMA_InitTypeDef, 73
 SC
 DSP_Typedef, 77
 SCR
 I2C_TypeDef, 91
 SDCFG
 MCPWM_TypeDef, 140
 SECOND2IWDGCNT
 lks32mc07x_sys.h, 496
 SELN
 CMP_SingleTypeDef, 64
 SELP
 CMP_SingleTypeDef, 64
 Set_CFG_STAT
 lks32mc07x_can.c, 619
 SetWakeIO
 lks32mc07x_wake.c, 848
 lks32mc07x_wake.h, 584
 SFT_RST
 SYS_TypeDef, 164
 SIF0
 lks32mc07x_sif.h, 472
 SIF_Clear_Irq
 lks32mc07x_sif.c, 766
 lks32mc07x_sif.h, 473
 SIF_DMA_EN
 SIF_InitTypeDef, 146
 SIF_DMA_Enable
 lks32mc07x_sif.h, 472
 SIF_DMA_IsEnable
 lks32mc07x_sif.c, 767
 lks32mc07x_sif.h, 474
 SIF_DONE
 SIF_InitTypeDef, 146
 SIF_DONE_VLD
 SIF_InitTypeDef, 146
 SIF_EN
 SIF_InitTypeDef, 146
 stru_DMA_REN, 157
 SIF_IE_Enable
 lks32mc07x_sif.h, 472
 SIF_Init
 lks32mc07x_sif.c, 768
 lks32mc07x_sif.h, 475
 SIF_InitTypeDef, 146
 SIF_DMA_EN, 146
 SIF_DONE, 146
 SIF_DONE_VLD, 146
 SIF_EN, 146
 SIF_IRQ_EN, 147
 SIF_IRQ_IF, 147
 SIF_MSB, 147
 SIF_RATIO, 147
 SIF_SYNC, 147
 SIF_SYNC_PULSE, 147
 SIF_SYNC_VLD, 147
 SIF_TDTH1, 147
 SIF_TOSC, 148
 SIF_TSTH1, 148
 SIF_IRQ_EN
 SIF_InitTypeDef, 147
 SIF_IRQ_IF
 SIF_InitTypeDef, 147
 SIF_Irq_IsEnable
 lks32mc07x_sif.c, 769
 lks32mc07x_sif.h, 476
 SIF_MSB
 SIF_InitTypeDef, 147
 SIF_RATIO
 SIF_InitTypeDef, 147
 SIF_RegTypeDef, 148
 SIFx_CFG, 148
 SIFx_IRQ, 148
 SIFx_TDTH1, 149

- SIFx_TOSC, [149](#)
- SIFx_TSTH1, [149](#)
- SIF_Senddata
 - lks32mc07x_sif.c, [770](#)
 - lks32mc07x_sif.h, [477](#)
- SIF_StructInit
 - lks32mc07x_sif.c, [771](#)
 - lks32mc07x_sif.h, [478](#)
- SIF_SYNC
 - SIF_InitTypeDef, [147](#)
- SIF_SYNC_PULSE
 - SIF_InitTypeDef, [147](#)
- SIF_SYNC_VLD
 - SIF_InitTypeDef, [147](#)
- SIF_TDTH1
 - SIF_InitTypeDef, [147](#)
- SIF_TH_set
 - lks32mc07x_sif.c, [771](#)
 - lks32mc07x_sif.h, [478](#)
- SIF_Timebase_set
 - lks32mc07x_sif.c, [772](#)
 - lks32mc07x_sif.h, [479](#)
- SIF_TOSC
 - SIF_InitTypeDef, [148](#)
- SIF_TSTH1
 - SIF_InitTypeDef, [148](#)
- SIFx_CFG
 - SIF_RegTypeDef, [148](#)
- SIFx_IRQ
 - SIF_RegTypeDef, [148](#)
- SIFx_TDTH1
 - SIF_RegTypeDef, [149](#)
- SIFx_TOSC
 - SIF_RegTypeDef, [149](#)
- SIFx_TSTH1
 - SIF_RegTypeDef, [149](#)
- SIN
 - DSP_Typedef, [77](#)
- SINC
 - DMA_InitTypeDef, [73](#)
- SIZE
 - SPI_TypeDef, [153](#)
- SLV_MODE
 - I2C_InitTypeDef, [89](#)
- SoftDelay
 - lks32mc07x_sys.c, [789](#)
- softIE
 - HALL_InitTypeDef, [85](#)
- source/lks32mc07x_adc.c, [591](#), [600](#)
- source/lks32mc07x_can.c, [602](#), [620](#)
- source/lks32mc07x_clu.c, [628](#), [629](#)
- source/lks32mc07x_cmp.c, [631](#), [638](#)
- source/lks32mc07x_crc.c, [639](#), [640](#)
- source/lks32mc07x_dac.c, [643](#), [649](#)
- source/lks32mc07x_dma.c, [650](#), [665](#)
- source/lks32mc07x_dsp.c, [666](#), [675](#)
- source/lks32mc07x_exti.c, [677](#), [681](#)
- source/lks32mc07x_flash.c, [686](#), [694](#)
- source/lks32mc07x_gpio.c, [695](#), [708](#)
- source/lks32mc07x_hall.c, [711](#), [725](#)
- source/lks32mc07x_i2c.c, [726](#)
- source/lks32mc07x_mcpwm.c, [730](#), [743](#)
- source/lks32mc07x_opa.c, [749](#), [753](#)
- source/lks32mc07x_qep.c, [755](#), [763](#)
- source/lks32mc07x_sif.c, [765](#), [773](#)
- source/lks32mc07x_spi.c, [774](#), [781](#)
- source/lks32mc07x_sys.c, [783](#), [806](#)
- source/lks32mc07x_tim.c, [811](#), [827](#)
- source/lks32mc07x_tmp.c, [829](#), [833](#)
- source/lks32mc07x_uart.c, [833](#), [844](#)
- source/lks32mc07x_wake.c, [846](#), [857](#)
- SPI0
 - lks32mc07x_spi.h, [482](#)
- SPI_ClearIRQFlag
 - lks32mc07x_spi.c, [774](#)
 - lks32mc07x_spi.h, [485](#)
- SPI_DMA_DISABLE
 - lks32mc07x_spi.h, [482](#)
- SPI_DMA_ENABLE
 - lks32mc07x_spi.h, [482](#)
- SPI_Duplex
 - lks32mc07x_spi.h, [484](#)
- SPI_FIRSTSEND_LSB
 - lks32mc07x_spi.h, [483](#)
- SPI_FIRSTSEND_MSB
 - lks32mc07x_spi.h, [483](#)
- SPI_Full
 - lks32mc07x_spi.h, [484](#)
- SPI_GetIRQFlag
 - lks32mc07x_spi.c, [775](#)
 - lks32mc07x_spi.h, [486](#)
- SPI_Half_Rx
 - lks32mc07x_spi.h, [484](#)
- SPI_Half_Tx
 - lks32mc07x_spi.h, [484](#)
- SPI_IF_CLEAR
 - lks32mc07x_sif.h, [472](#)
- SPI_IF_DataOver
 - lks32mc07x_spi.h, [483](#)
- SPI_IF_SSErr
 - lks32mc07x_spi.h, [483](#)
- SPI_IF_TranDone
 - lks32mc07x_spi.h, [483](#)
- SPI_Init
 - lks32mc07x_spi.c, [777](#)
 - lks32mc07x_spi.h, [487](#)
- SPI_InitTypeDef, [149](#)
 - BaudRate, [150](#)
 - ByteLength, [150](#)
 - CPHA, [150](#)
 - CPOL, [150](#)
 - CS, [150](#)
 - Duplex, [150](#)
 - EN, [151](#)
 - ENDIAN, [151](#)
 - IRQEna, [151](#)

- Mode, 151
- TRANS_MODE, 151
- Trig, 151
- SPI_IRQEna_DataOver
 - lks32mc07x_spi.h, 483
- SPI_IRQEna_Enable
 - lks32mc07x_spi.h, 484
- SPI_IRQEna_SSErr
 - lks32mc07x_spi.h, 484
- SPI_IRQEna_TranDone
 - lks32mc07x_spi.h, 484
- SPI_Master
 - lks32mc07x_spi.h, 485
- SPI_Mode
 - lks32mc07x_spi.h, 484
- SPI_ReadData
 - lks32mc07x_spi.c, 778
 - lks32mc07x_spi.h, 488
- SPI_RX_EN
 - stru_DMA_REN, 157
- SPI_SendData
 - lks32mc07x_spi.c, 779
 - lks32mc07x_spi.h, 489
- SPI_Slave
 - lks32mc07x_spi.h, 485
- SPI_StructInit
 - lks32mc07x_spi.c, 780
 - lks32mc07x_spi.h, 490
- SPI_TX_EN
 - stru_DMA_REN, 157
- SPI_TypeDef, 152
 - BAUD, 152
 - CFG, 152
 - IE, 153
 - RX_DATA, 153
 - SIZE, 153
 - TX_DATA, 153
- SQRT
 - DSP_Typedef, 77
- ss
 - CANTxMsg, 54
- STB_FIFOMode
 - lks32mc07x_can.h, 225
- STB_PriorityMode
 - lks32mc07x_can.h, 225
- STB_TransAll
 - lks32mc07x_can.h, 225
- STB_TransNo
 - lks32mc07x_can.h, 225
- STB_TransOne
 - lks32mc07x_can.h, 225
- STBTransMode
 - lks32mc07x_can.h, 225
- STBTransOption
 - lks32mc07x_can.h, 225
- STOP_IE
 - I2C_InitTypeDef, 89
- StopBits
 - UART_InitTypeDef, 184
- stru_DMA_REN, 153
 - __pad0__, 154
 - __pad1__, 154
 - __pad2__, 155
 - __pad3__, 155
 - __pad4__, 155
 - __pad5__, 155
 - __pad6__, 155
 - __pad7__, 155
 - __pad8__, 155
 - __pad9__, 155
 - ADC0_EN, 156
 - ADC1_EN, 156
 - B, 156
 - CMP_EN, 156
 - DSP_EN, 156
 - GPIO_EN, 156
 - HALL0_EN, 156
 - I2C0_RX_EN, 156
 - I2C0_TX_EN, 157
 - MCPWM_RE0_EN, 157
 - MCPWM_RE1_EN, 157
 - R, 157
 - SIF_EN, 157
 - SPI_RX_EN, 157
 - SPI_TX_EN, 157
 - SW_TRIG_EN, 157
 - TIMER0_EN, 158
 - TIMER1_EN, 158
 - TIMER2_EN, 158
 - TIMER3_EN, 158
 - UART0_RX_EN, 158
 - UART0_TX_EN, 158
 - UART1_RX_EN, 158
 - UART1_TX_EN, 158
- Stru_TempertureCof_Def, 159
 - nCofA, 159
 - nOffsetB, 159
- STT
 - UART_TypeDef, 187
- STT_HYST
 - MCPWM_InitTypeDef, 120
 - MCPWM_TypeDef, 140
- SW_TRIG_EN
 - stru_DMA_REN, 157
- Switch2HRC
 - lks32mc07x_wake.c, 850
 - lks32mc07x_wake.h, 585
- Switch2PLL
 - lks32mc07x_wake.c, 851
 - lks32mc07x_wake.h, 586
- Switch_CH0N_CH0P
 - MCPWM_InitTypeDef, 120
- Switch_CH1N_CH1P
 - MCPWM_InitTypeDef, 120
- Switch_CH2N_CH2P
 - MCPWM_InitTypeDef, 120

Switch_CH3N_CH3P
 MCPWM_InitTypeDef, 120
 Switch_CH4N_CH4P
 MCPWM_InitTypeDef, 120
 Switch_CH5N_CH5P
 MCPWM_InitTypeDef, 121
 SWT
 ADC_TypeDef, 39
 SYS
 lks32mc07x_sys.h, 496
 SYS_AnalogModule_ADC
 lks32mc07x_sys.h, 496
 SYS_AnalogModule_BGP
 lks32mc07x_sys.h, 497
 SYS_AnalogModule_CMP0
 lks32mc07x_sys.h, 497
 SYS_AnalogModule_CMP1
 lks32mc07x_sys.h, 497
 SYS_AnalogModule_CMP2
 lks32mc07x_sys.h, 497
 SYS_AnalogModule_DAC0
 lks32mc07x_sys.h, 497
 SYS_AnalogModule_DAC1
 lks32mc07x_sys.h, 497
 SYS_AnalogModule_OPA0
 lks32mc07x_sys.h, 497
 SYS_AnalogModule_OPA1
 lks32mc07x_sys.h, 497
 SYS_AnalogModule_OPA2
 lks32mc07x_sys.h, 498
 SYS_AnalogModule_OPA3
 lks32mc07x_sys.h, 498
 SYS_AnalogModule_PLL
 lks32mc07x_sys.h, 498
 SYS_AnalogModule_RCH
 lks32mc07x_sys.h, 498
 SYS_AnalogModule_TMP
 lks32mc07x_sys.h, 498
 SYS_AnalogModule_XTAL
 lks32mc07x_sys.h, 498
 SYS_AnalogModuleClockCmd
 lks32mc07x_sys.c, 790
 lks32mc07x_sys.h, 509
 SYS_ClearRst
 lks32mc07x_sys.c, 792
 lks32mc07x_sys.h, 510
 sys_clk
 lks32mc07x_sys.c, 806
 SYS_Clk_SPIDiv1
 lks32mc07x_sys.h, 498
 SYS_Clk_SPIDiv2
 lks32mc07x_sys.h, 498
 SYS_Clk_SPIDiv4
 lks32mc07x_sys.h, 499
 SYS_Clk_SPIDiv8
 lks32mc07x_sys.h, 499
 SYS_Clk_UARTDiv1
 lks32mc07x_sys.h, 499
 SYS_Clk_UARTDiv2
 lks32mc07x_sys.h, 499
 SYS_Clk_UARTDiv4
 lks32mc07x_sys.h, 499
 SYS_Clk_UARTDiv8
 lks32mc07x_sys.h, 499
 SYS_EnterSleep
 lks32mc07x_wake.c, 852
 lks32mc07x_wake.h, 587
 SYS_EVTCLR_Sleep
 lks32mc07x_wake.c, 852
 lks32mc07x_wake.h, 587
 SYS_FallSleep
 lks32mc07x_wake.c, 853
 lks32mc07x_wake.h, 588
 SYS_I2CClkDiv
 lks32mc07x_sys.c, 793
 lks32mc07x_sys.h, 512
 SYS_Init
 lks32mc07x_sys.c, 794
 lks32mc07x_sys.h, 513
 SYS_InitTypeDef, 160
 Clk_DivSPI, 160
 Clk_DivUART, 160
 Clk_FEN, 160
 Clk_Sel, 160
 PLL_DivSel, 161
 PLL_SrcSel, 161
 PORFilter_Ena, 161
 WDT_Ena, 161
 SYS_MCLK_12M_RC
 lks32mc07x_sys.h, 499
 SYS_MCLK_12M_XTAL
 lks32mc07x_sys.h, 499
 SYS_MCLK_24M_RC
 lks32mc07x_sys.h, 500
 SYS_MCLK_24M_XTAL
 lks32mc07x_sys.h, 500
 SYS_MCLK_32K_RC
 lks32mc07x_sys.h, 500
 SYS_MCLK_48M_RC
 lks32mc07x_sys.h, 500
 SYS_MCLK_48M_XTAL
 lks32mc07x_sys.h, 500
 SYS_MCLK_96M_RC
 lks32mc07x_sys.h, 500
 SYS_MCLK_96M_XTAL
 lks32mc07x_sys.h, 500
 SYS_MclkChoice
 lks32mc07x_sys.c, 795
 lks32mc07x_sys.h, 514
 SYS_MCU_POWER_3v3
 lks32mc07x_sys.h, 500
 SYS_MCU_POWER_5v0
 lks32mc07x_sys.h, 501
 SYS_Module_CAN0
 lks32mc07x_sys.h, 501
 SYS_Module_CL0

- lks32mc07x_sys.h, 501
- SYS_Module_CMP
 - lks32mc07x_sys.h, 501
- SYS_Module_CRC0
 - lks32mc07x_sys.h, 501
- SYS_Module_DSP0
 - lks32mc07x_sys.h, 501
- SYS_Module_GPIO
 - lks32mc07x_sys.h, 501
- SYS_Module_HALL0
 - lks32mc07x_sys.h, 501
- SYS_Module_I2C0
 - lks32mc07x_sys.h, 502
- SYS_Module_MCPWM0
 - lks32mc07x_sys.h, 502
- SYS_Module_QEP0
 - lks32mc07x_sys.h, 502
- SYS_Module_QEP1
 - lks32mc07x_sys.h, 502
- SYS_Module_SIF0
 - lks32mc07x_sys.h, 502
- SYS_Module_SPI0
 - lks32mc07x_sys.h, 502
- SYS_Module_TIMER0
 - lks32mc07x_sys.h, 502
- SYS_Module_TIMER1
 - lks32mc07x_sys.h, 502
- SYS_Module_TIMER2
 - lks32mc07x_sys.h, 503
- SYS_Module_TIMER3
 - lks32mc07x_sys.h, 503
- SYS_Module_UART0
 - lks32mc07x_sys.h, 503
- SYS_Module_UART1
 - lks32mc07x_sys.h, 503
- SYS_ModuleClockCmd
 - lks32mc07x_sys.c, 796
 - lks32mc07x_sys.h, 516
- SYS_PLLSRSEL_CRYSTAL
 - lks32mc07x_sys.h, 503
- SYS_PLLSRSEL_RCH
 - lks32mc07x_sys.h, 503
- SYS_POWER_2V5
 - lks32mc07x_sys.h, 503
- SYS_POWER_3V25
 - lks32mc07x_sys.h, 503
- SYS_POWER_3V5
 - lks32mc07x_sys.h, 504
- SYS_POWER_3V75
 - lks32mc07x_sys.h, 504
- SYS_POWER_4V0
 - lks32mc07x_sys.h, 504
- SYS_ReadMcuClk
 - lks32mc07x_sys.c, 798
 - lks32mc07x_sys.h, 518
- SYS_SetWakeInterval
 - lks32mc07x_wake.c, 854
 - lks32mc07x_wake.h, 589
- SYS_SoftReset_all
 - lks32mc07x_sys.c, 799
 - lks32mc07x_sys.h, 519
- SYS_SoftResetModule
 - lks32mc07x_sys.c, 800
 - lks32mc07x_sys.h, 520
- SYS_SPIClkDiv
 - lks32mc07x_sys.c, 802
 - lks32mc07x_sys.h, 522
- SYS_StructInit
 - lks32mc07x_sys.c, 803
 - lks32mc07x_sys.h, 523
- SYS_TypeDef, 161
 - CLK_CFG, 162
 - CLK_DIV0, 162
 - CLK_DIV1, 162
 - CLK_DIV2, 162
 - CLK_FEN, 163
 - DBG_CFG, 163
 - FLSE, 163
 - FLSP, 163
 - FLST, 163
 - IO_CFG, 163
 - MBIST, 163
 - null, 163
 - null10, 164
 - null7, 164
 - null8, 164
 - null9, 164
 - SBIST, 164
 - SFT_RST, 164
 - TRIM, 164
 - WR_PROTECT, 164
- SYS_UartClkDiv
 - lks32mc07x_sys.c, 804
 - lks32mc07x_sys.h, 524
- SYS_VolSelModule
 - lks32mc07x_sys.c, 805
 - lks32mc07x_sys.h, 525
- SYS_VolSelModuleEnableIRQ
 - lks32mc07x_sys.c, 805
 - lks32mc07x_sys.h, 525
- SYS_VolSelModuleIRQ
 - lks32mc07x_sys.c, 805
 - lks32mc07x_sys.h, 525
- SYS_WakeInterval_025s
 - lks32mc07x_wake.h, 580
- SYS_WakeInterval_05s
 - lks32mc07x_wake.h, 580
- SYS_WakeInterval_16s
 - lks32mc07x_wake.h, 581
- SYS_WakeInterval_1s
 - lks32mc07x_wake.h, 581
- SYS_WakeInterval_2s
 - lks32mc07x_wake.h, 581
- SYS_WakeInterval_32s
 - lks32mc07x_wake.h, 581
- SYS_WakeInterval_4s

lks32mc07x_wake.h, 581
 SYS_WakeInterval_8s
 lks32mc07x_wake.h, 581
 T0_Update0.INT_EN
 MCPWM_InitTypeDef, 121
 T0_Update1.INT_EN
 MCPWM_InitTypeDef, 121
 T1_Update0.INT_EN
 MCPWM_InitTypeDef, 121
 T1_Update1.INT_EN
 MCPWM_InitTypeDef, 121
 tbsel
 CANTxMsg, 54
 TBSel_PTB
 lks32mc07x_can.h, 225
 TBSel_STB
 lks32mc07x_can.h, 225
 TBSelOption
 lks32mc07x_can.h, 225
 TBUF
 CAN_TypeDef, 51
 TC_IE
 I2C_InitTypeDef, 89
 TCLK
 CMP_TypeDef, 66
 MCPWM_TypeDef, 141
 TCMD
 CAN_TypeDef, 51
 TCNT
 ADC_InitTypeDef, 28
 TCTRL
 CAN_TypeDef, 51
 TECNT
 CAN_TypeDef, 51
 TempSensor_Init
 lks32mc07x_tmp.c, 831
 lks32mc07x_tmp.h, 558
 TH
 HALL_TypeDef, 86
 TIM_QEPTTypeDef, 169
 TIM_TimerTypeDef, 181
 TH0
 MCPWM_InitTypeDef, 121
 MCPWM_TypeDef, 141
 TH00
 MCPWM_InitTypeDef, 122
 MCPWM_TypeDef, 141
 TH00_Match0.INT_EN
 MCPWM_InitTypeDef, 122
 TH00_Match1.INT_EN
 MCPWM_InitTypeDef, 122
 TH01
 MCPWM_InitTypeDef, 122
 MCPWM_TypeDef, 141
 TH01_Match0.INT_EN
 MCPWM_InitTypeDef, 122
 TH01_Match1.INT_EN
 MCPWM_InitTypeDef, 122
 TH1
 MCPWM_InitTypeDef, 123
 MCPWM_TypeDef, 141
 TH10
 MCPWM_InitTypeDef, 123
 MCPWM_TypeDef, 141
 TH10_Match0.INT_EN
 MCPWM_InitTypeDef, 123
 TH10_Match1.INT_EN
 MCPWM_InitTypeDef, 123
 TH11
 MCPWM_InitTypeDef, 123
 MCPWM_TypeDef, 142
 TH11_Match0.INT_EN
 MCPWM_InitTypeDef, 123
 TH11_Match1.INT_EN
 MCPWM_InitTypeDef, 124
 TH20
 MCPWM_InitTypeDef, 124
 MCPWM_TypeDef, 142
 TH20_Match0.INT_EN
 MCPWM_InitTypeDef, 124
 TH20_Match1.INT_EN
 MCPWM_InitTypeDef, 124
 TH21
 MCPWM_InitTypeDef, 124
 MCPWM_TypeDef, 142
 TH21_Match0.INT_EN
 MCPWM_InitTypeDef, 124
 TH21_Match1.INT_EN
 MCPWM_InitTypeDef, 125
 TH30
 MCPWM_InitTypeDef, 125
 MCPWM_TypeDef, 142
 TH31
 MCPWM_InitTypeDef, 125
 MCPWM_TypeDef, 142
 TH40
 MCPWM_InitTypeDef, 125
 MCPWM_TypeDef, 142
 TH41
 MCPWM_InitTypeDef, 125
 MCPWM_TypeDef, 143
 TH50
 MCPWM_InitTypeDef, 125
 MCPWM_TypeDef, 143
 TH51
 MCPWM_InitTypeDef, 126
 MCPWM_TypeDef, 143
 THETA
 DSP_Typedef, 78
 TIG_CH_EN
 DAC_InitTypeDef, 70
 TIM_ClearIRQFlag
 lks32mc07x_tim.c, 812
 lks32mc07x_tim.h, 541
 TIM_CMP0
 lks32mc07x_tim.c, 813

lks32mc07x_tim.h, 542
 TIM_CMP1
 lks32mc07x_tim.c, 814
 lks32mc07x_tim.h, 543
 TIM_Disable
 lks32mc07x_tim.c, 815
 TIM_Enable
 lks32mc07x_tim.c, 816
 TIM_GetFlag
 lks32mc07x_tim.c, 817
 lks32mc07x_tim.h, 544
 TIM_GetIRQFlag
 lks32mc07x_tim.c, 818
 lks32mc07x_tim.h, 545
 TIM_QEP_GetCount
 lks32mc07x_qep.c, 759
 lks32mc07x_qep.h, 465
 TIM_QEPCmd
 lks32mc07x_qep.c, 760
 lks32mc07x_qep.h, 466
 TIM_QEPInit
 lks32mc07x_qep.c, 761
 lks32mc07x_qep.h, 467
 TIM_QEPInitTypeDef, 165
 QEP_ClockDiv, 165
 QEP_FE_CNT_EN, 166
 QEP_Filter, 166
 QEP_IRQEna, 166
 QEP_Mode, 166
 QEP_TH, 167
 QEP_ZEC, 167
 QEP_ZLC, 167
 QEP_ZNC, 167
 QEP_ZPC, 167
 TIM_QEPStructInit
 lks32mc07x_qep.c, 762
 lks32mc07x_qep.h, 468
 TIM_QEPTypedef, 168
 CFG, 168
 CNT, 168
 IE, 168
 IF, 168
 TH, 169
 TIM_Timer_GetCMPT0
 lks32mc07x_tim.c, 819
 lks32mc07x_tim.h, 546
 TIM_Timer_GetCMPT1
 lks32mc07x_tim.c, 820
 lks32mc07x_tim.h, 547
 TIM_Timer_GetCount
 lks32mc07x_tim.c, 821
 lks32mc07x_tim.h, 548
 TIM_TimerCmd
 lks32mc07x_tim.c, 822
 lks32mc07x_tim.h, 549
 TIM_TimerInit
 lks32mc07x_tim.c, 823
 lks32mc07x_tim.h, 550
 TIM_TimerInitTypeDef, 169
 Timer0_CH0_DEFAULT, 171
 Timer0_CH1_DEFAULT, 171
 Timer0_FAIL_EN, 171
 Timer0_FAIL_POL, 171
 Timer0_FAIL_SEL, 171
 Timer0_HALT_PRT, 171
 Timer0_SHADOW, 172
 Timer0_UPDATE, 172
 Timer_CAP0_CLR_EN, 172
 Timer_CAP1_CLR_EN, 172
 Timer_CENTER, 172
 Timer_CH0_FE_CAP_EN, 172
 Timer_CH0_RE_CAP_EN, 173
 Timer_CH0_WorkMode, 173
 Timer_CH0Output, 173
 Timer_CH1_FE_CAP_EN, 173
 Timer_CH1_RE_CAP_EN, 173
 Timer_CH1_WorkMode, 173
 Timer_CH1Output, 174
 Timer_ClockDiv, 174
 Timer_CMP0, 174
 Timer_CMP1, 174
 Timer_CNT, 175
 Timer_DIR, 175
 Timer_EN, 175
 Timer_ETON, 175
 Timer_EVT, 175
 Timer_FLT, 176
 Timer_GATE_EN, 176
 Timer_IRQEna, 176
 Timer_ONE_TRIG, 177
 Timer_RL_EN, 177
 Timer_SRC0, 177
 Timer_SRC1, 178
 Timer_TH, 178
 Timer_XCLK_EN, 179
 TIM_TimerStrutInit
 lks32mc07x_tim.c, 825
 lks32mc07x_tim.h, 552
 TIM_TimerTypeDef, 179
 CFG, 180
 CMP0, 180
 CMP1, 180
 CNT, 180
 EVT, 180
 FLT, 180
 IE, 180
 IF, 181
 IO, 181
 TH, 181
 Timer0_CH0_DEFAULT
 TIM_TimerInitTypeDef, 171
 Timer0_CH1_DEFAULT
 TIM_TimerInitTypeDef, 171
 TIMER0_EN
 stru_DMA_REN, 158
 TIMER0_EVT_CH0

lks32mc07x_tim.h, 534
 TIMER0_EVT_CH1
 lks32mc07x_tim.h, 534
 Timer0_FAIL_EN
 TIM_TimerInitTypeDef, 171
 Timer0_FAIL_POL
 TIM_TimerInitTypeDef, 171
 Timer0_FAIL_SEL
 TIM_TimerInitTypeDef, 171
 Timer0_HALT_PRT
 TIM_TimerInitTypeDef, 171
 Timer0_IRQEna_FAIL
 lks32mc07x_tim.h, 541
 Timer0_IRQEna_FAIL_RE
 lks32mc07x_tim.h, 541
 Timer0_SHADOW
 TIM_TimerInitTypeDef, 172
 Timer0_UPDATE
 TIM_TimerInitTypeDef, 172
 TIMER1_EN
 stru_DMA_REN, 158
 TIMER1_EVT_CH0
 lks32mc07x_tim.h, 534
 TIMER1_EVT_CH1
 lks32mc07x_tim.h, 535
 TIMER2_EN
 stru_DMA_REN, 158
 TIMER2_EVT_CH0
 lks32mc07x_tim.h, 535
 TIMER2_EVT_CH1
 lks32mc07x_tim.h, 535
 TIMER3_EN
 stru_DMA_REN, 158
 TIMER3_EVT_CH0
 lks32mc07x_tim.h, 535
 TIMER3_EVT_CH1
 lks32mc07x_tim.h, 535
 Timer_CAP0_CLR_EN
 TIM_TimerInitTypeDef, 172
 Timer_CAP1_CLR_EN
 TIM_TimerInitTypeDef, 172
 Timer_CENTER
 TIM_TimerInitTypeDef, 172
 Timer_CH0_FE_CAP_EN
 TIM_TimerInitTypeDef, 172
 TIMER_CH0_IF
 lks32mc07x_tim.h, 535
 Timer_CH0_RE_CAP_EN
 TIM_TimerInitTypeDef, 173
 Timer_CH0_WorkMode
 TIM_TimerInitTypeDef, 173
 Timer_CH0Output
 TIM_TimerInitTypeDef, 173
 Timer_CH1_FE_CAP_EN
 TIM_TimerInitTypeDef, 173
 TIMER_CH1_IF
 lks32mc07x_tim.h, 536
 Timer_CH1_RE_CAP_EN
 TIM_TimerInitTypeDef, 173
 Timer_CH1_WorkMode
 TIM_TimerInitTypeDef, 173
 Timer_CH1Output
 TIM_TimerInitTypeDef, 174
 TIMER_CLK_DIV1
 lks32mc07x_tim.h, 536
 TIMER_CLK_DIV128
 lks32mc07x_tim.h, 536
 TIMER_CLK_DIV16
 lks32mc07x_tim.h, 536
 TIMER_CLK_DIV2
 lks32mc07x_tim.h, 536
 TIMER_CLK_DIV32
 lks32mc07x_tim.h, 536
 TIMER_CLK_DIV4
 lks32mc07x_tim.h, 537
 TIMER_CLK_DIV64
 lks32mc07x_tim.h, 537
 TIMER_CLK_DIV8
 lks32mc07x_tim.h, 537
 Timer_ClockDiv
 TIM_TimerInitTypeDef, 174
 Timer_CMP0
 TIM_TimerInitTypeDef, 174
 Timer_CMP1
 TIM_TimerInitTypeDef, 174
 Timer_CNT
 TIM_TimerInitTypeDef, 175
 Timer_DIR
 TIM_TimerInitTypeDef, 175
 Timer_EN
 TIM_TimerInitTypeDef, 175
 Timer_ETON
 TIM_TimerInitTypeDef, 175
 Timer_EVT
 TIM_TimerInitTypeDef, 175
 TIMER_EVT_CLU0
 lks32mc07x_tim.h, 537
 TIMER_EVT_CLU1
 lks32mc07x_tim.h, 537
 TIMER_EVT_CLU2
 lks32mc07x_tim.h, 537
 TIMER_EVT_CLU3
 lks32mc07x_tim.h, 538
 TIMER_EVT_TADC0
 lks32mc07x_tim.h, 538
 TIMER_EVT_TADC1
 lks32mc07x_tim.h, 538
 TIMER_EVT_TADC2
 lks32mc07x_tim.h, 538
 TIMER_EVT_TADC3
 lks32mc07x_tim.h, 538
 TIMER_FAIL_IF
 lks32mc07x_tim.h, 538
 Timer_FLT
 TIM_TimerInitTypeDef, 176
 Timer_GATE_EN

TIM_TimerInitTypeDef, 176
 TIMER_IF_CH0
 lks32mc07x_tim.h, 539
 TIMER_IF_CH1
 lks32mc07x_tim.h, 539
 TIMER_IF_FAIL
 lks32mc07x_tim.h, 539
 TIMER_IF_ZERO
 lks32mc07x_tim.h, 539
 Timer_IRQEna
 TIM_TimerInitTypeDef, 176
 Timer_IRQEna_All
 lks32mc07x_tim.h, 541
 Timer_IRQEna_CH0
 lks32mc07x_tim.h, 541
 Timer_IRQEna_CH0_RE
 lks32mc07x_tim.h, 541
 Timer_IRQEna_CH1
 lks32mc07x_tim.h, 541
 Timer_IRQEna_CH1_RE
 lks32mc07x_tim.h, 541
 Timer_IRQEna_None
 lks32mc07x_tim.h, 541
 Timer_IRQEna_ZC
 lks32mc07x_tim.h, 541
 Timer_IRQEna_ZC_RE
 lks32mc07x_tim.h, 541
 Timer_IRQEnaDef
 lks32mc07x_tim.h, 541
 Timer_ONE_TRIG
 TIM_TimerInitTypeDef, 177
 TIMER_OPMode_CAP
 lks32mc07x_tim.h, 539
 TIMER_OPMode_CMP
 lks32mc07x_tim.h, 539
 TIMER_POL_HIGH
 lks32mc07x_tim.h, 540
 TIMER_POL_LOW
 lks32mc07x_tim.h, 540
 Timer_RL_EN
 TIM_TimerInitTypeDef, 177
 Timer_SRC0
 TIM_TimerInitTypeDef, 177
 Timer_SRC1
 TIM_TimerInitTypeDef, 178
 Timer_TH
 TIM_TimerInitTypeDef, 178
 Timer_XCLK_EN
 TIM_TimerInitTypeDef, 179
 TIMER_ZC_IF
 lks32mc07x_tim.h, 540
 TMR0
 MCPWM_InitTypeDef, 126
 MCPWM_TypeDef, 143
 TMR0_Match0_INT_EN
 MCPWM_InitTypeDef, 126
 TMR0_Match1_INT_EN
 MCPWM_InitTypeDef, 126
 TMR0_RE0
 MCPWM_InitTypeDef, 126
 TMR0_RE1
 MCPWM_InitTypeDef, 126
 TMR1
 MCPWM_InitTypeDef, 127
 MCPWM_TypeDef, 143
 TMR1_Match0_INT_EN
 MCPWM_InitTypeDef, 127
 TMR1_Match1_INT_EN
 MCPWM_InitTypeDef, 127
 TMR1_RE0
 MCPWM_InitTypeDef, 127
 TMR1_RE1
 MCPWM_InitTypeDef, 127
 TMR2
 MCPWM_InitTypeDef, 127
 MCPWM_TypeDef, 143
 TMR2_Match0_INT_EN
 MCPWM_InitTypeDef, 128
 TMR2_Match1_INT_EN
 MCPWM_InitTypeDef, 128
 TMR2_RE0
 MCPWM_InitTypeDef, 128
 TMR2_RE1
 MCPWM_InitTypeDef, 128
 TMR2_TB
 MCPWM_InitTypeDef, 128
 TMR3
 MCPWM_InitTypeDef, 128
 MCPWM_TypeDef, 144
 TMR3_Match0_INT_EN
 MCPWM_InitTypeDef, 129
 TMR3_Match1_INT_EN
 MCPWM_InitTypeDef, 129
 TMR3_RE0
 MCPWM_InitTypeDef, 129
 TMR3_RE1
 MCPWM_InitTypeDef, 129
 TMR3_TB
 MCPWM_InitTypeDef, 129
 TMS
 DMA_InitTypeDef, 73
 TR0_AEC
 MCPWM_InitTypeDef, 129
 TR0_T0_RE0
 MCPWM_InitTypeDef, 130
 TR0_T0_RE1
 MCPWM_InitTypeDef, 130
 TR0_T0_UpdateEN
 MCPWM_InitTypeDef, 130
 TR0_T1_RE0
 MCPWM_InitTypeDef, 130
 TR0_T1_RE1
 MCPWM_InitTypeDef, 130
 TR0_T1_UpdateEN
 MCPWM_InitTypeDef, 130
 TR0_UP_INTV

MCPWM_InitTypeDef, 131
 TR1_AEC
 MCPWM_InitTypeDef, 131
 TR1_T0_RE0
 MCPWM_InitTypeDef, 131
 TR1_T0_RE1
 MCPWM_InitTypeDef, 131
 TR1_T0_UpdateEN
 MCPWM_InitTypeDef, 131
 TR1_T1_RE0
 MCPWM_InitTypeDef, 131
 TR1_T1_RE1
 MCPWM_InitTypeDef, 132
 TR1_T1_UpdateEN
 MCPWM_InitTypeDef, 132
 TR1_UP_INTV
 MCPWM_InitTypeDef, 132
 TRANS_MODE
 SPI_InitTypeDef, 151
 TRIG
 ADC_InitTypeDef, 28
 ADC_TypeDef, 39
 Trig
 SPI_InitTypeDef, 151
 TRIM
 SYS_TypeDef, 164
 TROVS
 ADC_InitTypeDef, 28
 TX
 can_parameter, 46
 TX_BUF_DMA_RE
 lks32mc07x_uart.h, 562
 TX_DATA
 SPI_TypeDef, 153
 TX_DMA_RE
 lks32mc07x_uart.h, 563
 TXD_INV
 UART_InitTypeDef, 184
 UART0
 lks32mc07x_uart.h, 563
 UART0_RX_EN
 stru_DMA_REN, 158
 UART0_TX_EN
 stru_DMA_REN, 158
 UART1
 lks32mc07x_uart.h, 563
 UART1_RX_EN
 stru_DMA_REN, 158
 UART1_TX_EN
 stru_DMA_REN, 158
 UART_ClearIRQFlag
 lks32mc07x_uart.c, 835
 lks32mc07x_uart.h, 568
 UART_FIRSTSEND_LSB
 lks32mc07x_uart.h, 563
 UART_FIRSTSEND_MSB
 lks32mc07x_uart.h, 563
 UART_GetIRQFlag
 lks32mc07x_uart.c, 836
 lks32mc07x_uart.h, 569
 UART_IF_CheckError
 lks32mc07x_uart.h, 563
 UART_IF_IDLE
 lks32mc07x_uart.h, 564
 UART_IF_LBD
 lks32mc07x_uart.h, 564
 UART_IF_RcvOver
 lks32mc07x_uart.h, 564
 UART_IF_RX_OV
 lks32mc07x_uart.h, 564
 UART_IF_SendBufEmpty
 lks32mc07x_uart.h, 564
 UART_IF_SendOver
 lks32mc07x_uart.h, 564
 UART_IF_StopError
 lks32mc07x_uart.h, 565
 UART_IF_TX_OV
 lks32mc07x_uart.h, 565
 UART_Init
 lks32mc07x_uart.c, 837
 lks32mc07x_uart.h, 570
 UART_InitTypeDef, 181
 BaudRate, 182
 Bit9Value, 182
 DMARE, 182
 DUPLEX, 183
 FirstSend, 183
 IRQEna, 183
 Match485Addr, 183
 MultiDropEna, 183
 ParityMode, 184
 RXD_INV, 184
 StopBits, 184
 TXD_INV, 184
 WordLength, 184
 UART_IRQEna_CheckError
 lks32mc07x_uart.h, 565
 UART_IRQEna_IDLE
 lks32mc07x_uart.h, 565
 UART_IRQEna_LBD
 lks32mc07x_uart.h, 565
 UART_IRQEna_RcvOver
 lks32mc07x_uart.h, 565
 UART_IRQEna_RX_OV
 lks32mc07x_uart.h, 566
 UART_IRQEna_SendBuffEmpty
 lks32mc07x_uart.h, 566
 UART_IRQEna_SendOver
 lks32mc07x_uart.h, 566
 UART_IRQEna_StopError
 lks32mc07x_uart.h, 566
 UART_IRQEna_TX_OV
 lks32mc07x_uart.h, 566
 UART_Parity_EVEN
 lks32mc07x_uart.h, 567
 UART_Parity_NO

lks32mc07x_uart.h, 567
 UART_Parity_ODD
 lks32mc07x_uart.h, 567
 UART_ParityMode
 lks32mc07x_uart.h, 567
 UART_ReadBaud
 lks32mc07x_uart.c, 838
 UART_ReadData
 lks32mc07x_uart.c, 839
 lks32mc07x_uart.h, 571
 UART_SendAddr_485
 lks32mc07x_uart.c, 840
 lks32mc07x_uart.h, 572
 UART_SendData
 lks32mc07x_uart.c, 841
 lks32mc07x_uart.h, 573
 UART_SendData_485
 lks32mc07x_uart.c, 842
 lks32mc07x_uart.h, 574
 UART_SetBaud
 lks32mc07x_uart.c, 843
 UART_STOPBITS_1b
 lks32mc07x_uart.h, 566
 UART_STOPBITS_2b
 lks32mc07x_uart.h, 567
 UART_StructInit
 lks32mc07x_uart.c, 843
 lks32mc07x_uart.h, 575
 UART_TypeDef, 185
 ADR, 186
 BUFF, 186
 CTRL, 186
 DIVH, 186
 DIVL, 186
 IE, 186
 IF, 186
 IOC, 187
 RE, 187
 STT, 187
 UART_WORDLENGTH_8b
 lks32mc07x_uart.h, 567
 UART_WORDLENGTH_9b
 lks32mc07x_uart.h, 567
 UPDATE
 MCPWM_TypeDef, 144
 Update0_INT_EN
 MCPWM_InitTypeDef, 132
 Update1_INT_EN
 MCPWM_InitTypeDef, 132
 UTIMER0
 lks32mc07x_tim.h, 540
 UTIMER1
 lks32mc07x_tim.h, 540
 UTIMER2
 lks32mc07x_tim.h, 540
 UTIMER3
 lks32mc07x_tim.h, 541
 UTIMER_Disable
 lks32mc07x_tim.c, 826
 lks32mc07x_tim.h, 553
 UTIMER_Enable
 lks32mc07x_tim.c, 826
 lks32mc07x_tim.h, 553
 valid
 CANRxMsg, 53
 W_PWM_HIGH_LEVEL
 lks32mc07x_cmp.h, 286
 W_PWM_LOW_LEVEL
 lks32mc07x_cmp.h, 286
 W_PWM_POL
 CMP_SingleTypeDef, 64
 WAKE_InitStruct
 lks32mc07x_wake.c, 857
 WAKE_InitTypeDef, 187
 AFE_REG5_RECORD, 188
 CLK_CFG_RECORD, 188
 wake_SoftDelay
 lks32mc07x_wake.c, 855
 WAKEIO_CLUOUT0
 lks32mc07x_wake.h, 582
 WAKEIO_CLUOUT1
 lks32mc07x_wake.h, 582
 WAKEIO_CLUOUT2
 lks32mc07x_wake.h, 582
 WAKEIO_CLUOUT3
 lks32mc07x_wake.h, 582
 WAKEIO_P0_0
 lks32mc07x_wake.h, 582
 WAKEIO_P0_11
 lks32mc07x_wake.h, 582
 WAKEIO_P0_14
 lks32mc07x_wake.h, 583
 WAKEIO_P0_2
 lks32mc07x_wake.h, 583
 WAKEIO_P0_6
 lks32mc07x_wake.h, 583
 WAKEIO_P2_15
 lks32mc07x_wake.h, 583
 WAKEIO_P2_4
 lks32mc07x_wake.h, 583
 WAKEIO_P2_7
 lks32mc07x_wake.h, 583
 WAKETRIG_HIGH
 lks32mc07x_wake.h, 584
 WAKETRIG_LOW
 lks32mc07x_wake.h, 584
 WDG_EN
 IWDG_InitTypeDef, 92
 WDT_Ena
 SYS_InitTypeDef, 161
 WIDTH
 CRC_InitTypeDef, 68
 HALL_TypeDef, 87
 WordLength
 UART_InitTypeDef, 184

WR_PROTECT

SYS_TypeDef, [164](#)

WTH

IWDG_InitTypeDef, [92](#)IWDG_TypeDef, [94](#)

X

DSP_Typedef, [78](#)

XOROUT

CRC_InitTypeDef, [68](#)

Y

DSP_Typedef, [78](#)

ZCS0.INT_EN

MCPWM_InitTypeDef, [132](#)

ZCS1.INT_EN

MCPWM_InitTypeDef, [133](#)

ZCS_DELAY

MCPWM_InitTypeDef, [133](#)MCPWM_TypeDef, [144](#)

ZCS_EN

MCPWM_InitTypeDef, [133](#)